

Transaction Security Services Processing Guide

BASE24[®]



Contents

About this document	xvi
What's new	xvii
Conventions used in this guide	xx
1: Introduction	21
Transaction Security Services process	21
Overview	22
Database relationships	24
Security data maintenance	27
Configuration	28
Integration in a BASE24 system	31
Implementation requirements	32
Software support	32
Security module support	38
ATM device support	40
POS device support	41
Interface support	41
PIN blocks and pad characters	42
Public key cryptography	43
BASE24 Automated Key Distribution System	43
Secure interchange logons and logoffs	44
Dutch security extensions	44
Large message MAC generation and verification	45
NCR PIN verification	45
RVT/RVS key management	45
Transaction Security Services environment support	47
User interface	47
Timer process	48
ACI Java server framework	49
User Security (USEC) application	50
Default user IDs	50
User auditing	53
Secure sockets layer (SSL) protocol	55

Files	55
Data access layer (DAL) file access	56
File formats.	58
Operations	59
Startup and shutdown	59
Process control	59
Process tracing	62
Master file key translation.	62
Event management	62
Transaction Management Facility (TMF)	63
File partitioning	63
Disaster recovery or dual-site configuration	63
Transaction processing examples	65
2: Database relationships and security data maintenance.	66
BASE24-atm terminal data files.	66
Channel Security Configuration window	69
Old key timer limit	69
SafeNet (Eracom) ProtectHost White Mark II HSMS.	69
SafeNet (Eracom) ProtectHost White AMB HSMS.	70
Banksys DEP HSMS	70
Thales e-Security HSMS	70
Future Keys.	70
BASE24-eps tabs	70
PIN Information tab.	71
MAC Information tab	72
BASE24-pos terminal data files	73
Channel Security Configuration window	75
Old key timer limit	75
SafeNet (Eracom) ProtectHost White Mark II HSMS.	75
SafeNet (Eracom) ProtectHost White AMB HSMS.	76
Banksys DEP HSMS	76
Future Keys.	76
BASE24-eps tabs	76
PIN Information tab.	76
MAC Information tab	77
Data Encryption Key Info tab	78
Data Encryption Exch Info tab.	79
Transaction Key Security tab.	80
Maintaining channel security data	80
Adding a new channel ID record	80
Modifying an existing channel ID record.	83
Deleting a channel ID record	87

Derivation Key file (KEYD)	91
DUKPT Derivation Key Profile Configuration window	93
Key file (KEYF)	94
BASE24 KEYF Configuration window	96
PIN Information tab	97
MAC Information tab	97
Message Information tab	98
Dynamic Key Limits tab	99
Dynamic Key Management tab	100
Interac Key Management tab	101
Interface Security Configuration window	103
Series, Use Series, and Key Hierarchy fields	103
Old key timer limit	103
SafeNet (Eracom) ProtectHost White AMB HSMS	104
Banksys DEP HSMS	104
Thales e-Security HSMS	104
Memory usage.	104
Future Keys.	105
BASE24-eps tabs	105
Exchange Information tab	106
PIN Information tab	108
MAC Information tab	110
Data Encr Exch Info tab	112
Data Encr Key Info tab	113
Master Keys tab	114
Maintaining KEYF and interface security data	114
Adding new KEYF and ICHG_SECURITY records	115
Modifying KEYF and ICHG_SECURITY records.	122
Deleting KEYF and ICHG_SECURITY records.	123
Key authorization files	125
Authentication Profile Configuration window	128
SafeNet (Eracom) ProtectHost White AMB HSMS	128
Banksys DEP HSMS	129
BASE24-eps tabs	129
DES (IBM 3624) PIN Verification tab	130
Visa PVV Keys tab	131
Diebold tab	132
NCR tab	133
Identikey tab	134
EMV tab	135
Chip Auth tab	137
Card tab	140
Contactless tab	142
Maintaining authentication profile security data	142

Displaying an authentication profile	143
Adding a new authentication profile	145
Modifying an existing authentication profile	155
Deleting an authentication profile	158
Key variant fields	161
External memory tables (OLTPs)	162
Rebuilding external memory tables (OLTPs)	165
External memory table (OLTP) size	168
Warmbooting the Transaction Security Services process	175
Internal memory tables	178
Warmbooting the Transaction Security Services process	180
3: Configuration	184
LCONF assigns	184
Primary and secondary TSS process access method	184
Round-robin TSS process access method	186
PPD names	188
LCONF params	189
Unused params	189
Access method	189
Reinstate time	189
Software PIN verification	190
Diebold PIN verification	190
Partial triple DES	190
Changed param usage	191
Logical network identifier	191
Multiple logical networks	192
Security Device Configuration window parameters	193
Transaction security services attributes	202
User interface list retrieval attributes	212
TSS process workload balancing	213
American Express card verification algorithms	215
4: Security module requirements.	218
ISO format 1 PIN block format	218
Atalla security modules	219
External key block format	219
Preparing data for the TSS files	220
Generate check digits command	223
Changing the master file key	223
Importing and exporting keys	224
Triple DES enabled banking commands	225

Triple DES migration software requirements	226
AKB conversion	227
ATM master key exchange	227
PIN translation	228
Message authentication for large messages	231
Device options and banking commands	231
NSP commands	235
Banksys DEP HSMs	240
Connectivity	241
Message formats	241
Supported functions.	241
External key block format	241
DEP key tokens	242
Key storage method.	245
Check digits	246
Message authentication for large messages	246
Preparing data for the TSS files.	246
Supported DEP HSM commands	248
Bull BNTng hardware security modules (HSMs).	250
Connectivity	250
Supported functions.	251
Key storage method.	251
BNTng key header	251
Check digits	254
BNTng initialization data.	254
Preparing data for the TSS database	255
Supported BNTng commands	257
SafeNet (Eracom) host security modules (HSMs)	258
Connectivity	258
Supported functions.	258
External key block format	260
Bad PIN and key synchronization errors.	260
AS2805 6.2 and 6.4 PIN block formats.	260
Proof-of-endpoint processing	261
Message authentication for large messages	261
Preparing data for the TSS files.	262
Supported SafeNet (Eracom) online functions.	267
Thales e-Security modules	271
External key block format	272
Double- and triple-length key encryption scheme	272
Decimalization table security.	272
Message authentication for large messages	273
Preparing data for the TSS files.	273
Generate check digits command	277

Changing local master key pairs	278
Triple DES enabled commands	278
Racal Access Module (RAM) process	278
HSM commands.	279
Asynchronous protocol and Thales HSM commands	283
5: TSS processing examples	285
Terminal acquired—not-on-us card	285
Terminal acquired—on-us card	287
Interface acquired—on-us card	289
Card verification	292
EMV card authentication.	293
Terminal acquired—on-us card—software verification allowed	294
Terminal acquired—on-us card—software verification not allowed	295
6: Message data field encryption	297
Overview	297
Cipher block chaining (CBC)	297
Electronic code book (ECB).	298
Encryption functions	298
Data encryption keys	299
Data encryption layers	300
Encryption data types	300
Message authentication codes (MACs)	300
Tokens	300
Configuring message data encryption for SPDH devices.	301
BASE24 database settings	301
LCONF	302
TSS database settings	303
Configuring message data encryption for interfaces	304
KEYF screen 4 configuration	304
Interface security configuration.	305
BASE24 KEYF configuration	307
Security module configuration.	308
Data encryption keys	308
Key exchange keys	308
Commands	309
Dynamic key management	310
7: TSS environment management	311
Environment configuration overview	311
Configuring environment control attributes.	312

System Interface Message Delivery Service Destination file (MDSDEST)	313
Contents	314
Security device	314
IS process	315
Creating the MDSDEST	316
Warmbooting the MDSDEST	317
Metadata external memory tables	317
SIS DAL processing	318
Metadata EMT build configuration and startup	321
Build command	321
Command entry methods	321
Recommended implementation	325
Modifying CONFCSV parameters	326
Adding a new security device	327
Adding Transaction Security Services processes	327
Adding timer processes	328
Changing IP addresses	329
Changing database access passwords	331
Report assign configuration and formatting	332
8: TSS event management	335
Event configuration overview	335
Event generation and suppression	335
Event logging	335
Event monitoring	336
event Response or Action	336
Log destination	336
Recognizing events	336
Cause, effect, and remedy	337
Subsystem ID and event number	337
Event Configuration and Management window	339
Suppressing an event message	340
Overview	341
Event Suppression Information tab	343
Log state	344
Event suppression methods	346
Configuring events	347
9: Process control	353
Process control overview	353
Process Control window	354
Destinations	355

Commands	356
Components	357
Command Text	358
Process control commands	358
AKDS Capacity report	359
Data Source component	359
Data Source Loader component	360
Environment component.	361
Event component.	363
Exception Log component.	366
Heap Manager component	367
Report memory allocated but not released for a thread	372
Key Conversion component.	377
Timer component	381
Trace component.	382
Transaction Security component	383
User Interface Auditing component	389
VSP (Version/Service Pack) Manager component.	390
10: Process tracing	392
Defining traces	392
Process tracing in a production environment	393
Use a filter	393
Partition the Trace data source	393
Trace a single process	393
Trace only the levels and components needed	394
Do not include trace header data	394
Masking sensitive cardholder data in trace output	394
Configuring traces	395
Process Type	396
Trace Output Assign.	397
Process Trace is Active	397
Trace Filter Profile	397
Include Header in Trace	398
Trace levels.	398
Trace Level, Component ID Filter, and User Data	402
Configuring trace filter profiles	406
Trace Filter Profile	407
Filter Type.	408
Filter Value	408
Trace operations	408
Controlling traces	409
Starting a trace	409

Modifying a running trace	409
Stopping a trace	410
Obtaining status information for a trace	410
Examining trace output	410
Viewing trace data using the trace viewer utility	411
Example external message trace output	414
Security device tracing procedure	416
11: OLTP management	419
OLTP warmboot files	419
OLTP Warmboot Management window	420
Toolbar functions	422
Sorting OLTP_Warmboot entries	422
Rebuilding OLTPs	423
Warmbooting OLTPs	424
Deleting OLTP_Warmboot records	425
Accessing the OLTP Warmboot Management window	425
12: Memory management and troubleshooting	427
About the ACI Heap Manager	427
Getting thread-level memory status information	430
Memory monitoring	431
Memory parcel handling with memory monitoring activated	431
Activating/deactivating memory monitoring	432
Using memory monitoring to identify memory leaks	432
Determine whether memory leaks exist	433
Find the memory leaks	436
Analyzing Heap Trace output	436
Memory verification	446
Enabling/disabling memory verification	446
Setting the memory verification interval	447
Memory parcel handling with memory verification activated	448
How memory is verified	449
Detect memory corruption	450
Analyzing process termination output	452
13: Dynamic card verification	456
Overview	456
EMV standard	457
Magnetic stripe standard	457
Visa contactless MSD with CVN 17	458
MasterCard PayPass contactless magnetic stripe requirements	460
Sample Track 1 and 2 IDD format	461

Visa payWave contactless magnetic stripe requirements	462
Sample Track 1 and 2 IDD format	463
Processing inputs	464
Processing	465
ATC checking	465
DCVD Checking	466
Configuration	467
BASE24 configuration	467
TSS configuration	472
14: Database conversion	473
Database conversion programs	475
15: Public key cryptography	477
Introduction	477
NCR and Tidel devices	479
NCR firmware emulating Diebold firmware	480
Diebold devices	481
Triton devices	481
Diebold firmware emulating NCR firmware	481
Devices supporting Wincor Nixdorf's shared signature specification firmware	482
Devices supporting Wincor Nixdorf's shared certificate specification firmware	483
Interfaces	484
BASE24 system	484
Device definitions	485
Device handler extensions	486
Terminology	487
Implementation requirements	491
NCR 5XXX devices	491
Diebold 10XX/478X devices	493
Shared Wincor Nixdorf signature devices	494
Shared Wincor Nixdorf certificate devices	495
Tidel devices	496
Triton devices	496
BASE24 system	496
Security device configuration	497
Atalla public key cryptography commands	497
AKB headers for RSA keys	498
Banksys DEP HSM public key cryptography commands	498
Thales e-Security public key cryptography commands	499
LCONF params	500
NCR 5XXX and Tidel terminal implementation procedures	502
Phase 1—generate an NCR RSA key pair	502

Phase 2—manufacture encrypting PIN pad (EPP)	502
Phase 3—generate RSA key pairs for the BASE24 system	503
Phase 4—authenticate and exchange public keys	506
Phase 5—download the master key	511
Migrating from single-length to double-length keys	512
Diebold 10XX/478X terminal implementation procedures	513
Phase 1—initial registration and certification for the EPP	513
Phase 2—initial registration and certification for the BASE24 system	514
Phase 3—exchange certificates	518
Phase 4—download the master key	520
Migrating from single-length to double-length keys	522
Triton terminal implementation procedures	523
Phase 1—initial registration and certification for the EPP	523
Phase 2—initial registration and certification for the BASE24 system	524
Phase 3—exchange certificates and download keys	533
Shared Wincor Nixdorf signature terminal implementation procedures	536
Phase 1—Generate a Wincor Nixdorf RSA key pair	537
Phase 2—Manufacture encrypting PIN pad (EPP)	537
Phase 3—Generate an RSA key pair for the BASE24 system	538
Phase 4—authenticate and exchange public keys	541
Phase 5—download the master key	542
Load all keys command	544
NCR 5XXX and Load All Keys Command	544
Diebold 10XX/478X load all keys command	545
Wincor Nixdorf load all keys command	546
Interface implementation	546
Phase 1—generate an interchange RSA key pair	547
Phase 2—Generate RSA key pairs for the BASE24 system and exchange public keys	547
Phase 3—Mutual authentication	550
Key Generate (KEYGEN) command	552
KEYGEN Command Configuration and Submission window	555
Process Control window	558
Disabling the KEYGEN command	559
Data access	561
Certificate Management window	563
Host Public Key Perusal window	590
Copying key text	594
Key reports	596
Channel Public Key Perusal window	598
Copying key text	603
Key reports	604
EPP Serial Number Configuration window	607
AKDS Capacity report	608

Configuring the report output location	612
Configuring the report page length	613
Generating the AKDS Capacity report.	613
16: Security device utilities and operations	616
Check digit validation and generation utility	616
Prerequisites	617
Procedures	617
Processing report.	621
AKB conversion utility	627
Prerequisites	627
Procedures	629
Processing report.	634
Optional AKB header environment attributes	635
Master file key conversion	637
Prerequisites	638
Accessing the utility	639
Atalla options	639
Thales Options	640
Key Translation report	643
Post-Translation procedures	645
Disabling a security device	646
Process Control window	647
Network control facility.	648
Enabling a security device	648
Process Control window	649
Network control facility.	650
Initializing a security device	650
Process Control window	651
Network control facility.	652
Obtaining the current status of a security device	652
17: Europay, MasterCard, and Visa (EMV)	654
Cryptographic support	654
EMV scheme and cryptogram version number	654
Cryptogram version numbers in EMV	655
Issuer script commands	670
Offline PIN change	670
Notification to host	670
Destination PIN block.	671
EMV 2000	671
Atalla NSPs	672

Banksys DEP HSMS	672
Bull BNTng HSMS.	672
SafeNet (Eracom) HSMS	672
Thales e-Security HSMS	673
Session key derivation function	673
Data block padding	674
Common session key derivation	674
Atalla NSPs	674
SafeNet (Eracom) ProtectHost White Mark II HSMS.	674
Thales e-Security HSMS	675
MasterCard proprietary session key derivation	675
EMV common session key derivation	675
Data block padding	676
Chip authentication program (CAP) token validation	676
Processing overview.	676
Initial CVR.	679
Implementation.	680
Configuration	680
Application transaction counter checking	681
18: Diebold PIN verification	684
Diebold number table	684
Atalla NSPs	684
Thales e-Security HSMS	685
Relative location	686
Downloading the Diebold number table (DNT)	686
Process Control window	686
Network control facility.	688
19: NCR PIN verification support.	689
NCR PIN verification	689
Input data	690
PIN encryption key and PIN source flag	690
PIN verification key	690
PIN	691
PIN block and PIN block format.	691
PIN data block.	691
PAN	692
Index.	693

About this document

The Transaction Security Services process provides all hardware security module functions for BASE24 processes. The Transaction Security Services process is built using ACI's BASE24-eps[®] object-oriented architecture. Files used by the process are maintained through a separate user interface. The **BASE24 Transaction Security Services Processing Guide** describes how the Transaction Security Services process is integrated into a BASE24 system. It describes how BASE24 hardware security data is converted and maintained in data files, how BASE24 processes are configured to use the Transaction Security Services process, and the implementation requirements for using the process. It also describes the operations facilities used to maintain the Transaction Security Services process and provides several transaction message flows.

Audience

This processing guide is intended for the system administrators and operational staff who determine how the BASE24 system is to handle transaction security and who set up the BASE24 database.

What's new

Post Release 11.1, Sep-2012

The following table highlights the major changes that have been made in the most recent update to the **BASE24 Transaction Security Services Processing Guide**. The first column of the table lists the sections in which major changes have been made. The second column of the table describes the major changes for each section. These changes update the manual to Release 6.0, Version 10 (TSS Post Release 11.1).

Section	Major Changes
All	Adds support for the Atalla Network Security Processor (NSP) model A10160 throughout the manual.

Post Release 11.1, Aug-2012

The following table highlights the major changes that have been made in the most recent update to the **BASE24 Transaction Security Services Processing Guide**. The first column of the table lists the sections in which major changes have been made. The second column of the table describes the major changes for each section. These changes update the manual to Release 6.0, Version 10 (TSS Post Release 11.1).

Section	Major Changes
1	Indicates that support for the American Express Card Security Code (CSC) Version 2.0 algorithm to verify iCSCs is added in this version.
2	Clarifies the card verification values verified and the algorithms used for the keys on the Card tab of the Authentication Profile Configuration window and adds support for iCSC verification.
3	Adds a description of the AMEX_CSC_ALGO Environment attribute. This attribute indicates which American Express card verification algorithm to use when verifying American Express cards.

Post Release 11.1, Mar-2012

The following table highlights the major changes that have been made in the most recent update to the **BASE24 Transaction Security Services Processing Guide**. The first column of the table lists the sections in which major changes have been made. The second column of the table describes the major changes for each section. These changes update the manual to Release 6.0, Version 10 (TSS Release 11.1).

Section	Major Changes
15	Removes the note indicating that other ATM vendors that support native NCR emulation, such as Nautilus Hyosung ATMs, are also supported by the BASE24 Automated Key Distribution System (AKDS) add-on product using the same procedures as NCR devices using NDC+ message formats. AKDS does not yet support Nautilus Hyosung ATMs.

Release 11.1

The following table highlights the major changes that have been made in the most recent update to the **BASE24 Transaction Security Services Processing Guide**. The first column of the table lists the sections in which major changes have been made. The second column of the table describes the major changes for each section. These changes update the manual to Release 6.0, Version 10 (TSS Release 11.1).

Section	Major Changes
2	Clarifies the usage of key variant values on the Channel Security Configuration and Interface Security Configuration windows. Adds support for the Interac Flash EMV scheme (CVN 85) on the EMV tab of the Authentication Profile Configuration window.
3	Removes the ANSI X9.17 – Thales option for the Key Scheme field on the Security Device Configuration window.
9	Adds new process control commands for the Heap Manager component for better memory management in the BASE24-eps (TSS) environment.
11	Adds a new section describing memory management in the BASE24-eps (TSS) environment.

Section	Major Changes
15	Clarifies the procedures for implementation of remote key transport processing for Triton terminals. Additional steps are required to reformat the certificates returned from Triton before they can be used by BASE24. Also clarifies that Triton is the certificate authority for Triton terminals with VeriSign acting as the service provider.
17	Adds support for the Interac Flash EMV scheme (CVN 85)

Release 09.2, Service Pack 6

The following table highlights the major changes that have been made in the most recent update to the **BASE24 Transaction Security Services Processing Guide**. The first column of the table lists the sections in which major changes have been made. The second column of the table describes the major changes for each section. These changes update the manual to Release 6.0, Version 10 (TSS Release 09.2 SP 6).

Section	Major Changes
All	Applies a new standard look and feel throughout the guide. All appendices have been converted to sections and section numbers are no longer provided in page numbering.
1	Updates the reference for permission IDs. Permission IDs have been moved from an appendix in the ACI Desktop User Interface Manual to configuration attribute tables for each window in the ACI Desktop Windows Reference .
2	Adds notes for the MAC Information tab on the Channel Security Configuration window to indicate that because many devices use the same MAC key for both inbound and outbound message authentication processing, the Inbound Key is used for MAC generation if you leave the Outbound Key blank. If the Outbound Key field is not blank, it is used for MAC generation. Thus, to use the same key for MAC verification and MAC generation, you must either configure the same key in both the Inbound Key and Outbound Key fields or leave the Outbound Key field blank.
7	Adds steps for logging off and logging back on to the ACI desktop user interface and stopping and restarting the ESWEB and TDAL processes when changing metadata. These procedures should only need to be performed infrequently as metadata in a production system should rarely change.
14	Adds support for NCR's enhanced signature remote key loading (RKL) authentication scheme.

Conventions used in this guide

This section explains how the security best practices symbol and syntax notation is used in this guide.

Security best practices symbol



This guide uses the symbol shown at the left to alert you to cross-industry best practices for protecting sensitive data that is stored, processed, or transmitted in a BASE24 system. Whenever you see this symbol, you should carefully review the information provided next to the symbol to ensure you are implementing cross-industry best practices for securing data. Refer to the **BASE24 Security Best Practices** for more information about configuring the BASE24 product to comply with cross-industry best practices.

Syntax notations

Syntax descriptions in this user guide use several symbols and conventions to denote how commands must be entered. The conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
<i>lowercase italics</i>	Identify variables that you must provide.
Brackets []	Enclose optional syntax items that you can enter in the command.
Braces { }	Enclose a group of items from which you are required to choose one item. These items are separated within the braces by a vertical bar ().
Vertical bar	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Notation	Meaning
Spaces	Are required as delimiters whenever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbols.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

1: Introduction

This section introduces the Transaction Security Services process. The Transaction Security Services process, built using ACI's Enterprise Services architecture, provides hardware security module services to BASE24 processes.

This section describes the Transaction Security Services process, explains why it was created to serve as an interface between the security utilities bound into BASE24 processes and hardware security modules, and introduces the following topics related to implementing the Transaction Security Services process into a BASE24 system:

- Security data maintenance
- Configuration
- Integration in a BASE24 system
- Advanced ATM key management
- Dutch security extensions
- Implementation requirements
- Transaction Security Services process
- Transaction Security Services environment
- Operations considerations
- Transaction processing

Transaction Security Services process

The Transaction Security Services process runs as an integrated server process in the Transaction Security Services environment. Integrated server processes are constructed from object-oriented components bound together into an executable process. Transaction Security Services processes use the TSSISLO executable object. The component within the Transaction Security Services process that performs most transaction security functions is called the Transaction Security component. The Key Conversion component performs key maintenance functions in hardware security modules such as changing the master file key. Other components within the Transaction Security Services process perform essential administrative or processing tasks. For example, the Process Control component enables you to enter process control commands for integrated server processes and the Environment component enables you to set attributes for processing.

Note: The AKDS process is a special Transaction Security Services process that contains all of the components of the Transaction Security Services process plus the components required to support public key cryptography for the BASE24 Automated Key Distribution System add-on product or secure logons to and logoffs from an interchange. The AKDS process uses the AKDSISLO executable object.

The Transaction Security Services environment is comprised not only of Transaction Security Services processes, but also a graphical user interface called the ACI desktop user interface, User Interface processes, a Timer process, a database, and user security and auditing functions for the ACI desktop user interface. The functions of the User Interface processes are divided between a C++ User Interface (XML Server) process running under the Guardian operating system and Java User Interface servlets running within a ACI Web Application Services - Java Services servlet container¹ process under Open System Services (OSS). Together, the C++ User Interface (XML Server) process and Java User Interface servlets are collectively referred to as the User Interface process throughout this processing guide, unless explicitly stated otherwise. Each component of the Transaction Security Services environment is described in more detail later in this section.

¹ If desired, you can replace the ACI Web Application Services - Java Services servlet container with any Java 2 Platform, Enterprise Edition (J2EE)-compliant servlet container.

Overview

The Transaction Security Services process serves as an interface between the security utilities bound into BASE24 processes and hardware security modules.

The Transaction Security Services process was created to ease the migration to triple DES. It also supports the Atalla Key Block (AKB) format and serves as the foundation for all future transaction security enhancements such as automated key distribution.

Beginning with version AKB 1.2 software, Atalla requires that all keys be stored in the database in the following format:

Header (8 bytes)	Encrypted key (48 bytes)	MAC (16 bytes)
------------------	--------------------------	----------------

The AKB format protects stored keys against known attacks. The Header field contains eight one-byte values describing the attributes of the key. Before a key in AKB format is used in an Atalla Network Security Processor, the content of the header block is validated. The Encrypted Key field contains the encrypted key block. Single-length and double-length keys are padded before encryption to hide their length. The MAC field contains a message authentication code (MAC) of the header and encrypted key. The MAC value represents a MAC of the 8 byte header and the 48 byte key and is generated by an Atalla hardware security module (HSM) when the key is generated. This MAC value cryptographically binds the header and the encrypted key together.

The header ensures the proper use of the key, the padding of the encrypted key block disguises the length of the key, and the MAC value ensures that the key block has not been tampered with.

Because Atalla does not support both the old key block format and the new key block format in a single HSM, all keys must be converted to the new key block format at the same time.

A set of database files used by the Transaction Security Services process supports extended length keys required for triple DES as well as the AKB key block format. These files are accessed by the Transaction Security Services process and are maintained using the ACI desktop user interface. If you are migrating to this release from a BASE24 release earlier than release 6.0, version 2, refer to [section 14](#) or the **BASE24 Release 6.0 Implementation Guide** for database conversion information.

The Transaction Security Services process supports all of the following transaction security functions currently supported in BASE24 systems:

- PIN encryption using single DES ensures that PIN information is maintained in a confidential manner.
- PIN verification ensures that the true cardholder is performing the transaction. The Transaction Security Services process supports the following methods of PIN verification:
 - Diebold (this method is only supported for Atalla AKB and Thales e-Security security devices)
 - IBM DES (3624-format) (this method is not supported for Bull BNTng HSMs)
 - Identikay (this method is supported only with Atalla security devices)
 - NCR (this method is supported only with Thales e-Security security devices and requires a custom HSM command)
 - Visa PVV (this method is the only PIN verification method supported for Bull BNTng HSMs)
- Message authentication ensures that data remains tamper-free while passing between an EFT device and its authorization destination.

Note: If using triple DES for message authentication, the encrypt operations performed by the sender and receiver of the message are replaced by encrypt/decrypt/encrypt operations using double or triple length keys.
- Card verification ensures that a valid card is being used to initiate the transaction.
- Chip authentication ensures that a credit or debit card with an embedded microprocessor chip is valid.
- Dynamic card verification ensures that a contactless magnetic stripe credit or debit card is valid (this function is supported only for Atalla AKB and Thales e-Security security devices).

Besides the existing security-related functions described above, the Transaction Security Services process provides the following security enhancements to a BASE24 system:

- Triple DES PIN encryption for stronger PIN protection and reduced potential fraud losses.
- Longer keys for protection against key exhaustion attacks.

- Stronger key storage techniques to protect against key compromise.
- Data field or full message encryption and decryption (this function is supported only with Atalla AKB and variant NSPs).
- RVT/RVS key management (this key management scheme is supported only with Thales e-Security security devices and requires custom HSM commands).
- Public key cryptography for the BASE24 Automated Key Distribution System add-on product and secure logons and logoffs to and from an interchange (these functions are supported only with Atalla AKB, Banksys DEP, and Thales e-Security security devices).

The Transaction Security Services process runs as an application process under the XPNET process (i.e., it runs under control of the XPNET process for the purpose of process restart and recovery only; the XPNET process does not send transaction processing messages to the Transaction Security Services process, except for the BASE24 Automated Key Distribution System add-on product). The same XPNET process control mechanisms available to other BASE24 processes can be used on the Transaction Security Services process. The process can be replicated as needed and can be started, stopped, etc., from a network control facility. Security utilities bound into BASE24 application processes interface with the Transaction Security Services process using a direct write/read interface.

Database relationships

When using the Transaction Security Services process, hardware-encrypted key information is maintained in the Transaction Security Services database. The hardware-encrypted key fields in the BASE24 database are set to a nonencrypted value known as a *key locator*. Key locator values are set when a new record is added to a BASE24 file (e.g., when adding a new terminal to the BASE24-atm Terminal Data files) or when existing records in a BASE24 file are converted. For more information on converting BASE24 files, refer to [section 14](#).

Key locator values are either set to the primary key value for the record (e.g., the terminal ID for hardware key fields in the BASE24-atm Terminal Data files and BASE24-pos Terminal Data files) or an index value created by the conversion program for the file (i.e., for the Key File). The Transaction Security Services process uses the key locator value as the primary key value to retrieve the actual hardware-encrypted key information from its database.

Note: If you are using the LN-IND param to partition the Transaction Security Services database by logical network, the two-character value of this param is added to the beginning of the key locator value. For more information on the LN-IND param, refer to [section 3](#).

The following table shows the relationship of existing BASE24 database files with key locator values to the corresponding Transaction Security Services database files. The first column lists the existing BASE24 files and the second column lists the files accessed by the Transaction Security Services process.

BASE24 file	TSS file
BASE24-atm Terminal Data files (ATD)	Channel Security File (Channel_Security)
BASE24-pos Terminal Data files (PTD)	
Card Prefix File (CPF)	Dynamic Card Verification File (DYN_CARD_VRFCTN)
	Chip Authentication File (Chip_Authentication)
	Interface Security File (ICHG_SECURITY)
Derivation Key File (KEYD)	Derivation Key File (Derivation_Key)
Key File (KEYF)	Interface Security File (ICHG_SECURITY)

When using the Transaction Security Services process, two BASE24 files are replaced by Transaction Security Services files and are no longer accessed in processing for verification in hardware security modules. These BASE24 files are used only if you need to support verification in software. The following table lists the Transaction Security Services files that replace these BASE24 files for verification processing in a hardware security module. These Transaction Security Services files use the same primary keys as the BASE24 files they replace.

BASE24 file	TSS file
Key Authorization File (KEYA)	Card Verification File (CARD_VRFCTN)
	Diebold Number Table File (DIEBOLD_NUM_TBL)
	IBM DES Verification File (IBM_DES)
	Identikey PIN Verification File (Identikey)
	Visa PVV Verification File (Visa_PVV)
Dutch Key Authorization File (KEYV)	NCR PIN Verification File (NCR_PIN_VRFY)
ICC Key File (KEYI)	EMV Authorization File (EMV_Security)

The Transaction Security Services database files are created during preparation for installation using the Metadata Manager (Metaman) utility. For the Transaction Security Services process, the Metaman utility creates both FUP command files used to create the

Transaction Security Services files and Comma Separated Values (CSV) files that list and define all elements, tables, assigns, and params. All Metaman utility output files are created by ACI prior to installation at a customer site.

The Metadata Configuration File (CONFCSV) table on the data subvolume defines the assigns for Transaction Security Services files. The assigns specify the file locations of the Enscribe disk files, the swap files for external memory tables, and the maximum size allowed for each external memory table. The CONFCSV also contains parameters that control the System Interface Services (SIS) layer of the TSS environment. These parameters are described in [section 7](#) of this guide. This file must be modified during installation for site-specific assign and parameter information.

The Meta Database Comma Separated Values (MDBCSV) file on the data subvolume defines the attributes and data elements for each TSS file and is the equivalent of a DDL file in the BASE24 environment.

Separate CONFCSV and MDBCSV files must exist on a different subvolume for the USEC and UAUD applications. This subvolume, referred to as the USEC/UAUD data subvolume, contains the user security and user auditing database files. The Environment File (Environment) and Event File (Event) assigns in this CONFCSV must point to the Environment and Event files residing on the TSS data subvolume. The USEC and UAUD applications use the same Environment and Event used by the Transaction Security Services process.

Note: With the advent of TSS release 04.1, the UAUD auditing database is no longer used. In TSS release 04.1, the UAUD application forwarded all auditing messages from the C++ User Interface (XMLS) process to the Java User Interface servlets (ESWEB) process for logging in the User Audit Log File (USRAULOG). With the advent of TSS release 05.4, the C++ User Interface (XMLS) process logs all auditing messages directly to the USRAULOG.

During installation, it is important to make sure that entries in the CONFCSV and MDBCSV are not truncated. Truncation can occur when transferring the files from the installation CD-ROM to the HP NonStop system using the HP NonStop Information Xchange Facility (IXF) file transfer program without increasing the default record size (132 characters) to a value large enough to accommodate the longest CONFCSV and MDBCSV entries. Truncation can also occur when using the HP NonStop TEDIT program to edit CONFCSV and MDBCSV entries. TEDIT also truncates lines to 132 characters. Therefore, use the HP NonStop EDIT program rather than TEDIT to modify CONFCSV and MDBCSV entries on the HP NonStop system.

Note: The CONFCSV and MDBCSV are specific to the TSS database for which they are used. For example, if you have different TSS databases for different XPNET systems (e.g., Production, Test, QA, Certification), you must configure a separate CONFCSV and MDBCSV on a separate subvolume for each system.

The key and key locator relationships between BASE24 files and the Transaction Security Services database files are described in detail in [section 2](#).

Security data maintenance

Hardware encrypted key information for the Transaction Security Service process is maintained using the ACI desktop user interface. The following table shows you the relationships between ACI desktop user interface windows and the files used by the Transaction Security Services process.

User interface windows	TSS file
Authentication Profile Configuration	Card Verification File (CARD_VRFCTN)
	Chip Authentication File (Chip_Authentication)
	Diebold Number Table File (DIEBOLD_NUM_TBL)
	Dynamic Card Verification File (DYN_CARD_VRFCTN)
	EMV Authorization File (EMV_Security)
	IBM DES Verification File (IBM_DES)
	Identikey PIN Verification File (Identikey)
	NCR PIN Verification File (NCR_PIN_VRFY)
	Visa PVV Verification File (Visa_PVV)
BASE24 KEYF Configuration ¹	Key File (KEYF)
Channel Security Configuration	Channel Security File (Channel_Security)
DUKPT Derivation Key Profile Configuration	Derivation Key File (Derivation_Key)
Interface Security Configuration	Interface Security File (ICHG_SECURITY)
Security Device Configuration ²	Security Device Configuration File (SEC_DEV_CONFIG)

¹ The BASE24 KEYF Configuration window is used to maintain data in the BASE24 Key File (KEYF) rather than a Transaction Security Services file.

² This window maintains security device configuration parameters similar to the LCONF assigns and params used to configure hardware security modules (HSMs) in an existing BASE24 system.

Context-sensitive ACI Online Help is provided for each of the above windows in the ACI desktop. All database maintenance activity performed from the ACI desktop user interface can be audited. Java server properties, in the form of `db.audited.db-service-name`, for the Java User Interface servlets process, and the `UI_AUDIT_LEVEL` environment attribute for the C++ User Interface process, allow delete, insert, update, and view actions on TSS files to be

logged. For the `db.audited.db-service-name` properties, *db-service-name* is the name of a Java database service that maintains the data in a particular TSS file. For more information on `db.audited.db-service-name` properties and a list of the properties supported for the TSS environment, refer to the **ACI Java Infrastructure Java Server Reference Guide**. For more information on auditing, refer to the **ACI Desktop User Interface Manual**.

Security data maintenance through the ACI desktop user interface is discussed in more detail in [section 2](#).

Configuration

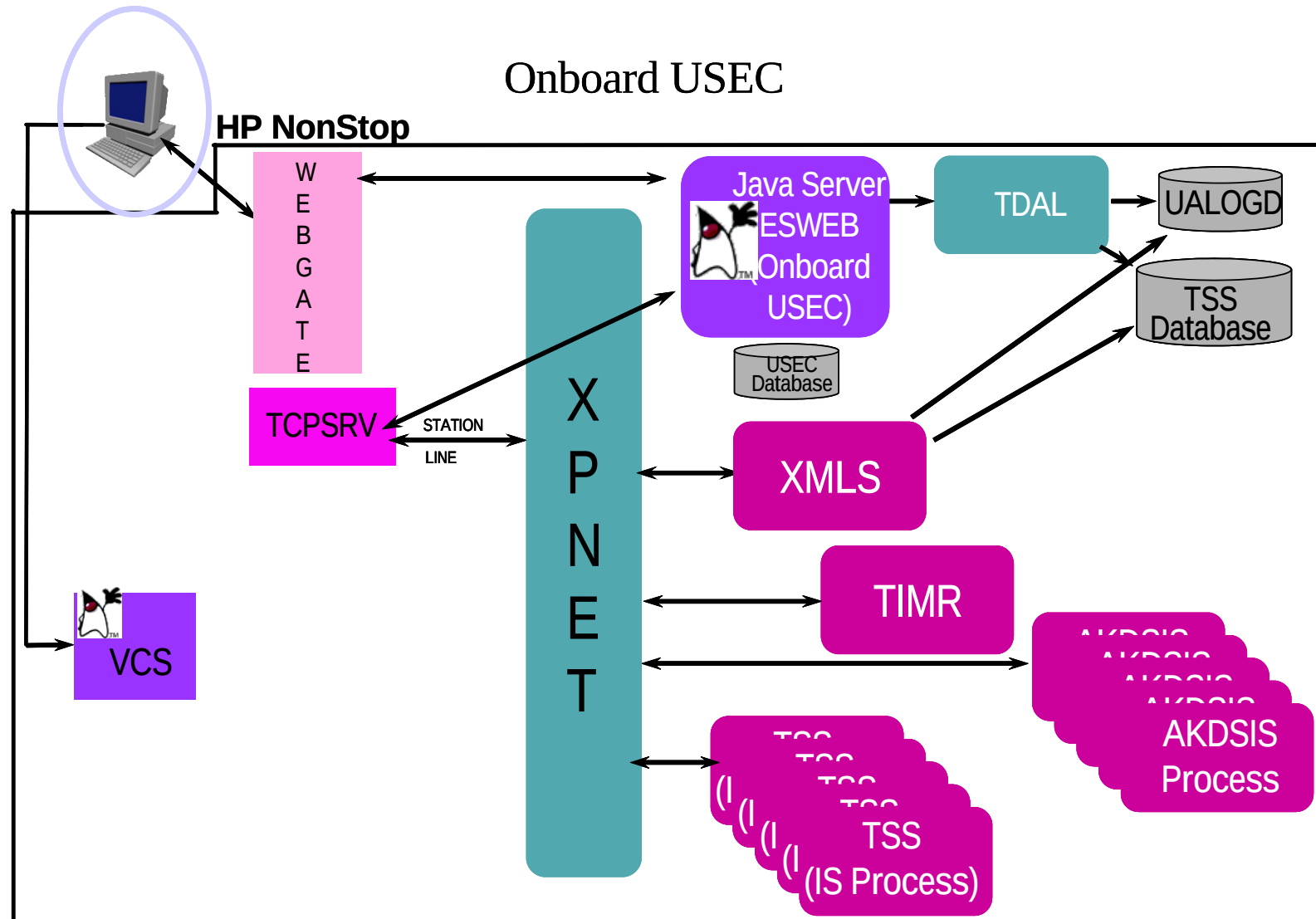
Each Transaction Security Services process and AKDS process must be configured as an application process to the XPNET system. In addition, Timer processes and User Interface Server processes must also be configured. The Timer processes set timers for keys stored in the database accessed by the Transaction Security Services process. The User Interface Server processes interface with the ACI desktop user interface, the User Security (USEC) application, and the TSS database.

ACI desktop user interface clients communicate with the HP NonStop system using the ACI Web Services (formerly WebGate) function of the ACI Communication Services (formerly ICE) product, and a servlet container.

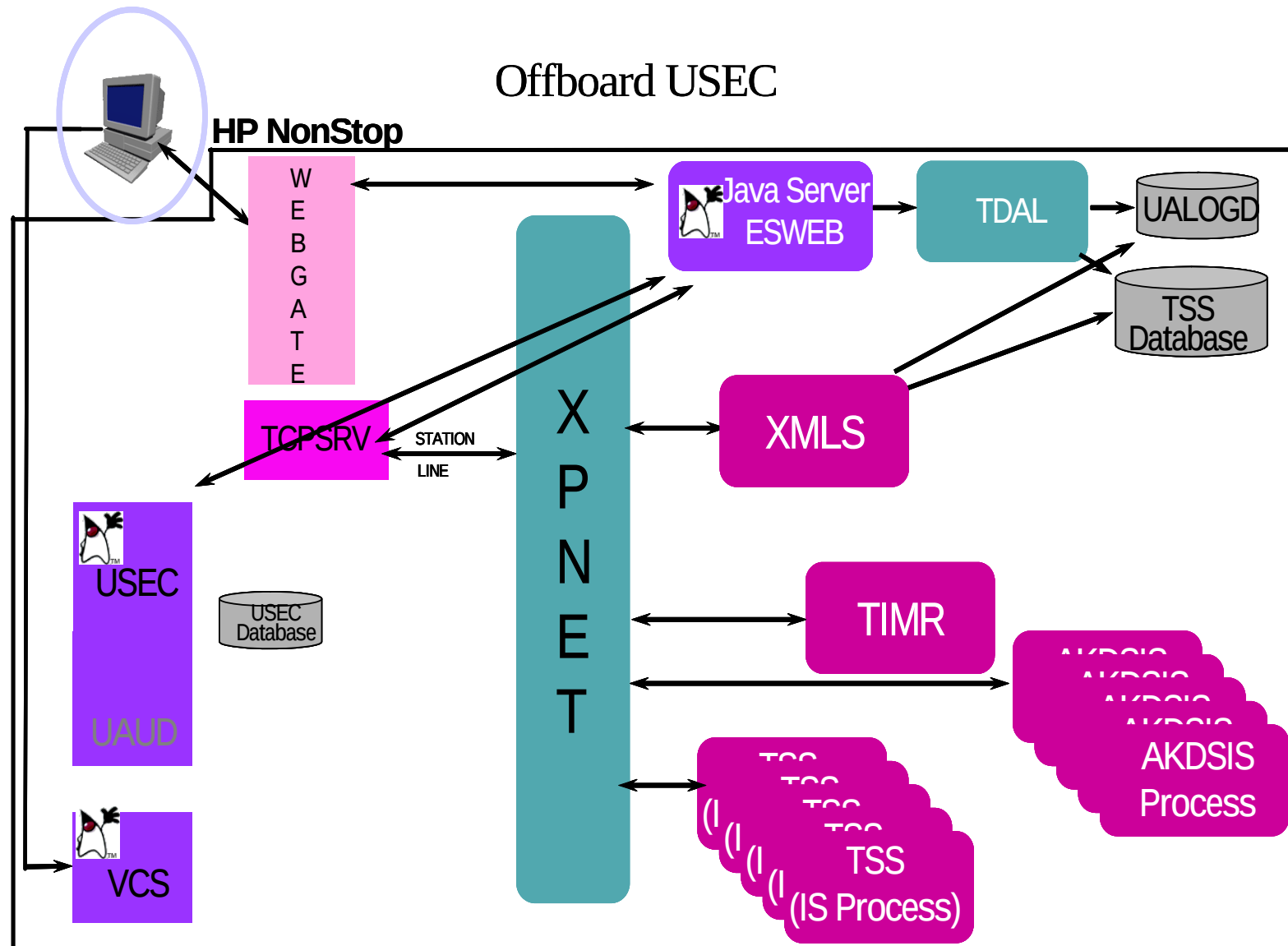
ACI Web Services provides Web server capability to the HP NonStop system and connects to ACI desktop user interface clients using Hypertext Transfer Protocol (HTTP) or HTTP over SSL (HTTPS) and Transmission Control Protocol/Internet Protocol (TCP/IP). The ACI Web Application Services - Java Services servlet container (ESWEB) process controls communication between ACI Web Services (formerly WebGate) and a Java Virtual Machine (JVM). Java User Interface servlets run within the ESWEB process. If the Java User Interface servlets receive a request that needs to be processed by the C++ User Interface (XML Server) process, the request is *passed through* the TCPSRV process and XPNET to the C++ User Interface (XML Server) process and the response is sent back through XPNET to the ESWEB process. The USEC application runs *onboard* within the ESWEB process or as a general Java process under OSS (if installed on the HP NonStop platform). The servlet container executes in a JVM and is responsible for invoking and managing resources. The relationships among all of the ACI desktop user interface components in the TSS environment are depicted in the illustrations on the following pages.

Note: The ACI Web Application Services - Java Services servlet container (ESWEB) process, along with the Version Checker and USEC applications, can reside on or off the HP NonStop platform. If they reside on the HP NonStop platform, the USEC application can run *onboard* (i.e., within the ESWEB process) or *offboard* (i.e., as a separate OSS process). If they reside on the HP NonStop platform, they are all started from TACL and run under OSS. If they reside off the HP NonStop platform, they must run in a Windows environment with ACI Web Application Services - Java Services version 5.0.28 or greater. A TCP/IP DAL Interface (TDAL) process, which runs under Guardian, is used to access the Enscribe database on the HP NonStop platform whether the ESWEB process resides on or off the HP NonStop platform. The TCPSRV process also runs under Guardian and both the TDAL and TCPSRV processes are

started from TACL. The ESWEB process uses a TCP/IP socket to connect with the TDAL process. The following illustration assumes these applications are installed on the HP NonStop platform and that the USEC application is running onboard the ESWEB process.



The next illustration depicts these applications installed on the HP NonStop platform with the USEC application running offboard the ESWEB process.



The detailed configuration for all of these entities is performed during installation and is beyond the scope of this guide.

Use of the Transaction Security Services process by BASE24 processes is controlled through LCONF assigns and params. BASE24 processes send messages directly to the Transaction Security Services processes identified in the SECURE-DEV1 and SECURE-DEV2 assigns, or in the SECURE-DEV-xx assigns, using a write/read interface when the SECURE-DEV-TYP param is set to a value of TSS in the LCONF. The SECURE-DEV-ACCESS-TYP param indicates whether a BASE24 application process communicates with a primary and secondary Transaction Security Services process identified by the SECURE-DEV1 and SECURE-DEV-2 assigns or with up to 20 Transaction Security Services processes identified by SECURE-DEV-xx assigns, where xx is any two characters. The SECURE-DEV1 assign specifies the primary Transaction Security Services process, while the SECURE-DEV2 assign specifies the secondary Transaction Security Services process. The SECURE-DEV-xx assigns specify up to 20 Transaction Security Services processes accessed on a *round-robin* (i.e., rotating) basis.

Some LCONF assigns and params are not used when implementing the Transaction Security Services process. For example, the SECURE-DEV-CONF-CMD100 and SECURE-DEV-CONF-TXT params are not used because they are replaced by similar parameters entered from the Security Device Configuration window.

A detailed discussion of how each LCONF [assign](#) and [param](#) related to HSMs need to be configured when using the Transaction Security Services process is provided in section 3.

Integration in a BASE24 system

When a BASE24 process requires the use of an HSM, the security utilities call the Transaction Security Services process specified in the SECURE-DEV1, SECURE-DEV2, or SECURE-DEV-xx assign with the key locator value and the requested function to be performed by the HSM. The Transaction Security Services process uses the key locator value to retrieve the actual hardware key value from the Transaction Security Services database.

The process uses the value of the transaction origin (based on the value in the ORIGINATOR field in the STM, PSTM, or TSTM) to determine whether it retrieves the PIN encryption key from the Channel_Security or the ICHG_SECURITY. For example, if the ORIGINATOR field is set to a value of 1 or 2, the process retrieves the PIN encryption key from the Channel_Security. If the ORIGINATOR field is set to a value of 7, the process would retrieve the PIN encryption key from the ICHG_SECURITY.

The process uses the HSM command information passed by the calling process to retrieve other key information needed to process the request. For example, if the HSM command requested specifies IBM DES (3624-format) PIN verification, the process retrieves the PIN verification key information from the IBM_DES.

After all the required key information is retrieved, the process formats the request to the HSM with the correct key values and returns the response to the requesting BASE24 process.

The illustration on the following page depicts how the Transaction Security Services process integrates into BASE24 transaction security processing.

The security utilities bound into BASE24 application processes communicate with a Transaction Security Services process using a direct write/read interface when a security function requiring the use of a hardware security module is needed, except for some public key cryptography functions used with the BASE24 Automated Key Distribution System add-on product. These functions require nowaited I/O communication with the Transaction Security Services process.

Each BASE24 process can communicate with either two Transaction Security Services processes (one primary and one secondary) or with up to 20 Transaction Security Services processes on a round-robin (i.e. rotating) basis. The Transaction Security Services process can be replicated as needed for load balancing and transaction throughput.

Implementation requirements

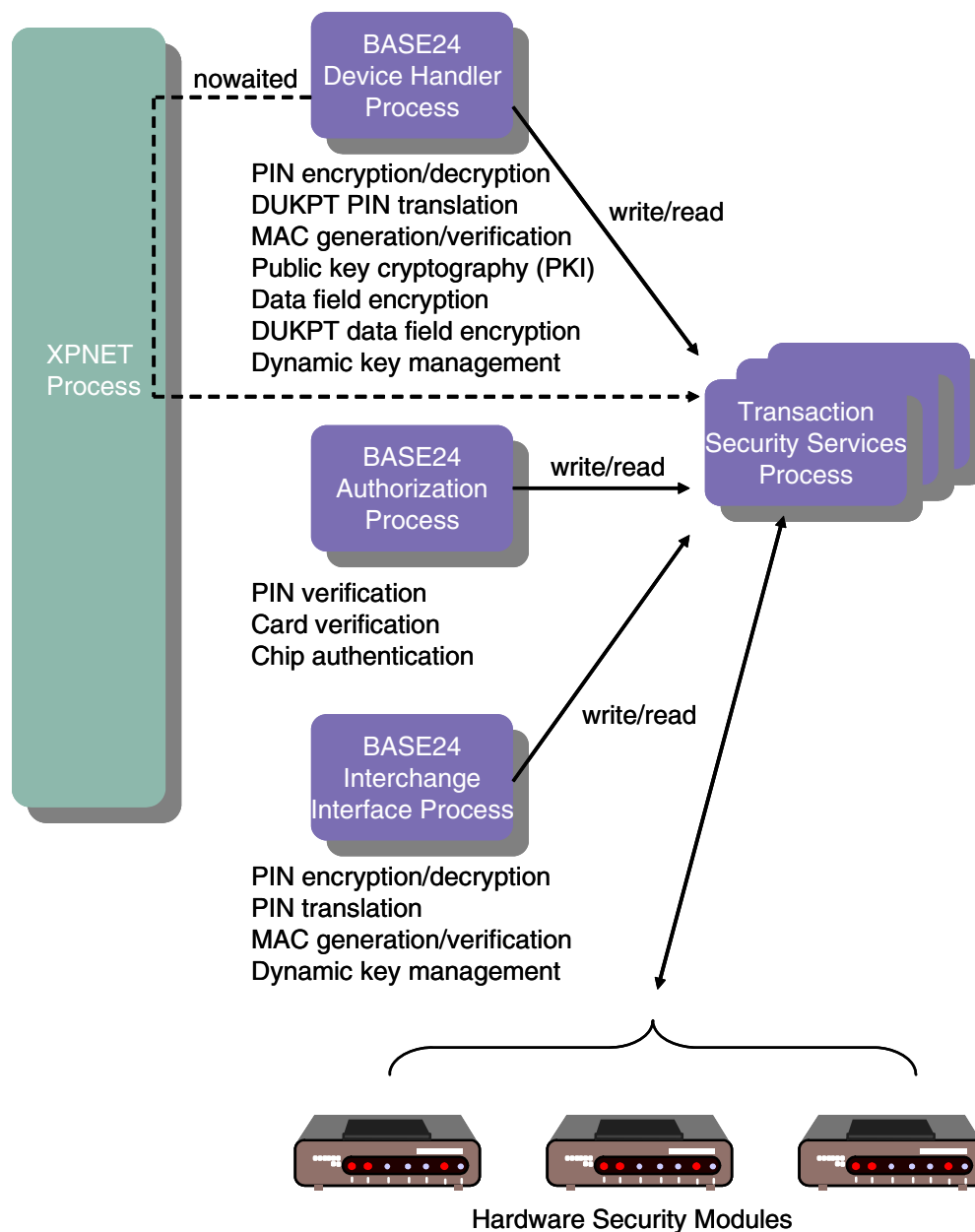
Use of the Transaction Security Services process is required in the following situations:

- When implementing the new Atalla key block format (i.e., migrating to AKB version software in Atalla HSMs)
- When implementing triple DES in ATM and POS devices
- When implementing triple DES across the BASE24 system
- When implementing the BASE24 Automated Key Distribution System add-on product or secure logons and logoffs to and from an interchange

The Transaction Security Services process supports both single DES and triple DES operations. It also supports Atalla keys stored in either variant or Atalla Key Block (AKB) format, however, it does not support a mix of keys in variant and AKB formats. After the decision is made to convert to the new AKB format, all keys in a BASE24 system must be converted at the same time.

Software support

The Transaction Security Services process requires a minimum of BASE24 release 6.0, version 2 software. ACI recommends that the BASE24 user be on a current Hewlett-Packard (HP) NSK-supported operating system to obtain maximum performance and support. The Transaction Security Services process is currently compatible with the HP NSK Guardian D46 (or higher) operating system on a K-series server. On an S-series server, it is compatible with the HP NSK Guardian Gxx (or higher) operating system. In addition, BASE24 release 6.0 requires the Transaction Management Facility (TMF). BASE24 release 6.0 software also requires that the BASE24 user be on the XPNET 3.0 or later system.



The XPNET process is used for nowaited PKI communication.

The TSS environment uses the XPNET process primarily for process startup and management. Except for certain public key commands required for the BASE24 Automated Key Distribution System add-on product, the TSS environment does not use the XPNET process for messaging. Messaging is performed directly using interprocess I/O.

Note: The ACI desktop user interface requires additional software. These software requirements are discussed under the [“Transaction Security Services environment support”](#) topic provided later in this section.

For BASE24 release 6.0, version 9, ten releases of Transaction Security Services (TSS) are supported: 04.1, 04.4, 05.2, 05.4, 06.2, 06.4, 07.4, 08.2, 08.2sp1, and 08.2sp2. Some functions of the Transaction Security Services process are supported only with TSS release 08.2, while others were introduced in earlier releases. The functions introduced in a TSS release are supported in all subsequent TSS releases. For example, the functions introduced in release 04.1 are supported in all subsequent TSS releases, while the functions or capabilities introduced in release 07.4 are supported only in release 07.4, 08.2, and service packs 1 and 2 for 08.2. The following paragraphs describe the functions or capabilities introduced in each release and list the releases in which the functions or capabilities are supported. Because the functions introduced in releases prior to 04.1 are available in all six subsequent TSS releases, they are not listed here.

Critical fixes

The following Transaction Security Services process functions or capabilities were introduced in a critical fix and are supported in all available TSS releases:

- The PIN_VRFY_FAIL_RETRY_IND Environment attribute specifies whether the Transaction Security Services process retries a PIN verification HSM operation using the previous key from the Channel Security Configuration File (Channel_Security) when the HSM returns a valid fail error (i.e., the cardholder PIN is considered to be invalid).

Post Release 11.1 (May 2012)

The following Transaction Security Services process functions or capabilities are supported only when TSS post release 11.1 or later is installed.

- Support for the American Express Card Security Code (CSC) Version 2.0 algorithm to verify American Express cards encoded with an iCSC.

Release 11.1

The following Transaction Security Services process functions or capabilities are supported only when TSS release 07.4 or later is installed.

- Interac Flash EMV scheme

Release 09.2

No new Transaction Security Services process functions or capabilities were added for release 09.2.

Release 08.2, service packs

No major Transaction Security Services process functions or capabilities were included in service packs 1–6 for release 08.2.

Release 08.2

The following Transaction Security Services process functions or capabilities are supported only when TSS release 08.2 or later is installed.

- Service packs
- VSP process control command (Lists the version level and service pack level information installed in a process)

Release 07.4

The following Transaction Security Services process functions or capabilities are supported only when TSS release 07.4 or later is installed.

- Support for the USEC application to run *onboard* (i.e., within the ESWEB process) instead of *offboard* (i.e., as a separate process).
- Chip authentication program (CAP) token validation for Thales e-Security hardware security modules (described in section 2 and section 17).
- Support of the following functions for SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs):
 - EMV 2000 and CCD support
 - Storage of PIN and MAC exchange keys (i.e., terminal master keys) in the Channel_Security rather than in the HSM
 - Message data encryption and decryption (described in section 6)
 - Proof-of-endpoint processing for interfaces
- Full or configurable message encryption for the BASE24 Standard POS Device Handler (SPDH) module (described in section 6)
- Support for metadata external memory tables that are used to access CONFCSV and MDBCSV information from memory rather than from disk (described in section 7)
- Message authentication for large messages using data chaining in Atalla variant and AKB NSPs, Banksys DEP HSMs, SafeNet (Eracom) ProtectHost White Mark II and AMB HSMs, and Thales e-Security HSMs
- Support of the following enhancements for the BASE24 Automated Key Distribution System add-on product:
 - Easier KEYGEN process control command configuration and submission
 - Certificate validity date verification
 - Certificate random nonce validation
 - Support for Wincor Nixdorf signature firmware installed on NCR hardware

- ❑ Support for Diebold signature emulation firmware
- ❑ Support for Triton devices

Note: AKDS support for Triton devices requires a CSM.

- Public key infrastructure (PKI) support for interchange interfaces to generate and verify signatures during the Logon exchange.
- Support for the following ACI desktop user interface enhancements:
 - ❑ Control of debugging and tracing for a user session
 - ❑ Ability to display the user ID logged on to an ACI desktop
 - ❑ Support for pluggable authentication with third party Java Authentication and Authorization Service (JAAS) authenticators
 - ❑ Field filtering at the user level rather than by permissions

Refer to the ***ACI Desktop User Interface Manual*** and the ***ACI Java Infrastructure Java Server Reference Manual*** for detailed descriptions of these features.

Release 06.4

The following Transaction Security Services process functions or capabilities are supported only when TSS release 06.4 or later is installed.

- Support for Banksys Data Encryption Peripheral (DEP) hardware security modules (described in section 4 and section 15).
- BASE24 Automated Key Distribution System add-on product support for shared Wincor Nixdorf certificate devices (i.e., Wincor Nixdorf certificate firmware running in 912 emulation or NCR NDC emulation mode) (described in section 15).
- Dynamic card verification (described in section 1 and section 13).
- EMV CCD authorization (described in section 2 and section 17).
- Message authentication support for APACS 30/40 point-of-sale devices.
- Full message data encryption between the BASE24 system and a host (described in section 6).
- User auditing enhancements (described in section 1)

Release 06.2

The following Transaction Security Services process functions or capabilities are supported only when TSS release 06.2 or later is installed.

- The ACI Web Application Services - Java Services servlet container process replaces the Servlet Engine (ESWEB) process in this version. This provides for multi-threaded execution within the container by isolating the single-threaded DAL layer into a smaller stand-alone process that can be dynamically replicated by an agent process as required for system load.

- Support for rebuilding and warmbooting external memory tables (also referred to as OLTPs because they are accessed in the online transaction processing path) from the OLTP Warmboot Management window.
- DES and Triple DES support for APACS 30/40 point-of-sale devices.
- Verification of CVD1 and CVD2 in a single call to the Transaction Security Services process.
- AKDS support for NCR certificate emulation firmware

Release 05.4

The following Transaction Security Services process functions or capabilities are supported only when TSS release 05.4 or later is installed.

- The C++ User Interface (XML Server) process logs auditing messages directly to the User Audit Log File (USRAULOG). The UAUD application, Audit Store and Forward File (Audit_Store_and_Forward), and Audit Store and Forward Configuration File (AUDIT_SAF_CONFIG) are no longer used.
- Support for Bull BNTng hardware security modules.
- BASE24 Automated Key Distribution System add-on product support for shared Wincor Nixdorf signature devices (i.e., Wincor Nixdorf firmware running in 912 emulation or NCR NDC emulation mode) and the AKDS Capacity Report.
- AKDS support for Tidel devices.

Release 05.2

The following Transaction Security Services process functions or capabilities are supported when TSS release 05.2 or later is installed.

- All user security validation origination from the Java User Interface servlets (ESWEB process). This eliminates the configuration formerly required for the C++ User Interface (XML Server) process to communicate with the USEC application.
- Access of Channel Security File (Channel_Security) records from memory or from disk (described in [section 2](#)).
- Additional security device configuration parameters, including the ability to automatically re-establish communication with a hardware security device after the device has reached its maximum consecutive errors limit (described in [section 3](#)).
- Enhanced process tracing facilities. Enables you to configure, filter, and trace C++ processes in the TSS environment using new ACI desktop user interface windows instead of entering process control commands to execute traces (described in [section 10](#)).

Release 04.4

The following Transaction Security Services process functions or capabilities are supported only when TSS release 04.4 or later is installed.

- American Express card security code (CSC) verification
- Automatic restart by the calling BASE24 process when communications with a Transaction Security Services process fails
- Derived unique key per transaction (DUKPT) PIN encryption for the BASE24-pos Hypercom Device Handler (HPDH) (described in [section 2](#))
- EMV 2000 authorization (described in [section 2](#) and [section 16](#))
- Encrypted IBM DES 3624-format decimalization table support
- SafeNet (Eracom) ProtectHost White Australian Major Banks (AMB) host security module (HSM) support (described in [section 4](#))
- AS2805 6.2 and AS2805 6.4 PIN blocks (described in [section 4](#))
- Master File Conversion utility support for the BASE24 Automated Key Distribution System add-on product (described in [section 15](#))
- *Round-robin* (rotating) access to up to twenty Transaction Security Services processes
- Triple DES support for Triton Device Handler processes

Release 04.1

The following Transaction Security Services process functions or capabilities are supported only when TSS release 04.1 or later is installed.

- Derived unique key per transaction (DUKPT) PIN encryption and message authentication for the BASE24-pos Standard POS Device Handler (SPDH) (described in [section 2](#))
- EMV derivation key index support (described in [section 2](#))
- Interac key management (described in [section 2](#))
- ISO format 1 PIN blocks (described in [section 4](#))
- Check Digit Validation and Generation utility (described in [section 16](#))
- File partitioning (described later in this section)

Note: The Node Controller application and the auditing database for the UAUD application are no longer used or required by the ACI desktop user interface in this release. The UAUD application forwards all auditing messages it receives from the C++ User Interface (XML Server) process to the Java User Interface servlets (ACI Web Application Services - Java Services servlet container) process to be logged to the User Audit Log File (USRAULOG).

Security module support

A number of vendors offer Tamper Resistant Security Modules (TRSMs) that are designed to perform PIN protection and verification, as well as other security-related functions within a physically secure environment. BASE24 currently supports the following security module models from five vendors—Atalla, Banksys, Bull, SafeNet (Eracom), and Thales e-Security:

Atalla Network Security Processor (NSP) models A8000, A8100, A8150, A9000, A9100, A9150, A10000E, A10100, A10150, and A10160.

All Axx100 models can run using the older variant key format or the Atalla Key Block (AKB) format. These devices can be configured to support the variant format or the AKB format simply by installing the appropriate CD-ROM in the device and rebooting. All Axx100, Axx150, and Axx160 models come standard with an AKB software CD-ROM. Appending the letter V to the Axx100, Axx150, or Axx160 model number when ordered (e.g., A10100V, A10150V, A10160V, A10160V, A9100V, A9150V, A8100V, and A8150V) ensures that the module is shipped from the factory with the variant software CD-ROM as well. If the letter V is not appended to the model number when ordering, the Axx100, Axx150, or Axx160 model is shipped from the factory only with the standard AKB software CD-ROM.

All other Atalla models can also be ordered with variant or AKB version software preloaded. While on-site upgrades from variant to AKB software are possible on the other Atalla models, this process is not as convenient as on the Axx100, Axx150, or Axx160 series.

- Banksys Data Encryption Peripheral (DEP) range of hardware security modules (HSMs)
- Bull BNTng family of hardware security modules (HSMs).
- SafeNet (Eracom) ProtectHost White Mark II and ProtectHost White Australian Major Banks (AMB) host security modules (HSMs).
- Thales e-Security (Racal) Host Security Module models RG7100 and RG7110, RG7300 and RG7310, RG7400, and HSM 8000.

Thales e-Security Host Security Modules were formerly named Racal-Guardata Host Security Modules.

If you are implementing full triple DES or the Atalla Key Block (AKB) format, please contact Atalla support to upgrade to the most current software available, and to confirm your current hardware is capable of supporting the necessary functions. The following minimum software versions are required for triple DES and/or AKB: AKB version 2.30 or 1.3 software. The 2.30 version software is used for A10100 AKB models while the 1.3 version software is used for A10000 AKB models. Note that Atalla recommends version 2.9 software as the minimum version when using the older variant key block format. This version is available from Atalla as a free upgrade for a limited time. It is the only version that allows an upgrade to the AKB versions of software, and it allows customers to make future upgrades in-house, without an HP CE visit. Note that the 2.90 equivalent software on the Axx100-V models is version 1.10. The Axx150 and Axx160 must run with version 3.0C or greater of the variant or Atalla System Program CD-ROM. Also note that CD-ROMs for the Axx100 are not compatible with the Axx150/Axx160 or vice versa.

Bull BNTng HSMs require software with a minimum version of 7.

Banksys DEP HSMs require firmware with a minimum version of FINTXS 1.0.e.

SafeNet (Eracom) HSMs require software with a minimum release date of 01/01/2000 or later. All software releases dated 01/01/2000 or later support triple DES.

Thales e-Security RG7000-series HSMs require a minimum of version 5.05/1.05 firmware to support full triple DES. The HSM 8000 requires a minimum software version of 2.0 to support full triple DES. (Triple DES support for DUKPT PIN encryption and message authentication is only provided in version 2.0 or greater software.)

If you are not implementing full triple DES, Atalla NSPs can use version 2.90 software, and Thales e-Security HSMs can use version 5.05/1.05 firmware. Atalla models A9000 and A10000E with version 2.90 software, Atalla models A8100-V, A9100-V, and A10100-V with version 1.10 software, and all Thales e-Security HSMs with 5.05/1.05 firmware, allow triple DES encrypted PIN blocks to be translated to single DES encrypted PIN blocks, and vice versa. The Atalla A8000 model does not support this functionality.

If you are implementing public key cryptography, you must use an Atalla NSP model with a minimum of AKB version 2.40 or 1.4 software, a Thales e-Security HSM with a minimum of version 6.02 firmware and an optional DSP plug-in (for RSA functions), or a Banksys DEP HSM with the appropriate firmware. For Atalla NSPs, the public key cryptography commands needed to support public key cryptography are available on Axx100, Axx150, and Axx160 models (i.e., A8100, A8150, A9100, A9150, A10100, A10150, and A10160) only.

The hardware security module requirements for the Transaction Security Services process are provided in [section 4](#).

ATM device support

The following BASE24-atm Device Handler processes currently support the use of the Transaction Security Services process, although the Transaction Security Services process is not required when using single DES for devices supported by these processes:

- Diebold 10XX/478X (includes Wincor Nixdorf signature- or certificate-based firmware running in 912 emulation mode and Diebold signature emulation firmware)
- Cash Source Plus
- IBM 3624, Version 8
- NCR 5XXX (includes 56XX and *personaS* devices as well as Wincor Nixdorf signature- or certificate-based firmware running in NDC emulation mode and NCR certificate emulation firmware; triple DES is not supported on 50XX devices)
- Tidel
- Triton

The Transaction Security Services process is required when ATM devices supported by these Device Handler processes are configured for triple DES. Contact your ACI account manager for information on the availability of other ATM Device Handler processes for use with the Transaction Security Service process.

The Diebold 10XX/478X, NCR 5XXX, and Triton Device Handler processes support the BASE24 Automated Key Distribution System (AKDS) add-on product in both leased line and dial-up modes. The Tidel Device Handler process supports AKDS in dial-up mode only.

POS device support

BASE24-pos Device Handler modules will be enhanced as needed to support the use of the Transaction Security Services process for triple DES, although the Transaction Security Services process is not required when using single DES. The Transaction Security Services process is required when POS devices are configured for triple DES.

The BASE24-pos Standard POS Device Handler (SPDH) module supports the use of the Transaction Security Services process for both triple DES or single DES, DUKPT, and dynamic key management.

The BASE24-pos Hypercom Device Handler (HPDH) module supports the use of the Transaction Security Services process for both triple DES or single DES PIN encryption and DUKPT. PIN keys can be single- or double-length. Hypercom does not support triple-length keys. MAC encryption keys are not supported because Hypercom utilizes a proprietary message authentication algorithm that supports only single-length keys.

The BASE24-pos APACS 30/40 Device Handler module supports the use of the Transaction Security Services process for both triple DES or single DES PIN encryption, DUKPT, and message authentication.

The BASE24-pos VISA II Device Handler module supports the use of the Transaction Security Services process for single DES only.

Interface support

The following Interchange Interface processes support the use of the Transaction Security Services process:

- BASE24 ISO (BIC ISO) Interchange Interface
- Banknet ISO Interchange Interface
- EPS-Net Interchange Interface
- Equens Interchange Interface
- LINK ISO Interchange Interface
- MDS/Cirrus ISO Interchange Interface
- MDS Maestro Interchange Interface
- VisaNet ISO Interchange Interface

Other Interchange Interface processes also support the use of the Transaction Security Services process. Refer to the **BASE24 Release 6.0 Implementation Guide** for a complete list of all Interchange Interface processes that support the Transaction Security Services process.

The Transaction Security Services process is not supported in releases prior to release 6.0, however, triple DES can be supported in certain release 6.0, version 3 Interchange Interface processes without the Transaction Security Services process in a *translate mode* only. For more information on this latter option, refer to the **BASE24 Transaction Security Manual**.

New key activation

When an acquirer Interface process generates a new key using the Transaction Security Services process, the new key is not activated (i.e., the current index for the key is changed to point to the new key) until after the Interface process receives a response from the secondary key processor or co-network indicating that the new key was successfully processed. In TSS releases prior to 02.4, the new key is activated immediately after it is generated. New key activation was implemented in a critical fix for TSS release 02.4.

PIN blocks and pad characters

The Transaction Security Services process supports the following PIN block formats:

- AS2805 6.2 and 6.4
- PIN/PAN (ANSI or ISO Format 0)
- PIN/PAD
- DUKPT

DUKPT PIN blocks are supported between a BASE24 Standard POS Device Handler (SPDH)-compliant device and the SPDH module and between a Hypercom device and the BASE24-pos Hypercom Device Handler (HPDH) module. Both the SPDH and HPDH modules translate DUKPT PIN blocks into PIN/PAN PIN blocks.

- ISO Format 1 (for Bull BNTng, Banksys DEP, and Thales e-Security HSMs only)



Most card associations do not allow the use of PIN PAD PIN block formats, as they are subject to certain cryptographic attacks. The use of ISO Format 0 (ANSI) PIN block formats is preferred.

The Transaction Security Services process also supports the destination PIN block formats required for the BASE24-atm EMV add-on product. These PIN block formats are described in [section 17](#) of this guide. The ISO Format 1 PIN block format is described in [section 4](#) of this guide. The AS2805 6.2 and AS2805 6.4 PIN block formats are also described in [section 4](#) of this guide. The PIN/PAN, PIN/PAD, and DUKPT PIN block formats are described in the **BASE24 Transaction Security Manual**.

A pad character is used for both PIN/PAD and ISO format 1 PIN blocks. The TSS database allows for the pad character to be set to any valid hexadecimal character (i.e., 0–9 and A–F) for PIN/PAD and ISO format 1 PIN blocks, except when using Thales e-Security security modules. The Thales e-Security security module restricts the pad character to hexadecimal F. Therefore, you must set the PIN Pad Character field to a value of F when using Thales e-Security HSMs for both PIN/PAD PIN blocks and ISO format 1 PIN blocks.

For PIN/PAN PIN blocks, the hexadecimal character F is always used for padding.

Public key cryptography

A public key scheme is one in which the sender and receiver each have their own unique pair of keys. One key in the pair is the *secret* (private) key. As its name implies, only its owner can know the secret key—it cannot be shared with any other party. The other key in the pair is the *public* key. The public key can be shared between the sender and receiver without threat to the integrity of the key pair. A mathematical relationship exists between the secret key and the public key, but it is virtually impossible to derive one from the other. When data (e.g., an ATM master key) is encrypted by one of the keys in the pair, only the other key can decrypt it. Thus, public-key cryptography differs from traditional DES cryptography in that the key used to encrypt data is different than the one used to decrypt the data. For this reason, PKI is referred to as an *asymmetrical* security scheme while DES is referred to as a *symmetrical* security scheme. More importantly, the key used to encrypt the data cannot be reused to decrypt the data. This characteristic means that the encrypting key (i.e., the public key) has no value to an adversary if exposed or intercepted while being transmitted.

The Transaction Security Services process uses public key cryptography for the following functions:

- To remotely and securely download new master keys to ATMs (BASE24 Automated Key Distribution System)
- To provide a secure means of logging on to and off an interchange

BASE24 Automated Key Distribution System

With the industry-wide move to improve key management practices and recent mandates requiring unique keys per terminal, security administrators have been searching for a cost-effective way to generate, manage, and distribute increasing numbers of ATM keys. To address this need, ACI has worked closely with leading ATM vendors, security device vendors, and major card associations to develop the BASE24 Automated Key Distribution System add-on product, also referred to as AKDS. This add-on product automates the otherwise error-prone and labor-intensive process of generating, protecting, and hand-entering ATM master keys at each ATM in your system.

The BASE24 Automated Key Distribution System add-on product uses public-key cryptography, often referred to as PKI, to remotely and securely establish the initial master key (i.e., the A-key) in the encrypting PIN pad (EPP) of an ATM. The BASE24 Automated Key Distribution System add-on product is a PKI implementation based on the Rivest-Shamir-Adleman (RSA) scheme, which is an algorithm based on prime number mathematics.

AKDS is available for NCR 5XXX, Diebold 10XX/478X, Tidel, and Triton Device Handler processes. These Device Handler processes must use a special Transaction Security Services process called the AKDS process to communicate with security modules for BASE24 Automated Key Distribution System add-on product cryptographic functions. The AKDS

process contains all of the components of the Transaction Security Services process plus the components required to support the BASE24 Automated Key Distribution System add-on product. AKDS functions are supported in hardware only; they are not supported in software. Atalla NSPs, Banksys DEP HSMs, and Thales e-Security HSMs are supported for the BASE24 Automated Key Distribution System add-on product; Bull BNTng and SafeNet (Eracom) HSMs are not supported for the BASE24 Automated Key Distribution System add-on product.

The configuration requirements and procedures for implementing the BASE24 Automated Key Distribution System add-on product are discussed in section 15 of this processing guide. This section also lists the security module commands required to support the BASE24 Automated Key Distribution System add-on product. The procedures for downloading public and master keys from BASE24 to a Tidel or Triton ATM are also discussed in section 15.

The procedures for downloading a public key or master key from BASE24 to an ATM are provided in the **BASE24-atm NCR 5XXX Device Support Manual** for NCR devices, Diebold devices using NCR certificate emulation firmware, and NCR devices using Wincor Nixdorf signature- or certificated-based firmware (running in NDC emulation mode), and in the **BASE24-atm Diebold 10XX/478X Device Support Manual** for Diebold devices, signature-based devices using Diebold signature emulation firmware, and Diebold devices using Wincor Nixdorf signature- or certificated-based firmware (running in 912 emulation mode).

Secure interchange logons and logoffs

Public key cryptography provides the means for certain BASE24 interfaces to securely log on to and off an interchange. For example, the SAMA SPAN2 Interchange Interface uses public key cryptography to validate digital signatures when the BASE24 system initiates an 1804 logon or logoff to the interchange.

Dutch security extensions

The Transaction Security Services process provides cryptographic support in Thales e-Security HSMs for the following security functions used by interfaces to Equens N.V.—a national switch in the Netherlands:

- Large message MAC generation and verification
- NCR PIN verification method *
- RVT/RVS key management *

The cryptographic functions used to support the above functions are available only on Thales e-Security HSMs. The functions identified with an asterisk (*) require custom HSM commands that are not supported in the standard Thales e-Security product. All of these cryptographic functions are described in section 4.

Large message MAC generation and verification

Equens supports message authentication codes (MACs) for large messages using the ANSI X9.19 method and double-length MAC encryption keys. The data encrypted can be binary or expanded hexadecimal format. This type of message authentication is provided by the Thales e-Security MS (Generate MAC using ANSI X9.19 Method for a Large Message) host command. The MS command is used primarily for support of the ANSI X9.19 method.

NCR PIN verification

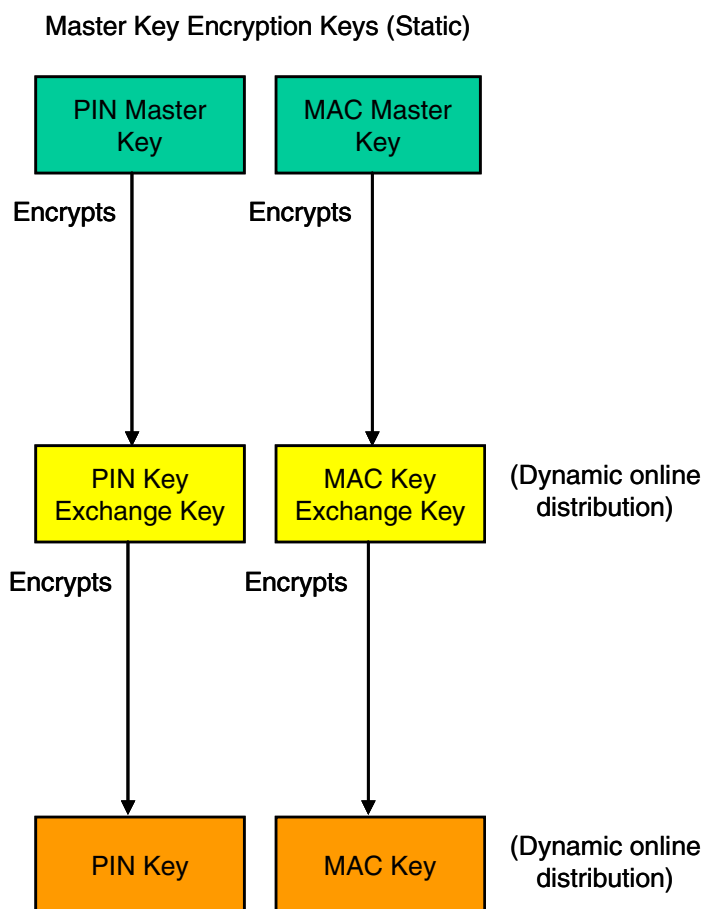
Equens uses the NCR PIN verification method. The NCR PIN block can be encrypted under a terminal PIN key or under an interface encryption key. The Transaction Security Services process uses keys stored in either the Channel_Security or ICHG_SECURITY to decrypt the PIN block and the PIN verification key stored in the NCR_PIN_VRFY to verify the PIN. The NCR PIN verification key is entered on the NCR tab of the Authentication Profile Configuration window. This tab is described in section 2.

The cryptographic function used to support NCR PIN verification is available only on customized Thales e-Security HSMS. This cryptographic function is described in section 4.

RVT/RVS key management

Equens uses a three-tier hierarchy of keys for key management instead of the two-tier key hierarchy used throughout the rest of BASE24. This three-tier key management scheme uses static master keys and random values to encrypt and decrypt transport (key exchange) and session (working) keys for dynamic online distribution.

A two-tier hierarchy of keys is used for most interfaces where working keys (session keys) are encrypted under key exchange keys (transport keys) for online distribution. For an Equens interface, a third tier of keys called master key encryption keys are used to encrypt key exchange keys for online distribution. This three-tier hierarchy of keys is depicted in the following illustration.



Master key encryption keys comprise the top tier in the three-tier key hierarchy. Key exchange keys comprise the second tier, while working keys make up the third tier. Keys in the second and third tiers are dynamically generated using a 64-bit or 128-bit random value. The random value used to generate key exchange keys (transport keys) is referred to as the RVT. The random value used to generate working keys (session keys) is referred to as the RVS. Both the RVT and the RVS can be generated by the Equens interface or can be received from Equens.

The static offline master keys used to encrypt key exchange keys are entered on the Master Keys tab of the Interface Security Configuration window. This tab is described in [section 2](#).

The cryptographic functions used to support RVT/RVS key management are available only on customized Thales e-Security HSMs. These cryptographic functions are described in [section 4](#).

Transaction Security Services environment support

A number of additional facilities are required in the Transaction Security Services environment to support the Transaction Security Services process. These facilities are described below.

User interface

C++ User Interface Server processes (XML Server processes) and Java User Interface servlets are required to support requests from ACI desktop user interface clients running on user workstations. The ACI Web Services HTTP Server and XPNET Servlet Engine processes serve as an interface between multiple ACI desktop user interface clients and the server-side User Interface Server processes. If the Java User Interface servlets running in a Servlet Engine process receive a request that needs to be processed by the C++ User Interface (XML Server) process, the request is routed through the XPNET process to the C++ process. You can configure multiple URLPATHs to the ACI Web Services HTTP Server to access different TSS networks (e.g., test, quality assurance, production, etc.). In this case, desktop users can select the URLPATH to be accessed after they log on to the ACI desktop. For detailed information on configuring URLPATHs, refer to the **WebGate Administrators Guide**.

All Java-based ACI desktop user interface components, including clients, servlets, Version Checker server, and User Security (USEC) application require the Java Virtual Machine (JVM) runtime environment. In addition, the JVM must be 32-bit rather than 64-bit for compatibility with the TSS database.

All ACI desktop user interface components, except for clients, can run on the HP NonStop system or off platform in a Windows environment. Refer to the **BASE24-eps Hardware and Software Requirements** document for more information. On ACI desktop user interface client workstations, the URLPATHs a user can connect to are configured in a Remote.cfg file while the ACI desktop user interface client is configured in a Local.cfg file.

On the HP NonStop system, the Java User Interface servlets must run under the Open System Services (OSS) environment, which coexists with the Guardian environment and runs under the NonStop Kernel operating system. The OSS environment has a structure and feel similar to a UNIX system.

The C++ User Interface (XML Server) processes run under the Guardian environment.

When an ACI desktop user interface client is started, the Version Checker client communicates with the Version Checker server, also running in the JVM runtime environment, to compare the current version of Java archive files (JARs) used by ACI desktop client applications to the current published versions of these files in the JAR Repository. If the current published versions are newer than the client versions or are missing from the client set, the Version Checker server returns the required JARs to the Version Checker client. The client then notifies the user that the JAR files need to be updated. The user can then select the JARs to be updated. The Version Checker server communicates with Desktop clients over a raw socket TCP/IP connection.

The C++ User Interface Server processes and Java User Interface servlets (running within a servlet container provided by Servlet Engine processes running under the OSS environment) perform the following functions:

- Validate ACI desktop user interface user requests with the USEC application
- Read, add, update, and delete items in TSS files
- Load TSS files into memory tables and warmboot the processes that read the memory tables
- Initialize, enable, disable, or status hardware security modules
- Create optional audit records for logging in the User Audit Log File (USRAULOG)

Timer process

A Timer process is required to set the expiration times for old (i.e., “previous”) keys in the TSS database. If needed, you can add Timer processes for performance and throughout purposes or for redundancy. When a Transaction Security Services process needs to set a new timer, it accesses the available Timer processes in a round-robin manner.

Note: The Timer process does not set the timers used for dynamic key management thresholds. These timers are still maintained within the respective Interface or Device Handler processes or modules.

ACI Java server framework

The Java User Interface servlets for the ACI desktop user interface are based on ACI's Java Server Framework (AJSF), which is part of ACI's Java Infrastructure (AJI). The ASJF includes the following files:

- Application Configuration File (APPLCNFG)
- Event File (Event)
- Event Documentation File (Event_Document)
- External Connection File (EXTRCNCT)
- Listener File (LISTENER)
- Process File (PROCESS)

All of the above files are populated during the installation process. After installation, you can view the data contained in the AJSF files from the ACI Java Server Framework windows (accessed from **System Operations > Server Management**) and the Event Configuration and Management window (accessed from **System Operations > Event > Event Management**). If you have any custom software modifications (CSMs), you may need to change some of the configuration settings on the ACI Java Server Framework windows. The following table shows you the ACI desktop user interface windows that enable you to view or change data in the ACI Java Server Framework database files.

ACI desktop window	ACI Java Server Framework database file
Process Configuration	Process File (PROCESS)
Event Configuration and Management	Event File (Event)
	Event Documentation File (Event_Document)
Application Configuration	Application Configuration File (APPLCNFG)
Listener Configuration	Listener File (LISTENER)
Connection Configuration	External Connection File (EXTRCNCT)

You can also monitor ACI Java Server Framework services using the server administration windows accessed from **System Operations > Server Management > Server Administration**. From these windows, you can view various statistics on how ACI Java Server Framework services are being used, such as how many database connections are open, what external connections are established, etc.

Refer to the **ACI Java Infrastructure Java Server Reference Guide** for detailed information on the ACI Java Server Framework configuration requirements for the Java User Interface servlets used for the ACI desktop user interface.

User Security (USEC) application

The User Security (USEC) application provides operator security for ACI desktop user interface functions.

Each time the Java User Interface servlets receive a message from an ACI desktop user interface client, they send a request to the USEC application to verify that the message is authentic and that the operator has the appropriate security to perform the requested task or function. For failover purposes with a multiple node configuration, all USEC applications share a common security database. In the event of a failure, the Java User Interface servlets can switch over to another USEC application, without requiring the user to log in again. In addition, the security database is backed up. If the USEC application is unable to access the primary security database, it can switch over to the backup database, without interruption.

Additional software is required for the USEC application to run on the HP NonStop system. The application must run under the Open System Services (OSS) environment, which coexists with the Guardian environment and runs under the NonStop Kernel operating system on an HP NonStop system.

The OSS environment has a structure and feel similar to a UNIX system. In addition, the USEC application requires one of the following, depending on whether you are implementing the USEC database using SQL or Enscribe:

- If you are implementing an SQL database, runtime NonStop SQL/MP with Java bound in and either the NonStop SQL/MP database driver or the NonStop SQL/MX database driver is required. ACI recommends the NonStop SQL/MX database driver because of its better error reporting capabilities. This option is not available for K-series servers.
- If you are implementing an Enscribe database, ACI's Java Database Connectivity (JDBC) driver called the Java DAL Database Connectivity (JDAL) driver is required for Java server applications to interface with the Enscribe database.

The USEC application can run onboard (i.e., within the ESWEB process) or offboard (i.e., as a separate process). If the USEC application runs offboard, it must be configured as a general process in the OSS configuration file.

Default user IDs

Four preconfigured user IDs are provided for the Transaction Security Services environment at installation. These user IDs and their passwords are listed in the following table and described below.

User ID (name)	Password
TSSSysAdmin	TSYSAdmin01
SecAdmin	SecAdmin01
SysAdmin	SysAdmin01

User ID (name)	Password
TSSAdmin	TSAdmin01



Note: For security reasons, the system administrator or security administrator should change the passwords for these preconfigured user IDs immediately after installation. Cross-industry best practices recommend that you do not use vendor-supplied default passwords, that you use passwords containing both numeric and alphabetic characters with a minimum password length of at least seven characters, that you change passwords at least every 90 days, and that a new password is not the same as any of the last four passwords used.



The TSSSysAdmin and TSSAdmin preconfigured user IDs are specific to the Transaction Security Services environment while the SecAdmin and SysAdmin preconfigured user IDs are generic to all BASE24-eps applications. The preconfigured TSSSysAdmin user ID has access to all TSS windows and functions available on the ACI desktop, including the User Security and ACI Java Server Framework windows and functions. The preconfigured TSSAdmin user ID has access to all TSS and ACI Java Server Framework windows and functions on the ACI desktop, except for the User Security windows and functions. The SecAdmin user ID has access to the User Security windows and functions only. The SysAdmin user ID has access to almost all windows and functions in a BASE24-eps system, including all TSS and User Security, and ACI Java Server Framework windows and functions. Therefore, you can use the TSSSysAdmin, SecAdmin, or SysAdmin user IDs to set up initial security access for other users, however, the SysAdmin user ID has access to several windows and functions that are not applicable to the TSS environment. You can retain or delete default user IDs as needed for your security environment. Cross-industry security best practices recommend that you identify all users with a unique user ID before allowing them to access system components.

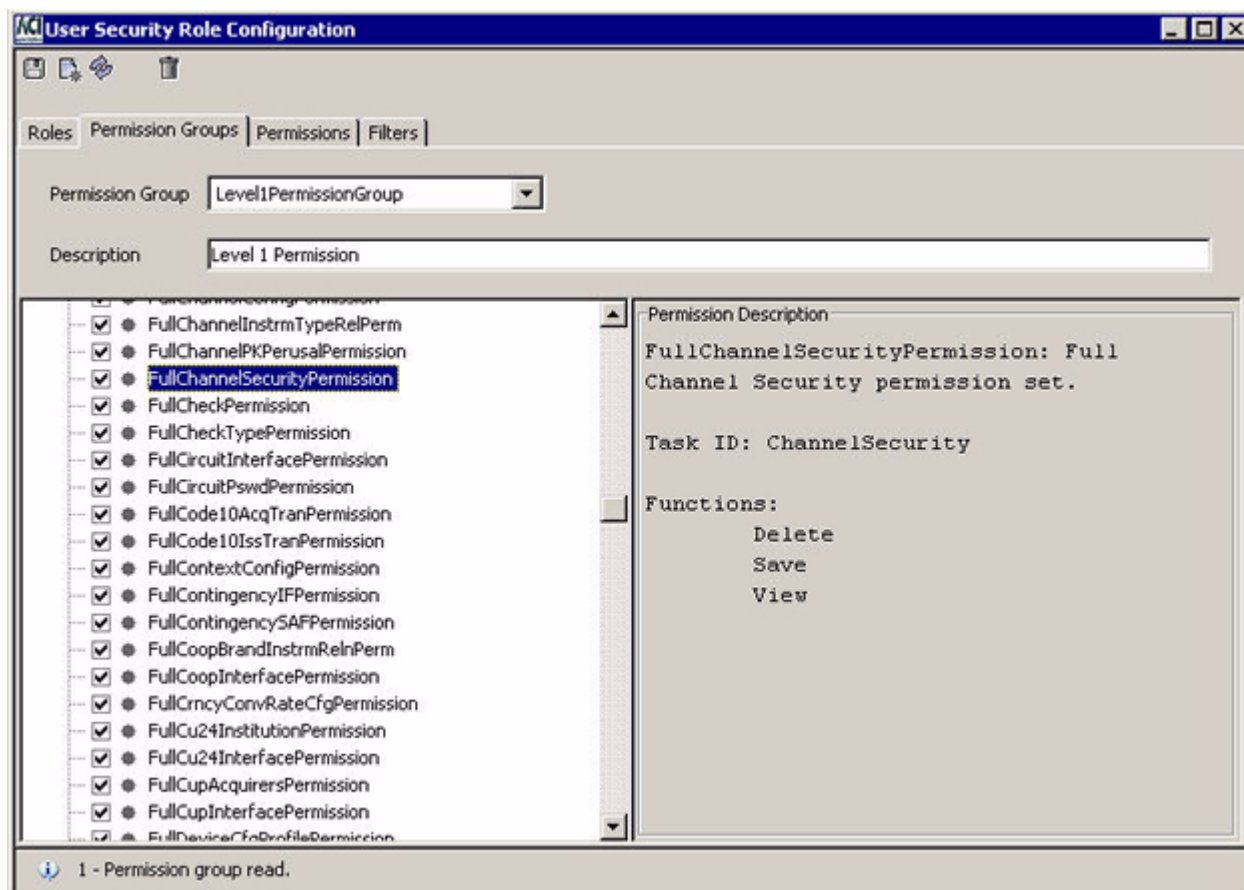
The following table shows you the default roles configured for each of the default user IDs provided at installation.

User ID	Role
SecAdmin	SecAdminRole
TSSAdmin	TSSAdminRole
TSSSysAdmin	TSSSysAdminRole
SysAdmin	SysAdminRole

You can view the individual permissions configured for each of these default user IDs on the Permission Groups tab of the User Security Role Configuration window. The TSSSysAdminPermissionGroup is assigned to the TSSSysAdmin user ID, the

TSSAdminPermissionGroup is assigned to the TSSAdmin user ID, the SysAdminPermissionGroup is assigned to the SysAdmin user ID, and the SecAdminPermissionGroup is assigned to the SecAdmin user ID.

The individual permissions configured to these permission groups are displayed in a list in the lower left-hand portion of the window. Check marks are displayed next to permissions configured to the group. You can right click on a selected permission to view a description of the permission, the task IDs associated with the permission, and the functions available, as shown in the following illustration.



Note: The permissions list displays all permissions available in a BASE24-eps system. The permissions checked for the TSSSysAdminPermissionGroup include all of the full permissions available for the TSS environment. If a full permission is not checked for this permission group, you can ignore it.

The following table summarizes the permissions available for each permission group configured for default user IDs. Permission IDs for each application window in the the TSS environment are listed in the configuration attributes table for each applicable window in the **ACI Desktop Windows Reference**.

User ID	Permission group	Permissions
TSSSysAdmin	TSSSysAdminPermissionGroup	All permission IDs listed for TSS.
TSSAdmin	TSSAdminPermissionGroup	All permissions IDs listed for TSS except for the following: FullUSCtlPermission FullUSGCfgPermission FullUSPPPermission FullUSRCfgPermission
SecAdmin	SecAdminPermissionGroup	FullUSCtlPermission FullUSGCfgPermission FullUSPPPermission FullUSRCfgPermission
SysAdmin	SysAdminPermissionGroup	All permission IDs listed for TSS as well as additional permission IDs listed for BASE24-eps, that are not relevant to the TSS environment for BASE24 customers.

A specific task ID is associated with each window on the ACI desktop user interface. Task IDs are combined with permissions to provide delete, save, and/or view functions to users for each window. For windows that retrieve a list of values for a field from another window, you must at least give a user view access to the window from which the list of values are retrieved. In addition, some windows that are based on a channel or interface framework, also require access to the server component for the framework. A specific task ID (i.e., ChannelConfiguration and InterfaceConfiguration) is associated with each of these server components.

Refer to the **ACI Desktop User Interface Manual** and ACI Online Help for the ACI desktop for a complete list of the task IDs needed to access each window available for TSS users.

User auditing

Optional user auditing enables you to audit all online operator activity performed through the ACI desktop user interface. The database auditing mechanism enables you to track all adds, updates (before and after images), and deletes to the application database by user ID and time of change. If you update or delete a profile, all data records for the profile are updated or deleted and are included in the audit trail. User logouts, password verifications, and password changes are also recorded. When a TSS file is audited, records are written to a User Audit Log File (USRAULOG). You can configure the files audited when your ACI system

is installed and control the extent to which files are audited. For full auditing, header information as well as detail information about the before and after images of the affected record are logged to the USRAULOG. For medium auditing, only the header information is logged to the USRAULOG.

The Java User Interface servlets (ACI Web Application Services - Java Services servlet container) process writes auditing records directly to the USRAULOG. The C++ User Interface (XML Server) process can write auditing records directly to the USRAULOG or it can write nowaited messages to an Audit Store and Forward File (Audit_Store_and_Forward) for transmission to the UAUD application. A value of Y for the USE_LOCAL_USER_AUDIT environment attribute indicates that the XML Server writes directly to the USRAULOG. A value of N for this attribute indicates that the XML Server process writes to the Audit_Store_and_Forward. The only situation in which the XML Server process cannot write directly to the USRAULOG is if the USRAULOG has a database type (e.g., SQL) that is different from the database type (i.e., Enscribe) for the rest of the TSS database.

You can extract user auditing records from the USRAULOG to a specified extract file or to the Audit Report Viewer window using the User Audit window on the ACI desktop user interface (accessed from **View > User Audit**). An application configuration property named report.userauditlog.loc specifies the directory where the extract file is created (e.g.: /user/B24es/ESWeb/reports/). Refer to the **ACI Desktop User Interface Manual** for detailed procedures on using the User Audit and Audit Report Viewer windows. The **ACI Java InfrastructureJava Server Reference Guide** provides a detailed description of the report.userauditlog.loc property.

You can also view user configuration and status information from the User Permissions and User Status windows, respectively, on the ACI desktop user interface. The User Permissions window (accessed from **View > User Permissions**) displays the assigned roles, permissions, and filters for each configured user ID. The User Status window (accessed from **View > User Status**) displays the last login date and time for a user ID, the number of failed logins, date and time of the last failed login, and the date and time the password was last changed for the user ID. Refer to the **ACI Desktop User Interface Manual** for detailed descriptions of both of these windows.

The UAUD application must be configured as a general process in the OSS configuration file.

Additional software is required for the UAUD application to run on the HP NonStop system. The application must run under the Open System Services (OSS) environment, which coexists with the Guardian environment runs under the NonStop Kernel operating system on an HP NonStop system.

Most of the user interface auditing in the TSS environment is controlled by db.audited.db-service-name properties configured for the Java User Interface servlets on the Application Configuration window of the ACI desktop user interface, where db-service-name is the name of a particular Java database service. For each Java database service, you can configure the servlets to perform no auditing, headers only auditing, or full auditing. For files maintained by the C++ User Interface process, the UI_AUDIT_LEVEL attribute for the Transaction Security Services environment indicates the level of user interface auditing performed by the C++ User Interface process. Using this attribute, you can configure the C++ User Interface process to perform no auditing, headers only auditing, or full auditing. Each of these options is explained in detail in [section 3](#).

Refer to the **ACI Java Infrastructure Java Server Reference Guide** for a list of all possible `db.audited.db-service-name` properties that apply to the TSS environment. If a TSS file and associated ACI desktop window is not represented in the list in the **ACI Java Infrastructure Java Server Reference Guide**, auditing for the file is controlled by the `UI_AUDIT_LEVEL` attribute.

The **ACI Desktop User Interface Manual** describes UAUD processing in further detail.

Secure sockets layer (SSL) protocol

Messaging endpoints of the ACI desktop user interface (i.e., Desktop clients, ACI Web Services HTTP Server, USEC, and UAUD applications) provide support for the Secure Sockets Layer (SSL) protocol. You can optionally configure any or all of the above ACI desktop user interface components to use the SSL protocol when the ACI desktop is installed. The ACI desktop uses Sun Microsystems' Java Secure Sockets Extension (JSSE) version 1.0.3_01. This version of JSSE supports SSL version 3.0.



Cross-industry best practices advocates using strong cryptography and security protocols such as secure sockets layer (SSL) / transport layer security (TLS) and Internet protocol security (IPSEC) to safeguard sensitive cardholder data during transmission over open, public networks.

Note: If you want to provide SSL support for the ACI Web Services HTTP Server, you must purchase it directly from Insession Technologies.

The SSL protocol helps ensure that user interface message data is not read by an unauthorized party (through the use of certificates) or tampered with during transmission over the user interface network (using data encryption). Hashing and encryption algorithms can be used to encrypt data transferred between network endpoints. Using SSL with Remote Method Invocation (RMI) or basic socket connections provides encryption security to the transmitted data. When an SSL-enabled server authenticates itself to an SSL-enabled client, the client receives a certificate to accept or reject. If the client accepts the certificate, both the client and server establish an encrypted connection.

If desired, you can use the trace facility provided with the ACI Communications Services product to trace SSL negotiation sequences for ACI Web Services. The ICE Probe utility provided on the ACI Web Services installation CD-ROM is a Windows-based tool that provides a user friendly interface for reading trace files.

SSL configuration requirements are discussed in the **ACI Desktop User Interface Manual** and the **ACI Java Infrastructure Java Server Reference Guide**.

Files

The Transaction Security Services environment requires the following additional files other than those security-related TSS files listed earlier. The assigns and params for these files, as well as the security-related TSS files listed earlier, are configured in the `CONFCSV`.

- The Environment File (Environment) contains attributes for the Transaction Security Services environment similar to params defined in the LCONF. The Environment Management window in the ACI desktop user interface enables you to configure these attributes. The Transaction Security Services attributes required for support of the Transaction Security Services process are described in [section 3](#).
- The Event File (Event) contains event messages that can be generated by processes in the Transaction Security Services environment. The Event Document File (Event_Document) contains event message and token documentation. The Event Configuration window in the ACI desktop user interface enables you to view the cause and remedy of event messages, modify the text and documentation for event messages, and suppress the generation of certain event messages.
- The Exception Log File (Exception_Log) contains Extensible Markup Language (XML) messages that the C++ User Interface process could not interpret. This file can be used as a debugging tool.
- The System Interface Message Delivery Service Destination File (MDSDEST) maps security device assign names to PPD names (e.g., ATALLA1 to \$BOX1). It also maps Transaction Security Services processes to PPD names to enable the use of INFO and STATUS process control commands for these processes.
- The Trace File (Trace) contains the output of process traces generated for traces configured in the Trace Configuration File (Trace_Config) and the Trace Filter Profile Configuration File (Trace_Filter).

The OSS environment and Servlet Engine JVM environment also require several additional files other than those security-related TSS files listed earlier.

Several files are required to support the ACI Java Server Framework. See the [“ACI Java server framework”](#) topic provided earlier in this section for a list of these files.

Several files are required to support file partitioning. See the [“File partitioning”](#) topic provided later in this section for a list of these files.

For detailed information on user security and auditing files, refer to the **ACI Desktop User Interface Manual**.

Data access layer (DAL) file access

The Data Access Layer (DAL) of BASE24-eps processes, including the Transaction Security Services process uses the data source assign (or metadata *long* name) rather than the metadata *short* name to access TSS files. For example, to access the Channel Security File, DAL uses the Channel_Security long name rather than the CSECD short name to read the CONFCSV to obtain the physical file location. Therefore, all TSS files are referenced by their long name throughout this manual, except where use of the long name is awkward or inefficient, as in graphics or tables. In these instances, the short name is used instead for convenience. The short name is also required to be used in the command syntax for some process control commands.

The following table relates the long name and short name to each file available in the TSS environment.

TSS file	Long name	Short name
Application Configuration File	APPLCNFG	APPLD
Audit Store and Forward Configuration File	AUDIT_SAF_CONFIG	ASAFCD
Audit Store and Forward File	Audit_Store_and_Forward	ASAFD
BNTng Header File	BNTng_Header	BNTHD
BNTng Initialization File	BNTng_Init_Data	BNTID
Card Verification File	CARD_VRFCTN	CRDVD
Channel Public Key Security File	CHAN_PKEY_SEC	CHNPkd
Channel Security File	Channel_Security	CSECD
Channel Security Future File ¹	CHAN_SEC_FUTURE	CSECFD
Chip Authentication File	Chip_Authentication	CAD
Data Source OLTP Relationship File	DS_OLTP_Relation	DOLTPD
Derivation Key File	Derivation_Key	DRVKD
Diebold Number Table File	DIEBOLD_NUM_TBL	DNTD
Dynamic Card Verification File	DYN_CARD_VRFCTN	DCRDVD
EMV Authorization File	EMV_Security	EMVSD
Environment File	Environment	ENVMTD
EPP Serial Number File	EPP_SERIAL_NUM	EPNUMD
Event Definition File	Event	EVENTD
Event Document File	Event_Document	EVDODD
Exception Log File	Exception_Log	EXLOGD
External Connection File	EXTRCNCT	EXTCND
File Partition Catalog File	file_part_catalog	FPCTGD
File Partition Fields File	file_part_fields	FPFLDD

TSS file	Long name	Short name
File Partition List File	file_part_list	FPLSTD
Host Public Key Security File	HOST_PUBLIC_KEY	HSPKD
IBM DES Verification File	IBM_DES	IDESD
Identikey PIN Verification File	Identikey	IDNTD
Interface Security File	ICHG_SECURITY	ISECD
Interface Security Future File ¹	ICHG_SEC_FUTURE	ISECFD
Listener File	LISTENER	LSTND
NCR PIN Verification File	NCR_PIN_VRFY	NCRD
OLTP Warmboot File	OLTP_Warmboot	OOLTPD
Process File	PROCESS	PROCD
Security Device Configuration File	SEC_DEV_CONFIG	SECDCD
Trace Configuration File	Trace_Config	TRCCFD
Trace File	Trace	TRCD
Trace Filter Profile Configuration File	Trace_Filter	TRCFD
User Audit Log	USRAULOG	UALOGD
Visa PVV Verification File	Visa_PVV	VPVVD

¹Although these files are installed for TSS, they are not used. There are no TSS messages to activate future keys and promote them to the current keys table.

File formats

A WinZip archive (S18880 TSS File Format Folders.zip) located on **InfoLink > HELP24 > Solution Attachments**, provides two files for each TSS version—an HTML file and a COBOL text file. The HTML file (.htm) contains a report of all the tables, element names, element data types, and element descriptions, etc., for all the tables in the TSS application. This report includes data source and data element descriptions and is similar to DDLs. The COBOL text file provides the layout of all the TSS files and data elements in COBOL format.

Operations

As stated previously, the Transaction Security Services process can be controlled through commands entered from an XPNET network control facility. Additional operational controls are available for the Transaction Security Services environment. Each of these operations functions is described below.

Startup and shutdown

Detailed step-by-step procedures for startup and shutdown of Transaction Security Services environment components is provided in appendix A and B, respectively, in the **BASE24-eps and TSS HP NonStop & HP Integrity NonStop Install Guide**.

Process control

You can control all TSS-related processes, lines, and stations configured to the XPNET system from a network control facility. Refer to the **NET24-XPNET Network Control Command Reference Manual** for a discussion of process, line, and station control commands. Typically, these commands are used to configure and start the Transaction Security Services processes, Timer process, C++ User Interface processes, Java User Interface servlets (ACI Web Application Services - Java Services servlet container) process, and the UAUD components.

You can control the ACI Web Services HTTP Server and associated resources using the Node Operator Facility (NOF) of the ACI Communications Services product. Refer to the **NOF Command Reference and Resource Definition Guide** for detailed information on the commands used to control these resources.

Additional operational control mechanisms are available through the Process Control facility of the Transaction Security Services environment. Through this facility, security personnel operators can enter commands to perform the following functions:

- Rebuild external memory tables (OLTPs)¹
- Warmboot a rebuilt external memory table (OLTP)¹ or start rebuilding an internal memory table
- Refresh individual Channel Security File (Channel_Security) records from disk to an internal memory table
- Initialize, enable, and disable all hardware security modules (HSMs) or a single HSM
- Obtain the current status of one or all HSMs
- Download the Diebold number table (DNT) into the memory of an HSM
- Convert keys in the TSS database from non-AKB format (i.e., variant format) to AKB format

- Erase the old LMK key pair for a Thales e-Security HSM or replace the current MFK with the pending MFK for an Atalla NSP after converting keys using the Master File Key Conversion utility.
 - Generate a new RSA key pair for the host BASE24 system
This process control function is only available if you have licensed the BASE24 Automated Key Distribution System add-on product or the ability to securely log on and off an interchange.
- ¹ You can also use the OLTP Management window to rebuild and warmboot an external memory table (OLTP). This window is described in [section 11](#) of this guide.

All of these commands are entered from the Process Control window in the ACI desktop user interface. For detailed information on using the Process Control window to rebuild external memory tables (OLTPs), warmboot OLTPs, and start rebuilding internal memory tables, refer to [section 2](#). For detailed information on entering HSM commands, refer to [section 9](#) and [section 16](#) in this guide. For detailed information on generating a host RSA key pair, refer to [section 15](#) of this guide. For more information on the Process Control window, refer to [section 9](#) and the ACI Online Help provided with the ACI desktop user interface.

The process control mechanisms available for the Transaction Security Services process are depicted in the following illustration.

Process tracing

Through the ACI desktop user interface, you can configure, execute, and filter traces of C++ processes (e.g., Transaction Security Services processes) in the TSS environment. The Trace Configuration window (accessed from **System Operations > Trace > Trace**) enables you to configure and execute a process trace. The Trace Filter Profile Configuration window (accessed from **System Operations > Trace > Trace Filter Profile**) allows you filter the messages for which trace output is provided. For detailed information on using these trace facilities, refer to [section 10](#).

Master file key translation

ACI provides a Master File Key Conversion tool that enables you to translate hardware-encrypted keys in the TSS database from encryption under one master key file (i.e., MFK for Atalla or LMK pair for Thales e-Security) to encryption under another master key file. This conversion tool is executed from the Master File Key Conversion window, accessed as **Configure > Transaction Security > Master File Key Conversion** on the ACI desktop. Using this tool, you can translate all the keys in the TSS database or just the keys of a selected type. For detailed information on using this tool, refer to the [Master Key File Conversion](#) topic provided in section 16.

The Master File Key Conversion tool is not currently supported for Banksys DEP, Bull BNTng, or SafeNet (Eracom) HSMS.

Event management

Event messages generated by processes running in the Transaction Security Services environment on the HP NonStop system are sent to an EMS collector process and are displayed on the EMS log just like all BASE24 modules.

You should also monitor the EMS log for Transaction Management Facility (TMF) events that require attention. You can use the Event Configuration window in the ACI desktop user interface to view event message documentation for the Transaction Security Services environment. For detailed information on managing events for the TSS environment, refer to [section 8](#).

Note: Event messages generated by the ACI Web Services HTTP Server are documented in the **Messages and Codes Reference** in the ACI Application Services Suite documentation set.

Security device errors are mapped to error messages provided by the security utilities bound into BASE24 processes.

Transaction Management Facility (TMF)

All of the Enscribe files maintained by the Transaction Security Services process are audited by TMF. The Transaction Security Services process uses TMF to back out updates on a per-transaction basis. More sophisticated features of TMF, such as rollback and roll forward, are not required or used. For more information on TMF operations, refer to HP's ***NonStop TM/MP Operations and Recovery Guide***.

File partitioning

File partitioning enables you to partition a single TSS database file across multiple data sets. On an HP NonStop system, file partitioning allows you to expand the Enscribe limit of 16 partitions for a single file. File partitioning also spreads I/O across multiple data sets, thereby reducing the bottleneck associated with a single data set. Any database file in the TSS environment that is not accessed from an external memory table (OLTP) can be partitioned. You use the File Partition Configuration window (accessed from **System Operations > Database Management > File Partition**) to configure file partitions. Refer to the ***BASE24-eps File Partitioning User Guide*** for detailed information on using the File Partition Configuration window and procedures for partitioning files. Data entered on the File Partition Configuration window is stored in the following files:

- File Partition Catalog File (file_part_catalog)
- File Partition Fields File (file_part_fields)
- File Partition List File (file_part_list)

Data in the file_part_fields and file_part_list is accessed from external memory tables during processing.

Disaster recovery or dual-site configuration

Note: The following is an example of TSS files to replicate in a disaster recovery or dual-site configuration. Your files may differ based on your specific disaster recovery or dual-site processing requirements. You can change the TSS files to be replicated or not replicated, however, a file must be audited in order for it to be replicated.

Typically, you would replicate all TSS files that are audited by the Transaction Monitoring Facility (TMF) for disaster recovery purposes or dual-site use. In a typical TSS installation that includes the BASE24 Automated Key Distribution System add-on product, this would include the following files:

- BNTng Header File (BNTng_Header)
- BNTng Initialization File (BNTng_Init_Data)
- Card Verification File (CARD_VRFCTN)
- Channel Public Key Security File (CHAN_PKEY_SEC)

- Channel Security File (Channel_Security)
- Channel Security Future (CHAN_SEC_FUTURE)¹
- Chip Authentication File (Chip_Authentication)
- Data Source OLTP Relationship File (DS_OLTP_Relation)
- Derivation Key File (Derivation_Key)
- Diebold Number Table File (DIEBOLD_NUM_TBL)
- Dynamic Card Verification File (DYN_CARD_VRFCTN)
- EMV Authorization File (EMV_Security)
- Environment File (Environment)
- EPP Serial Number File (EPP_SERIAL_NUM)
- Event Document File (Event_Document)
- File Partition Catalog File (file_part_catalog)
- File Partition Fields File (file_part_fields)
- File Partition List File (file_part_list)
- Host Public Key Security File (HOST_PUBLIC_KEY)
- IBM DES Verification File (IBM_DES)
- Identikey PIN Verification File (Identikey)
- Interface Security File (ICHG_SECURITY)
- Interface Security Future (ICHG_SEC_FUTURE)¹
- NCR PIN Verification File (NCR_PIN_VRFY)
- OLTP Warmboot File (OLTP_Warmboot)
- Security Device Configuration File (SEC_DEV_CONFIG)
- Trace Filter Profile Configuration File (Trace_Filter)
- Visa PVV Verification File (Visa_PVV)

¹ Even though these data sources are delivered with BASE24 Transaction Security Services, the functionality provided by them is only available in BASE24-eps.

Typically, you would not replicate the following files, including all external memory table (OLTP) swap files:

- Application Configuration File (APPLCNFG)
- Audit Store and Forward Configuration File (AUDIT_SAF_CONFIG)²
- Audit Store and Forward File (Audit_Store_and_Forward)²
- BNTng Header external memory table swap file (BNTHOD)
- Card Verification external memory table swap file (CRDVOD)
- Chip Authentication external memory table swap file (CAOD)
- Derivation Key external memory table swap file (DRVKOD)
- Diebold Number Table external memory table swap file (DNTOD)

- Dynamic Card Verification external memory table swap file (DCRDOD)
- EMV Authorization external memory table swap file (EMVSOD)
- Event File (Event)
- Event external memory table swap file (EVTOD)
- Exception Log File (Exception_Log)
- External Connection File (EXTRCNCT)
- IBM DES Verification external memory table swap file (IDESOD)
- Identkey PIN Verification external memory table swap file (IDNTOD)
- Interface Security external memory table swap file (ISECOD)
- Listener File (LISTENER)
- NCR PIN Verification external memory table swap file (NCROD)
- Process File (PROCESS)
- Trace Configuration File (Trace_Config)
- Trace File (Trace)
- User Audit Log (USRAULOG)
- Visa PVV Verification external memory table swap file (VPVVOD)

² These files may or may not be used in TSS release 05.4 or later. A value of N for the USE_LOCAL_USER_AUDIT Environment attribute indicates they are used. A value of Y for this attribute indicates they are not used. These data sources only need to be used if the USRAULOG is maintained in a different type of database (e.g., SQL) from the rest of the TSS database (i.e., Enscribe).

Transaction processing examples

The Transaction Security Services process is used in the processing of every transaction that requires a hardware security module function. For transaction security functions performed in software, the Transaction Security Services process is not used.

Example transaction processing flows using the Transaction Security Services process are provided in [section 5](#).

2: Database relationships and security data maintenance

This section describes the relationships between BASE24 database files and the Transaction Security Services database files. It also describes how security data is maintained.

BASE24-atm terminal data files

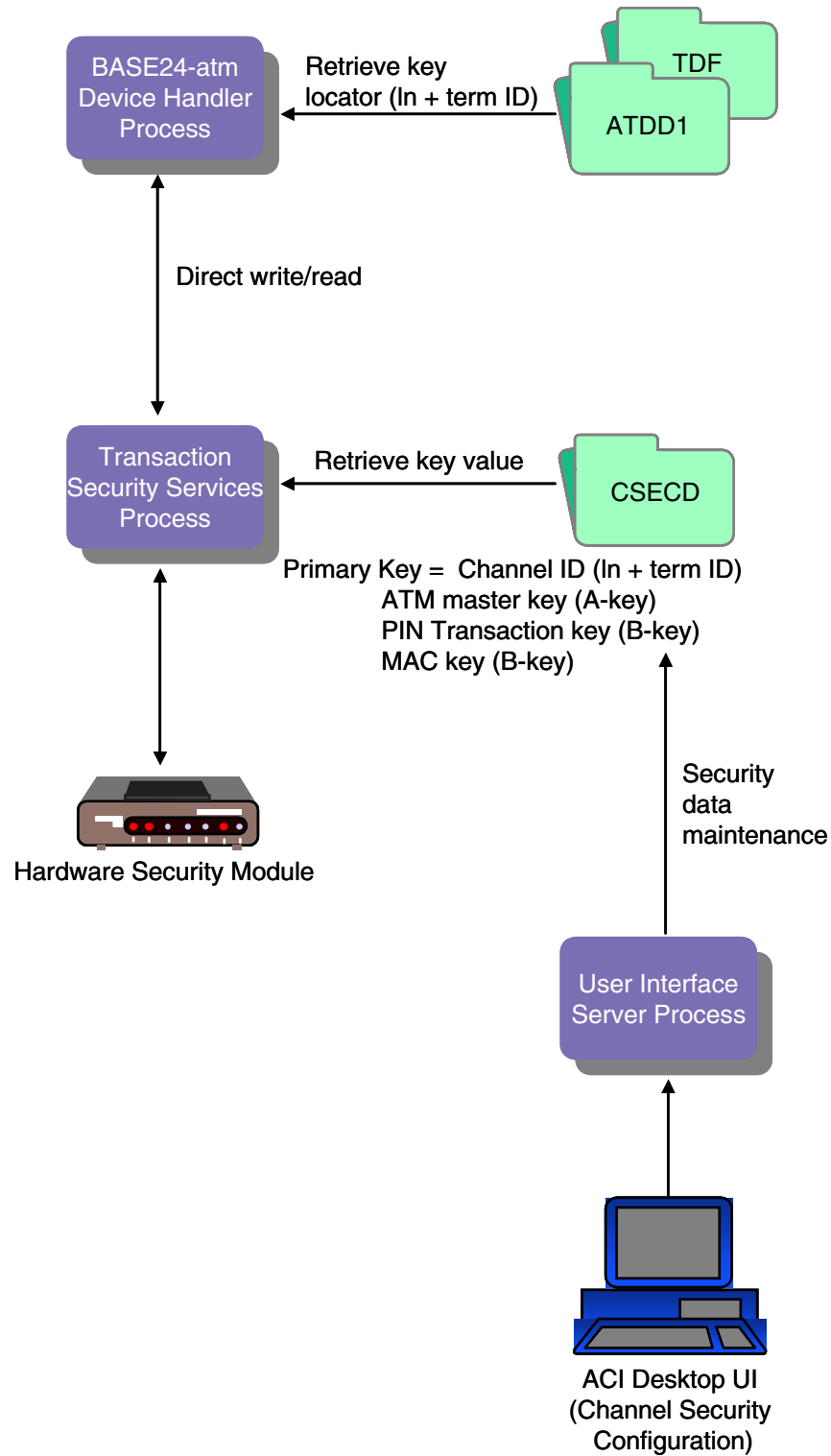
When the BASE24-atm Terminal Data files are converted, the MASTER KEY field on the device-specific ATD screen 9 for records that use hardware-encrypted keys when the Transaction Security Services process is implemented (i.e., the SECURE-DEV-TYP param is set to a value of TSS) are set to key locator values. In addition, these fields are not checked for all hexadecimal characters. If you change the key locator value in the MASTER KEY field of a converted record to a value other than the terminal ID, you must add a corresponding record for this key locator value on the Channel Security Configuration window.

Regardless of whether a record is converted, all encryption type fields (i.e., PIN ENCRYPT TYPE, PIN CHANGE ENCRYPT TYPE, and MAC SUPPORT TYPE TYPE fields on the device-specific ATD screen 9) for a particular terminal's record must contain compatible values. You cannot mix software encryption and hardware encryption types in the same record. For example, if the PIN encryption type value specifies to use a security device, the MAC support type value cannot specify that MAC support is in software, and vice versa.

The following diagram illustrates the relationship between BASE24-atm Terminal Data files and the Channel Security File (Channel_Security) after file conversion. The hardware encrypted key fields in the BASE24-atm Terminal Data files are set to the value of the two-character logical network identifier (specified by the LN-IND LCONF param if you are partitioning the Channel_Security) added to the beginning of the terminal ID, and the actual key values are stored in the Channel_Security. Data in the Channel_Security is maintained from the Channel Security Configuration window. The terminal ID value entered in the TERM ID field on BASE24-atm Terminal Data files (ATD) screen 1 must match the value entered in the Channel ID field on the Channel Security Configuration window. If you add additional terminal records to the BASE24-atm Terminal Data files, you enter the encryption keys into corresponding Channel_Security records from the Channel Security Configuration window on the ACI desktop user interface. Refer to [section 3](#) for more information on the LN-IND param.

Note: If an ATM is configured in the BASE24-atm Terminal Data files to send PINs in the clear to any interchanges on the BASE24 system, a Channel_Security record must be manually added for the encrypted intermediate key with a PIN block type of PIN/PAD.

If an Interchange Interface process supports the conversion of incoming encrypted PINs to clear PINs internally, an ICHG_SECURITY record must be manually added for the encrypted intermediate key with a PIN block type of ANSI.



Channel Security Configuration window

The Channel Security Configuration window enables you to configure keys and parameters required by ATM devices for PIN encryption and message authentication. The Data Encryption Exch Info, Data Encryption Key Info, Transaction Key Security, AS2805 Specific Keys, and German Info tabs on this window are not currently used for any ATM devices. Samples of the PIN Information tab and MAC Information tab of Channel Security Configuration window for a Diebold 10XX/478X device are shown below. For ATM devices, the ATM master key is used as the exchange key for both PIN and MAC encryption keys. Therefore, the same ATM master key must be used in the Exchange Key field on both the PIN Information and MAC Information tabs. Refer to the ACI Online Help provided with the ACI desktop user interface for detailed descriptions of this window and its fields.

Note: You must configure a security device using the Security Device Configuration window before you can configure channel security information using the Channel Security Configuration window. The Data Encryption tab is only enabled for Atalla NSPs and SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs). The Transaction Key Security tab is disabled for all types of security devices except for Thales e-Security HSMs.

Old key timer limit

The Old Key Timer Limit field located at the top of the Channel Security Configuration window specifies the maximum length of time, in seconds, that a PIN, MAC, or data encryption key can be used after a successful dynamic key exchange before it is automatically cleared.

This timer limits the amount of time that an old key can be used for PIN verification, message authentication, or data encryption after a successful key exchange has taken place.

Currently, this limit is only used for Triton ATMs.

SafeNet (Eracom) ProtectHost White Mark II HSMs

When you use SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs) in your system, you can store channel PIN and MAC exchange keys (i.e., terminal master keys) in the TSS database or in the memory of the HSMs.

- To store these exchange keys in the TSS database, set the Key Length field on the PIN Information or MAC Information tab to a value of “Single” or “Double” and enter the encrypted key value.
- To store these exchange keys in the HSM, set the Key Length field to a value of “Key Index” and enter a five decimal index value, which is stored as a four-hexadecimal-character value.

Data encryption key exchange keys must always be stored in the HSM and entered as five decimal index values.

SafeNet (Eracom) ProtectHost White AMB HSMs

When you use SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs) in your system, the fields displayed on the Channel Security Configuration window for entering keys differ significantly from those shown and described in this section. For detailed information on configuring channel keys for SafeNet (Eracom) ProtectHost White AMB HSMs, refer to [section 4](#).

Banksys DEP HSMs

When you use Banksys Data Encryption Peripheral (DEP) HSMs in your system, the fields displayed on the Channel Security Configuration window for entering keys also differ significantly from those shown and described in this section. For detailed information on configuring channel keys for Banksys DEP HSMs, refer to [section 4](#).

Thales e-Security HSMs

When you use Thales e-Security HSMs in your system, the Key Type fields displayed on the MAC Information and Data Encryption Key Info tabs are not used. Also, the AS2805 Specific Keys tab is not used for TSS processing. These fields and tab are currently only used for BASE24-eps processing.

Future Keys

The “Future” key rows displayed on the Channel Security Configuration window are not applicable to BASE24 Transaction Security Services processing. These rows are only used for BASE24-eps processing. Only the “Current” key rows are used for BASE24 transaction Security Services processing.

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

BASE24-eps tabs

The AS2805 Specific Keys and German Info tabs are not applicable to BASE24 Transaction Security Services processing. These tabs are only used for BASE24-eps processing.

PIN Information tab

An example of the PIN Information tab on the Channel Security Configuration window is shown below.

The screenshot shows the 'Channel Security Configuration' window with the 'PIN Information' tab selected. The window has a title bar with standard Windows controls. Below the title bar is a toolbar with icons for file operations. The main area contains several configuration fields and two tables.

Configuration Fields:

- Channel ID: Enter
- External Key Block Format: Standard Atalla
- Old Key Timer Limit: 30
- Key Hierarchy: Two Levels
- Exchange Key Option: Separate Key exchange keys
- PIN Block Format: ANSI PIN (ISO Format 0)
- PIN Pad Character: F

Exchange Keys Section:

Exchange Keys Key Variant: 0 - Key Exchange Key Variant Key Length: Single

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

Inbound Keys Section:

Inbound Keys Key Length: Single

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

MAC Information tab

An example of the MAC Information tab on the Channel Security Configuration window is shown below.

The screenshot shows the 'Channel Security Configuration' window with the 'MAC Information' tab selected. The window has a title bar with standard Windows controls. Below the title bar, there are several configuration fields: 'Channel ID' with a text box containing 'Enter' and a help icon; 'External Key Block Format' set to 'Standard Atalla'; 'Old Key Timer Limit' set to '30'; 'Key Hierarchy' set to 'Two Levels'; and 'Exchange Key Option' set to 'Separate Key exchange keys'. Below these fields is a tabbed interface with four tabs: 'Data Encryption Key Info', 'Transaction Key Security', 'A52805 Specific Keys', and 'German Info'. The 'Transaction Key Security' tab is active, and within it, the 'MAC Information' sub-tab is selected. This sub-tab contains 'MAC Option' set to 'Single length DES Cipher Block Chaining' and 'MAC Length' set to '32 bits'. Below these are two sections: 'Exchange/Inbound' and 'Outbound'. The 'Exchange/Inbound' section has a sub-tab 'Exchange/Inbound' selected, showing 'Exchange Keys' with 'Export Variant' and 'Key Variant' both set to '0 - Key ...' and 'Key Length' set to 'Single'. It contains a table with columns 'Header', 'Key Part 1', 'Key Part 2', and 'Key Part 3', and rows 'Current' and 'Future'. The 'Outbound' section has a sub-tab 'Outbound' selected, showing 'Inbound Keys' with 'Key Length' set to 'Single'. It also contains a table with the same columns and rows as the 'Exchange/Inbound' section. Both tables are currently empty.

The optional Export Variant field on this tab specifies the variant under which a new device MAC key is generated when you are using Atalla security devices without AKB software. Otherwise, this field is not used. If the MAC key is to be downloaded under variant 0 of the KEK, the Transaction Security Services process uses Atalla command 11D to generate the new device MAC key. If the MAC key is to be downloaded under variant 5 of the KEK, the Transaction Security Services process uses Atalla command 10 (Generate Working Key, Any Type) and the specified export variant to generate the new device MAC key. Valid values are as follows:

- 0 = Key Exchange Key variant (default)
- 3 = MAC Encryption Key variant

Note: Because many devices use the same MAC key for both inbound and outbound message authentication processing, the Inbound Key is used for MAC generation if you leave the Outbound Key blank on the MAC Configuration tab of the Channel Security Configuration window. If the Outbound Key field is not blank, it is used for MAC generation. Thus, to use the same key for MAC verification and MAC generation, you must either configure the same key in both the Inbound Key and Outbound Key fields or leave the Outbound Key field blank.

BASE24-pos terminal data files

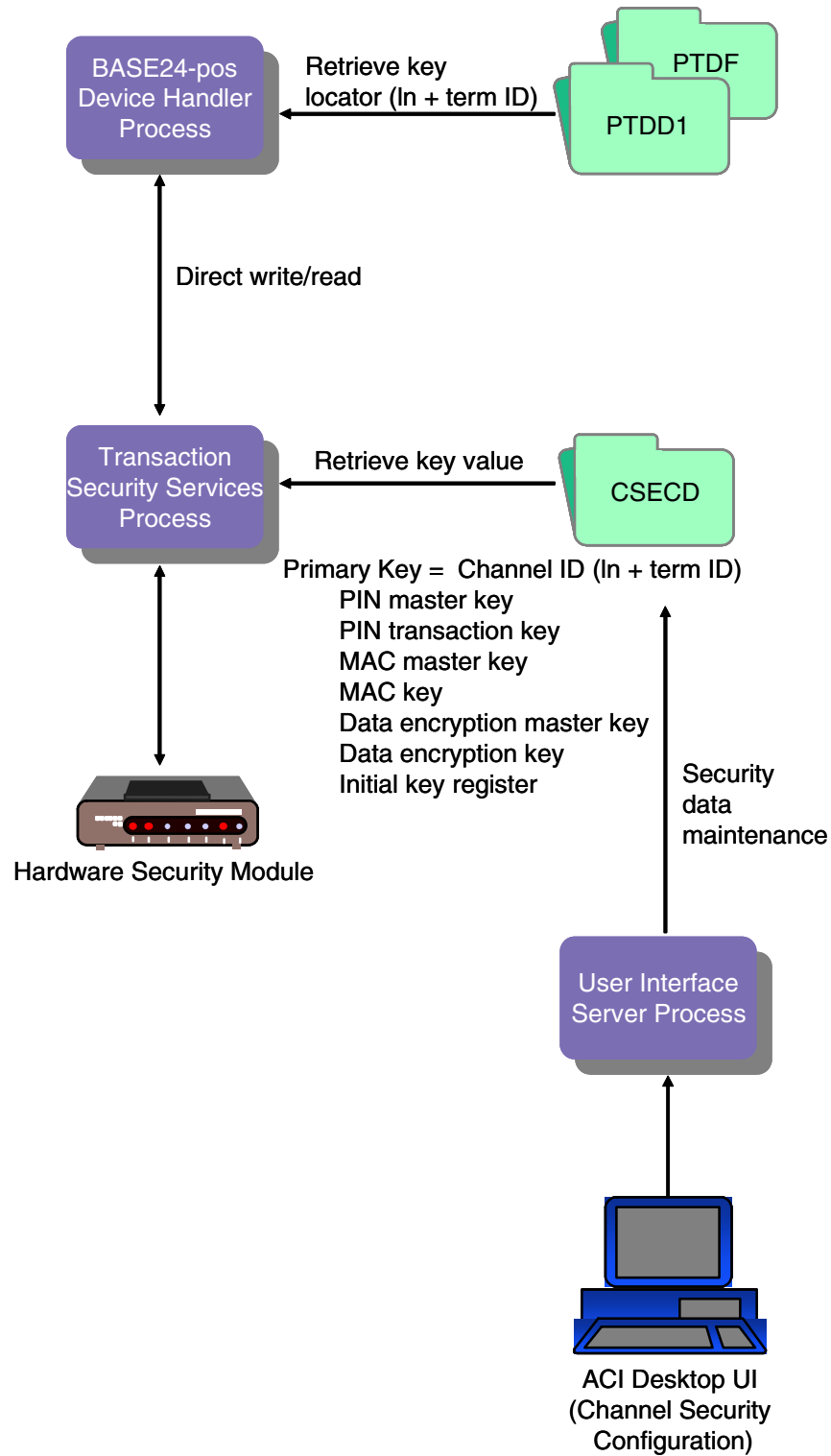
When the BASE24-pos Terminal Data files are converted, the PIN MASTER KEY and MAC MASTER KEY fields on PTD screen 7 for records that use hardware-encrypted keys when the Transaction Security Services process is implemented (i.e., the SECURE-DEV-TYP param is set to a value of TSS) are set to key locator values. In addition, these fields are not checked for all hexadecimal characters. If you change the key locator value in the PIN MASTER KEY or MAC MASTER KEY field of a converted record to a value other than the terminal ID, you must add a corresponding record for this key locator value on the Channel Security Configuration window.

Regardless of whether a record is converted, the encryption type fields (i.e., PIN ENCRYPTION TYPE and MAC TYPE fields on PTD screen 7) for a particular terminal's record must contain compatible values. You cannot mix software encryption and hardware encryption types in the same record. For example, if the PIN encryption type value specifies to use a security device, the MAC type value cannot specify that MAC support is in software, and vice versa.

The following diagram illustrates the relationship between BASE24-pos Terminal Data files and the Channel Security File (Channel_Security) after file conversion. The hardware encrypted key fields in the BASE24-pos Terminal Data files are set to the value of the two-character logical network identifier (specified by the LN-IND LCONF param if you are partitioning the Channel_Security) added to the beginning of the terminal ID, and the actual key values are stored in the Channel_Security. Data in the Channel_Security is maintained from the Channel Security Configuration window. The terminal ID value entered in the TERMINAL ID field on BASE24-pos Terminal Data files (PTD) screen 1 must match the value entered in the Channel ID field on the Channel Security Configuration window. If you add additional terminal records to the BASE24-pos Terminal Data files, you enter the encryption keys into corresponding Channel_Security records from the Channel Security Configuration window on the ACI desktop user interface. Refer to [section 3](#) for more information on the LN-IND param.

Note: If a POS device is configured in the BASE24-pos Terminal Data files to send PINs in the clear to any interchanges on the BASE24 system, a Channel_Security record must be manually added for the encrypted intermediate key with a PIN block type of PIN/PAD.

If an Interchange Interface process supports the conversion of incoming encrypted PINs to clear PINs internally, an ICHG_SECURITY record must be manually added for the encrypted intermediate key with a PIN block type of ANSI.



Channel Security Configuration window

The Channel Security Configuration window enables you to configure keys and parameters required by POS devices for PIN encryption, message authentication, and data encryption. The Data Encryption Exch Info and Data Encryption Key Info tabs are currently used for SPDH-compliant devices only. The Transaction Key Security tab is currently used for APACS 30/40 devices only. The AS2805 Specific Keys and German Info tabs on this window are not currently used for any POS devices. The PIN, MAC, and data encryption master keys are entered in the Exchange Key field on the appropriate tab. For POS devices, these keys can all be different. Samples of the PIN Information, MAC Information, Data Encryption Exch Info, and Data Encryption Key Info tabs of the Channel Security Configuration window for an SPDH-compliant device are shown below. Refer to the ACI Online Help provided with the ACI desktop for detailed descriptions of this window and its fields.

Note: You must configure a security device using the Security Device Configuration window before you can configure channel security information using the Channel Security Configuration window. The Data Encryption Exch Info and Data Encryption Key Info tabs are only enabled for Atalla NSPs and SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs). The Transaction Key Security tab is disabled for all types of security devices except for Thales e-Security HSMs.

Old key timer limit

The Old Key Timer Limit field located at the top of the Channel Security Configuration window specifies the maximum length of time, in seconds, that a PIN, MAC, or data encryption key can be used after a successful dynamic key exchange before it is automatically cleared.

This timer limits the amount of time that an old key can be used for PIN verification, message authentication, or data encryption after a successful key exchange has taken place.

SafeNet (Eracom) ProtectHost White Mark II HSMs

When you use SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs) in your system, you can store channel PIN and MAC exchange keys (i.e., terminal master keys) in the TSS database or in the memory of the HSMs.

- To store these exchange keys in the TSS database, set the Key Length field on the PIN Information or MAC Information tab to a value of “Single” or “Double” and enter the encrypted key value.
- To store these exchange keys in the HSM, set the Key Length field to a value of “Key Index” and enter a five decimal index value, which is stored as a four-hexadecimal-character value.

Data encryption key exchange keys must always be stored in the HSM and entered as five decimal index values.

SafeNet (Eracom) ProtectHost White AMB HSMs

When using SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs) in your system, the fields displayed on the Channel Security Configuration window for entering keys differ significantly from those shown and described in this section. For detailed information on configuring channel keys for SafeNet (Eracom) ProtectHost White AMB HSMs, refer to [section 4](#).

Banksys DEP HSMs

When you use Banksys Data Encryption Peripheral (DEP) HSMs in your system, the fields displayed on the Channel Security Configuration window for entering keys also differ significantly from those shown and described in this section. For detailed information on configuring channel keys for Banksys DEP HSMs, refer to [section 4](#).

Future Keys

The “Future” key rows displayed on the Channel Security Configuration window are not applicable to BASE24 Transaction Security Services processing. These rows are only used for BASE24-eps processing. Only the “Current” key rows are used for BASE24 transaction Security Services processing.

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

BASE24-eps tabs

The AS2805 Specific Keys and German Info tabs are not applicable to BASE24 Transaction Security Services processing. These tabs are only used for BASE24-eps processing.

PIN Information tab

See the “[PIN Information tab](#)” topic above for an example of this tab.

Note: If an SPDH-compliant terminal uses the derived unique key per transaction (DUKPT) method of PIN encryption, you must set the PIN Block Format field to a value of DUKPT and enter an intermediate PIN block encryption key in the Inbound Key table. When the PIN block in an incoming transaction is in DUKPT format, the SPDH module translates the block to be encrypted under the intermediate PIN block encryption key. This intermediate PIN block encryption key must be encrypted under variant 1 for an Atalla NSP and under key pair 06/07 for a Thales e-Security HSM.

MAC Information tab

See the “[MAC Information tab](#)” topic above for an example of this tab.

Note: If you are configuring message authentication for an SPDH-compliant device using the derived unique key per transaction (DUKPT) method of message authentication, you must set the MAC Option field to a value of “Old Style Visa DUKPT.”

Pseudo triple DES (ANSI X9.19) message authentication is specified by the “Pseudo 3DES Cipher Block Chaining” value in the MAC Option field. Check the firmware version requirements for support of this type of message authentication in Atalla, Banksys DEP, SafeNet (Eracom), and Thales e-Security hardware security devices.

The optional Export Variant field on this tab specifies the variant under which a new device MAC key is generated when you are using Atalla security devices without AKB software. Otherwise, this field is not used. If the MAC key is to be downloaded under variant 0 of the KEK, the Transaction Security Services process uses Atalla command 11D to generate the new device MAC key. If the MAC key is to be downloaded under variant 5 of the KEK, the Transaction Security Services process uses Atalla command 10 (Generate Working Key, Any Type) and the specified export variant to generate the new device MAC key. Valid values are as follows:

- 0 = Key Exchange Key variant (default)
- 3 = MAC Encryption Key variant

Note: Because many devices use the same MAC key for both inbound and outbound message authentication processing, the Inbound Key is used for MAC generation if you leave the Outbound Key blank on the MAC Configuration tab of the Channel Security Configuration window. If the Outbound Key field is not blank, it is used for MAC generation. Thus, to use the same key for MAC verification and MAC generation, you must either configure the same key in both the Inbound Key and Outbound Key fields or leave the Outbound Key field blank.

Data Encryption Key Info tab

An example of the Data Encryption Key Info tab on the Channel Security Configuration window is shown below for an Atalla NSP.

Note: The Data Type and Encryption Layers fields are disabled for SafeNet (Eracom) ProtectHost White Mark II HSMs.

The screenshot shows the 'Channel Security Configuration' window with the 'Data Encryption Key Info' tab selected. The window has a title bar with standard Windows controls. Below the title bar, there are several configuration fields:

- Channel ID:** A text box containing 'Enter'.
- External Key Block Format:** A dropdown menu set to 'Standard Atalla'.
- Old Key Timer Limit:** A numeric spinner box set to '30'.
- Key Hierarchy:** A dropdown menu set to 'Two Levels'.
- Exchange Key Option:** A dropdown menu set to 'Separate Key exchange keys'.

Below these fields is a tabbed interface with the following tabs: 'Export Master Info', 'PIN Information', 'MAC Information', 'Data Encryption Exch Info', 'Data Encryption Key Info' (selected), 'Transaction Key Security', 'A52805 Specific Keys', and 'German Info'.

Under the 'Data Encryption Key Info' tab, there are three more fields:

- DES Mode:** A dropdown menu set to 'ECB'.
- Data Type:** A dropdown menu set to 'Unpacked ASCII...'.
- Encryption Layers:** A numeric spinner box set to '1'.

Below these fields are two tabs: 'Inbound' (selected) and 'Outbound'.

Under the 'Inbound' tab, there is a section titled 'Inbound Keys' with a 'Key Length' dropdown set to 'Single'. Below this is a table with the following structure:

	Header	Key Part 1	Key Part 2	Key Part 3
Current 1				
Current 2				
Future 1				
Future 2				

At the bottom of the 'Inbound Keys' section, there is a horizontal scrollbar.

Data Encryption Exch Info tab

An example of the Data Encryption Exch Info tab on the Channel Security Configuration window is shown below for an Atalla NSP.

Note: The Key Variant field is disabled for SafeNet (Eracom) ProtectHost White Mark II HSMS.

The screenshot shows the 'Channel Security Configuration' window with the 'Data Encryption Exch Info' tab selected. The window has a title bar with standard Windows controls. Below the title bar is a toolbar with icons for file operations. The main area contains several configuration fields and two tables.

Configuration Fields:

- Channel ID: Enter
- External Key Block Format: Standard Atalla
- Old Key Timer Limit: 30
- Key Hierarchy: Two Levels
- Exchange Key Option: Separate Key exchange keys

Tabbed Interface:

- Top tabs: Data Encryption Key Info, Transaction Key Security, AS2805 Specific Keys, German Info
- Bottom tabs: Export Master Info, PIN Information, MAC Information, Data Encryption Exch Info (selected)

Exchange Keys Section:

Exchange Keys Key Variant: 0 - Key Exchange Key Vari... Key Length: Single

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

Initialization Vector Section:

	Header	Key Part 1	Key Part 2	Key Part 3
1				

Transaction Key Security tab

An example of the Transaction Key Security tab on the Channel Security Configuration window is shown below. Currently, this tab is only used for an APACS 30/40 device.

This allows you to enter initial key register values that are provided by the device manufacturer and are hardcoded into the terminal for each dataset maintained. This value is used in the computation of a transaction key between the terminal and the BASE24 system. For more information on how this value is used in processing, refer to the **BASE24-pos APACS 30/40 Device Handler User's Guide**.

The screenshot shows the 'Channel Security Configuration' window with the 'Transaction Key Security' tab selected. The window contains several input fields and tabs. The 'Initial Key Register' section is expanded, showing a table with three columns: 'Key Part 1', 'Key Part 2', and 'Key Part 3'. The first row of the table has a '1' in the first column, indicating the key register number.

	Key Part 1	Key Part 2	Key Part 3
1			

Maintaining channel security data

The following paragraphs describe procedures for adding, modifying, and deleting channel ID records in the Channel Security File (Channel_Security).

Adding a new channel ID record

Use the following procedure when adding a new ATM terminal or POS device to your BASE24 system.

1. Log on to the BASE24 user access system and add the new terminal record on the BASE24-atm Terminal Data file (ATD) or BASE24-pos Terminal Data file (PTD) screens. Set the PIN encryption type and the MAC type for the record to perform both PIN encryption and message authentication in a security device (i.e., in hardware rather than software).

When you add an ATD record, the MASTER KEY field on device-specific ATD screen 9 is automatically set to the terminal ID, which is used as the key locator value (channel ID) for Channel_Security records.

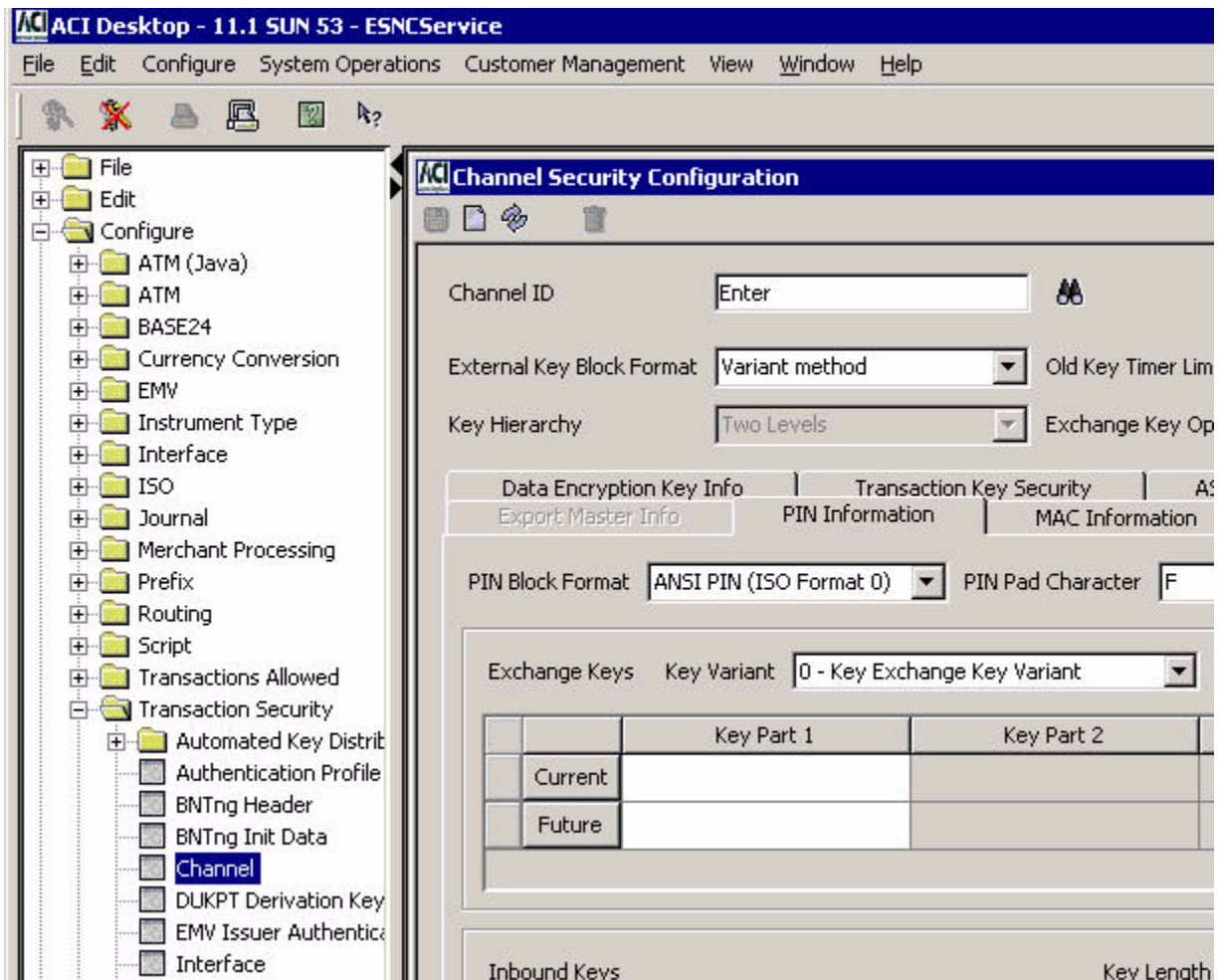
When you add a PTD record, the PIN MASTER KEY and MAC MASTER KEY fields on PTD screen 7 are automatically set to the terminal ID, which is used as the key locator value (channel ID) for Channel_Security records.

In addition, if the LN-IND LCONF param is set, the value of this param is prepended to the terminal ID key locator value set in the hardware key field. For example, if the LN-IND is set to US and the terminal ID is set to S1AATM04, the resulting key locator value will be USS1AATM04.

Note: The above processing is only true if the SECURE-DEV-TYP LCONF assign is set to TSS and the PIN encryption or MAC type field associated with the key field (i.e., PIN ENCRYPT TYP field on device-specific ATD screen 9, PIN ENCRYPTION TYPE and MAC TYPE fields on device-specific PTD screen 7) are also set to a value indicating encryption or message authentication in a security device. If the SECURE-DEV-TYP LCONF assign is not set to TSS and these fields are set to indicate no encryption, no message authentication, or encryption or message authentication in software, the associated key field is not automatically set to the terminal ID when you add a new record.

2. Log on to the ACI desktop user interface. Refer to the ***ACI Desktop User Interface Manual*** for instructions on logging on.
3. Display the Channel Security Configuration window by accessing **Configure > Transaction Security > Channel**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.

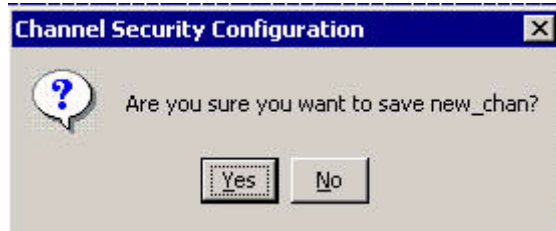


4. Type the terminal ID from step 1 in the Channel ID field. If you press the **Enter** key at this point, a message similar to the following is displayed at the bottom of the window. This message indicates that the record does not exist.

0 - 09:13 AM > Unable to retrieve NewChanRec. Channel not found. Enter data and Save to insert.

5. Enter or change data in the appropriate fields for your security requirements. Refer to the "Configure Security Information for a Channel" procedure provided in the ACI Online Help for the Channel Security Configuration window for details.

6. Click Save () and answer Yes to the “Are you sure you want to save” dialog box.



7. The following message is displayed at the bottom of the window to indicate that the channel ID record was successfully saved.



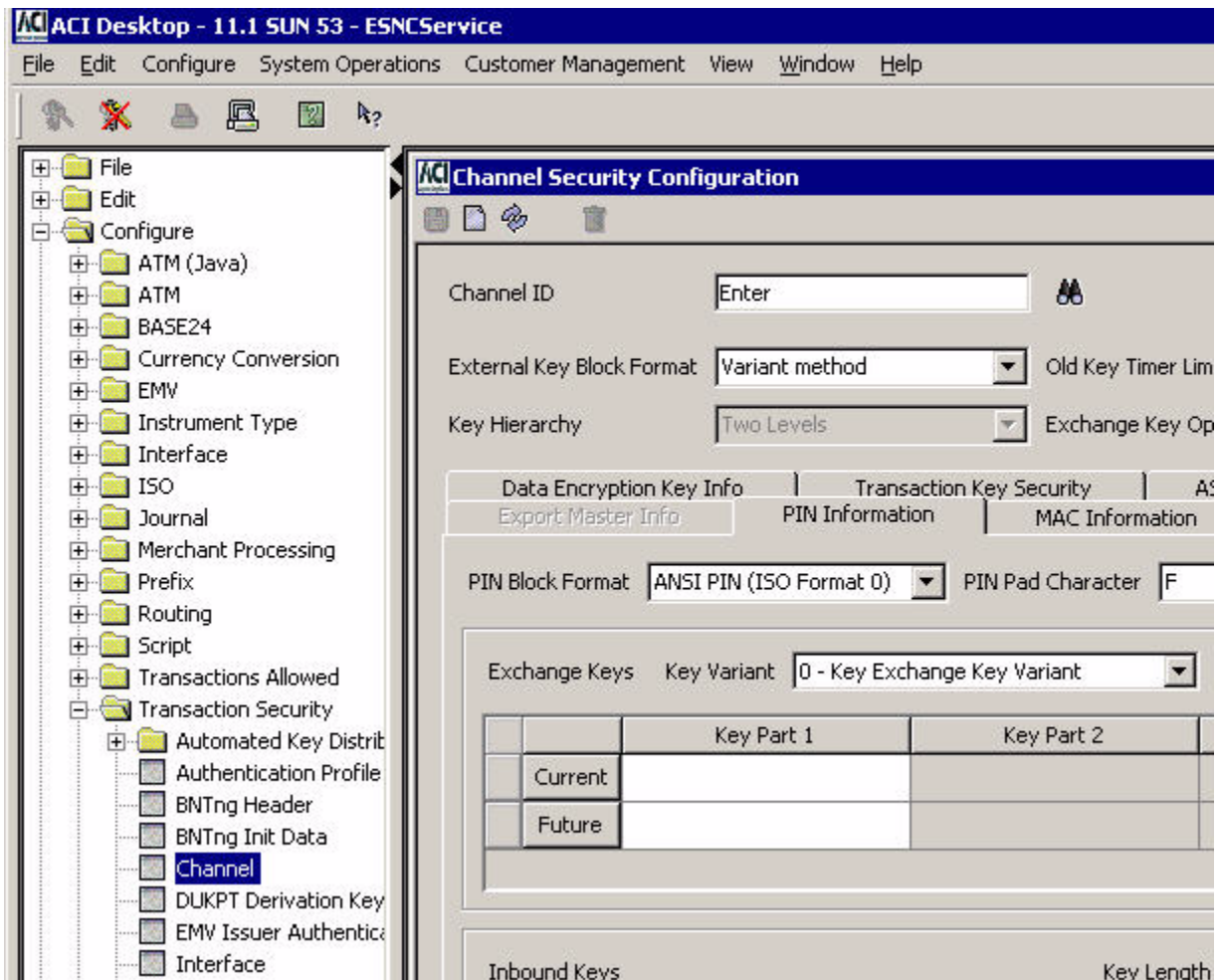
Note: Because Channel_Security records are read dynamically from disk into an internal memory table as they are needed in online transaction processing, you do not need to enter an Alter Data Source command when adding a new Channel_Security record. You may, however, want to enter a CSEC REFRESH process control command to replace the memory copy with the newly updated record from disk.

Modifying an existing channel ID record

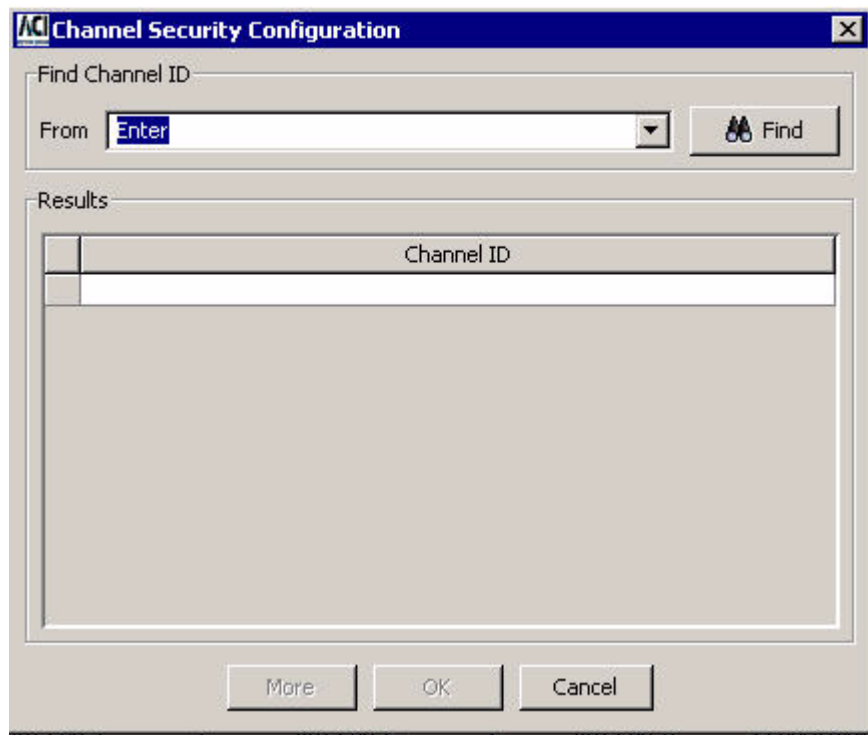
Use the following procedure when modifying an existing channel ID record for an ATM terminal or POS device in your BASE24 system.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Channel Security Configuration window by accessing **Configure > Transaction Security > Channel**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

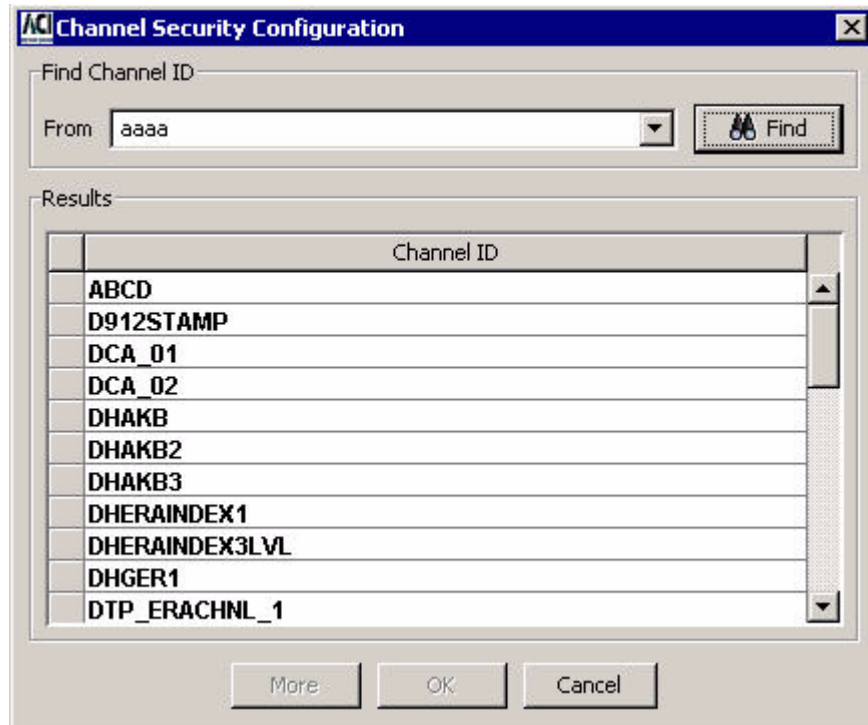
Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Type the channel ID in the Channel ID field and press the **Enter** key. If you do not know the channel ID value, click Find (). If you click Find, the following dialog box is displayed.



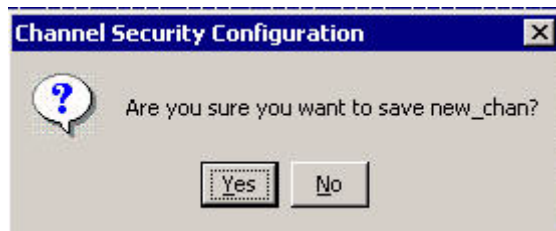
To find the channel ID, enter a partial value less than what you think the channel ID value might be in the From field and click the Find button ( Find). All configured channel IDs equal to or greater than the partial value you entered are displayed in the Results area. For searching, characters are ordered from 0–9 and A–Z, where 0 is the lowest and Z is the highest.



Highlight the record you want to retrieve and click OK. The record is retrieved and displayed on the Channel Security Configuration window. A message similar to the following is displayed at the bottom of the window. This message indicates that the record was retrieved.

0 - 01:08 PM > Channel Security Information Retrieved

4. Change data as needed in the appropriate fields for your security requirements. Refer to the “Configure Security Information for a Channel” procedure provided in ACI Online Help for the Channel Security Configuration window for details.
5. Click Save () and answer Yes to the “Are you sure you want to save new_chan?” dialog box.



6. The following message is displayed at the bottom of the window to indicate that the channel ID record was successfully saved.



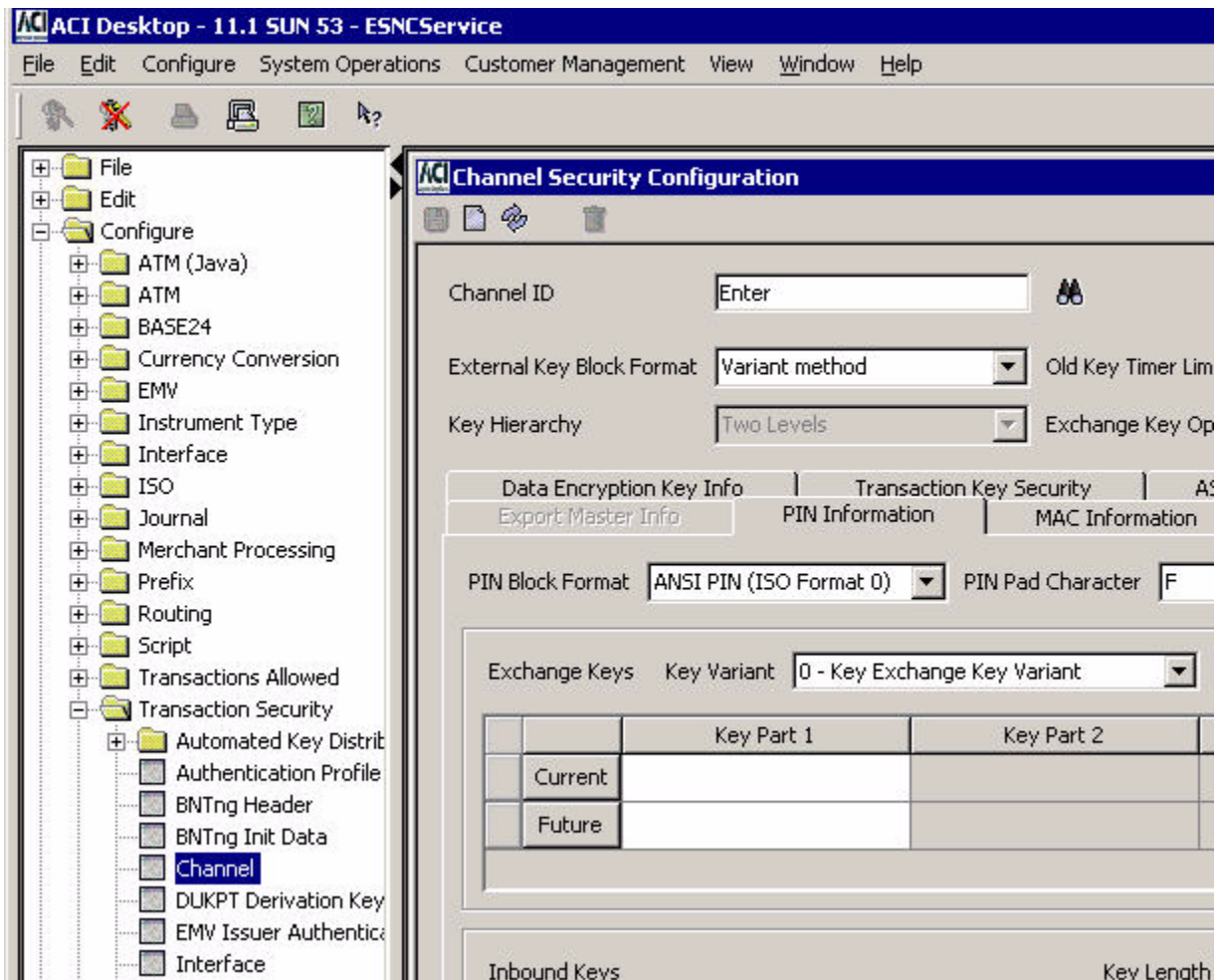
7. Warmboot the Transaction Security Services process to begin using the changed data in processing. Refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section.
8. Using the Device Control Terminal (DCT) or an XPNET network control facility, download the updated key information to the ATM terminal or POS device.

Deleting a channel ID record

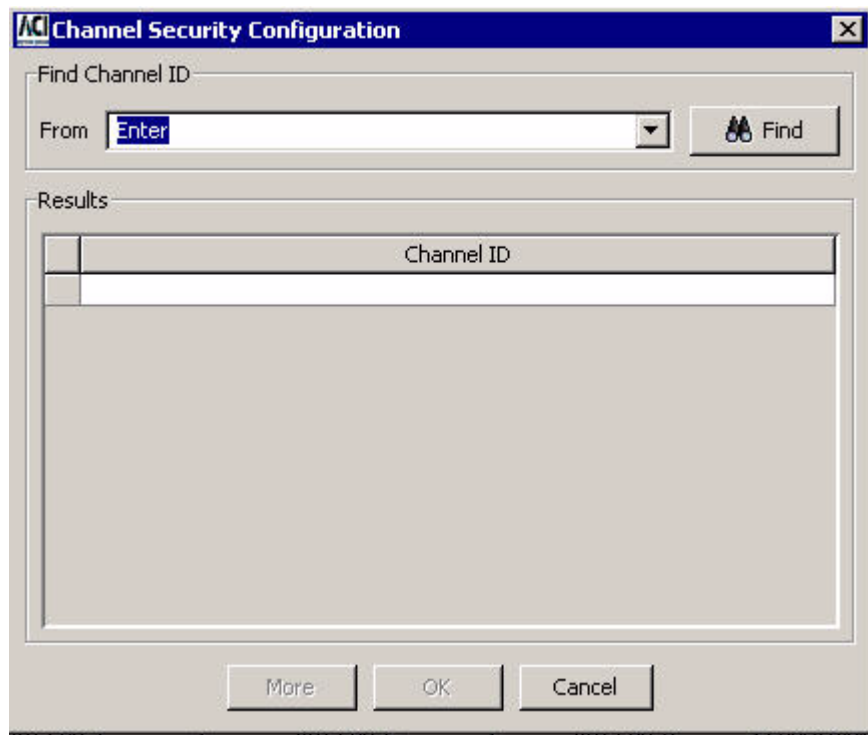
Use the following procedure when deleting a channel ID record for an ATM terminal or POS device in your BASE24 system.

1. Log on to the ACI desktop user interface. Refer to the ***ACI Desktop User Interface Manual*** for instructions on logging on.
2. Display the Channel Security Configuration window by accessing **Configure > Transaction Security > Channel**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

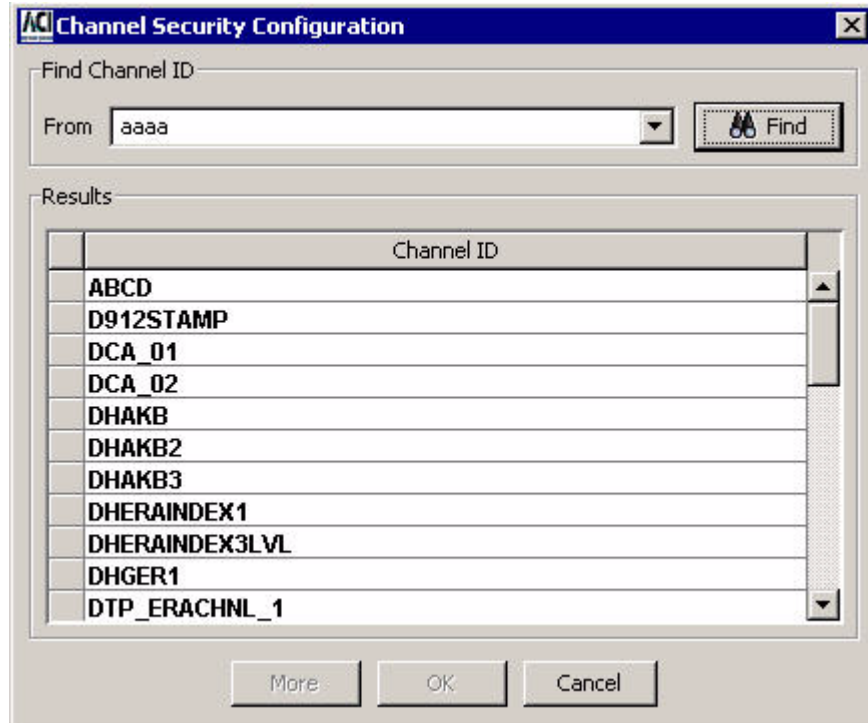
Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Type the channel ID in the Channel ID field and press the **Enter** key. If you do not know the channel ID value, click Find (). If you click Find, the following dialog box is displayed.



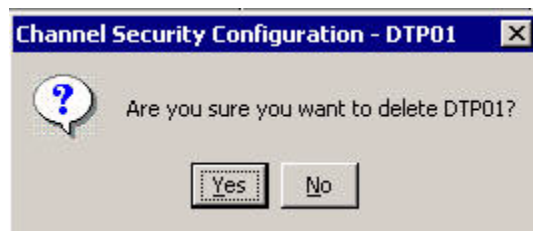
To find the channel ID, enter a partial value less than what you think the channel ID value might be in the From field and click the Find button ( Find). All configured channel IDs equal to or greater than the partial value you entered are displayed in the Results area. For searching, characters are ordered from 0–9 and A–Z, where 0 is the lowest and Z is the highest.



Highlight the record you want to delete and click OK. The record is retrieved and displayed on the Channel Security Configuration window. A message similar to the following is displayed at the bottom of the window. This message indicates that the record was retrieved.

0 - 01:08 PM > Channel Security Information Retrieved

4. Click Delete () and answer Yes to the “Are you sure you want to delete” dialog box.



5. The following message is displayed at the bottom of the window to indicate that the channel ID record was successfully deleted.

1 - 02:00 PM > Channel Security Information Deleted

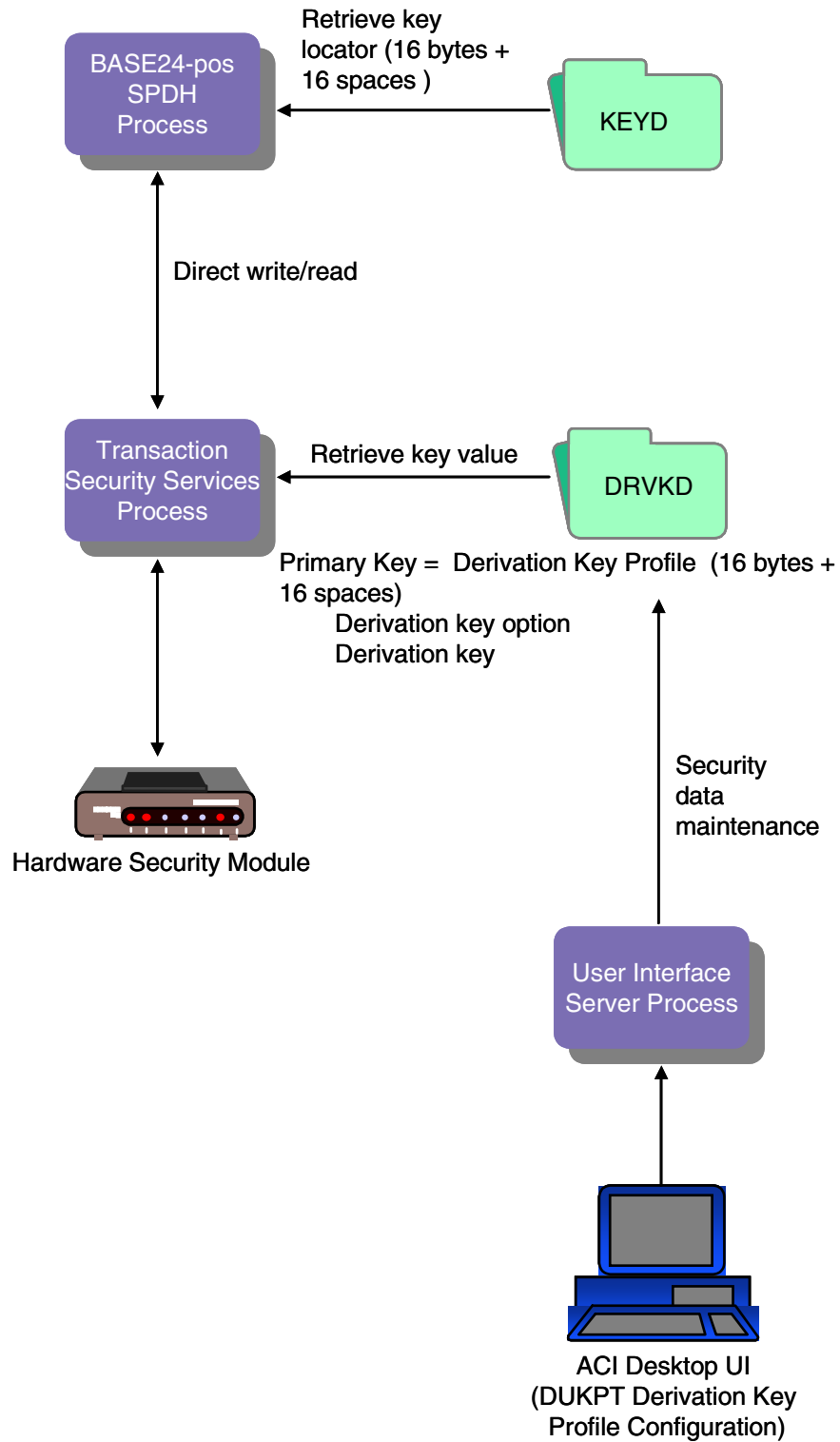
6. If you are deleting the terminal data completely from the BASE24 system, log on to the BASE24 user access system and delete the ATD or PTD record for the terminal associated with the channel ID record.

Derivation Key file (KEYD)

When the Derivation Key File (KEYD) is converted, the DERIVATION KEY field on KEYD screen 1 is set to a key locator value. This key locator value consists of the first 16 bytes of the original derivation key value appended with 16 bytes of spaces. This key locator value is used as the derivation key profile, which is the primary key to the Derivation Key File (Derivation_Key) used by the Transaction Security Services process.

If you change the key locator value in the DERIVATION KEY field of a converted record to a value other than the first 16 bytes of the original derivation key value appended with 16 bytes of spaces, you must add a corresponding record for this key locator value on the DUKPT Derivation Key Profile Configuration window.

The following diagram illustrates the relationship between the Derivation Key File (KEYD) and the Derivation Key File (Derivation_Key) after file conversion. Data in the Derivation_Key is maintained from the DUKPT Derivation Key Profile Configuration window. The key locator value entered in the DERIVATION KEY field on KEYD screen 1 must match the value entered in the Derivation Key Profile field on the DUKPT Derivation Key Profile Configuration window. If you add additional records to the KEYD after it is converted, you enter the derivation keys into corresponding Derivation_Key records from the DUKPT Derivation Key Profile Configuration window on the ACI desktop user interface.



DUKPT Derivation Key Profile Configuration window

The DUKPT Derivation Key Profile Configuration window (access **Configure > Transaction Security > DUKPT Derivation Key Profile**) enables you to configure derivation keys and parameters used by the BASE24-pos Standard POS Device Handler (SPDH) module and the BASE24-pos Hypercom Device Handler (HPDH) module for derived unique key per transaction (DUKPT) PIN block translation and DUKPT message authentication (SPDH module only).

The DUKPT Derivation Key Profile Configuration window maintains data in the Derivation Key File (Derivation_Key). The Derivation Key Profile specifies the primary key to Derivation_Key records and must match a key locator value in the KEYD. The Derivation Key Option field specifies whether the security device is to use the single key derivation algorithm or the double key derivation algorithm with this derivation key. This option does not specify the length of the derivation key. The length of the derivation key is specified in the Key Length field. For the single key derivation algorithm, the derivation key can be a single or double length key. For the double key derivation algorithm, the derivation key must be a double length key. The Derivation Key table contains the encrypted derivation key. Refer to the ACI Online Help provided with the ACI desktop for detailed descriptions of this window and its fields.

Note: You must configure a security device using the Security Device Configuration window before you can configure derivation key information using the DUKPT Derivation Key Profile Configuration window.

An example of the DUKPT Derivation Key Profile Configuration window is shown below.

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				

Key file (KEYF)

The following diagram illustrates the relationship between the BASE24 Key File (KEYF) and the Interface Security File (ICHG_SECURITY) after file conversion. The hardware encrypted key fields in the KEYF are set to the value of the two-character logical network identifier (specified by the LN-IND LCONF param if you are partitioning the ICHG_SECURITY) added to the beginning of an index value generated by the conversion program, and the actual key values are stored in the ICHG_SECURITY. The only exception is the hardware encrypted intermediate key (refer to the note below). Refer to [section 3](#) for more information on the LN-IND param.

Data in the ICHG_SECURITY is maintained from the Interface Security Configuration window. Also after file conversion, the remaining data in the KEYF, such as the PIN and MAC encrypt types, PIN block format, all dynamic key information, and IMNI interface-specific information, is maintained from the BASE24 KEYF Configuration window on the ACI desktop user interface. This allows all security data to be maintained from a single user interface. The Key Locator field on the BASE24 KEYF Configuration window is used to access records in the ICHG_SECURITY on the Interface Security Configuration window. The value in the Interface Name field on the Interface Security Configuration window must match the value in the Key Locator field on the BASE24 KEYF Configuration window. You can have multiple records in the KEYF with the same key locator value as long as the external interface or interchange treats all the links to the BASE24 system as a single logical connection and shares the same keys among all the links.

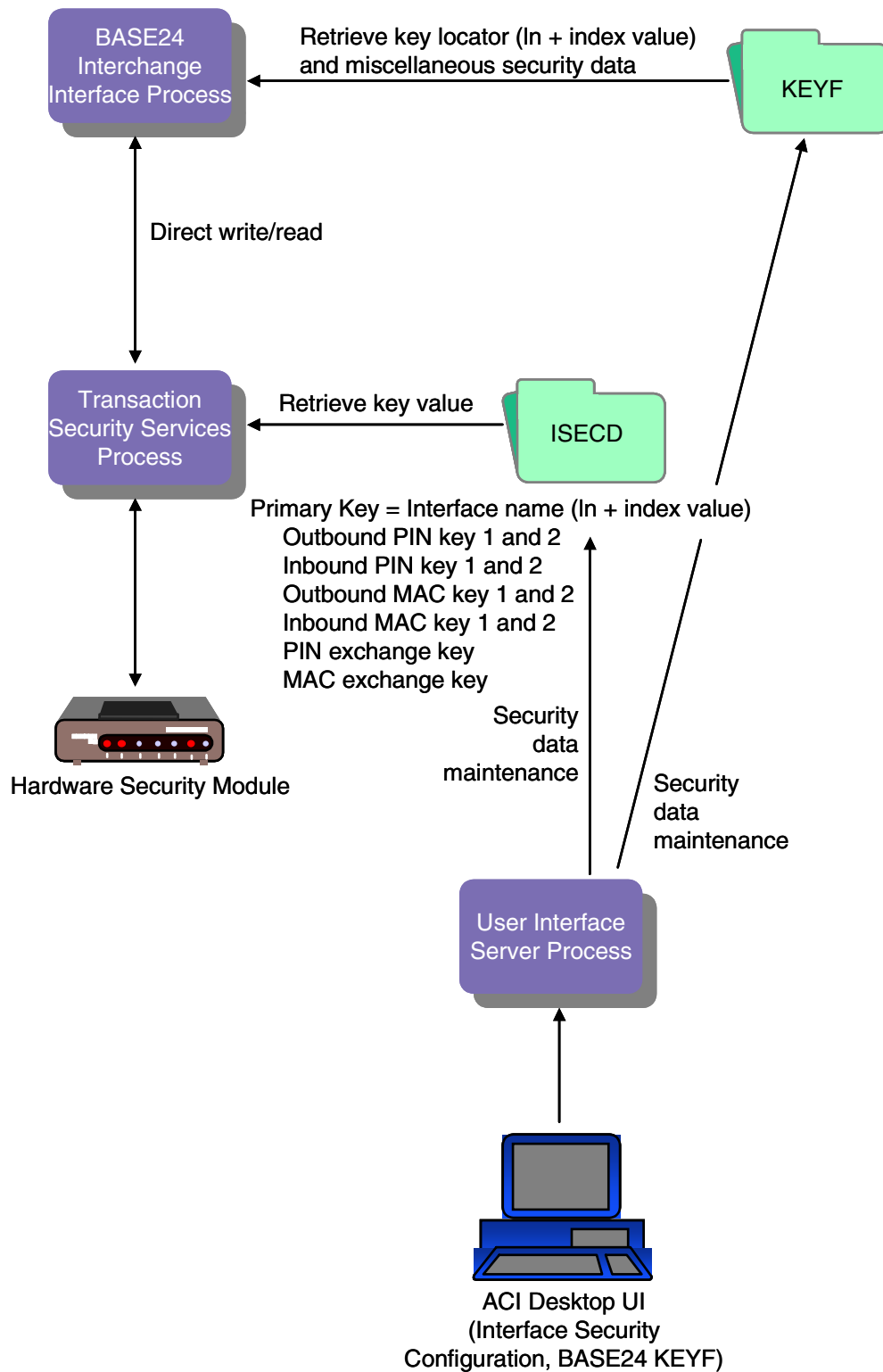
If you need to add KEYF and ICHG_SECURITY records after conversion, you must add a new KEYF record from the BASE24 KEYF Configuration window before you can add the new ICHG_SECURITY record from the Interface Security Configuration window.

Note: If an interchange encrypts PINs in hardware and the BASE24 system processes PINs internally in the clear, or if an interchange uses clear PINs while the BASE24 system uses PINs encrypted in software, an intermediate key must be configured in the KEYF using the Pathway-based Key File (KEYF) screens. The ACI desktop user interface does not grant access to intermediate keys.

If an acquiring host or Interchange Interface process is configured to send PINs in the clear to any interchange in the BASE24 system that supports encrypted PINs, an ICHG_SECURITY record must be manually added for the encrypted intermediate key with a PIN block type of PIN/PAD.

If an acquiring host or Interchange Interface process encrypts PINs and an interchange interface supports clear PINs, an ICHG_SECURITY record must be manually added for the encrypted intermediate key with a PIN block type of ANSI.

If you are using the MULT-LN-PIN-KEY LCONF param, an ICHG_SECURITY record must be manually added for the common PIN encryption key. Refer to [section 3](#) for more information on the MULT-LN-PIN-KEY param.



BASE24 KEYF Configuration window

The BASE24 KEYF Configuration window on the ACI desktop user interface enables you to maintain PIN and MAC encrypt types, PIN block format, and all dynamic key information stored in the BASE24 Key File (KEYF). Fields are displayed on six tabs on this window: PIN Information, MAC Information, Message Information, Dynamic Key Limits, Dynamic Key Management, and Interac Key Management. Fields displayed on the Message Information tab are specific to the ISO Host Interface process. Fields displayed on the Interac Key Management tab are specific to IMNI interfaces. Samples for each one of these tabs on the BASE24 KEYF Configuration window are shown below. Refer to the ACI Online Help provided with the ACI desktop for detailed descriptions of this window and its fields.

If your TSS environment is implemented into a BASE24 system with multiple logical networks, you can select the logical network for the KEYF you want to access by entering the logical network identifier in the Logical Net field. A BASE24_KEYF assign must be defined in the Metadata Configuration File (CONFCSV) for each logical network in your BASE24 system. The format of these assigns is as follows:

`"BASE24_KEYF $\textit{logical-net}$ "`

where:

logical-net is 1 to 4 alphanumeric characters identifying a logical network. For consistency, ACI recommends that this logical network identifier match the value of the LOGICAL-NET param in the Logical Network Configuration File (LCONF) for each logical network.

In addition, the CONFCSV must contain one BASE24_KEYF assign without the logical network identifier appended to it. This assign defines the default KEYF accessed if you do not specify a logical network in the Logical Net field.

Sample entries in the CONFCSV for the KEYF are shown below. In these entries, the default KEYF accessed is for the PRO1 logical network. Users could access the KEYF for the PRO1, PRO2, and PRO3 logical networks in the Logical Net field.

```
"BASE24_KEYF", "BASE24_KEYF", "1.0", "$DATA.PRO1DATA.KEYF", "Enscribe", "", ""
"BASE24_KEYFPRO1", "BASE24_KEYF", "1.0", "$DATA.PRO1DATA.KEYF", "Enscribe", "", ""
"BASE24_KEYFPRO2", "BASE24_KEYF", "1.0", "$DATA.PRO2DATA.KEYF", "Enscribe", "", ""
"BASE24_KEYFPRO3", "BASE24_KEYF", "1.0", "$DATA1.PRO3DATA.KEYF", "Enscribe", "", ""
```

If you attempt to access a KEYF that is not properly defined in the CONFCSV, a data not found error is displayed.

Note: Any time you change the CONFCSV, you must stop and restart the process that uses the changed data. To ensure that no transactions currently in progress are lost, you should stop the process in an orderly fashion. The change does not take effect until after the process is stopped and restarted.

PIN Information tab

An example of the PIN Information tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'PIN Information' tab selected. The window has a title bar with standard Windows icons. Below the title bar is a 'Keys Section' with fields for 'Logical Net', 'DPC/FIID' (a dropdown menu showing 'Select/Enter'), 'Interface Process Name' (a dropdown menu showing 'Select/Enter'), and 'Key Locator' (a dropdown menu showing 'SELECT/ENTER'). Below this section is a tabbed interface with tabs for 'PIN Information', 'MAC Information', 'Message Information', 'Dynamic Key Limits', 'Dynamic Key Management', and 'Interac Key Management'. The 'PIN Information' tab is active, displaying several configuration options: 'Encryption Type' (Security Module), 'BASE24 Encryption Type' (Security Module), 'PIN Block Format' (ANSI PIN (ISO Format 0)), 'ANSI PAN Format' (12 Right, no Check Digit), 'Keys Supported' (Combined Keys), 'Notarization' (Not supported), 'Key Length' (Single), and 'PIN Pad Character' (F).

Keys Section			
Logical Net		DPC/FIID	Select/Enter
Interface Process Name	Select/Enter		
Key Locator	SELECT/ENTER		

PIN Information			
Encryption Type	Security Module	Keys Supported	Combined Keys
BASE24 Encryption Type	Security Module	Notarization	Not supported
PIN Block Format	ANSI PIN (ISO Format 0)	Key Length	Single
ANSI PAN Format	12 Right, no Check Digit	PIN Pad Character	F

MAC Information tab

An example of the MAC Information tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'MAC Information' tab selected. The window has a title bar with standard Windows icons. Below the title bar is a 'Keys Section' with fields for 'Logical Net', 'DPC/FIID' (a dropdown menu showing 'Select/Enter'), 'Interface Process Name' (a dropdown menu showing 'Select/Enter'), and 'Key Locator' (a dropdown menu showing 'SELECT/ENTER'). Below this section is a tabbed interface with tabs for 'PIN Information', 'MAC Information', 'Message Information', 'Dynamic Key Limits', 'Dynamic Key Management', and 'Interac Key Management'. The 'MAC Information' tab is active, displaying several configuration options: 'MAC Encryption Type' (No MAC Processing), 'Fields included in MAC' (Selected Fields), and 'MAC Data Character Set' (ASCII).

Keys Section	
Logical Net	DPC/FIID Select/Enter
Interface Process Name	Select/Enter
Key Locator	SELECT/ENTER

MAC Information	
MAC Encryption Type	No MAC Processing
Fields included in MAC	Selected Fields
MAC Data Character Set	ASCII

Message Information tab

An example of the Message Information tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'Message Information' tab selected. The window has a title bar with standard Windows controls. Below the title bar is a toolbar with icons for help, save, undo, redo, and delete. The main content area is divided into two sections. The top section, titled 'Keys Section', contains three fields: 'Logical Net' (a text box), 'DPC/FIID' (a dropdown menu with 'Select/Enter' selected), and 'Interface Process Name' (a dropdown menu with 'Select/Enter' selected). Below these is a 'Key Locator' label and a dropdown menu with 'SELECT/ENTER' selected. The bottom section contains a tabbed interface with six tabs: 'PIN Information', 'MAC Information', 'Message Information' (which is active), 'Dynamic Key Limits', 'Dynamic Key Management', and 'Interac Key Management'. Below the tabs are two fields: 'Full Message Encryption' (a dropdown menu with 'No Encryption' selected) and 'Message Encryption Type' (a dropdown menu with 'No Encryption' selected).

Keys Section	
Logical Net	
DPC/FIID	<select><option>Select/Enter</option></select>
Interface Process Name	<select><option>Select/Enter</option></select>
Key Locator	<select><option>SELECT/ENTER</option></select>

Message Information	
Full Message Encryption	<select><option>No Encryption</option></select>
Message Encryption Type	<select><option>No Encryption</option></select>

Dynamic Key Limits tab

An example of the Dynamic Key Limits tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'Dynamic Key Limits' tab selected. The window has a title bar with standard Windows icons. Below the title bar is a toolbar with icons for file operations. The main content area is divided into sections. The 'Keys Section' at the top contains fields for 'Logical Net', 'DPC/FIID' (a dropdown menu), 'Interface Process Name' (a dropdown menu), and 'Key Locator' (a dropdown menu). Below this is a tabbed interface with six tabs: 'PIN Information', 'MAC Information', 'Message Information', 'Dynamic Key Limits' (which is active), 'Dynamic Key Management', and 'Interact Key Management'. The 'Dynamic Key Limits' tab is divided into three sections: 'PIN Key', 'MAC Key', and 'Full Message Encryption'. Each section contains a table of limits. The 'PIN Key' section has 'Transaction Limit', 'Consecutive Error Limit', 'Timer Interval' (set to 'Minutes'), 'Translation Error Limit', and 'Timer Limit'. The 'MAC Key' section has 'Transaction Limit', 'Consecutive Error Limit', 'Timer Interval' (set to 'Minutes'), 'Transaction Error Limit', 'Timer Limit', and 'KMAC Sync Error Limit'. The 'Full Message Encryption' section has 'Transaction Limit', 'Consecutive Error Limit', 'Timer Interval' (set to 'Minutes'), 'Translation Error Limit', and 'Timer Limit'. All limit fields are set to '0'.

Section	Limit Type	Value
PIN Key	Transaction Limit	0
	Consecutive Error Limit	0
	Timer Interval	Minutes
	Translation Error Limit	0
	Timer Limit	0
MAC Key	Transaction Limit	0
	Consecutive Error Limit	0
	Timer Interval	Minutes
	Transaction Error Limit	0
	Timer Limit	0
	KMAC Sync Error Limit	0
Full Message Encryption	Transaction Limit	0
	Consecutive Error Limit	0
	Timer Interval	Minutes
	Translation Error Limit	0
	Timer Limit	0

Dynamic Key Management tab

An example of the Dynamic Key Management tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'Dynamic Key Management' tab selected. The window has a title bar with standard OS icons. Below the title bar is a toolbar with icons for file operations. The main content area is divided into two sections. The top section, titled 'Keys Section', contains three fields: 'Logical Net' (a text box), 'DPC/FIID' (a dropdown menu with 'Select/Enter' selected), and 'Interface Process Name' (a dropdown menu with 'Select/Enter' selected). Below these is a 'Key Locator' field (a dropdown menu with 'SELECT/ENTER' selected). The bottom section contains a tabbed interface with five tabs: 'PIN Information', 'MAC Information', 'Message Information', 'Dynamic Key Limits', and 'Dynamic Key Management' (which is the active tab). Below the tabs are three fields: 'Exchange Receiver' (a text box), 'Exchange Originator' (a text box), and 'Management Type' (a dropdown menu with 'NONE' selected).

Keys Section	
Logical Net	
DPC/FIID	Select/Enter
Interface Process Name	Select/Enter
Key Locator	SELECT/ENTER

Dynamic Key Management	
Exchange Receiver	
Exchange Originator	
Management Type	NONE

Interac Key Management tab

The Interac Key Management tab displays exchange key sequence number fields for inbound and outbound keys, fields indicating the index (i.e., 1 or 2) of the inbound and outbound PIN, MAC, and data encryption keys to use for the interface, and a field for entering a new key exchange key. All of the fields on this tab are specific to the Inter Member Network Interchange (IMNI) Interface. Financial institution members of the Interac association in Canada use this interface to route transactions to and from other Interac association members. This tab is enabled only if the Security Device Type field on the Security Device Configuration window is set to Atalla and only applies to Interac association members.

Exchange key sequence numbers

Two exchange key sequence number counters are maintained in the KEYF—one for the inbound (import) key exchange keys and one for the outbound (export) key exchange keys. These counters are displayed in the Inbound KEK Counter and Outbound KEK Counter fields, respectively, on the IMNI tab. When the IMNI interface exchanges an inbound or outbound working key (i.e., PIN, MAC, or data) the associated counter value is increased by one. When an inbound working key is exchanged, the inbound key exchange key sequence number counter is increased by one. When an outbound working key is exchanged, the outbound key exchange key sequence number counter is increased by one. The valid values for each counter are 0–999999999999999. You can manually reset the values in either field by entering a new value in the field and pressing the Save icon ().

Key indexes

This tab displays an index field for each inbound and outbound working key stored in the KEYF. These fields indicate which of the two keys (1 or 2) stored for each inbound or outbound PIN, MAC, or data encryption key is currently in use for the interface. You can manually change the index value for a key by selecting one of the selectable list values in the associated key index field and pressing the Save icon ().

Transaction acquirers pass the index of a PIN, MAC, or data encryption key to be used in a transaction request instead of repeating an attempt with more than one key. BASE24 stores the specified indexes to be used for a transaction in the TSS Index token (BC) and passes this information in calls to the Transaction Security Services process.

Key exchange key

From this tab you can also change the current key exchange keys in the Interface Security File (ICHG_SECURITY) by entering a new key exchange key value in the New Key Exchange Key field and pressing the Save icon (). If the new key exchange key is formatted correctly, all of the inbound and outbound key exchange keys for the interface in the ICHG_SECURITY (i.e., PIN key exchange keys, MAC key exchange keys, and data encryption key exchange keys) are changed to the specified key value. When this occurs, both the inbound and outbound key counters in the KEYF are reset to 0.

An example of the Interac Key Management tab on the BASE24 KEYF Configuration window is shown below.

The screenshot shows the 'BASE24 KEYF Configuration' window with the 'Interac Key Management' tab selected. The window has a title bar with standard icons and a menu bar. Below the menu bar is a 'Keys Section' with fields for 'Logical Net', 'DPC/FIID' (a dropdown menu), 'Interface Process Name' (a dropdown menu), and 'Key Locator' (a dropdown menu). Below this is a tabbed interface with tabs for 'PIN Information', 'MAC Information', 'Message Information', 'Dynamic Key Limits', 'Dynamic Key Management', and 'Interac Key Management'. The 'Interac Key Management' tab is active and contains a note: 'Note: This tab is to be used only for Interac Security setup.' Below the note are four pairs of input fields: 'Inbound KEK Counter' and 'Outbound KEK Counter' (both with text boxes containing '0'), 'Inbound PIN Index' and 'Outbound PIN Index' (both with dropdown menus showing '1'), 'Inbound MAC Index' and 'Outbound MAC Index' (both with dropdown menus showing '1'), and 'Inbound KME Index' and 'Outbound KME Index' (both with dropdown menus showing '1'). Below these fields is a section titled 'New Key Exchange Key' which includes a 'Key Length' dropdown menu set to 'Single'. At the bottom is a table with four columns: 'Key Part 1', 'Key Part 2', 'Key Part 3', and 'Check Digits'. The first row of the table has a small box containing the number '1' in the first column.

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				

Interface Security Configuration window

The Interface Security Configuration window enables you to configure keys and parameters required by interfaces for PIN encryption, message authentication, and data encryption. Key exchange keys for PIN and MAC keys are entered on the Exchange Information tab. Key exchange keys for data encryption keys are entered on the Data Encr Exch Info tab. You can use the same key exchange key (import key) for both decrypting keys received from the interface and sending keys to the interface, or you can use a separate key (export key) for sending keys to the interface. PIN, MAC, and data encryption working keys are entered on the PIN Information, MAC Information, and Data Encr Key Info tabs, respectively. Static master PIN and MAC keys used to encrypt and decrypt transport (key exchange) and session (working) keys for dynamic online distribution are entered on the Master Keys tab. The Data Encr Exch Intf and Data Encr Key Info tabs are currently used for the IMNI interface only. The Master Keys tab is currently used for the Equens interface only.

Samples of the Exchange Information, PIN Information, MAC Information, Data Encr Exch Intf, Data Encr Key Intf, and Master Keys tabs of the Interface Security Configuration window are shown on the following pages. Refer to the ACI Online Help provided with the ACI desktop for detailed descriptions of this window and its fields.

Note: You must configure a security device using the Security Device Configuration window before you can configure interface security information using the Interface Security Configuration window. The Data Encr Exch Intf and Data Encr Key Info tabs are only enabled for Atalla NSPs or SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs). The Master Keys tab is disabled for all types of security devices except for Thales e-Security HSMs.

Series, Use Series, and Key Hierarchy fields

The Series, Use Series, and Key Hierarchy fields displayed at the top of the Interface Security Configuration window are all specific to BASE24-eps processing and are not used for BASE24 Transaction Security Services processing.

Old key timer limit

The Old Key Timer Limit field located at the top of the Interface Security Configuration window specifies the maximum length of time, in seconds, that a PIN, MAC, or data encryption key can be used after a successful dynamic key exchange before it is automatically cleared.

This timer limits the amount of time that an old key can be used for PIN verification, message authentication, or data encryption after a successful key exchange has taken place.

If you set this field to a value of 0, the previous key is not deleted from the Interface Security File (ICHG_SECURITY). When a key exchange occurs, the current key becomes the previous key and the new key becomes the current key.

SafeNet (Eracom) ProtectHost White AMB HSMs

When using SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs) in your system, the fields displayed on the Interface Security Configuration window for entering keys differ significantly from those shown and described in this section. For detailed information on configuring interface keys for SafeNet (Eracom) ProtectHost White AMB HSMs, refer to [section 4](#).

Banksys DEP HSMs

When you use Banksys Data Encryption Peripheral (DEP) HSMs in your system, the fields displayed on the Interface Security Configuration window for entering keys also differ significantly from those shown and described in this section. For detailed information on configuring interface keys for Banksys DEP HSMs, refer to [section 4](#).

Thales e-Security HSMs

When you use Thales e-Security HSMs in your system, the Import and Export Key Type fields displayed on the Exchange Information tab and the Key Type fields displayed on the MAC Information and Data Encryption Key Info tabs are not used. These fields are currently only used for BASE24-eps processing.

Memory usage

Interface security key data is accessed from different sources during transaction processing, depending on how each interface is configured on the Interface Security Configuration window. The security key data can be obtained from an internal process memory collection and the ICHG_SECURITY, which is an Enscribe disk file, or from the Interface external memory table (OLTP), which is a hash table designed to improve performance for interfaces that do not exchange keys.

The Transaction Security Services process always first attempts to access an interface record from the Interface external memory table (OLTP). If the Option field on the Exchange Information tab of the Interface Security Configuration window for the record is set to a value of “No keys exchanged”, the security key data for the interface is obtained from the external memory table (OLTP) during processing. If the Option field is set to any other value, the interface exchanges keys. In this case, the Transaction Security Services process attempts to access the interface record from an internal process memory collection to obtain the security key data. If the key data is not present in the internal memory collection, the process reads the record from the ICHG_SECURITY disk file and loads it into the internal memory collection.

When you change values on the Interface Security Configuration window for an interface that does not exchange keys, you must rebuild the Interface external memory table (OLTP) and warmboot the OLTP, or stop and restart, the Transaction Security Services process to reread the Interface external memory table (OLTP). The new values will be used in processing for the next transaction for the interface that requires security key data.

Refer to the “[External memory tables \(OLTPs\)](#)” and “[Internal memory tables](#)” topics provided later in this section for procedures on rebuilding external memory tables (OLTPs), warmbooting OLTPs, and warmbooting Transaction Security Services processes to reread internal memory tables.

Future Keys

Future (“Future” and “(F)”) key rows displayed on the Interface Security Configuration window are not applicable to BASE24 Transaction Security Services processing. These rows are only used for BASE24-eps processing. Only current rows (“Current” and “(C)”) are used for BASE24 Transaction Security Services processing.

	Header	Key Part 1	Key Part 2	Key Part 3
Current				
Future				

	Header	Key Part 1	Key Part 2	Key Part 3	
Import(C)					
Export(C)					
Import(F)					
Export(F)					

BASE24-eps tabs

The AS2805 Specific Keys, German Info, Master Info, and Level 2 Import Master Info tabs are not applicable to BASE24 Transaction Security Services processing. These tabs are only used for BASE24-eps processing.

Exchange Information tab

The Exchange Information tab enables you to configure key exchange key information for an interface.

The Keys Generated Online field specifies whether key exchange keys and working keys for the interface are generated online (dynamically). If the interface uses a three-tier key management hierarchy, such as for the Dutch Security Extensions, you must enter a check mark in this field and enter the appropriate static offline PIN and MAC master keys on the Master Keys tab. If you have not licensed the Dutch Security Extensions, this field is disabled.

When the Keys Generated Online field is checked, you can enter information in the Option, Key Variant, and Key Length fields for both PIN and MAC key exchange keys, but not in the Import or Export key fields, since the keys in these fields are generated dynamically. When the Keys Generated Online field is not checked or disabled, you can enter data in all fields on this tab.

The Keys Generated Online field is not stored in the Interface Security File (ICHG_SECURITY) when you save a record. Its only purpose is to let you save a record without entering key exchange key values in the Import and Export fields on this tab. When you enter a check mark in this field, you can save records without entering key exchange key values in these fields. If you do not enter a check mark in this field, the system will not allow you to save a record without key exchange key values in these fields.

To support the Dutch Security Extensions, set the Option fields for both PIN keys and MAC keys to “Use import key for both functions”, and the Key Length fields for both PIN keys and MAC keys to either Single (for single DES) or Double (for triple DES).

Note: When implementing the Dutch Security Extensions, the Key Length fields on this tab, the PIN Information tab, and the MAC Information tab must all be set to the same value (i.e., either Single or Double).

An example of the Exchange Information tab on the Interface Security Configuration window is shown below.

The screenshot shows the 'Interface Security Configuration' window with the 'Exchange Information' tab selected. The window has a title bar and a menu bar. Below the menu bar, there are several configuration fields and a table.

Configuration fields include:

- Interface Name:
- Series:
- Use Series: ☐
- External Key Block Format:
- Old Key Timer Limit:
- Key Hierarchy:
- Exchange Key Option:

Below these fields are several tabs: MAC Information, Data Encr Exch Info, Data Encr Key Info, German Info, and AS2805 Specific Keys. The 'Exchange Information' tab is currently selected.

Under the 'Exchange Information' tab, there are fields for:

- Keys Generated Online: ☐
- Import Key Type:
- Export Key Type:

Below these fields are two tabs: PIN Keys and MAC Keys. The 'PIN Keys' tab is selected.

Under the 'PIN Keys' tab, there are fields for:

- Option:
- Key Variant:
- Key Length:

At the bottom of the window is a table with 5 columns: Key Part 1, Key Part 2, Key Part 3, and Check Digits. The table has 4 rows, each with a label in the first column: Import(C), Export(C), Import(F), and Export(F).

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Import(C)				
Export(C)				
Import(F)				
Export(F)				

PIN Information tab

The PIN Information tab enables you to enter outbound and inbound PIN encryption keys for an interface.

Note: When the Keys Generated Online field on the Exchange Information tab is checked, you can enter information in the Key Length and Current Index fields for both outbound and inbound PIN keys, but not in the Key Part or Check Digits fields since these keys are generated dynamically. To support the Dutch Security Extensions for an interface, set the Key Length fields for both outbound and inbound PIN keys to either Single (for single DES) or Double (for triple DES) and the Current Index fields for both outbound and inbound PIN keys to a value of 1 or 2. The Key Length fields on this tab, the Exchange Information tab, and the MAC Information tab must all be set to the same value (i.e., either Single or Double).

You can enter data in the PIN Block Format and PIN PAD Character fields, regardless of how the Keys Generated Online field is set.

The Current Index fields indicate which of two outbound PIN encryption keys or two inbound PIN encryptions keys are currently used for an interface. Some interfaces, such as the IMNI Interface, can specify which key to use for a transaction, regardless of which key is identified as currently being used.

An example of the PIN Information tab on the Interface Security Configuration window is shown below.

The screenshot shows the 'Interface Security Configuration' window with the 'PIN Information' tab selected. The window has a title bar and standard Windows window controls. Below the title bar is a toolbar with icons for file operations. The main area contains several configuration sections:

- Interface Name:** A dropdown menu showing 'SELECT/ENTER'.
- Series:** A dropdown menu showing 'SELECT/ENTER'.
- Use Series:** An unchecked checkbox.
- External Key Block Format:** A dropdown menu showing 'Variant method'.
- Old Key Timer Limit:** A numeric field with the value '30'.
- Key Hierarchy:** A dropdown menu showing 'Two Levels'.
- Exchange Key Option:** A dropdown menu showing 'Separate Key exchange keys'.
- MAC Information:** A sub-tab labeled 'Master Info'.
- Data Encr Exch Info:** A sub-tab labeled 'Level 2 Import Master Info'.
- Data Encr Key Info:** A sub-tab labeled 'Exchange Information'.
- German Info:** A sub-tab.
- AS2805 Specific Keys:** A sub-tab labeled 'PIN Information'.
- PIN Block Format:** A dropdown menu showing 'ANSI PIN (ISO Format 0)'.
- PIN PAD Character:** A dropdown menu showing 'F'.
- Outbound Keys:** A section with a 'Key Length' dropdown set to 'Single', a 'Current Index' spinner set to '1', and a 'Key Counter' set to '0'. Below this is a table with columns 'Key Part 1', 'Key Part 2', 'Key Part 3', and 'Check Digits'. The table has three rows: 'Current 1' (highlighted in blue), 'Current 2', and 'Future'.
- Inbound Keys:** A section with a 'Key Length' dropdown set to 'Single', a 'Current Index' spinner set to '1', and a 'Key Counter' set to '0'. Below this is a table with columns 'Key Part 1', 'Key Part 2', 'Key Part 3', and 'Check Digits'. The table has three rows: 'Current 1' (highlighted in blue), 'Current 2', and 'Future'.

MAC Information tab

The MAC Information tab enables you to enter outbound and inbound MAC encryption keys for an interface.

Note: When the Keys Generated Online field on the Exchange Information tab is checked, you can enter information in the Key Length and Current Index fields for both outbound and inbound MAC keys, but not in the Key Part or Check Digits fields since these keys are generated dynamically. To support the Dutch Security Extensions for an interface, set the Key Length fields for both outbound and inbound MAC keys to either Single (for single DES) or Double (for triple DES) and the Current Index fields for both outbound and inbound MAC keys to a value of 1 or 2. The Key Length fields on this tab, the Exchange Information tab, and the PIN Information tab must all be set to the same value (i.e., either Single or Double).

You can enter data in the MAC Option and MAC Length fields, regardless of how the Keys Generated Online field is set.

Pseudo triple DES (ANSI X9.19) message authentication is specified by the “Pseudo 3DES Cipher Block Chaining” value in the MAC Option field. Check the firmware version requirements for support of this type of message authentication in Atalla, Banksys DEP, SafeNet (Eracom), and Thales e-Security hardware security devices.

When viewing MAC keys on this window that were generated dynamically, the Key Length field must be set to the same length as the key that was dynamically generated. Otherwise, an “incorrect key size” error message is displayed.

The Current Index fields indicate which of two outbound MAC encryption keys or two inbound MAC encryptions keys are currently used for an interface. Some interfaces, such as the IMNI Interface, can specify which key to use for a transaction, regardless of which key is identified as currently being used.

An example of the MAC Information tab on the Interface Security Configuration window is shown below.

The screenshot shows the 'Interface Security Configuration' window with the 'MAC Information' tab selected. The window has a title bar with standard Windows controls. Below the title bar is a toolbar with icons for file operations. The main area contains several configuration fields and tabs.

Configuration Fields:

- Interface Name:
- Series:
- Use Series: ☐
- External Key Block Format:
- Old Key Timer Limit:
- Key Hierarchy:
- Exchange Key Option:
- MAC Option:
- MAC Length:

Tabs:

- Master Info
- Level 2 Import Master Info
- Exchange Information
- PIN Information
- MAC Information** (selected)
- Data Encr Exch Info
- Data Encr Key Info
- German Info
- AS2805 Specific Keys

Outbound Keys Section:

Key Length: Current Index: Key Counter 0: Key Type:

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Current 1				
Current 2				
Future				

Inbound Keys Section:

Key Length: Current Index: Key Counter 0: Key Type:

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Current 1				
Current 2				
Future				

Data Encr Exch Info tab

An example of the Data Encr Exch Info tab on the Interface Security Configuration window is shown below. This tab is only valid for Atalla NSPs or SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs).

Note: When using a SafeNet (Eracom) ProtectHost White Mark II HSM, the Key Length field is hidden because it is not used. Also, the Key Variant field is disabled for the same reason. Data encryption exchange keys must be stored in the Mark II HSM and entered as a five-decimal index value in the Key Part 1 field.

The screenshot shows the 'Interface Security Configuration' window with the 'Data Encr Exch Info' tab selected. The window contains various configuration fields and tables.

Configuration Fields:

- Interface Name:
- Series:
- Use Series: ☐
- External Key Block Format:
- Old Key Timer Limit:
- Key Hierarchy:
- Exchange Key Option:

Tabbed Interface:

- Master Info
- Level 2 Import Master Info
- Exchange Information
- PIN Information
- MAC Information
- Data Encr Exch Info** (selected)
- Data Encr Key Info
- German Info
- A52805 Specific Keys

Exchange Keys Section:

Exchange Keys Option: Key Variant: Key Length:

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Import(C)				
Export(C)				
Import(F)				
Export(F)				

Initialization Vector Section:

	Key Part 1	Key Part 2	Key Part 3
1			
2			

Data Encr Key Info tab

The Data Encr Key Info tab enables you to enter outbound and inbound data encryption keys for an interface.

The Current Index fields indicate which of two outbound data encryption keys or two inbound data encryptions keys are currently used for an interface. Some interfaces, such as the IMNI Interface, can specify which key to use for a transaction, regardless of which key is identified as currently being used.

Note: When using a SafeNet (Eracom) ProtectHost White Mark II HSM, the Data Type and Encryption Layers fields are disabled.

An example of the Data Encr Key Info tab on the Interface Security Configuration window is shown below. This tab is only valid for Atalla NSPs or SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs).

The screenshot shows the 'Interface Security Configuration' window with the 'Data Encr Key Info' tab selected. The window includes various configuration fields and a table for inbound keys.

Interface Name: SELECT/ENTER **Series:** SELECT/ENTER

Use Series: ☐

External Key Block Format: Variant method **Old Key Timer Limit:** 30

Key Hierarchy: Two Levels **Exchange Key Option:** Separate Key exchange keys

Master Info | Level 2 Import Master Info | Exchange Information | PIN Information

MAC Information | Data Encr Exch Info | Data Encr Key Info | German Info | AS2805 Specific Keys

DES Mode: ECB **Data Type:** Unpacked ASCII hex... **Encryption Layers:** 1

Inbound | Outbound

Inbound Keys: Key Length: Single **Current Index:** 1 **Key Type:** ZEK

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Current 2				
Future				

Master Keys tab

The Master Keys tab is used to configure offline static PIN and MAC master keys used with random values to encrypt and decrypt transport (key exchange) and session (working) keys for dynamic online distribution. This tab is only available when you license the Dutch Security Extensions and use Thales e-Security HSMs. Otherwise, the tab is disabled.

Note: The ACI desktop does not validate check digits entered on this window.

An example of the Master Keys tab on the Interface Security Configuration window is shown below.

The screenshot shows the 'Interface Security Configuration' window with the 'Master Keys' tab selected. The window has a title bar with standard Windows controls. Below the title bar, there's a section for 'Interface Name' with a dropdown menu showing 'SELECT/ENTER'. To the right are icons for save, print, refresh, delete, and help. Below this is a section for 'External Key Block Format' with a dropdown menu showing 'Variant method' and 'Old Key Timer Limit' with a numeric input set to '30'. The main area has four tabs: 'Exchange Information', 'PIN Information', 'MAC Information', and 'Master Keys' (which is active). Under 'Master Keys', there are two sub-sections: 'PIN Key' and 'MAC Key'. Each sub-section contains a table with columns for 'Key Part 1', 'Key Part 2', 'Key Part 3', and 'Check Digits'. The 'PIN Key' table has a row with a '1' in the first column. The 'MAC Key' table also has a row with a '1' in the first column.

Interface Security Configuration				
Interface Name		SELECT/ENTER		
External Key Block Format		Variant method		
Old Key Timer Limit		30		
Exchange Information		PIN Information		MAC Information
Data Encr Exch Info	Data Encr Key Info	German Info	Master Keys	AS2805 Specific Keys
PIN Key				
	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				
MAC Key				
	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				

Maintaining KEYF and interface security data

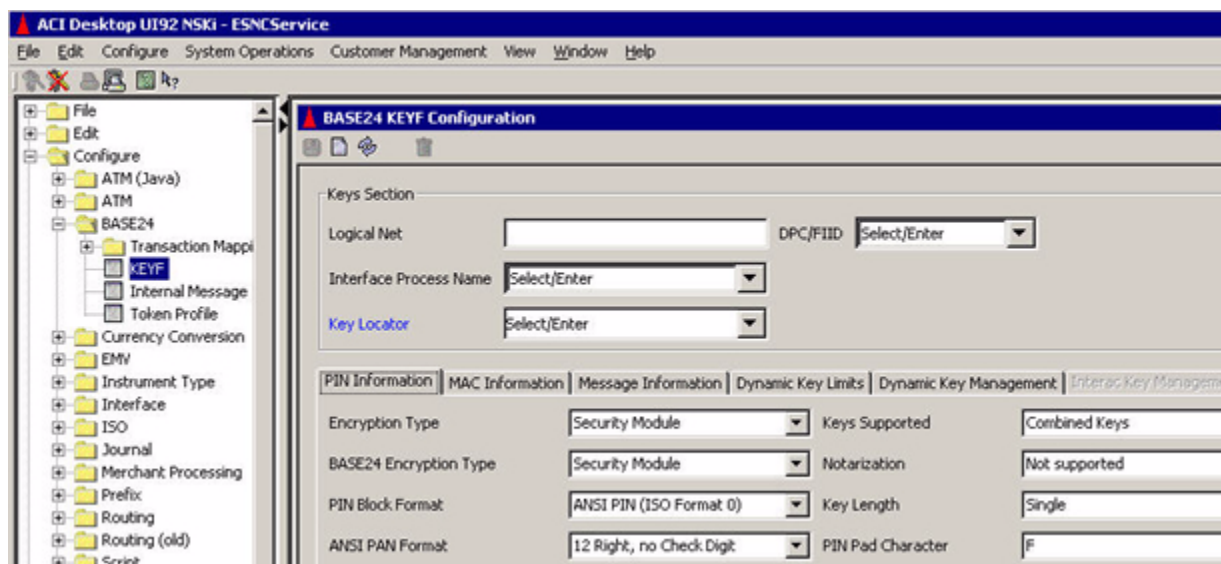
The following paragraphs describe procedures for adding, modifying, and deleting interface records in the Key File (KEYF) and in the Interface Security File (ICHG_SECURITY).

Adding new KEYF and ICHG_SECURITY records

Use the following procedure to add a new KEYF and ICHG_SECURITY record when adding an interface to your BASE24 system.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the BASE24 KEYF Configuration window by accessing **Configure > BASE24 > KEYF**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Optionally enter the logical network of the KEYF you want to access in the Logical Net field.
 - If you do not specify a logical network value, the application uses the first entry that is defined for the BASE24_KEYF assign in the Metadata Configuration File (CONFCSV).
 - If you do specify a logical network value, it must match one of the BASE24_KEYF assigns in the CONFCSV (the logical network value is appended to the BASE24_KEYF assign). If you specify a logical network value that does not exist in a BASE24_KEYF assign, the application will notify you that the data could not be found.

- Place the cursor in the DPC/FIID field, type a new DPC or FIID value, and press the **Enter** key. The Interface Process Name field becomes active.

BASE24 KEYF Configuration

Keys Section

Logical Net: [] DPC/FIID: 0001

Interface Process Name: Select/Enter

Key Locator: Select/Enter

PIN Information | MAC Information | Message Information | Dynamic Key Limits | Dynamic Key Management | Interac Key Management

Encryption Type: Security Module Keys Supported: Combined Keys

BASE24 Encryption Type: Security Module Notarization: Not supported

PIN Block Format: ANSI PIN (ISO Format 0) Key Length: Single

ANSI PAN Format: 12 Right, no Check Digit PIN Pad Character: F

- Click once in the Interface Process Name field, type the symbolic name of the new Interface process, and press the **Tab** key.

BASE24 KEYF Configuration - P1A^HISO01

Keys Section

Logical Net: [] DPC/FIID: 0001

Interface Process Name: P1A^HISO01

Key Locator: Select/Enter

PIN Information | MAC Information | Message Information | Dynamic Key Limits | Dynamic Key Management | Interac Key Management

Encryption Type: Security Module Keys Supported: Combined Keys

BASE24 Encryption Type: Security Module Notarization: Not supported

PIN Block Format: ANSI PIN (ISO Format 0) Key Length: Single

ANSI PAN Format: 12 Right, no Check Digit PIN Pad Character: F

6. Enter any 16 character hexadecimal value in the Key Locator field and click on the Key Locator field label hypertext link to display the Interface Security Configuration window.

BASE24 KEYF Configuration - P1A^HISO01

Keys Section

Logical Net: DPC/FIID:

Interface Process Name:

Key Locator:

[Click here to go to the Interface Security Configuration window.](#)

PIN Information | MFA | Dynamic Key Limits | Dynamic Key Management | Interac Key Management

Encryption Type: Keys Supported:

BASE24 Encryption Type: Notarization:

PIN Block Format: Key Length:

ANSI PAN Format: PIN Pad Character:

7. Determine the next available key locator value. Click on the selectable list for the Interface Name field to find the last key locator value used for the logical network you are logged on to. If you have multiple logical networks in your system, the first two characters of the key locator value identifies the logical network. For example, if you are adding an interface to logical network 01 (i.e., the LN-IND param is set to 01) and the last key locator value used is 0100000000000009, the next key locator value would be 0100000000000010.

Note: If BASE24 Enhanced Authorization with the IMT Interface is installed on your system, the interface names associated with the IMT Interface will also be listed in the selectable list for the Interface Name field.

Interface Security Configuration

Interface Name: SELECT/ENTER
 Series: SELECT/ENTER

Use Series: INTF_IMT_ATM, INTF_IMT_ISS_ATM, INTFSVS_99, JEANIE, JIK, KKK, MDS, MM01 TEST

External Key Block: [] Old Key Timer Limit: 30

Key Hierarchy: []

MAC Information: MDS, MM01 TEST

Keys Generated Online: ☐

PIN Keys | MAC Keys

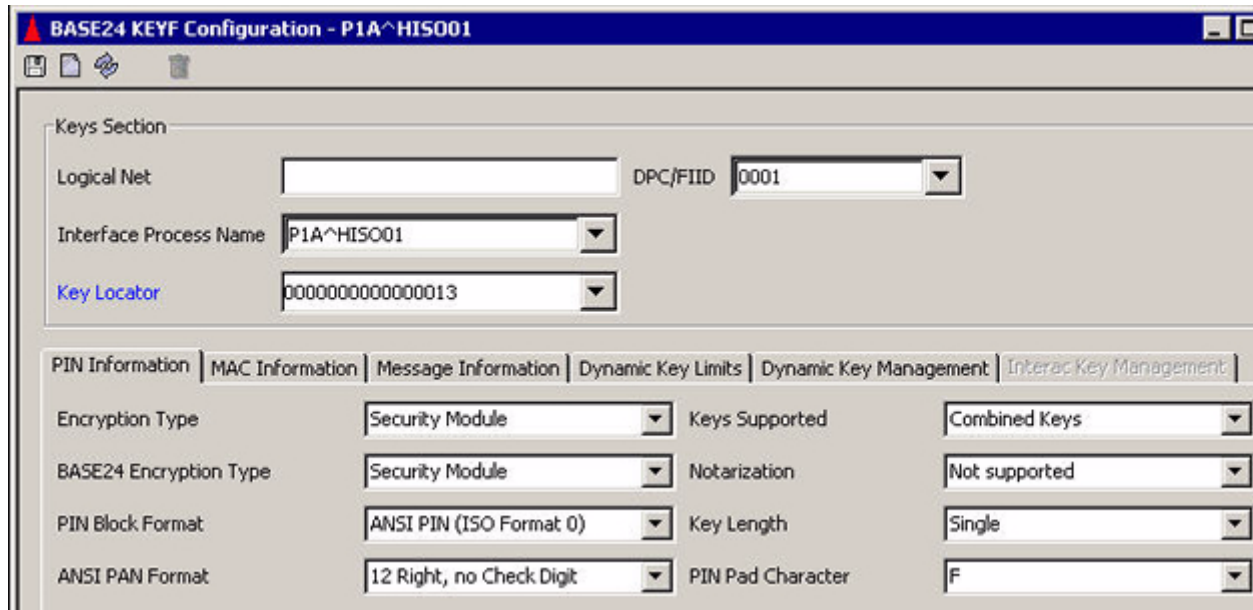
PIN Keys Option: Use import key for both... Key Variant: 0 - Key Exchang... Key Length: Single

	Key Part 1	Key Part 2	Key Part 3	Check Digits
Import(C)				
Export(C)				
Import(F)				
Export(F)				

Actual key locator values will be displayed in the selectable list. The only interface names displayed will be for the IMT interface if BASE24 Enhanced Authorization is installed.

- Exit the Interface Security Configuration window without saving changes and return to the BASE24 KEYF Configuration window.

- Click in the Key Locator field, enter the new key locator value determined from step 7, copy it to your clipboard, and press the **Enter** key.



BASE24 KEYF Configuration - P1A^HISO01

Keys Section

Logical Net: DPC/FIID:

Interface Process Name:

Key Locator:

PIN Information | MAC Information | Message Information | Dynamic Key Limits | Dynamic Key Management | *Interac Key Management*

Encryption Type: Keys Supported:

BASE24 Encryption Type: Notarization:

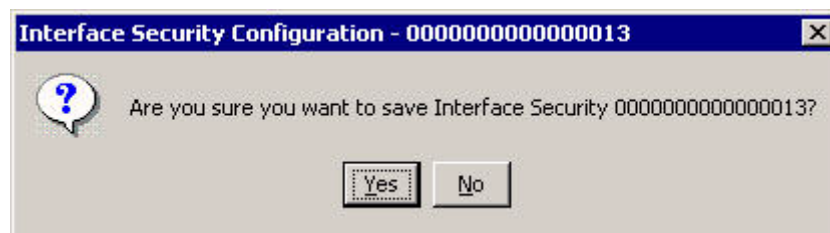
PIN Block Format: Key Length:

ANSI PAN Format: PIN Pad Character:

10. Click on the Key Locator field label again. Paste the new key locator value in the Interface Name field on the Interface Security Configuration window.

11. Enter all required and optional data as needed for the ICHG_SECURITY record and click Save (). Answer Yes to the "Are you sure you want to save" dialog box.

Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields for each tab on this window.

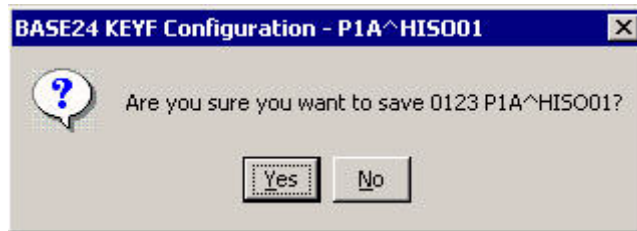


If the record is successfully added, a message similar to the following is displayed at the bottom of the window.

13 - 02:43 PM > Successful insert of Interface Security 0100000000000010 .

12. Exit the Interface Security Configuration window and return to the BASE24 KEYF Configuration window.
13. Enter all required and optional data as needed for the KEYF record and click Save (). Answer Yes to the “Are you sure you want to save” dialog box.

Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields for each tab on this window.



If the record is successfully added, a message indicating that “BASE24 KEYF Information Inserted” is displayed at the bottom of the window.

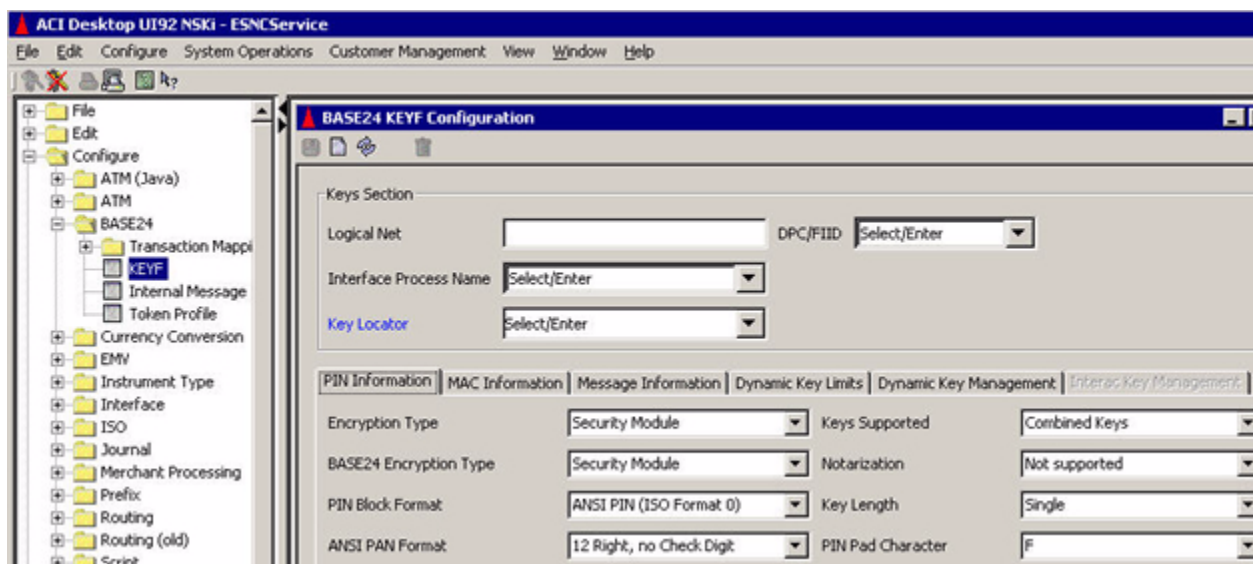
14. If the interface for which you added an ICHG_SECURITY record does not support dynamic key management (indicated by a value of “No keys exchanged” in the Options field on the Exchange Information tab of the Interface Security Configuration window), the ICHG_SECURITY record is accessed in an external memory table (OLTP), which must be rebuilt. Otherwise, the ICHG_SECURITY record is accessed as an internal memory table, which does not need to be rebuilt. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for instructions on rebuilding external memory tables (OLTPs).
15. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. This step is required only if the ICHG_SECURITY is accessed as an external memory table (OLTP). Refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for external memory tables (OLTPs).

Modifying KEYF and ICHG_SECURITY records

Use the following procedure to modify KEYF and ICHG_SECURITY records for an interface to your BASE24 system.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the BASE24 KEYF Configuration window by accessing **Configure > BASE24 > KEYF**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Enter the logical network ID (e.g., PRO1, PRO2, etc.) in the Logical Net field and click Refresh (🔄). The system determines whether the logical network is defined. If not, an error message is displayed.
4. Select the DPC of the host or FIID of the interchange from the selectable list for the DPC/ FIID field.
5. Select the interface process symbolic name from the selectable list for the Interface Process Name field. Click once in the Interface Process Name field to refresh the window. The KEYF record to be modified should be displayed.
6. Update any fields as needed for the KEYF record.

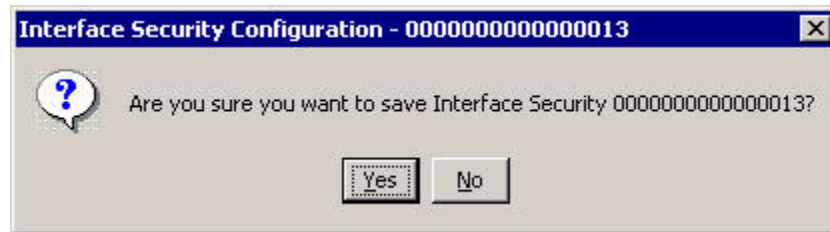
Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields and their values for each tab on this window.

7. If you need to update any key information for this KEYF record, click on the Key Locator field label. The Interface Security Configuration window is displayed with the related ICHG_SECURITY record.

If you need to update any key values, double-click in the appropriate field (e.g., Key Part 1). The key value is unformatted so that you can edit it. When you are finished editing the key value, press the **Enter** key to reformat the value.

Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields and their values for each tab on this window.

When you are finished updating the ICHG_SECURITY record, click Save (). Answer Yes to the “Are you sure you want to save” dialog box.



If the record is successfully updated, a message indicating that update was successful is displayed at the bottom of the window.

Exit the Interface Security Configuration window.

If you made any changes on the BASE24 KEYF Configuration window, click Save (). Answer Yes to the “Are you sure you want to save” dialog box.

8. If you updated an ICHG_SECURITY record and the interface does not support dynamic key management (indicated by a value of “No keys exchanged” in the Options field on the Exchange Information tab of the Interface Security Configuration window), the ICHG_SECURITY record is accessed in an external memory table (OLTP), which must be rebuilt. Otherwise, the ICHG_SECURITY record is accessed as an internal memory table, which does not need to be rebuilt. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for instructions on rebuilding external memory tables (OLTPs).
9. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. This is required whether the ICHG_SECURITY is accessed as an external memory table (OLTP) or an internal memory table.
 - If the ICHG_SECURITY is accessed as an external memory table (OLTP), refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for external memory tables (OLTPs).
 - If the ICHG_SECURITY is accessed as an internal memory table, refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for internal memory tables.

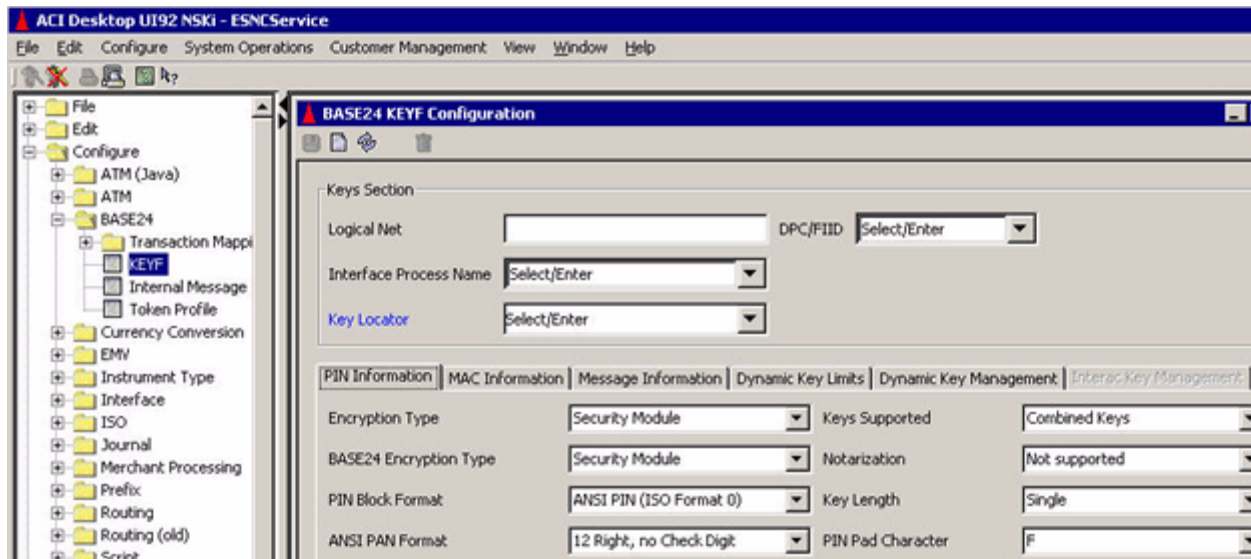
Deleting KEYF and ICHG_SECURITY records

Use the following procedure to delete KEYF and ICHG_SECURITY records for an interface from your BASE24 system.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.

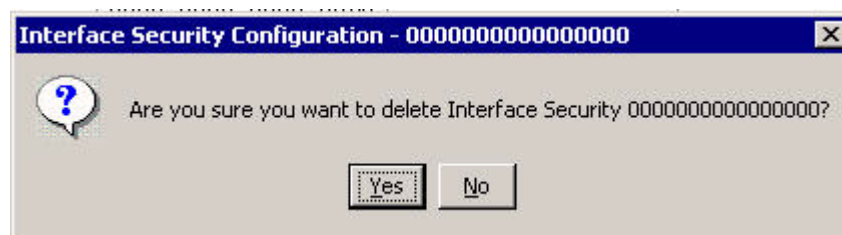
2. Display the BASE24 KEYF Configuration window by accessing **Configure > BASE24 > KEYF**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure



3. Enter the logical network ID (e.g., PRO1, PRO2, etc.) in the Logical Net field and click Refresh (🔄). The system determines whether the logical network is defined. If not, an error message is displayed.
4. Select the DPC of the host or FIID of the interchange from the selectable list for the DPC/ FIID field.
5. Select the interface process symbolic name from the selectable list for the Interface Process Name field. Click once in the Interface Process Name field to refresh the window. The KEYF record to be deleted should be displayed.
6. If you need to delete any key information associated with this KEYF record, click on the Key Locator field label. The Interface Security Configuration window is displayed with the related ICHG_SECURITY record.

Click Delete (🗑️). Answer Yes to the “Are you sure you want to delete” dialog box.



If the record is successfully deleted, a message indicating that the delete was successful is displayed at the bottom of the window.

Exit the Interface Security Configuration window.

7. On the BASE24 KEYF Configuration window, click Delete (). Answer Yes to the “Are you sure you want to delete” dialog box.
If the record is successfully deleted, a message indicating that the delete was successful is displayed at the bottom of the window.
8. If you deleted an ICHG_SECURITY record and the interface does not support dynamic key management (indicated by a value of “No keys exchanged” in the Options field on the Exchange Information tab of the Interface Security Configuration window), the ICHG_SECURITY record is accessed in an external memory table (OLTP), which must be rebuilt. Otherwise, the ICHG_SECURITY record is accessed as an internal memory table, which does not need to be rebuilt. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for instructions on rebuilding external memory tables (OLTPs).
9. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. This is required whether the ICHG_SECURITY is accessed as an external memory table (OLTP) or an internal memory table.
 - If the ICHG_SECURITY is accessed as an external memory table (OLTP), refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for external memory tables (OLTPs).
 - If the ICHG_SECURITY is accessed as an internal memory table, refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for internal memory tables.

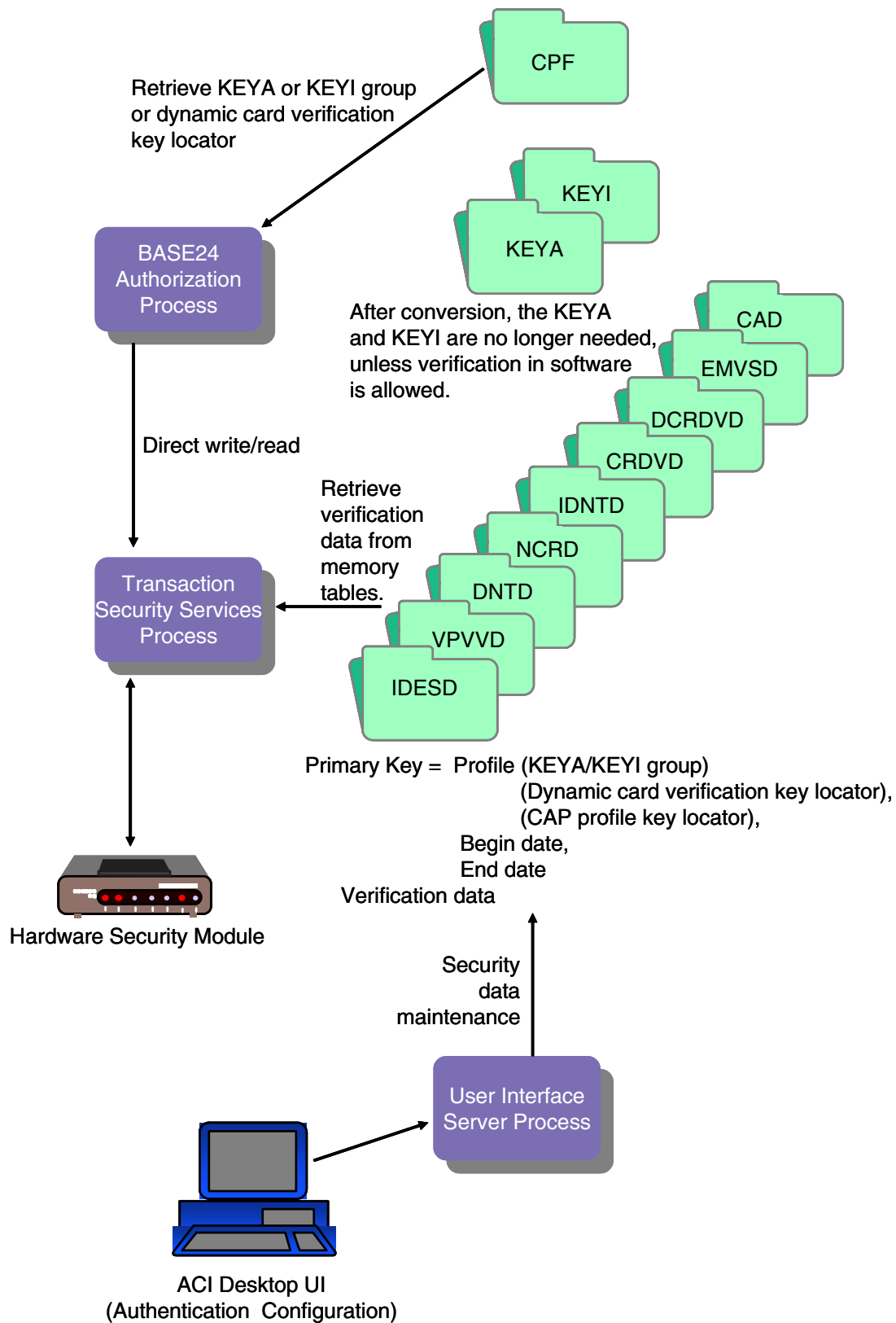
Key authorization files

The following diagram illustrates the relationships between the BASE24 key authorization files—Key Authorization File (KEYA) and ICC Key File (KEYI) and the key authorization files maintained by the Transaction Security Services process after file conversion. The hardware encrypted key fields in the KEYA and KEYI are set to binary zeros and the actual key values are stored in the IBM DES Verification File (IBM_DES), Diebold Number Table File (DIEBOLD_NUM_TBL), Identikkey PIN Verification File (Identikkey), NCR PIN Verification File (NCR_PIN_VRFY), Visa PVV Verification File (Visa_PVV), EMV Authorization File (EMV_Security), or Card Verification File (CARD_VRFCTN). The same primary key values used in the converted KEYA and KEYI records are copied to the IBM_DES, DIEBOLD_NUM_TBL, Identikkey, Visa_PVV, CARD_VRFCTN, DYN_CARD_VRFCTN, or EMV_Security. Data in the IBM_DES, DIEBOLD_NUM_TBL, Identikkey, Visa_PVV, CARD_VRFCTN, and EMV_Security is maintained from the Authentication Profile Configuration window. After the KEYA and KEYI have been converted, they are no longer accessed during transaction processing, unless PIN verification, card verification, or PIN generation in software is required and it is allowed. If PIN verification, card verification, or PIN generation in software is required and allowed, the Authorization process still accesses the KEYA and KEYI for key information.

Note: Due to their recent implementation, the Dynamic Card Verification File (DYN_CARD_VRFCTN) and Chip Authentication File (Chip_Authentication) have no related hardware encrypted key fields in the KEYA or KEYI.

The following table shows you the fields in the Card Prefix File (CPF) that are used to access the appropriate records in the TSS database. The value in these fields must match a value entered in the Profile field on the Authentication Profile Configuration window.

CPF field	TSS file
PIN VERIFICATION KEYA GROUP (CPF screen 2)	IBM_DES, DIEBOLD_NUM_TBL, Identikay, NCR_PIN_VRFY, Visa_PVV
KEYI GROUP (CPF screen 11)	EMV_Security
CAP GROUP (CPF screen 13)	Chip_Authentication
CV KEYA GROUP (CPF screen 2)	CARD_VRFCTN
MANUAL CV KEYA GROUP (CPF screen 2)	
SIV KEYA GROUP (CPF screen 6)	
DYN CARD VERIF KEY LOCATOR (CPF screen 3)	DYN_CARD_VRFCTN
NEW PIN TRANSPORT KEY LOCATOR (CPF screen 11)	ICHG_SECURITY



Authentication Profile Configuration window

The Authentication Profile Configuration window on the ACI desktop user interface enables you to maintain PIN, card, dynamic card (contactless), and EMV verification keys and parameters. This window supports the entry of keys and parameters for the following methods of PIN verification: IBM DES (3624-format), Visa PVV, Diebold, NCR, and Identikey. You can also enter card verification and dynamic card verification key pairs as well as keys and parameters required for processing EMV transactions and CAP token validation transactions. Fields are displayed on ten tabs on this window: DES (IBM 3624) PIN Verification, Visa PVV Keys, Diebold, NCR, Identikey, EMV, Chip Auth, Card, Contactless, and Dual PVN PIN (this tab is only supported in BASE24-eps). The PIN and card verification types you select in one or more rows at the top of the window dictate which tabs are active on the window. Checkmarks in the IBM DES, Diebold, NCR, and Identikey columns and “Visa PVV” in the Visa PVV column indicate which types of PIN verification are configured for the profile. Values in the Visa PVV column indicate whether Visa PVV, dual PVN, or both are configured for the profile (dual PVN is only supported in BASE24-eps). Values in the EMV column indicate whether EMV authentication, chip authentication (CAP token validation), or both are configured for the profile. Values in the Card column indicates whether card verification, contactless card verification, or both are configured for the profile. A sample of this table is shown below.

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
December 2011	December 2011	☐	None ☐	☐	☐	☐	None ☐	None ☐

Samples for each of the supported tabs on the Authentication Profile Configuration window are described on the following pages. Refer to the ACI Online Help provided with the ACI desktop for detailed descriptions of this window and its fields.

Note: You must configure a security device using the Security Device Configuration window before you can configure authentication security information using the Authentication Profile Configuration window.

SafeNet (Eracom) ProtectHost White AMB HSMs

When you use SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs) in your system, the fields displayed on the Authentication Profile Configuration window for entering keys differ significantly from those shown and described in this section. For detailed information on configuring keys for SafeNet (Eracom) ProtectHost White AMB HSMs, refer to [section 4](#).

Banksys DEP HSMs

When you use Banksys Data Encryption Peripheral (DEP) HSMs in your system, the fields displayed on the Authentication Profile Configuration window for entering keys also differ significantly from those shown and described in this section. For detailed information on configuring keys for Banksys DEP HSMs, refer to [section 4](#).

BASE24-eps tabs

The Dual PVN PIN tab is not applicable to BASE24 Transaction Security Services processing. This tab is only used for BASE24-eps processing.

DES (IBM 3624) PIN Verification tab

The DES (IBM 3624) PIN Verification tab of the Authentication Profile Configuration window contains the input data used in IBM DES (3624-format) PIN verification. Data entered on this tab is stored in the IBM DES Verification File (IBM_DES).

Note: There has been a well publicized attack on customer PINs that can be facilitated by repeatedly altering the decimalization table and sending PIN verification commands to the HSM. To protect your cardholder's PINs from such an attack, you can store the decimalization table in the memory of the security device or encrypt the decimalization table value in the TSS database. The options available depend on the type of security devices installed in your system as described below.

- For Atalla NSPs, the decimalization table value sent in commands must be in the clear. You can store the clear decimalization table in the memory of the NSP using the SCT or SCA. In this case, you enter the index of this memory location instead of the clear decimalization table value in the Decimalization Table field.
- For SafeNet (Eracom) host security modules (HSMs), the actual value of the decimalization table is stored in the memory of the SafeNet (Eracom) HSM. Therefore, the Decimalization Table field is not used.
- For Thales e-Security HSMs, decimalization table value sent in commands can be encrypted or in the clear. When an HSM configure security parameter is set to support encrypted decimalization tables, you can encrypt the clear decimalization table value using a console command and enter the encrypted value in the Decimalization Table field. Both the clear decimalization table and the encrypted decimalization table consist of 16 hexadecimal characters. Note that you cannot mix clear and encrypted decimalization tables. Either all decimalization tables must be in the clear or must be encrypted. For more information on this configure security parameter, refer to [section 4](#).

An example of the DES (IBM 3624) PIN Verification tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

Identkey	EMV	Chip Auth	Card	Contactless	Dual PWN PIN
DES (IBM 3624) PIN Verification			Visa PWN Keys	Diebold	NCR
Key Length: Single					
		Key Part 1	Key Part 2	Key Part 3	Check Digits
	1				
Decimalization Table PAN Verify Length 4 PAN Pad Character PAN Verify Offset 0					

Visa PVV Keys tab

The Visa PVV Keys tab of the Authentication Profile Configuration window contains the PIN verification key pairs input data used in Visa PVV PIN verification. Data entered on this tab is stored in the Visa PVV Verification File (Visa_PVV).

An example of the Visa PVV Keys tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

Identkey		EMV	Chip Auth	Card	Contactless	Dual PVN PIN	
DES (IBM 3624) PIN Verification				Visa PVV Keys		Diebold	NCR
	Key Pair	Key Part 1	Key Part 2	Key Part 3	Check Digits		
☐	1						
☐							
☐	2						
☐							
☐	3						
☐							
☐	4						
☐							
☐	5						
☐							
☐	6						

Diebold tab

The Diebold tab of the Authentication Profile Configuration window contains the Diebold number table (DNT) information used in Diebold PIN verification. Data entered on this tab is stored in the Diebold Number Table File (DIEBOLD_NUM_TBL). Diebold PIN verification is only supported for Atalla AKB NSPs and Thales e-Security HSMs. This tab is disabled for all other types of security devices.

An example of the Diebold tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

Identkey | EMV | Chip Auth | Card | Contactless | Dual PVN PIN
DES (IBM 3624) PIN Verification | Visa PVV Keys | **Diebold** | NCR

Relative Location Algorithm Number Entry Type

Diebold Number Table

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				

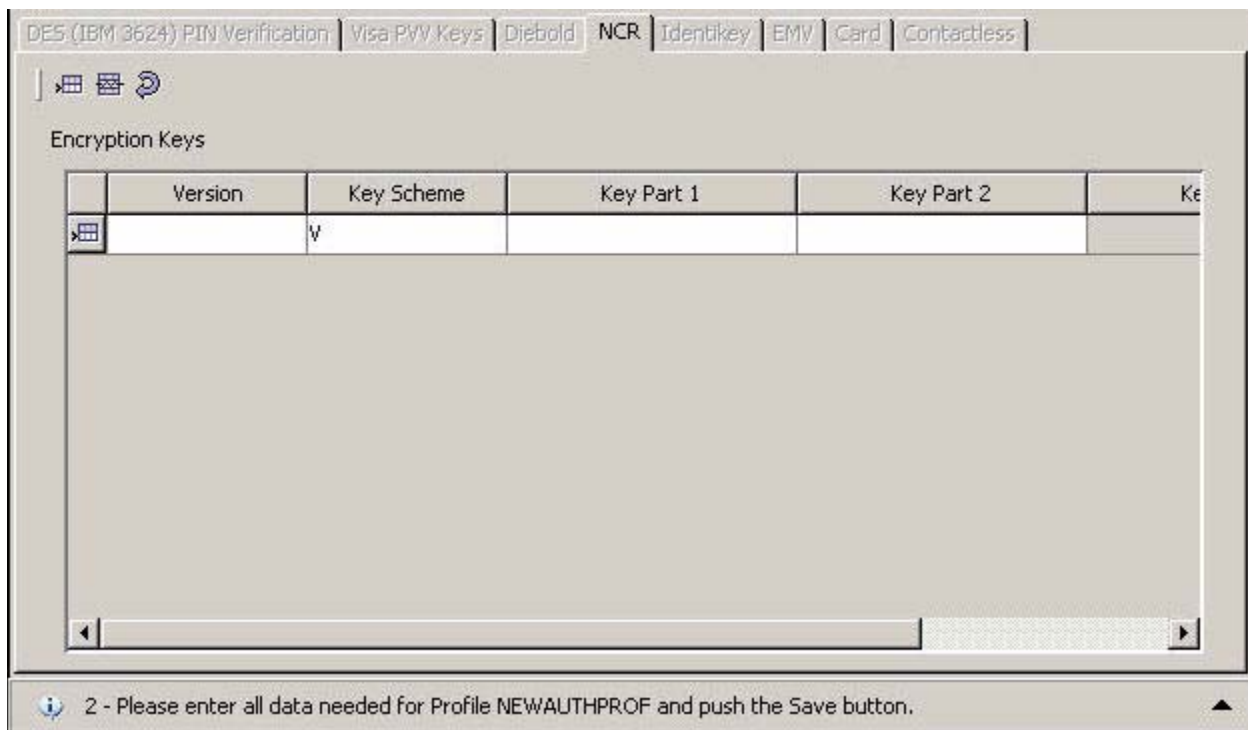
NCR tab

The NCR tab of the Authentication Profile Configuration window contains the encryption keys used for NCR PIN verification. Data entered on this tab is stored in the NCR PIN Verification File (NCR_PIN_VRFY). This tab is only available when you license the Dutch Security Extensions and use Thales e-Security HSMs. Otherwise, the tab is disabled and cannot be accessed.

Note: On this tab, the Begin Date and End Date fields are not used. The Version field is used instead, in combination with the Profile field, to access records in the NCR_PIN_VRFY. The version is a unique number from 000-999 that is assigned by the vendor to the key that you are using to configure PIN verification. After the version number is entered and saved, it cannot be changed.

The Key Scheme field in the Encryption Keys table specifies whether the Thales e-Security HSM is configured to use the old (ANSI X9.17 method) non-variant (32H) key encryption scheme or the variant (1A+32H) key encryption scheme. For more information on these encryption schemes, refer to the Thales **Host Security Module, RG7000 Programmer's Manual**.

An example of the NCR tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.



DES (IBM 3624) PIN Verification | Visa PIV Keys | Diebold | **NCR** | Identkey | EMV | Card | Contactless

Encryption Keys

	Version	Key Scheme	Key Part 1	Key Part 2	Key Part 3
		V			

2 - Please enter all data needed for Profile NEWAUTHPROF and push the Save button.

Identikey tab

An example of the Identikey tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

DES (IBM 3624) PIN Verification		Visa PVV Keys		Diebold	NCR
Identikey	EMV	Chip Auth	Card	Contactless	Dual PVN PIN
Bank ID					
PAN Verify Offset	0				
PAN Verify Length	4				
Compare Indicator	Not Used				

EMV tab

An example of the EMV tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

DES (IBM 3624) PIN Verification		Visa PVV Keys		Diebold		NCR	
Identkey	EMV	Chip Auth	Card	Contactless	Dual PVV PIN		
EMV Scheme	Visa	Branch/Height Parameters			Branch factor 2; Tree Hei...		
Cryptogram Version Number	0E	Destination PIN Block Type			Visa Format Without usin...		
Secure Message Key for Integrity							
		Key Part 1	Key Part 2	Key Part 3	Check Digits		
1							
Secure Message Key for Confidentiality							
		Key Part 1	Key Part 2	Key Part 3	Check Digits		
1							
Cryptographic Keys							
	Key Index	Key Part 1	Key Part 2	Key Part 3	Check Digits		
1							

EMV scheme and cryptogram version number

The combination of EMV scheme and cryptogram version number dictates the type of security processing performed by the Transaction Security Services process.

The EMV Scheme field specifies the EMV implementation used for this authentication profile. Possible values are CCD, Interac, MasterCard/Europay, SECCOS6, or Visa. The SECCOS6 EMV scheme is only supported on BASE24-eps.

The Cryptographic Version Number field specifies the cryptogram version number supported for each EMV implementation, and is used if the cryptogram version number is not present in an EMV transaction request. For the Visa EMV scheme, the possible values in this field are 01, 0A, and 0E; for the MasterCard EMV scheme, the possible values are 00-09, 0A-0F, and 10-15; for the CCD EMV scheme, the possible values are A5, B5, C5, D5, E5, and F5; and for the Interac scheme, the only supported value is 85.

When you select the a scheme in the EMV Scheme field, the Destination PIN Block Type field is a display-only field set to the value needed for the scheme.

Branch / height parameters

The Branch/Height Parameters field sets the branch factor and tree height parameters used for EMV 2000 processing. Refer to the “[EMV 2000](#)” topic in section 17 of this guide for detailed information on how these parameters are used in processing.

Cryptographic keys

Note: You can enter up to 24 cryptographic keys on this window, which are stored in the EMV_Security. When the Transaction Security Services process receives a request that requires a cryptographic key, it uses the derivation key index value of 0–255 passed in the command request to retrieve the cryptographic key used for the function from the EMV_Security. Therefore, you must specify a unique key index value of 0–255 in the Key Index column of the Cryptographic Keys table for each cryptographic key entered on this tab.

If a cryptographic key is not entered in one of the 24 rows of the Cryptographic Keys table, the Key Index column for the table entry will be blank. For these entries, a default key index value of -1 is stored in the EMV_Security.

Chip Auth tab

The Chip Auth tab specifies processing parameters for validating chip authentication program (CAP) tokens. This tab is displayed only for Thales e-Security HSMs.

Because CAP token validation requires a double-length issuer master key (cryptographic key) and cryptographic version number, this tab contains an EMV Key Profile field to retrieve this data from an EMV_Security record. These can be configured in a separate record used only for CAP token validation or in a record that is also used for EMV payment transactions. In either case, the EMV profile record must exist before you create a Chip Authentication File (Chip_Authentication) record from this tab.

An example of the Chip Auth tab on the Authentication Profile Configuration window is shown below, followed by a discussion of its fields.

The screenshot displays the 'Chip Auth' tab within the 'Authentication Profile Configuration' window. The window features a series of tabs at the top, with 'Chip Auth' currently selected. Below the tabs, the 'EMV Cryptography' section contains two dropdown menus: 'EMV Key Profile' (set to 'AUT3') and 'EMV Issuer Application Data Format' (set to 'VISA'). The 'Card Configuration' section follows, with fields for 'Application Interchange Profile', 'Predicted Card Verification Results', 'Use of Initial Card Verification Results' (set to 'Never'), and 'Initial Card Verification Results'. The 'Chip Authentication Configuration' section at the bottom includes 'Application' (set to 'CAP'), 'Internet Authentication Flags', 'Issuer Proprietary Bitmap', 'Number of Resynch Attempts' (set to '0'), and 'HSM Specific Parameters'. Search icons are visible next to the 'Predicted Card Verification Results', 'Initial Card Verification Results', 'Issuer Proprietary Bitmap', and 'HSM Specific Parameters' fields.

EMV cryptography

The EMV Cryptography section of the Chip Auth tab specifies parameters related to EMV cryptography. The EMV Key Profile field allows you to select an EMV_Security configuration that contains the cryptographic keys to be used for CAP token validation. You can use the same configuration for both EMV payment authentication and CAP token validation or you can use a separate configuration for just CAP token validation. The Issuer Application Data field specifies the format (i.e., Visa, M/Chip 2, or M/Chip 4) of issuer application data. The cryptogram version number is retrieved from the EMV_Security configuration.

Card configuration

The fields in the Card Configuration section specify processing parameters for chip authentication cards.

The Application Interchange Profile field specifies the application functions (in four hexadecimal digits) that are supported by the chip authentication application in the ICC and is coded according to EMV specifications. For stand-alone chip authentication applications, this is usually 0x10000, which would be entered as 2000 in hexadecimal format. When chip authentication is combined with EMV payment applications in the ICC, a different value may be required.

The Predicted Card Verification Results field specifies the predicted card verification results (CVR) for a CAP token in hexadecimal format. If the ICC contains a separate CAP authentication application, the CVR bits can be predicted (e.g., a successful CAP transaction includes successful offline PIN verification by design) and do not need to be included in the CAP token. If the ICC contains an EMV payment application that is also used for CAP authentication, then the CVR cannot be predicted and must be included in the CAP token. The bits in the predicted CVR are overwritten with any CVR bits in the CAP token to obtain the final CVR used for application cryptogram generation. The hexadecimal digits in this field must be at least as long as implied by the issuer application data format: 8 hexadecimal digits for Visa, 8 hexadecimal digits for M/Chip 2, and 12 hexadecimal digits for M/Chip 4.

The Use Initial Card Verification Results and Initial Card Verification Results fields enable you to specify different predicted card verification results when the application transaction counter (ATC) is 0 or 1. You would only use these fields if the CVR is expected to be different for the initial use of a CAP authentication card. Otherwise, set the Use Initial Card Verification Results field to "Never".

The Initial Card Verification Results field contains the predicted Card Verification Results (CVR) of the application used for chip authentication when the application is used for the first time. The value in this field must be in hexadecimal digits and must be at least as long as implied by the issuer application data format: 8 hexadecimal digits for Visa, 8 hexadecimal digits for M/Chip 2, and 12 hexadecimal digits for M/Chip 4.

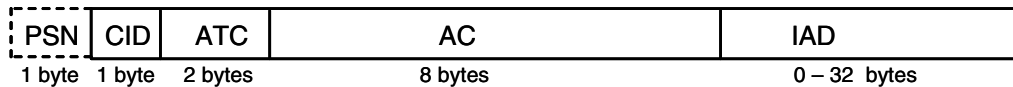
Chip authentication configuration

The fields in the Chip Authentication Configuration section specify parameters specific to CAP token validation in a security device. The Transaction Security Services process uses this data to format the command sent to the security device.

The Application field specifies whether this configuration supports the MasterCard (CAP) or Visa (DPA) CAP authentication programs.

The Internet Authentication Flags field specifies two hexadecimal characters that represent the internet authentication flag (IAF) bits used in the CAP token calculation by the personal card reader (PCR). Bits in the IAF indicate whether the transaction currency and amount are used in generating the CAP token by the PCR.

The Issuer Proprietary Bitmap field contains anywhere from 2 to 88 hexadecimal characters that represent the bits to be included (selected) in the compressed CAP token. When validating a CAP token, the security device selects a subset of the bits in the CAP token according to a supplied Issuer Proprietary Bitmap (IPB), and compares the selected bits to the generated partial AC. The number of hexadecimal characters in this field must be divisible by 2. The IPB is structured as follows:



Note: The PAN sequence number (PSN) is optional.

The Number of Resynch Attempts field indicates the number of attempts supported to automatically resynchronize the ATC upon CAP token validation failure. Currently, no attempt (0) or one or more attempts is supported. For each attempt, the process increases the value of the ATC by 2^n , where n is the number of ATC bits in the IPB. This field defaults to 0.

The HSM Specific Parameters field enables you enter free-format text specific to the type of security device you are using for CAP token validation. Currently, this field is only used to input the 4-byte IPB MAC value required in the K2 command for Thales e-Security HSMs. This MAC value is generated using the MI console command.

Card tab

The Card tab enables you to maintain the card verification key or key pairs used for Card Verification Value (CVV), Integrated Circuit Card Verification Value (iCVV), Card Validation Code (CVC), Integrated Circuit Card Validation Code (Chip CVC), Card Security Code (CSC) verification, and Integrated Circuit Card Security Code (iCSC).

Note: CSC verification is supported only when you are using an Atalla security device with AKB or variant (i.e., non-AKB) software, a Banksys DEP HSM, a Bull BNTng HSM, or a Thales e-Security HSM. It is not currently supported for SafeNet (Eracom) host security modules. iCSC verification is supported only when using an Atalla security device with AKB software or a Thales e-Security HSM.

An example of the Card tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

DES (IBM 3624) PIN Verification		Visa PVV Keys		Diebold	NCR
Identitykey	EMV	Chip Auth	Card	Contactless	Dual PIN PIN
Card Verification Algorithm: Visa/MasterCard CVV/CVC					
Key Pair	Key Part 1	Key Part 2	Key Part 3	Check Digits	
1					

The Verification Algorithm field specifies whether the card verification key or key pairs are used for verification of cards issued by Visa or MasterCard, or cards issued by American Express.

Note: The Verification Algorithm field is only displayed when you are using an Atalla NSP with variant software or a Thales e-Security HSM. This field is not displayed when you are using an Atalla NSP with AKB software, a Banksys DEP HSM, a Bull BNTng HSM, or a SafeNet (Eracom) host security module.

Atalla NSPs (with AKB or variant software), Banksys DEP HSMs, Bull BNTng HSMs, and Thales e-Security HSMs use a different algorithm for verifying the American Express card security code (CSC) from the algorithm used to verify either the Visa card verification value (CVV) or MasterCard card verification code (CVC). Two single-length keys (key pair) are used to verify a CVV or CVC. A single, double-length key is used to verify a CSC. The key entry fields displayed vary according to the value you select in the Verification Algorithm field. When the Verification Algorithm field is set to a value of AMEX CSC, you must enter a single double-length card verification key. When the Verification Algorithm field is set to a value of Visa/MasterCard CVV/CVC, you must enter two single-length card verification keys (key pair).

Although Atalla NSPs with AKB software, Banksys DEP HSMS, and Bull BNTng HSMS also use a different algorithm for verifying the American Express card security code (CSC) from the algorithm used to verify either the Visa card verification value (CVV) or MasterCard card verification code (CVC), the key fields displayed do not change for the different algorithms. Therefore, the Verification Algorithm field is not displayed when using Atalla NSPs with AKB software, Banksys DEP HSMS, or Bull BNTng HSMS.

Atalla NSPs with AKB software and Thales e-Security HSMS also use a different algorithm for verifying American Express integrated circuit card security codes (iCSCs). Use of this algorithm is controlled by the [AMEX_CSC_ALGO](#) Environment attribute.

Contactless tab

The Contactless tab enables you to maintain the dynamic card verification key used for Dynamic Card Verification Value (dCVV) and Card Verification Code 3 (CVC3) verification. This tab is only displayed for Atalla AKB and variant NSPs and Thales e-Security HSMs.

An example of the Contactless tab on the Authentication Profile Configuration window for a Thales e-Security HSM is shown below.

The Algorithm field indicates whether to perform MasterCard or Visa dynamic card verification.

Maintaining authentication profile security data

The following paragraphs describe procedures for displaying, adding, modifying, and deleting authentication profile security data from the Authentication Profile Configuration window. Data entered on the Authentication Profile Configuration window is stored in different files, depending on the tab from which the data is entered. The following table shows you the security data files that correspond to each tab.

Tab	File
DES (IBM 3624) PIN Verification	IBM DES Verification File (IBM_DES)
Visa PVV Keys	Visa PVV Verification File (Visa_PVV)
Diebold	Diebold Number Table File (DIEBOLD_NUM_TBL)
NCR	NCR PIN Verification File (NCR_PIN_VRFY)
Identikey	Identikey PIN Verification File (Identikey)
Card	Card Verification File (CARD_VRFCTN)
Contactless	Dynamic Card Verification File (DYN_CARD_VRFCTN)

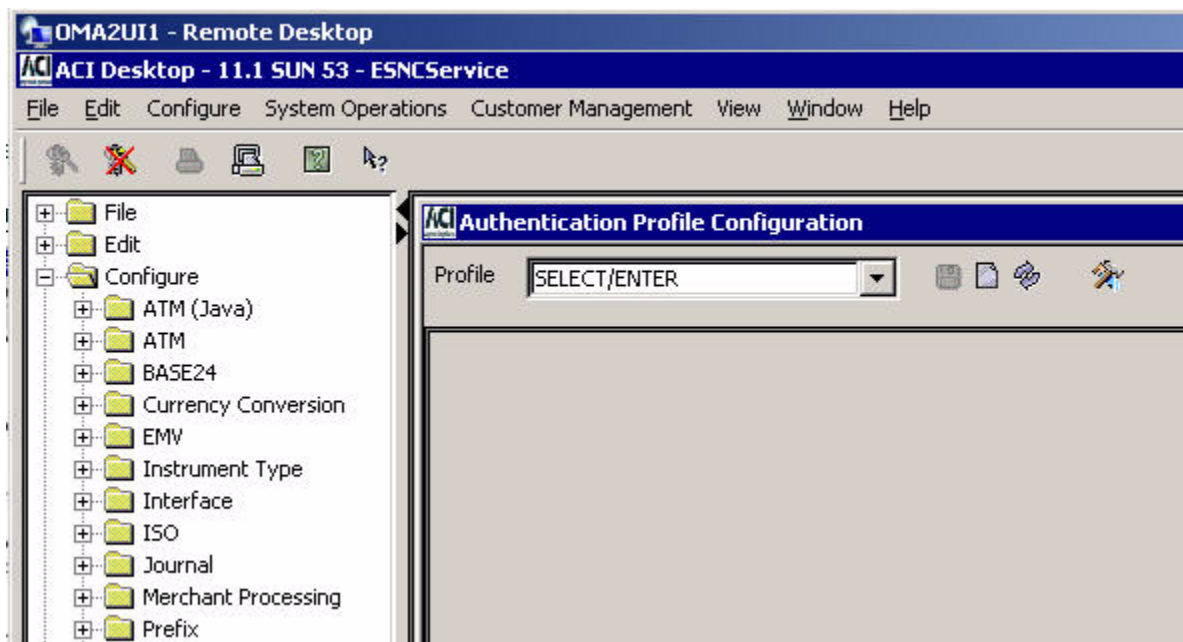
Tab	File
EMV	EMV Authorization File (EMV_Security)
Chip Auth	Chip Authentication File (Chip_Authentication)

Displaying an authentication profile

To display an authentication profile, perform the following steps:

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Authentication Profile Configuration window by accessing **Configure > Transaction Security > Authentication Profile**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Enter an authentication profile name in the Profile field and press the **Enter** key or select a name from the selectable list. The table below the field displays the various entries that make up the profile. The verification types that have been defined for the profile are indicated by a check mark in a box that is labeled with a verification type name, except for the Visa PVV, EMV, and Card columns. In the Visa PVV column, a value indicates whether Visa PVV PIN verification (Visa PVV), dual PVN PIN verification (Dual PVN), both (Both Visa PVV and Dual PVN tabs), or neither (None) have been defined in the profile. In the EMV column, a value indicates whether EMV validation, CAP token validation (Chip Auth), both (Both EMV and Chip Auth tabs), or neither (None) have been defined in the

profile. In the Card column, a value indicates whether card verification (CVD1/CVD2), contactless card verification (CVD3), both (Both Card and Contactless tabs), or neither (None) have been defined in the profile.

Authentication Profile Configuration - NEW_PROFILE

Profile: NEW_PROFILE

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
December 2011	December 2011	☐	None ☐	☐	☐	☐	None ☐	None ☐

- To display the configuration details for the verification types in an entry, select a row by clicking anywhere in the row. Only the tabs below the table for the verification types that have been configured for the row are active; the inactive tabs appear dimmed so that you cannot access them.

Authentication Profile Configuration - NEW_PROFILE

Profile: **NEW_PROFILE**

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
December 2011	December 2011	☐	None ☐	☐	☐	☐	EMV ☐	None ☐

DES (IBM 3624) PIN Verification				Visa PVV Keys		Diebold	NCR
Identikey	EMV	Chip Auth	Card	Contactless	Dual PVN PIN		
	EMV Scheme: Visa					Branch/Height Parameters: Branch factor 2; Tree Height...	
	Cryptogram Version Number: 0E					Destination PIN Block Type: Visa Format Without using...	
Secure Message Key for Integrity							
	Key Part 1	Key Part 2	Key Part 3	Check Digits			
1							
Secure Message Key for Confidentiality							
	Key Part 1	Key Part 2	Key Part 3	Check Digits			
1							
Cryptographic Keys							
	Key Index	Key Part 1	Key Part 2	Key Part 3	Check Digits		
1							

2 - Please enter all data needed for Profile NEW_PROFILE and push the Save button.

- Click the active tabs to display the configurations of the various verification types.

Adding a new authentication profile

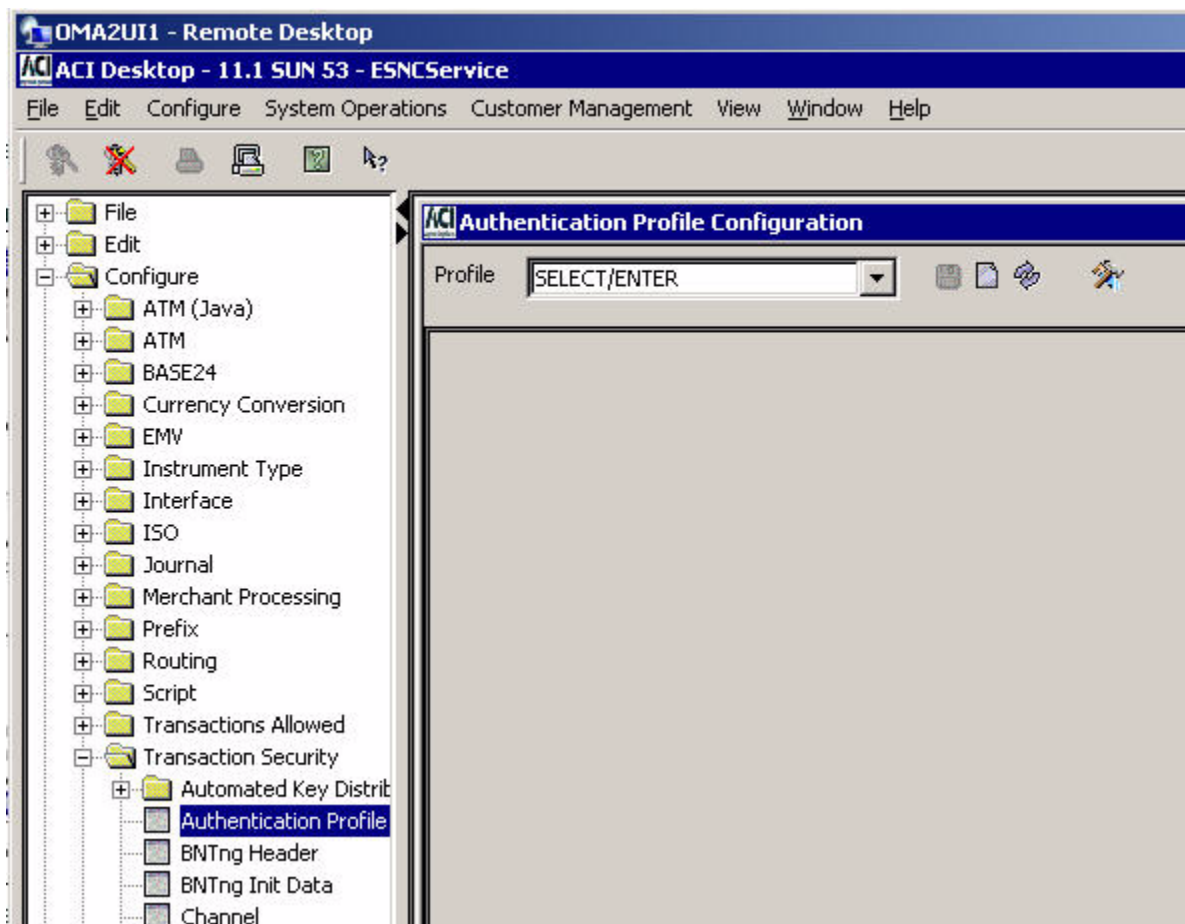
You can add a new authentication profile in two ways. You can create a new authentication profile from the very beginning or you can use an existing authentication profile as a *template* for creating a new profile. Procedures for both methods of adding a new authentication profile are provided below.

Creating a new authentication profile from the beginning

Use the following procedure to add a new authentication profile from the very beginning.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Authentication Profile Configuration window by accessing **Configure > Transaction Security > Authentication Profile**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



- Enter the name of the authentication profile to be added in the Profile field and press the **Enter** key. The main body of the window is displayed and you are prompted to enter all the data needed for the new profile as shown below.

Authentication Profile Configuration - NEWAUTHPROFILE

Profile: **NEWAUTHPROFILE**

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
December 2011	December 2011	☐	None ☐	☐	☐	☐	None ☐	None ☐

Identikey | EMV | Chip Auth | Card | Contactless | Dual PVN PIN
 DES (IBM 3624) PIN Verification | Visa PVV Keys | Diebold | NCR

Key Length: **Single**

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				

Decimalization Table:
 PAN Pad Character:
 PAN Verify Length:
 PAN Verify Offset:

2 - Please enter all data needed for Profile NEWAUTHPROFILE and push the Save button.

- Select the beginning date and ending date for the profile in the Begin Date and End Date fields. You can configure multiple date range entries for the authentication profile by inserting additional rows in the table near the top of the window. This step and all remaining steps in this procedure pertain to each row in this table.

These fields specify the range of card expiration dates to which this authentication profile applies. The expiration date in the transaction data is compared to the range of card expiration dates to determine which entry of the authentication profile to use, and therefore, which PIN verification, card verification, and EMV parameters to use. For example, if there are two authentication profile entries and one has a range of April 2002–2003, and another has a range of April 2003–2004, and a transaction contains an expiration date of September 2003, the application would use the second entry to authenticate the transaction.

Valid values are January 1000 through December 9999. You can change the month and year in this field by selecting either value and using the up and down arrows to change them.

5. Select the boxes or values for the verification types to be configured in this profile by clicking in the appropriate boxes or selecting a value from a selectable list. You must select at least one type of verification for the profile. The tabs for the verification types selected are activated.

Authentication Profile Configuration - NEWAUTHPROFILE

Profile: **NEWAUTHPROFILE**

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
December 2011	December 2011	☒	None ☐	☐	☐	☐	None ☐	Both C... ☐

DES (IBM 3624) PIN Verification

Identikey | EMV | Chip Auth | **Card** | Visa PVV Keys | Diebold | NCR

Card Verification Algorithm: **Visa/MasterCard CVV/CVC**

Key Pair	Key Part 1	Key Part 2	Key Part 3	Check Digits
1				

2 - Please enter all data needed for Profile NEWAUTHPROFILE and push the Save button.

6. Enter the data required for each type of verification selected for this authentication profile on the active tabs and click Save ().

Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields for each type of verification.

7. If the authentication profile information is successfully saved, a message similar to the following is displayed at the bottom of the window.

9 - 09:41 AM > Successful insert of Profile NEWAUTHPROF April 2003 - December 2005 .

8. Rebuild the external memory tables (OLTPs) for the verification types included in the new authentication profile. A separate external memory table (OLTP) is maintained for each tab on the Authentication Profile Configuration window. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for procedures.
9. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. Refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for procedures.
10. Log on to the BASE24 user access system and access the Card Prefix File (CPF) screens. Add or update the appropriate fields as shown in the following table with the new authentication profile value. These fields contain the key locator values used by the Transaction Security Services process to access authentication profile information.

The following table shows you the fields in the Card Prefix File (CPF) that are used to access the appropriate records in the TSS database. The value in these fields must match a value entered in the Profile field on the Authentication Profile Configuration window.

CPF field	TSS file
PIN VERIFICATION KEYA GROUP (CPF screen 2)	IBM_DES, DIEBOLD_NUM_TBL, Identikey, NCR_PIN_VRFY, Visa_PVV
KEYI GROUP (CPF screen 11)	EMV_Security
CAP GROUP (CPF screen 13)	Chip_Authentication
CV KEYA GROUP (CPF screen 2)	CARD_VRFCTN
MANUAL CV KEYA GROUP (CPF screen 2)	
SIV KEYA GROUP (CPF screen 6)	
DYN CARD VERIF KEY LOCATOR (CPF screen 3)	DYN_CARD_VRFCTN
NEW PIN TRANSPORT KEY LOCATOR (CPF screen 11)	ICHG_SECURITY

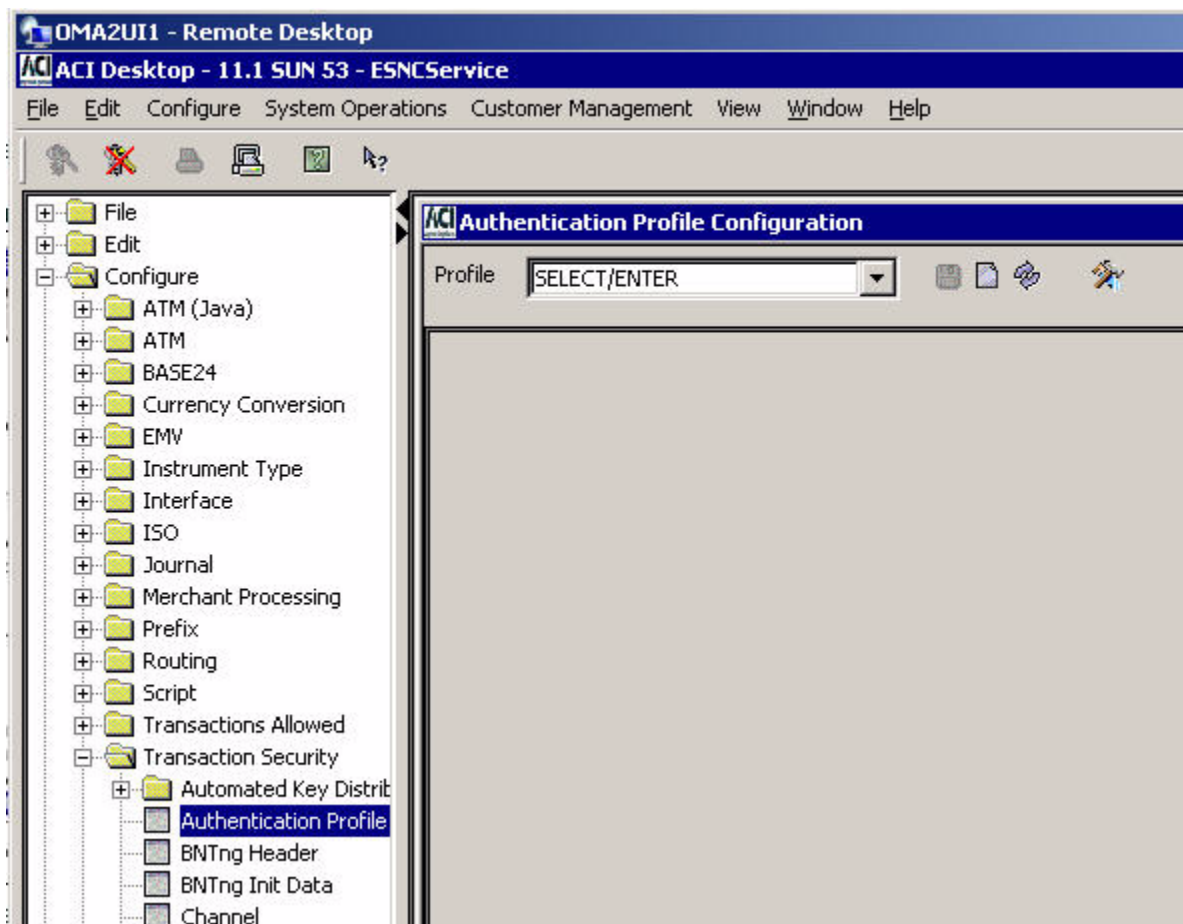
The PIN VERIFICATION KEYA GROUP field specifies the authentication profile to use to retrieve PIN verification method information. The CV KEYA GROUP, MANUAL CV KEYA GROUP, DYN CARD VERIF KEY LOCATOR, and SIV KEYA GROUP fields specify the authentication profile to use to retrieve card or dynamic card verification keys. The KEYI GROUP and CAP GROUP fields specify the authentication profile from which EMV or CAP token validation processing information is retrieved. The NEW PIN TRANSPORT KEY LOCATOR field specifies a host interface for notifying a card management system of offline PIN changes.

Adding a new authentication profile from a template

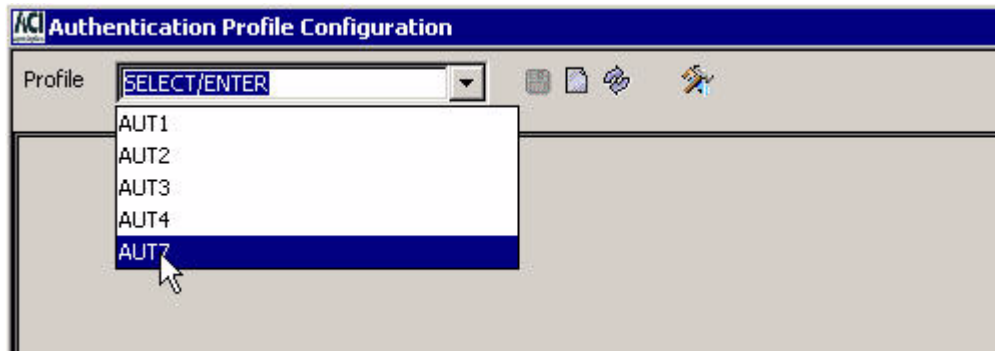
Use the following procedure to create a new authentication profile by using an existing authentication profile as a *template*.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Authentication Profile Configuration window by accessing **Configure > Transaction Security > Authentication Profile**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

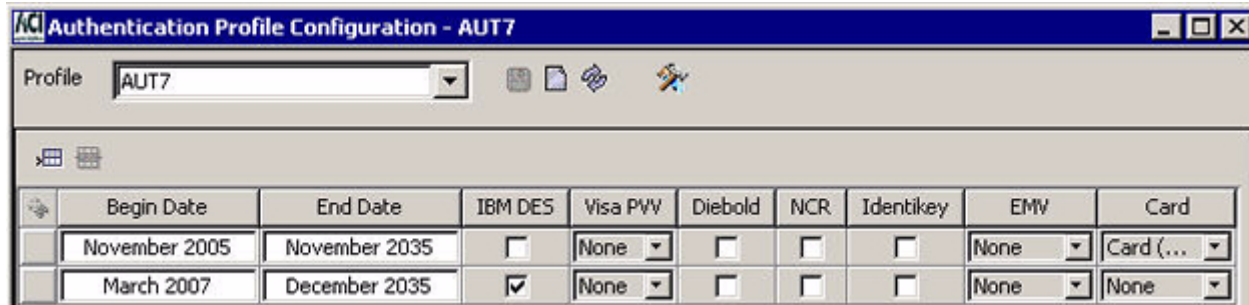
Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Enter the name of the authentication profile you want to use as a template in the Profile field and press the **Enter** key or select the profile from the selectable list.



4. All the date range entries for the profile are displayed. In this example, two date ranges have been specified for the AUT7 authentication profile. One date range includes card verification only while the other includes IBM DES PIN verification only.



5. Click anywhere in a date range row to retrieve the configuration data for that data range.

Authentication Profile Configuration - AUT7

Profile: **AUT7**

	Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
	November 2005	November 2035	☐	None	☐	☐	☐	None	Card (...)
	March 2007	December 2035	☒	None	☐	☐	☐	None	None

Identikey | EMV | Chip Auth | Card | Contactless | Dual P/VN PIN

DES (IBM 3624) PIN Verification | Visa PVV Keys | Diebold | NCR

Key Length: **Single**

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1	FCBA 7CF5 972C F0DD			

Decimalization Table: **0123456789012345**

PAN Pad Character: **F**

PAN Verify Length: **9**

PAN Verify Offset: **3**

12 - Successful view of Profile AUT7 March 2007 - December 2035.

- Change the name in the Profile field to the name of the new profile. Enter the beginning and ending dates for the date range of this new profile and re-select the verification types to be included in the profile. Verification data on the tabs that were active for the profile being used as a template is still displayed.

Authentication Profile Configuration - NEWAUTHPROF

Profile: **NEWAUTHPROF**

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikkey	EMV	Card
January 2012	December 2013	☒	None ☐	☐	☐	☐	None ☐	None ☐

DES (IBM 3624) PIN Verification

Key Length: **Single**

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1	FCBA 7CF5 972C F0DD			

Decimalization Table: **0123456789012345**

PAN Pad Character: **F**

PAN Verify Length: **9**

PAN Verify Offset: **3**

14 - Please enter all data needed for Profile NEWAUTHPROF and push the Save button.

- Click Save (), and answer Yes to the “Are you sure you want to save” dialog box.

Authentication Profile Configuration - NEWAUTHPROF

Are you sure you want to save Profile NEWAUTHPROF?

Yes **No**

If the new profile is successfully saved, a message similar to the following is displayed at the bottom of the window.



26 - Successful insert of Profile NEWAUTHPROF October 2009 - October 2010.

8. Rebuild the external memory tables (OLTPs) for the verification types included in the new authentication profile. A separate external memory table (OLTP) is maintained for each tab on the Authentication Profile Configuration window. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for procedures.
9. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. Refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for procedures.
10. Log on to the BASE24 user access system and access the Card Prefix File (CPF) screens. Add or update the appropriate fields as shown in the following table with the new authentication profile value. These fields contain the key locator values used by the Transaction Security Services process to access authentication profile information.

The following table shows you the fields in the Card Prefix File (CPF) that are used to access the appropriate records in the TSS database. The value in these fields must match a value entered in the Profile field on the Authentication Profile Configuration window.

CPF field	TSS file
PIN VERIFICATION KEYA GROUP (CPF screen 2)	IBM_DES, DIEBOLD_NUM_TBL, Identkey, NCR_PIN_VRFY, Visa_PVV
CV KEYA GROUP (CPF screen 2)	CARD_VRFCTN
MANUAL CV KEYA GROUP (CPF screen 2)	
SIV KEYA GROUP (CPF screen 6)	
DYN CARD VERIF KEY LOCATOR (CPF screen 3)	DYN_CARD_VRFCTN
KEYI GROUP (CPF screen 11)	EMV_Security
NEW PIN TRANSPORT KEY LOCATOR (CPF screen 11)	ICHG_SECURITY

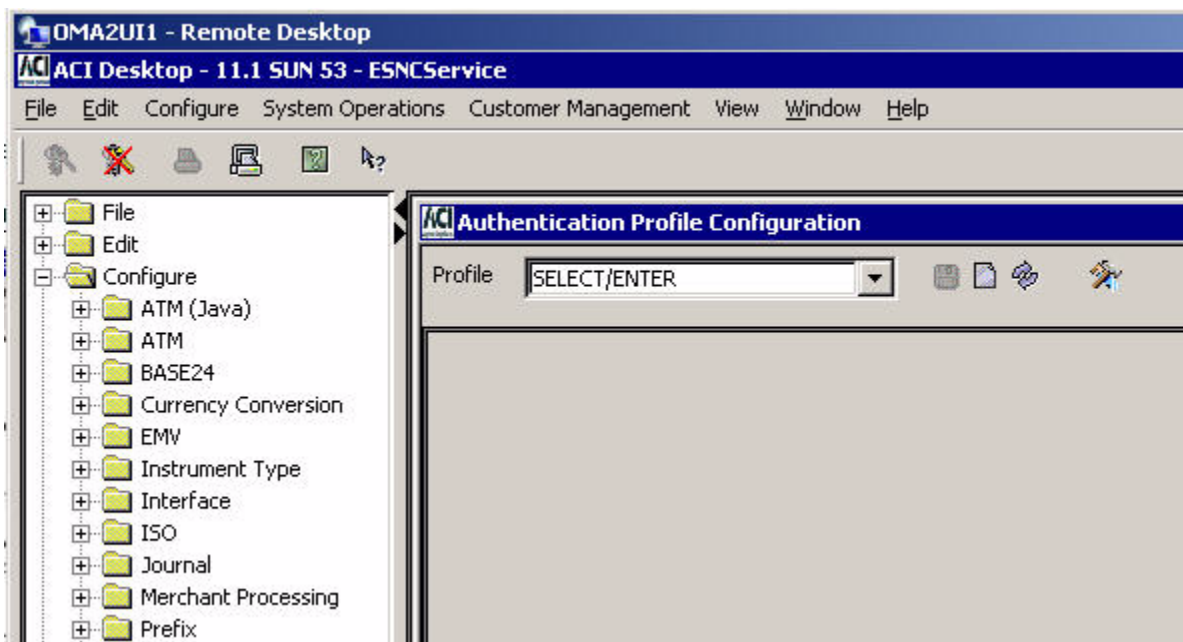
The PIN VERIFICATION KEYA GROUP field specifies the authentication profile to use to retrieve PIN verification method information. The CV KEYA GROUP, MANUAL CV KEYA GROUP, DYN CARD VERIF KEY LOCATOR, and SIV KEYA GROUP fields specify the authentication profile to use to retrieve card or dynamic card verification keys. The KEYI GROUP and CAP GROUP fields specify the authentication profile from which EMV or CAP token validation processing information is retrieved. The NEW PIN TRANSPORT KEY LOCATOR field specifies a host interface for notifying a card management system of offline PIN changes.

Modifying an existing authentication profile

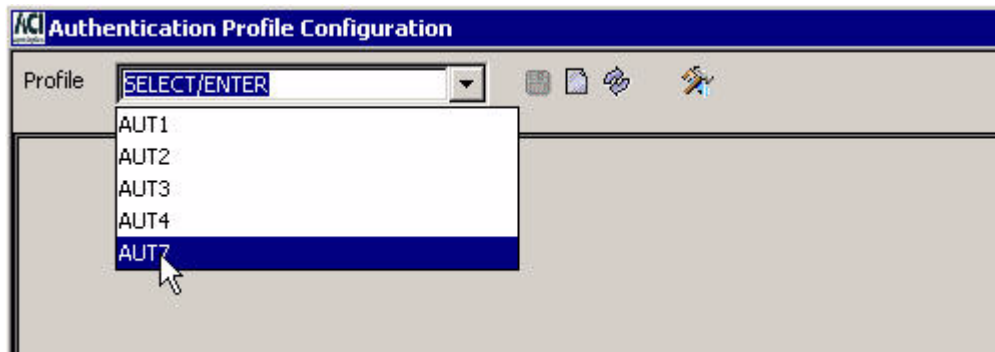
Use the following procedure to make changes to an existing authentication profile.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Authentication Profile Configuration window by accessing **Configure > Transaction Security > Authentication Profile**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



3. Enter the authentication profile you want to change in the Profile field and press the **Enter** key or select the profile from the selectable list.



4. All the date range entries for the profile are displayed. In this example, two date ranges have been specified for the AUT7 profile. This profile includes IBM DES PIN verification only for one date range and card verification only for the other date range.

The screenshot shows the 'Authentication Profile Configuration - AUT7' window. The 'Profile' dropdown is set to 'AUT7'. Below the profile name is a table with two rows representing date ranges and their associated verification methods.

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
November 2005	November 2035	☐	None	☐	☐	☐	None	Card (...)
March 2007	December 2035	☒	None	☐	☐	☐	None	None

5. Click anywhere in a date range row to retrieve the configuration data for that data range.

The screenshot shows the 'Authentication Profile Configuration - AUT7' window with the 'March 2007' to 'December 2035' date range selected. The configuration details for this range are displayed below the table.

Identikey	EMV	Chip Auth	Card	Contactless	Dual PVN PIN
DES (IBM 3624) PIN Verification	Visa PVV Keys	Diebold	NCR		

Key Length:

	Key Part 1	Key Part 2	Key Part 3	Check Digits
1	FCBA 7CF5 972C F0DD			

Decimalization Table: PAN Verify Length:

PAN Pad Character: PAN Verify Offset:

12 - Successful view of Profile AUT7 March 2007 - December 2035.

- Update the existing data for the profile on activated tabs as needed. If you need to add an additional verification type to the profile, select the verification type in the date range row and enter the necessary data on the activated tab.

Refer to the procedures and field descriptions in ACI Online Help for detailed information on the required and optional fields for each type of verification.

In this example, Identikey PIN verification data is being added to the AUT7 authentication profile.

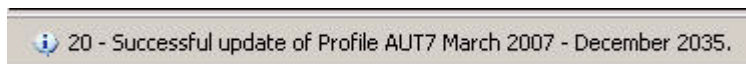
Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identikey	EMV	Card
November 2005	November 2035	☐	None	☐	☐	☐	None	Card (...)
March 2007	December 2035	☒	None	☐	☐	☒	None	None

DES (IBM 3624) PIN Verification				Visa PVV Keys		Diebold	NCR
Identikey	EMV	Chip Auth	Card	Contactless	Dual PVN PIN		
Bank ID: 123456 PAN Verify Offset: 2 PAN Verify Length: 4 Compare Indicator: Right							

- Click Save (), and answer Yes to the “Are you sure you want to save” dialog box.



- If the authentication profile information is successfully updated, a message similar to the following is displayed at the bottom of the window.



- Rebuild the external memory tables (OLTPs) for the verification types in the updated authentication profile. A separate external memory table (OLTP) is maintained for each tab on the Authentication Profile Configuration window. Refer to the [“Rebuilding external memory tables \(OLTPs\)”](#) topic provided later in this section for procedures.

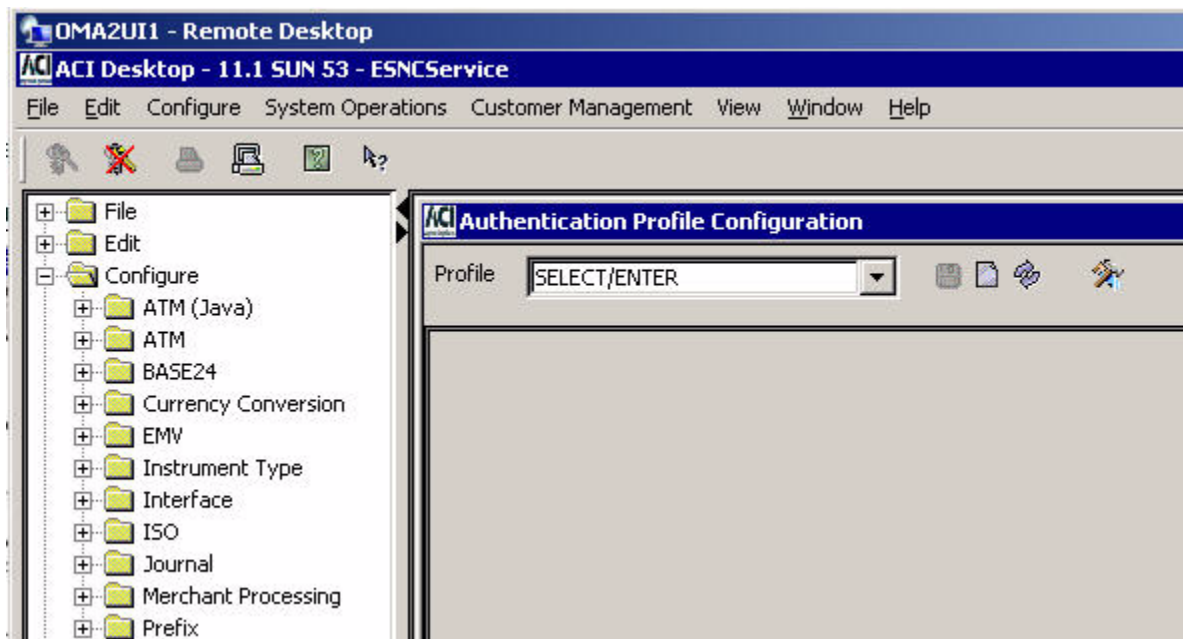
10. Warmboot the Transaction Security Services process to begin using the new external memory table (OLTP) data in processing. Refer to the [“Warmbooting the Transaction Security Services process”](#) topic provided later in this section for procedures.

Deleting an authentication profile

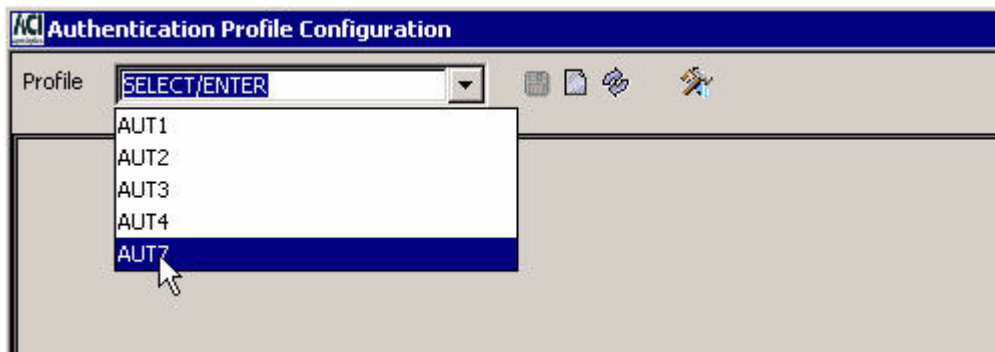
Use the following procedure to delete an authentication profile.

1. Log on to the ACI desktop user interface. Refer to the **ACI Desktop User Interface Manual** for instructions on logging on.
2. Display the Authentication Profile Configuration window by accessing **Configure > Transaction Security > Authentication Profile**. You can use the Configure drop-down menu at the top of the user interface or the navigation pane on the left to access this window.

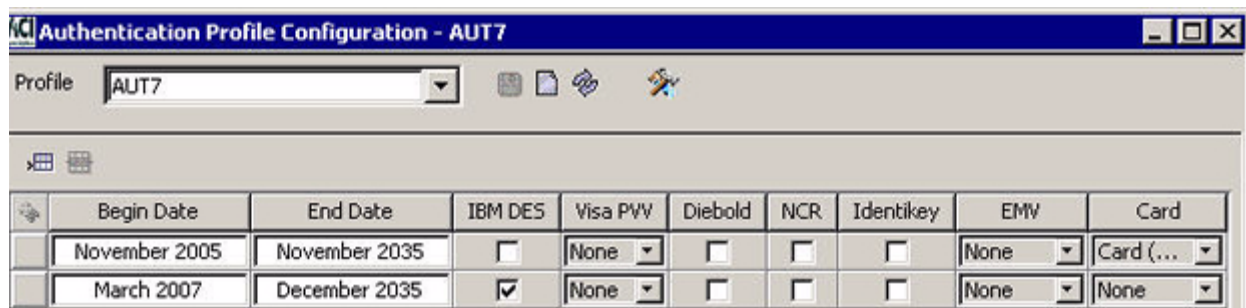
Note: The navigation pane in the following example shows all the Configure windows available in BASE24-eps. For BASE24 TSS, only the Transaction Security, BASE24, and User Security folders are available under Configure.



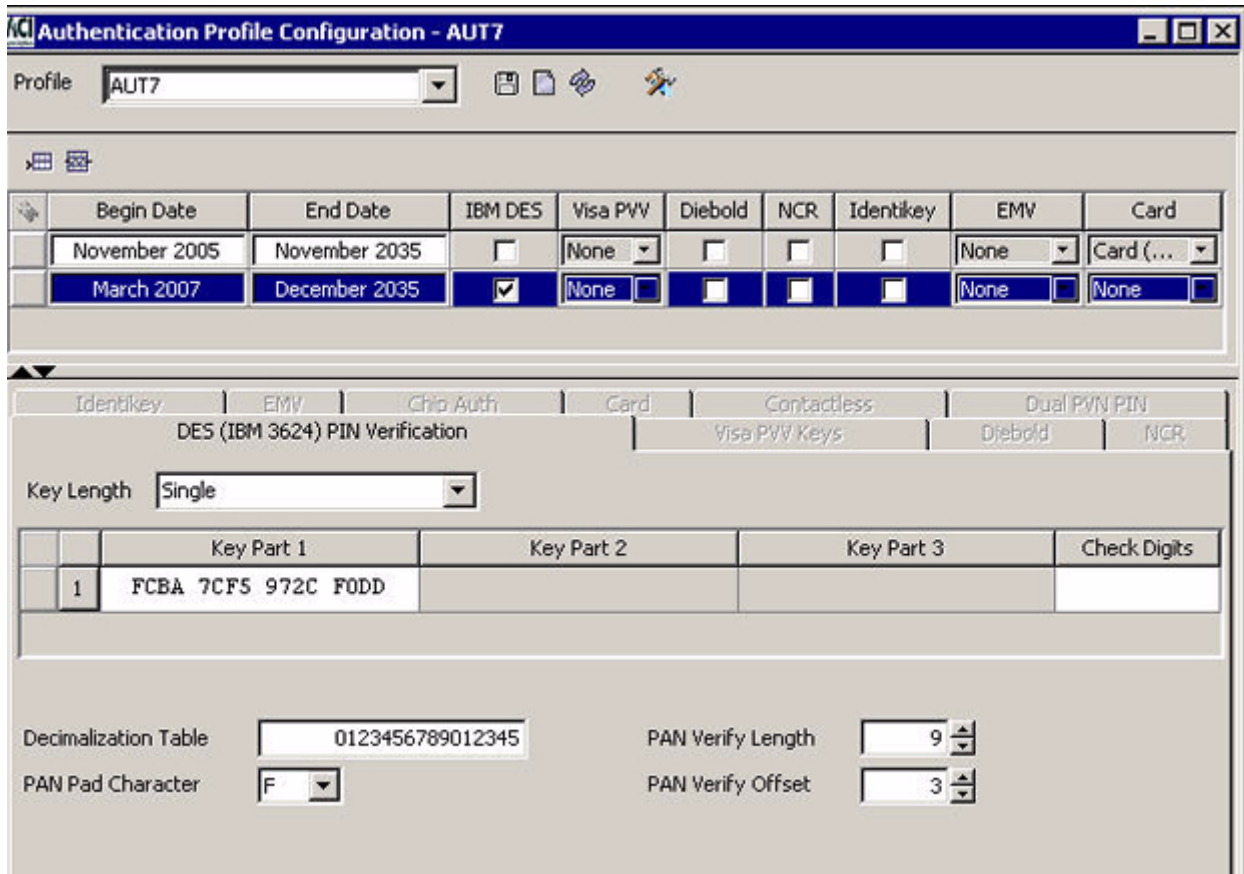
3. Enter the name of the authentication profile you want to delete in the Profile field and press the **Enter** key or select the profile from the selectable list.



4. All the date range entries for the profile are displayed. In this example, two date ranges have been specified for the AUT7 authentication profile. This profile includes IBM DES PIN verification only for one date range and card verification only for the other date range.



5. Click anywhere in a date range row to select the configuration data for that data range.



Authentication Profile Configuration - AUT7

Profile: AUT7

Begin Date	End Date	IBM DES	Visa PVV	Diebold	NCR	Identkey	EMV	Card
November 2005	November 2035	☐	None	☐	☐	☐	None	Card (...)
March 2007	December 2035	☒	None	☐	☐	☐	None	None

DES (IBM 3624) PIN Verification

Key Length: Single

Key Part 1	Key Part 2	Key Part 3	Check Digits
1 FCBA 7CF5 972C F0DD			

Decimalization Table: 0123456789012345

PAN Pad Character: F

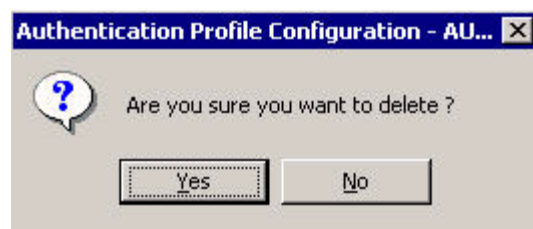
PAN Verify Length: 9

PAN Verify Offset: 3

6. Click Delete Row (). The date range row is now marked for deletion.



7. Click Save (), and answer Yes to the "Are you sure you want to delete" dialog box.



Authentication Profile Configuration - AU...

Are you sure you want to delete ?

Yes No

If the profile is successfully deleted, a message similar to the following is displayed at the bottom of the window.

22 - Successful delete of Profile AUT7 200703 203512.

Key variant fields

The Key Variant fields on various tabs of the Channel Security Configuration window and the Interface Security Configuration window specify the key variant to be applied to the PIN, MAC, or data encryption key exchange key (KEK) when translating PIN, MAC, or data encryption keys.

For example, if the working key provided by an external processor has been encrypted with the KEK using a non-Atalla security device and the BASE24 system uses Atalla security devices, a variant must be used. The variants available for each type of working key are described below.

PIN working key variants

The following variants are available in the Key Variant field for PIN key exchange keys.

Key variant	Description
0 - Key Exchange Key Variant	Translate the PIN key directly from the PIN KEK parts.
1 - PIN Encryption Key Variant	Include the variant 1 constant with the data encryption KEK parts when translating the data encryption key. The Atalla variant 1 constant is hexadecimal 0800 0000 0000 0000. This value is displayed only when Atalla security devices are used in your system.
5 - ATM Key Variant ¹	Include the variant 5 constant with the PIN KEK parts when translating the PIN key. The Atalla 5 constant is hexadecimal 2800 0000 0000 0000.

¹ This variant is only available on the Channel Security Configuration window.

MAC working key variants

The following variants are available in the Key Variant field for MAC key exchange keys.

Key variant	Description
0 - Key Exchange Key Variant	Translate the MAC key directly from the MAC KEK parts.
3 - MAC Encryption Key Variant	Include the variant 3 constant with the MAC KEK parts when translating the MAC key. The Atalla 3 constant is hexadecimal 1800 0000 0000 0000. This value is displayed only when Atalla security devices are used in your system.

Key variant	Description
5 - ATM Key Variant ¹	Include the variant 5 constant with the MAC KEK parts when translating the MAC key. The Atalla 5 constant is hexadecimal 2800 0000 0000 0000.

¹ This variant is only available on the Channel Security Configuration window.

Data encryption working key variants

The following variants are available in the Key Variant field for data encryption key exchange keys.

Key variant	Description
0 - Key Exchange Key Variant	Translate the data encryption key directly from the data encryption KEK parts.
2 - Data Encryption Key Variant	Include the variant 2 constant with the data encryption KEK parts when translating the data encryption key. The Atalla 2 constant is hexadecimal 1000 0000 0000 0000. This value is displayed only when Atalla security devices are used in your system.

External memory tables (OLTPs)

Several of the files maintained by the Transaction Security Services process are accessed as external memory tables during online transaction processing (OLTP). Because external memory tables are accessed during online transaction processing, they are often referred to as OLTPs. These external memory tables (OLTPs) are similar to extended memory tables in the BASE24 processing environment. As with extended memory tables, external memory tables (OLTPs) must be built or rebuilt when the underlying file is changed (through an add, update, or delete) and the application process that accesses the external memory table (i.e., the Transaction Security Services process) must be warmbooted before it starts using the new external memory table. OLTPs are rebuilt by entering Deliver Data Source Loader commands (also referred to as OLTP Rebuild commands). Processes are warmbooted to start using a rebuilt external memory table (OLTP) by entering Alter Data Source commands (also referred to as OLTP Warmboot commands). The following files for the Transaction Security Services process are accessed as external memory tables (OLTPs):

- BNTng Header File (BNTng_Header)
- Card Verification File (CARD_VRFCTN)
- Chip Authentication File (Chip_Authentication)

- Derivation Key File (Derivation_Key)
- Diebold Number Table File (DIEBOLD_NUM_TBL)
- Dynamic Card Verification File (DYN_CARD_VRFCTN)
- EMV Authorization File (EMV_Security)
- Event File (Event)¹
- IBM DES Verification File (IBM_DES)
- Identikey PIN Verification File (Identikey)
- Interface Security File (ICHG_SECURITY)²
- NCR PIN Verification File (NCR_PIN_VRFY)
- Visa PVV Verification File (Visa_PVV)

¹ Use of the Event_OLTP is optional. If the Event_OLTP is built and warmbooted, the Event component retrieves data from the OLTP. If the Event_OLTP is not available, it retrieves data from the Event File on disk. Data retrieval performance is better using the Event_OLTP, however, its use requires you to consume more disk space.

² Data for an interface in the ICHG_SECURITY is accessed from an external memory table (i.e., ISECOD) only when the interface does not support dynamic key management. This is indicated by a value of “No keys exchanged” in the Options field on the Exchange Information tab of the Interface Security Configuration window. Otherwise, data for an interface is accessed from an internal memory table (or the ICHG_SECURITY disk file).

When you change values on the Interface Security Configuration window for an interface that does not exchange keys, you must rebuild the Interface external memory table (OLTP) and warmboot, or stop and restart, the Transaction Security Services process to reread the Interface external memory table (OLTP). The new values will be used in processing for the next transaction for the interface that requires security key data.

When you change key values on the Interface Security Configuration window for an interface that does exchange keys, you do not have to warmboot the Transaction Security Services process to reread the ICHG_SECURITY, or stop and restart the Transaction Security Services process. The Transaction Security Services process reads the new key values from the ICHG_SECURITY disk file for the next transaction for the interface that requires security key data if it encounters a key synchronization error using the key in the internal memory table or if the key is an outbound key. If, however, you add or delete a record for an interface that does exchange keys, or if you change the value of the Options field on the Exchange Information tab for an interface to start or stop exchanging keys, you must rebuild the Interface extended memory table before warmbooting the Transaction Security Services process to reread the ICHG_SECURITY or stopping and restarting the Transaction Security Services process. As an alternative to changing key exchange key values from the ACI desktop user interface, you can change them by entering a KEY CHANGE text command from a network control facility. In this case, the changed key data is automatically reread from the ICHG_SECURITY disk file and there is no need to warmboot the Transaction Security Services process to reread the ICHG_SECURITY, or to stop and restart the Transaction Security Services process.

Refer to the “[Internal memory tables](#)” topic for more information on warmbooting the Transaction Security Services process for the ICHG_SECURITY internal memory table.

A separate file assign is required for each file that is accessed as an external memory table (OLTP). These file assigns specify the location of *swap* files used when loading the file data into external memory. By design, these assigns always have `_OLTP` appended to the assign name and cannot specify the same file location as the file assigns for the associated disk files from which the external memory tables (OLTPs) are built. Also, ACI inserts the letter O into the swap file names specified by these `_OLTP` assigns. For example, the external memory table (OLTP) swap file for the `ICHG_SECURITY` file is `ISECOD`.

All of these files are maintained from the Authentication Profile Configuration window, except for the `ICHG_SECURITY`, which is maintained from the Interface Security Configuration window. You can rebuild external memory tables (OLTPs) and warmboot the Transaction Security Services process to reread the `ICHG_SECURITY` from the Process Control or OLTP Warmboot Management windows on the ACI desktop user interface or from an XPNET network control facility as described below.

Rebuilding external memory tables (OLTPs)

You can rebuild external memory tables (OLTPs) in the TSS environment in the following three ways:

- Enter a DELIVER PROCESS command from an XPNET network control facility.
- Enter a Deliver Data Source command (also referred to as the OLTP Rebuild command) from the Process Control window on the ACI desktop user interface.
- Enter an OLTP Rebuild command from the OLTP Warmboot Management window on the ACI desktop user interface.

Procedures for the first two methods of rebuilding external memory tables (OLTPs) are provided below. Procedures for using the OLTP Warmboot Management window to rebuild an external memory table (OLTP) are provided in [section 11](#). Commands to rebuild external memory tables (OLTPs) should be sent to the User Interface (XML Server) process instead of a Transaction Security Services process to avoid any impact to online transaction processing performance.

XPNET network control facility

To rebuild an external memory table (OLTP) from an XPNET network control facility, use the following command.

Syntax

DELIVER PROCESS *object-name*, "DELIVER DSLDR *oltp-assign-name*"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

oltp-assign-name — The assign name specified for the external memory table (OLTP) in the CONFCSV on the data subvolume. The following table lists the assign names for each of the files accessed as external memory tables (OLTPs).

Assign name	TSS file
BNTNG_HEADER_OLTP	BNTng Header File (BNTHOD)
CARD_VERIFICATION_OLTP	Card Verification File (CRDVOD)
CHIP_AUTHENTICATION_OLTP	Chip Authentication File (CAOD)
DERIVATION_KEY_OLTP	Derivation Key File (DRVKOD)
DIEBOLD_NUMBER_TABLE_OLTP	Diebold Number Table File (DNTOD)
DYNAMIC_CARD_VERIFICATION_OLTP	Dynamic Card Verification File (DCRDOD)

Assign name	TSS file
EMV_SECURITY_OLTP	EMV Authorization File (EMVSOD)
EVENT_OLTP	Event File (EVTOD)
IBM_DES_OLTP	IBM DES Verification File (IDESOD)
IDENTIKEY_OLTP	Identikey PIN Verification File (IDNTOD)
INTERCHANGE_SECURITY_OLTP	Interface Security File (ISECOD)
NCR_PIN_VERIFICATION_OLTP	NCR PIN Verification File (NCROD)
VISA_PVV_OLTP	Visa PVV Verification File (VPVOD)

Example

The following is an example of a DELIVER PROCESS command sent to the User Interface process to rebuild the external memory table (OLTP) for the Visa PVV Verification File.

```
deliver process p1a^xmll01,"deliver dsldr VISA_PVV_OLTP"
```

Process Control window

To rebuild an external memory table (OLTP) from the Process Control window on the ACI desktop user interface, perform the following steps:

1. After logging on, display the Process Control window by selecting System Operations and then Process Control.
2. In the Destination field, select XMLS from the list of values, or enter the symbolic name of the User Interface process.
3. In the Command field, select Deliver from the list of values. The XPNET process sends Deliver commands to a single process.
4. In the Component field, select Data Source Loader from the list of values.
5. In the Command Text field, enter the name of the file to be loaded into memory. This is the assign name specified for the external memory table (OLTP) in the CONFCSV on the data subvolume. The following table lists the assign names for each of the files accessed as external memory tables (OLTPs).

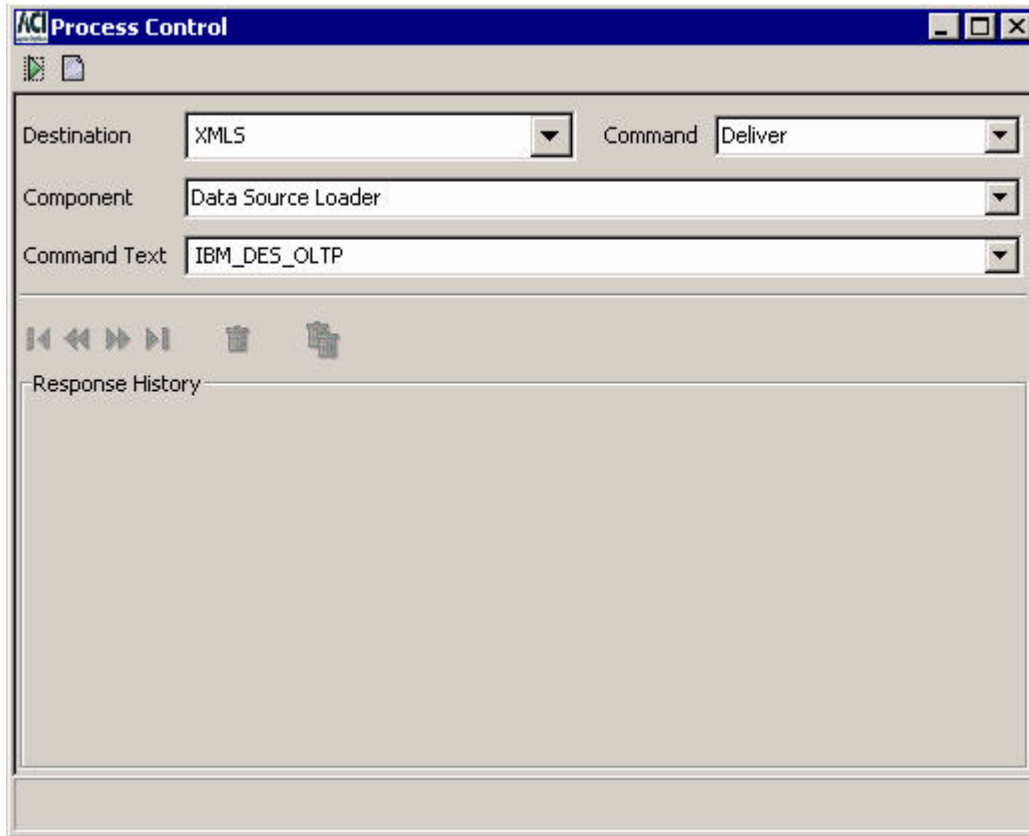
Assign name	TSS file
BNTNG_HEADER_OLTP	BNTng Header File (BNTHOD)
CARD_VERIFICATION_OLTP	Card Verification File (CRDVOD)
CHIP_AUTHENTICATION_OLTP	Chip Authentication File (CAOD)

Assign name	TSS file
DERIVATION_KEY_OLTP	Derivation Key File (DRVKOD)
DIEBOLD_NUMBER_TABLE_OLTP	Diebold Number Table File (DNTOD)
DYNAMIC_CARD_VERIFICATION_OLTP	Dynamic Card Verification File (DCRDOD)
EMV_SECURITY_OLTP	EMV Authorization File (EMVSOD)
EVENT_OLTP	Event File (EVTOD)
IBM_DES_OLTP	IBM DES Verification File (IDESOD)
IDENTIKEY_OLTP	Identikey PIN Verification File (IDNTOD)
INTERCHANGE_SECURITY_OLTP	Interface Security File (ISECOD)
NCR_PIN_VERIFICATION_OLTP	NCR PIN Verification File (NCROD)
VISA_PVV_OLTP	Visa PVV Verification File (VPVVOD)

6. Click Submit () at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window. If the table was successfully rebuilt, an event message similar to the following will be displayed on the EMS log.

```
MM-DD-YY;HH:MM:SS.HHH \SYS.$B4XM  ACI.1033.1000      60
Mgr PPD:\SYS.$BB4AN Generator: P1A^XML01
DSLDR: File IBM_DES_OLTP successfully loaded.
```

An example of the Process Control window is shown below.



External memory table (OLTP) size

If the build of an external memory table (OLTP) fails, it's possible that the memory size allowed for the table is not big enough. If so, you can increase the size allotted for the external memory table (OLTP) in its assign configuration (e.g., "-MT_SIZE=500000") by editing the Metadata Configuration File (CONFCSV) on the data subvolume. See the ["Calculating maximum hash table memory requirements"](#) topic below for detailed information on calculating the size for hash tables accessed in external memory.

Note: Any time you change the CONFCSV, you must stop and restart the process that uses the changed data. The change does not take effect until after the process is stopped and restarted. In addition, the process should be shut down in an orderly fashion so that any transactions currently in progress are not lost.

Calculating maximum hash table memory requirements

Currently, all external memory tables (OLTPs) used by the Transaction Security Services process are [hash tables](#). When determining the amount of memory required for a hash table, you must consider the following three data areas required to build a hash table:

- The hash table header, which is always a fixed length of 4184 bytes.
- The data records and hash data record headers. The space required for data records depends on the [aligned length of each record](#) in the file and the total number of records. The hash data record header is a fixed length of 22 bytes.
- The bucket space, which depends on the total number of records in the file and the [hash algorithm](#) selected to build the hash table. The size of each [hash bucket](#) is a fixed length of 22 bytes.

Calculating the size required for the data records and data record headers involves multiplying the total number of records in the file by the length of each record, including the hash data record header.

Calculating the bucket space required is more problematic because the Data Source Loader component attempts 108 different algorithms to determine the most efficient hash algorithm. The most efficient algorithm is the one eventually used but the component requires space to attempt all 108 algorithms, one at a time. Some algorithms require more space than others and the most efficient algorithm does not necessarily conserve the most space.

The most important function of the hash algorithm is to eliminate key [collisions](#) when calculating hash keys to optimize the speed of data retrieval. Keys are distributed across buckets. Therefore the best algorithm results in the least number of keys in a bucket to eliminate the additional overhead required for searching a chain of keys that hash to the same bucket (when searching for an exact match to a record).

Modulo divide hash algorithm

The modulo divide hash algorithms requires the number of buckets to be a prime number greater than or equal to the number of hash records times the bucket factor. The maximum bucket factor is always 6. Therefore, this algorithm has the potential to allocate a number of buckets equal to the next prime number greater than or equal to the total number of data records in the file multiplied by a factor of 6.

Multiplicative hash algorithm

For the multiplicative hash algorithm, the number of buckets need only be greater than or equal to the number of hash records times the maximum bucket factor of 6. For example, if there are 10,000 data records in a file, the multiplicative hash algorithm requires 10,000 or more buckets times 6, or 60,000 or more buckets.

Mid-square hash algorithm

For the mid-square hash algorithm, the number of buckets must be a power of 2 (i.e., 2, 4, 8, 16, 32, 64, 128, 256, etc.), greater than or equal to the number of hash records times the maximum bucket factor of 6. For example, if there are 50 data records in a file, the mid-square hash algorithm requires a power of two greater than or equal to 50 times 6 buckets, or 300 buckets. The next power of 2 greater than or equal to 300 is 512.

Data alignment

Although the total number of data records is multiplied by 6, the data itself is only loaded once. However, records need to be aligned on an 8-byte boundary. Therefore, you must add enough bytes to the record length so that each record starts on the proper boundary. This length is referred to as the aligned record length. Structures are adjusted to be on 4-byte boundaries.

Example calculation

The following is an example for calculating the maximum space required for a hash table using the modulo divide hash algorithm.

Hash table header:	4184 bytes
Hash data record header:	24 bytes (the actual definition is 22 bytes, but the next 8-byte boundary is 24 bytes)
Data records:	nn bytes (total number of data records times the aligned record length)
Hash bucket size:	24 bytes (the actual definition is 22 bytes, but the next 8-byte boundary is 24 bytes)

Suppose the following for the source file to be hashed:

Record length:	121 bytes
Number of records:	2031

In this case, the aligned record length is 128 bytes because the record length of 121 bytes does not fall on an 8-byte boundary. You must add 7 bytes to reach the next 8-byte boundary, which is 128 bytes.

The amount of space required for data records and hash data record headers is calculated as follows:

Data record space = (hash data record header + aligned record length) $\times$ number of data records

$$= (24 + 128) \times 2031$$

$$= 308,712$$

The maximum number of buckets is the next prime number equal to or greater than the number of records in the file (2031) times the maximum bucket factor of 6.

Next prime $\geq$ (maximum bucket factor $\times$ number of records):

= next prime $\geq$ (6 $\times$ 2031)

= next prime $\geq$ 12186

= 12197

Therefore the maximum bucket space required is 12,197 buckets times the size of each bucket (24 bytes). $12,197 \times 24 = 292,728$.

After you have determined the size of each of the three data areas, you can add them together to obtain the maximum size required for the hash table in memory.

Table size = Hash table header + Data records and hash data headers + Bucket space

= 4184 + 308,712 + 292,728

= 605,624

Therefore, the -MT_SIZE param for the external memory table (OLTP) in this example must be set to a value equal to or greater than 605624. If it is set to a lesser value, such as 500000, which is the default value set for most external memory tables (OLTPs) in the CONFCSV at installation, the build of the external memory table (OLTP) will fail. Note that when the Data Source Loader component has finished determining the most efficient hash algorithm to use, it resizes the memory segment to the actual number of bytes used to build the table. This memory table is then flushed to disk (i.e., in the disk location specified by the _OLTP assign) as an unstructured file where it only occupies the space needed.

CONFCSV parameters

By setting certain optional parameters for an external memory table (OLTP) assign (e.g., _OLTP) in the CONFCSV, the Data Source Loader component generates a hash table processing summary report when you build an external memory table (OLTP). By default, the report is not suppressed. When the -SUPPRESS_RPT param is not present or is set to a value of N, the summary report is generated and written to a spooler or write-only file location specified for stream data in the assign name specified in the -STATS_DSRC_ASSIGN parameter. For example, you could set the -STATS_DSRC_ASSIGN to "MTRCRC," which is write-only file (WOF) for stream data provided in the default installed CONFCSV configuration. Samples of these assigns and params in the CONFCSV are shown below.

Assign Entries:

```
"MTRCRC", "Stream", "1.0", "$bard.EDUCdata.mtrcrc", "WOF", "", ""  
"INTERCHANGE_SECURITY_OLTP", "Interchange_Security_OLTP", "1.  
0", "$bard.  
educdata.isecod", "HASHDS", "-MT_SIZE=500000", "ICHG_SECURITY"
```

Param Entries:


```
"isecod_SUPPRESS_RPT","1.0","N","bt_stringf",1,1,False,""  
"isecod_STATS_DSRC_ASSIGN","1.0","MTRCRC","bt_stringf",1,32,False,""  
"isecod_MT_SIZE","1.0","500000","bt_stringf",1,1,False,""
```

Note that the hash table processing summary report is only written if it is not suppressed and a valid write-only location is specified. This summary processing report lists the bucket factor used by the optimum algorithm. You can replace the maximum bucket factor used in the example calculations above with the bucket factor listed on this report to obtain a close approximation of the required size of the final table built.

Two other optional parameters for external memory table (OLTP) assigns in the CONFCSV control how the Data Source Loader component builds external memory tables (OLTPs). The -KEY_TYPE parameter indicates whether the key for the hash table is alphanumeric (ASCII or A), binary (BINARY or B), or both alphanumeric and binary (ANY or *). This parameter defaults to a value of NONE.

The -BLD_DSRC_ASSIGN parameter is an optional parameter that is used by those applications that need to connect to a read-only external memory table (OLTP) where the memory table has not already been built. Typically the application could check the error from the file to determine that the memory table did not exist. If it does not exist, the application could use the PreLoad From attribute of the assign to recreate the file with the read-only flag set to false. This would cause the table to be built using the PreLoad from file as input. After the table was built the application would then disconnect so that the memory table is flushed to disk, and then reconnect with the read-only flag set to true.

Hash table processing summary report

When the processing summary report is not suppressed and a valid write-only location is specified for an external memory table (OLTP), the Data Source Load component generates a hash table processing summary report when you rebuild an external memory table (OLTP). The hash table summary report details the hashing algorithms and bucket factors used to build the optimum hash table for a file.

A sample hash table summary report is shown below, followed by descriptions of the fields that appear on the report.

```

1: load oltp <oltp-assign-name>;
  Table Size:                5288 bytes

  Total Data Records:        9
  Maximum Record Length:     46 bytes

  Hash Statistics:
  Key Type:                   NONE
  Key Length:                 16 bytes
  Key Algorithm:              DOUBLE BYTE XOR
  Hash Algorithm:             MODULO DIVIDE

  Bucket Factor:             2
  Total Buckets:              19
  Total Buckets Used:         9
  Percent Buckets Used:      47%
  Total Barrels:             1

  Total Collisions:           0
  Percent Collisions:         0%
  Total Collision Chains:     0
  Longest Collision Chain:    0
  Average Collision Chain:    0

  Build Attempts:            20
Load of <oltp-assign-name> successful.

```

Table Size — The total size, in bytes, required for the external memory table (OLTP) using the optimum hash algorithm. Note that this may not be the maximum space used because the Data Source Load component tests 108 different algorithms one at a time when building the hash table. Some algorithms may require more space than the optimum algorithm.

Total Data Records — The total number of data records included in the external memory table (OLTP).

Maximum Record Length — The maximum length, in bytes, of each record included in the external memory table (OLTP).

Key Type — The type of record key specified for the build of this external memory table (OLTP). Valid types are: ALL ASCII (alphanumeric), ALL BINARY (the binary data cannot exceed four bytes), ASCII AND BINARY (combination of alphanumeric and binary data), and NONE.

Key Length — The total length, in bytes, of the primary key to this external memory table (OLTP).

Key Algorithm — The optimum key algorithm used to hash the key from this file in building the optimum hash table. The Data Source Loader component tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built. Valid key algorithms include the following: SINGLE BYTE XOR, DOUBLE BYTE XOR, TRIPLE BYTE XOR, QAUDRUPLE BYTE XOR, SINGLE BYTE SUMMATION, DOUBLE BYTE SUMMATION, and SINGLE BYTE POLYNOMIAL.

Hash Algorithm — The optimum algorithm used to hash key values from this file in building the optimum hash table. The Data Source Loader component tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built. Valid hash algorithms include the following: MULTIPLICATIVE, MID-SQUARE, and MODULO DIVIDE.

Bucket Factor — The bucket factor used in building the hash table. The bucket factor is the ratio of the number of buckets that will exist to the actual number of data records in the hash table. The Data Source Loader component tries all combinations of allowed hash algorithms, key algorithms, and bucket factors until a table with the fewest collisions is built. For example, a bucket factor of 3 indicates that there would be three times as many buckets as there are hash records determined by the hash algorithm.

Total Buckets — The total number of buckets used for this combination of hash algorithm, key algorithm, and bucket factor.

Total Buckets Used — The total number of buckets that actually contain data records for this combination of hash algorithm, key algorithm, and bucket factor.

Percent Buckets Used — The percentage of buckets containing data items in the optimum hash table.

Total Barrels — The total number of barrels used in the optimum hash table. A barrel is equal to a group of buckets.

Total Collisions — The total number of collisions that occurred on the optimum hash table. A collision occurs when two or more data records hash to the same bucket.

Percent Collisions — The percentage of collisions that occurred on the optimum hash table. A collision occurs when two or more data records hash to the same bucket.

Total Collision Chains — The total number of collision chains that occurred on the optimum hash table. A collision chain occurs when two or more data records hash to the same bucket.

Longest Collision Chain — The longest collision chain that occurred on the optimum hash table. A collision chain occurs when two or more data records hash to the same bucket.

Average Collision Chain — The average length of all collision chains that occurred on the optimum hash table. A collision chain occurs when two or more data records hash to the same bucket.

Build Attempts — The total number of combinations attempted to obtain the optimum hash table.

Terminology

The following terms are essential to understanding hash tables and how they are used in processing.

Hash table — A hash table uses a hash function to provide rapid access to data items which are distinguished by some key. Each data item to be stored is associated with a key (e.g, the name of a person). A hash function is applied to the item's key and the resulting hash value is used as an index to select one of a number of hash buckets in a hash table. Records in a hash table are retrieved by exact key value. A hash bucket contains a list of records that were hashed to that specific bucket. The list of records is search sequentially until an entry matches the key value being used.

Hash bucket — A notional receptacle, a set of which might be used to apportion data items for sorting or lookup purposes. For example, when you look up a name in the phone book, you typically hash it by extracting its first letter; the hash buckets are the alphabetically ordered letter sections. This term is used for software with respect to code that uses actual hash functions; in jargon, it is used for human associative memory as well. Thus, two things *in the same hash bucket* are more difficult to discriminate, and may be confused. For example, if you hash English words only by length, you get too many common grammar words in the first couple of hash buckets. Compare to hash collision.

Hash collision — A hash collision occurs when two or more different keys hash to the same value, that is, to the same location in a hash table.

Hash function — A hash coding function which assigns a data item distinguished by some key into one of a number of possible hash buckets in a hash table. The hash function is usually combined with another more precise function.

For example a program might take a string of letters and put it in one of twenty six lists depending on its first letter. Ideally, a hash function should distribute items evenly between the buckets to reduce the number of hash collisions. If, for example, the strings were names beginning with "Mr.", "Miss" or "Mrs." then taking the first letter would be a very poor hash function because all names would hash the same.

Warmbooting the Transaction Security Services process

After you have successfully rebuilt an external memory table (OLTP), you must either warmboot the Transaction Security Services process, or stop and restart the process, before it will begin using the new external memory table (OLTP) in processing. You can stop and restart a Transaction Security Services process from an XPNET network control facility using the STOP PROCESS and START PROCESS commands. You can warmboot Transaction Security Services processes to start using a rebuilt external memory table (OLTP) using any of the following three methods:

- Enter a DELIVER PROCESS command from an XPNET network control facility.
- Enter an Alter Data Source command (also referred to as the OLTP Warmboot command) from the Process Control window on the ACI desktop user interface.

- Enter an OLTP Warmboot command from the OLTP Warmboot Management window on the ACI desktop user interface.

Procedures for the first two methods of warmbooting Transaction Security Services processes to start using a rebuilt external memory table (OLTP) are provided below. Procedures for using the OLTP Warmboot Management window to warmboot Transaction Security Services processes to start using a rebuilt external memory table (OLTP) are provided in [section 11](#). For detailed information on entering STOP PROCESS and START PROCESS commands, refer to the **NET24-XPNET Network Control Command Reference Manual**.

XPNET network control facility

To warmboot a Transaction Security Services process from an XPNET network control facility for an external memory table (OLTP), use the following command.

Syntax

DELIVER PROCESS *object-name*, "ALTER DASRC *oltp-assign-name*"

object-name — The symbolic name assigned to a Transaction Security Services process in the XPNET node.

oltp-assign-name — The assign name specified for the external memory table (OLTP) in the CONFCSV on the data subvolume. For example, the assign name for the IDESOD external memory table (OLTP) is IBM_DES_OLTP.

Example

The following is an example of a DELIVER PROCESS command sent to a Transaction Security Services process to reread the external memory table (OLTP) for the IBM DES Verification File.

```
deliver process p1a^tss01,"alter dasrc IBM_DES_OLTP"
```

Process Control window

To warmboot Transaction Security Services processes from the Process Control window on the ACI desktop user interface for an external memory table (OLTP), perform the following steps:

1. Log on to the ACI desktop user interface and display the Process Control window by selecting System Operations and then Process Control.
2. In the Destination field, select IS or PCTL from the list of values. For the IS destination, the ALTER command is broadcast to all Transaction Security Services processes configured in the XPNET node for the "IS" service. The destination of PCTL sends commands to all components registered to use the command, regardless of the type of

process in which they are found. The PCTL destination causes processes of all types to close and reopen (re-read) the memory table, thereby accessing the new data in processing. When using the PCTL destination, you may see informational event messages logged for any processes that attempt to close and reopen the memory table if they do not actually use the table.

3. In the Command field, select Alter from the list of values.
4. In the Component field, select Data Source from the list of values.
5. In the Command Text field, enter the assign name of the file to be warmbooted. This is the assign name specified for the external memory table (OLTP) in the CONFCVS on the data subvolume. For example, the assign name for the IDESOD external memory table (OLTP) is IBM_DES_OLTP.
6. Click Submit () at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window.

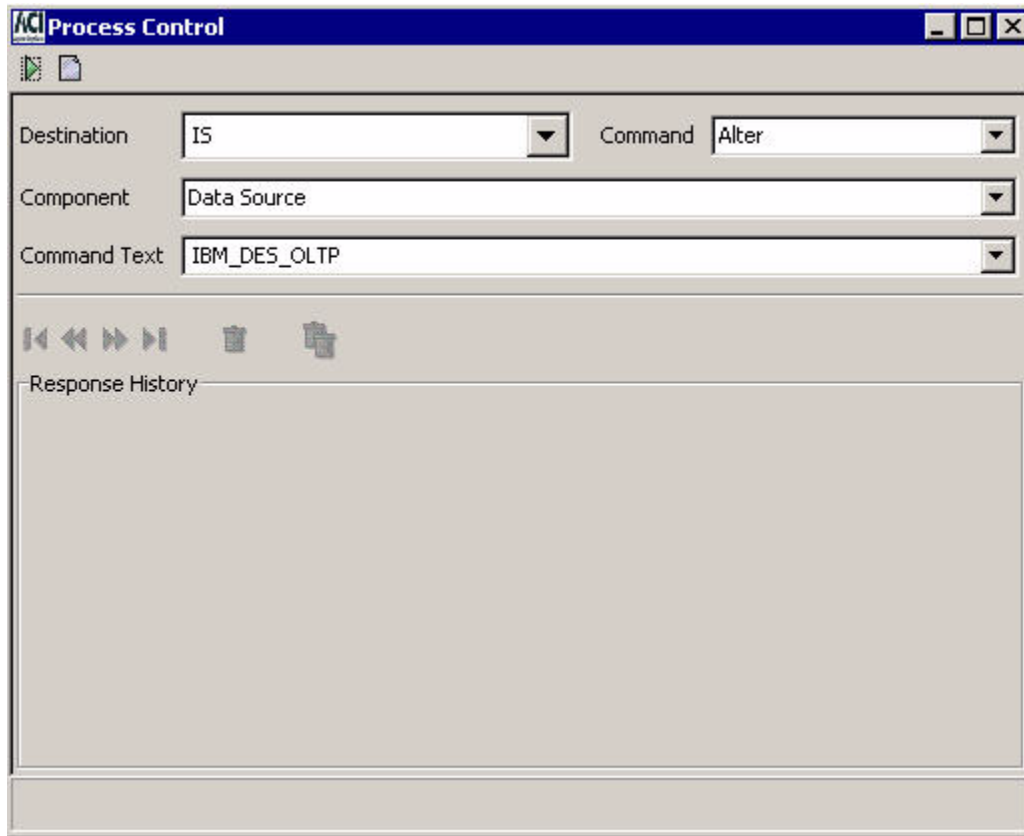
If the table was successfully warmbooted for a process, an event message similar to the following will be displayed on the EMS log.

```
MM-DD-YY;HH:MM:SS.HHH \SYS.$B4XM  ACI.1033.1000      60
Mgr PPD:\SYS.$BB4AN Generator: P1A^IAS01
DASRC: Successful warmboot of data source IBM_DES_OLTP
```

If a process does not have the specified external memory table (OLTP) open, the following message is displayed:

```
Warmboot not needed
```

An example of the Process Control window is shown below.



Internal memory tables

Some of the files maintained by the Transaction Security Services process are accessed as internal memory tables during online transaction processing. The Transaction Security Services processes that access these files must be warmbooted when you make a change to the file from which the internal memory tables are built. The following files for the Transaction Security Services process are accessed internally as memory tables:

- Channel Security File (Channel_Security)¹
- Interface Security File (ICHG_SECURITY)²
- Security Device Configuration File (SEC_DEV_CONFIG)

¹ Data for a channel in the Channel_Security can be accessed from an internal memory table or from disk, depending on how the CSEC_MEMORY_USE Environment attribute is set. When this attribute is set to Y, a limited number of Channel_Security records are maintained in internal memory. In this case, the MAX_CSEC_IN_MEM Environment

attribute specifies the maximum number of Channel_Security records that can be maintained in memory. When the CSEC_MEMORY_USE Environment attribute is set to N, all Channel_Security records are accessed from disk.

Due to hardware memory and swap space limitations, it may not be possible for a single Transaction Security Services process to maintain all Channel_Security records in memory if the channel (terminal) population exceeds a maximum number of devices. If you have more Channel_Security records than can be handled in memory for your hardware configuration, you must either always access Channel_Security records from disk by setting the CSEC_MEMORY_USE attribute to N or limit the number of Channel_Security records that can be maintained in memory. The MAX_CSEC_IN_MEM Environment attribute limits the number of Channel_Security records that can be maintained in internal memory when the CSEC_MEMORY_USE attribute is set to Y. When the number of Channel_Security records added to memory reaches this limit and a Channel_Security record is read from disk, the first entry is deleted from the memory table and the record just read from disk is inserted into the memory table. In effect, records are swapped in and out of memory when this limit is reached.

For Channel_Security records maintained in memory, recursive function calls are made when either a PIN verify or PIN translate function fails. This is done because the PIN key in memory may not be current. In this case, the Channel_Security record is reread from disk and the security device call is made again. For Channel_Security records accessed from disk, recursive function calls are not made when either a PIN verify or PIN translate function fails. For MAC verify, PVV generate, MAC generate, and data encrypt/decrypt/translate function calls, the first call is always forced to disk so no recursive calls are ever made.

- ² Data for an interface in the ICHG_SECURITY is accessed from an internal memory table only, except for outbound keys, when the interface supports dynamic key management. When an interface supports dynamic key management, outbound keys are always retrieved from the ICHG_SECURITY disk file. Dynamic key management is indicated by a value of "Use import key for both options" or "Use separate keys" in the Options field on the Exchange Information tab of the Interface Security Configuration window. Otherwise, data for an interface is accessed from the external memory table (OLTP).

When you change key values on the Interface Security Configuration window for an interface that does exchange keys, you do not have to warmboot the Transaction Security Services process to reread the ICHG_SECURITY, or stop and restart the Transaction Security Services process. The Transaction Security Services process reads the new key values from the ICHG_SECURITY disk file for the next transaction for the interface that requires security key data if it encounters a key synchronization error using the key in the internal memory table or if the key is an outbound key. If, however, you add or delete a record for an interface that does exchange keys, or if you change the value of the Options field on the Exchange Information tab for an interface to start or stop exchanging keys, you must rebuild the Interface extended memory table before warmbooting the Transaction Security Services process to reread the ICHG_SECURITY, or stopping and restarting the Transaction Security Services process.

When you change values on the Interface Security Configuration window for an interface that does not exchange keys, you must always rebuild the Interface extended memory table and warmboot, or stop and restart, the Transaction Security Services process to reread the Interface extended memory table. The new values will be used in processing for the next transaction for the interface that requires security key data.

Refer to the [“External memory tables \(OLTPs\)”](#) topic for more information on rebuilding external memory tables (OLTPs) and warmbooting the Transaction Security Services process for the ISECOD external memory table (OLTP).

The Channel_Security is maintained from the Channel Security Configuration window, the ICHG_SECURITY is maintained from the Interface Security Configuration window, and the SEC_DEV_CONFIG is maintained from the Security Device Configuration window.

When you warmboot a Transaction Security Services process for a file accessed as an internal memory table using the Alter Data Source process control command, the process deletes all records from internal memory and reloads the memory table as described below. For the Channel_Security, you can also warmboot a Transaction Security Services process to reread a single record from disk using the CSEC Refresh process control command. In this case, all of the other records in internal memory are retained.

For the Channel_Security (when the CSEC_MEMORY_USE Environment attribute is set to Y) and ICHG_SECURITY, the process adds entries to the internal memory table as each transaction is processed. When a Channel_Security record is needed, the process first searches for the record in the internal memory table. If the record is not found in memory, the process reads the record from the disk file and adds it to the internal memory table. If adding the record to memory exceeds the limit imposed by the MAX_CSEC_IN_MEM Environment attribute, the first entry is deleted from the memory table before the record just read from disk is inserted into the memory table. When an ICHG_SECURITY record is needed for an interface that supports dynamic key management, except for outbound keys, the process first searches for the record in the internal memory table. If the record is not found in internal memory, the process reads the record from the disk file and adds it to the internal memory table. For an outbound key, the record is always read from the disk file. Thus, the internal memory tables for Channel_Security and ICHG_SECURITY records are rebuilt dynamically as transactions are processed.

For the SEC_DEV_CONFIG, the process immediately reloads all records from the disk file to the internal memory table.

Warmbooting the Transaction Security Services process

After you have changed data in the Channel_Security, ICHG_SECURITY, or SEC_DEV_CONFIG, you must warmboot the Transaction Security Services process before it will begin using the new data in processing. You can warmboot Transaction Security Services processes by entering a DELIVER PROCESS command from an XPNET network control facility or by entering a command from the Process Control window or OLTP Warmboot window (for the ICHG_SECURITY only) on the ACI desktop user interface. Procedures for both methods of warmbooting Transaction Security Services processes are provided below. Procedures for warmbooting the Transaction Security Services process to reread the ICHG_SECURITY from the OLTP Management window are provided in [section 11](#).

XPNET network control facility

To warmboot a Transaction Security Services process from an XPNET network control facility for an internal memory table, use the following command.

Syntax

DELIVER PROCESS *object-name*, "ALTER DASRC *data-source-name*"

object-name — The symbolic name assigned to a Transaction Security Services process in the XPNET node.

data-source-name — The assign name specified for the disk file in the CONFCSV on the data subvolume. The internal memory table is rebuilt from the disk file. The following table lists the assign names for each of the disk files accessed as internal memory tables.

Assign name	TSS file
CHANNEL_SECURITY	Channel Security File (Channel_Security)
ICHG_SECURITY	Interface Security File (ICHG_SECURITY)
SEC_DEV_CONFIG	Security Device Configuration File (SEC_DEV_CONFIG)

Example

The following is an example of a DELIVER PROCESS command sent to a Transaction Security Services process to reload the internal memory table for the Channel Security File.

```
deliver process p1a^tss01,"alter dasrc CHANNEL_SECURITY"
```

Process Control window

To warmboot Transaction Security Services processes from the Process Control window on the ACI desktop user interface for an internal memory table, perform the following steps:

1. Log on to the ACI desktop user interface and display the Process Control window by selecting System Operations and then Process Control.
2. In the Destination field, select IS or PCTL from the list of values. For the IS destination, the ALTER command is broadcast to all Transaction Security Services processes configured in the XPNET node for the "IS" service. The destination of PCTL sends commands to all components registered to use the command, regardless of the type of process in which they are found. The PCTL destination causes processes of all types to close and reopen (re-read) the memory table, thereby accessing the new data in

processing. When using the PCTL destination, you may see informational event messages logged for any processes that attempt to close and reopen the memory table if they do not actually use the table.

3. In the Command field, select Alter from the list of values.
4. In the Component field, select Data Source from the list of values.
5. In the Command Text field, enter the assign name of the file to be warmbooted. This is the assign name specified for the disk file in the CONFCSV on the data subvolume from which the internal memory table is built. The following table lists the assign names for each of the disk files accessed as internal memory tables.

Assign Name	TSS File
CHANNEL_SECURITY	Channel Security File (Channel_Security)
ICHG_SECURITY	Interface Security File (ICHG_SECURITY)
SEC_DEV_CONFIG	Security Device Configuration File (SEC_DEV_CONFIG)

6. Click Submit () at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window.

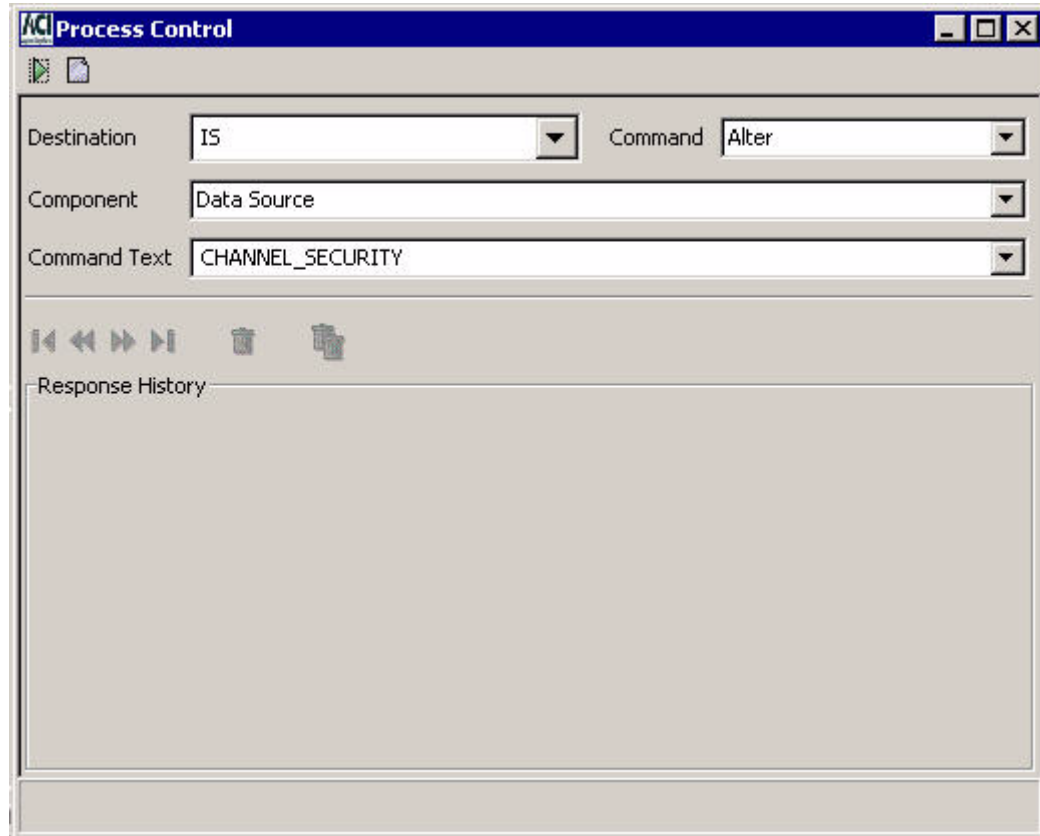
If the table was successfully warmbooted for a process, an event message similar to the following will be displayed on the EMS log.

```
MM-DD-YY;HH:MM:SS.HHH \SYS.$B4XM ACI.1033.1000 60
Mgr PPD:\SYS.$BB4AN Generator: P1A^IAS01
DASRC: Successful warmboot of data source CHANNEL_SECURITY
```

If a process does not have the specified internal memory table open, the following message is displayed:

```
Warmboot not needed for CHANNEL_SECURITY
```

An example of the Process Control window is shown below.



3: Configuration

This section describes the application configuration requirements for implementing the Transaction Security Services process. Additional configuration requirements for XPNET, TCP/IP, HTTP, and OSS should be determined during installation and are beyond the scope of this guide.

LCONF assigns

When implementing the Transaction Security Services process, all SECURE-DEV1, SECURE-DEV2, or SECURE-DEV-xx assigns must be set to the process pair directory (PPD) name of a Transaction Security Services process. BASE24 application processes send messages directly to the Transaction Security Services process or AKDS process identified in the SECURE-DEV1, SECURE-DEV2, or SECURE-DEV-xx assigns using a write/read interface when the SECURE-DEV-TYP param is set to a value of TSS in the LCONF. The SECURE-DEV-ACCESS-TYP param indicates whether a BASE24 application process communicates with a primary and secondary Transaction Security Services process identified by the SECURE-DEV1 and SECURE-DEV-2 assigns or with up to 20 Transaction Security Services processes identified by SECURE-DEV-xx assigns, where xx is any two characters. When using the SECURE-DEV-xx assigns, the BASE24 application process accesses Transaction Security Services processes on a *round-robin* (i.e., rotating) basis.

Note: The AKDS process is a special Transaction Security Services process that contains all of the components of the Transaction Security Services process plus the components required to support the BASE24 Automated Key Distribution System add-on product or secure logons and logoffs to and from an interchange. NCR 5XXX or Diebold 10XX/478X Device Handler processes or Interchange Interface processes must use the AKDS process instead of the regular Transaction Security Services process when using the BASE24 Automated Key Distribution System add-on product or secure logons and logoffs to and from an interchange. Configuration requirements for the AKDS process are the same as that of the regular Transaction Security Services process.

Primary and secondary TSS process access method

When the SECURE-DEV-ACCESS-TYP param in the LCONF is set to a value of 0, the BASE24 application process uses the SECURE-DEV1 and SECURE-DEV2 assigns to access Transaction Security Services processes.

The SECURE-DEV1 assign specifies the PPD name of the primary Transaction Security Services process for a BASE24 application process, while the SECURE-DEV2 assign specifies the PPD name of the secondary Transaction Security Services process for a BASE24 application process. You can assign different Transaction Security Services processes to different BASE24 application processes by configuring multiple SECURE-DEV1 and SECURE-DEV2 assigns in the LCONF and assigning each to a specific symbolic process name in the READ BY field on the LNCF Assign screen. In addition, the USER FIELD for the SECURE-DEV1 and SECURE-DEV2 assigns on the LNCF Assign screen must be left blank. The security utilities bound into BASE24 processes use this value to define the device subtype. Since there are no valid device subtypes associated with calls to the Transaction Security Services process, the utilities generate an error event message when a value is present.

When a transaction is received by a BASE24 process that requires the use of a security module, the BASE24 process first attempts to communicate with its primary Transaction Security Services process. If the request to the primary Transaction Security Services process times out or the primary Transaction Security Services process is considered inoperative, the BASE24 process attempts to communicate with its secondary Transaction Security Services process to complete the transaction.

The security utilities for a BASE24 process can report a 9100 error message with hardware error 201 (the current path to the device is down) when the Transaction Security Services processes specified by the SECURE-DEV1 and SECURE-DEV2 assigns have been stopped and restarted after the BASE24 process is initialized. In this case, you must warmboot, or stop and restart, the BASE24 process reporting the error.

Note: When the security device associated with the primary Transaction Security Services has reached the [maximum consecutive error count](#) and the security device associated with the secondary Transaction Security Services is in use, you must warmboot the BASE24 process to start using the security device associated with the primary Transaction Security Services process again.

You can also use the SECURE-DEV-MONITOR-THRESHOLD param to check the secondary Transaction Security Services process at specified intervals or balance requests evenly between the primary and secondary Transaction Security Services processes.

Checking the secondary Transaction Security Services process

To minimize the possibility of both Transaction Security Services processes becoming inoperative, the BASE24 system can be configured to occasionally perform one request using the secondary Transaction Security Services process instead of the primary Transaction Security Services process. The request is sent to the secondary Transaction Security Services process at intervals based on the number of requests performed by the primary Transaction Security Services process.

Each BASE24 process using the Transaction Security Services process maintains its own request counter. The value specified in the SECURE-DEV-MONITOR-THRESHOLD param is the number of requests processed by the primary Transaction Security Services process before the BASE24 process sends a single request to the secondary Transaction Security Services process. For example, if this param were set to a value of 1000, the BASE24

process would send 999 requests to the primary Transaction Security Services process before it would send one request to the secondary Transaction Security Services process, thereby checking its availability. The request counter is reset each time the threshold is reached.

Occasionally using the secondary Transaction Security Services process permits the identification and resolution of any problems with the secondary Transaction Security Services process while the primary Transaction Security Services process is functioning properly.

When using the SECURE-DEV-MONITOR-THRESHOLD param to periodically check the availability of the secondary Transaction Security Services process, ACI recommends that the primary Transaction Security Services process assigned to a BASE24 application process run in the same CPU as the requesting BASE24 process for performance reasons. By sending the majority of requests to a Transaction Security Services process running in the same CPU, the additional overhead associated with communicating over the inter processor bus between CPUs is avoided.

Balancing requests between transaction Security Services processes

Although you could use the SECURE-DEV-MONITOR-THRESHOLD param to balance the number of requests processed by the primary and secondary Transaction Security Services processes, it is not recommended because the Transaction Security Services process automatically balances requests across all security devices. The Transaction Security Services process alternates requests among all security devices (or interface module processes) configured in the Security Device Configuration File one request at a time.

When this param is set to a value of 1, each BASE24 process alternates requests between the primary and secondary Transaction Security Services processes. Each BASE24 process uses the primary Transaction Security Services process for one transaction, uses the secondary Transaction Security Services process for one transaction, and then repeats the cycle.

Round-robin TSS process access method

When the SECURE-DEV-ACCESS-TYP param in the LCONF is set to a value of 1, the BASE24 application process uses the SECURE-DEV-xx assigns to access Transaction Security Services processes. You can configure a BASE24 application process to access up to twenty different Transaction Security Services processes on a *round-robin* (i.e., rotating) basis by configuring up to twenty SECURE-DEV-xx assigns, where xx is any two characters, with each assign specifying a different Transaction Security Services process.

For this method of Transaction Security Services process access, the security utilities in the BASE24 application process maintains the status of each Transaction Security Services process in the Security Device Control Block (SDCB) as well as a time to attempt to reopen the Transaction Security Services process. In addition, the SDCB contains fields indicating the next Transaction Security Services process to use, the number of Transaction Security Services processes open, and the time to delay before attempting to reinstate a Transaction Security Services process. This reinstate delay time is set by the SECURE-DEV-REINSTATE-TIME param.

As each call to a Transaction Security Services process is made, the security utilities check the index in the SDCB to determine which Transaction Security Services process to use. The security utilities then increment the index to point to the next Transaction Security Services process to be used for the next transaction. If the SDCB flag indicates that a particular Transaction Security Services process is down, that Transaction Security Services process is skipped in the rotation.

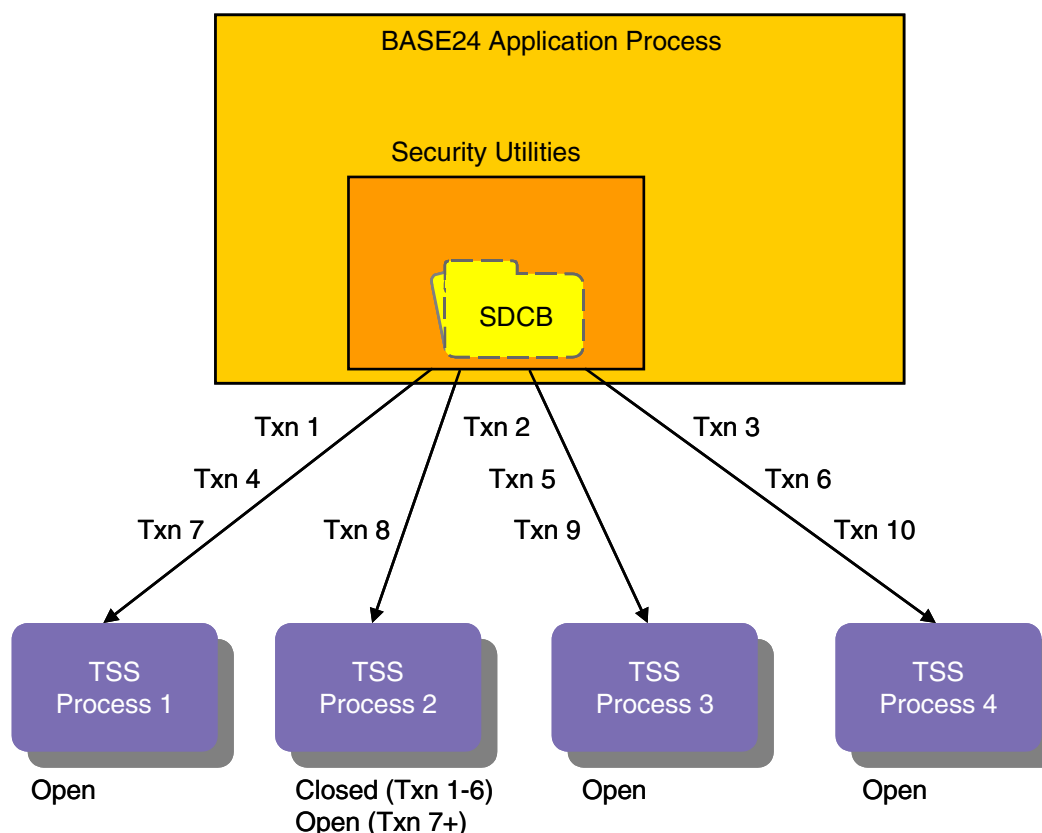
If a Transaction Security Services process is marked down, the security utilities checks the reinstate time. If the reinstate time has passed, the security utilities attempt to restart the Transaction Security Services process. If the restart is successful, the SDCB is updated to indicate that the Transaction Security Services process is in an open state.

The security utilities generate an event message each time a Transaction Security Services process state is modified. The security utilities also log the status of the Transaction Security Services process and how many Transaction Security Services processes are up. If no Transaction Security Services processes are up, the utilities generates an event message indicating this fact.

The security utilities reads the LCONF for SECURE-DEV-xx assigns until no more assigns are found or until twenty assigns have been read. For each assign, the utilities initializes the SDCB with the Transaction Security Services process name, file number, subtype, fail count, state and reopen time.

Use of the round-robin Transaction Security Services process access method is depicted in the following illustration. In this example, there are four Transaction Security Services (TSS) processes. TSS process 2 is marked as closed in the SDCB when the BASE24 application process requires a hardware security device for transactions 1–6. Therefore, it is excluded

from the round-robin distribution. By the time transaction 7 needs to be processed, TSS process 2 is now marked as open and is reinstated into the round-robin distribution and receives transaction 8.



PPD names

The PPD names entered for the SECURE-DEV1, SECURE-DEV2, or SECURE-DEV-xx assigns must match the PPD names assigned to the primary, secondary, or multiple Transaction Security Services processes in the XPNET configuration.

The Security Device Name field on the Security Device Configuration window of the ACI desktop contains an assign name for a particular security device. All Transaction Security Services processes can access all of the security devices defined in this window. This assign name is associated with the actual PPD name of the security device (i.e., the SYSGEN utility, DSC, or SCF name of the device) or the interface module process (i.e., Boxcar, Racal Access Module, HSM Interface) in the System Interface Message Delivery Service Destination File (MDSDEST) on the data subvolume. This file must be created on the HP NonStop system when installing the ACI desktop user interface. For detailed information on creating the MDSDEST, refer to [section 7](#).

Note: All Atalla implementations using the TCP/IP protocol require the Boxcar process.

LCONF params

As stated previously, the SECURE-DEV-TYP param in the LCONF must be set to a value of TSS when implementing the Transaction Security Services process.

Unused params

When using the Transaction Security Services process, two params in the LCONF are not used because they are replaced by similar parameters specified in the Security Device Configuration window of the ACI desktop. The security utilities generate a warning event message when either of the following params are read from the LCONF and the SECURE-DEV-TYP param is set to TSS:

- SECURE-DEV-CONF-CMD100
This param is replaced by the Initialization String field on the Security Device Configuration window of the ACI desktop.
- SECURE-DEV-CONF-TXT
This param is replaced by the Configuration String field on the Security Device Configuration window of the ACI desktop.

The RSM-TERM-MASTER-KEY param is replaced by the RSM_TERM_MASTER_KEY attribute set in the Transaction Security Services environment. Refer to the [“Transaction security services attributes”](#) topic later in this section for a description of this attribute.

Access method

The SECURE-DEV-ACCESS-TYP param indicates whether a BASE24 application process communicates with a primary and secondary Transaction Security Services process identified by the SECURE-DEV1 and SECURE-DEV-2 assigns or with up to 20 Transaction Security Services processes identified by SECURE-DEV-xx assigns. Valid values are as follows:

- 0 = Access primary and secondary Transaction Security Services processes using the SECURE-DEV1 and SECURE-DEV2 assigns. Default.
- 1 = Access multiple Transaction Security Services processes on a round-robin basis using the SECURE-DEV-xx assigns.

Reinstate time

The SECURE-DEV-REINSTATE-TIME param indicates the number of minutes from the time a Transaction Security Services process is marked closed to the time the BASE24 application process attempts to restart the Transaction Security Services process. A Transaction Security Services process is marked closed when three or more consecutive critical errors have occurred since initialization or warmboot. The valid range for this param is 0–32000.

A value of 0 indicates that the BASE24 application process does not attempt to reopen a Transaction Security Services process based on the time interval. This param is only used when the SECURE-DEV-ACCESS-TYP param is set to a value of 1.

The SECURE-DEV-REINSTATE-TIME param provides an additional means of determining when the BASE24 application process attempts to reopen a Transaction Security Services process. The reopen is attempted either when the given time interval specified by this param has been reached or when the number of times the device has come up in the SDCB round-robin list since the last attempt to open the process is equal to the value of the SECURE-DEV-FAIL-NOTIFY param. You can configure both the SECURE-DEV-REINSTATE-TIME and SECURE-DEV-FAIL-NOTIFY when using the SECURE-DEV-xx assigns. If both params are configured, then the Transaction Security Services process is reopened when either threshold is reached.

Software PIN verification

For BASE24-atm Authorization processes and BASE24-pos Device Handler/Router/Authorization processes, a param called SOFTWARE-PIN-VERIFICATION-ALWD, indicates whether PIN verification, card verification, or PIN offset generation is allowed to be performed in software or in the clear within any BASE24-atm Authorization processes and BASE24-pos Device Handler/Router/Authorization process in the BASE24 system when the SECURE-DEV-TYP param is set to a value of TSS. A value of Y in this param indicates that PIN verification, card verification, or PIN offset generation is allowed in software or in the clear when the Transaction Security Services process is used. A value of N in this param indicates that PIN verification, card verification, or PIN offset generation is not allowed in software or in the clear when the Transaction Security Services process is used. In the latter case, for any transactions in which PIN verification, card verification, or PIN offset generation is attempted in software or in the clear, the transaction is denied. This param defaults to a value of N. The Transaction Security Services process itself does not perform PIN verification, card verification, or PIN offset generation in software or in the clear. This processing must be performed within a BASE24 authorization process.

Diebold PIN verification

When using Diebold PIN verification, the ATM-4000-MASTER param for BASE24-atm Authorization processes and the POS-4000-MASTER param for BASE24-pos Device Handler/Router/Authorization processes are not used and do not need to be configured. The Transaction Security Services process controls downloading of the Diebold number table rather than the BASE24 authorization processes.

Partial triple DES

For Interchange Interface processes, the DES-TRIPLE-SINGLE param must be set to a value of N. This prevents an Interchange Interface process from attempting to read the Key 6 File (KEY6), which is not used in processing by the Transaction Security Services process.

Changed param usage

Three existing LCONF params, SECURE-DEV-MONITOR-THRESHOLD, SECURE-DEV-FAIL-NOTIFY, and SECURE-DEV-TIM-LMT have different meanings and usage when the Transaction Security Services process is implemented.

The SECURE-DEV-MONITOR-THRESHOLD param specifies the number of requests a BASE24 process sends to its primary Transaction Security Services process before it sends a single request to its secondary Transaction Security Services process. You can use this param to check the secondary Transaction Security Services process at specified intervals or balance requests evenly between the primary and secondary Transaction Security Services processes. Refer to the [“Checking the secondary Transaction Security Services process”](#) and [“Balancing requests between transaction Security Services processes”](#) topics provided earlier for detailed information on how this param is used with Transaction Security Services processes.

The SECURE-DEV-FAIL-NOTIFY param specifies the intervals (i.e., the number of transactions processed) at which the security utilities attempt to automatically reopen a Transaction Security Services process when the process has been marked closed due to three or more consecutive critical errors since initialization or warmboot. For example, if this param is set to 20, the security utilities will attempt to reopen the Transaction Security Services process once after the third failure to communicate with it, and then once every twenty transactions afterward. This param applies when using both the primary/secondary and round-robin methods to access Transaction Security Services processes.

The SECURE-DEV-TIM-LMT specifies the maximum time, in hundredths of a second, to wait for a response from the Transaction Security Services process before the request is considered to be timed out. This param must be set in consideration of the Timeout Limit field value on the Security Device Configuration window. The Timeout Limit field value specifies the maximum time, in seconds, to wait for a response from an HSM before the Transaction Security Services process considers the request to be timed out. When a call to the HSM times out, the Transaction Security Services process increments the consecutive retries counter by 1. If the maximum retries (hardcoded to 3) has not been reached, the process attempts the HSM call again using the next available HSM from the pool of HSMs, which may or may not be the same HSM on which the timeout occurred.

The SECURE-DEV-TIM-LMT param value must be set to a value large enough to include the Timeout Limit field value plus the time required to send a request and receive the response from the Transaction Security Services process. Thus, it should always be set to a value greater than that set in the Timeout Limit field on the Security Device Configuration window.

Logical network identifier

In certain logical network configurations, it may be desirable for performance reasons to partition the Channel_Security and ICHG_SECURITY by logical network. Partitioning is also required if you have duplicate device names between logical networks. For example, when separate BASE24 logical networks reside on separate HP NonStop systems, each logical

network would have its own CARD_VRFCTN, DIEBOLD_NUM_TBL, IBM_DES, EMV_Security, Visa_PVV, SEC_DEV_CONFIG, and KEYF. The Channel_Security and ICHG_SECURITY could be partitioned by logical network using the LN-IND param as described below.

The LN-IND param contains a two-character hexadecimal code representing the logical network name specified in the LOGICAL-NET param for this LCONF. The value of this param is used to support the Transaction Security Services process for multiple BASE24 logical networks residing on separate HP NonStop systems. It enables the physical partitioning of the Channel_Security and ICHG_SECURITY accessed by the Transaction Security Services process. Valid values are 0–F.

When adding or updating a BASE24-atm or BASE24-pos Terminal Data files record, the ATD and PTD requester and server processes add the value of this param to the beginning of the terminal ID to create key locator values that are placed in hardware-encrypted key fields (if the SECURE-DEV-TYP param is set to a value of TSS). If the terminal ID for the record being added or updated has more than 14 characters, the last one or two characters are truncated when the key locator value is created. If the LN-IND param does not exist, key locators are set to the full value of the terminal ID.

TSS database conversion programs also use a runtime parameter called LN-IND to create a key locator value when converting existing BASE24 files. When they are run on a BASE24-atm Terminal Data file or BASE24-pos Terminal Data file, the two-character LN-IND value is added to the beginning of the 1–16 character terminal ID. If the terminal ID for the terminal record being converted has more than 14 characters, the last one or two characters are truncated when the key locator value is created.

When converting Key File (KEYF) or Key 6 File (KEY6) records to the TSS database, the two-character LN-IND runtime param value is added to the beginning of a 14-character key locator index value generated by the conversion program.

If you need to partition the TSS database by logical network, the value of the LN-IND LCONF param and the LN-IND runtime parameter used by the TSS database conversion programs must be set to the same value.

If you do not need to partition the TSS database, the LN-IND LCONF param should not be configured and the LN-IND runtime parameter used by the TSS database conversion programs must be set to a value of 00.

The following BASE24 processes use the LN-IND LCONF param:

BASE24-atm Terminal Data files requester and server objects
BASE24-pos Terminal Data files requester and server objects

Multiple logical networks

If the Transaction Security Services process is implemented into a BASE24 system with multiple logical networks and a separate TSS database for each network, the MULT-LN-PIN-KEY LCONF param is required to ensure that PINs are properly translated when a transaction is SPROUTE-routed from one logical network to another using the CP file type.

In BASE24 logical networks using the Transaction Security Services process, the MULT-LN-PIN-KEY param contains a key locator value to a common single-length PIN encryption key in the ICHG_SECURITY. You must manually add this record to the ICHG_SECURITY in each TSS database. In BASE24 logical networks that do not use the Transaction Security Services process, the param contains an encrypted single-length PIN encryption key. The MULT-LN-PIN-KEY param can contain up to 16 hexadecimal characters. PIN blocks translated using the MULT-LN-PIN-KEY param are always converted to ANSI PIN block format.

The following processes use this param, however, it must be configured with a READ BY value of all asterisks.

BASE24-atm Authorization
BASE24-atm ISO Host Interface
BASE24-pos Device Handler/Router/Authorization
BASE24-pos ISO Host Interface

BASE24-atm Authorization processes and BASE24-pos Device Handler/Router/Authorization processes use this param when processing transactions that meet the following conditions:

- The prefix for the transaction was not found in the Card Prefix File (CPF) for the current logical network.
- The transaction is SPROUTE-routed to another logical network using the LGNT routing method and the CP file type.
- The transaction contains an encrypted PIN block.
- The MULT-LN-PIN-KEY param was successfully read during initialization or warmboot processing and contained valid hexadecimal characters.

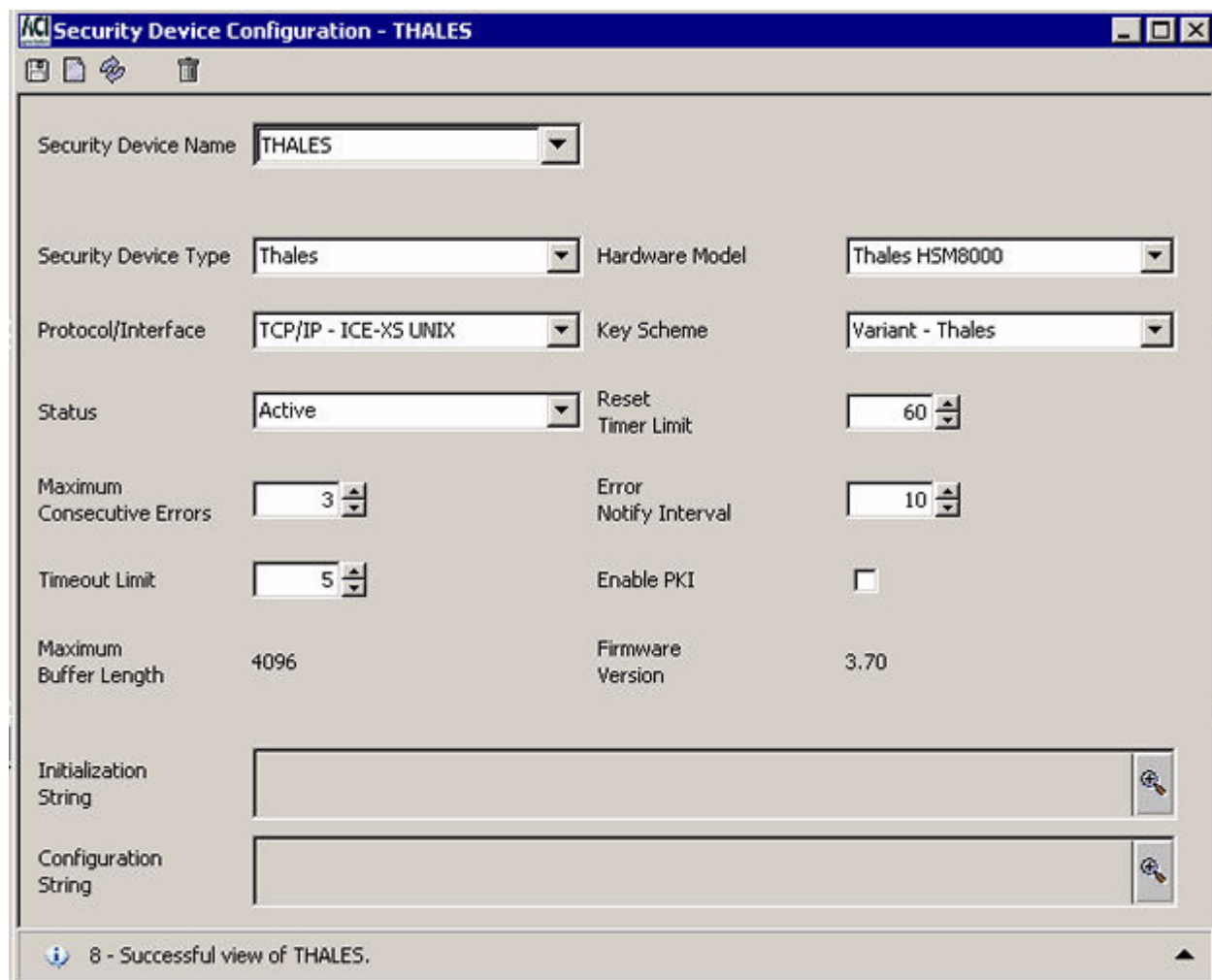
If the above conditions are met, the process translates the PIN block in the internal message to encryption under the PIN encryption key specified by the MULT-LN-PIN-KEY and adds the Multiple Logical Network token to the transaction. If the transaction is a PIN Change transaction, the new PIN block is translated in the same manner.

When a BASE24-atm Authorization, BASE24-pos Device Handler/Router/Authorization, BASE24-atm ISO Host Interface, or BASE24-pos ISO Host Interface process receives a transaction containing the Multiple Logical Network token, it sets the ORIGINATOR field in the internal message to 1 (interface). This serves two purposes. If you are using a Thales e-Security HSM, the common PIN encryption key must be a Zone (interface) PIN key. If the Transaction Security Services process is used in the current logical network, this value allows the process to retrieve the common PIN encryption key from the ICHG_SECURITY.

Security Device Configuration window parameters

Each hardware security device in your system must be configured on the Security Device Configuration window. This window enables you to set several parameters specific to the type of hardware security device used. A sample of the Security Device Configuration window is shown below, followed by descriptions of its fields.

Note: When you add a new security device or change the parameters for a security device, you must [warmboot the Transaction Security Services process to reread the internal table for the Security Device Configuration File \(SEC_DEV_CONFIG\)](#). When you add or upgrade a security device, you must also [initialize the security device](#) before the firmware version and maximum buffer length are displayed on this window for the device.



Security Device Configuration - THALES

Security Device Name: THALES

Security Device Type: Thales Hardware Model: Thales HSM8000

Protocol/Interface: TCP/IP - ICE-XS UNIX Key Scheme: Variant - Thales

Status: Active Reset Timer Limit: 60

Maximum Consecutive Errors: 3 Error Notify Interval: 10

Timeout Limit: 5 Enable PKI: ☐

Maximum Buffer Length: 4096 Firmware Version: 3.70

Initialization String: 

Configuration String: 

8 - Successful view of THALES.

Security device name

The Security Device Name field contains an assign name for a particular security device. All Transaction Security Services processes can access all of the security devices defined in this window. This assign name is associated with the actual PPD name of the security device (i.e., the SYSGEN utility, DSC, or SCF name of the device) or the interface module process (i.e., Boxcar, Remote Application Manager, Racal Access Module, HSM Interface) in the System Interface Message Delivery Service Destination File (MDSDEST) on the data subvolume. This file must be created on the HP NonStop system when installing the ACI desktop user interface. For detailed information on creating the MDSDEST, refer to [section 7](#).

Security device type

The Security Device Type field identifies the manufacturer of the hardware security device. The Transaction Security Services process currently supports devices manufactured by Atalla, Thales e-Security, SafeNet (Eracom), Bull (BNTng models), and Banksys (DEP models). IBM security devices are only supported on BASE24-eps.

Note: You cannot configure devices from more than one vendor in the same logical network. For example, you cannot combine Thales and Atalla devices because they each only recognize their own scheme for storing cryptograms. The two HSM brands are not interoperable. For example, a Thales HSM cannot interpret the format of an Atalla-encrypted key block. You can, however, mix different devices in the same BASE24 system using separate logical networks if a single vendor type is used in each logical network and each logical network is configured with its own TSS database. To route between the logical networks, you would need to configure LGNT routing (CPF routing) over SPROUTE routing and configure the Authorization process or module to translate the PIN block to encryption under a common intermediate key configured on all logical networks.

Hardware model

The Hardware Model field indicates the specific model of the hardware security device for the vendor specified in the Security Device Type field. The Transaction Security Services process uses the settings in this field and the Protocol/Interface and Key Scheme fields to determine the command functions and function parameters it can send to the security device. The possible hardware model values for each supported vendor are described in the following table.

Vendor	Hardware model value	Description
Atalla	Atalla Ax000	Specifies Atalla A8000, A9000, and A10000E NSPs.
	Atalla Ax100	Specifies Atalla high performance A8100, A8150, A9100, A9150, A10100, A10150, and A10160 NSPs.
Thales	Thales RG7400	Specifies Thales e-Security RG7400 HSMs.
	Thales RG71x0	Specifies Thales e-Security RG7100 and RG7110 HSMs.
	Thales RG73x0	Specifies Thales e-Security RG7300 and RG7310 HSMs.
	Thales HSM8000	Specifies Thales e-Security HSM 8000 devices.

Vendor	Hardware model value	Description
SafeNet (Eracom)	Eracom Mark II	Specifies SafeNet (Eracom) ProtectHost White Mark II host security modules.
	Eracom AMB	Specifies SafeNet (Eracom) ProtectHost White AMB host security modules.
Bull BNTng	BNTng V7	Specifies Bull BNTng HSMs running with version 7 or greater software.
DEP	DS2	Specifies Banksys DEP HSMs using the DEP System 2 (DS2) message format. Reserved for future use.
	DS3	Specifies Banksys DEP HSMs using the DEP System 3 (DS3) message format.
	DS4	Specifies Banksys DEP HSMs using the DEP System 4 (DS4) message format. Reserved for future use.

Protocol/interface

The Protocol/Interface field indicates the protocol or interface process used to communicate with the hardware security device. Only values that end with “NSK” are used with the Transaction Security Services process. Values that end with “UNIX” or “IBM” are only applicable to BASE24-eps systems. The possible values for this field depend on the supported hardware vendors and models as shown in the following table.

Vendor	Hardware model	Protocol / interface value	Description
Atalla	N/A	Async – NSK	Specifies an asynchronous connection to the NSP.
		TCP/IP – Boxcar NSK	Specifies the Boxcar process to communicate with the NSP using TCP/IP.

Vendor	Hardware model	Protocol / interface value	Description
Thales	Thales RG7400	Async – NSK	Specifies an asynchronous connection to an RG7400 HSM.
	Thales RG73x0	SDLC - HIP NSK	Specifies the HSM Interface Process (HIP) to communicate with an RG73x0 HSM using a synchronous data-link control (SDLC) emulation mode connection
	All other models	TCP/IP – RAM NSK	Specifies the RAM process to communicate with the HSM using TCP/IP.
		UDP/IP – URAM NSK	Specifies the URAM process to communicate with the HSM using UDP/IP.
SafeNet (Eracom)	N/A	Async – NSK	Specifies an asynchronous connection to the HSM.
		Ethernet - Blocker NSK	Specifies the Blocker process to communicate with the HSM using the Ethernet protocol.
		TCP/IP – RAM NSK	Specifies the RAM process to communicate with the HSM using TCP/IP.
Bull BNTng	BNTng V7	TCP/IP – RAM NSK	Specifies the RAM process to communicate with the HSM using TCP/IP.
DEP	N/A	TCP/IP – RAM NSK	Specifies the RAM process to communicate with the HSM using TCP/IP.

Note: For SafeNet (Eracom) and Thales e-Security devices, the value of the Protocol/Interface field also indicates whether the Transaction Security Services process communicates with the device with bracketed or unbracketed messages. Bracketed messages include start-of-text (STX) and end-of-text (ETX) control characters. Bracketed messages are used with the asynchronous protocol (i.e., Async – NSK) and are not used with the TCP/IP or UDP/IP protocols.

Key scheme

The Key Scheme field indicates the key block format used by Atalla NSPs or the key encryption scheme used by Thales e-Security HSMs. This field is not applicable for SafeNet (Eracom) host security modules (HSMs), Bull BNTng hardware security modules (HSMs), and Banksys DEP HSMs. The possible values for this field are described in the following table.

Key scheme value	Description
N/A	Not applicable. This value is set for SafeNet (Eracom) host security modules (HSMs), Bull BNTng hardware security modules (HSMs), and Banksys DEP HSMs.
AKB – Atalla	Specifies the Atalla Key Block (AKB) format.
Variant – Atalla	Specifies the standard Atalla (variant) key block format.
Variant – Thales	Specifies Thales' variant (1A+32H) key encryption scheme.

Status

The Status field displays the current logical status of a device. This field is set to Active when the device is intentionally placed into service (e.g., by sending an ENABLE process control command to the device) or Inactive when the device is intentionally removed from service (e.g., by sending a DISABLE process control command to the device). Security devices are also marked as Inactive while RSA key generation is in progress for public key cryptography.

The value of the Status field is only read from disk when a security device is initialized or when you warmboot the Security Device Configuration File (SEC_DEV_CONFIG). Thus, you can preconfigure security devices on the Security Device Configuration window before they are put into service.

Note: The status of a device must be Active and the consecutive error count must be less than the maximum consecutive limit for a security device to be used by the Transaction Security component in transaction processing. If the consecutive error count for a device is exceeded, a device may still display a logical status of Active even though the device will no longer be used in processing until the error counter is reset to 0.

Reset timer limit

The Reset Timer Limit field specifies the number of seconds the Transaction Security component waits before automatically attempting to re-establish communications with a hardware security device after the device has reached its maximum consecutive errors limit. Valid values are 0–9999 seconds. A value of 0 in this field indicates that the Transaction Security component does not automatically attempt to reconnect with the device. If you set this field to a value greater than 0, the Transaction Security component sends an echo test, or HSM status request message to the device at intervals defined by this field until the device successfully responds to the request. For example, if this field is set to 30 seconds, the

Transaction Security component waits 30 seconds after the device is initially marked inactive and then sends an echo test or HSM status message request to the device. If Transaction Security component does not receive a successful response to the message after another 30 seconds have elapsed, it sends another echo test or HSM status message to the device. This continues indefinitely until a successful response is received. At that time, the consecutive error count is reset to 0 and the Status field is set to Active. Note that a device may still display a logical status of Active after the maximum consecutive error limit is reached until communications are re-established with the device.

Maximum consecutive errors

The Maximum Consecutive Errors field defines the maximum number of consecutive critical errors that can be returned from a security device before it is marked down and taken out of service. When a security device is taken out of service, it is no longer included in the rotation (distribution in a round-robin manner) of transactions sent from the Transaction Security Services process. The value of the Reset Timer Limit field indicates whether the process attempts to automatically re-establish communications with the device when the maximum consecutive error limit is reached.

Timeout limit

The Timeout Limit field value specifies the maximum time, in seconds, to wait for a response from a security device before the Transaction Security Services process considers the request to be timed out. When a call to the security device times out, the Transaction Security Services process increments the consecutive retries counter by 1. If the maximum retries (hardcoded to 3) has not been reached, the process attempts the security device call again using the next available security device from the pool of security devices, which may or may not be the same security device on which the timeout occurred.

Error notify interval

The Error Notify Interval field specifies the interval (i.e., the number of times the error occurs) at which critical event messages are generated indicating the error. Valid values are 1–99. The default is 10.

Public key infrastructure

A check mark in the Enable PKI field indicates that the security device is used for public key infrastructure (PKI) functions required for the BASE24 Automated Key Distribution System add-on product or for secure logons and logoffs to and from an interchange. Refer to the [“Public Key Cryptography”](#) section for detailed information on PKI command functions.

Maximum buffer length

The Max Buffer Length field displays the maximum input buffer length, in bytes, supported by the device. This is a display-only field that is automatically set when a security device is initialized. No data is displayed in this field until the device is initialized.

For Atalla NSPs, this field displays the maximum MAC data buffer length (e.g., 4096 bytes).

For SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs), the maximum buffer length depends on the protocol used to connect to the HSM as follows:

Protocol	Maximum buffer length
Ethernet	1496 bytes
TCP/IP	2048 bytes
Async	4096 bytes

For Thales e-Security HSMs, the value of this field is set according to the I/O Buffer Size value returned from the HSM in response to the HSM status message sent to the HSM when it is initialized as shown below.

I/O buffer size	Maximum buffer length
0	2000 bytes
1	8000 bytes
2	16000 bytes
3	32000 bytes

Note: Although the maximum buffer length for Thales e-Security HSMs allows from 2000–32000 bytes, the MAC Generate/Verify (MS) command is restricted to 1975 bytes of MAC data.

For SafeNet (Eracom) ProtectHost White AMB host security modules, the maximum buffer length returned in the response to an Echo Test (0000) function is displayed in this field. If the Echo Test (0000) function fails, the default values listed above for SafeNet (Eracom) ProtectHost White Mark II host security modules are used.

For Bull BNTng HSMs, the maximum buffer length for the TCP/IP protocol is 2048 bytes.

For Banksys DEP HSMs, the maximum buffer length is 2048 bytes.

Firmware version

The Firmware Version field displays the version of software for Atalla NSPs (e.g., 2.62), the firmware version (e.g., 0352.E001 or 1053.0801) for Thales e-Security HSMs, and the specification, revision, and customization level for SafeNet (Eracom) host security modules, the version, release, build, and software type for Bull BNTng HSMs, and the firmware name and revision (e.g., FINTXS 1.0.e) for Banksys DEP HSMs. This field is automatically set for a security device when that device is initialized. No data is displayed in this field until the device is initialized.

- For Atalla NSPs, the Transaction Security Services process sends the 1110 command to all configured NSPs during initialization to obtain the software version information.
- For Thales e-Security HSMs, the process sends the HSM Status (N0) command to all configured HSMs to obtain the firmware version.
- For SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs), the process sends the Echo Test (0000) function to all configured HSMs to obtain the specification, revision, and customization level. The SafeNet (Eracom) ProtectHost White Mark II host security module does not support an equivalent function.
- For Bull BNTng HSMs, the process sends the self-test (F2) function to all configured Bull BNTng HSMs to obtain the major version number, release number, build number, and software type identifier (e.g., I for International). These values are returned in the DF 1E tag of the FF 12 self-test response and are displayed in *VVRR-EEL* format, where *VV* is the major version number, *RR* is the release number, *EE* is the build number, and *L* indicates the software type (e.g., I=International).
- For Banksys DEP HSMs, the process sends the HSM Status (02000900) command to all configured Banksys DEP HSMs to obtain the major release number, minor release number, and subrelease number of the application firmware in each device. The first eight bytes of the response contain free form ASCII text, byte 9 contains the major release number, byte 10 contains the minor release number, and byte 11 contains the subrelease number.

Because vendors sometimes support different functionality across software, specification and revision, or firmware releases for a given security device model, the Transaction Security Services process uses this information to determine which command functions are available for each configured device.

Initialization string

The Initialization String field defines the text that the Transaction Security Services process sends to an Atalla Network Security Processor (NSP) model A10000E in a 100 command during initialization or warmboot. This field does not apply to Thales e-Security, SafeNet (Eracom), Bull BNTng, or Banksys DEP hardware security devices or Atalla NSP models other than the A10000E. This field replaces the LCONF param SECURE-DEV-CONF-CMD100 when using the Transaction Security Services process with an Atalla A10000E model. Refer to the description of the SECURE-DEV-CONF-CMD100 param in the **BASE24 Transaction Security Manual** for detailed information on the data to enter in this field.

Configuration string

The Configuration String field defines the configuration text the Transaction Security Services process sends to an Atalla Network Security Processor (NSP) model A10000E in a 101 command during initialization or warmboot. This field does not apply to Thales e-Security, SafeNet (Eracom), Bull BNTng, or Banksys DEP hardware security devices or Atalla NSP models other than the A10000E. This field replaces the LCONF param SECURE-DEV-CONF-TXT when using the Transaction Security Services process with an Atalla A10000E model. Refer to the description of the SECURE-DEV-CONF-TXT param in the **BASE24 Transaction Security Manual** for detailed information on the data to enter in this field.

Note: When using the Boxcar process to interface with Atalla NSPs, you should configure all command options in the NSP's CONFIG.PRM file rather than using this field. For more information, refer to the "Option 023 and the Boxcar Interface Module" topic in the **BASE24 Transaction Security Manual**.

Transaction security services attributes

Configuration of the following attributes for the Transaction Security Services environment must be considered when implementing the Transaction Security Services process. These attributes are set on the Environment Management window of the ACI desktop user interface. If you change an environment attribute on the Environment Configuration window, the new value does not take effect until after you send an [Alter Attribute](#) command (using the Process Control window or an XPNET network control facility) to the processes that use the attribute, or stop and restart the processes that use the attribute. Each of these attributes is described below.

Note: The Atalla Key Block Conversion utility uses three Environment attributes to specify the new AKB-formatting pending MFK under which keys are encrypted when translating keys from non-AKB format to AKB format. This utility also uses the AKB_KEY_IMPORT_VARIANT_SOURCE attribute to convert keys to AKB format using the key variant or the key type and several optional Environment attributes to specify the AKB header value used for different key types when the keys are translated. If an attribute is not configured, a default header value is used instead. All of these attributes and their associated default values are described for the "[AKB conversion utility](#)" topic in section 16.

AKDS_CAP_RPT_PG_LGTH — The number of report lines printed on each page of the AKDS Capacity Report. If you do not configure this attribute on the Environment Configuration window, it defaults to 60 lines per page. The AKDS Capacity Report component inserts a page break whenever the number of lines specified by this attribute is reached in the report output.

Default value: 60
 Component: AKDS Capacity Report
 Process usage: XML Server (XMLS)

AMEX_CSC_ALGO — A code indicating which American Express card verification algorithm to use when verifying American Express cards. Valid values are as follows:

- 1 = Perform card verification using the American Express CSC version 1 algorithm.
- 2 = Perform card verification using the American Express CSC version 2 algorithm.
- 3 = Perform card verification by first using the American Express CSC version 1 algorithm, and if that fails, use the American Express CSC version 2 algorithm.
- 4 = Perform card verification by first using the American Express CSC version 2 algorithm, and if that fails, use the American Express CSC version 1 algorithm.

For newly issued contact chip cards, you must use the CSC Version 2 algorithm. Service codes are used with the CSC Version 2 algorithm as follows:

- Service code 999 is used for iCSC generation.
- Service code 201 is used for 5CSC generation.
- Service code 000 is used for both 4CSC and 3CSC generation.

For the newly issued contactless chip cards, you must use the CSC Version 2.0 algorithm for iCSC generation and a service code of 702.

For previously issued contactless chip cards, CSC Version 1.0 algorithm must be used for 5CSC, 4CSC, and 3CSC validation. Because a service code is not used in CSC version 1 algorithm, the service code value does not matter with 5CSC, 4CSC, and 3CSC validation with these contactless chip cards.

The same fields in the American Express GNS message are used for both iCSC verification and 5CSC verification.

For magnetic stripe only cards, you must use CSC algorithm Version 1.0 to generate 5CSC, 4CSC, and 3CSC values.

The [“American Express card verification algorithms”](#) topic depicts how the value of the AMEX_CSC_ALGO Environment attribute influences card verification processing.

Default value: 1
Component: Transaction Security (Atalla AKB and Thales e-Security)
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

ATALLA_CBC_DATA_FRMT — A flag indicating whether data is unpacked or left as is before sending an encrypt or translate message data command to an Atalla NSP as discussed below.

If this attribute is set to Y when translating data from encryption with a source key to encryption with a destination key (and the DES Mode field is set to CBC or 3DES CBC for the source key on the Channel or Interface Security Configuration window), the data buffer is unpacked before encrypting with the destination key.

If this attribute is set to Y when encrypting data (and the Data Type field on the Channel or Interface Security Configuration window is set to “Unpacked ASCII Hexadecimal”), the data buffer is unpacked before submitting the command to the Atalla NSP.

Valid values are as follows:

- Y = Yes, unpack data before encrypting or translating data.
- N = No, do not unpack data before encrypting or translating data. Leave the data as is.

Default value: Y
Component: Transaction Security (Atalla)
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

ATALLA_CHK_DGTS_LGTH — Specifies whether the Check Digit Validation and Generation utility, AKB Conversion utility, and Master File Key Conversion utility generate and validate four or six check digits when using Atalla NSPs. Valid values are as follows:

- 4 = The Check Digit Validation and Generation utility, AKB Conversion utility, and Master File Key Conversion utility generate and validate four check digits. Default.
- 6 = The Check Digit Validation and Generation utility, AKB Conversion utility, and Master File Key Conversion utility generate and validate six check digits.

If this attribute contains an invalid value or is not configured, the Check Digit Validation and Generation utility, AKB Conversion utility, and Master File Key Conversion utility generate and validate four check digits.

Note: Although users can enter four to six check digits on the ACI desktop user interface, only the first four digits are validated.

Default value: 4
Component: Key Conversion
Process usage: XML Server (XMLS)

CHAN_DATA_KEY_EXPORT_VARIANT — The variant used to generate data encryption keys in an Atalla NSP. If this attribute is not configured, data encryption keys are generated under variant 2. Valid values for this attribute are as follows:

- 0 = Key exchange key variant. Translate the data encryption key directly from the data encryption KEK parts.
- 2 = Data encryption key variant. Include the variant 2 constant with the data encryption KEK parts when translating the data encryption key. The Atalla 2 constant is hexadecimal 1000 0000 0000 0000. This value is displayed only when Atalla security devices are used in your system.

CSEC_MEMORY_USE — A flag indicating whether the Transaction Security Services process maintains Channel Security File (Channel_Security) records in internal memory. Valid values are as follows:

- Y = Yes, Channel_Security records are maintained in internal memory.
- N = No, Channel_Security records are not maintained in internal memory. They are always read from disk.

Default value: Y
 Component: Transaction Security
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

CUSTOMER_NAME — The name of the customer displayed in the header on the AKDS Capacity Report. You must change the value of this attribute from its default value to your actual customer name.

Default value: CUSTOMER NAME
 Component: AKDS Capacity Report
 Process usage: XMLS

DECIMAL_TBL_ENCRYPT — An optional attribute indicating whether you are using encrypted decimalization tables in your system when using Thales e-Security HSMs. The Master File Key Conversion utility uses this attribute to properly translate decimalization tables from the old LMK pair to the new LMK pair. Valid values are as follows:

- Y = Yes, decimalization tables are encrypted.
- N = No, decimalization tables are not encrypted.

If this attribute is not configured or contains an invalid value, it defaults to a value of N.

Default value: N
 Component: Key Conversion
 Process usage: XMLS

DFLT_DAT_FRMT — Specifies the date format used for any reports generated by the BASE24-eps product, including the AKDS Capacity Report. Possible values for this attribute are shown below. In these values, DD is the day of the month (01–31), DDD is the Julian

date (001–365), MM is the number of the month (01–12), MMM is text representing the month (e.g., Jun, Jul, Aug, etc.), YY is the last two digits of the year (00–99), and YYYY is all four digits of the year (0001–9999). An example of the date formatted for June 29, 2006 is shown for each value.

DD/MM/YYYY	= 29/06/2006
DD-MMM-YYYY	= 29-Jun-2006
DD MMM YYYY	= 29 Jun 2006
DD-MMM-YY	= 29-Jun-06
DD MMM YY	= 29 Jun 06
MM/DD/YY	= 06/29/06
YY/DDD	= 06/180
YYYY/DDD	= 2006/180

Default value: DD MMM YYYY
 Component: Foundation Report Stream Class
 Process usage: All

EVT_DEST — Specifies the name of the EMS Collector process (e.g., \$0) to which event messages generated by processes in the Transaction Security Services environment are sent. You may want to set this attribute to the same value set for the XPNET node, so that all event messages are sent to the same location.

Default value: \$0
 Component: Event
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

MAX_CSEC_IN_MEM — An optional attribute specifying the maximum number of Channel_Security records that the Transaction Security Services process can maintain in memory when the CSEC_MEMORY_USE attribute is set to a value of Y. If the CSEC_MEMORY_USE attribute is set to a value of N, the Transaction Security Services process uses a value of 1 for this attribute. Valid values are 1–9999999999.

Default value: 20000 if the CSEC_MEMORY_USE attribute is set to Y.
 1 if the CSEC_MEMORY_USE attribute is set to N.
 Component: Transaction Security
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

NON_ATALLA_KEY_EXPORT — A code indicating the variant to use for exporting Atalla keys to non-Atalla interchange endpoints. When this attribute is set to a value of F (False), keys are exported using the key type (i.e., 1 = PIN, 2 = Data Encryption, and 3 = MAC) as the variant. When this attribute is set to a value of T (True), keys are exported using the key

exchange key (KEK) variant (set in the Key Variant field on the Exchange Information tab of the Interface Security Configuration window) if it is set to a value of 0 (Key exchange key). If it is not set to a value of 0, the key type is used for the variant. Valid values are as follows:

- T = True. Use the KEK variant from the Interface Security file (ICHG_SECURITY) if it is 0 (Key exchange key). If it is not set to 0, use the key type for the variant.
- F = False. Use the key type (i.e., 1 = PIN, 2 = Data Encryption, and 3 = MAC) as the variant.

Default value: F
Component: Transaction Security
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

PIN_LGTH_ERR_RETRY_IND — A code specifying whether the Transaction Security Services component retries an HSM operation using the previous key from the Channel Security Configuration File (Channel_Security) when the HSM returns a sanity check error or a PIN length error. Valid values are as follows:

- Y = Yes, retry both sanity check errors and PIN length errors using the previous key.
- N = No, do not retry PIN length errors using the previous key.

Default value: Y
Component: Transaction Security
Process usage: Transaction Security Services (IS)

PIN_VRFY_FAIL_RETRY_IND — A code specifying whether the Transaction Security Services process retries a PIN verification HSM operation using the previous key from the Channel Security Configuration File (Channel_Security) when the HSM returns a valid fail error (i.e., the cardholder PIN is considered to be invalid).

When using the PIN/PAD PIN block format, an HSM can sometimes return a valid fail error even though the correct PIN was entered. In this case, the HSM successfully decrypts the PIN block (using the wrong key) and then fails the PIN validation. To ensure that correctly entered PINs are processed successfully, you should always set this attribute to Y when using the PIN/PAD PIN block format. Because this situation rarely occurs with other PIN block formats, you can leave this attribute set to the default value of N when using any other PIN block format. Valid values are as follows:

- Y = Yes, retry valid fail errors using the previous key.
- N = No, do not retry valid fail errors using the previous key.

Default value: N
Component: Transaction Security
Process usage: Transaction Security Services (IS)

PIN_XLATE_DISK_READ — A code specifying whether the Transaction Security Services process always reads keys from the Channel Security or Interface Security disk files rather than from the associated internal memory table when translating a PIN. Reading the key from disk ensures that the correct key is always used.

Some security devices, such as a SafeNet (Eracom) host security module (HSM), can return a successful result for translating a PIN/PAD PIN block even if the wrong key is used after a key change. For PIN/PAD PIN blocks, a SafeNet (Eracom) HSM checks only for a hexadecimal pad character to delimit the PIN. As long as the decrypted PIN block contains an A–F hexadecimal character, the SafeNet (Eracom) HSM considers the PIN block to have been successfully translated, even if the wrong key is used. After a key change on a BASE24-eps system with more than one Integrated Server process, all processes other than the one that initiated the change will continue to attempt to use the old key. Forcing the process to always read the key from disk ensures that the correct key is always used.

Because this situation rarely occurs with PIN block formats other than the PIN/PAD PIN block format, you can leave this attribute set to the default value of N when translating any other PIN block format. Valid values are as follows:

- Y = Yes, read the key from disk instead of the internal memory table for all PIN translation functions.
- N = No, do not read the key from disk instead of the internal memory table for all PIN translation functions.

Default value: N
 Component: Transaction Security
 Process usage: Transaction Security Services (IS)

RAM_MAX_BUF_LGTH — The maximum length of data, in bytes, allowed for the data buffer in the RAM process. This prevents the RAM process from sending data that is too long in DUKPT MAC Generate and DUKPT MAC Verify functions to Thales e-Security HSMs. These functions allow for a maximum of 4096 bytes.

Default value: 3972
 Component: Transaction Security (Racal)
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

RSM_EMV_KEY_SCHEME — A code specifying the key scheme used by Thales e-Security security devices for EMV commands. Valid values are as follows:

- 0 = Use old key scheme (16 or 32 hexadecimal characters)
- 1 = Use new key scheme (1A + 32 or 48 hexadecimal characters)

For the new key scheme, the following rules apply to the first character of a key value:

- If the first character of the key value is a hexadecimal character (i.e., 0–9, A–F), the key is the default length for its intended function (single length or double length as appropriate).

- If the first character of the key value is “U”, the key is a double length key and must be followed by 32 hexadecimal characters.
- If the first character of the key value is “T”, the key is a triple length key and must be followed by 48 hexadecimal characters.

Default value: 1
Component: Transaction Security
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

RSM_KEY_STORE_LGTH — A code indicating the length of keys stored in the memory of a Thales e-Security security device, including Diebold number table entries. Valid values are as follows:

- 1 = Single length keys
- 2 = Double length keys
- 3 = Triple length keys

Default value: 1
Component: Transaction Security
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

RSM_TERM_MASTER_KEY — The encrypted terminal master key used with Thales e-Security security devices. This key is required when the Transaction Security Services process uses Thales e-Security security devices to generate message authentication code (MAC) encryption keys on behalf of an Interchange Interface process. This attribute is not required if the Transaction Security Services process only generates MAC encryption keys on behalf of BASE24 Device Handler processes or when Thales e-Security security devices are not used to generate MAC encryption keys.

The BASE24 ISO Host Interface process and the BIC ISO Interface process use this key in an intermediate step that is performed when BASE24 is responsible for generating and distributing MAC encryption keys. The key in this attribute does not coincide with any other keys being used by the external processor. For ease of maintenance, ACI recommends using one RSM_TERM_MASTER_KEY attribute for a BASE24 network.

When this attribute is required, the default value of zeros must be changed. The Thales e-Security console command K can be used to obtain the encrypted key for this attribute. Refer to the appropriate Thales e-Security manual for information regarding the console command.

Default value: 0000000000000000
Component: Transaction Security
Process usage: XML Server (XMLS) and Transaction Security Services (IS)

TSEC_STARTUP_MSGS — A flag indicating whether the Transaction Security component generates event messages during startup processing. Setting this attribute to a value of false is intended to alleviate problems when BASE24-eps is installed on an IBM zSeries platform. Therefore, BASE24 customers can leave it set to its default value of true. Valid values are as follows:

- True = Yes, generate event messages during startup processing.
- False = No, do not generate event messages during startup processing.

Default value: True
 Component: Transaction Security
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

UI_AUDIT_LEVEL — A code indicating the level of user interface auditing performed by the C++ User Interface Server process. This attribute is optional. When performing full auditing, the C++ User Interface Server process includes all record fields and their values in auditing messages written directly to the User Audit Log File (USRAULOG). When performing headers only auditing, the User Interface process includes the record's primary key fields only in auditing messages. When performing full auditing, the User Interface process includes all primary key and modified fields for deletes and inserts, all modified fields for updates, and primary key fields only for views (reads).

Note: Most of the user interface auditing in the TSS environment is controlled by db.audited.db-service-name properties configured for the Java User Interface servlets on the Application Configuration window of the ACI desktop user interface, where *db-service-name* is the name of a particular Java database service. The UI_AUDIT_LEVEL attribute controls auditing only for files accessed by C++ servers. Refer to the **ACI Java Infrastructure Java Server Reference Guide** for a list of all possible db.audited.db-service-name properties that apply to the TSS environment. Thus, if a file and associated ACI desktop window is not represented in the list in the **ACI Java Infrastructure Java Server Reference Guide**, auditing for the file is controlled by the UI_AUDIT_LEVEL attribute.

Note that if you have never used the ACI desktop user interface to view, add, update, or delete data and then change the value of the UI_AUDIT_LEVEL attribute, an event message indicating an "Unknown Attribute Name" is generated. This is normal and the updated value will take effect on the first request that requires auditing. Valid values are as follows:

- 0 = No user interface auditing
- 1 = Headers only auditing (primary key fields only)
- 2 = Full auditing (all primary key and modified fields)

Default value: 1
 Component: User Interface Auditing Implementation
 Process usage: XML Server (XMLS)

UI_AUDIT_OUTPUT_DATA_TYP — A flag indicating whether the C++ User Interface (XML Server) process converts the format (character set) of auditing detail when it writes auditing records directly to the User Audit Log File (USRAULOG) and whether it converts the format to ASCII or EBCDIC. Valid values are as follows:

- A = Yes, convert the format of audit detail to ASCII.
- E = Yes, convert the format of audit detail to EBCDIC.
- N = No, do not convert the format of audit detail written to the USRAULOG. Write the detail in the native format of the XML Server process.

Default value: N
 Component: User Interface Auditing Implementation
 Process usage: XML Server (XMLS)

URAM_MAX_BUF_LGTH — The maximum length of data, in bytes, allowed for the data buffer in the URAM process. This prevents the URAM process from sending data that is too long in DUKPT MAC Generate and DUKPT MAC Verify functions to Thales e-Security HSMs. These functions allow for a maximum of 4096 bytes.

Default value: 3972
 Component: Transaction Security (Racal)
 Process usage: XML Server (XMLS) and Transaction Security Services (IS)

USE_LOCAL_USER_AUDIT — A flag indicating whether the C++ User Interface (XML Server) process writes auditing records directly to the User Audit Log File (USRAULOG). If not, the process writes nowaited auditing messages to the Audit Store and Forward File (Audit_Store_and_Forward) for transmission to the UAUD application. When the UAUD application receives an auditing message, it routes the message through ACI Web Services to the ACI Web Application Services - Java Services servlet container process to be logged to the USRAULOG.

A value of Y for the USE_LOCAL_USER_AUDIT environment attribute indicates that the XML Server writes directly to the USRAULOG. A value of N for this attribute indicates that the XML Server process writes to the Audit_Store_and_Forward. The only situation in which the XML Server process cannot write directly to the USRAULOG is if the USRAULOG has a database type (e.g., SQL) that is different from the database type (i.e., Enscribe) for the rest of the TSS database.

Valid values are as follows:

- Y = Yes, the C++ User Interface (XML Server) process writes auditing records directly to the USRAULOG.
- N = No, the C++ User Interface (XML Server) process does not write auditing records directly to the USRAULOG.

Default value: N
 Component: User Interface Auditing Implementation
 Process usage: XML Server (XMLS)

User interface list retrieval attributes

The following table shows you Environment attributes for C++ user interface servers that limit the number of items that can be viewed or returned to an ACI desktop user interface window in a single response and the default values for each of these attributes. The items can range from profile values listed in a single field to rows from a data source displayed in a table. The attributes are organized according to the user interface windows to which they apply. The default value is listed for each attribute if it is not configured.

Warning: ACI strongly advises that you do not change the values of these attributes from their default values without a full understanding of the implications and risks involved. The C++ User Interface (XMLS) process can run out of memory if these attributes are set to values that are too high.

ACI desktop window	Environment attribute name	Default value
Authentication Profile Configuration	CRD_VRFY_LIST_MAX_LINES	15
	EMV_LIST_MAX_LINES	15
	IDKEY_PRFL_LIST_MAX_LINES	15
	NCR_LIST_MAX_LINES	50
	PIN_PRFL_LIST_MAX_LINES	15
	AKEY_SUMM_VIEW_MAX_LINES	15
Channel Public Key Perusal	CHNPK_LIST_MAX_LINES	50
Host Public Key Perusal	HOST_PK_SEC_LIST_MAX_LINES	15
Environment Configuration	ENV_LIST_MAX_LINES	30
Interface Security Configuration	INTF_SEC_LIST_MAX_LINES	15
BASE24 KEYF Configuration	KEYF_FIID_LIST_MAX_LINES	50
	KEYF_NAM_LIST_MAX_LINES	50

For Java user interface servers, the equivalent functionality for limiting the number of items returned in a response is controlled within transaction map XML files. These parameters should not be changed, however, except for customer specific modifications. If changed, you must restart the Java User Interface (ESWEB) process before the changes take effect. The following is an example of code in the OltpWarmbootMgmnt.xml transaction map file that limits the number of OLTP Warmboot data source (OLTP_Warmboot) rows returned in a single response to the OLTP Management window to 20.

```

<entry>
  <tag>OltpWarmbootMngmntViewRq</tag>
  <task>OltpWarmbootManagement</task>
  <function>View</function>
  <formatter>
    <class>es.server.formatter.xml.OltpWarmbootMgmnt.
XMLFormatter</class>
  </formatter>
  <command>
    <class>es.server.command.ESSimpleCommand</class>
    <transaction>
      <component>es.server.datasource.OLTPWarmbootComponent</
component>
      <method maxRows="20">getWarmbootData</method>
    </transaction>
  </command>
</entry>

```

TSS process workload balancing

Note: Because hardware security device requests are automatically distributed evenly among all available Transaction Security Services processes when you are using the round-robin access method (i.e., SECURE-DEV-ACCESS-TYP param set to 1), the following paragraphs only apply when using the primary and secondary Transaction Security Services process access method (i.e., SECURE-DEV-ACCESS-TYP param set to 0).

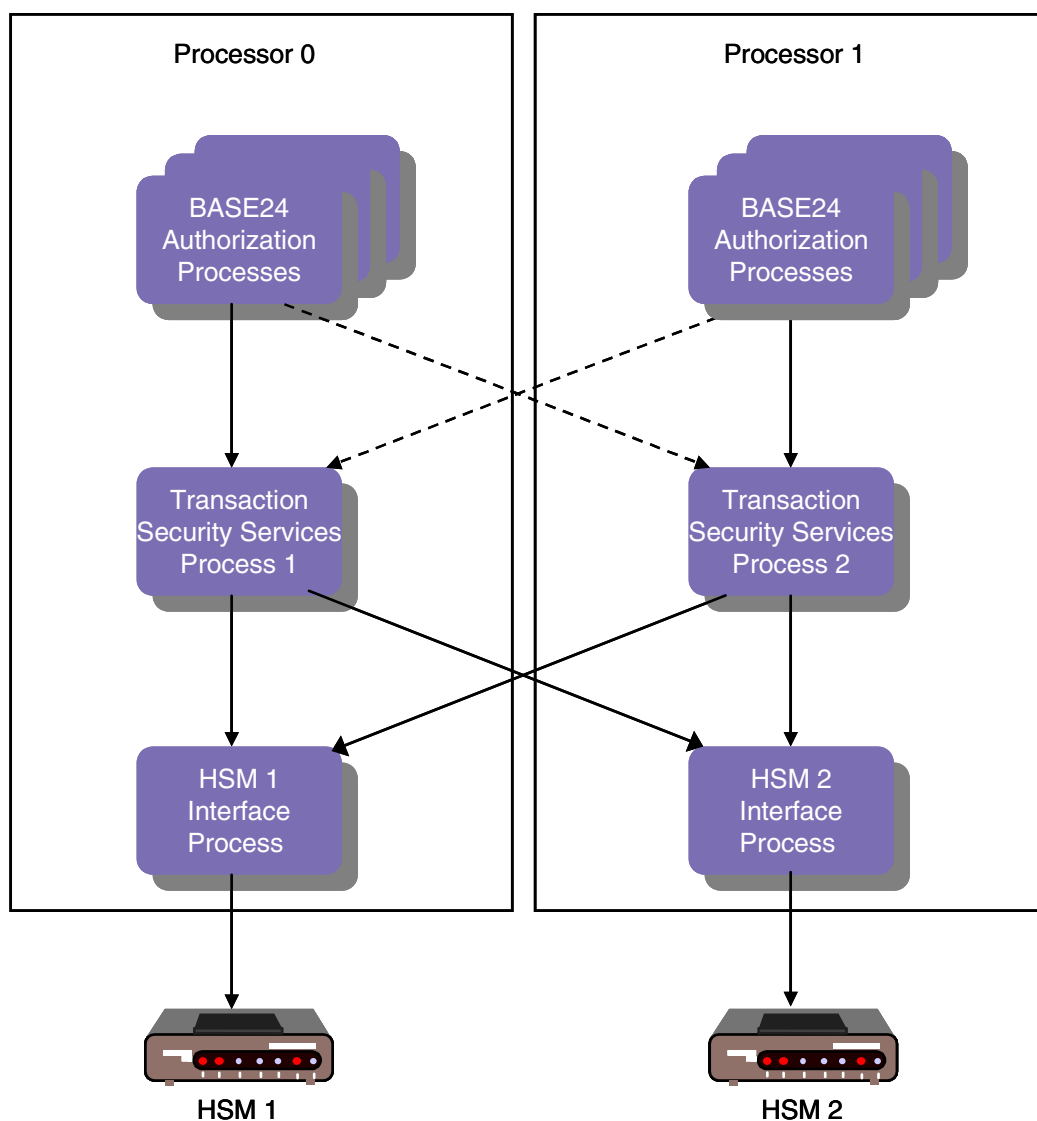
When you implement the Transaction Security Services process using the primary and secondary Transaction Security Services process access method, two levels of workload balancing are used. You can change the number of requests sent to primary and secondary Transaction Security Services processes using the SECURE-DEV-MONITOR-THRESHOLD param. Each Transaction Security Services process can send requests to any security module (or HSM interface process) defined in the SEC_DEV_CONFIG. Each instance of the Transaction Security Services process maintains a list of security modules (or HSM interface process) in memory and alternates requests one at a time to each device. Because the Transaction Security Services process alternates requests to HSMs one at a time, ACI recommends that you do not use the SECURE-DEV-MONITOR-THRESHOLD param to alternate requests between primary and secondary Transaction Security Services processes running in separate CPUs. Otherwise, a single request may travel across the inter processor bus twice, resulting in a possible degradation of performance.

You can monitor the number of transaction messages processed by a Transaction Security Services process using the Start, Status, and Stop Statistics process control commands. For more information on these commands, refer to [section 9](#).

The following diagram depicts a highly simplified example distribution of Transaction Security Services processes and security modules in a BASE24 system with two CPUs and two HSMs. The primary path for requests is denoted with solid lines while the secondary path for request is denoted with dotted lines. In this example, the SECURE-DEV-MONITOR-THRESHOLD

param would be configured to intermittently send a single request (e.g., once every 5000 transactions) to the secondary Transaction Security Services process to make sure the process is available.

Note: This example is for illustrative purposes only, the optimum configuration for any BASE24 system can only be obtained through ongoing resource allocation analysis and performance tuning.



American Express card verification algorithms

Card verification for American Express cards can use either the American Express CSC version 1 or version 2 algorithm, depending on the card input mode for the transaction and the setting of the AMEX_CSC_ALGO Environment attribute. The following table identifies the possible combinations between card input mode, CSC values (5-CSC, 4-CSC and 3-CSC), service code (SVC), and American Express CSC algorithm (CSC Version 1.0, CSC Version 2.0).

The card data input mode from the Point of Service Data TDE and the presence of 5-CSC, 4-CSC or 3-CSC values inside different TDEs are used to determine the service code value for American Express CSC version 2 algorithm.

Note 1: In all scenarios, the American Express algorithm to be used for card verification, and whether to try both algorithms for the same transaction, depends on the value of the AMEX_CSC_ALGO Environment attribute.

Card Data Input Mode	5-CSC or iCSC	4-CSC	3-CSC	Algorithm	SVC	Comments
crd_input_mde_manual crd_input_mde_key_entry	No	Yes	Yes	1 or 2 (Note 1)	000	Algorithm 1 does not use a service code value. Algorithm 2 uses a service code value of 000 for 4-digit and 3-digits CSCs. These would be manually key entered transactions for EMV cards.

Card Data Input Mode	5-CSC or iCSC	4-CSC	3-CSC	Algorithm	SVC	Comments
crd_input_mde_manual crd_input_mde_key_entry	Yes	No	No	1 or 2 (Note 1)	201	This is the rare scenario when a 5-digit CSC is received in DE53 from American Express GNS. Somewhere at the acquiring system or in American Express, the swiped transaction is turned into an American Express GNS keyed transaction with a 5-digit CSC in DE53. DE22 indicates key entry and DE35 is not present. In this case, the service code (SVC) is not available in track data and the Transaction Security component cannot determine if an ICC card or a magnetic stripe only card was used in the transaction. Algorithm 1 does not use a service code value. Algorithm 2 uses a service code value of 201 (i.e., assumes an ICC card - service code = 201).

Card Data Input Mode	5-CSC or iCSC	4-CSC	3-CSC	Algorithm	SVC	Comments
crd_input_mde_mag_stripe crd_input_mde_compl_mag_stripe	Yes	Yes	Yes	1 or 2 (Note 1)	201 and 000	Algorithm 1 does not use a service code value. Algorithm 2 uses a service code value of 201 for the 5-digit CSC. If a 4- or 3-digit CSC is also present, then an additional HSM call is made using a service code value of 000 for the 4- or 3-digit CSC. This would be for EMV fallback transactions or EMV transactions performed at a terminal without a chip reader.
crd_input_mde_icc	Yes	No	No	1 or 2 (Note 1)	999	Algorithm 1 does not use a service code value. Algorithm 2 uses a service code value of 999.
crd_input_mde_prox_icc	Yes	No	No	1 or 2 (Note 1)	702	Algorithm 1 does not use a service code value. Algorithm 2 uses a service code value of 702.
crd_input_mde_prox_mag_stripe	N/A	N/A	N/A	N/A	N/A	Dynamic card verification is not currently supported for American Express cards.

4: Security module requirements

This section describes the security module requirements needed to support the functions available with the Transaction Security Services process.

ISO format 1 PIN block format

Bull BNTng, Banksys DEP HSMS, and Thales e-Security HSMS support the ISO format 1 PIN block format in addition to the PIN/PAN, PIN/PAD, and DUKPT PIN block formats described in the **BASE24 Transaction Security Manual**. To use the ISO format 1 PIN block for a terminal device, set the PIN Block Format field on the PIN Information tab of the Channel Security Configuration window to a value of ISO Format 1. To use the ISO format 1 PIN block for an interface, set the PIN Block Format field on the PIN Information tab of the Interface Security Configuration window and BASE24 KEYF Configuration window to a value of ISO Format 1.

The ISO format 1 PIN block is represented by the following 16 hexadecimal values:

1NppppppppppppRR

Where:

1 is a control field character that identifies the PIN block as an ISO format 1 PIN block.

N is length of the PIN in hexadecimal representation. Since PINs can be from 4 to 12 digits in length, the hexadecimal representation of the length is 4–C as shown in the following table:

PIN digits	Hexadecimal representation
4	4
5	5
6	6
7	7
8	8

PIN digits	Hexadecimal representation
9	9
10	A
11	B
12	C

p is the 4–12 digit PIN. If the PIN is less than 12 digits, characters to the right are randomly padded with any valid hexadecimal character (i.e., 0–9 and A–F).

R is a random padding hexadecimal character (i.e., 0–9 and A–F).

Note: The random padding characters should not be transmitted and are not required in order to translate the PIN block to another format since the PIN length is known.

Atalla security modules

Atalla security functions are available through separate utilities, the ACI desktop user interface, the Atalla NSPDIAG tool and the Secure Configuration Terminal (SCT), or the Atalla Secure Configuration Assistant (SCA). You can only use the SCA with Atalla x100 HSMs with software version 2.10 or later.

Check digits are automatically validated by the security device when the keys and check digits are entered from the ACI desktop user interface. This provides immediate validation of keys, rather than having to use a batch utility to validate keys. You can also use the Check Digit Validation and Generation utility to validate or generate check digits for all the keys in one or more TSS files. Refer to [section 16](#) for more information on this utility.

External key block format

The external key block format is the format of the key block that an external entity (e.g., device or interface) supports—Standard Atalla (variant format) or Atalla Key Block (AKB) format. The format is set in the External Key Block Format field on the Channel Security Configuration window and the Interface Security Configuration window. Currently, no devices or interfaces in the BASE24 system support the AKB format. Therefore, this field should always be set to Standard Atalla.

Preparing data for the TSS files

A secure key loading device such as the Secure Configuration Terminal (SCT) or Secure Configuration Assistant (SCA) is used to perform offline key management and the programming of the Master File Key (MFK) directly into the NSP.

A secure key loading device, such as the SCT or SCA, can be used to encrypt or translate the keys under the proper MFK variant for storage in the TSS files, as shown in the following table. The table identifies the TSS field name, the window on the ACI desktop user interface on which the field appears, MFK variant, key type, and AKB header value. The AKB header values are shown in parentheses in the Key Type column of the table.

Note: Although the AKB NSP supports both triple- and double-length MFKs, Atalla strongly encourages that you use a triple-length MFK. If you do use a double-length MFK, you cannot use any triple-length keys.

Field name on User Interface window	Window	MFK variant	Key type (AKB header)
PIN Information: Exchange Keys: Current	Channel	5	ATM A key (1ADNE000)
PIN Information: Inbound Keys: Current	Channel	1	PIN encryption key (1PUNE000) ¹ AS2805-compliant PIN encryption key (1PUNN000)
MAC Information: Exchange/ Inbound: Exchange Keys: Current	Channel	0	Key exchange key (1KDNE000)
MAC Information: Exchange/ Inbound: Inbound Keys: Current	Channel	3	Message Authentication Code Key (1MDNE000) AS2805-compliant MAC key (1MDGN000)
Data Encryption Exch Info: Exchange Keys: Current	Channel	0	Key exchange key (1KDNE000)
Data Encryption Key Info: Outbound: Outbound Keys: Current 1 and 2	Channel	2	Data Encryption Key for both encryption and decryption (1DDNE000)
Derivation Key	DUKPT Derivation Key	8	Derivation key (1dDNE000 or 1dDNN000)
EMV: Cryptographic Keys	Authentication	9	EMV ACC master keys (1mENE000)

Field name on User Interface window	Window	MFK variant	Key type (AKB header)
EMV: Secure Message Key For Integrity	Authentication	13	EMV MAC master key (1mENE00M)
EMV: Secure Message Key for Confidentiality	Authentication	14	PIN encryption key (1mENE00E)
DES (IBM 3624) PIN Verification ²	Authentication	4	PIN verification key (1V3NE000)
Diebold: Diebold Number Table ³	Authentication	–	Diebold Number Table Row (1ndNE000)
VISA PVV Keys	Authentication	4	PIN verification key (1VVNE000)
Card: Card Verification ⁴	Authentication	3	Card verification key A or B (1CDNE000) American Express Card Security Code KCSC (1mXNE000)
Contactless	Authentication	9	Issuer master key (1mENN000)
Exchange Information: PIN Keys: Import(C)	Interface	0	Key exchange key (1PUNN010)
Exchange Information: PIN Keys: Export(C)	Interface	0	Key exchange key (1KDEE000)
Exchange Information: MAC Keys: Import(C)	Interface	0	Key exchange key (1MDNN010)
Exchange Information: MAC Keys: Export(C)	Interface	0	Key exchange key (1KDEE000)
PIN Information: Outbound Keys: Current 1 and 2	Interface	1	PIN encryption key (1PUNE000) AS2805-compliant PIN encryption key (1PUNN000)
PIN Information: Inbound Keys: Current 1 and 2	Interface	1	PIN encryption key (1PUNE000) AS2805-compliant PIN encryption key (1PUNN000)

Field name on User Interface window	Window	MFK variant	Key type (AKB header)
MAC Information: Outbound Keys: Current 1 and 2	Interface	3	Message authentication key (1MDNE000) AS2805-compliant MAC key (1MDGN000)
MAC Information: Inbound Keys: Current 1 and 2	Interface	3	Message authentication key (1MDNE000) AS2805-compliant MAC key (1MDVN000)
Data Encr Exch Info: Exchange Keys: Import(C)	Interface	0	Key exchange key (1DDNN010)
Data Encr Exch Info: Exchange Keys: Export(C)	Interface	0	Key exchange key (1KDNE000)
Data Encr Key Info: Inbound: Inbound Keys: Current 1 and 2	Interface	2	Data Encryption Key for both encryption and decryption (1DDNE000)
Data Encr Key Info: Outbound: Outbound Keys: Current 1 and 2	Interface	2	Data Encryption Key for both encryption and decryption (1DDNE000)

- ¹ For ATM or POS devices configured to send PINs in the clear to any interchanges on the BASE24 system, the intermediate key value of 9999999999999999 (16 hexadecimal 9s) must be stored in a Channel_Security record.
- ² For security purposes, you can store the clear decimalization table value in the memory of the NSP using the SCT or SCA. In this case, you enter the index of this memory location instead of the clear decimalization table value in the Decimalization Table field on the ACI desktop.
- ³ Anytime you change the DNT in the TSS database, you must rebuild the Diebold Number Table external memory table (OLTP), warmboot the Transaction Security Services processes that access the external memory table (OLTP), and execute the DNTLOAD process control command to reload the DNT in the memory of the NSP. Refer to [section 2](#) for information on rebuilding external memory tables (OLTPs) and warmbooting processes that access them. Refer to [section 9](#) and [section 18](#) for more information on entering the DNTLOAD command.
- ⁴ For verification of MasterCard or Visa cards, you must enter a key pair of two single-length keys encrypted under MFK variant 3 or with an AKB header of 1CDNE000. For verification of American Express cards, you must enter a single double-length key encrypted under MFK variant 3 or with an AKB header of 1mXNE000.

With AKB version software, Atalla has made several enhancements to the Secure Configuration Terminal (SCT), including a PC-based “helper application” to simplify key generation. In addition, Atalla has developed the Secure Configuration Assistant (SCA), which is a secure configuration device based on the HP iPAQ Personal Data Assistant. The SCA is used with Smartcards that perform cryptographic operations and store security-relevant data. The implementation of Smartcards used with the SCA provides identity-based authorization.

The security windows in the ACI desktop user interface are designed to allow you to cut and paste key cryptograms from these Atalla applications directly into the BASE24 database, thereby eliminating the manual entry of cryptograms. The ACI desktop automatically parses parts of the key cryptogram into the correct fields (i.e., Header, Key Part 1, Key Part 2, Key Part 3, MAC, and Check Digits) on the user interface window. Copy key text and paste key text options are available from a pop-up menu when you position the cursor in a key field and right click. For more information, refer to ACI Online Help for the ACI desktop user interface.

Generate check digits command

The User Interface process uses the 7E (Generate Check Digits) command when you enter a new key from the ACI desktop user interface, or when you validate or generate check digits using the Check Digit Validation and Generation utility. Refer to [section 16](#) for more information on this utility. The 7E command accepts single-length keys only if option 6C is enabled in your security policy using commands 108 and 109. This option applies only to Atalla NSPs using AKB software.

The Transaction Security Services process also uses the 7E command to generate check digits for a specified key on behalf of BASE24 AS2805-compliant Device Handler or Interface processes or modules.

The Transaction Security Services process uses the 7E command to generate check digits using the leftmost four digits or the leftmost six digits method as specified in the TSS request message. You must enable option 88 in your security policy to use the leftmost six digits method.

The User Interface process, Check Digit Validation and Generation utility, AKB Conversion utility, and Master File Key Conversion utility use the 7E command to generate check digits using the leftmost four digits or the leftmost six digits method. The `ATALLA_CHK_DGTS_LGTH` Environment attribute specifies whether to generate and validate four or six check digits. Note that the ACI desktop user interface only validates the first four digits if six check digits are entered.

Changing the master file key

ACI provides a Master File Key Conversion tool that enables you to translate hardware-encrypted AKB- and non-AKB-formatted keys in the TSS database from encryption under one Atalla Master File Key (MFK) to another MFK. This conversion tool is executed from the Master File Key Conversion window, accessed as **Configure > Transaction Security >**

Master File Key Conversion on the ACI desktop. Using this tool, you can translate all the keys in the TSS database or just the keys of a selected type (i.e., card verification, PIN verification, EMV, channel, or interface). For detailed information on using this tool, refer to the [Master File Key Conversion](#) topic provided in section 16.

Good security practice as well as several network and card associations require the MFK to be replaced periodically. This conversion tool enables you to convert keys from encryption under the old MFK to encryption under the new MFK and automatically replace them in the TSS database files. NSPs must contain the old MFK as the current MFK and contain the new MFK as the pending MFK before this utility is used. The Master File Key Conversion utility requires the 9E command (Translate Working Key From Under the Current MFK to the Pending MFK) to translate keys. After keys are converted, use the Erase LMK or MFK command on the Process Control window make the pending MFK the current MFK. This process control command executes the Atalla 9A command (Network Security Processor Status Key) and 9F command (Replace the Current MFK with the Pending MFK) on the specified NSPs.

Importing and exporting keys

If you are using an Atalla NSP with AKB software, you must enable the key types of working keys that can be imported and exported as well as key exchange keys that can be used to both import and export working keys. Refer to the “Importing and Exporting Working Keys” topic in the ***Atalla Key Block Banking Command Reference Manual*** for detailed information on the options used to define the keys that can be imported or exported.

Import keys

You must enable any of the following key types to be imported to your BASE24 system in option E0 using the 108 and 109 security policy commands:

Key type	Description
D	Data Encryption (currently only used for Interac support in Canada)
M	Message Authentication (MAC)
P	PIN Encryption Key
V	PIN Verification Key (although supported, this key type typically is not imported in a BASE24 system)

Export keys

You must enable any of the following key types to be exported from your BASE24 system in options E1 using the 108 and 109 security policy commands:

Key type	Description
D	Data Encryption (currently only used for Interac support in Canada)
M	Message Authentication (MAC)
P	PIN Encryption Key
V	PIN Verification Key (although supported, this key type typically is not exported in a BASE24 system)

Key exchange keys

For ATM master keys to be used as key exchange keys for both importing and exporting working keys, you must enable options E3 and E2 using the 108 and 109 security policy commands. In option E2, you must enable key type A (ATM master keys). Option E3 enables ATM master key headers.

Triple DES enabled banking commands

With AKB version software, the following banking commands used by BASE24 are triple DES enabled for all Atalla security device models: 10, 11, 13, 15, 31, 32, 33, 37, 5E, 7E, 98, 99, 9E, 335, 350, 351, 352, BD, and D0. Note that the 335 and BD commands are not available for the A8000 model. The A8100 and A8100-V models support the 335 and BD commands, however, you cannot use binary data in the BD command when connected to a security device over an asynchronous port.

The 11D command has been retired in AKB version software. This command is replaced by the 10 and 1A commands in AKB software. For Atalla NSPs running with non-AKB software, the [Export Variant](#) field on the MAC Information tab of the Channel Security Configuration window specifies whether the 11D or 10 command is used to generate a new MAC key for a device. If this field is set to a value other than 0, the 10 command is used. Otherwise, the 11D command is used. The 10 command is always used to generate a new data encryption key.

AKB versions of Atalla security device models also support the 1A and 11B commands. The 1A command is used to export a working key in AKB format to non-AKB format. The 11B command is used to import a working key in non-AKB format to AKB format. Neither of these commands are enabled in the default factory security policy. To use these commands you must add them to your security policy using commands 108 and 109.

Currently, all Atalla Network Security Processors (NSPs) running AKB software support double- and triple-length keys for triple DES operations in the default factory security policy. The NSP default factory security policy does not allow single length keys for triple DES operations. If you must support single length keys in triple DES operations, you must enable option 6C in your security policy using commands 108 and 109.

Triple DES encryption using a double length key is accomplished by replicating the first key as the third key. The data is encrypted using the first key, the result is decrypted using the second key, and this second result is encrypted again using the first key. When decrypting data, the process is reversed. The encrypted data is decrypted using the first key, the result is encrypted using the second key, and this second result is decrypted again using the first key.

For triple DES encryption using triple length keys, all three key parts must be unique. Atalla NSPs ensure this is true when requested to generate a triple length DES key.

When encrypting data using triple length keys, the data is first encrypted using the first key, the result is decrypted using the second key, and this second result is encrypted using the third key. When decrypting data, the process is reversed. The encrypted data is first decrypted using the third key, the result is encrypted using the second key, and this second result is decrypted using the first key.

Triple DES migration software requirements

The following table summarizes the software version requirements for Atalla devices when migrating a BASE24 system to triple DES and use of the Transaction Security Services process. The first column of the table lists the BASE24 software releases for which triple DES functionality and the Transaction Security Services (TSS) process are available. The second column lists the minimum software versions required to support single DES. The third column lists the minimum software versions required to support the translation of single DES to triple DES PIN blocks for interchanges using the Key 6 File (KEY6). This functionality enables you to meet triple DES mandates for individual interchanges before full triple DES encryption is required in acquiring devices. The fourth column lists the minimum software versions required to support triple DES across the entire BASE24 system.

BASE24 release and TSS usage	Single DES	Triple DES translation ¹	Full triple DES
5.x without TSS	Variant	Variant	Not available
6.0 without TSS	Variant	Variant	Not available
6.0 with TSS	Variant or AKB	Not available	AKB only

¹ This function requires command 335, which is available in all Atalla models and minimum software versions described below (except the A8000).

Software	A8000, A9000, A10000	A8100, A8150, A9100, A9150, A10100, A10150, A10160
Variant	2.9	2.3
AKB	1.4	2.30 (2.31 for public key cryptography)

Note that Atalla devices running variant software cannot interoperate with Atalla devices running AKB software because of the difference in key block formats between the versions. Therefore, when you convert Atalla devices to AKB software, all your devices must be converted at the same time. ACI provides a conversion program to convert keys to the new key block format. Refer to the [“AKB conversion”](#) topic below for more information.

Triple DES translation of ANSI PIN blocks is supported for both the Atalla minimum variant and AKB software versions when using the Transaction Security Services process. When using Atalla variant software, triple DES PIN Pad PIN verification is not supported.

Only the AKB version of Atalla software can be used in a full triple DES environment. With few exceptions, older versions of Atalla software do not support triple DES PIN verification. One exception is triple DES Visa PIN verification using double length keys.

AKB conversion

ACI provides a conversion program to convert working keys stored in the TSS database in the variant key format (non-AKB format) to the Atalla Key Block (AKB) format. The program enables you to convert the keys in one or more files by entering the Convert AKB command on the Process Control window of the ACI desktop. For detailed information on the AKB Conversion utility, refer to the [AKB Conversion](#) topic provided in section 16. You can use the Check Digit Validation and Generation utility to validate or generate check digits for all the keys in one or more TSS files prior to converting keys.

ATM master key exchange

The Transaction Security Services process supports the generation of a new ATM master key encrypted under the previous master key. At the request of a BASE24-atm Device Handler process, the Transaction Security Services process calls an NSP to generate a new ATM terminal master key encrypted under the previous ATM terminal master key for downloading to a terminal. The new master key is stored as both the PIN master key and the MAC master key (key exchange keys) in the Channel Security File (Channel_Security). This is done because ATMs only support one master key. Therefore, the PIN key exchange key and the

MAC key exchange key must be the same in the Channel_Security. If the download fails, the Device Handler process can request that the new key be backed out, up to the expiration of the old key timer limit in the Channel_Security.

In an NSP with the AKB version of Atalla software, the new ATM master key is generated using the 10 command (Generate Working Key, Any Type). In an NSP with variant (non-AKB) version of Atalla software, the new ATM master key is generated using both the 10 command and the 14#1 command (Load ATM Master Key, Primary Node – Diebold).

Note: Support of this functionality requires a CSM for the BASE24-atm Device Handler process.

PIN translation

Atalla's support for PIN translation function varies based on the PIN block formats, key lengths, and version of Atalla software used as described below.

If the destination PIN Block is ANSI, the 31 command is used. When using AKB software, the 6C command must be enabled to allow the use of single-length keys. Variant software on xx100 NSPs accepts both single- and double-length keys as is. Variant software version 2.84 only accepts single-length keys.

If the destination PIN Block is PIN/PAD and the destination key length is single, the 33 command is used. When using AKB software, the 6C command must be enabled to allow the use of single-length keys. Variant software on xx100 NSPs accepts both single- and double-length keys as is. Variant software version 2.84 only accepts single-length keys.

If the destination PIN Block is PIN/PAD and the destination key length is double, or the source key length is double, the 335 command is used. If the source key length is single, the 6C option must be enabled. If the destination key length is single, the 6A option must be enabled because the destination key length in the 335 command must be double length and the 6A option allows a replicated double-length key. Note that the 335 command is not used when the Atalla Key Block (AKB) is used.

The following table shows you the scenarios for which options 6A or 6C must be enabled for the 335 command.

Source PIN block type	Source key length	Destination PIN block type	Destination key length	Option 6A required?	Option 6C required?
ANSI	Single	PIN/PAD	Double		✓
ANSI	Double	PIN/PAD	Double		
PIN/PAD	Single	PIN/PAD	Double		✓
PIN/PAD	Double	PIN/PAD	Single	✓	

Source PIN block type	Source key length	Destination PIN block type	Destination key length	Option 6A required?	Option 6C required?
PIN/PAD	Double	PIN/PAD	Double		
PIN/PAD	Double	ANSI	Single	✓	

The Transaction Security Services process uses command 335 in some cases where commands 31 or 33 could have been used. For example, if the NSP software is variant, but is not version 2.84 or 2.9, the Transaction Security Services process could have used the 31 and/or 33 commands. However, since the Transaction Security Services process does not differentiate between different versions of variant software, command 335 is used for consistency.

Note: The following PIN translation information applies only to Atalla NSPs running variant software. Triple-length keys are not supported by Atalla variant software.

In order to translate a PIN block from encryption under a double-length acquirer key (in any valid PIN block format, excluding DUKPT) to encryption under a single-length issuer key in PIN/PAD PIN block format, you must use a double-length key with both key parts set to the same value that is passed to the Atalla NSP in the 335 command. This requires that you enable the Atalla premium option 6C in the NSP. If this option is not enabled, these PIN translations are not allowed. The following table shows you which PIN translation scenarios are supported by the Transaction Security Services process. The first column indicates the PIN block format of the acquirer. The second column indicates the PIN block format of the issuer. The third column indicates the Atalla command used to perform the PIN translation or indicates "N/A" if the PIN translation scenario is not available.

Acquirer	Issuer	Command
ANSI (single-length)	ANSI (single-length)	31
ANSI (single-length)	ANSI (double-length)	31
ANSI (single-length)	ANSI (triple-length)	N/A
ANSI (double-length)	ANSI (single-length)	31
ANSI (double-length)	ANSI (double-length)	31
ANSI (double-length)	ANSI (triple-length)	N/A
ANSI (triple-length)	ANSI (single-length)	N/A
ANSI (triple-length)	ANSI (double-length)	N/A
ANSI (triple-length)	ANSI (triple-length)	N/A
ANSI (single-length)	PIN/PAD (single-length)	33

Acquirer	Issuer	Command
ANSI (single-length)	PIN/PAD (double-length)	335 ¹
ANSI (single-length)	PIN/PAD (triple-length)	N/A
ANSI (double-length)	PIN/PAD (single-length)	33
ANSI (double-length)	PIN/PAD (double-length)	335
ANSI (double-length)	PIN/PAD (triple-length)	N/A
ANSI (triple-length)	PIN/PAD (single-length)	N/A
ANSI (triple-length)	PIN/PAD (double-length)	N/A
ANSI (triple-length)	PIN/PAD (triple-length)	N/A
PIN/PAD (single-length)	ANSI (single-length)	335 ¹
PIN/PAD (single-length)	ANSI (double-length)	335
PIN/PAD (single-length)	ANSI (triple-length)	N/A
PIN/PAD (double-length)	ANSI (single-length)	31
PIN/PAD (double-length)	ANSI (double-length)	31
PIN/PAD (double-length)	ANSI (triple-length)	N/A
PIN/PAD (triple-length)	ANSI (single-length)	N/A
PIN/PAD (triple-length)	ANSI (double-length)	N/A
PIN/PAD (triple-length)	ANSI (triple-length)	N/A
PIN/PAD (single-length)	PIN/PAD (single-length)	33
PIN/PAD (single-length)	PIN/PAD (double-length)	335 ¹
PIN/PAD (single-length)	PIN/PAD (triple-length)	N/A
PIN/PAD (double-length)	PIN/PAD (single-length)	335 ¹
PIN/PAD (double-length)	PIN/PAD (double-length)	335 ¹
PIN/PAD (double-length)	PIN/PAD (triple-length)	N/A
PIN/PAD (triple-length)	PIN/PAD (single-length)	N/A
PIN/PAD (triple-length)	PIN/PAD (double-length)	N/A
PIN/PAD (triple-length)	PIN/PAD (triple-length)	N/A

¹ Refer to the previous table for the options required by the 335 command.

Message authentication for large messages

The Transaction Security Services process supports message authentication of large messages (e.g., ATM statements, EMV issuer scripts) in both variant and AKB Atalla NSPs using data chaining (multiple calls to the NSP) in 98, 99, 348, 386, and 352 commands. Data chaining is not supported for message authentication in 346, 347, and 350 commands. Refer to Atalla documentation for the maximum length of data supported by these commands.

Device options and banking commands

Some device options and banking commands required by BASE24 must be configured or enabled in Atalla devices before they can be used. These options and commands are discussed below.

Device options

For all Atalla devices, the option for omitting carriage return and line feed characters from the messages they return to BASE24 must be configured in a 101 (Configure Security Processor Options) command in the Configuration String field on the Security Device Configuration window of the ACI desktop user interface. For the A10000E device, the options for setting the minimum PIN length and specifying the response code the NSP returns for a PIN length error are also configured in a 101 command in the Configuration String field on the Security Device Configuration window. BASE24 products require a minimum PIN length of 4 digits. For all other Atalla devices, these options must be configured using the 108 (Define Security Policy) and 109 (Confirm Security Policy) commands in the NSPDIAG tool or using the NSP Option Value Configuration screen in the Atalla Secure Configuration Assistant (SCA). These commands are entered using the NSPDIAG tool with the (A0) and (A1) options, respectively. For more information on the 101 command, defining a security policy, and the SCA, refer to the appropriate Atalla documentation.

If you need to convert ATM master keys to the AKB format, you must enable option E3 for commands 10 and 1A to support ATM master key headers. This option is not enabled in the default factory security policy. To use this option, you must add it to your security policy using commands 108 and 109.

If you need to use the Generate Check Digits (7E) command to generate six check digits instead of four check digits, you must enable option 88 for command 7E. This option is not enabled in the default factory security policy. To use this option, you must add it to your security policy using commands 108 and 109.

The following table lists premium options, which must be purchased from Atalla in a command 105 or command 100 (A10000E only) before they can be used in an Atalla NSP. The first column of the table lists each premium option, while the second column identifies the BASE24 function for which the premium option is required. You only need to purchase the options for the BASE24 functions implemented in your system.

Premium option	BASE24 function
62 (Allow command 31 DUKPT)	Required for the derived unique key per transaction (DUKPT) encryption method. This option allows you to use the Translate PIN—Visa DUKPT (31#7) command to translate DUKPT-formatted PIN blocks to ANSI PIN blocks encrypted under an outbound PIN encryption key.
66 (Do not validate old PIN for command 37)	Required for PIN Change transaction processing. BASE24 processes require the 37 command to be performed without the old PIN, regardless of the PIN verification method used.
8C (Allow a replicated single length Visa PVV PIN verification key)	Premium option 8C must be enabled for the Verify PIN—Visa PVV Method (32#3) and PIN Change—Visa PVV (37#3) commands to allow single-length Visa PVV keys to be replicated in triple DES operations. Therefore, if you convert single-length Visa PVV keys to AKB format using the AKB Conversion utility, you must enable option 8C to use the 32#3 and 37#3 commands.

Device banking commands

Some of the Atalla NSP banking commands used in BASE24 processing may need to be enabled before transaction processing begins. All Atalla NSPs, except for the A8000 and A10000E NSPs, support the 105 (Enable Premium Value Command and Options), 108 (Define Security Policy) and 109 (Confirm Security Policy) commands to enable Atalla NSP banking commands. For the A10000E NSP, premium commands are enabled using the 100 command (Configure Security Processor Commands). The A8000 NSP does not support the 100 or 105 commands because it does not have any premium value commands or options. For all other NSPs, premium commands are enabled using the 105 command.

Some of the banking commands that BASE24 supports are disabled when an Atalla device is delivered from the factory because of the security exposure involved with the command. These commands can be enabled using only the 108 and 109 security policy commands. For example, the D0 (Verify Clear PIN) command is disabled in Atalla devices due to security exposure. Other banking commands that BASE24 supports are identified as *premium* commands for Atalla NSPs. These commands are disabled when an Atalla device is delivered from the factory and can only be enabled in the security policy commands by purchasing a 105 command from Atalla. The 105 command is based on the device serial number and is unique to each device.

The following table lists premium commands, which must be purchased from Atalla in a command 105 or command 100 (A10000E only) before they can be used in an Atalla NSP. The first column of the table lists each premium command, while the second column identifies the BASE24 function for which the premium command is required. You only need to purchase the commands for the BASE24 functions implemented in your system.

Premium command	BASE24 function
11D (Generate ATM MAC Key) ¹	This command is needed if message authentication is performed, the Export Variant field on the MAC Information tab of the Channel Security Configuration window is set to a value of 0, and the NSP is used to generate a new MAC transaction key for a terminal. This command is not needed to generate a new MAC key for an interface process.
12B (Translate Public or Private Key AKB from MFK to PMFK)	Required to support the conversion of public and private keys by the Master File Key Conversion utility.
31#7 (Translate PIN—Visa DUKPT)	Required for the derived unique key per transaction (DUKPT) encryption method. This command translates DUKPT-formatted PIN blocks to ANSI PIN blocks encrypted under an outbound PIN encryption key. You must purchase option 62 in the form of a command 105, and enable it in the NSP's security policy to use this command.
37 (PIN Change)	Required for PIN Change transaction processing. BASE24 processes require the 37 command to be performed without the old PIN, regardless of the PIN verification method used.
75 (Enter Key Part)	This command is not recommended because of security risks, but is needed if you must enter Diebold number table values in the clear on the Authentication Profile Configuration window. For more information on this command, refer to section 18 .
114 (Import 2805 Working Key)	Required to import a PIN or MAC key from an AS2805-compliant interface using cipher block chaining (CBC) encryption. Electronic code book (ECB) encryption is not currently supported.

Premium command	BASE24 function
115 (Generate Non-AKB Working Key for AS2805 Export)	Required to generate and download/export a non-AKB PIN or MAC key to an AS2805-compliant device or interface using CBC encryption. ECB encryption is not currently supported.
120 (Generate RSA key pair)	Required for public key cryptography. Refer to section 15 for more information on these commands.
122 (Generate AKB for root public key)	
123 (Verify public key and generate AKB of public key)	
124 (Generate digital signature)	
125 (Verify digital signature)	
126 (Generate message digest)	
12F (Generate ATM master key and encrypt with public key)	
351 (EMV PIN Change)	Required for validation of offline PIN changes to EMV cards.
D0 (Verify Clear PIN)	This command is not recommended because of security risks, but is needed if you require clear PIN processing using the Identkey PIN verification method.

¹ The 11D command is replaced by the 10 and 1A commands in AKB version software. The 10 and 1A commands are not premium commands. For Atalla NSPs running non-AKB software, the [Export Variant](#) field on the MAC Information tab of the Channel Security Configuration window specifies whether the 11D or 10 command is used to generate a new MAC key for a device. The 10 command is always used to generate a new data encryption key.

All of the NSP banking commands to be enabled in an Atalla device, except for the A10000E, must be specified in the security policy established using the 108 and 109 commands in the NSPDIAG tool or the NSP Configuration Management menu in the Atalla Secure Configuration Assistant (SCA).

The 105, 108 and 109 commands must be entered outside of BASE24. The 105 command to enable the 37 premium command must be obtained from Atalla and entered into the Atalla device using the NSPDIAG tool or the SCA. After this is done, you can establish the security policy using the 108 command in the NSPDIAG tool or the NSP Configuration Management menu in the SCA. When using the NSPDIAG tool, the 208 command response returned for the 108 command then must be entered at the NSP using the Secure Configuration Terminal

(SCT). The NSP encrypts this result using variant 30. The result from this is then entered as a 109 command using the NSPDIAG tool. For detailed information on the 105, 108, and 109 commands, NSPDIAG tool, SCT, and SCA, refer to the appropriate Atalla documentation.

To enable banking commands for A8000 devices, use the 108 and 109 commands entered from the SCT. For more information, refer to the **A8000 Installation, Operations, and Command Reference Guide** and the **Banking Command Reference Manual**.

If you are translating PIN/PAD PIN blocks from single DES to triple DES, and vice versa, as an interim step in migrating interchanges to triple DES, the 335 (PIN Translate) command and options 6A (allow the outgoing double-length PIN encryption key to be a replicated single-length key) and 6C (allow one key-triple DES (single-length) working keys) are required. Option 6A is required if the destination uses single-length keys. Option 6C is required if the source uses single-length keys. This banking command is available in all Atalla models (except the A8000) with AKB 2.1.3 or later software and in the following models for software version 2.9 or later software: A8100V, A9000, A9100V, A9150V, A10000E, A10100V, A10150V, and A10160V. Note that the 2.90 equivalent software on Axx100-V models is version 1.10.

If you need to convert keys from variant format to AKB format and any of the keys (KPE, KEK, MAC, etc.) to be converted to AKB format are single length keys or if any keys to be converted are comprised of key pairs (CVV, PVV) and have the same value for key 1 and key 2, option 6C must be enabled in your security policy for the 11C (Convert Atalla DES Key to 3DES AKB Format) command.

If you need options 6A or 6C for the 335 or 11C commands in your BASE24 system, you must enable them in the NSP's security policy using the 108 and 109 commands. These options are disabled in the NSP's default factory security policy. Note that with AKB version 2.51 or later software, you must enable the 11C command with a specific count of the number of times the command can be executed. For more information on the 335 and 11C commands and their options, refer to Atalla documentation.

Note: For NSPs running software prior to release 2.40 or 1.4, you must obtain option 6A in a 105 command from Atalla before you can enable it.

NSP commands

The following table lists the Atalla NSP commands required to support BASE24 security functions using the Transaction Security Services process. The first two columns of the table list the Atalla commands and associated message ID. The third column describes the BASE24 function for which the command is used.

Atalla NSP command	Message ID	BASE24 function
Echo Test Message	00	Enabling a security device

Atalla NSP command	Message ID	BASE24 function
Generate Working Key, Any Type	10	PIN and MAC key management (device and interface) ATM master key exchange (requires a CSM for the Device Handler process) Also used to generate data encryption keys
Translate Working Key for Distribution	11	Key management (interface)
Import Non-AKB formatted Working Keys	11B ²	Key management
Convert Atalla DES Key to 3DES AKB Format	11C	Key management (AKB conversion utility)
Generate ATM MAC or Data Encryption Key	11D ^{1, 4}	MAC key management (device)
Translate Working Key for Distribution to Non-Atalla Node	1A	Key management (device and interface)
Translate Public or Private Key AKB from MFK to PMFK	12B	Public key cryptography (Master File Key Conversion utility)
Generate ATM Master Key and Encrypt with Public Key	12F	BASE24-atm Automated Key Distribution add-on product This command is supported only in Axx100, Axx150, and Axx160 models.
Translate Working Key for Local Storage (switch-to-switch)	13	Key management (interface)
Load ATM Master Key, Primary Node – Diebold (non-AKB only)	14#1#55	ATM master key exchange (requires a CSM for the Device Handler process)
Change Transaction Key	15#2	PIN key management (device)
Translate PIN—ANSI format	31#1	PIN translation (interface)
Translate PIN—PIN Pad format	31#3	
Translate PIN—Visa DUKPT	31#7	Derived unique key per transaction (DUKPT) PIN encryption

Atalla NSP command	Message ID	BASE24 function
Verify PIN—Identkey method	32#1	PIN verification
Verify PIN—IBM 3624-format method	32#2	
Verify PIN—Visa PVV method	32#3	
Verify PIN—Diebold method ⁷	32#5	
Translate PIN—ANSI to PIN Pad	33#13	PIN translation (interface)
Translate PIN—PIN Pad to PIN Pad	33#33	
Verify AMEX CSC	35A	Card verification (American Express cards)
Encrypt Or Decrypt Data Or Translate	55	Data field encryption
Generate MAC and Encrypt or Translate Data	59	
Load Diebold Number Table	74	Downloading the Diebold Number Table (DNT)
Enter Key Part	75	This command is not recommended because of security risks, but is needed if you must enter Diebold number table values in the clear on the Authentication Profile Configuration window. For more information on this command, refer to section 18 .
Generate Random Number	93	BASE24-atm Automated Key Distribution add-on product
Reformat Initialization Vector	95	Data field encryption
Encrypt Or Decrypt Data	97	
Generate MAC	98 ⁵	Message authentication (device and interface)
Verify MAC	99 ⁵	

Atalla NSP command	Message ID	BASE24 function
Network Security Processor Status Key	9A	Master File Key Conversion utility
Translate Working Key From Under the Current MFK to the Pending MFK	9E	
Replace the Current MFK with the Pending MFK	9F	
Import 2805 Working Key	114	Key management (AS2805-compliant interface)
Generate Non-AKB Working Key for AS2805 Export	115	Key management (AS2805-compliant device or interface)
Generate RSA Key Pair	120	BASE24-atm Automated Key Distribution add-on product These commands are supported only in Axx100, Axx150, and Axx160 models.
Generate AKB for Root Public Key	122	
Verify Public Key and Generate AKB of Public Key	123	
Generate Digital Signature	124	
Verify Digital Signature	125	
Generate Message Digest	126	
PIN Translate—Double Length KPE	335 ⁴	Triple DES/DES translation (interface)
PIN Translate—Visa DUKPT to 3DES KPE with DUKPT MAC Verify	346 ⁶	DUKPT message authentication
DUKPT to DES PIN Translate with DES MAC Generate	347 ⁶	
DUKPT MAC Verify	348 ⁶	
Verify EMV ARQC	350 ⁴	EMV processing
EMV PIN Change	351 ⁴	
Generate EMV MAC	352 ⁴	
Visa dCVV Verify	357	Dynamic card verification
MasterCard CVC3 Verify	359	

Atalla NSP command	Message ID	BASE24 function
PIN Change—Identikey	37#1 ³	PIN change processing
PIN Change—IBM 3624	37#2 ³	
PIN Change—Visa PVV	37#3 ³	
PIN Change—Diebold ⁶	37#5 ³	
DUKPT MAC Generate	386 ⁶	DUKPT message authentication
Get System Configuration Information	1110	NSP initialization (to obtain software version)
Generate CVV/CVC	5D#3	Requires a CSM in the security utilities to use.
Verify CVV, Track 2, Online	5E#3	Card verification (MasterCard and Visa cards)
Generate Check Digits	7E	ACI desktop key entry Check Digit Validation and Generation utility Key management (AS2805-compliant device or interface)
Translate PIN and Generate MAC—ANSI	BD#1 ⁴	PIN translation and message authentication (interface)
Translate PIN and Generate MAC—PIN Pad	BD#3 ⁴	
Verify Clear PIN	D0#1	This command is not recommended because of security risks, but is needed if you require clear PIN processing using the Identikey PIN verification method.

¹ The 11D command has been retired in AKB version software. This command is replaced by the 10 and 1A commands in AKB software. For Atalla NSPs running non-AKB software, the [Export Variant](#) field on the MAC Information tab of the Channel Security Configuration window specifies whether the 11D or 10 command is used to generate a new MAC key for a device. The 10 command is always used to generate a new data encryption key.

² The 11B command translates a working key in non-AKB format from encryption under a KEK, to encryption under the MFK in AKB format.

- ³ The 37 message ID can be configured by Atalla to be performed with or without the old PIN, depending on the value used in command 100/105 or in security policy commands (108 and 109) when this command is activated. For the BASE24 system, it must be configured to be performed without the old PIN. This configuration is called option 66.
- ⁴ The BD,11D, 335, 350, 351, and 352 functions are not supported by the A8000 model. The A8100-V model supports all of these commands, however, you cannot send binary data in the BD command when sending the command over an asynchronous port. The A8100 model does not support the 11D command when it is configured with AKB version software.

Additional parameters are passed in the 350, 351, and 352 functions for [EMV 2000](#) and [EMV CCD](#) processing.

- ⁵ These functions support the X9.19 method of message authentication, also known as pseudo triple DES, as well as single DES and triple DES message authentication.
- ⁶ These functions are only supported on NSP's running with AKB release 2.5.1 or later software. The 347 and 386 commands are defaulted to off due to high security exposure. You must enable these commands in your NSP's security policy before you can use them in processing.

In the 2.82 release for Ax100 NSP models, this command was enhanced to support two additional MAC Type (algorithm) codes. The "VL" code uses the Visa DUKPT algorithm and returns the left half of the calculated MAC. The "VR" code uses the Visa DUKPT algorithm and returns the right half of the calculated MAC. TSS Message 50 (DUKPT MAC Generate) contains an optional input parameter (MAC Option) to determine whether the Transaction Security component sets the MAC Type to VL (for an approved transaction) or to VR (for a denied transaction) in the request to the NSP. This variation of DUKPT message authentication may be required for BASE24 Scandinavian customers.

- ⁷ The Transaction Security Services process only supports the Diebold PIN verification method in Atalla AKB NSPs.

Banksys DEP HSMs

This subsection describes how Banksys Data Encryption Peripheral (DEP) HSMs must be configured to work with the Transaction Security Services process. It does not describe the proprietary workings of the security modules, but rather the functions of each that are supported by the Transaction Security Services process and the information a user will require for configuring the modules for use by the Transaction Security Services process.

Banksys DEP HSM security functions are available through separate utilities and the ACI desktop user interface.

Connectivity

The Transaction Security Services process connects to a Banksys DEP HSM using a single TCP/IP connection. You must use a Remote Application Manager (RAM) process between the Transaction Security Services processes and the HSM to facilitate multiple Transaction Security Services processes communicating with a single Banksys DEP HSM.

Message formats

The Transaction Security Services process supports the DEP System 3 (DS3) message format. DEP System 2 (DS2) and DEP System 4 (DS4) formats are not currently supported.

Supported functions

Banksys DEP HSMs used with the Transaction Security Services process currently support PIN translation, PIN verification (for the IBM DES and Visa PVV PIN verification methods only), PVV generation (for IBM DES and Visa PVV only), card verification, message authentication, derived unique key per transaction (DUKPT) PIN encryption, EMV functions, and public key cryptography. The Diebold, Identikey, and NCR methods of PIN verification, PIN encryption, and data encryption and decryption are not currently supported. In addition, the Master File Key Conversion utility is not currently supported by the Transaction Security Services process for Banksys DEP HSMs.

Banksys DEP HSMs provide DES key management including key translation and key generation of single, double, and triple length keys. For card verification and generation, DEP HSMs support the CVV/CVC, CVV2/CVC2, CAVV, and the American Express CSC. For message authentication code (MAC) verification and generation, DEP HSMs support single DES, triple DES, and pseudo 3DES (X9.19). For EMV, DEP HSMs support ARQC verification and ARPC generation, verifying encrypted counters, generating MACs for issuer script commands, and offline PIN changes. Both EMV'96 and EMV2000 are supported. For the public key cryptography, DEP HSMs support the following RSA functions: RSA key generate, signature verify, signature generate, HASH generate, import public key, and ATM master key generate (BASE24 Automated Key Distribution System add-on product only). Finally, for DUKPT, DEP HSMs support PIN translate (DUKPT to ANSI or PIN/Pad formats), DUKPT MAC generate, and DUKPT MAC verify functions.

External key block format

DEP HSMs do not use different key block formats. Therefore, the External Key Block Format field on the Channel Security Configuration window and the Interface Security Configuration window should always be set to Standard DEP when using DEP HSMs.

DEP key tokens

The DEP HSM utilizes a *key token* for handling host-stored cryptographic keys. The Transaction Security Services process does not support keys stored in the DEP Key Table (DKT); keys must be stored in the TSS database. The structure of the key token is shown in the following table. Note that the lengths provided are for binary data. Key token data that is stored in the TSS database is in binary format, but is displayed on ACI desktop user interface windows in hexadecimal format. As a result, the user interface key fields are 96 hexadecimal digits in length but the resulting key stored on disk will consume only 48 bytes.

Name	Position	Length	Description / values
Version	0	1	The version of the key token being used. Currently, the only valid value is 00.
Key Tag	1	4	This four byte (binary) field is divided into four subfields as follows: The first subfield contains the tag type, which is always set to 04 for a cryptographic key. The second subfield contains the library ID value. See the “DEP library definitions” topic below for valid values. The third subfield contains the cryptographic function ID value. See the “DEP function definitions for keys” topic below for valid values. The last subfield defines the instance of a key. Several key instances (INS) can exist in a library for the same cryptographic functionality.
Host Master Key Instance	Variable	1	Identifies the version of the Host Master Key (HMK) under which the key is encrypted.
Key Cryptogram	5	Variable	Contains the encrypted key cryptogram. The format is a two-byte (binary) length followed by 8, 16, or 24 bytes (binary) depending on the actual key length, and a four-byte (binary) value containing the MAC calculated over all of the preceding fields.
Key Check Value	Variable	3	A three-byte (binary) value containing the key check digits.

When you configure a DEP HSM on the Security Device Configuration window, the following columns are displayed for key entry on all applicable Transaction Security windows:

- Key Part 1

- Check Digits

Key Part 1	Key Part 2	Key Part 3	Check Digits

You enter the key token in its entirety in the Key Part 1 field. The Key Check Value is entered in the Check Digits field.

DEP library definitions

The following table describes the library ID values that can be placed in the Key Part 1 field for a DEP HSM key token.

Library identification	Value	Description
STD	00	Reserved for HMK and other internal HSM stored keys.
ECI	1D	Card Verification
CPIN	1E	PIN Processing
EMV	26	EMV Processing
KMF	2B	Key Management
SCRYP	31	MAC Processing
DUKPT	28	DUKPT Processing
PKI	25	RSA Processing (used for public key cryptography)

DEP function definitions for keys

The following table describes the library function values that can be placed in the Key Part 1 field for a DEP HSM key token.

Library	Function	Value	Description
STD	HMK	07	Host Master Key
ECI	VISA PVV	02	VISA PVV Key
ECI	CVC/CVV	05	CVC/CVV Key
ECI	CAVV	11	CAVV_CVV Key

Library	Function	Value	Description
CPIN	IBM3624_GEN_PIN	21	IBM DES Verification Key
CPIN	PIN_ENC_KEY	26	PIN Encryption Key
EMV ¹	SM_ID	1	Secure Message Key for Integrity
EMV ¹	SM_MAC	2	Secure Message Key for Confidentiality
EMV ¹	TRXION	3	Cryptographic Key(s)
KMF	STD_KEK	30	Key Exchange Key
SCRYP	MAC	00	MAC Verification/ Generation Key
PKI	RSA_SIGN	02	RSA Signature Generation Key
PKI	RSA_VERIF	03	RSA Signature Verification Key
DUKPT	BDK	00	DUKPT Base Derivation Key

¹ For EMV keys, the function value is limited to a single digit to allow for a three- digit instance field.

The function ID subfield of the Key Tag configures the function definition for the key. The valid values available in this subfield depend on the value of the library ID subfield entered for the Key Tag as described below.

If the library ID subfield is set to 1D (Card verification), valid values for the function ID subfield are as follows:

- 02 = VISA PVV Key
- 05 = CVC/CVV Key
- 11 = CAVV_CVV Key

If the Library field is set to 1E (PIN processing), valid values for the Function field are as follows:

- 21 = IBM DES Verification Key
- 26 = PIN Encryption Key

If the library ID subfield is set to 26 (EMV processing), valid values for the Function ID subfield are as follows:

- 1 = Secure Message Key for Integrity
- 2 = Secure Message Key for Confidentiality
- 3 = Cryptographic Key(s)

If the library ID subfield is set to 2B (Key management), the only valid value for the function ID subfield is 30 (Key exchange key).

If the library ID subfield is set to 31 (MAC processing), the only valid value for the function ID subfield is 00 (MAC verification/generation key).

If the library ID subfield is set to 28 (DUKPT processing), the only valid value for the function ID subfield is 00 (DUKPT base derivation key).

If the library ID subfield is set to 25 (RSA processing), valid values for the function ID subfield are as follows:

- 02 = RSA Signature Generation Key
- 03 = RSA Signature Verification Key

The instance subfield of the Key Tag contains the host master key instance, which is automatically set to 00 or 000, depending on the value entered in the function ID subfield of the Key Tag.

Key storage method

Banksys provides an application to create the keys for the DEP HSM. After a key is created, you must store it in the TSS database using the key token storage method described below.

As stated previously, you can enter the key token in its entirety in the Key Part 1 field and the key check value in the Check Digits field on ACI desktop user interface windows.

The tag type and instance subfield values for the Key Tag field from the key token are not stored in the TSS database. The tag type is always assumed to be 04 and the key instance is always assumed to be 00 or 000. The values entered for the version, library, function, and instance of the key token are all stored in the appropriate `_hdr` field in the TSS database as shown in the following table.

Key token field	_hdr position	Comments
Version	0	Currently, must always be set to 01.

Key token field	_hdr position	Comments
Key Tag Subfield 2 (Library)	2	Identifies the key type attribute (e.g., EMV, PIN, MAC, Key Management). See the “ DEP library definitions ” topic above for valid values.
Key Tag Subfield 3 (Function)	4	Identifies the key usage attribute (e.g., PIN verification, PIN encryption, etc.). See the “ DEP function definitions for keys ” topic above for valid values.
Host Master Key Instance	5 or 6	Identifies the version of the host master key under which the associated key has been encrypted.

Check digits

Check digits are required for any keys entered into the TSS database for DEP HSMs because they are part of the key token used to represent keys. Check digits are automatically validated by the HSM when you enter new keys and check digits from the ACI desktop user interface. This provides immediate validation of keys, rather than having to use a batch utility to validate keys.

The User Interface (XML Server) process uses the Generate Check Digits (02001F00) command when you enter a new key from the ACI desktop user interface.

Message authentication for large messages

The Transaction Security Services process supports message authentication of large messages (e.g., ATM statements, EMV issuer scripts) in Banksys DEP HSMs using data chaining (multiple calls to the HSM) in DUKPT MAC Verify (02280800), DUKPT MAC Generate (02280900), MAC Generate (02310000), and MAC Verify (02310100) commands. Data chaining is not supported in EMV Script MAC Generate (02262500) commands. The issuer script data is limited to 2048 bytes for this command.

Preparing data for the TSS files

A secure key loading device such as the C-ZAM/DEP is used to perform offline key management and the programming of the DEP Master Key (DMK) directly into the DEP Crypto Module.

The C-ZAM/DEP is a secure configuration device that uses smart cards, DEP control cards (DCCs) that perform cryptographic operations and store security-relevant data. The use of DCCs provides identity-based authorization.

A secure key loading device, such as the C-ZAM/DEP, can be used to encrypt or translate the keys under the proper DEP master key (DMK) instance for storage in the TSS files, as shown in the following table. The table identifies the TSS field name, the window on the ACI desktop user interface on which the field appears, and the Version, Library, Function, and Instance subfield values for the key token. If a field is not listed in this table, it is not currently supported for the DEP HSM.

Field name on user interface window	Window	Ver	Lib	Func	Inst
PIN Information: Exchange Keys: Current	Channel	00	2B	30	00
PIN Information: Inbound Keys: Current ¹	Channel	00	1E	26	00
MAC Information: Exchange/ Inbound: Exchange Keys: Current	Channel	00	2B	30	00
MAC Information: Exchange/ Inbound: Inbound Keys: Current	Channel	00	31	00	00
Derivation Key	DUKPT Derivation Key	00	28	00	00
EMV: Cryptographic Keys	Authentication	00	26	3	000
EMV: Secure Message Key For Integrity	Authentication	00	26	1	000
EMV: Secure Message Key for Confidentiality	Authentication	00	26	2	000
DES (IBM 3624) PIN Verification ²	Authentication	00	1E	21	00
VISA PVV Keys	Authentication	00	1E	02	00
Card: Card Verification ³ (For CVC/CVV verification)	Authentication	00	1D	05	00
Card Verification ³ (For CAVV/CVV verification)		00	1D	11	00
Exchange Information: PIN Keys: Import(C)	Interface	00	2B	30	00
Exchange Information: PIN Keys: Export(C)	Interface	00	2B	30	00
Exchange Information: MAC Keys: Import(C)	Interface	00	2B	30	00

Field name on user interface window	Window	Ver	Lib	Func	Inst
Exchange Information: MAC Keys: Export(C)	Interface	00	2B	30	00
PIN Information: Outbound Keys: Current 1 and 2	Interface	00	1E	26	00
PIN Information: Inbound Keys: Current 1 and 2	Interface	00	1E	26	00
MAC Information: Outbound Keys: Current 1 and 2	Interface	00	31	00	00
MAC Information: Inbound Keys: Current 1 and 2	Interface	00	31	00	00

- ¹ For ATM or POS devices configured to send PINs in the clear to any interchanges on the BASE24 system, the intermediate key value of 9999999999999999 (16 hexadecimal 9s) must be stored in a Channel_Security record.
- ² The decimalization table is stored (in the clear) in the TSS database but the call to the HSM actually performs a load of the table into the HSM at the same time the PIN Verify/PVV Generate function is performed.
- ³ For verification of all cards, you only enter a single key token that contains both keys.

Supported DEP HSM commands

The DEP message format does not lend itself to easily identifying the command being performed. The general structure of each request message sent to a DEP HSM is provided in four blocks of data as described below.

- The first block contains FF, which is a fixed value indicating the start of a DEP request message.
- The second block contains a variable number of input data tags and associated data. Data tags always begin with x01.
- The third block contains one or more interface (elementary command) tags. Interface tags always begin with x02.
- The fourth block contains one or more output data tags.

For detailed information on the structure of DEP DS3 message formats, refer to the Banksys **DEP DS/3 and DS/4 Principles** document.

In order to identify the specific request type, the Transaction Security Services process performs the following logic:

- Checks the first byte of the inbound message for a value of xFF. If it is, the command is a DEP request and DEP processing should be performed.
- Parses the message buffer in reverse order to determine the command to be performed.
- For each 4-byte value in the message buffer, the process determines if the tag is an output data tag (begins with x01) or an interface data tag (begins with x02).
- Uses the last interface data tag to determine the command type for most messages. In some cases, however, additional interface data tags must be parsed to determine the command type.

The following table identifies the possible request types and associated interface tag(s).

Request type	Interface tag(s)
Firmware Version Get	02000B00
HSM Status	02000900
Key Generate	022B3000
Key Translate	022B3100
Generate Check Digits	02001F00
MAC Generate	02310000
MAC Verify	02310100
CVV Generate	021D0700
CVV/CVC Verify	021D0800
CAVV Verify	021D1300
AMEX CSC Verify	021D1400
EMV ARQC Verify/ARPC Generate	02260A00
EMV Script MAC Generate	02262500
EMV Verify Encrypted Counters	02262E00
EMV Offline PIN Change	02261B00 (MasterCard scheme) or 02260F00 (VISA scheme) Response is based on the scheme.
PIN Verify – IBM DES	021E0400 and 02000000
PIN Verify – VISA PVV	021D0500

Request type	Interface tag(s)
PVV Generate – IBM DES	021E0300
PVV Generate – VISA PVV	021D0D00
PIN Translate	021E1D00
DUKPT PIN Translate	021E1C00
DUKPT MAC Verify	02280800
DUKPT MAC Generate	02280900
RSA Key Pair Generate	02252900
RSA Signature Verify	02250D00 or 02251000
RSA Signature Generate	02250C00 or 02250F00
RSA Import Public Key	02252A00
RSA Generate ATM Master Key	02252B00
RSA Hash Generate	02310800

Bull BNTng hardware security modules (HSMs)

This subsection describes how Bull BNTng HSMs must be configured to work with the Transaction Security Services process. It does not describe the proprietary workings of the security modules, but rather the functions of each that are supported by the Transaction Security Services process and the information a user will require for configuring the modules for use by the Transaction Security Services process.

Bull BNTng HSM security functions are available through separate utilities, the ACI desktop user interface, or the Key Management Center (KMC), which consists of an application on a Linux personal computer and a BNTng HSM. Keys can be arranged by group and assigned to HSMs either individually or by group.

Connectivity

The Transaction Security Services process connects to a Bull BNTng HSM using a single TCP/IP connection. You must use a Remote Application Manager (RAM) process between the Transaction Security Services processes and the HSM to facilitate multiple Transaction Security Services processes communicating with a single Bull BNTng HSM.

Supported functions

BNTng HSMs used with the Transaction Security Services process currently support PIN translation, PIN verification (for the Visa PVV PIN verification method only), PVV generation (for Visa PVV only), card verification, message authentication, and EMV functions. The IBM DES, Diebold, Identikey, and NCR methods of PIN verification, PIN encryption, and data encryption and decryption are not currently supported. In addition, the public key cryptography and the Master File Key Conversion utility are not currently supported by the Transaction Security Services process for BNTng HSMs. The RSA commands needed for public key cryptography are supported by the BNTng HSM, but the Transaction Security Services process does not yet support these functions for the BNTng HSM.

Key storage method

Bull provides an application to create the keys for the BNTng HSM. After a key is created, you must store it in the TSS database using the key token storage method described below.

For the key token storage method, you must enter a *header locator* value in the Header field for the key that corresponds to a BNTng header record in the BNTng_Header, the key cryptogram in the Key Part 1 and Key Part 2 fields, and the cryptogram check digits in the Check Digits field. The Header field for a key cryptogram must contain 8 alphanumeric characters. You must also enter the header record on the BNTng Header Configuration window where the Header Key field contains the header locator value you entered in the Header field for the key. The Transaction Security Services process sends the *key token*, which contains the 32- or 42-byte binary header, the key cryptogram and the key check value, in request messages to the Bull BNTng HSM.

Note: Binary data is entered in hexadecimal character format in the TSS database.

BNTng key header

When storing key cryptograms in the TSS database, all key cryptograms used by the BNTng HSM are stored with a 32- or 42-byte binary¹ key header that specifies the following for each key stored in the TSS database:

- Key ID type
- Decryption key ID (i.e., the identifier of the storage or transport key used to encrypt this key)
- Exchanged key ID (i.e. the identifier of this key)
- Key usage
- Encryption method

¹ Binary values are entered and stored as hexadecimal characters in the TSS database. Key headers are referred to as *labels* in the Bull BNTng nomenclature.

Because the size of the BNTng header is too large to fit in the existing TSS database with the key cryptograms, it is stored in a separate file and is mapped from the key cryptogram record using a *header locator* value. You enter the header locator value in the Header field for a particular key cryptogram. This value must match the value you enter in the Header Key field on the BNTng Key Header Configuration window. An example of a header locator value is shown below.

Exchange Key	Key Variant	0 - Key Exchange Key Variant				Key Length	Double						
	Header	Key Part 1				Key Part 2				Key Part 3			
1	KTKatmmk	3333	3333	3333	3333	4444	4444	4444	4444				

The data for BNTng headers is entered on the BNTng Key Header Configuration window (accessed from **Configure > Transaction Security > BNTng Header** on the ACI desktop user interface). The data entered on this window is stored in the BNTng Header File (BNTng_Header) and is accessed in online transactions processing from the BNTng Header external memory table (BNTHOD). An example of the BNTng Key Header Configuration window is shown below, followed by descriptions of its fields.

BNTng Key Header Configuration

Header Key

Select/Enter

Key ID Type

Decryption Key ID

TK:

IdP:

IdC:

NK:

Exchange Key ID

TK:

IdP:

IdC:

NK:

Key Usage

Encryption Method

Header key

The Header Key field specifies a header locator value defined for the key in the Header field of the TSS database record to which this header record pertains. The value in this field serves as the primary key to BNTng_Header records. Because the Header field for a key cryptogram must contain 8 alphanumeric characters, the value you enter in the Header Key field must also contain 8 alphanumeric characters.

Key ID type

The Key ID Type field specifies a value of 01 or 03 that indicates the format of the key. The 01 value is used for interbanking identifiers of the type 02 in the issuing environment for French interbanking and is not used for RSA keys. The 03 value is recommended for all other cases. Bull strongly recommends that you only use the 03 value.

Decryption key ID

The Decryption Key ID field contains the BNTng key identifier of the key used to encrypt this key for local storage. Refer to the Bull ***BNT EMV Cryptographic Principles*** document for a detailed description of key identifiers.

The value in this field must consist of 28 hexadecimal characters if the Key ID Type field is set to 01, or 38 hexadecimal characters if the Key ID Type field is set to 03.

Exchange key ID

The Exchange Key ID field contains the BNTng key identifier of the key used to encrypt this key for transport and storage. Refer to the Bull ***BNT EMV Cryptographic Principles*** document for a detailed description of key identifiers.

The value in this field must be consist of 28 hexadecimal characters if the Key ID Type field is set to 01, or 38 hexadecimal characters if the Key ID Type field is set to 03.

Key usage

The Key Usage field contains two bytes of binary controls that limit the usage of the key to the functions available to its key type. These two bytes of binary data are represented by four hexadecimal characters in the BASE24-eps database. Therefore, you must enter the hexadecimal representation of the binary data in this field. Refer to the Bull ***BNT EMV Cryptographic Principles*** document for a detailed description of these key usage control elements.

Encryption method

The Encryption Method field specifies the encryption method used for the key. The encryption method defines the computational algorithm of the key cryptogram on the basis of its clear value and possibly other key attributes from the key header. Valid values are as follows:

DES ECB (Single or double length keys)	= DES electronic code book (ECB) encryption of a single or double length key by a double length key with a control vector.
DES CBC (Double length keys)	= DES cipher block chaining (CBC) encryption by a double length key with a control vector.
DES ECB (Double length keys with additional data)	= DES electronic code book (ECB) encryption of a double length key with additional data by a double length key with a control vector.

Refer to the Bull ***BNT EMV Cryptographic Principles*** document for a detailed description of these encryption methods and the types of keys encrypted under each method.

Check digits

Check digits are required for any keys entered into the TSS database for Bull BNTng HSMs because they are part of the key token used to represent keys. Check digits are automatically validated by the HSM when you enter new keys and check digits from the ACI desktop user interface. This provides immediate validation of keys, rather than having to use a batch utility to validate keys.

The User Interface (XML Server) process uses the ED (Secret key verification) command when you enter a new key from the ACI desktop user interface.

BNTng initialization data

Data required to initialize a BNTng is stored in the BNTng Initialization File (BNTng_Init_Data). The BNTng_Init_Data contains a separate record for each BNTng HSM in your BASE24 system. The primary key to each record is the assign name of the HSM as defined in the Security Device Name field on the Security Device Configuration window and a user-defined initialization data identifier.

Each record contains the data to be loaded in BNTng key distribution file format (i.e., tag DF 14). Each HSM must be loaded with a particular KDK file containing the master key of the institution (the equivalent of the Atalla MFK) encrypted under the master key of the HSM. Note that it is valid to load KDKs for two distinct institutions into one HSM.

The data is held as raw binary in the BNTng_Init_Data. Because it is assumed that the BNTng_Init_Data will always be loaded programmatically, there is no configuration window on the ACI desktop user interface for this file.

When you enter a process control INIT command for a Bull BNTng HSM, the initialization data for the HSM from the BNTng_Init_Data is loaded into the HSM.

Preparing data for the TSS database

To aid in the entry of keys, the following table lists the relevant key entry fields in the TSS database with the key mnemonic (e.g., KTK, KT) associated with that key in the Bull BNTng documentation. Refer to the Bull **BNT EMV Cryptographic Principles** document for a detailed description of these key mnemonics, the header data associated with each type of key, and the KMC commands used to generate the key. All keys are double-length keys unless specified otherwise.

Field or tab name on user interface window	Window	Bull key mnemonic
PIN Information > Exchange Key	Channel ¹	KTK
PIN Information > Inbound Key		KT KT1 (single length)
MAC Information > Exchange Key		KTK
MAC Information > Inbound Key		KSC KSC1 (single length)
VISA PVV Keys	Authentication ²	KPV
Card Verification > Card		KCVX, KVCV, or KCSC
Card Verification > EMV > Secure Message Key for Integrity		IMKsm ³
Card Verification > EMV > Cryptographic Keys		IMKac
Card Verification > EMV > Secure Message Key for Confidentiality		IMKsm ³

Field or tab name on user interface window	Window	Bull key mnemonic
Exchange Information > PIN Keys > Import	Interface ¹	KTK
Exchange Information > PIN Keys > Export		KTK
Exchange Information > MAC Keys > Import		KTK
Exchange Information > MAC Keys > Export		KTK
PIN Information > Outbound Keys		KT KT1 (single length)
PIN Information > Inbound Keys		
MAC Information > Outbound Keys		KSC KSC1 (single length)
MAC Information > Inbound Keys		

- ¹ The Data Encryption tabs on the Channel Security Configuration window and the Interface Security Configuration window, as well as the Master Keys tab on the Interface Security Configuration window, are not supported when using a Bull BNTng HSM.
- ² When using a Bull BNTng HSM, the DES (IBM 3624) PIN Verification, Diebold, NCR, and Identikey tabs on the Authentication Profile Configuration window are not supported when using a Bull BNTng HSM.
- ³ For the key stored in the Secure Message Key for Integrity field, the U_mac bit must be set in the Key Usage field for the key header on the BNTng Key Header Configuration window. For the key stored in the Secure Message Key for Confidentiality, the U_conf bit must be set in the Key Usage field.

Copy/paste key text

When using a BNTng HSM, the Copy Key Text and Paste Key Text functions are disabled on Transaction Security windows.

Supported BNTng commands

The following table lists the Bull BNTng HSM request functions required to support Transaction Security Services functions. The first two columns of the table list the Bull BNTng HSM request functions and associated function codes. The third column describes the TSS function for which the command is used. If a Bull BNTng HSM function is not listed, it is not currently supported by the Transaction Security Services process.

Bull BNTng HSM function	Function code	TSS function
MAC computation	E4	EMV processing (Script MAC generation)
Application cryptogram (AC) verification Authorization Response Cryptogram (ARPC) computation	E2 and E3	EMV processing (Card and issuer authentication)
Token verification	ED	Check digit generation
Self-test	F2	HSM initialization
Card verification	FD	Card verification (CVV and CVC)
CVX2 verification	FE	Card verification (CVV, CVV2, CVC, and CVC2)
CSC verification	FF 43	American Express card verification (CSC)
CVX computation	FF 2A	CVD generation (CVV and CVC)
CVX2 computation	FF 2B	CVD generation (CVV2 and CVC2)
DES key generation	E9	Key generation
DES key import	EB	Key translation
Generic MAC computation	F8	MAC generation
Generic MAC verification	F9	MAC verification
Code verification	FC	PIN verification (Visa PVV method)
PIN translate	FB	PIN translation
PIN verification value computation	FF 29	PVV generation

SafeNet (Eracom) host security modules (HSMs)

SafeNet (Eracom) HSM security functions are available through separate utilities, the ACI desktop user interface, or the ProtectHost White console.

Connectivity

The Transaction Security Services process currently supports both the SafeNet (Eracom) ProtectHost White Mark II host security module and the SafeNet (Eracom) ProtectHost White AMB host security module.

ProtectHost White Mark II HSM

The ProtectHost White Mark II host security module (HSM) can be connected to the BASE24 host computer using either of the following two communications interfaces:

- Asynchronous Serial (V.24/RS232)
- Ethernet (IEEE 802.3)

Refer to SafeNet (Eracom)'s ***ProtectHost White Communications Guide*** and the ***BASE24 ERACOM Security Module (ESM) Reference Manual*** for detailed information on the configuration requirements for each of these protocols. Procedures for configuring the HSM for each of the above interfaces are provided in SafeNet (Eracom)'s ***ProtectHost White Mark II Console User Guide***.

ProtectHost White AMB HSM

The ProtectHost White AMB host security module (HSM) can be connected to the BASE24 host computer using the Ethernet (IEEE 802.3) communications interface or a TCP/IP protocol interface.

Refer to SafeNet (Eracom)'s ***ProtectHost White Communications Guide*** for detailed information on the configuration requirements for these protocols. Procedures for configuring the HSM for the above interfaces are provided in SafeNet (Eracom)'s ***ProtectHost White AMB Console User Guide***.

Supported functions

SafeNet (Eracom) HSMs require software with a minimum release date of 01/01/2000 or later. All software releases dated 01/01/2000 or later support triple DES.

The functions supported by the Transaction Security Services process differs between the SafeNet (Eracom) ProtectHost White Mark II host security module (HSM) and the ProtectHost White AMB HSM. Therefore, the functions supported are described separately for each type of HSM.

Although both the ProtectHost White Mark II HSM and ProtectHost White AMB HSM generate Key Verification Codes (KVCs) (i.e., check digits) when generating keys, they do not support a *generate check digits* command that generates Key Verification Codes (KVCs) for working or session keys. The ACI desktop user interface requires such a command to validate the check digits entered by a user on Transaction Security windows. Therefore, when a ProtectHost White Mark II or ProtectHost White AMB HSM is used, the Desktop has no means of validating the check digits value entered by a user. As such, the Check Digit fields on ACI desktop user interface windows are either disabled (i.e., greyed-out) or not displayed when the Security Device Type field on the Security Device Configuration window is set to Eracom. This prevents users from entering check digit values.

ProtectHost White Mark II HSM

ProtectHost White Mark II host security modules (HSMs) used with the Transaction Security Services process support PIN encryption, PIN verification, PVV generation, card verification (for MasterCard and Visa cards), message authentication, message data field encryption and decryption, EMV functions, and proof-of-end-point processing. For PIN verification, the HSM supports the IBM DES (3624-format) and Visa PVV PIN verification methods. The Diebold, Identikey, and NCR methods of PIN verification, card verification (for American Express cards), public key cryptography, and the Master File Key Conversion utility currently are not supported for SafeNet (Eracom) ProtectHost White Mark II HSMs. Although the ProtectHost White Mark II HSM does not support the translation of keys from encryption under the local storage key (Domain Master Key) to encryption under a key exchange key for sending to an interchange, key generation is supported. The ProtectHost White Mark II HSM supports the generation of PIN encryption, data encryption, and MAC keys for terminals and interchanges. Therefore, the ProtectHost White Mark II HSM supports key exchanges to a terminal or interchange as well as key change network control commands.

ProtectHost White AMB HSM

ProtectHost White AMB host security modules (HSMs) used with the Transaction Security Services process support PIN encryption, PIN verification, PVV generation (for Visa PVV only), card verification (for MasterCard and Visa cards), message authentication, and proof-of-end-point processing. For PIN verification, the HSM supports the IBM DES (3624-format) and Visa PVV PIN verification methods. The Diebold, Identikey, and NCR methods of PIN verification, card verification (for American Express cards), EMV functions, data encryption and decryption, public key cryptography, and the Master File Key Conversion utility currently are not supported by the Transactions Security Services process for ProtectHost White AMB devices. Data encryption and decryption are supported by the ProtectHost White AMB HSM, but the Transaction Security Services process does not currently support these functions for the ProtectHost White AMB HSM.

The ProtectHost White AMB HSM supports key generation for sending keys to an interchange and the translation of keys from encryption under a key exchange key to encryption under the local storage key (Domain Master Key) for keys received from an interchange. The ProtectHost White AMB HSM supports the generation of PIN encryption and MAC keys for terminals and interchanges. The ProtectHost White AMB HSM supports key exchanges to a terminal or interchange as well as key change network control commands.

Although the ProtectHost White AMB HSM does not return check digits when working or session keys are generated; check digits can be generated using the KVC Request function. Interfaces use the KVC Request function when they need check digits (e.g., for a Key Verification message).

External key block format

SafeNet (Eracom) security modules do not use different key block formats. Therefore, the External Key Block Format field on the Channel Security Configuration window and the Interface Security Configuration window should always be set to Standard Eracom when using either ProtectHost White Mark II or ProtectHost White AMB host security modules (HSMs).

Bad PIN and key synchronization errors

Because the minimum PIN length for ANSI and all ISO PIN block formats is 4, the ProtectHost White Mark II host security module (HSM) returns an error code of 07 (PIN format error) for PIN lengths of 3 or less since this would typically be caused by keys being out of synchronization. This functionality assumes that the PIN entry device will not accept a PIN of less than 4 digits to be placed in the PIN block, which is typically the case.

If the minimum PIN length configured in BASE24 is greater than 4 digits and the entered PIN is less than the minimum and greater than 3 digits, the Mark II HSM returns an error code of 0B (CheckLength error) for the IBM 3624 PIN verification function. For example, if the minimum PIN length configured is set to 6, PIN lengths of 4 and 5 will return an error code of 0B while a PIN length of 3 returns an error code of 07.

AS2805 6.2 and 6.4 PIN block formats

ProtectHost White AMB HSMs support the AS2805 6.2 and 6.4 PIN block formats defined by Standards Australia in the AS 2805.3-2000 standard, ***Electronic funds transfer - Requirements for interfaces - PIN management and security***. Refer to this standard for detailed information on these PIN block formats. Besides the ANSI PIN (ISO Format 0)

PIN block format, you can select these PIN block formats in the PIN Block Format field on the PIN Information tab of the Channel Security Configuration window using one of the following values:

- AS2805 6.2 = AS2805 6.2 PIN block format
- AS2805 6.4 = AS2805 6.4 PIN block format

Note: When you select one of the above PIN block format values on the PIN Information tab of the Channel Security Configuration window, the only other field on this tab for which you can enter information is the Key Length field. Also, the MAC Information tab is disabled.

Proof-of-endpoint processing

The ProtectHost White Mark II and AMB HSMs perform proof-of-endpoint processing as prescribed by Standards Australia in the AS2805.6.3 2000 standard, ***Electronic funds transfer - Requirements for interfaces - PIN management and security***. The Transaction Security Services process supports the following functions for proof-of-endpoint processing for AS2805 compliant POS devices and interfaces: TERMPROOF 6.4, NODEPROOF, NODERESP.

The TERMPROOF - 6.4 function is designed to perform the cryptographic part of the terminal proof-of-endpoint processing. The PIN Pad Acquirer Serial Number (PPASN) is enciphered using the *KEK and the left-hand 32 bits are compared with the value received from the terminal. This function is only supported on ProtectHost White AMB HSMs using the E500 (APCA) function.

The NODEPROOF function generates the single-length random key to be forwarded to the remote node as part of the inter-nodal proof-of-endpoint processing. The Random Key (KRs) is inverted to form KRr. KRr and KRs are returned to the host encrypted by the KEKs in the request. This function is supported on ProtectHost White Mark II HSMs using the EE3033 function and on ProtectHost White AMB HSMs using the E520 (APCA) function.

The NODERESP function performs the response part of the inter-nodal proof-of-endpoint processing. The function decrypts a single-length key (KRs) using the KEKr in the request. KRr is formed by inverting KRs and is returned encrypted under KEKr. This function is supported on ProtectHost White Mark II HSMs using the EE3034 function and on ProtectHost White AMB HSMs using the E530 (APCA) function.

Message authentication for large messages

The Transaction Security Services process supports message authentication of large messages (e.g., ATM statements, EMV issuer scripts) in SafeNet (Eracom) ProtectHost White Mark II HSMs using data chaining (multiple calls to the HSM) in Generate MAC (EE0701), and Verify MAC (EE0702) commands. Data chaining is not supported in EMV Script Crypto

(EE2007) and EMV 2000 Script Crypto (EE2013) commands. The issuer script data for these commands are limited to the HSM's device buffer size (based on the communications protocol configured).

Preparing data for the TSS files

Keys for SafeNet (Eracom) ProtectHost White Mark II and AMB HSMs are entered, stored, and displayed in different formats in the TSS database. The following paragraphs describe the different formats.

ProtectHost White Mark II HSM

Offline HSM Domain Master Key loading and key management is accomplished using a user-supplied VGA-compatible video monitor and standard 104-type keyboard attached to the video output and keyboard input HSM ports. This terminal is known as the HSM console. For detailed procedures on entering keys, refer to the ***ProtectHost White Mark II Console User Guide***.

The table below summarizes how keys must be stored in the TSS database for a ProtectHost White Mark II HSM. Key fields listed with "N/A" in the Domain Master Key Variant column of the table are stored as four hexadecimal character index values in the TSS database except for the card verification key. The card verification key is stored as a two-numeric-character (00–99) index value. When a command request is sent to the HSM, the HSM uses the index value for the key sent in the request to retrieve the actual single- or double-length key value from memory in the HSM. The number of index locations available for a key depends on the key type. The Domain Master Key variant is not applicable for keys that are stored as index values in the TSS database.

Note: Some ProtectHost White Mark II HSM key index values entered on the Channel Security Configuration, Interface Configuration, and Authentication Profile Configuration windows are entered and displayed as five decimal characters that are converted to hexadecimal characters for storage in the TSS database. This applies to the following keys:

- PIN, MAC, and data encryption exchange keys on the Channel Security Configuration window.
- PIN, MAC, and data encryption import and export exchange keys on the Interface Security Configuration window.
- IBM DES and Visa PVV keys on the Authentication Profile Configuration window.

For the Channel Security Configuration window, you can store channel PIN and MAC exchange keys (i.e., terminal master keys) in the TSS database or in the memory of the HSMs. To store these exchange keys in the TSS database, set the Key Length field on the PIN Information or MAC Information tab to a value of "Single" or "Double" and enter the encrypted key value. To store these exchange keys in the HSM, set the Key Length field to a value of "Key Index" and enter a five decimal index value, which is stored as a four-hexadecimal-character value. Data encryption key exchange keys must always be stored in the HSM and entered as five decimal index values on the Channel Security Configuration window.

For key fields in which the keys are stored as index values, the index value is entered and displayed in the Key Part 1 field for the key field on the ACI desktop window. The Key Part 2 and Key Part 3 fields are disabled (i.e., dimmed).

Key fields listed with a number in the Domain Master Key Variant column of the table below must be encrypted under the specified Domain Master Key variant for storage. For these key fields, the actual single- or double-length key value is entered on the ACI desktop window and stored in the TSS database.

Because the HSM does not support the generation of check digits, the Check Digits fields for all key fields are greyed-out when a SafeNet (Eracom) HSM is configured on the Security Device Configuration window. This prevents you from entering or displaying check digit information in any of the key fields listed in this table.

Field name on user interface window	Window	Domain master key variant
PIN Information > Exchange Key	Channel	N/A or 5 ¹
PIN Information > Inbound Key		1
MAC Information > Exchange Key		N/A or 5 ¹
MAC Information > Inbound Key		2
Data Encryption Exch Info > Exchange Key		N/A
Data Encryption Key Info > Outbound Keys		0
DES (IBM 3624) PIN Verification ²	Authentication	N/A
VISA PVV Keys		
Cryptographic Keys		30
Secure Message Key For Integrity		31
Secure Message Key for Confidentiality		32
Card Verification		N/A ³
Exchange Information > PIN Keys > Import	Interface ⁴	N/A
Exchange Information > PIN Keys > Export		N/A
Exchange Information > MAC Keys > Import		N/A
Exchange Information > MAC Keys > Export		N/A
PIN Information > Outbound Keys		1
PIN Information > Inbound Keys		1

Field name on user interface window	Window	Domain master key variant
MAC Information > Outbound Keys		2
MAC Information > Inbound Keys		2
Data Encr Exch Info > Exchange Keys		N/A
Data Encr Key Info > Inbound Keys		0
Data Encr Key Info > Outbound Keys		0

- ¹ Channel PIN and MAC exchange keys can be stored in the TSS database or in the ESM.
- To store these exchange keys in the TSS database, set the Key Length field on the appropriate tab to a value of “Single” or “Double” and enter the encrypted key value.
 - To store these exchange keys in the ESM, set the Key Length field to a value of “Key Index” and enter a five decimal index value, which is stored as a four-hexadecimal-character value.
- ² The actual value of the decimalization table is stored in the memory of the SafeNet (Eracom) security module.
- ³ This field is stored as a two-numeric-character index field instead of a four-hexadecimal-character index field.
- ⁴ The Master Keys tab on the Interface Security Configuration window is not supported when using a SafeNet (Eracom) security module.

ProtectHost White AMB HSM

Offline AMB HSM Domain Master Key loading and key management is accomplished using a user-supplied VGA-compatible video monitor and standard 104-type keyboard attached to the video output and keyboard input AMB HSM ports. This terminal is known as the AMB console. For detailed procedures on entering keys, refer to the ***ProtectHost White AMB Console User Guide***.

The table below summarizes how keys must be stored in the TSS database for an AMB HSM when using ANSI PIN (ISO Format 0) blocks.

When keys are stored in the TSS database and sent in a command to the AMB HSM, the key field must include the format code, KM index or key index (if applicable), and encrypted key (if applicable). When using SafeNet (Eracom) ProtectHost White AMB host security modules (HSMs), the following samples show you how these fields are displayed on Transaction Security windows for entry of keys cryptograms.

		Format Code	KM Index	Key Part 1	Key Part 2	Key Part 3
	1	▼				

		Format Code	Key Index
	1	▼	

The Format Code column specifies the format codes you can enter in the Format Code field when entering and storing a key in the TSS database. Format codes specify attributes of the key such as the key length (i.e., single, double, or triple), mode of operation (i.e., encrypted, electronic code book (ECB) encrypted, or cipher block chain (CBC) encrypted) used to encrypt the key, algorithm used to encrypt the key, etc.

Possible format codes are:

- 02 = Key stored in AMB HSM in BCD
- 03 = Key stored in AMB HSM in binary
- 20 = Encrypted key – single length – 64 bit
- 21 = CBC encrypted key – double length – 128 bit
- 22 = CBC encrypted key – triple length – 192 bit
- 23 = ECB encrypted key – double length – 128 bit
- 30 = Encrypted key – single length – 64 bit (no KM index; KEK encrypted)
- 31 = CBC encrypted key – double length – 128 bit (no KM index, KEK encrypted)
- 32 = CBC encrypted key – triple length – 192 bit (no KM index, KEK encrypted)
- 33 = ECB encrypted key – double length – 128 bit (no KM index, KEK encrypted)

Format codes 22 and 30–33 are not currently used and are reserved for future use.

Note: Because the key length is specified in the Format Code field value for AMB HSMs, the Key Length field is disabled.

The KM Index/Key Index column lists the valid key index values you can enter on ACI desktop windows for these fields. Key index values must be entered as either four numeric characters or two hexadecimal characters. They are stored in the TSS database as either binary coded decimal (BCD) or binary values.

When a command request is sent to the AMB HSM, the AMB HSM uses the index value for the key sent in the request to retrieve the actual single-length, double-length, or triple-length key value from memory in the AMB HSM. The number of index locations available for a key depends on the key type.

You enter the format code and the numeric or hexadecimal key index value in the Format Code and KM Index/Key Index fields, respectively, on ACI desktop transaction security windows.

For keys stored in the TSS database, you enter the encrypted key values in the Key Part 1, Key Part 2, and Key Part 3 fields as required for the key length. For key fields in which the keys are stored as index values in the TSS database, the Key Part 1, Key Part 2, and Key Part 3 fields are not displayed.

Because the AMB HSM does not support the generation of check digits for working or session keys, the Check Digits fields for all key fields are not displayed when an AMB HSM device is configured on the Security Device Configuration window. This prevents you from entering or displaying check digit information in any of the key fields listed in this table.

Field name on user interface window	Window	Format code	KM index / key index
PIN Information > Exchange Key	Channel ¹	20, 21, 23	00–FF
PIN Information > Inbound Key		20, 21	
MAC Information > Exchange Key		20, 21, 23	
MAC Information > Inbound Key		20, 21	
DES (IBM 3624) PIN Verification ²	Authentication ³	02	0000–9999
VISA PVV Keys		03	0000–FFFF
Card Verification		20, 21, 23	00–FF

Field name on user interface window	Window	Format code	KM index / key index
Exchange Information > PIN Keys > Import	Interface ¹	20, 21	00–FF
Exchange Information > PIN Keys > Export			
Exchange Information > MAC Keys > Import ⁴			
Exchange Information > MAC Keys > Export ⁴			
PIN Information > Outbound Keys			
PIN Information > Inbound Keys			
MAC Information > Outbound Keys			
MAC Information > Inbound Keys			

- ¹ The Data Encryption tabs on the Channel Security Configuration window and Interface Security Configuration window, and the Master Keys tab on the Interface Security Configuration window, are not supported when using a SafeNet (Eracom) security module.

If you select a value of AS2805 6.2 or AS2805 6.4 in the PIN Block Format field on the PIN Information tab, enter a value in the Key Length field only on this tab. None of the key table fields are displayed on this tab for these PIN block formats. In addition, the MAC Information tab is disabled.

- ² The actual value of the decimalization table is stored in the memory of the SafeNet (Eracom) security module.
- ³ When using a SafeNet (Eracom) ProtectHost White AMB host security module, the Secure Message Key For Integrity, Secure Message Key for Confidentiality, and Cryptographic Keys fields under the EMV tab are not supported.
- ⁴ The exchange of MAC keys is reserved for future use. Therefore, these fields are currently disabled.

Supported SafeNet (Eracom) online functions

This section lists the SafeNet (Eracom) ProtectHost White Mark II and AMB host security module (HSM) functions required to support Transaction Security Services functions.

ProtectHost White Mark II HSM

The following table lists the ProtectHost White Mark II HSM functions required to support Transaction Security Services functions. The first two columns of the table list the ProtectHost White Mark II HSM functions and associated function codes. The third column describes the TSS function for which the command is used. If a ProtectHost White Mark II HSM function is not listed, it is not currently supported by the Transaction Security Services process.

SafeNet (Eracom) function	Function code	TSS function
HSM Status	01	Enabling a security device
Generate CVV/CVC	9B	Requires a CSM in the security utilities to use.
Verify CVV/CVC	9C	Card verification
Generate initial device PIN key	EE0400	PIN key management (device)
Generate initial device MAC key	EE0400	MAC key management (device)
Generate initial interface PIN key	EE0402	PIN key management (interface)
Generate initial interface MAC key	EE0402	MAC key management (interface)
Translate PIN key for storage	EE0403	Key management (interface)
Translate MAC key for storage	EE0403	Key management (interface)
Translate PIN (both PIN block format and/or PIN encryption key)	EE0602	PIN translation
Verify PIN—IBM 3624 (ANSI PIN block)	EE0603	PIN verification—IBM 3624 method
Verify PIN—IBM 3624 (PIN/Pad PIN block)	EE0603	PIN verification—IBM 3624 method
PIN offset generate—IBM 3624 (ANSI PIN block)	EE0604	PIN change and PIN selection
PIN offset generate—IBM 3624 (PIN/Pad PIN block)	EE0604	PIN change and PIN selection
Verify PIN—Visa PVV (ANSI PIN block)	EE0605	PIN verification—Visa PVV method
Verify PIN—Visa PVV (PIN/Pad PIN block)	EE0605	PIN verification—Visa PVV method

SafeNet (Eracom) function	Function code	TSS function
PIN offset generate—Visa PVV	EE0607	PIN change and PIN selection
Generate MAC ¹	EE0701	Message authentication
Verify MAC ¹	EE0702	Message authentication
Encipher 2	EE0800	Message data field encryption
Decipher 2	EE0801	
EMV CCD Script Crypto	EE2007	EMV processing
EMV 2000 Script Crypto	EF2013	
EMV CCD PIN Change/Unblock	EE2016	
EMV 2000 PIN Change/Unblock	EE2017	
EMV Verify AC/Generate ARPC	EE2018	
NODEPROOF (Proof-of-endpoint processing – request – interchange)	EE3033	Proof-of-endpoint (interface) processing
NODERESP (Proof-of-endpoint processing – response - interchange)	EE3034	Proof-of-endpoint (interface) processing

¹ These functions support the X9.19 method of message authentication, also known as pseudo triple DES, as well as single DES and triple DES message authentication.

ProtectHost White AMB HSM

The following table lists the ProtectHost White AMB host security module (HSM) functions required to support Transaction Security Services functions. The first two columns of the table list the ProtectHost White AMB HSM functions and associated function codes. Function codes on the ProtectHost White AMB HSM that conforms to Australian Payments Clearing Association (APCA) standards are denoted by "(APCA)" in the Function Code column. The third column describes the TSS function for which the command is used. If a ProtectHost White AMB HSM function is not listed, it is not currently supported by the Transaction Security Services process.

SafeNet (Eracom) function	Function code	TSS function
Echo Test	0000	Security module initialization

SafeNet (Eracom) function	Function code	TSS function
TERMKEYGEN1 (Generate Terminal Session Keys and Roll KEK1)	3000 3500 (APCA)	Key management: active terminal master key (AS2805-compliant POS channel)
TERMKEYGEN2 (Generate Terminal Session Keys using KEK2 and Roll KEK1/KEK2)	3010 3510 (APCA)	Key management: backup terminal master key (AS2805-compliant POS channel)
NODEKEYGEN - 6.3 (Generate	3A00 (APCA)	Key management (interface)
MKEYGEN (Generate ATM Master Key)	3B20 (APCA)	Key management (ATM channel)
RTMK (Key Translation - Receive) (Translate received interchange session keys)	4500 (APCA)	Key translation (AS2805-compliant interface)
MACGEN - 6.3 and 6.4	5500 (APCA)	Message authentication (POS channel and interface)
MACVERIFY - 6.3 and 6.4	5600 (APCA)	Message authentication (POS channel and interface)
PINVERIFY 2000 (transaction from interchange; ANSI PIN Block format; DES (IBM3624) PIN Verify)	6500 (APCA)	PIN verification (interface acquired)—IBM 3624 method
PINVERIFY 2000 – VISA 6.3 (transaction from interchange; ANSI PIN Block format; Visa PVV PIN Verify)	6501 (APCA)	PIN verification (interface acquired)—Visa PVV method
PINVERIFY - 6.4 (transaction from POS AS2805 6.4 terminal; ANSI PIN Block format; DES (IBM3624) PIN Verify)	6510 (APCA)	PIN verification (POS AS2805 channel acquired)—IBM 3624 method
PINVERIFY – VISA 6.4 (transaction from POS AS2805 6.4 terminal; ANSI PIN Block format; Visa PVV PIN Verify)	6511 (APCA)	PIN verification (POS AS2805 channel acquired)—Visa PVV method
PVVGen - using given PIN	65B4 (APCA)	PIN change and PIN selection
PINBLOCKTRANS - 6.3 to 6.3 (PIN Block translation – interchange to interchange)	6600 (APCA)	PIN key management (interface)
PINBLOCKTRANS - 6.4 to 6.3 (PIN Block translation – POS terminal to interchange)	6610 (APCA)	PIN key management (interface)

SafeNet (Eracom) function	Function code	TSS function
KVC Request (Return Verification Code of a key)	7510 (APCA)	Check digit verification for Domain Master Key (KM) (interface and POS channel under AS2805 6.2)
CVVVERIFY	8520 (APCA)	Card verification
TERMPROOF - 6.4 (Proof-of- endpoint processing – POS terminal)	E500 (APCA)	Proof-of-endpoint (AS2805-compliant POS channel) processing
NODEPROOF (Proof-of-endpoint processing – request – interchange)	E520 (APCA)	Proof-of-endpoint (interface) processing
NODERESP (Proof-of-endpoint processing – response - interchange)	E530 (APCA)	Proof-of-endpoint (interface) processing

Considerations

Currently, the Transaction Security Services process requests one key at a time, even though the “EE” key generation commands for a ProtectHost White Mark II HSM allow multiple keys to be generated in a single request message.

The Transaction Security Services process always uses the 12 rightmost digits, excluding the check digit, for the PAN string used to build an ANSI PIN block. It does not support the 12 rightmost digits, including the check digit, or the 12 leftmost digits, for the PAN string used to build an ANSI PIN block.

For keys stored in the TSS database for a ProtectHost White Mark II HSM, the TSS application uses the SafeNet (Eracom) key specifier format of 10 (encrypted key—single length) for single-length keys and the SafeNet (Eracom) key specifier format of 11 (encrypted key—double-length—ECB) for double-length keys.

For keys stored in the ProtectHost White Mark II HSM, the TSS application assumes a key specifier format of 03 (index—long/binary).

Thales e-Security modules

Thales security functions are available through separate utilities, the ACI desktop user interface, or the HSM console.

Management of key check values (check digits) has been enhanced so that check digits are automatically validated by the security device when the keys and check digits are entered from the ACI desktop user interface. This provides immediate validation of keys, rather than having to use a batch utility to validate keys.

External key block format

The external key block format is the format of the key block that an external entity (e.g., device or interface) supports. The format is set in the External Key Block Format field on the Channel Security Configuration window and the Interface Security Configuration window. If the external entity uses a Thales e-Security HSM, you can set this field to ANSI X9.17 method or Variant method. Refer to the Thales e-Security vendor documentation for descriptions of these key encryption schemes and how they are used in processing.

Double- and triple-length key encryption scheme

The Transaction Security Services process requires all double- and triple-length keys for Thales e-Security HSMs stored in the TSS database to be encrypted using Thales' variant (1A+32H) key encryption scheme instead of the old (ANSI X9.17 method) non-variant (32H) key encryption scheme. For more information on these encryption schemes, refer to the Thales ***Host Security Module, RG7000 Programmer's Manual***.

If a variant of the LMK is required for the key type (see the Master Key Pair column in the table for the "Preparing Data for the TSS Files" topic on the following page), this variant and is always applied regardless of the key encryption scheme used (ANSI X9.17 or variant) and is applied to the left half of the LMK.

For double- and triple-length keys encrypted using the variant key encryption, another variant (two or three different ones depending on the length of the key being encrypted or decrypted) is applied to the right half of the LMK. Refer to the "Local Master Key Triple DES Variant Scheme" topic in the Thales ***Host Security Module, RG7000 Programmer's Manual*** for a description of how these variants are applied.

Note: Although Thales e-Security HSMs treat key pairs like double-length keys, the Transaction Security Services process views key pairs as two single-length keys. Because of this, the Transaction Security Services process does not support the variant (1A+32H) key encryption scheme for these keys. Therefore, Visa PVV PIN verification and card verification key pairs must be encrypted using the non-variant (32H) key encryption scheme in the TSS database.

Decimalization table security

For Thales e-Security HSMs, the decimalization table value sent in commands can be encrypted or in the clear. If you are using the IBM DES (3624-format) PIN verification method, you can encrypt the clear decimalization table value using a console command, and enter the encrypted value in the Decimalization Table field on the DES (IBM 3624) PIN

Verification tab of the Authentication Profile Configuration window. The Configure Security Parameter “Encrypted/Plaintext Decimalization Tables” indicates whether the HSM supports Encrypted or Plaintext decimalization tables. The valid values for this parameter are:

- E = Encrypted decimalization tables are used. Default.
- P = Plaintext (in the clear) decimalization tables are used.

The “Encrypted/Plaintext Decimalization Tables” configure security parameter is available beginning in the following firmware/software version numbers for all HSM ranges.

HSM type	Version number
HSM 8000	2.0
RG7x10	3.0
RG7x00	7.0

If you choose to use encrypted decimalization tables, all decimalization tables must be encrypted. You cannot have some decimalization tables in the clear and some encrypted. In addition, you must set the optional environment attribute `DECIMAL_TBL_ENCRYPT` to a value of Y for the Master File Key Conversion utility to be able to properly translate encrypted decimalization tables from the old LMK pair to the new LMK pair.

Message authentication for large messages

The Transaction Security Services process supports message authentication of large messages (e.g., ATM statements, EMV issuer scripts) in Thales e-Security HSMs using data chaining (multiple calls to the HSM) in Generate MAC (M6), Verify MAC (M8), and MAC Generate/Verify (MS) commands. Data chaining is not supported for message authentication in GW, KU, KW, MK, and MM commands. Therefore, the size of a message to be authenticated in these commands is limited by the configured buffer size of the HSM and communications protocol used to interface with the HSM.

Preparing data for the TSS files

Offline HSM Master Key loading and key management are accomplished using a user-supplied asynchronous ASCII terminal connected to the HSM asynchronous port. This asynchronous ASCII terminal is known as the HSM console.

The table on the following pages summarizes the HSM console commands used to create, encrypt, and translate keys for storage in the TSS database. Note that the HSM Master Keys must have already been loaded and the HSM placed in the authorized state before the commands can be performed.

Field name on user interface window	Window	Master key pair	HSM console command
PIN Information: Exchange Key	Channel	14–15	K = Encrypt a terminal master key
PIN Information: Inbound Key	Channel	14–15	K = Encrypt a terminal PIN key ¹
MAC Information: Exchange Key	Channel	14–15	K = Encrypt a terminal master key
MAC Information: Inbound Key	Channel	16–17	FK = Format key from components
Derivation Key	Derivation Key	28–29	DE = Form a ZMK from clear components DG = Generate a base derivation key
Cryptographic Keys	Authentication	Variant 1 of 28–29	FK = Format key from components
Secure Message Key For Integrity	Authentication	Variant 2 of 28–29	FK = Format key from components
Secure Message Key for Confidentiality	Authentication	Variant 3 of 28–29	FK = Format key from components
DES (IBM 3624) PIN Verification	Authentication	14–15	K = Encrypt a PIN verification key
Decimalization Table	Authentication	18–19	ED = Encrypt decimalization table
Diebold Number Table	Authentication	10–11	R = Load and encrypt a clear Diebold table ⁵
VISA PVV Keys	Authentication	14–15	K = Encrypt a PIN verification key
Card Verification	Authentication	Variant of 14–15 ⁶	KA = Generate a CVK pair

Field name on user interface window	Window	Master key pair	HSM console command
Contactless	Authentication	Variant 1 or 7 of 28–29 ⁷	FK = Format key from components
NCR: Encryption Keys	Authentication	Variant of 20–21	UD or UI = Translate PVK from KTR to LMK ⁸
Exchange Information: PIN Keys: Import	Interface	04–05	EC = Encrypt clear component FK = Format key from components KI = Import key ²
Exchange Information: PIN Keys: Export	Interface	04–05	EC = Encrypt clear component FK = Format key from components KI = Import key ²
Exchange Information: MAC Keys: Import	Interface	04–05	EC = Encrypt clear component FK = Format key from components KI = Import key ²
Exchange Information: MAC Keys: Export	Interface	04–05	EC = Encrypt clear component FK = Format key from components KI = Import key ²
PIN Information: Outbound Keys	Interface	06–07	FK = Format key from components ³
PIN Information: Inbound Keys	Interface	06–07	FK = Format key from components ³
MAC Information: Outbound Keys	Interface	16–17	FK = Format key from components ⁴
MAC Information: Inbound Keys	Interface	16–17	FK = Format key from components ⁴
Master Keys: PIN Key	Interface	06–07	EC = Encrypt clear component FK = Format key from components KI = Import key

Field name on user interface window	Window	Master key pair	HSM console command
Master Keys: MAC Key	Interface	16–17	EC = Encrypt clear component FK = Format key from components KI = Import key

- ¹ The intermediate key value of 9999999999999999 (16 hexadecimal 9s) encrypted under the HSM Master Key pair 06–07 must be stored in a Channel_Security record for ATM or POS devices configured to send PINs in the clear to any interchanges on the BASE24 system.

For SPDH-compliant devices or Hypercom devices using the DUKPT method of PIN encryption, this field must contain an intermediate key encrypted under the HSM Master Key pair 06–07. Although the HSM Master Key pair 06–07 represents an interchange rather than a terminal key, it is required because Thales e-Security only supports a command to translate from DUKPT encryption to encryption under an interchange master/session key.

- ² A key exchange key is the key under which working keys are encrypted for exchange between BASE24 and an external processor. It equates to the Thales e-Security Zone Master Key (ZMK).

An encrypted ZMK need not be placed in the Exchange Information fields on the Interface Security Configuration window unless automatic key exchange is supported. (BASE24 only supports automatic key exchange with DPCs and certain interchanges.)

To generate a ZMK, the console command EC must be used first to obtain three encrypted ZMK components. Console command FK is then used to format the key from the three encrypted ZMK components. Finally, the KI command is used to translate the key from encryption under the ZMK to encryption under the LMK for storage in the Exchange Information fields.

- ³ If the BASE24 operations staff is responsible for generating the inbound and outbound keys for PIN encryption, console command B or FK can be used to generate and encrypt these keys under the HSM Master Key pair 06–07. These keys can then be placed in the PIN Information fields on the Interface Security Configuration window, and the ZMK-encrypted forms of these keys can be conveyed to the external processor.

If the inbound and outbound working keys are generated by an external processor, the keys can be distributed automatically or manually. If these keys are distributed automatically when the interface process is first brought up, inbound and outbound entries in the PIN Information fields must be left as zeros. The ZMK must be provided in the Exchange Information fields on the Interface Security Configuration window (refer to the first footnote for more information about the ZMK).

Otherwise, the BASE24 operations staff receives ZMK-encrypted versions or clear versions of these keys. The ZMK-encrypted versions of the keys must be used to obtain versions encrypted under the HSM Master Key pair 06–07 for storage in the TSS database. Since the inbound and outbound keys must be encrypted or decrypted using the ZMK, depending on whether the keys are being sent to or received from an external processor, the ZMK must be provided in the HSM console command B or FK.

If the operations staff receives clear versions of these keys, they must be encrypted using the HSM console command BK to generate the encrypted zone PIN keys (ZPKs). The key type must be set to a value of 2 for this command. If only one component is used, it should be entered three times. Otherwise, at least two different components should be entered. The resulting cryptogram can then be placed in the PIN Information fields, and the ZMK-encrypted forms of these keys can be conveyed to the external processor.

- ⁴ If zone authentication keys are not distributed automatically, they must be generated manually and encrypted using the HSM console FK command.
- ⁵ Handling the Diebold Number Table — Initially, the clear Diebold number table (DNT) must be manually entered using the RSM console command R. This command causes the clear DNT to be encrypted and loaded into the RSM. It also returns an encrypted version of the DNT to the BASE24 system administrator. The BASE24 system administrator can then copy the encrypted form of the DNT and enter it on the Authentication Profile Configuration window.

Anytime you change the DNT in the TSS database, you must rebuild the Diebold Number Table external memory table (OLTP), warmboot the Transaction Security Services processes that access the external memory table (OLTP), and execute the DNTLOAD process control command to reload the DNT in the memory of the HSM. Refer to [section 2](#) for information on rebuilding external memory tables (OLTPs) and warmbooting processes that access them. Refer to [section 9](#) and [section 18](#) for more information on using the DNTLOAD command.

- ⁶ The HSM encrypts card verification keys under a variant of key pair 14-15. This variant, unlike the variants used with Atalla security devices, is an unknown value. HSM card verification keys must be encrypted using HSM console command KA. This command incorporates the variant that is required to obtain the proper results.
- ⁷ For dynamic card verification of the dCVV for Visa Wave cards, variant 1 is required. For dynamic card verification of the CVC3 for MasterCard PayPass cards, variant 7 is required.
- ⁸ Use the UA–UD commands for single DES encryption and the UF–UI commands for two-key triple DES encryption.

Generate check digits command

The User Interface process uses the BU (Generate a Key Check Values) command when you enter a new key from the ACI desktop user interface.

Changing local master key pairs

ACI provides an optional Master File Key Conversion tool that enables you to translate hardware-encrypted keys in the TSS database from encryption under one Local Master Key (LMK) pair value to another LMK pair value, automating what otherwise is a manual process. This conversion tool is executed from the Master File Key Conversion window, accessed as **Configure > Transaction Security > Master File Key Conversion** on the ACI desktop. Using this tool, you can translate all the keys in the TSS database, or just the keys encrypted under a particular LMK pair (i.e., 04–05, 06–07, 14–15, 16–17, 18–19, 20–21, 28–29, 34–35, or 36–37). For detailed information on using this tool, refer to the [Master file key conversion](#) topic provided in section 16.

Good security practice as well as several network and card associations require the LMK pairs to be replaced periodically. This conversion tool enables you to convert keys from encryption under the old LMK pair to encryption under the new LMK pair and automatically replace them in the TSS database files. HSMs must contain the old LMK pair as the current LMK pair and contain the new LMK pair as the pending LMK pair before this utility is used. The Master File Key Conversion utility requires the BW command (Translate Keys From Old LMK to New LMK) to translate keys. After keys are converted, use the Erase LMK or MFK command on the Process Control window to erase the old LMK pair. This process control command executes the Thales e-Security BS command (Erase the Key Change Storage) on the specified HSMs.

Triple DES enabled commands

All HSM commands required for BASE24 processing are triple DES-enabled in firmware version 5.05/1.05. This release also includes an option to disable single DES in HSM commands. A minimum of version 6.02 firmware and an optional DSP plug-in (for RSA functions) is required to support public key cryptography.

Racal Access Module (RAM) process

The RAM process provides direct IP support and supports the User Datagram Protocol/Internet Protocol (UDP/IP), a connectionless protocol, in addition to TCP/IP. These features eliminate the I/O to the XPNET process and the check-pointing overhead associated with the XPNET system. Thus, BASE24 environments using the RAM process should see performance improvement, even with the additional I/O to the Transaction Security Services process. For more information on the RAM process, refer to the **BASE24 Transaction Security Manual** and the following informal publications:

- The **Remote Application Manager Implementation and Installation Guide** describes the use of the RAM process with TCP/IP. This guide is a TGAL document located in the RAMUPDT file on the BA10RAM subvolume.
- The **RAM/UDP Update Guide** describes the use of the URAM process with UDP/IP. The RAM process is referred to as the URAM process when used with UDP/IP. This guide is a Microsoft Word document. The binary format of this document is located in the

URAMUDOC file on the BA10RAM subvolume. To view this document, perform a binary transfer of the URAMUDOC file to your PC and add a .DOC extension to the file name. You can then open the document using the Word application.

URAM-MSG-MDFY parameter

The URAM-MSG-MDFY parameter in the URAMCNFG file indicates what type of message modification processing the URAM process performs. When using the Transaction Security Services process, you must set this parameter to a value of 0 (no message modification is performed). For more detailed information on the URAM-MSG-MDFY parameter and its possible values, refer to the global definition comments in the BA10RAM.URAMCNFG file.

HSM commands

The following table lists the Thales e-Security commands required to support BASE24 security functions using the Transaction Security Services process. The first two columns of the table list the Thales e-Security commands and associated transaction codes. The third column describes the BASE24 function for which the command is used.

Thales e-Security command	Transaction code	BASE24-eps function
Generate a key	A0	Public key cryptography
Translate a TMK, TPK, or PVK from LMK to Another TMK, TPK, or PVK	AE	PIN key translation (device)
Erase the Key Change Storage	BS	Master File Key Conversion utility
Generate a Key Check Value	BU	ACI desktop key entry
Translate Keys From Old LMK to New LMK	BW	Master File Key Conversion utility
Translate PIN (encryption under LMK pair 14-15 to encryption under LMK pair 06-07)	CA	PIN translation—terminal originated (interface)
Translate PIN (encryption under LMK pair 06-07 to encryption under LMK pair 06-07)	CC	PIN translation—interface originated (interface)
Generate a Diebold PIN Offset	CE	PIN change—Diebold PIN verification method (device and interface)
Verify PIN from Terminal—Diebold method	CG	Diebold PIN verification (device)

Thales e-Security command	Transaction code	BASE24-eps function
Translate PIN (BDK encryption under LMK pair 28–29 to ZPK encryption under LMK pair 06–07)	CI	Derived unique key per transaction (DUKPT) PIN block translation using the single key derivation algorithm
Generate CVV/CVC	CW	Requires a CSM in the security utilities to use.
Verify a Visa CVV	CY	Card verification (verify CVV/CVC) (MasterCard and Visa cards)
Verify PIN from Terminal—IBM 3624-format	DA	IBM 3624 PIN verification method (device)
Verify PIN from Terminal—Visa PVV	DC	Visa PVV PIN verification method (device)
Generate IBM 3624 PIN Offset	DE	PIN change—IBM 3624 PIN verification method (device and interface)
Generate Visa PVV	DG	PIN change—Visa PVV PIN verification method (device and interface)
Interchange PIN Verify—IBM 3624-format	EA	IBM 3624 PIN verification method (interface)
Interchange PIN Verify—Visa PVV	EC	Visa PVV PIN verification method (interface)
Interchange PIN Verify—Diebold method	EG	Diebold PIN verification method (interface)
Generate an RSA Key Set	EI	Public key cryptography
Generate a MAC on a Public Key	EO	
Validate a Certificate and Generate MAC on its Public Key	ES	
Generate a Signature	EW	
Validate a Signature	EY	
Translate a Secret Key from the Old LMK to a New LMK	EM	Public key cryptography (Master File Key Conversion utility)
Translate a MAC on a Public Key	EU	

Thales e-Security command	Transaction code	BASE24-eps function
Interchange Key Translate (encryption under ZMK to encryption under LMK pair 06-07)	FA	PIN key translation (interface)
Translate DUKPT PIN to DES PIN – Triple DES	G0	Derived unique key per transaction (DUKPT) PIN block translation using the double key derivation algorithm
Interchange Key Translate (encryption under LMK pair 06-07 to encryption under ZMK) ¹	GC	PIN key translation (interface)
Export a DES key	GK	Public key cryptography
Hash a Block of Data	GM	
MAC Verify/MAC Generate ²	GW	Derived unique key per transaction (DUKPT) message authentication
Generate TAK under TMK, TPK, or LMK	HA	MAC key management (device and interface)
Generate a New Terminal Master Key or PIN Key	HC	PIN key management (device) Changing ATM master key (device)
Interchange Key Generate (encryption under ZMK and LMK pair 06-07)	IA	PIN key management (interface)
Translate PIN from TMK or TPK to LMK	JC	PIN change—Diebold, IBM 3624, or Visa PVV PIN verification methods (device)
Translate PIN from ZWK to LMK	JE	PIN change—Diebold, IBM 3624, or Visa PVV PIN verification methods (interface)
Generate a Key Check Value (Not Double Length ZMK)	KA	ACI desktop key entry
ARQC (or TC/ACC) Verification or ARPC Generation	KQ	EMV authorization The enhanced KU command (available in Thales version 6 or higher software) is required for validation of offline PIN changes to EMV cards.
Generate Secure Message with Integrity and optional Confidentiality and PIN Change	KU	

Thales e-Security command	Transaction code	BASE24-eps function
Generate Secure Message with Integrity and optional Confidentiality and PIN Change (EMV 2000)	KY	EMV 2000 and EMV CCD authorization
ARQC (or TC/ACC) Verification or ARPC Generation (EMV 2000)	KW	
Verify Encrypted Counters - M/Chip 4 ³	K0	
Verify Truncated Application Cryptogram (MasterCard CAP)	K2	CAP token validation
Load Data to User Storage	LA	Diebold PIN verification—loading the encrypted DNT to HSM memory
Translate Decimalization Table from Old to New LMK	LO	Master File Key Conversion utility (Encrypted decimalization table)
Generate MAC	M6	Message authentication (supports data chaining for large messages)
Verify MAC	M8	
Translate TAK from LMK to ZMK	MG	MAC key management (interface) MAC key translation (interface)
Translate TAK from ZMK to LMK	MI	MAC key translation (interface)
Generate a binary MAC ⁴	MK	Message authentication
Verify a binary MAC ⁴	MM	Message authentication
MAC Generate/Verify ⁵	MS	Dutch security extensions (message authentication for large messages) or X9.19 method of message authentication
HSM Status	NO	Enabling a security device
Verify Dynamic CVV	PM	Dynamic card verification
Verify Card Security Codes	RY	Card verification (American Express cards)

Thales e-Security command	Transaction code	BASE24-eps function
PIN Generate RVT ⁶	SA	Dutch security extensions (Equens interface key management)
PIN Receive RVT ⁶	SC	
PIN Generate RVS ⁶	SE	
PIN Receive RVS ⁶	SG	
Verify PIN—NCR method ⁶	SI	Dutch security extensions NCR PIN verification (device and interface)

¹ In some HSM firmware versions, the GC transaction code may be available only when the HSM is in the authorized state. As a result, the Repeat Key command for dynamic key management may not work. Check with your Thales e-Security customer support representative.

² Only eight-byte MACs are supported for the GW command.

³ Proprietary authentication data is not supported for the K0 command.

⁴ The MK and MM transaction codes are not supported in the standard Thales e-Security product. They must be obtained from Thales e-Security.

⁵ This command uses the X9.19 method of message authentication, also known as pseudo triple DES.

⁶ These commands are not available in standard Thales e-Security products and must be specially ordered.

Asynchronous protocol and Thales HSM commands

The following commands for Thales e-Security hardware security modules (HSMs) are not supported over an asynchronous protocol:

- ARQC (or TC/ACC) Verification or ARPC Generation (KQ)
- Generate Secure Message with Integrity and optional Confidentiality and PIN Change (KU)
- Generate Secure Message with Integrity and optional Confidentiality and PIN Change (EMV 2000) (KY)
- ARQC (or TC/ACC) Verification or ARPC Generation (EMV 2000) (KW)
- MAC Verify/MAC Generate (GW)
- Generate RSA Key Set (EI)

- Generate a MAC on a Public Key (EO)
- Validate a Certificate and Generate a MAC on its Public Key (ES)
- Generate a Signature (EW)
- Validate a Signature (EY)
- Export a DES Key (GK)
- Hash a Block of Data (GM)

5: TSS processing examples

This section presents example message flows showing how the Transaction Security Services process is incorporated into online BASE24 transaction processing. All message flows assume hardware encryption for PINs and MACs, except where noted.

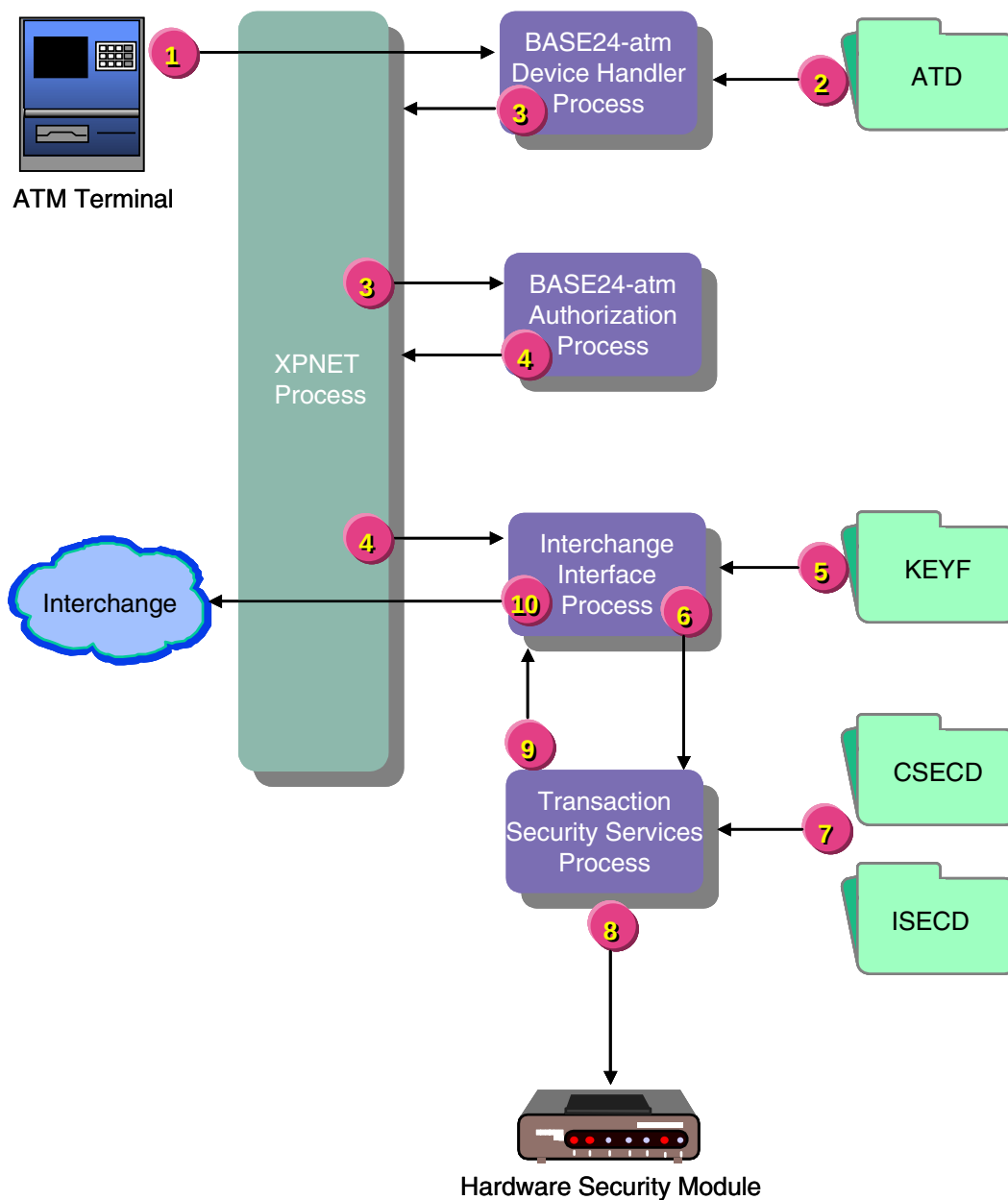
The message flows provided in this section are intended to provide an overall understanding of how the Transaction Security Services process fits in the flow of transaction security processing. This section is not comprehensive and is not intended to depict Transaction Security Services processing for every possible transaction security function.

Note: The flows in this section depict transaction processing steps integral to transaction security only.

Terminal acquired—not-on-us card

The diagram below illustrates the flow of PIN block processing for a transaction that originates at an ATM and is authorized by an interchange. The steps depicted in the illustration are described following the diagram.

1. A transaction including a PIN originates at an ATM terminal. The XPNET process passes the request containing a single DES or triple DES encrypted PIN block to the BASE24-atm Device Handler process.
2. The Device Handler process retrieves the key locator value (i.e., the terminal ID) for the terminal's PIN key from the BASE24-atm Terminal Data files and places this value in the STM along with the encrypted PIN block.
3. The Device Handler process sends the transaction to the XPNET process for delivery to the Authorization process for authorization and routing.
4. The Authorization process determines that the transaction needs to be routed to an Interchange Interface process. The Authorization process passes the transaction to the XPNET process for delivery to the Interchange Interface process.
5. The Interchange Interface process retrieves the key locator value (i.e., an index value created by the KEYF or KEY6 conversion program) for the outbound PIN key from the Key File (KEYF).
6. The security utilities bound into the Interchange Interface process pass the ATM PIN key locator value, outbound interface PIN key locator value, the encrypted PIN block and other PIN-related information to the Transaction Security Services process. The request does not pass through the XPNET process.



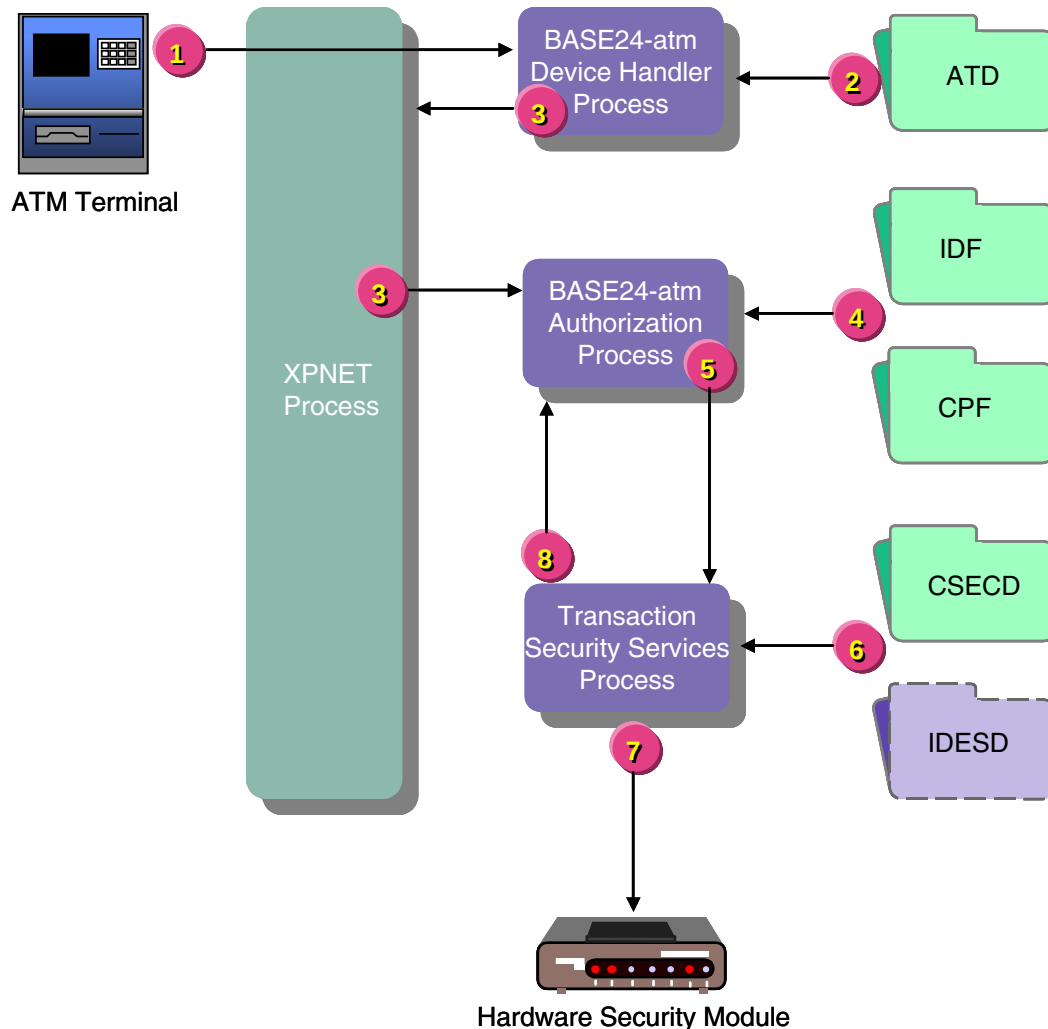
7. The Transaction Security Services process uses the key locator values to retrieve the actual ATM PIN key from the Channel_Security and the outbound PIN key from the ICHG_SECURITY.
8. The Transaction Security Services process calls the HSM to translate the encrypted PIN block from encryption under the ATM PIN key to encryption under the interface's outbound PIN key. Depending on the lengths of the keys retrieved, the translation can be any of the following: single DES to triple DES, single DES to single DES, triple DES to single DES, or triple DES to triple DES.
9. The Transaction Security Services process passes the translated PIN block back to the Interchange Interface process.

10. The Interchange Interface process sends the transaction, with the translated PIN block, to the XPNET process for delivery to the interchange for authorization.

Terminal acquired—on-us card

The diagram below illustrates the flow of PIN verification processing for a transaction that originates at an ATM and is authorized on the BASE24 system. The steps depicted in the illustration are described following the diagram.

In this transaction flow, the issuing institution uses the IBM DES (3624-format) PIN verification method. If a different PIN verification method were used, the flow would be essentially the same except that the Transaction Security Services process would access a different external memory table (OLTP) to retrieve the PIN verification information.



1. A transaction including a PIN originates at an ATM terminal. The XPNET process passes the request containing a single DES or triple DES encrypted PIN block to the BASE24-atm Device Handler process.
2. The Device Handler process retrieves the key locator value (i.e., the terminal ID) for the terminal's PIN key from the BASE24-atm Terminal Data files and places this value in the STM along with the encrypted PIN block.
3. The Device Handler process sends the transaction to the XPNET process for delivery to the Authorization process for authorization and routing.

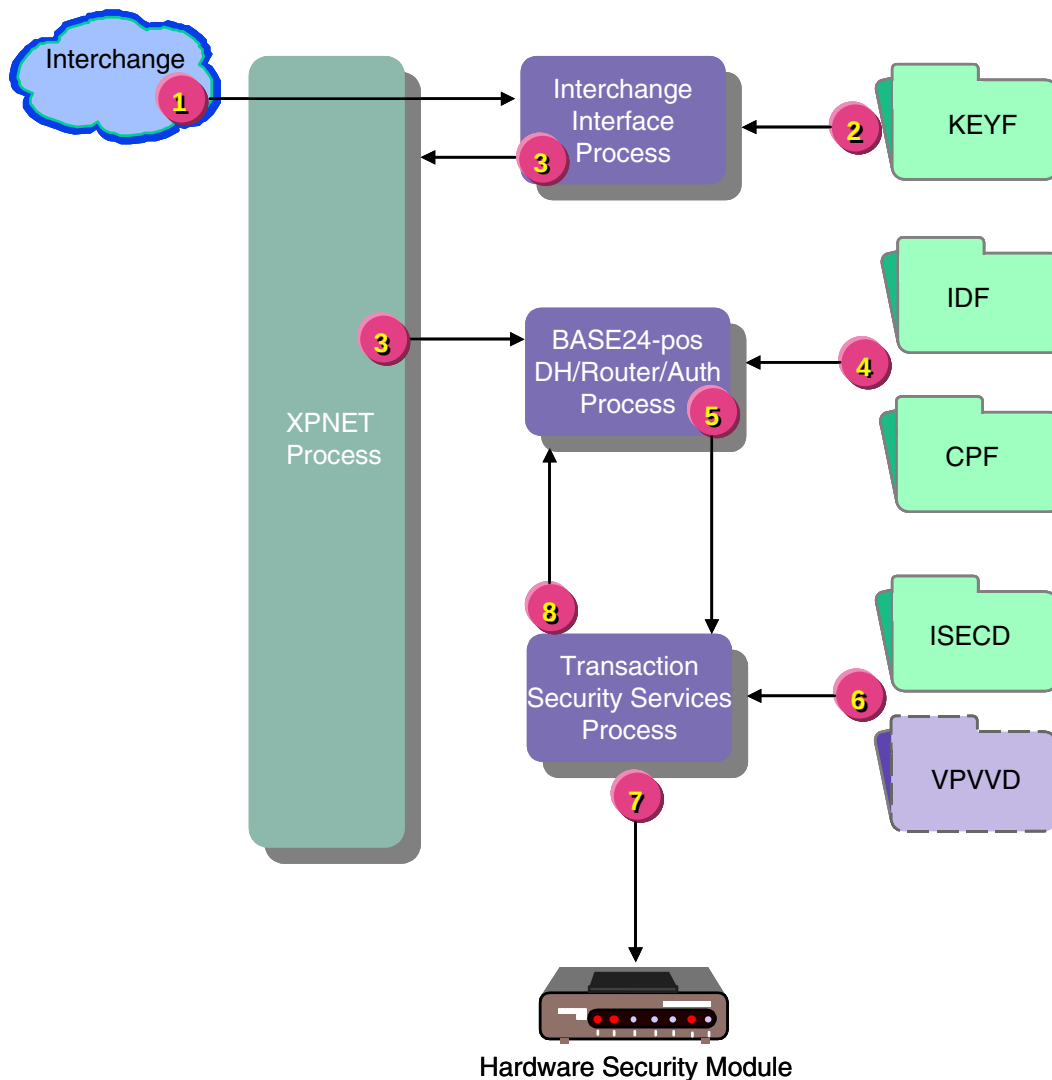
4. The Authorization process determines that the transaction needs to be authorized on the BASE24 system and retrieves the PIN check type value (i.e., the PIN verification method) from the Institution Definition File (IDF) or the Card Prefix File (CPF) and the PIN verification KEYA group value for the transaction from the CPF. Note that the Authorization process does not access the KEYA at all.
5. The security utilities bound into the Authorization process pass the ATM PIN key locator value, the PIN verification type, KEYA group, card expiration date, the encrypted PIN block and other PIN-related information to the Transaction Security Services process. The request does not pass through the XPNET process.
6. The Transaction Security Services process uses the ATM PIN key locator value to retrieve the actual ATM PIN key from the Channel_Security and the KEYA group and card expiration date values to retrieve the IBM DES (3624-format) PIN verification information from the IBM_DES external memory table (OLTP).
7. The Transaction Security Services process calls the HSM to decrypt the encrypted PIN block using the ATM PIN key and verify the PIN using the PIN verification key. Depending on the lengths of the keys retrieved, the HSM can perform single or triple DES decryption and verification.
8. The Transaction Security Services process passes the result of the PIN verification back to the Authorization process.

Interface acquired—on-us card

The diagram below illustrates the message flow of PIN verification processing when an interchange sends a transaction to the BASE24 system for authorization. The steps depicted in the illustration are described following the diagram.

In this transaction flow, the issuing institution uses the Visa PVV PIN verification method. If a different PIN verification method were used, the flow would be essentially the same except that the Transaction Security Services process would access a different external memory table (OLTP) to retrieve the PIN verification information.

1. An interchange sends a transaction request to the BASE24 system with an encrypted PIN. The XPNET process delivers the request to the Interchange Interface process.
2. The Interchange Interface process checks the value in the ENCRYPT TYPE and BASE24 ENCRYPT TYPE fields on Key File (KEYF) screen 1. In this case, both fields are set to a value of 1 indicating that PIN encryption and PIN management are performed in a security module. The Interchange Interface process retrieves the key locator value (i.e., an index value created by the KEYF or KEY6 conversion program) for the inbound PIN key from the Key File (KEYF) and places it in the PSTM.
3. The Interchange Interface process passes the transaction to the XPNET process for delivery to the BASE24-pos Device Handler/Router/Authorization process for authorization.

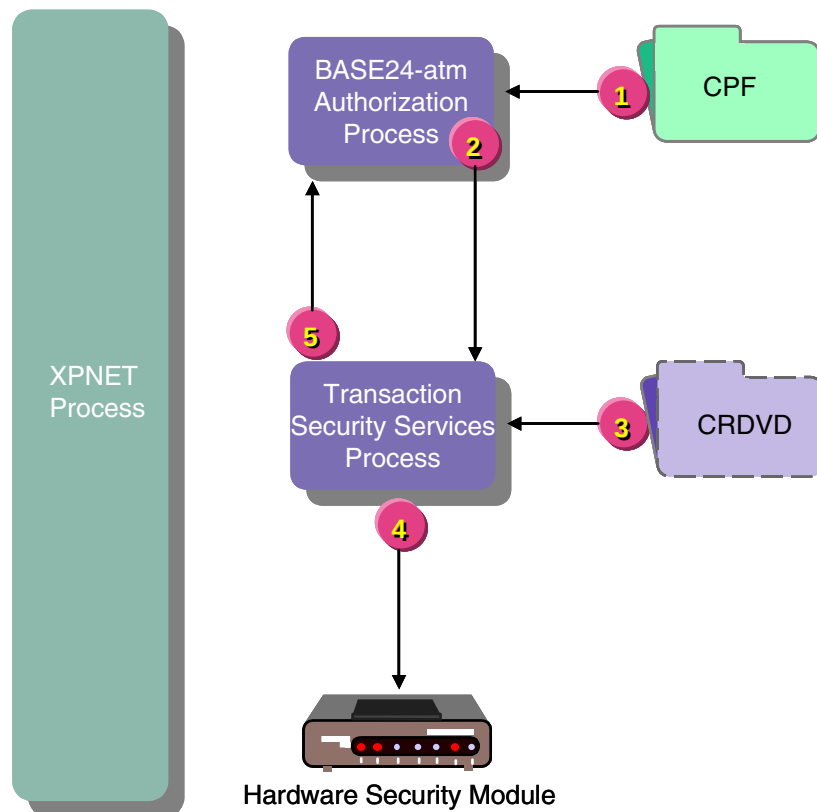


4. The Authorization module in the BASE24-pos Device Handler/Router/Authorization process retrieves the PIN check type value (i.e., the PIN verification method) from the Institution Definition File (IDF) or the Card Prefix File (CPF) and the PIN verification KEYA group value for the transaction from the CPF. Note that the Authorization process does not access the KEYA at all.
5. The security utilities bound into the BASE24-pos Device Handler/Router/Authorization process pass the inbound PIN key locator value, the PIN verification type, KEYA group, card expiration date, the encrypted PIN block and other PIN-related information to the Transaction Security Services process. The request does not pass through the XPNET process.

6. The Transaction Security Services process uses the inbound PIN key locator value to retrieve the actual inbound PIN key from the ICHG_SECURITY and the KEYA group and card expiration date values to retrieve the Visa PVV PIN verification information from the Visa_PVV external memory table (OLTP).
7. The Transaction Security Services process calls the HSM to decrypt the encrypted PIN block using the inbound PIN key and verify the PIN using the PIN verification key. Depending on the lengths of the keys retrieved, the HSM can perform single or triple DES decryption and verification.
8. The Transaction Security Services process passes the result of the PIN verification back to the Authorization module.

Card verification

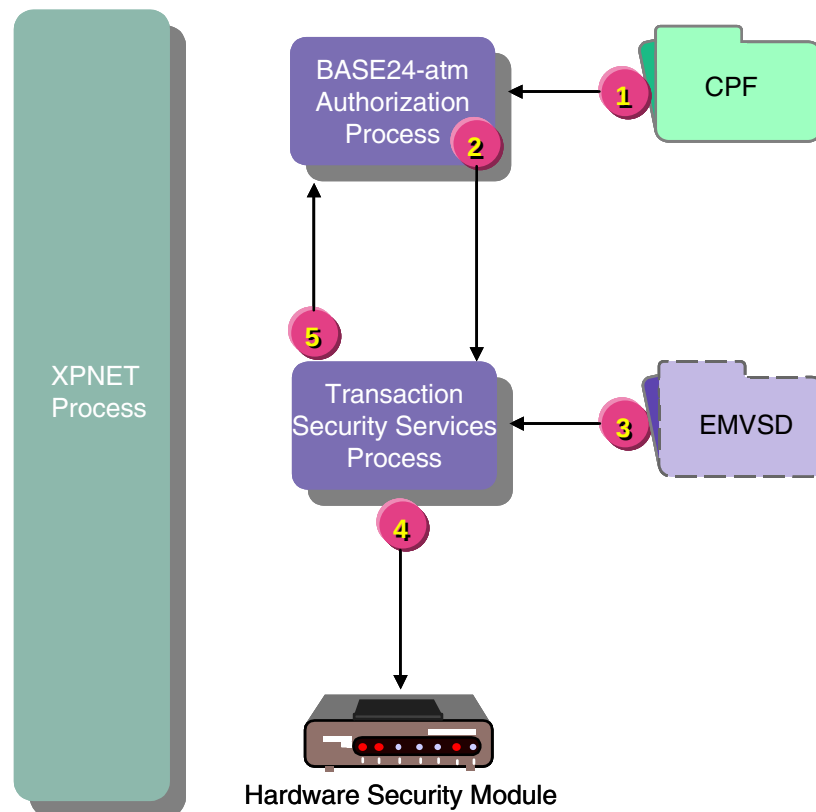
The diagram below illustrates the message flow for card verification. This message flow assumes that PIN verification has already been performed. The steps depicted in the illustration are described following the diagram.



1. The Authorization process retrieves the card verification check type and the card verification KEYA group values for the transaction from the CPF. Assume that card verification is required for this transaction.
2. The security utilities bound into the Authorization process pass the card verification KEYA group, card expiration date, and the card verification data to the Transaction Security Services process. The request does not pass through the XPNET process.
3. The Transaction Security Services process uses the card verification KEYA group value and card expiration date to retrieve the security module-encrypted Card Verification Key pair (CVK pair) from the CARD_VRFCTN external memory table (OLTP).
4. The Transaction Security Services process calls the HSM to verify the card using the key pair retrieved in the previous step.
5. The Transaction Security Services process passes the result of the card verification back to the Authorization process.

EMV card authentication

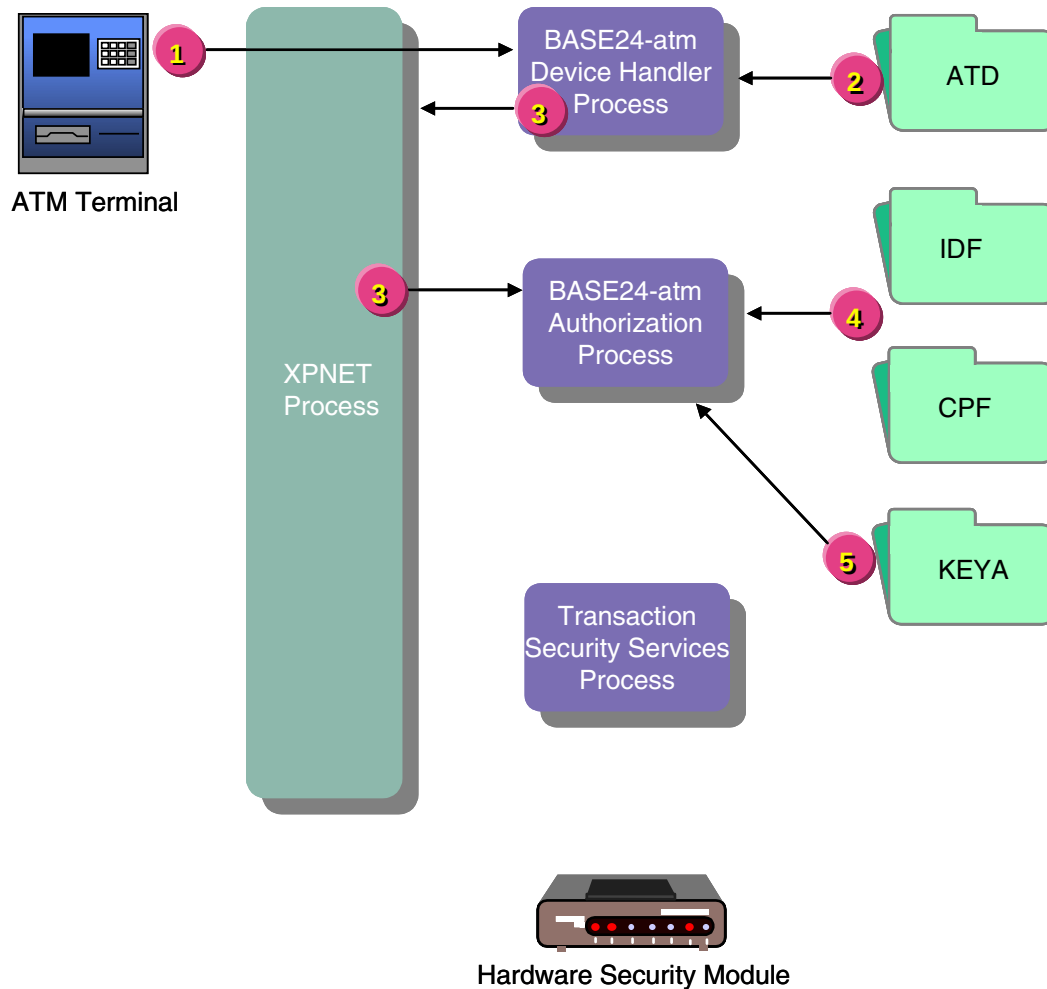
The diagram below illustrates the security processing message flow for EMV card authentication. This message flow assumes that PIN verification has already been performed. The steps depicted in the illustration are described following the diagram.



1. The Authorization process retrieves the KEYI group and EMV check values for the transaction from the CPF. Assume that EMV checking is required for this transaction.
2. The security utilities bound into the Authorization process pass the KEYI group, card expiration date, and the EMV authentication data to the Transaction Security Services process. The request does not pass through the XPNET process.
3. The Transaction Security Services process uses the KEYI group value and card expiration date to retrieve the appropriate issuer master key or secure messaging key from the EMV_Security external memory table (OLTP).
4. The Transaction Security Services process calls the HSM to verify the request cryptogram and generate the response cryptogram.
5. The Transaction Security Services process passes the result of the authentication processing back to the Authorization process.

Terminal acquired—on-us card—software verification allowed

The diagram below illustrates the flow of PIN verification for a transaction that originates at an ATM and is authorized on the BASE24 system. In this case, the issuing institution verifies PINs in software and software verification is allowed (i.e., the SOFTWARE-PIN-VERIFICATION-ALWD param is set to a value of Y). The steps depicted in the illustration are described following the diagram.



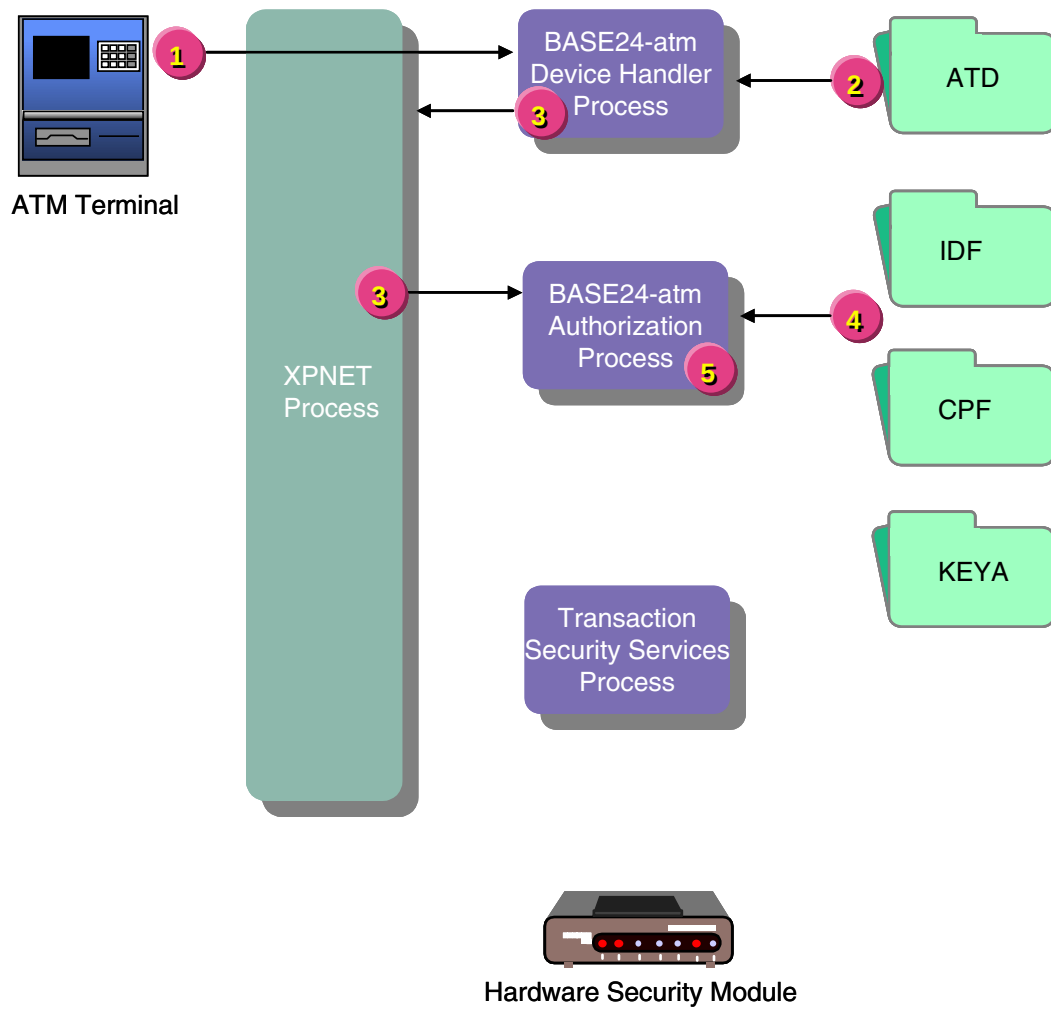
1. A transaction including a PIN originates at an ATM terminal. The XPNET process passes the request containing a clear PIN block to the BASE24-atm Device Handler process.
2. The Device Handler process checks the PIN encrypt type value for the terminal from the BASE24-atm Terminal Data files and determines no PIN encryption is used. The process places this value in the STM along with the clear PIN block.
3. The Device Handler process passes the transaction to XPNET process for delivery to the Authorization process for authorization and routing.

4. The Authorization process retrieves the PIN check type value (i.e., the PIN verification method) from the Institution Definition File (IDF) or the Card Prefix File (CPF) and the PIN verification KEYA group value for the transaction from the CPF.
5. The Authorization process retrieves the clear key for the PIN verification method specified in the STM from the KEYA and verifies the PIN in software using the specified verification method. The security utilities and the Transaction Security Services process are not involved in processing.

Terminal acquired—on-us card—software verification not allowed

The diagram below illustrates the flow of PIN verification for a transaction that originates at an ATM and is authorized on the BASE24 system. In this case, the issuing institution verifies PINs in software, but software verification is not allowed (i.e., the SOFTWARE-PIN-VERIFICATION-ALWD param is set to a value of N). The steps depicted in the illustration are described following the diagram.

1. A transaction including a PIN originates at an ATM terminal. The XPNET process passes the request containing a clear PIN block to the BASE24-atm Device Handler process.
2. The Device Handler process checks the PIN encrypt type value for the terminal from the BASE24-atm Terminal Data files and determines no PIN encryption is used. The process places this value in the STM along with the clear PIN block.
3. The Device Handler process passes the transaction to the XPNET process for delivery to the Authorization process for authorization and routing.
4. The Authorization process retrieves the PIN check type value (i.e., the PIN verification method) from the Institution Definition File (IDF) or the Card Prefix File (CPF) and the PIN verification KEYA group value for the transaction from the CPF.
5. The Authorization process determines that PIN verification in software is not allowed and denies the transaction.



6: Message data field encryption

This section describes the data encryption support provided by the Transaction Security Services process for message field data. This support enables you to encrypt select fields (configurable) in a message or an entire (full) message transmitted between BASE24 and certain endpoints. This section describes message field encryption processing, the available options, and how to configure the BASE24 and TSS databases to enable it.

Overview

Data encryption and decryption enables you to encrypt sensitive data, other than PINs, in messages transmitted between endpoints. An endpoint can be an ATM, POS device, interchange, or host, and the BASE24 system. Both the transmitting endpoint and the receiving endpoint must either share the same working key called a data (or message) encryption key.

The transmitting endpoint uses the data encryption key to encrypt a selected data field. The receiving endpoint uses the data encryption key to decrypt the data field. As with other working keys in the BASE24 system, you can change data encryption keys dynamically using dynamic key management thresholds.

Note: On the ACI desktop user interface windows, message data encryption is referred to as data encryption. On the KEYF screen, message data encryption is referred to as message encryption. The key used to encrypt and decrypt message data is referred to as the data encryption key or message data encryption key throughout this section.

When encrypting a data field or the entire message for transmission, you can use two different modes of the DES algorithm to encrypt and decrypt the data—cipher block chaining (CBC) or electronic code book (ECB). You can also use either one or two layers of data encryption.

Cipher block chaining (CBC)

Cipher block chaining (CBC) is a mode of operation that uses feedback in the encryption process. The result of one encryption operation is used to *feed* the encryption of the next block. The plaintext is combined with the previous ciphertext block using an exclusive-OR operation before it is encrypted. Thus, the encryption of each block depends on all the previous blocks. This also requires that all encrypted blocks be processed sequentially when decrypting the data.

CBC mode requires a random initialization vector, which is combined with the first data block using an exclusive-OR operation before it is encrypted. An initialization vector is a sequence of random bytes appended to the front of the plaintext block before encryption. The initialization vector does not have to be kept secret. The Transaction Security Services process uses a static initialization vector. The process calls the security module to generate the initialization vector the first time one is needed for a CBC operation. The initialization vector does not change thereafter.

One of the key characteristics of the CBC mode is that it uses a chaining mechanism that causes the decryption of a block of ciphertext to depend on all the preceding ciphertext blocks. As a result, the entire validity of all preceding blocks is contained in the immediately previous ciphertext block. A single bit error in a ciphertext block affects the decryption of all subsequent blocks. Rearrangement of the order of the ciphertext blocks also causes decryption to yield a corrupted value.

Electronic code book (ECB)

Electronic Code Book (ECB) is a mode of operation for a block cipher, with the characteristic that each possible block of plaintext has a defined corresponding ciphertext value and vice versa. In other words, for a given data encryption key, the same plaintext value will always result in the same ciphertext value. You can use the ECB mode when a volume of plaintext is separated into several blocks of data, each of which is then encrypted independently of other blocks.

ECB mode is not considered to be effective with small block sizes (for example, smaller than 40 bits) and identical encryption modes. This is because some words and phrases can be reused often enough so that the same repetitive part-blocks of ciphertext can emerge, laying the groundwork for a codebook attack where the plaintext patterns are fairly obvious. However, security can be improved if random pad bits are added to each block. On the other hand, 64-bit or larger blocks should contain enough unique characteristics to make a codebook attack unlikely to succeed.

Encryption functions

The Transaction Security Services process supports data encryption, data decryption, and data translation. Using the Transaction Security Service process, you can:

- Encrypt a specified data field for transmission to a POS device or certain interfaces and decrypt data fields received from certain interfaces.
- Encrypt an entire (full) message for transmission to a BASE24 host and decrypt an entire message received from a BASE24 host. With this option, all messages between the BASE24 system and the host are encrypted except for all network management (0800) messages. When a full message is encrypted, all fields starting with the primary bitmap to the end-of-text (ETX) character are encrypted. The start-of-BASE24-header, BASE24 header, and message type identifier fields are never encrypted.
- Translate encrypted data fields from encryption under one data encryption key to encryption under another data encryption key.

The BASE24 system can carry individual data fields internally either in the clear or in encrypted format as required, however, it cannot carry a full message internally in encrypted format.

The following table lists the various combinations of encrypted or clear values supported for the BASE24 system as the sending endpoint, the receiving endpoint, and internally. It also lists the encrypt, decrypt, or translate actions required for each valid combination.

Receiving endpoint		Internal		Sending endpoint	
Data	Action	Data	Action	Data	Action
Clear		Clear		N/A	
Clear		Clear		Clear	
Clear		Clear		Encrypted	Encrypt
Clear	Encrypt	Encrypted	Decrypt	N/A	
Clear	Encrypt	Encrypted		Clear	Decrypt
Clear	Encrypt	Encrypted		Encrypted	Translate
Encrypted	Decrypt	Clear		N/A	
Encrypted	Decrypt	Clear		Clear	
Encrypted	Decrypt	Clear		Encrypted	Encrypt
Encrypted		Encrypted	Decrypt	N/A	
Encrypted		Encrypted		Clear	Decrypt
Encrypted		Encrypted		Encrypted	Translate

Data encryption keys

The same data encryption keys used to encrypt data by the sending endpoint must be used by the receiving endpoint to decrypt the encrypted data field or message. The BASE24 system supports single, double, or triple length keys and both single DES or triple DES encryption. For one layer of data encryption, only one data encryption key is needed. For two layers of data encryption, two data encryption keys are required.

Data encryption layers

When encrypting data, you can use one or two layers of encryption. When using one layer, the data is encrypted and decrypted using a single data encryption key. When using two layers, the data encrypted or decrypted using a data encryption key is encrypted or decrypted again using a second data encryption key. The number of data encryption layers required by the endpoint typically dictates the number of layers to use.

Note: When using SafeNet (Eracom) ProtectHost White Mark II HSMs or when using DUKPT data encryption, you can only use one layer of data encryption.

Encryption data types

You can encrypt data in two different data types, binary or unpacked ASCII hexadecimal, depending on the DES encryption mode used. For the CBC mode, both binary and unpacked ASCII hexadecimal data is supported. For the ECB mode, only unpacked ASCII hexadecimal data is supported. The data type required by the endpoint usually dictates the type of encryption mode to use.

Note: Only unpacked ASCII hexadecimal data can be encrypted when using DUKPT data encryption.

Message authentication codes (MACs)

If an endpoint uses message authentication codes (MACs), the BASE24 system generates a MAC for encrypted data it transmits to an endpoint and verifies the MAC for encrypted data it receives from an endpoint.

Tokens

The data encryption key locator and encryption mode (i.e., CBC or ECB) used to encrypt data fields is carried in the Data Encryption Key (BN) token. The SPDH module supports encryption of the Available Balance field (FID J) only, encryption of selected fields¹ (configurable message encryption), or encryption of all fields¹ (full message encryption). The IMNI interface may encrypt the available balance field only before it is sent to the SPDH module. If the available balance field is encrypted, the SPDH module either translates or decrypts the balance as needed. For configurable or full message encryption, the SPDH module can encrypt or decrypt the configured fields or all fields in a message. The BASE24 ISO Host Interface process can encrypt or decrypt a full message.

¹ For configurable or full message encryption in the SPDH module, some optional fields are never encrypted because they are already encrypted under another key or created by another key. Additionally, the data encryption key cannot be used to encrypt itself in the Data Encryption Key field (FID I).

The TSS Key Index (BC) token indicates the index (1 or 2) of the PIN, MAC, or data encryption keys to use for a transaction when specified by the sending or acquiring interface. If the key index in the TSS Key Index token specifies a value of 1 for a key, the Transaction Security Services process uses the first key defined in the database. If the key index value in the TSS Key Index token specifies a value of 2 for a key, the Transaction Security Services process uses the second key defined in the database. If the key index value is not specified or the TSS Key Index token is not present for a transaction, the Transaction Security Services process uses the key currently defined as the current key in the database.

Configuring message data encryption for SPDH devices

The BASE24 Standard POS Device Handler (SPDH) module supports full message or configurable message encryption in any message sent to or received from an SPDH-compliant POS device. Also, the SPDH module supports the encryption of the available balance field only (in ASCII or binary format) returned in an authorization response to an SPDH-compliant POS device.

To support these message encryption options, you must configure the type of data encryption to be performed and whether the available balance is returned in a response in the BASE24-pos Terminal Data files (PTD). Also, for configurable message encryption, you must configure the fields to be encrypted in the ACNF field map for each transaction request or response message in which you want to encrypt data.

For all message encryption options, you must generate and store the data encryption key and indicate whether to use the CBC or ECB mode of encryption. These configuration options and values are set in the BASE24 database and TSS database as described below.

BASE24 database settings

The following paragraphs discuss the data encryption configuration requirements for the BASE24 database.

The available balance must be included in a response for it to be encrypted. The BALANCE RETURNED field on BASE24-pos Terminal Data files (PTD) screen 2 indicates whether the BASE24 system includes the available balance in response messages sent to the specified POS terminal.

To enable data encryption for a POS device, set the DATA ENCRYPTION TYPE field on PTD screen 7 to a value of 01 (Available balance encryption – ASCII), 02 (Available balance encryption – binary), 03 (Full message encryption), 04 (Configurable message encryption), 05 (Full message encryption – DUKPT), or 06 (Configurable message encryption – DUKPT). To disable data encryption for a POS device, set this field to a value of 00 (No encryption).

To support configurable message encryption, you must identify the fields to be encrypted in each transaction request and response message by entering an “E” for each encrypted field on the ACI Standard Device Configuration File (ACNF) Field Map screens.

To support dynamic key management of the data encryption key, you must set either the DATA KEY THRESHOLD or DATA KEY ERROR THRESHOLD field on the Key Threshold screen of the ACI Standard Device Configuration File (ACNF) to a nonzero value.

Finally, either the PIN MASTER KEY field on PTD screen 7 must be set to a key locator value or the DERIVATION KEY GROUP field on PTD screen 7 must be set to a KEYD record. A key locator value in the PIN MASTER KEY field is used to access the data encryption key configuration in the TSS database. The KEYD record specified in the DERIVATION KEY GROUP field must be configured with a key locator value in the DERIVATION KEY field to a TSS Derivation Key File (Derivation_KEY) record. The key locator value in the KEYD is used to access the DUKPT derivation key configuration in the TSS database.

For detailed information on PTD screens, refer to the **BASE24-pos Files Maintenance Manual**. For detailed information on ACNF screens, refer to the **BASE24-pos Standard POS Device Configuration Manual**. For detailed information on the KEYD screen, refer to the **BASE24 Files Maintenance Manual**.

You can convert the configuration of a TSM in an SPDH device to DUKPT message field data encryption by adding the corresponding KEYD and TSS DRVKD records, setting the DERIVATION KEY GROUP field on PTD screen 7, setting the POS-DH-DUKPT-UPDATE-METHOD LCONF param to a value of Y and changing out the TSM. If the following conditions are all met, the SPDH Device Handler module automatically updates the DATA ENCRYPTION TYPE field and processes the transaction:

- The KSN is present in the message (FID 6 sub-FID T).
- The DATA ENCRYPTION TYPE field on PTD screen 7 is not set to 05 (full message encryption - DUKPT) or 06 (configurable message encryption - DUKPT).
- The POS-DH-DUKPT-UPDATE-METHOD LCONF param is not set to a value of N.

If the DATA ENCRYPTION TYPE field was set to 04 (configurable message encryption), then the module updates the field to 06 (configurable message encryption - DUKPT). Otherwise, if the DATA ENCRYPTION Type field was set to 00 (no encryption) or 03 (full message encryption), the module updates the field to 05 (full message encryption - DUKPT) as the default.

LCONF

POS-DH-DUKPT-UPDATE-METHOD — A code indicating whether the Standard POS Device Handler (SPDH) module can automatically update the DATA ENCRYPTION TYPE field on BASE24-pos Terminal Data files (PTD) screen 7 to a value of 05 (full message encryption - DUKPT) or 06 (configurable message encryption - DUKPT) when the Key Serial Number and Descriptor field (SubFID T of FID 6) is received in a message for the first time and a Derivation Key File (KEYD) record exists for the SPDH terminal. If the DATA ENCRYPTION TYP field was set to 04 (configurable message encryption), then the module updates the field

to 06 (configurable message encryption - DUKPT). Otherwise, if the DATA ENCRYPTION Type field was set to 00 (no encryption) or 03 (full message encryption), the module updates the field to 05 (full message encryption - DUKPT). Valid values are as follows:

- Y = Yes, automatically update the data encryption method in the BASE24-pos Terminal Data files.
- N = No, do not automatically update the data encryption method in the BASE24-pos Terminal Data files.

Param Default: N

TSS database settings

The following fields must be configured on the Data Encryption Key Info and Data Encryption Exch Info tabs of the Channel Security Configuration window to support data encryption and decryption using data encryption keys:

- Data Encryption Key Info > DES Encr Mode (specifies whether to use the ECB or CBC mode of DES)
- Data Encryption Key Info > Data Type (specifies whether you are using unpacked ASCII hexadecimal or binary data encryption)
- Data Encryption Exch Info > Exchange Key (contains the transport key for downloading a new data encryption key to a device).

For Atalla NSPs, this key must be encrypted under variant 0 (Key Exchange Key Variant) of the MFK.

For SafeNet (Eracom) ProtectHost White Mark II HSMs, data encryption exchange keys must be stored in the Mark II HSM and entered as a five-decimal index value in the Key Part 1 column.

- Data Encryption Key Info > Outbound Keys (contains one or two data encryption keys, depending on whether you are using one or two layers of data encryption)
When using SafeNet (Eracom) ProtectHost White Mark II HSMs, you can only enter one data encryption key because Mark II HSMs always use just one layer of data encryption.
- Data Encryption Key Info > Encryption Layers (specifies whether to use one or two layers of data encryption)
When using SafeNet (Eracom) ProtectHost White Mark II HSMs, this field is disabled. Mark II HSMs always use just one layer of data encryption.
- Data Encryption Exch Info > Initialization Vector (specifies a sequence of random bytes appended to the front of the first plaintext block before encryption using the CBC mode of operation).

When using Atalla NSPs, the Initialization Vector field is a display only field. The BASE24 system calls the security module to generate the initialization vector the first time one is needed for a CBC operation. The value does not change thereafter. When using SafeNet (Eracom) ProtectHost White Mark II HSMs, the Initialization Vector field is active and you can optionally enter a value in the field. If the Transaction Security Services process cannot retrieve an initialization vector from the Channel_Security, the process calls the HSM to generate one and stores it in the Channel_Security.

All of this information is stored in the Channel Security File (Channel_Security). A sample of the Data Encryption Exch Info and Data Encryption Key Info tabs on the Channel Security Configuration window are shown in [section 2](#). Refer to the ACI Online Help provided for the ACI desktop user interface for detailed information on this window and its fields.

Configuring message data encryption for interfaces

Data encryption support is provided for the following interface processes in a BASE24 system:

- IMNI Interface
- ISO Host Interface
- Merchant Link Interface

Configuration support for these interfaces is provided on KEYF screens, the Interface Security Configuration window, and the BASE24 KEYF Configuration window.

KEYF screen 4 configuration

Screen 4 of the KEYF configures full message encryption for the ISO Host Interface process. Brief descriptions of the fields on KEYF screen 4 are provided below. For detailed descriptions of these fields, refer to the **BASE24 Base Files Maintenance Manual**.

The FULL MSG ENCRYPTION field must be set to a value of 1 (Full message encryption) to support full message encryption processing by the ISO Host Interface process. The MSG ENCRYPT TYPE field indicates whether the ISO host expects to receive encrypted messages or messages in the clear from the BASE24 system. This field must be set to 1 (Security module message encryption) to implement full message encryption in the ISO Host Interface process.

The CHECK DIGITS field contains the last four check digits of the current outbound message data encryption key. You can obtain these check digits from the utility used to encrypt the key or from the BASE24 ASMCOM and RSMCOM utilities. The value from this field is appended with two spaces and is placed before the BASE24 header of encrypted external messages because it must be in the clear. The host can compare the check digits received in an external message from the BASE24 system against the check digits of its current message data encryption key in use. If they are different, the host signals a key synchronization error by setting an appropriate status code in the BASE24 header and returning a 9xxx message. When the ISO Host Interface process receives such a message, it invokes the Transaction Security Services process to generate a new outbound message data encryption key and sends the new key to the host whenever a key synchronization error occurs. Note that a 9800 can still be sent but if it is a MAC sync error, the reject will be followed by a MAC key exchange.

The OUTBOUND KEY COUNTER field indicates the number of times the outbound message data encryption working key has been changed by dynamic key management processing. The number in this field is increased each time the outbound message data encryption working key is changed by the BASE24 transaction processing system. You only need to enter a value in this field if the DPC expects a value other than zero when dynamic key management is first established; otherwise the ISO Host Interface process increments it automatically and attempts to resynchronize it with the DPC.

The INBOUND KEY COUNTER field indicates the number of times the inbound message data encryption working key has been changed by dynamic key management processing. The number in this field is increased each time the inbound message data encryption working key is changed by the BASE24 transaction processing system. You only need to enter a value in this field if the DPC expects a value other than zero when dynamic key management is first established; otherwise the ISO Host Interface process increments it automatically and attempts to resynchronize it with the DPC.

All of the fields displayed under the LIMITS heading on the screen configure dynamic key management parameters for the inbound and outbound message data encryption keys. If you do not want to use a parameter for dynamic key management, leave it set to a default value of 0.

- The MSG KEY TIMER VALUE field specifies the maximum number of minutes, hours, or days the inbound or outbound message data encryption key can be used before it must be changed. The MSG KEY TIMER INTERVAL field indicates whether the value in the MSG KEY TIMER VALUE field is in minutes, hours, or days.
- The MSG KEY TRAN field specifies the maximum number of transactions that can be processed using the current inbound or outbound message data encryption key before it must be changed.
- The MSG KEY ERROR field specifies the maximum number of message encryption or decryption errors that can occur with the current inbound or outbound message data encryption key before it must be changed.
- The CONSECUTIVE MSG KEY ERROR field specifies the maximum number of consecutive message encryption or decryption errors that can occur with the current inbound or outbound message data encryption key before it must be changed.

Interface security configuration

To support message data encryption for the ISO Host Interface or IMNI Interface process, you must configure the data encryption keys and data encryption key exchange keys on the Interface Security Configuration window of the ACI desktop user interface. This record must correspond to an existing record in the KEYF. If a record does not already exist in the KEYF for the interface, you must first add one using the BASE24 KEYF Configuration window on the ACI desktop user interface or the KEYF screen in the BASE24 user access system.

The Merchant Link Interface simply carries the encrypted data (i.e., available balance and data encryption key information in tokens between the transaction acquiring BASE24 system and the transaction accepting BASE24 system.

Note: For the IMNI Interface, dynamic key management thresholds for the data encryption key are not supported in the KEYF. Therefore, the exchange of a new data encryption key can only be initiated by the IMNI interface. Dynamic key management thresholds for the data (message) encryption key for the ISO Host Interface process are supported in the KEYF and exchange of a new data (message) encryption key can be initiated by the ISO Host Interface process or by the host.

Several fields must be configured on two tabs of the Interface Security Configuration window on the ACI desktop user interface to support data encryption and decryption.

Data Encr Exch Info tab

Configure the Exchange Keys fields on the Data Encr Exch Info tab for the key exchange keys to import or export a new data encryption key. The import key is used to decrypt a new data encryption key received from an interface. The export key is used to encrypt a new data encryption key transmitted to an interface. If desired, the import key can be used for both operations. Note the following security device key entry requirements for exchange keys:

- For Atalla NSPs, these keys must be encrypted under variant 0 (Key Exchange Key Variant) of the MFK.
- For SafeNet (Eracom) ProtectHost White Mark II HSMs, data encryption exchange keys must be stored in the Mark II HSM and entered as a five-decimal index value in the Key Part 1 column.

Configure the Initialization Vector fields if needed. Row 1 contains the inbound initialization vector (IV) while row 2 contains the outbound IV.

- When using Atalla NSPs, the Initialization Vector fields are display only fields. The BASE24 system calls the security module to generate the initialization vector the first time one is needed for a CBC operation. The value does not change thereafter.
- When using SafeNet (Eracom) ProtectHost White Mark II HSMs, the Initialization Vector fields are active and you can optionally enter values in the fields. If the Transaction Security Services process cannot retrieve an initialization vector from the Interface Security File (ICHG_SECURITY), the process calls the HSM to generate one and stores it in the ICHG_SECURITY.

Data Encr Key Info tab

The following fields on the Data Encr Key Info tab must be configured for the inbound and outbound data encryption keys. The inbound key is used to decrypt or translate encrypted data received from the interface. The outbound key is used to encrypt or translate data to be sent to the interface.

- Inbound Keys (contains one or two data encryption keys, depending on whether you are using one or two layers¹ of data encryption). These keys are used to decrypt or translate encrypted data received from the interface.

- Outbound Keys (contains one or two data encryption keys, depending on whether you are using one or two layers¹ of data encryption). These keys are used to encrypt or translate encrypted data transmitted to the interface.
- DES Encr Mode (specifies whether to use the ECB or CBC mode of DES)
- Data Type (specifies whether you are using unpacked ASCII hexadecimal or binary data encryption)
- Encryption Layers (specifies whether to use one or two layers² of data encryption)

¹ When using SafeNet (Eracom) ProtectHost White Mark II HSMs, you can only enter one data encryption key because Mark II HSMs always use just one layer of data encryption.

² When using SafeNet (Eracom) ProtectHost White Mark II HSMs, this field is disabled. Mark II HSMs always use just one layer of data encryption.

All of this information entered on the Interface Security Configuration window is stored in the Interface Security File (ICHG_SECURITY). Samples of the Data Encr Exch Info and Data Encr Key Intf tabs on the Interface Security Configuration window are shown in [section 2](#).

BASE24 KEYF configuration

On the BASE24 KEYF Configuration window, you can configure fields related to message data field encryption for the ISO Host Interface process on the Message Information and Dynamic Key Limits tabs and for the IMNI Interface process on the Interac Key Management tab. Samples of these tabs on the BASE24 KEYF Configuration window are shown in [section 2](#).

On the Message Information tab of the BASE24 KEYF Configuration window, you can configure the Full Message Encryption and Message Encryption Type fields for the ISO Host Interface process. These fields are equivalent to the FULL MSG ENCRYPTION and MSG ENCRYPT TYPE fields on KEYF screen 4.

On the Dynamic Key Limits tab of the BASE24 KEYF Configuration window, you can configure dynamic key management parameters for the inbound and outbound message data encryption keys used for full message encryption by the ISO Host Interface process. The following table associates the fields on this tab with the equivalent fields on KEYF screen 4.

BASE24 KEYF	KEYF screen 4
Transaction Limit	MSG KEY TRAN
Translation Error Limit	MSG KEY ERROR
Consecutive Error Limit	CONSECUTIVE MSG KEY ERROR
Timer Limit	MSG KEY TIMER VALUE
Timer Interval	MSG KEY TIMER VALUE

On the Interac Key Management tab of the BASE24 KEYF Configuration window, you can view and update counters for inbound and outbound key exchange keys, change the index of the inbound or outbound data encryption key, and change the key exchange key for the IMNI interface. When the IMNI interface exchanges a new working key, the associated inbound or outbound key counter is incremented by one. These counters are reset to 0 when you enter and save a new key exchange key in the New Key Exchange Key field. Key exchange keys entered in this field are validated and stored in the Interface Security File (ICHG_SECURITY).

Security module configuration

Currently, data encryption is available only with Atalla NSPs or SafeNet (Eracom) ProtectHost White Mark II HSMs. Data encryption is supported for NSPs using AKB or non-AKB (variant) software. The following paragraphs discuss the configuration requirements for an Atalla NSP or SafeNet (Eracom) ProtectHost White Mark II HSM to support data encryption.

Data encryption keys

The following paragraphs discuss how data encryption keys are entered and stored in the TSS database for Atalla NSPs and SafeNet (Eracom) ProtectHost White Mark II HSMs.

Atalla NSPs

For storage in the Channel_Security or ICHG_SECURITY, the data encryption keys must be encrypted under variant 2 of the MFK or be generated with an AKB header value of 1DDNE000, depending on whether the NSP is using variant or AKB software. The same data encryption key can be used for both encryption and decryption.

SafeNet (Eracom) ProtectHost White Mark II HSMs

For storage in the Channel_Security or ICHG_SECURITY, the data encryption keys must be encrypted under variant 0 of the KM. The same data encryption key can be used for both encryption and decryption.

Key exchange keys

The following paragraphs discuss how data encryption exchange keys are entered and stored in the TSS database for Atalla NSPs and SafeNet (Eracom) ProtectHost White Mark II HSMs.

Atalla NSPs

As with other key exchange keys, the key exchange keys used for data encryption keys must be encrypted under variant 0 of the MFK or be generated with an AKB header value of 1KDNE000, depending on whether the NSP is using variant or AKB software.

SafeNet (Eracom) ProtectHost White Mark II HSMs

The key exchange keys used for data encryption keys must be stored in the memory of the HSM and entered as five-decimal index values on the Channel and Interface Security Configuration windows. These index values are stored as four-character-hexadecimal index values in the Channel_Security or ICHG_SECURITY.

Commands

Atalla NSPs and SafeNet (Eracom) ProtectHost White Mark II HSMs require specific commands or functions to be enabled in the security device to support message data encryption as described below.

Atalla NSPs

The mode of the DES algorithm used determines which commands are required in the Atalla NSP to support data encryption and decryption. To encrypt and decrypt data using the ECB mode of DES and generate the static initialization vector, you must enable the 55 command (Encrypt Or Decrypt Data Or Translate) and the 59 command (Generate MAC and Encrypt or Translate Data). To encrypt and decrypt data using the CBC mode of DES, you must enable the 55 command, the 97 command (Encrypt Or Decrypt Data), the 95 command (Reformat Initialization Vector), and the 59 command. The 10 command is always used to generate a new data encryption key. All of these commands are described below.

The 55 command (Encrypt Or Decrypt Data Or Translate) provides data encrypt or decrypt functions using the ECB mode. This command is also required to generate the static initialization vector for the CBC mode. This command is not enabled in the default factory security policy due to its high security exposure. You must purchase this command in the form of a 105 command, and enable it in the NSP's security policy using the SCT and the 108 and 109 commands, or the NSP Configuration Management menu in the SCA, if you are using the ECB mode of DES.

The 97 command (Encrypt Or Decrypt Data) provides data encrypt or decrypt functions using the CBC mode. This command is not enabled in the default factory security policy and must be enabled in the NSP's security policy using the SCT and the 108 and 109 commands, or the NSP Configuration Management menu in the SCA, if you are using the CBC mode of DES.

The 95 command (Reformat Initialization Vector) is used to reformat an initialization vector. As with the 55 command, this command is also not enabled in the default factory security policy and must be enabled in the NSP's security policy using the SCT and the 108 and 109 commands, or the NSP Configuration Management menu in the SCA, if you are using the CBC mode of DES.

The 59 command (Generate MAC and Encrypt or Translate Data) generates a MAC and encrypts or translates data. This command supports both the ECB and CBC modes of DES. This command is not enabled in the default factory security policy and must be enabled in the NSP's security policy using the SCT and the 108 and 109 commands, or the NSP Configuration Management menu in the SCA.

The 10 command (Generate Working Key, Any Type) generates a new data encryption key when the exchange of a new data encryption key is initiated by the interface.

SafeNet (Eracom) ProtectHost White Mark II HSMS

The EE0800 function supports both the ECB and CBC modes of the DES algorithm to encrypt data. Also, the EE0801 function supports both the ECB and CBC modes of the DES algorithm to decrypt data. The EE0701 and EE0702 functions are used to generate and verify MACs of the encrypted data, respectively.

Dynamic key management

You can configure dynamic key management thresholds for which a new data encryption key is automatically downloaded when one of the specified thresholds is exceeded.

For the IMNI Interface and an SPDH-compliant device, the BASE24 system forces the data encryption key to be changed at least once every daily cutover period. You can also configure two thresholds for dynamic key management of the data encryption key for an SPDH-compliant device. These thresholds are configured on the Key Thresholds screen of the ACNF.

For the ISO Host Interface process, you can configure several dynamic key management thresholds for the message data encryption key on KEYF screen 4 or on the Dynamic Key Limits tab of the BASE24 KEYF Configuration window.

For detailed information on configuring dynamic key management of data encryption keys, refer to the ***BASE24 Transaction Security Manual***.

Note: Dynamic key management does not apply to DUKPT data encryption.

7: TSS environment management

This section discusses the following topics related to managing your TSS processing environment:

- Configuring environment attributes that control various aspects of TSS processing
- Creating the System Interface Message Delivery Service Destination File (MDSDEST) for your TSS environment
- Managing metadata and metadata external memory tables
 - Modifying Metadata Configuration File (CONFCSV) parameters
 - Accessing CONFCSV and MDBCSV data from memory rather than from disk. By accessing metadata from memory rather than from disk, performance is improved and new versions of TSS data sources can be implemented in Transaction Security Services processes without incurring a complete outage of the entire TSS environment.
- Performing common tasks to maintain your TSS environment
 - Adding security devices
 - Adding Transaction Security Services processes
 - Adding Timer processes
 - Changing the IP address and database access password for the TSS environment
- Configuring and formatting reports generated from the TSS environment

Environment configuration overview

The Environment Configuration window enables configuration of environment-level control attributes. Environment-level control attributes affect general processing. For example, the environment control attribute EVT_DEST is used to control the destination for all event messages generated by the application servers.

The environment attributes specific to the TSS environment are described in [section 3](#) and [section 16](#) of this processing guide. The attributes described in section 3 are relevant to the entire TSS environment while the attributes described in section 16 are attributes used by the AKB Conversion utility only.

All application servers, regardless of whether they handle online transactions or requests from the ACI desktop user interface, access the Environment Configuration control attribute. In the event the appropriate control attribute is missing or is invalid, the application uses a default value.

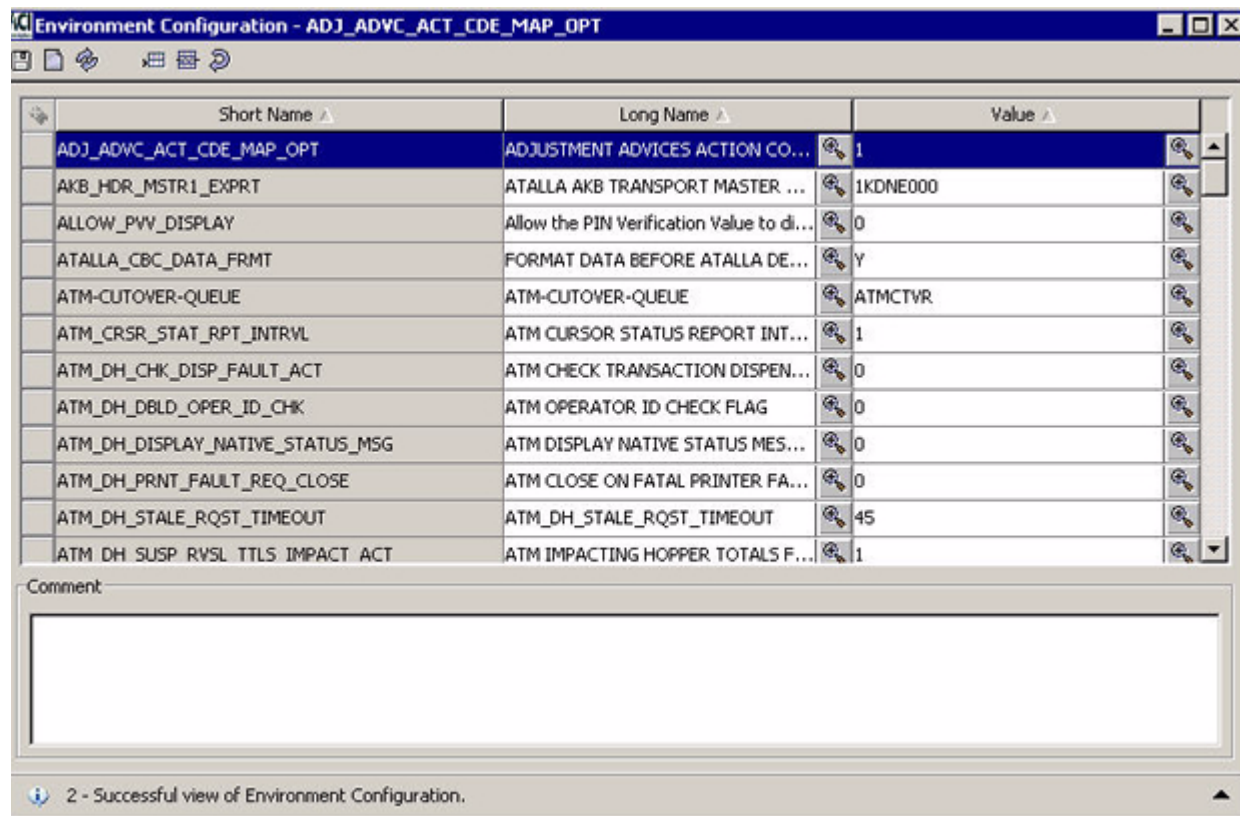
Configuring environment control attributes

The Environment Configuration user interface window is used to configure environment control attributes. A complete description of this window and configurable attributes is available from the ACI desktop user interface.

Using the ACI desktop user interface, the Environment Configuration window facilitates the configuration of environment control attributes. Environment control attributes are referred to by their *short name*.

Upon entry into the window, all environment control attributes configured in the Environment File (Environment) are displayed. The list below is not necessarily a complete list. Refer to the “[Transaction security services attributes](#)” topic in section 3 for descriptions of all environment attributes relevant to the TSS environment. When you select a row, a description of the attribute is displayed in the Comment field. If an attribute is not configured, its default value is used.

Note: Many of the attributes displayed in this example are specific to a BASE24-eps system and are not used in a TSS environment.



System Interface Message Delivery Service Destination file (MDSDEST)

The System Interface Message Delivery Service Destination File (MDSDEST) is accessed by the System Interface Services (SIS) layer of the integrated server (IS). The MDSDEST is used to map remote destinations to HP NonStop NSK Process Pair Directory (PPD) names. These include the following remote destinations:

- Hardware security device
- Timer process
- IS processes (when an XML Server process needs to send a message to an IS process for STATUS and INFO commands only)

Contents

The MDSDEST contains one entry for each remote destination. Each entry is made up of three fields: symbolic name, PPD name, and a yes or no value that indicates the format of the text sent to the remote destination. The yes or no value is referred to as the *raw send* flag.

Example Entry

```
ATALLA1, $BOX1, Y
```

In this example entry, ATALLA1 is the symbolic name and \$BOX1 is the PPD name of the Atalla Boxcar process. The Y value indicates that raw text is sent to the destination. When the indicator is set to a value of N, text is sent with XPNET headers.

Security device

For each security device (e.g., Atalla, Banksys, Bull, SafeNet (Eracom), or Thales) configured on the Security Device Configuration window, there must be a corresponding entry in the MDSDEST. The physical location of the MDSDEST is specified by the SIMDS_DEST_FILENAME parameter set using the Metadata Manager tool.

In the case of an Atalla security device connected using the TCP/IP protocol, each MDSDEST entry points to a Boxcar PPD name. For example, if a customer has three Atalla security devices using Boxcar processes (e.g., \$BOX1, \$BOX2, and \$BOX3) and the security device names on the ACI desktop user interface Security Device Configuration window are ATALLA1, ATALLA2, and ATALLA3, the MDSDEST edit file would contain the following lines:

```
ATALLA1, $BOX1, Y
ATALLA2, $BOX2, Y
ATALLA3, $BOX3, Y
```

Note: All Atalla implementations using the TCP/IP protocol require the Boxcar process.

For Banksys DEP hardware security modules, each entry points to the Remote Application Manager (RAM) process. In the following example MDSDEST entries, the names of the devices on the Security Device Configuration window are DEP1 and DEP2:

```
DEP1, $RAM.#DEP1, Y
DEP2, $RAM.#DEP2, Y
```

For Bull BNTng hardware security modules, each entry points to the Remote Application Manager (RAM) process. In the following example MDSDEST entries, the names of the devices on the Security Device Configuration window are BNTNG1 and BNTNG2:

```
BNTNG1, $RAM, Y
BNTNG2, $RAM, Y
```

For SafeNet (Eracom) security devices, each MDSDEST entry points to the SafeNet (Eracom) device itself, to a Blocker process PPD name, or to a Remote Application Manager (RAM) process, depending on whether the device is connected using an asynchronous, Ethernet, or TCP/IP communications protocol, respectively.

In the following example MDSDEST entry for an asynchronous connection, the PPD name of the device is \$EMS2000.

```
ERACOM1, $EMS2000.#TERM, Y
```

In the following example MDSDEST entry for an Ethernet connection, the entry points to a Blocker process.

```
ERACOM1, $BLKR1, Y
```

In the next example MDSDEST entry for a TCP/IP connection, the entry points to a Remote Application Manager (RAM) process:

```
ERACOM1, $RAM2.#MARKII, Y
```

For Thales e-Security HSMS, each entry points to the Racal Access Module (RAM) process. In the following example MDSDEST entries, the names of the devices on the Security Device Configuration window are RACAL1 and RACAL2:

```
RACAL1, $RAM.#RACAL1, Y  
RACAL2, $RAM.#RACAL2, Y
```

Timer processes

You must configure a corresponding entry in the MDSDEST for each timer process in the TSS environment, as shown below. You can configure multiple timer processes as needed, with each process defined for the SIS Timer Process (#STP) symbolic name. The SIS layer distributes requests to timer processes with the same symbolic name on a round-robin basis. ACI recommends that you configure more than one timer process in case one of the processes stops running. In that case, an alternate process is available. Timers contained in a Timer process are lost when the Timer process stops running.

```
#STP, $TTIM1, N  
#STP, $TTIM2, N  
#STP, $TTIM3, N
```

Note that in all these entries, the raw text flag is set to N, indicating that XPNET headers are prefixed to messages sent to timer processes.

IS process

An entry must exist in the MDSDEST for at least one Integrated Server (IS) process to which you want to send STATUS and INFO process control commands. These entries are needed in the MDSDEST to support synchronous messaging between the XML Server process and an IS

process. STATUS and INFO commands are handled through inter-process communications rather than through the XPNET process. For STATUS and INFO commands, the XML Server process must receive a response from an IS process before it can send the command response back to the ACI desktop user interface client. If an IS process does not have an entry in the MDSDEST, the XML Server process cannot send STATUS and INFO commands to that process when the destination for the command is IS.

You can either configure just one IS process in the MDSDEST to handle all STATUS and INFO commands, or you can configure all IS processes in the MDSDEST.

If you configure just one IS process, STATUS and INFO commands are sent to that process when you specify "IS" as the destination.

```
IS, $TSI1, N
```

You can either configure just one IS process in the MDSDEST to handle all STATUS and INFO commands, or you can configure all IS processes in the MDSDEST. In either case, you must enter the MDSDEST symbolic name of an individual IS process (e.g., IS1, IS2, IS3) as the destination for a STATUS or INFO command to send the command to a specified IS process. An entry must exist in the MDSDEST for the IS process specified as the destination. The symbolic names for IS processes must be unique for each IS process and start with "IS". Example process entries are shown below:

```
IS1, $TSI1, N
IS2, $TSI2, N
IS3, $TSI3, N
IS4, $TSI4, N
IS5, $AKI1, N
```

Note: IS processes include all Transaction Security Services processes and AKDS processes in the TSS environment.

Creating the MDSDEST

To create the MDSDEST entries, first enter them in an edit-type file (e.g., MDSDESTS). After you have created this edit file, you must convert it to an unstructured, uneditable file using the System Interface Services (SIS) Remote Destination File Loader program. The following is an example command for running this program on an edit file named MDSDESTS.

```
TACL> run $vol.subvol.sirdfll/in MDSDESTS/MDSDEST
```

This command creates a MDSDEST if one does not exist, otherwise it inserts any new entries from the source edit file into the existing MDSDEST.

The following is an example obey file named MDSDESTC that executes the above command. It first purges data from the existing MDSDEST before inserting entries from the source edit file.

```
FUP PURGEDATA $vol.subvol.mdsdest
```

```
run $vol.subvol.SIRDFLL0/IN $vol.subvol.MDSDESTS/vol.subvol.  
mdsdest
```

You would execute this obey file by entering the following command at the TACL prompt:

```
TACL> O[BEY] MDSDESTC
```

Warmbooting the MDSDEST

The MDSDEST is automatically warmbooted according to the value of the [SIMDS_DEST_WARMBOOT_DELTA](#) parameter in the CONFCSV.

Metadata external memory tables

Metadata external memory tables are used to access CONFCSV and MDBCSV information from memory rather than from disk. CONFCSV information is accessed from the CONFEMT; MDBCSV information is accessed from the MDBEMT. By accessing metadata from memory rather than from disk, performance is improved and new versions of data sources can be implemented in TSS environment processes on a rolling basis without incurring a complete outage of the entire TSS environment system.

Whenever you change data in the CONFCSV or MDBCSV, you must rebuild the associated external memory table using the Metadata EMT Build process. The Metadata EMT Build process broadcasts a message to all Transaction Security Services processes to inform them that the CONFEMT or MDBEMT have been replaced. If application coding changes are made in conjunction with the CONFCSV or MDBCSV changes, such that a new version of some data sources are being introduced into the system, you must also stop and restart Transaction Security Services processes on a rolling basis. If application changes are not necessary (e.g. , for changing the physical location of an assign or the MT_SIZE param for an assign), you need only warmboot the affected data sources.

The Java User Interface servlets (ESWEB) process and TCP/IP DAL Interface (TDAL) process do not receive SIS broadcast notifications automatically when the CONFEMT or MDBEMT is rebuilt. Only C++ Integrated Servers do. Therefore, when you change the CONFCSV or MDBCSV and rebuild the CONFEMT and MDBEMT, respectively, you must always stop and restart the ESWEB and TDAL processes. You must also log off and log back on the ACI desktop user interface. Thus, after rebuilding CONFEMT or MDBEMT, you must perform the following steps.

1. Log off from the ACI desktop user interface.
2. Stop the ESWEB process .
3. Stop the TCP/IP DAL Interface (TDAL) process,
4. Restart the TDAL process
5. Restart the ESWEB process
6. Log in to the ACI desktop user interface.

Note: These steps should not need to be performed very often in a production system because the metadata in a production system should rarely change.

When installing this TSS release 07.4 or later, you should also ensure that the location for the BASE24_KEYF assign is correct in the CONFCSV. The value of this assign must match the value of the KEYF/KEY6 assign in the LCONF.

This topic explains how metadata is used in SIS DAL processing, how the Metadata EMT Build program works, and provides procedures for running the Metadata EMT Build program and stopping and restarting Transaction Security Services processes on a rolling basis or warmbooting individual data sources.

SIS DAL processing

When the SIS DAL layer in a TSS process reads a record from the database, the version information is validated against the configured metadata. If the version of the record read is newer than the configured metadata, a SIS DAL error is returned.

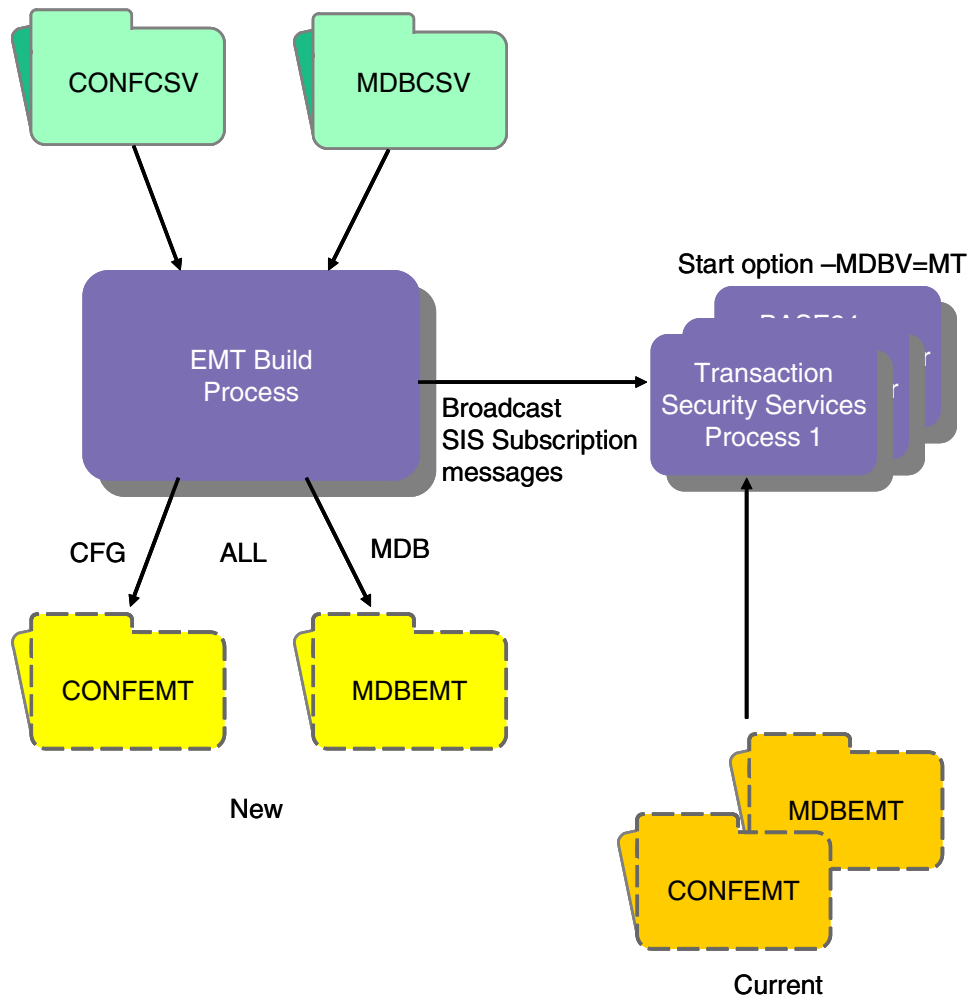
When the Metadata EMT Build program is run, the EMT Build process sends a SIS subscription message to each process started with the MT (Memory Table) value for the -MDBV start option.

- If the EMT Build process is run with the MDB option, the subscription message notifies the SIS Data Factory that a new MDBEMT external memory table is available and the process automatically switches to the new table.
- If the EMT Build process is run with the CFG option, the subscription message notifies the SIS Data Factory that a new CONFEMT external memory table is available and the process automatically switches to the new table.
- If the EMT Build process is run with the ALL option, the subscription message notifies the SIS Data Factory that new CONFEMT and MDBEMT external memory tables are available and the process automatically switches to the new tables.

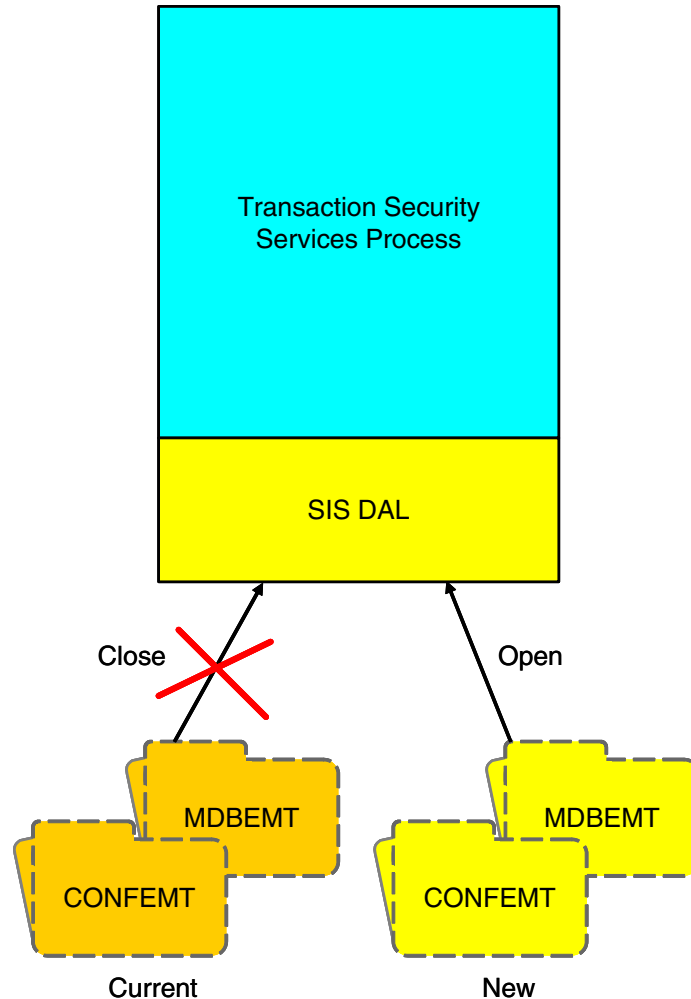
If application coding changes were made in conjunction with the metadata changes, you must also stop and restart Transaction Security Services processes on a rolling basis to pick up any new code and start using the new metadata. During the window of time when some processes are using the new version of metadata and some processes are still using the old version of metadata, if a newer version record is read, the SIS DAL layer automatically starts using the newer version instead of returning an error.

Note: Since most metadata changes also include application code changes to use the new metadata layout, processes must be stopped and restarted with new executables to pick up the new code.

The following diagram depicts EMT Build processing in a TSS environment.



After Transaction Security Services processes receive the broadcast SIS subscription message from the EMT Build process, they close the current CONFEMT and/or MDBEMT and open the new CONFEMT and/or MDBEMT as shown below.



At this point, the TSS environment process is using the new metadata, but if application coding changes were made in conjunction with the metadata changes, the code changes will not be used until the process is stopped and restarted with the new executable. You can selectively stop and restart TSS environment processes on a rolling basis and in any order to start using the new data. To minimize the impact on a production system, new metadata and accompanying application changes should only be introduced during offpeak hours.

To minimize down time, you should have several Transaction Security Services processes configured and running. You can then stop and restart some with the new executable while others continue to run. A short outage (which should be too short to be detected by an ACI

desktop user interface user) will occur when the XML Server process is stopped and restarted. The starting and stopping of Transaction Security Services processes should be coordinated with the roll-in of new executables.

You do not need to stop and restart the SIS Timer process because it does not do any DAL (i.e. database) access and does not access the MDBEMT. It uses the CONFEMT to retrieve parameters, but since it receives a subscription event when the CONFEMT is built, no user intervention is required.

Metadata EMT build configuration and startup

You can configure the Metadata EMT Build process to run as a stand-alone batch process or as a continuously running process. When started as a stand-alone process, you send the Build command when the program is started. When started as a continuously running process, the program waits until it receives the Build command.

Build command

The Build command specifies the metadata external memory tables to be built. The syntax for this command is as follows:

Syntax

BUILD *option*

where:

option is the build option the EMT Build program is to use. Valid options are as follows:

ALL	= Rebuild both the CONFEMT and MDBEMT.
CFG	= Rebuild the CONFEMT only.
MDB	= Rebuild the MDBEMT only.

Command entry methods

You can send the Build command using any of the following means:

- TACL run command or TACL macro
- Network control facility (e.g., NCPCOM)
- Process Control window

When invoking the Metadata EMT Build process from a TACL run command or TACL macro, you can run the program as a stand-alone batch process. The process stops running when finished. You would use this method when initially creating the CONFEMT and MDBEMT. In this case, a broadcast SIS subscription message is not sent to Transaction Security Services processes.

To invoke the Metadata EMT Build process from an XPNET network control facility or from the Process Control window, you must configure the process to run continuously under the XPNET process. You would use this method if changing the CONFCSV or MDBCSV after the initial CONFEMT and MDBEMT have been created. In this case, a broadcast SIS subscription message is sent to Transaction Security Services processes.

TACL run command

Use the following syntax to run the Metadata EMT Build program as a stand-alone batch process from a TACL prompt. The process stops running when finished. In this case, you do not have to configure the Metadata EMT Build process to run under the XPNET process.

Syntax

```
run $vol.objvol.EMTBLD/name $EMTB, lib $vol.xnlib.xnlib/  
-MDBV=CSV -AI=TSS -STRT=$vol.cnfgvol -BUILD={CFG | MDB | ALL}
```

Where:

vol.objvol is the volume and subvolume where the EMTBLD executable object is located.

vol is the volume where the native mode application process library (XNLIBN) resides.

vol.cnfgvol is the volume and subvolume where the MDBCSV and CONFCSV files are located.

CFG indicates to build the CONFEMT only.

MDB indicates to build the MDBEMT only.

ALL indicates to build both the CONFEMT and MDBEMT.

Example

```
TACL> RUN $DATA1.ES74CNFG.EMTBLD/NAME $EMTB, LIB $SYS.XNLIB.XNLIB/ - MDBV=CSV -  
AI=TSS -STRT=$DATA1.ESDATA -BUILD=ALL
```

TACL Macro

For convenience, you can encapsulate the above TACL run command within one or more TACL macros.

Syntax

```
?TACL MACRO
  $vol.objvol.EMTB LD/name $EMTB, lib $vol.xnlib.xnlibn/
  -MDBV=CSV -AI=TSS -STRT=$vol.cnfgvol -BUILD={CFG | MDB | ALL}
```

The following is an example TACL macro that builds both the both the CONFEMT and MDBEMT.

Example

```
?TACL MACRO
  run $DATA1.ES74CNFG.EMTB LD/name $EMTB, lib $SYS.XNLIB.XNLIBN/ -MDBV=CSV
  -AI=TSS -STRT=$DATA1.ESDATA -BUILD=ALL
```

XPNET configuration

To run the Metadata EMT Build program as a process running under the XPNET process, configure the process with the following startup options for the STARTOPTIONS attribute. In this case, the EMT Build process is always running, although it does not perform any processing until it receives a Build command:

Syntax

```
STARTOPTIONS -MDBV=CSV -AI=TSS -STRT=$vol.subvol
```

where:

vol.subvol is the volume and subvolume (e.g., \$DATA1.ES74CNFG) where the EMT Build process executable resides.

The following are example commands and attributes for creating and running the EMT Build process under the XPNET process.

```
RESET PROCESS
SET PROCESS LIBRARY $VOLUME.XNLIB.XNLIBN
SET PROCESS PROGRAM $VOLUME.ES74CNFG.EMTB LD
SET PROCESS PPD $TIS4
SET PROCESS PRIORITY 147
SET PROCESS CPU 1
SET PROCESS QAT 30
SET PROCESS QMT 75
SET PROCESS HOMETERM $SUDO
SET PROCESS SERVICE EMTB
SET PROCESS STARTOPTIONS "-MDBV=CSV -AI=TSS -STRT=$VOLUME.
ES74CNFG"
SET PROCESS REPLACEDST OFF
```



```
SET PROCESS NSKSTARTSEQ ON
```

```
ADD PROCESS P1A^EMTBLD, UNDER SYSNAME \TANDEM, UNDER NODE P1A^NODE
```

Network control facility (NCPCOM)

Using NCPCOM, you can send the Build command in a Deliver Process command. The syntax of this command is described below.

Syntax

```
DELIVER PROCESS sym-proc-name, "BUILD option"
```

where:

sym-proc-name is the symbolic name of the EMT Build process (e.g., p1a^emtld).

option is the build option the EMT Build program is to use. Valid options are as follows:

ALL	= Rebuild both the CONFEMT and MDBEMT.
CFG	= Rebuild the CONFEMT only.
MDB	= Rebuild the MDBEMT only.

Example

```
DELIVER PROCESS P1A^EMTBLD, "BUILD ALL"
```

Process Control Window

To send the Build command from the Process Control window, use the following syntax.

Process Control Syntax

Destination:	Metadata EMT Build Symbolic Process Name (e.g., P1A^EMTBLD)
Command:	Deliver
Component:	Select any component from the selectable list. It does not matter which one you select because a component is not used.

Command text: BUILD {CFG | MDB | ALL}

For example:

BUILD CFG

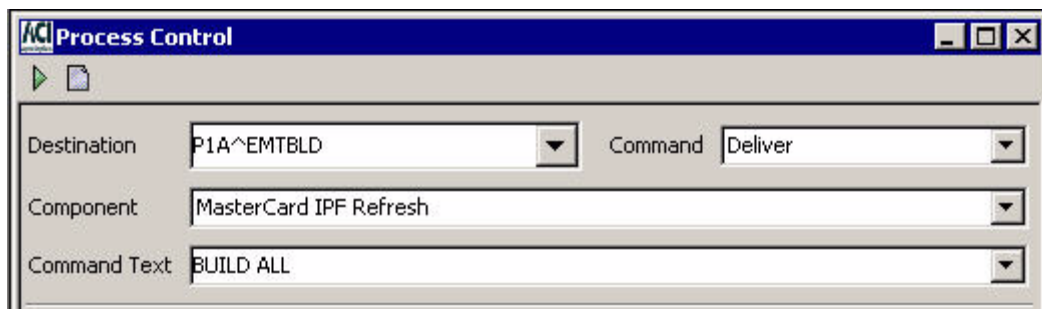
or

BUILD MDB

or

BUILD ALL

The following illustration shows the Build command being sent to the Metadata EMT Build process from the Process Control window.



Recommended implementation

The following are recommended procedures for implementing metadata external memory tables into your TSS environment.

1. Execute an initial batch run of the Metadata EMT Build process using the "BUILD ALL" command to build the initial CONFEMT and MDBEMT.

Execute the command from TACL.

The Metadata EMT Build process should be continuously running for all subsequent runs of the BUILD command.

2. Make the metadata changes using the Metadata Manager (Metaman) utility, create new CONFCSV and MDBCSV files, and replace the existing CONFCSV and MDBCSV files with the new files.
3. Send a "BUILD MDB" command to the Metadata EMT Build process. A subscription service message is broadcast to all Transaction Security Services processes running with the -MDBV=MT option. Transaction Security Services processes are now aware of the new MDBEMT, but are not yet using the updated assigns from the CONFEMT.
4. Send a "BUILD CFG" command to the Metadata EMT Build process. A subscription service message is broadcast to all Transaction Security Services processes running with the -MDBV=MT option. Transaction Security Services processes are now aware of the new assigns in the CONFEMT and will use them with any new data sources.

5. If TSS application changes accompany the metadata changes (typical scenario), stop and restart Transaction Security Services processes with the new executables on a rolling basis. Once restarted, the Transaction Security Services processes will be using the new CONFEMT and MDBEMT.

If TSS application changes are not necessary (e.g., for changing the physical location of an assign or the MT_SIZE param for an assign), then existing data sources remain open with old assigns until you send an Alter Data Source process control command to force Transaction Security Services processes to close and reopen the specified data source.

Note: If you just changed a configuration parameter value in the CONFCSV in step 2, you do not need to warmboot Transaction Security Services processes since these configuration parameters are only used by SIS functions. The Metadata EMT Build process broadcasts a subscription service message in step 4 to all SIS processes registered for the SUBEVT_PARMCHG_D subscription event.

Modifying CONFCSV parameters

Two parameters in the Metadata Configuration File (CONFCSV) control processing in the TSS environment.

The SIMDS_DEST_FILENAME parameter specifies the location of the MDSDEST. This parameter must be modified during installation to contain the physical location of the MDSDEST on your system. A sample of the SIMDS_DEST_FILENAME parameter in the CONFCSV is shown below.

```
"SIMDS_DEST_FILENAME", "1.0", "$vol.subvol.mdsdest",  
"bt_stringf", 1, 32, False, ""
```

The SIMDS_DEST_WARMBOOT_DELTA parameter specifies the interval between automatically warmbooting the MDSDEST when an unknown destination is encountered or the interval between the last two sends exceeds this configured interval. In either case, the MDSDEST is only warmbooted if the last modification timestamp of the file has changed or the MDSDEST file name has changed. Thus, there is no need to manually warmboot the MDSDEST after a change has been made. If this parameter is not configured, it defaults to 60 seconds. If this parameter is set to a value of 0, you must stop and restart processes before a modified MDSDEST file takes effect in processing.

For an unknown destination, up to three retries are performed to prevent *thrashing* (i.e., repeatedly performing an operation that keeps failing). The retries are reset when a send and wait operation completes successfully.

Note: In releases prior to TSS 07.4, the TSS data source assigns and params are read from the CONFCSV, which is read only at process startup. Therefore, there is no way to detect when the MDSDEST file name has changed. In release TSS 07.4 and later, the TSS data source assigns and params are read from the CONFEMT. Whenever the CONFEMT is built using the Metadata EMT Build utility, the new metadata is automatically read and a change in the MDSDEST file name can be detected.

A sample of the SIMDS_DEST_WARMBOOT_DELTA parameter in the CONFCSV is shown below. In this example, the parameter is set to wait 45 seconds before attempting to retry a connection to a failed destination.

```
"SIMDS_DEST_WARMBOOT_DELTA", "1.0", "45", "bt_stringv", 1, 5, True, ""
```

Note: Beginning in version 07.4 and later, any time you change the CONFCSV, you must rebuild the CONFEMT and stop and restart Transaction Security Services processes on a rolling basis. To ensure that no transactions currently in progress are lost, you should stop and restart processes in an orderly fashion. The change does not take effect until after the Transaction Security Services or Timer process is stopped and restarted.

Adding a new security device

For performance and throughput purposes, you may need to add additional security devices after your TSS environment is initially installed. Perform the following steps to add a new security device.

1. Install and configure the security device according to the vendor documentation and the information provided in [section 4](#).
2. Edit the MDSDESTS file and add an assign and PPD entry for the new security device.

```
ATALLA4, $BOX4, Y
```
3. Recreate the MDSDEST. Refer to the [“Creating the MDSDEST”](#) topic for more information.
4. Add the new security device on the Security Device Configuration window (accessed from **Configure > Transaction Security > Security Device**). Refer to [section 3](#) and [section 4](#) for detailed information on configuring security devices.
5. From the Process Control window, warmboot the Security Device Configuration File (SEC_DEV_CONFIG) using the Alter command, a destination of IS, and the Data Source component. Refer to [section 2](#) for detailed information on warmbooting files accessed as internal memory tables.

Adding Transaction Security Services processes

For performance and throughput purposes, you may need to add Transaction Security Services processes after your TSS environment is initially installed. Perform the following steps to add a Transaction Security Services process.

Note: Regular Transaction Security Services processes use the TSSISLO executable object while AKDS processes use the AKDSISLO executable object.

1. Add the new Transaction Security Services process to the XPNET node. Use the following commands to create a Transaction Security Services process from a network control facility. Modify the commands as needed for site-specific information.

Note: The `–STRT` param for the `STARTOPTIONS` attribute must be set to the volume and subvolume where the Metadata files (`CONFCSV` and `MDBCSV`) are located.

```
RESET PROCESS
SET PROCESS LIBRARY $VOLUME.XNLIBN.XNLIBN
SET PROCESS PROGRAM $VOLUME.TSS6TOBJ.TSSISLO
SET PROCESS PPD $TIS4
SET PROCESS PRIORITY 147
SET PROCESS CPU 1
SET PROCESS QAT 30
SET PROCESS QMT 75
SET PROCESS HOMETERM $SUDO
SET PROCESS SERVICE IS
SET PROCESS STARTOPTIONS "-MDBV=MT -AI=TSS -STRT=$VOLUME.
TSSTCNFG"
SET PROCESS REPLACEDEST OFF
SET PROCESS NSKSTARTSEQ ON
ADD PROCESS P1A^TSSIS4, UNDER SYSNAME \TANDEM, UNDER NODE
P1A^NODE
```

2. Edit the `MDSDESTS` file and add an assign and PPD entry for the new process.

```
IS3,$TSI4,N
```

3. Recreate the `MDSDEST`. Refer to the [“Creating the MDSDEST”](#) topic for more information.
4. Add or modify the `LCONF` assigns and params for the `BASE24` processes that you want to start using the new Transaction Security Services process. Most notably, the `SECURE-DEV1` and/or `SECURE-DEV2` assigns, or a `SECURE-DEV-xx` assign, in the `LCONF` must be set to the PPD of the new Transaction Security Services process.
5. Warmboot the `BASE24` processes for which the `LCONF` assigns and params were added or altered in the previous step.

Adding timer processes

For performance and throughput purposes, or for redundancy, you may need to add Timer processes after your TSS environment is initially installed. Perform the following steps to add a Timer process. The TSS environment accesses Timer processes in a round-robin manner when starting timers. If a Timer process is unavailable, the Message Delivery Subsystem (MDS) of the System Interface Services (SIS) within the TSS environment uses the next Timer process in its table to start the timer.

1. Add the new Timer process to the `XPNET` node. Use the following commands to create a Timer process from a network control facility. Modify the commands as needed for site-specific information.

Note: The `–STRT` param for the `STARTOPTIONS` attribute must be set to the volume and subvolume where the Metadata files (`CONFCSV` and `MDBCSV`) are located.

```
RESET PROCESS
SET PROCESS LIBRARY $VOLUME.XNLIBN.XNLIBN
SET PROCESS PROGRAM $VOLUME.TSS6TOBJ.SISTPLO
SET PROCESS PPD $TTIM2
```

```

SET PROCESS PRIORITY 147
SET PROCESS CPU 1
SET PROCESS QAT 30
SET PROCESS QMT 75
SET PROCESS HOMETERM $SUDO
SET PROCESS STARTOPTIONS "-MDBV=MT -AI=TSS -STRT=$VOLUME.
TSSTCNFG"
SET PROCESS REPLACEDEST OFF
SET PROCESS NSKSTARTSEQ ON
SET PROCESS SAVEABEND ON
ADD PROCESS P1A^TIMR02, UNDER SYSNAME \HP, UNDER NODE P1A^NODE

```

2. Edit the MDSDESTS file and add an assign and PPD entry for the new process.

```
#STP, $TTIM2, N
```

3. Recreate the MDSDEST. Refer to the [“Creating the MDSDEST”](#) topic for more information.

Changing IP addresses

The following is a procedure for changing IP addresses for the TSS environment. IP addresses may need to be changed for any number of reasons, including network conflicts, moving networks, or security concerns.

1. Change the IP address in all of the following configuration files that apply to your installation.

- HOSTS file on the configuration subvolume of the HP NonStop platform. This file contains the TSS environment IP address and a *dummy* IP address of "127.0.0.1 localhost". You need only change the TSS environment IP address.

```

172.19.66.18      oma3t02      localhost
127.0.0.1         localhost

```

- ACI desktop user interface client files (on each Windows PC):

- `<desktop-install-folder>\ES\Desktop\local.cfg`

```
ESNCSERVICE=OMA3T02:36742/ESWeb/ESTHTTPServlet
```

- `<desktop-install-folder>\ES\Desktop\localVerChk.cfg`

```
Host=OMA2U1
```

- OSS: `<eshome>/Server/VersionChecker/VersionChecker.cfg`

```
Host=OMA3T02
```

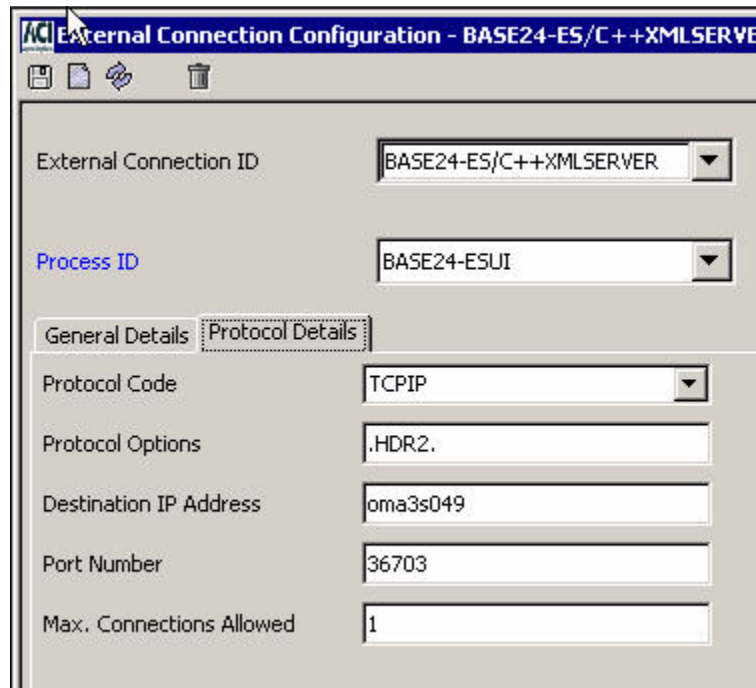
Note: If the Version Checker server resides on the same Windows server as the ACI desktop user interface client, the Host property does not need to be configured.

- OSS: `<eshome>/ESWeb/ESConfig/TEST/P1A^NODE/config.properties`

```
usec.host=1.160.10.240
```

Note: If you start the ESWEB process from OSS instead of TACL on the HP NonStop platform, the config.properties file will be named `no_ems_config.properties`.

2. Log on to the ACI desktop user interface and access the External Connection Configuration window (access **System Operations** > **Server Management** > **External Connection**).
3. Update the IP address in the Destination IP Address field on the Protocol Details tab for both of the XML Server process records (e.g., "BASE24-ES/C++XMLSERVER" and "BASE24-ES/C++XMLSERVER FORMATTED").



External Connection Configuration - BASE24-ES/C++XMLSERVER

External Connection ID: BASE24-ES/C++XMLSERVER

Process ID: BASE24-ESUI

General Details | Protocol Details

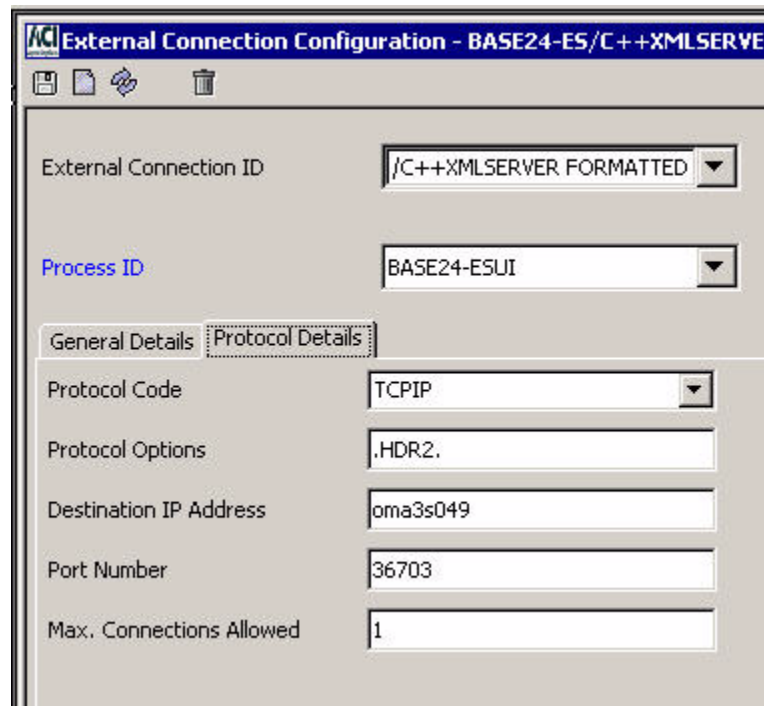
Protocol Code: TCPIP

Protocol Options: .HDR2.

Destination IP Address: oma3s049

Port Number: 36703

Max. Connections Allowed: 1



- Restart all of the following: TCPSRV process, TDAL process, Version Checker server, ESWEB process, and XML Server process.

Changing database access passwords

For security reasons, you may need to periodically change the database access password. The database access password is configured in the `db.user.password` property of the `config.properties` file for the ESWEB server process. Perform the following steps to change the database access password.

- Change the password value in the `db.user.password` property in the `config.properties` file for the ESWEB server process. Make a note of the new clear password value for use in step 3.
- Encrypt the password value in the `config.properties` file by running the `ConfigPropertyBuilder` utility. Refer to the **ACI Java Infrastructure Java Server Reference Guide** for detailed procedures.
- Encrypt the password from step 1 by running the `encpasswd` command on the new clear password value.


```
> $SISLIB/encpasswd clear-password-value
```
- Copy the resulting encrypted password value and paste it into the `MLSETUP` metadata file for the `db-user-id_UID` parameter, replacing the current `UID's PARAM_VALUE`. In the following example `MLSETUP` file, the database user ID is "tsaespa" and the encrypted password is "C48ED805".


```

metadata>cat MLSETUP
CREATE PARAM tsaespa_UID
(
  PARAM_VERSION 1.0,
  PARAM_TYPE bt_stringf(64),
  PARAM_VALUE C48ED805,
);

```

5. Run the GENERATE CONFCSV command from the MetaMan utility. Refer to the **BASE24-eps MetaMan Utility User Guide** for detailed information on the GENERATE CONFCSV command and running the MetaMan utility.

```

runmeta
generate CONFCSV DB2ZOS CONFCSV_OUT;

```

The MetaMan utility will generate a *db-user-id*_UID parameter entry in the config.csv for the parameter.

```
"tsaespa_UID", "1.0", "C48ED805", "bt_stringf", 1, 64, False, ""
```

6. Send a “BUILD CFG” command to the Metadata EMT Build process. See the “[Build command](#)” topic for details.
7. Stop and restart the ESWEB server process.

Report assign configuration and formatting

Report assigns (e.g., REFRESH_REPORT, IPF_REPORT_*interface-name*, etc.) in the BASE24-eps system always require the use of a stream data source of some type. Stream data sources are always Write Only Files (WOFs) on all supported platforms and are used either for input data sources from which data is read or for output data sources to which data is written as it is generated. ACI has created two types of stream data sources for use in the BASE24-eps system—Stream and Stream132. Each contains one record per line to be written to a WOF. The Stream data source defines up to 4062 characters per record while the Stream132 data source defines up to 132 characters per record. The Stream data source is intended for the input (e.g., refresh input data sources) or output (e.g., extract or trace data sources) of data records while the Stream132 data source is intended for generating 132-character reports. Therefore, report assigns should never use the Stream data source and always use the Stream132 (or a Stream80) data source. If desired, you can add a Stream80 table (using the Metadata Manager (Metaman) utility) to the CONFCSV and MDBCSV to contain only 80 characters per record, although many ACI reports have been designed to be 132 characters in width. Each record in the Stream132 table or a Stream80 table represents a single line in generated report output.

Note: If a report is designed to be 132 characters in width and is configured with a Stream80 data source, the page header of the report is truncated to the page width and the report data wraps to the next line with checks to make sure that words are split between multiple lines.

Caution: If you are not sure whether a configured assign is for a report or for data input or output, leave the assign entry as is! Changing an extract output or refresh input assign to use the Stream132 table or a Stream80 table instead of the Stream table will seriously corrupt extract or refresh processing.

If you add any assigns for additional processing reports in the CONFCSV, they should be configured using the MetaMan utility to use the Stream132 table or a user-defined Stream80 table, depending on the width of the report to be generated.

The following example Assign entry from the CONFCSV is configured to use the Stream132 table.

"Assigns"

```
"AssignName","TableName","TableVersion","PhysicalName","DataSourceType","ConfigString","PreloadFromAssignName"
"TBP_REPORT",,"1.0","$.#TBPRPT","WOF","",""
```

The following example Table and Element entries, respectively, from the MDBCSV are configured for the Stream132 table.

"Tables"

```
"GenAppId","GenTableId","TableName","TableVersion","SmallName","MaxPhysicalRecordSize","EndOfFixedFields","NumLogicals","NumKeys","PriKeyName","PriKeyTinyName","PriKeyOffset","PriKeyLength","SISTblVer","LegacyFileFlag","AuditFlag"
299,493,,"1.0","strm2d",,0,1,0,"","","",,0,True,False
```

"Elements"

```
"GenAppId","GenTableId","LogicalOrdinal","NumCopies","LogicalCopyNum","PhysOrdinal","ElmtName","BaseTypeName","BaseTypeSize","ElmtDim","Offset","Length","DimFlag","VarLenFlag","PadFlag","OffsetFlag","OverHeadBytes","DfltVal","DeltaFlag"
299,493,1,1,1,1,"stream_element","bt_stringv",1,,0,,True,True,False,False,0,"",False
```

If you modify the CONFCSV or MDBCSV, you must rebuild the associated external memory tables and stop and restart BASE24-eps processes on a rolling basis before the changes will take effect.

Note: ACI recommends that you modify the CONFCSV and MDBCSV only using the Metadata Manager (Metaman) utility due to unforeseen consequences that can result when modifying these metadata files using a text editor.

8: TSS event management

This section discusses the facilities and procedures used to monitor event messages generated by components in the TSS environment. It includes an overview of configuring events, recognizing events, determining the cause, effect, and remedy for an event, and detailed procedures for configuring and suppressing events.

Event configuration overview

An event is a condition of the application (e.g., initialization, termination, missing file, disk full, etc.) that warrants notification to the user. Notification occurs in the form of an event message. Event messages provide information which give clear indication as to the state or status of a component or other system entity at the instant an event is triggered. There are generally five aspects involving events. These are described below.

Event generation and suppression

Component-generated events are triggered according to conditions set by software developers. Component software developers decide if and when events will be triggered.

Event messages may be suppressed (i.e., not logged), however. An event may be suppressed always, never, or generated once for a specified time period and/or number of times the event has been triggered. When an event is suppressed, there is no indication the event occurred.

Event logging

When an event is triggered, a component supplies the subsystem ID, event number, and any required parameters that are used for text substitution. The Event component retrieves the event message information (i.e., text and emphasis) from the database and formats an event message using the supplied parameters in the message text. The event message is then sent to a logging destination. This may be a file, application, or output device.

Event monitoring

Event monitoring is the action of observing an event. This may be performed manually or automatically by external software and hardware systems.

event Response or Action

When events are observed, decisions must be made as to whether or not a response or action is required. Some events are information-only events and do not require action on the part of the observer while other events indicate symptoms of a problem (e.g., unable to write records to a file because the disk device is full).

Often times, human operators do not notice events until well after a failure has occurred. That is why many large payment system environments also use an automated event monitoring system. Automated monitoring systems continuously monitor events and are capable of correcting many of the resulting situations. Of course, the automated monitoring system is only as good as the rules by which it operates.

Log destination

The destination to which events are logged must be specified via the EVT_DEST parameter in the Environment Management window of the ACI desktop user interface. The parameter value must be the name of a valid terminal, printer, file, or application.

Recognizing events

Events appear as event messages and may be observed on any output device (i.e., terminal and printer) capable of receiving such messages. An example of an event message is shown below.

```
02-05-13;10:55:53.332 \ARGUS.$W3E10      ACI.1022.1000      20
Mgr PPD: \ARGUS.$WEDU  Generator: P3E^XML01
PCTL: Command = STATUS not supported by service = EVT
```

Ordinarily, event messages contain data such as date and time, system and subsystem, event number, and event text. The message sample above is the *brief* form of an event message. In the detailed form of an event message, additional data are available. The preceding event message can be dissected and explained as follows.

Data	Meaning
02-05-13	Date the event message was generated.

Data	Meaning
10:55:53.332	Time the event message was generated.
\ARGUS.\$W3E10	System where the event message was generated and process name.
P3E^XML01	Symbolic process name of the process which generated the event message.
1022	Subsystem ID of the event message.
20	Event message number
\ARGUS.\$WEDU	System where the event message was generated and network name.
PCTL:	Component that generated the event message.
Command = STATUS...	Event message text

Date, time, system, subsystem, process name, network name, and event number are provided by the component at the time the event is generated. The text and emphasis of the event message is retrieved from the database. Additional event message information is contained in the database and is available for viewing via the ACI desktop user interface. This information includes event cause, effect, and remedy.

Cause, effect, and remedy

Each event message is configured with a cause, effect, and remedy information. This information is intended to provide you adequate investigative information to determine the operational status of the environment and resolve any problems. Cause, effect, and remedy may be viewed using the Event Management window of the ACI desktop user interface.

Note: Java User Interface servlets (ESWEB) process error events related to the creation and connection with the database pool are not configured on the Event Management window because you cannot access this window when they occur. If you see one of these errors in your event log, check to make sure the database server is up and running verify that the database connection properties configured in the config.properties file are correct.

Subsystem ID and event number

To determine the cause, effect, and remedy, you will first need the following two pieces of data from the event message:

- Subsystem ID

- Event number

Given the following event message sample, the first line indicates the Subsystem ID is 1022 and the Event number is 20.

```
02-05-13;10:55:53.332 \ARGUS.$W3E10      ACI.1022.1000      20
Mgr PPD: \ARGUS.$WEDU  Generator: P3E^XML01
PCTL: Command = STATUS not supported by service = EVT
```

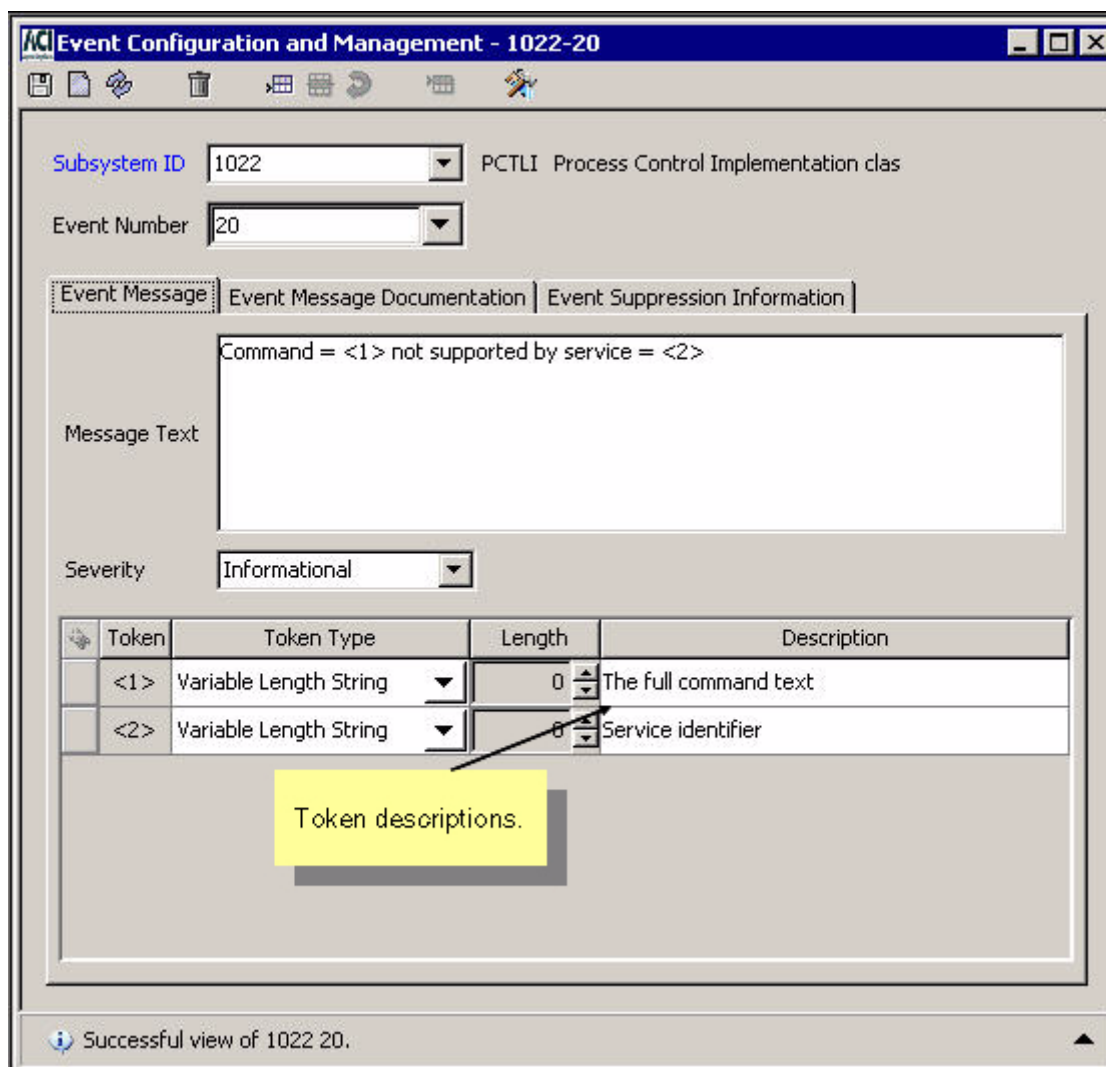
Note: Unlike other BASE24 software, subsystem IDs for components in the TSS environment are not defined in the EMCLDDLS file on the SCRIBE subvolume of your HP NonStop system. Instead, they are defined in the Event Document File (Event_Document) and mapped to events in the Event File (Event). You can view all the subsystem IDs in the Event_Document for TSS components from the Event Subsystem ID Configuration window of the ACI desktop user interface.

Event Configuration and Management window

Using the subsystem ID and event number, you can observe additional information about the event on the Integrated Server Event Configuration and Management window.

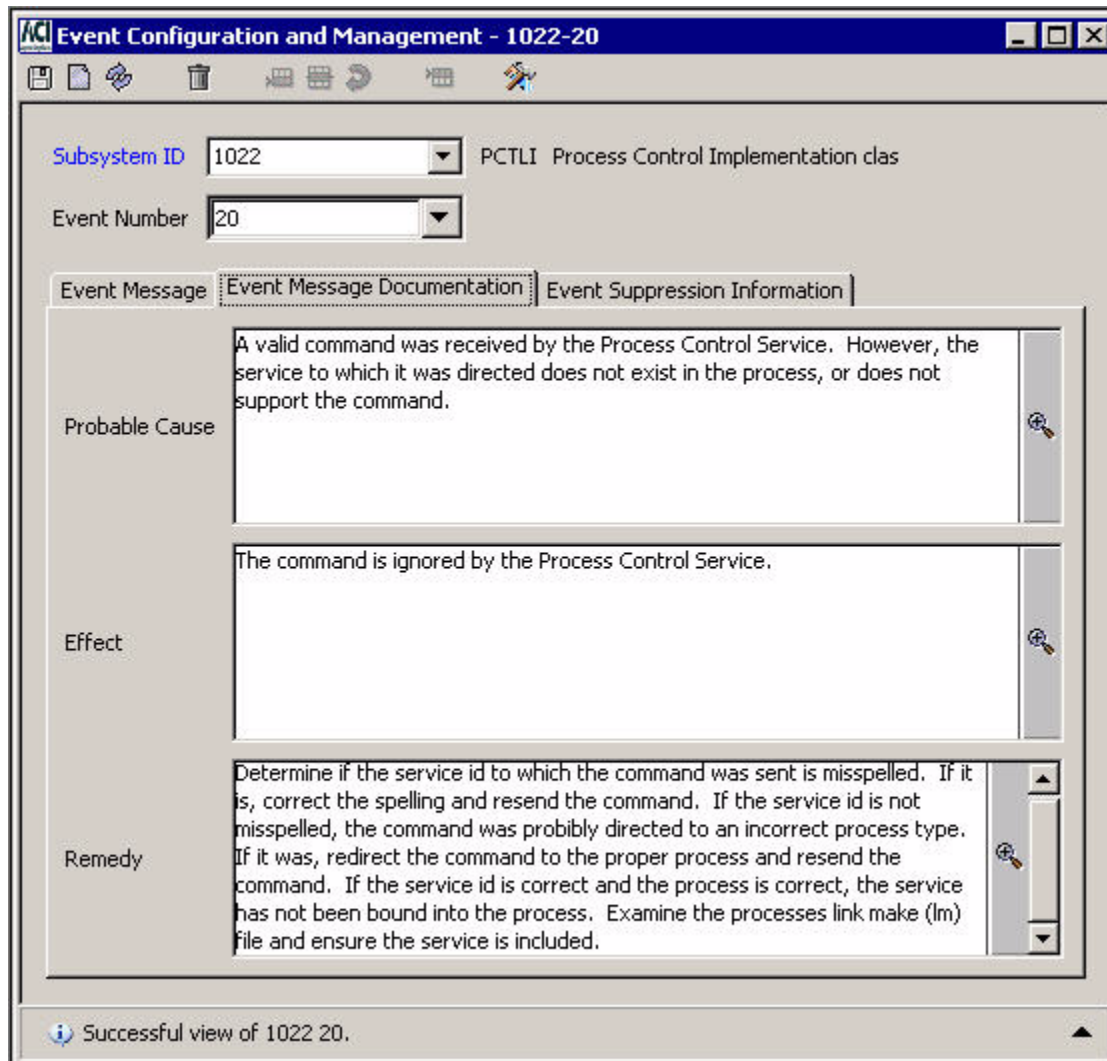
Event Message tab

On the Event Message tab of the Integrated Server Event Configuration and Management window, you can view documentation of any tokens used in the event message. The following image shows descriptions of the tokens used in the preceding event message example. In the TSS environment, tokens are variables for which the generating component substitutes specific text or values.



Event Message Documentation tab

On the Event Message Documentation tab of the Integrated Server Event Configuration and Management window, you can view the probable cause, effect, and remedy for an event. The following image shows the cause, effect, and remedy information that corresponds to the preceding event message example.



Suppressing an event message

Event suppression provides a means of reducing the number of times an event message is generated by components in the TSS environment. Event suppression may be desirable when multiple components or processes report the same error condition or when a component or process reports the same error continually.

This subsection describes how event suppression works and how it is configured in the TSS environment and includes the following topics:

- An overview of event suppression
- A discussion of event suppression methods
- A discussion of event suppression configuration on the Event Suppression Information tab of the Integrated Server Event Configuration and Management window.
- A discussion of event suppression performance impacts

Warning: Indiscriminate use of event suppression may adversely affect your ability to monitor system processing by limiting your ability to detect significant processing events if and when they should occur. Therefore, any attempts at event suppression should not be undertaken until after you have had significant experience running your TSS software in a production environment and you are familiar with the types of event messages generated.

Event suppression control is available to indicate whether the event is logged and whether to skip events. Skipping allows a number of events to be skipped once an event suppression threshold has been reached. By default, all events are logged.

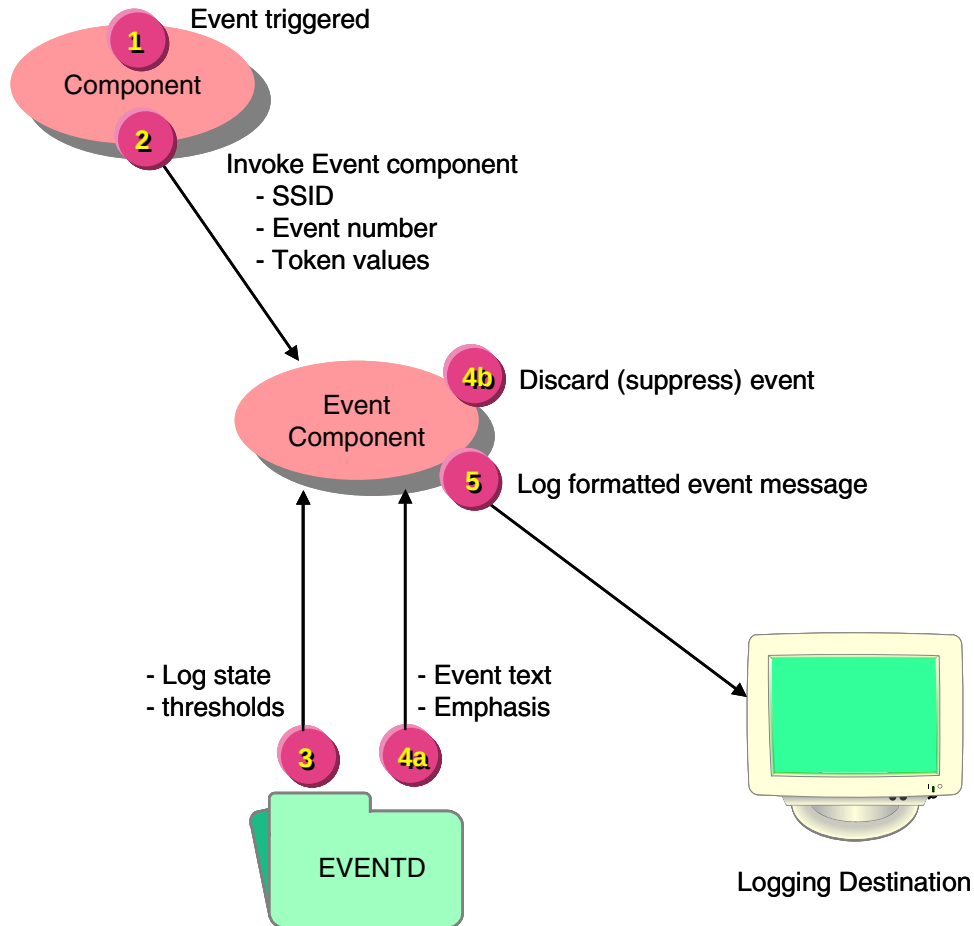
Overview

This subsection provides an overview of event suppression processing and introduces each component involved in determining whether an event is suppressed.

Note: Events associated with file errors should never be suppressed, because the same event can be generated with different file codes.

Event suppression is activated by subsystem IDs (SSIDs) and event numbers. The Event component suppresses events for the TSS environment. When an event is generated, the component that generates the event invokes the Event component using the Log Event Message member function. The Event component uses the SSID and event number provided in the request to read the Event File (Event) to determine whether the event is logged or suppressed. Suppression data entered on the Event Suppression Information tab of the

Integrated Server Event Configuration and Management window is stored in the Event. The Event component then either logs or suppresses (discards) the event accordingly. The processing involved in suppressing an event is depicted in the following illustration.



1. An event is triggered by a component.
2. The event-generating component invokes the Event component to log an event message. The originating component provides the subsystem ID, event number, and token values in the request.
3. Using the SSID and event number, the Event component retrieves the event suppression information from the Event File and determines whether the event is logged or discarded (suppressed).
4. The Event component logs or discards the event.
 - a. If the event is to be logged, the component retrieves the event text and emphasis from the Event File and formats the event.
 - b. If the event is to be suppressed, the component discards the event.
5. If the event is not suppressed, the Event component sends the formatted event to a designated logging destination.

Event Suppression Information tab

Event suppression is configured on the Event Suppression Information tab of the Integrated Server Event Configuration and Management window. A sample of the Event Suppression Information tab is shown below.

The screenshot shows a window titled "Event Configuration and Management - 1022-20". The window has a menu bar with icons for file operations and a toolbar with icons for configuration and management. The main area is divided into three tabs: "Event Message", "Event Message Documentation", and "Event Suppression Information". The "Event Suppression Information" tab is selected. It contains the following fields:

- Subsystem ID:** A dropdown menu showing "1022".
- Event Number:** A dropdown menu showing "20".
- Time Suppression Unit:** A dropdown menu showing "Minutes".
- Time Suppression Interval:** A text box with "0" and a spin button.
- Suppression Option:** A dropdown menu showing "Always Log the Event".
- Event Suppression Number:** A text box with "0" and a spin button.

At the bottom of the window, there is a status bar with the text "Successful view of 1022 20." and a small upward-pointing arrow.

Log state

The Log State field indicates the event suppression action, if any, to be taken when logging an event. Four states of event suppression are supported:

- Always Log the Event
- Never Log the Event
- Log at Network Threshold
- Log at Process Threshold

If you select either the Log at Network Threshold or Log at Process Threshold value in the Log State field, the event is suppressed according to configured thresholds. A threshold can be a count as defined in the Number of events to skip field or a time duration as defined in the Time Unit and Time Interval fields. Either or both types of thresholds can be configured for these log states.

When an event is configured to be suppressed at the network threshold, the event thresholds apply to the entire system. When an event is configured to be suppressed at the process threshold, the event thresholds apply only to a single running process so multiple events can occur across processes.

If you select the Always Log the Event value, the event is always logged. If you select the Never Log the Event value, the event is never logged.

The default log state value for an event is Always Log the Event.

Each of these log state values are described in more detail below.

Always log the event

When you select the “Always Log the Event” log state for an event, the event is never suppressed. Thus, an event will **always** be logged when this level of event suppression is selected. Since this is the default log state for handling an event, it is not necessary to configure all SSIDs and event numbers for this state. You might manually set this value for an event if you were changing from logging at the network or process threshold and you want to always allow logging of the event while retaining the threshold information used with these logging states.

Never log the event

When you select the “Never Log the Event” log state, the event is always suppressed. Thus, an event will never be logged if this log state is selected. This log state effectively cancels out an event completely.

Caution: Extreme caution and careful consideration should be made before any event is specified as *not logged*, since this level effectively prohibits the event from ever being logged.

Log at process threshold

When you select the “Log at Process Threshold” log state, the event is suppressed by each individual process running in the TSS environment. There is no global coordination of events with this log state because process-controlled suppression is implemented within the Event component in the local process. Each process monitors its own event stream. When an event is logged for which the “Log at Process Threshold” log state is used, subsequent occurrences of the event will be suppressed, based on the time interval or the number of events to skip. Thus with this level, any event will appear at least once per process before it is suppressed.

Log at network threshold

When you select the “Log at Network Threshold” log state, the event is suppressed across multiple processes running in the TSS environment. Thus, logging at a network threshold is analogous to logical network control. With this log state, the global coordination of events is performed by the Event component, which monitors events being logged within a logical network. For this log state, the event is logged when the count or time threshold is reached across all running processes. A single event is logged once during a time threshold and each running process accumulates into one count threshold. Fewer events are logged when suppressing at the network level than when suppressing at the process level, which reduces overall system resource usage.

Event suppression methods

Event suppression methods determine how an event is suppressed—either by time interval or by the number of events to skip. These event suppression methods are only used for the following log states:

- Log at Process Threshold
- Log at Network Threshold

These event suppression methods are not used for the following log states:

- Always Log the Event
- Never Log the Event

One or both of the methods can be configured for an event. If both methods are configured, whichever condition occurs first causes the event to be logged again.

Suppression by number of events to skip

Suppression by the number of events to skip causes some attempts to log an event to be suppressed. Each attempt to log the event increases a counter by one. When the counter reaches the defined threshold, the event is logged and the counter is reset. The threshold for this event suppression method is set in the Number of events to skip field, which can be set from 0–9999 and defaults to 0. When this field is set to 0, suppression by the number of events to skip is not used.

Suppression by time interval

Suppression by time interval causes all subsequent occurrences of an event to be suppressed for a specified time interval, after the event is initially logged once. This threshold is set in the Time Interval and Time Unit fields:

- The Time Unit field indicates whether time interval value is measured in seconds or minutes.
- The Time Interval field defaults to a value of 0, which indicates that suppression by time interval is not used.

When the Time Unit field is set to Minutes, you can set the Time Interval field to a value from 0–166 (minutes). When the Time Unit field is set to Seconds, you can set the Time Interval field to a value from 0–9999 (seconds).

Note: If you select Seconds in the Time Unit field and enter a number in the Time Interval field that is evenly divisible by 60, the system will convert that number to minutes. When you redisplay the event suppression configuration information, the Time Unit field will display Minutes and the Time Interval field will contain a value in minutes. If the number of seconds that you originally enter in the Time Interval field is not evenly divisible by 60, the system will not convert the value to minutes.

Configuring events

There are two areas of configuration related to event management. These are:

- **Environment Management**

The Environment Management window is used to configure the event message destination device name parameter.

- **Event**

The Integrated Server Event Configuration and Management window is used to configure the event message text, emphasis, cause, effect, and remedy. Additionally, control is available that enables suppression of event messages.

The Event Subsystem ID Configuration window is used to configure subsystem ID, name, and description.

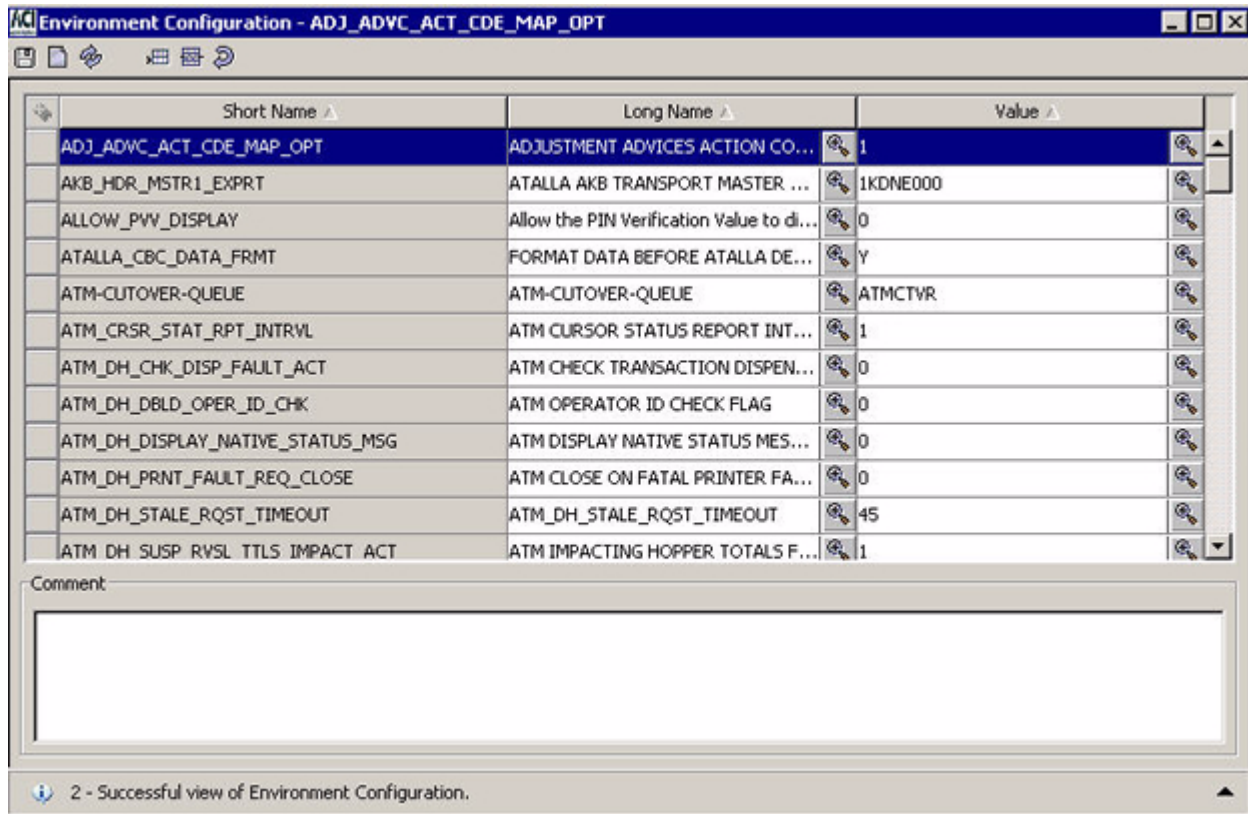
Using the ACI desktop:

1. Access **System Operations > Environment**

Configure the environment parameter called EVT_DEST so that the parameter value is a valid event message destination.

The following illustration shows an example Environment Configuration window.

Note: Many of the attributes displayed in this example are specific to a BASE24-eps system and are not used in a TSS environment.



Short Name	Long Name	Value
ADJ_ADVC_ACT_CDE_MAP_OPT	ADJUSTMENT ADVICES ACTION CO...	1
AKB_HDR_MSTR1_EXPRT	ATALLA AKB TRANSPORT MASTER ...	1KDNE000
ALLOW_PVV_DISPLAY	Allow the PIN Verification Value to di...	0
ATALLA_CBC_DATA_FRMT	FORMAT DATA BEFORE ATALLA DE...	Y
ATM-CUTOVER-QUEUE	ATM-CUTOVER-QUEUE	ATMCTVR
ATM_CRSR_STAT_RPT_INTRVL	ATM CURSOR STATUS REPORT INT...	1
ATM_DH_CHK_DISP_FAULT_ACT	ATM CHECK TRANSACTION DISPEN...	0
ATM_DH_DBLD_OPER_ID_CHK	ATM OPERATOR ID CHECK FLAG	0
ATM_DH_DISPLAY_NATIVE_STATUS_MSG	ATM DISPLAY NATIVE STATUS MES...	0
ATM_DH_PRNT_FAULT_REQ_CLOSE	ATM CLOSE ON FATAL PRINTER FA...	0
ATM_DH_STALE_RQST_TIMEOUT	ATM_DH_STALE_RQST_TIMEOUT	45
ATM_DH_SUSP_RVSL_TTLS_IMPACT_ACT	ATM IMPACTING HOPPER TOTALS F...	1

Comment:

2 - Successful view of Environment Configuration.

2. Access **System Operations > Event > Event Management**

Though no configuration is required, text, emphasis, cause, effect, and remedy information may be reconfigured (i.e., information added, changed, or deleted) to suit local operational requirements. However, it is suggested that you do this with caution. Changing and deleting this information may impact your ability to resolve problems efficiently.

The following illustration shows an example of event message text and emphasis. The text with $\langle n \rangle$ is a token, where n is a number where text substitution occurs with items that vary between event messages.

Event Configuration and Management - 1022-20

Subsystem ID: 1022 PCTLI Process Control Implementation clas

Event Number: 20

Event Message | Event Message Documentation | Event Suppression Information

Message Text: Command = $\langle 1 \rangle$ not supported by service = $\langle 2 \rangle$

Severity: Informational

Token	Token Type	Length	Description
$\langle 1 \rangle$	Variable Length String	0	The full command text
$\langle 2 \rangle$	Variable Length String	0	Service identifier

Successful view of 1022 20.

Event Message Documentation (i.e., cause, effect, and remedy information) is available on the Event Message Documentation tab.

The following illustration shows an example of event management documentation.

Event Configuration and Management - 1022-20

Subsystem ID: 1022 PCTLI Process Control Implementation clas

Event Number: 20

Event Message | **Event Message Documentation** | Event Suppression Information

Probable Cause
A valid command was received by the Process Control Service. However, the service to which it was directed does not exist in the process, or does not support the command.

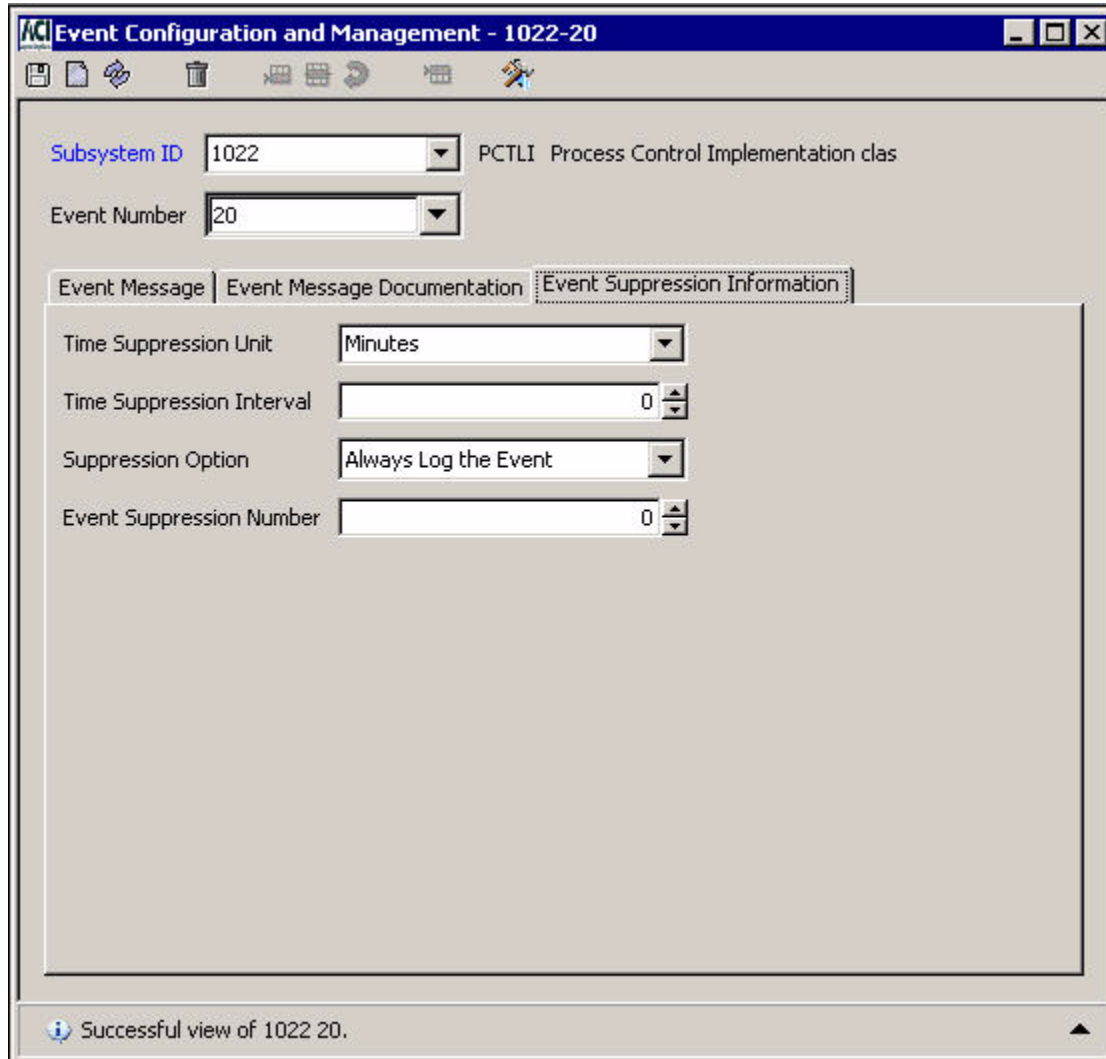
Effect
The command is ignored by the Process Control Service.

Remedy
Determine if the service id to which the command was sent is misspelled. If it is, correct the spelling and resend the command. If the service id is not misspelled, the command was probably directed to an incorrect process type. If it was, redirect the command to the proper process and resend the command. If the service id is correct and the process is correct, the service has not been bound into the process. Examine the processes link make (lm) file and ensure the service is included.

Successful view of 1022 20.

Event suppression control is available to indicate whether or not the event is logged and whether or not to skip events. If logged, further suppression control is available according the process and network threshold values. Skipping allows a number of events to be skipped once an event suppression threshold has been reached. By default, all events are logged. ACI Online Help, available from the ACI desktop user interface, provides an explanation of the event suppression fields.

The following illustration shows an example of event suppression information.

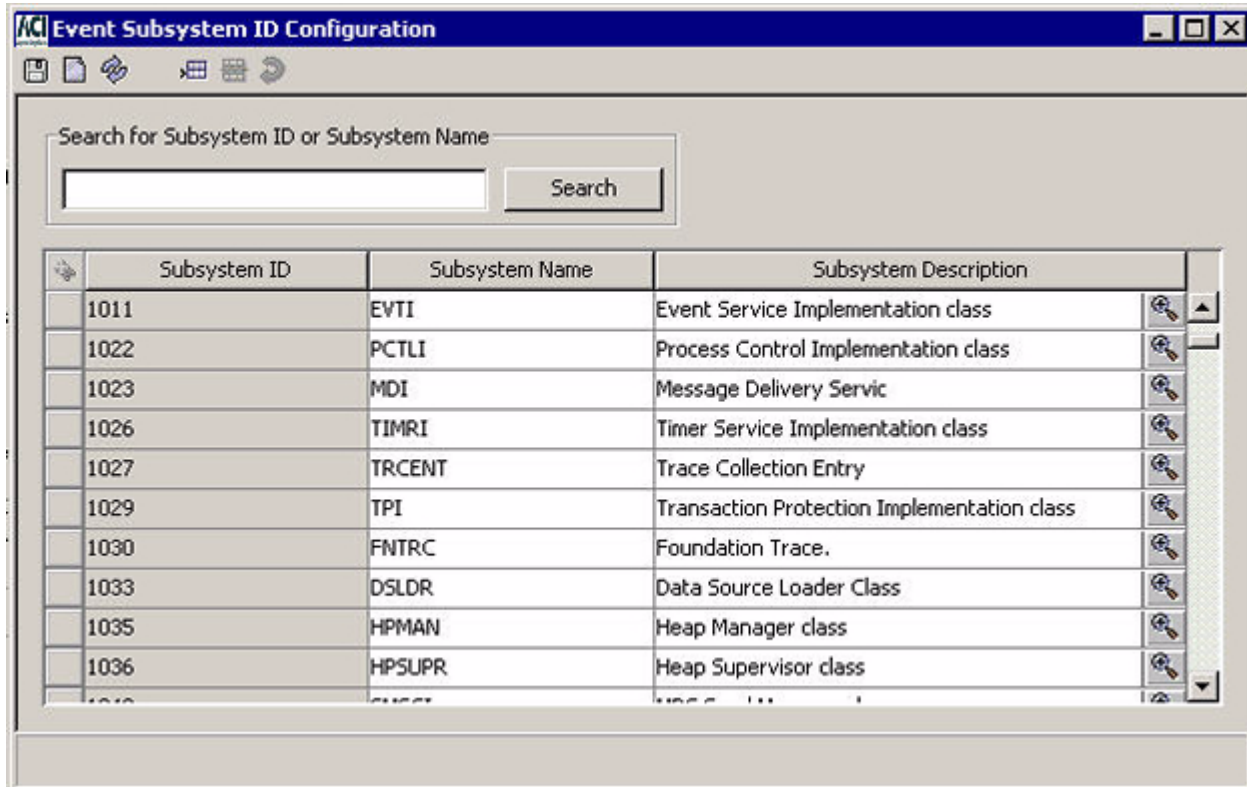


In this example events are not suppressed. All events are allowed to the event logging destination.

3. Rebuild and warmboot the optional Event external memory table (Event_OLTP) if you change any data on the Event Message or Event Suppression Information tabs on the Event Configuration and Management window. You enter the process control commands from the OLTP Management window or the Process Control window.
4. Stop and restart the ESWEB Server process. The ESWEB Server does not start using the rebuilt Event_OLTP until after it is stopped and restarted.
5. Access **System Operations > Event > Subsystem ID**.

No configuration is required here. Subsystems are preconfigured and should not be altered.

The following illustration shows an example Event Subsystem ID Configuration window.



9: Process control

This section describes the process control facilities provided in the TSS environment. It includes an overview of the functions you can perform from the Process Control window, describes the Process Control window and its fields, and lists the process control commands available in the TSS environment.

Process control overview

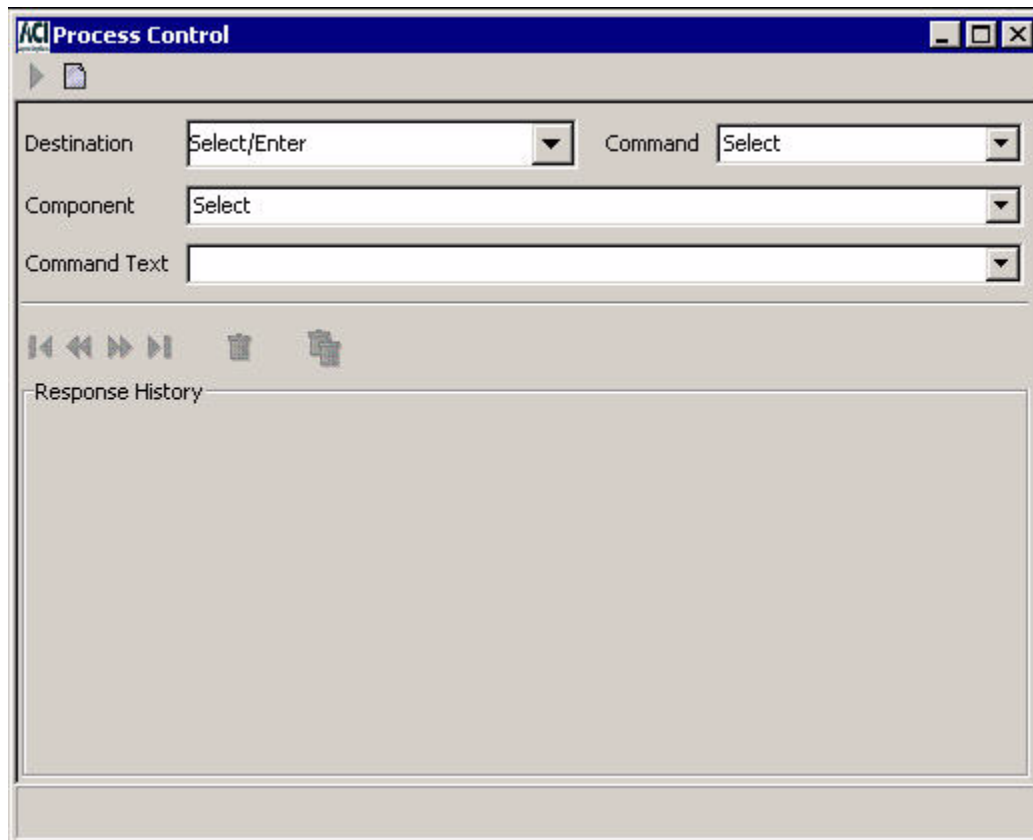
Process Control is an interactive facility that provides users operational control of the TSS environment. Operational requirements vary from environment to environment.

The following are some of the common operational control tasks you can perform.

- Obtaining and altering environment configuration information
- Obtaining status information
- Reloading an external or internal memory table
- Obtaining and resetting statistics
- Initializing, enabling, and disabling a security device
- Enabling or disabling audit store-and-forward processing

Process Control window

The Process Control window is available under the System Operations menu option of the ACI desktop user interface. Control is facilitated using selectable lists, destination entry, and keyword entry. A sample of the Process Control window is shown below.

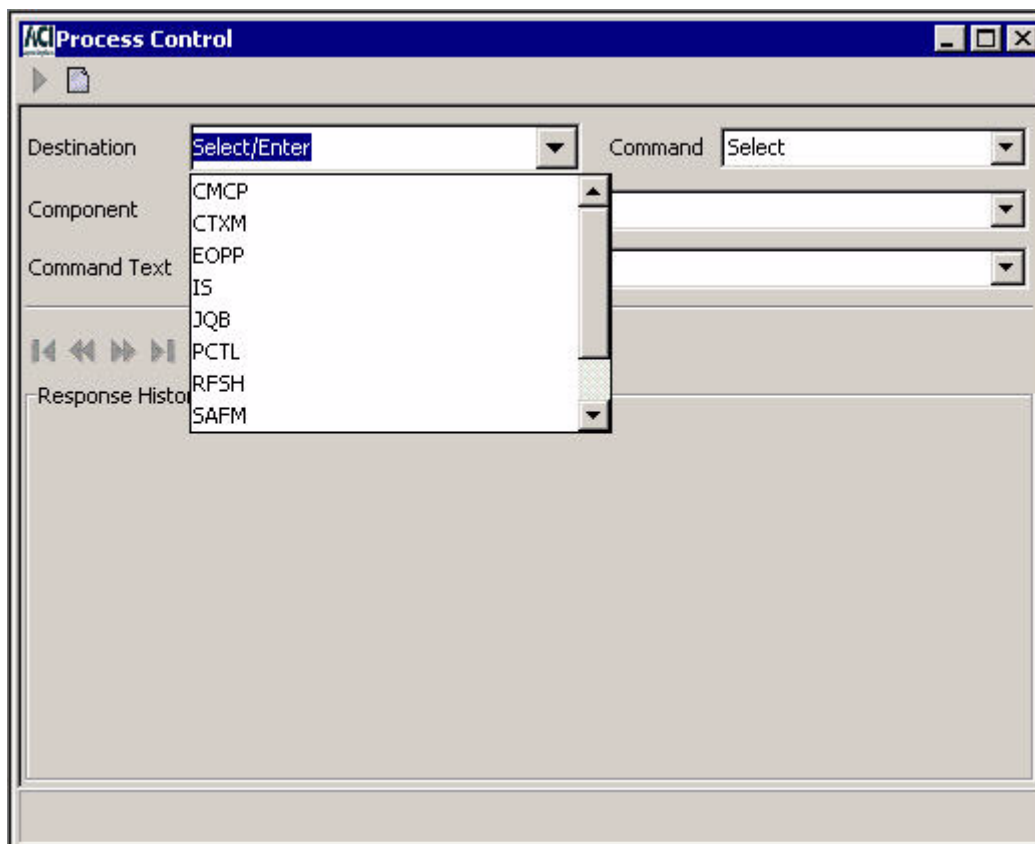


Destinations

Generic destinations provided in a selectable list include all the application servers available in a BASE24-eps system. Only the Process Control (PCTL), Integrated Server (IS), and C++ User Interface Server (XMLS) process destinations are used with the Transaction Security Services process in a BASE24 system. The destination of PCTL sends commands to all components registered to use the command, regardless of the type of process in which they are found.

Because the Transaction Security Services process statically registers as an IS process just like the Integrated Server process in a BASE24 for Enhanced Authorization system, you can run into process control destination conflicts if both the Transaction Security Services process and BASE24 for Enhanced Authorization are installed on the same system. A workaround for this conflict is to configure a separate XPNET node for just your Transaction Security Services processes and another XPNET node with the XML Server process and at least one Integrated Server process. By doing so, all Deliver commands are directed to the local “IS” process (Transaction Security Services or Integrated Server) within that node and all broadcast messages sent to a destination of IS will also be delivered to the Transaction Security Services processes.

Note: The CMCP, CTXM, EOPP, JQB, RFSH, SAFM, and TBP selections are not applicable and are not used in the TSS environment.

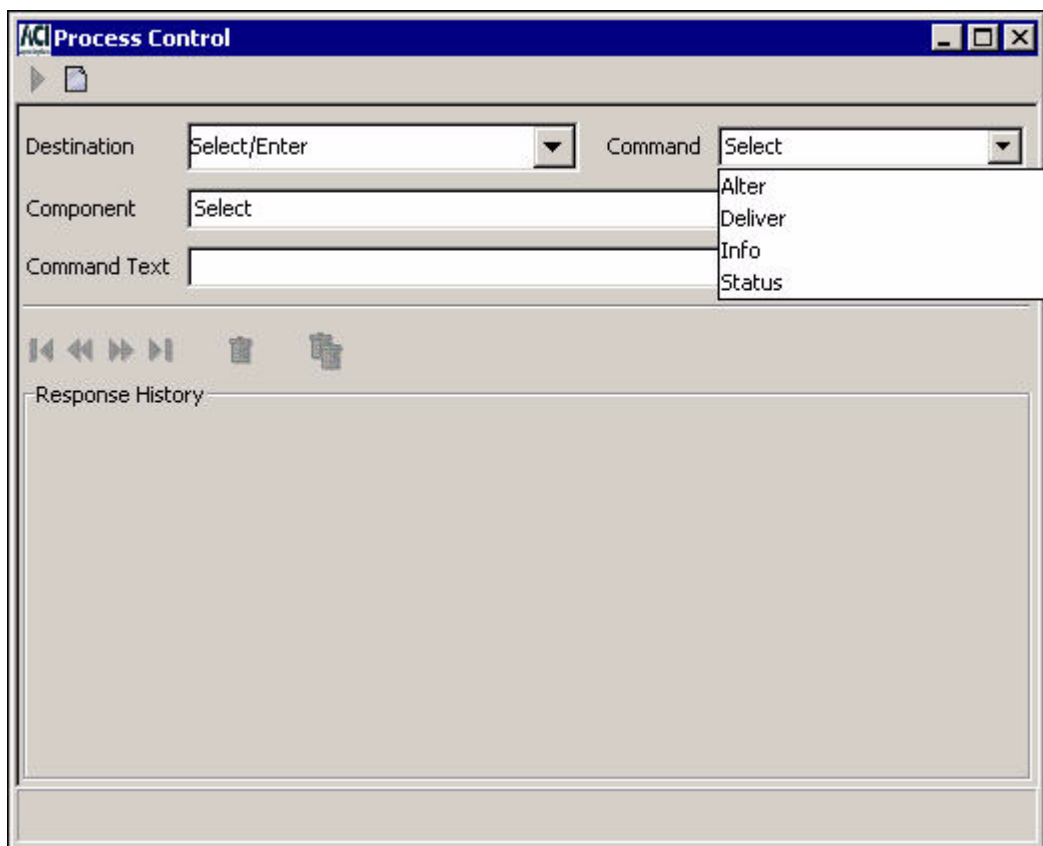


Besides the entries in the selectable list, you can also enter the symbolic name of a single process when entering an ALTER or DELIVER command. When you send an ALTER command to an entry in the selectable list (other than the PCTL destination), it is broadcast to all processes configured with a matching SERVICE attribute configured in the XPNET node. When you send an ALTER command to the symbolic name of an individual process, it is not broadcast to any other processes.

When entering an INFO or STATUS command, you must enter the assign specified for a single Transaction Security Services process in the System Interface Message Delivery Service Destination File (MDSDEST). For example, assume you have 12 Transaction Security Services processes with assign names of "IS n " in the MDSDEST, where n is a number from 1 to 12. To send an INFO or STATUS command to the first Transaction Security Services process, you would enter "IS1" in the Destination field. To send an INFO or STATUS command to the twelfth Transaction Security Services process, you would enter "IS12" in the Destination field.

Commands

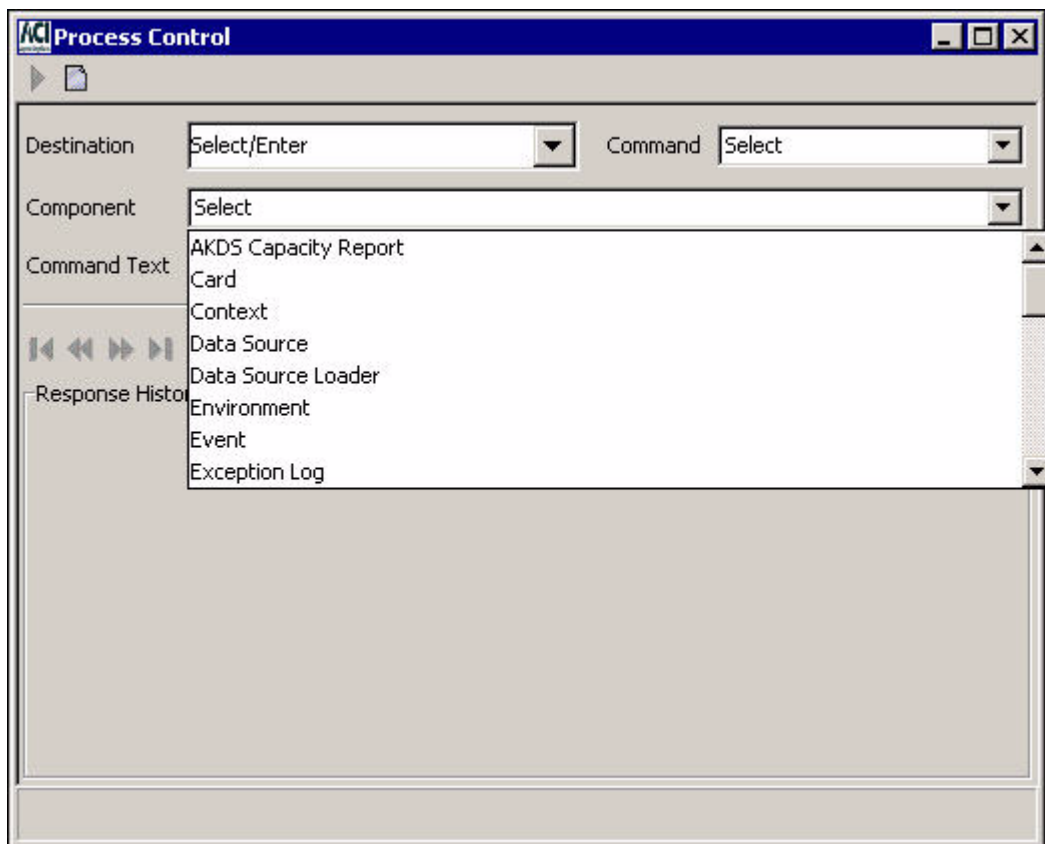
Allowed commands include Alter, Deliver, Info, and Status. Refer to ACI Online Help for an explanation of the commands.



ALTER commands are broadcast to all processes that have been configured to the XPNET node with a service value equal to the value entered in the Destination field. For example, ALTER commands are sent to all Transaction Security Services processes configured to the XPNET node with a SERVICE attribute value of "IS" when you enter "IS" in the Destination field. DELIVER commands are sent to a specified process. If you send an ALTER command to symbolic process name, it is only sent to the specified process. For DELIVER commands, you can enter a generic destination (i.e., IS or XMLS) or a symbolic process name (e.g., P1A^TSSIS1, P1A^TSSIS3, P1A^XMLS01) in the Destination field. INFO and STATUS commands are sent to a single Transaction Security Services process. For these commands, the value entered in the Destination field must match the assign name defined for the Transaction Security Services process in the MDSDEST.

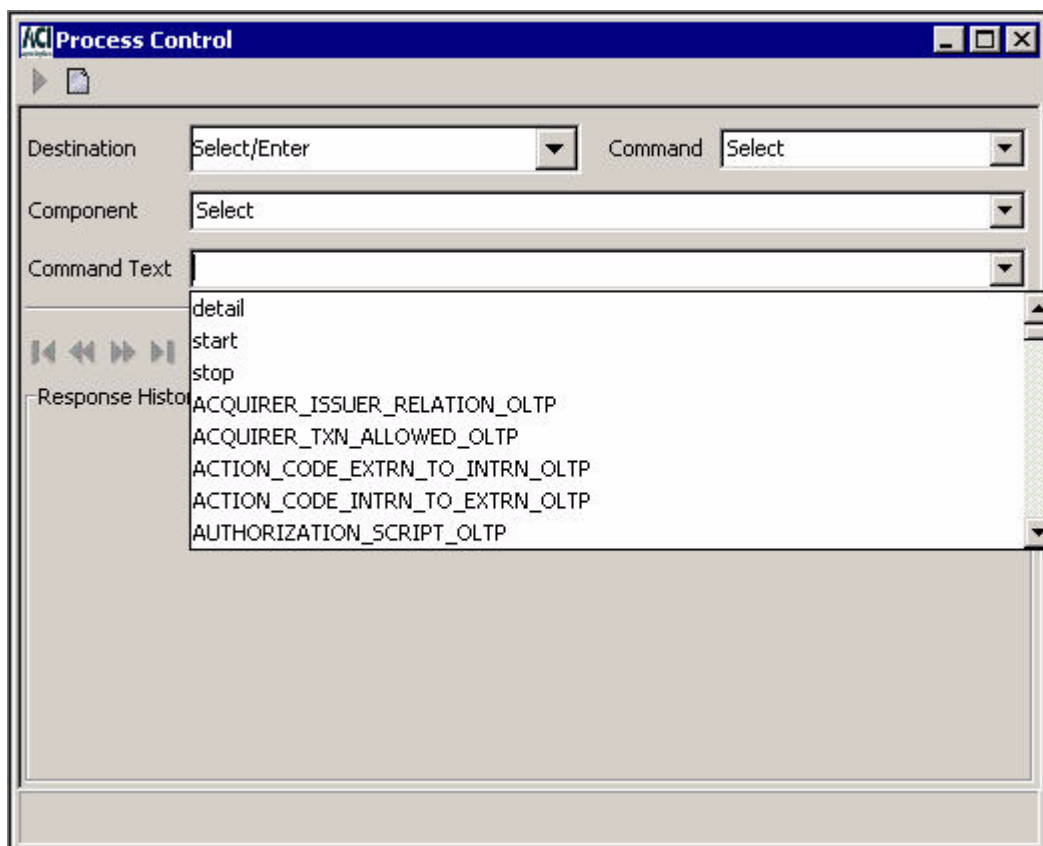
Components

Available components vary from system to system and may include some or all of those shown below.



Command Text

The default command text keywords are shown below. The supported command text keywords depend on the components available in the respective application servers. Refer to the [“Process control commands”](#) topic or ACI Online Help for the ACI desktop user interface for a list of components and the related command text keywords.



When the text for a command includes multiple items or variables, you can separate the items or variables using a comma or a space in the Command Text field.

Process control commands

The following table describes the Process Control commands available for the TSS environment.

You can enter all of these commands from the Process Control window on the ACI desktop. The syntax for entering commands on the Process Control window is provided for each command. You can also enter most of these commands from an XPNET network control facility (i.e., NCS screen, NCPCOM). If a command can be entered from a network control facility, the NCPCOM command syntax is also provided.

AKDS Capacity report

The AKDS Capacity Report component generates a report of the total number of ATM terminals that have been authenticated and exchanged public keys with the BASE24 system. ACI uses this report to reconcile your AKDS usage (number of ATM terminals) against your AKDS capacity license.

Generate AKDS Capacity report

The Generate AKDS Capacity Report command generates the AKDS Capacity Report. When you enter this command with the optional DETAIL key word, the component includes the channel ID of each ATM terminal defined in the Channel Public Key Security File (CHAN_PKEY_SEC) in the report output, along with the total number of terminals defined in the CHAN_PKEY_SEC. When you enter this command without the optional DETAIL key word, the component only includes the total number of terminals defined in the CHAN_PKEY_SEC in the report output.

Command syntax

Destination:	XMLS
Command:	Deliver
Component:	AKDS Capacity Report
Command text:	[DETAIL]

Data Source component

The following process control commands are available to the Data Source component.

Alter data source

The Alter Data Source command (also referred to as the OLTP Warmboot command) causes the Data Source component to open, reread, and close the specified file (i.e., external or internal memory table). You can use this command to perform the following:

- Start using an external memory table (OLTP) after it has been reloaded using the Load Data Source command
- Start rebuilding an internal memory table after its associated disk file has been modified

- Start using new partitions and closing old partitions in a partitioned file.

The memory tables used in the TSS environment are described in [section 2](#).

You can also use this command to open a new file for any file in the CONFCSV if the physical file has changed, the CONFCSV entry was changed, or the old file was renamed and a new file was put in its place. This applies to any file in the CONFCSV rather than just those files with associated memory tables.

Note: When you send this command to the PCTL destination, processes of all types in the system attempt to close and reopen (re-read) the memory table, thereby accessing the new data in processing. When using the PCTL destination, you may see informational event messages logged for any processes that attempt to close and reopen the memory table if they do not actually use the table.

Process control syntax

Destination: IS or PCTL

Command: Alter

Component: Data Source

Command text: *data-source-name*

where: *data-source-name* is the assign name specified for the external memory table (OLTP) or disk file in the CONFCSV on the data subvolume. For example, the assign name for the IDESOD external memory table (OLTP) is IBM_DES_OLTP. The assign name for the Channel Security Configuration File (Channel_Security) is CHANNEL_SECURITY.

NCPCOM syntax

DELIVER PROCESS *object-name*, "ALTER DASRC *data-source-name*"

object-name — The symbolic name assigned to a Transaction Security Services process in the XPNET node.

data-source-name — See process control command text above.

Data Source Loader component

The following process control commands are available to the Data Source Loader component.

Deliver data source

The Deliver Data Source command (also referred to as the OLTP Rebuild command) loads the specified file (i.e., external memory table) from another file (i.e., data file). Use this command to reload an external memory table (OLTP) after you have added, updated, or deleted data in the underlying data file. After the external memory table (OLTP) is reloaded, use the Alter Data Source command to begin using the external memory table (OLTP) in processing. The relationships between ACI desktop windows and the underlying data files they maintain are described in [section 2](#).

Process control syntax

Destination: XMLS
Commands to rebuild external memory tables (OLTPs) should be sent to the User Interface (XML Server) process instead of a Transaction Security Services process to avoid any impact to online transaction processing performance.

Command: Deliver

Component: Data Source Loader

Command text: *oltp-assign-name*
where: *data-source-name* is the assign name specified for the external memory table (OLTP) in the CONFCSV on the data subvolume. For example, the assign name for the IDESOD external memory table (OLTP) is IBM_DES_OLTP.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER DSLDR *oltp-assign-name*"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

oltp-assign-name — See process control command text above.

Environment component

The following process control commands are available to the Environment component.

Alter attribute

The following attribute change command rereads the specified attribute or all attributes (denoted by *) and notifies registered clients to begin using the new values.

Use this command after you have added, updated, or deleted attributes on the Environment Configuration window. Modified attribute values are not used in processing until they are reread using this command.

Process control syntax

Destination: Any selection from the Destination field selectable list or a symbolic process name

Command: Alter

Component: Environment

Command text: *attribute-name* | *

where: *attribute-name* is the short name of a specified environment attribute. An asterisk (*) specifies all environment attributes. The environment attributes available in the TSS environment are described in [section 3](#) and [section 16](#).

NCPCOM syntax

DELIVER PROCESS *object-name*, "ALTER ENVMT *attribute-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

attribute-name — See process control command text above.

Info attribute

The Info Attribute command reports on a specified attribute or on all attributes (denoted by *) from C++ classes that have been constructed (i.e., used) by the specified destination process since the process was last started. Thus, you may not see all the Environment attributes that could be used in the response to this command. This process control command is intended to be used only in a support situation to view the values processes are using for the environment variables they have registered. It does not provide a view of all Environment attributes that have been configured and could be used by the specified process. If an attribute specified in this command has not been registered, an event message is generated.

This command displays the current value and default value for the specified attribute or all attributes that have been registered. The reported information for the attributes is displayed in the Response History area of the Process Control window.

Process control syntax

Destination: XMLS or the assign name defined for a Transaction Security Services process in the MDSDEST

Command: Info

Component: Environment

Command text: *attribute-name* | *

where: *attribute-name* is the short name of a specified environment attribute. An asterisk (*) specifies all environment attributes.

Event component

The following process control commands are available to the Event component.

Reset suppression

The Reset Suppression command clears all events in the current memory collection that contain event counts and timers for suppressed events.

Process control syntax

Destination: Any selection from the Destination field selectable list or a symbolic process name

Command: Alter

Component: Event

Command text: RESETSUPPRESION

NCPCOM syntax

DELIVER PROCESS *object-name*, "ALTER EVT RESETSUPPRESSION"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

Event report generation

The REPORT command generates a report of all possible events that can be generated by a specified subsystem in the TSS environment or by all subsystems in the TSS environment. For each event, the report provides the same information you can view online on the Event Message and Event Message Documentation tabs of the Event Configuration and Management window on the ACI desktop or in the ***BASE24-eps Event Documentation*** document. The event information is retrieved from the Event data source (Event) and the Event Documentation data source (Event_Document). The major difference between the generated report and the ***BASE24-eps Event Documentation*** document is that the report does not include a table of contents and forces a page break each time the SSID changes. The generated report output includes an ACI copyright statement and has a title of "BASE24-eps Event Documentation."

The generated output for the REPORT command is written to the physical location specified by the EVT_REPORT assign in the CONFCSV. The EVT_REPORT assign is configured as a Stream data source.

Command syntax

Destination: XMLS
 Command: Deliver
 Component: Event
 Command text: REPORT [ssid]
 where:
 ssid is an optional subsystem ID.

An example of the report output for the es.server.formatter.xml.
 MessageSchemaManagersubsystem (SSID 5512) is shown below.

© 2009 by ACI Worldwide Inc. All rights reserved.

All information contained in this documentation, as well as the software described in it, is confidential and proprietary to ACI Worldwide, Inc., or one of its subsidiaries, is subject to a license agreement, and may be used or copied only in accordance with the terms of such license. Except as permitted by such license, no part of this documentation may be reproduced, stored in a retrieval system, or transmitted in any form or by electronic, mechanical, recording, or any other means, without the prior written permission of ACI Worldwide, Inc., or one of its subsidiaries.

ACI, ACI Worldwide, and the ACI product names used in this documentation are trademarks or registered trademarks of ACI Worldwide, Inc., or one of its subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Event Documentation
 BASE24-eps™

This document contains the documentation for the log messages that may be generated by the BASE24-eps.
 Log messages are documented in order by log message number.

```
*****
SSID : 5512 UIJS      : es.server.formatter.xml.MessageSchemaManager
*****
Event Number  : 1      (SSID 5512)
Severity      : Critical
```

Text : Exception occurred during processing of {0}: {1}
Probable Cause: Exception could be due to configuration or message issues.
Exception logged should assist to identify problem.
Action Taken : No further processing is performed. Denied response returned.
Remedy : Identify/correct exception in configuration or class code.

Event Number : 2 (SSID 5512)
Severity : Informational
Text : MessageSchemaManager starting.
Probable Cause: Standard startup message.
Action Taken : MessageSchemaManager is performing startup/initialization requirements.
Remedy : None required.

Event Number : 3 (SSID 5512)
Severity : Informational
Text : MessageSchemaManager started.
Probable Cause: Standard startup message.
Action Taken : MessageSchemaManager has completed startup/initialization requirements.
Remedy : None required.

Event Number : 4 (SSID 5512)
Severity : Informational
Text : Loading message schema: {0}.
Probable Cause: Standard startup message.
Action Taken : MessageSchemaManager is loading the specified message schema into memory.
Remedy : None required.

Event Number : 5 (SSID 5512)
Severity : Critical
Text : Root element of schema document {0} does not equal {1}.
Probable Cause: Root element for specified file did not equal expected value.
Action Taken : No further processing is performed on the file.
Remedy : Verify/correct specified file.

Event Number : 7 (SSID 5512)
Severity : Critical
Text : No xml.msgschema.aci or xml.msgschema.csm entries found in config properties.
Probable Cause: Specified properties were not found in the APPLCNFG.
Action Taken : No further processing is performed.
Remedy : Update the APPLCNFG with the missing entries.

Event Number : 9 (SSID 5512)
Severity : Critical
Text : Error during retrieval of schema file location.
Probable Cause: An error occurred during the retrieval of a schema file.
Probably due to a missing file or incorrect home directory specification.
Action Taken : No further processing on the file is performed.
Remedy : Locate the missing file or correct configuration if necessary.

Event Number : 10 (SSID 5512)
Severity : Warning

Text : Message schema entry {0} does not exist in config directory or any config jars. Processing of this file is halted.
Probable Cause: The specified file could not be found for processing.
Action Taken : No further processing on the file is performed.
Remedy : Locate the missing file and correct configuration if necessary.

Event Number : 11 (SSID 5512)
Severity : Critical
Text : Data Library include not found in element schema file {0}.
Probable Cause: The specified schema file did not include any <xsd:include...> statements.
Action Taken : No further processing on the file is performed.
Remedy : Correct the specified schema file.

Event Number : 12 (SSID 5512)
Severity : Warning
Text : Error during processing of {0}: message schema for {1} was previously found and processed from {2}; this schema version will not be loaded.
Probable Cause: The specified schema file was already processed. Probably due to duplicate configuration.
Action Taken : No further processing on the file is performed.
Remedy : Locate and correct the duplicate schema file configuration.

Event Number : 13 (SSID 5512)
Severity : Informational
Text : Message schema {0} in {1} will override equivalent product schema.
Probable Cause: Standard startup message with CSM'ed schemas involved.
Action Taken : MessageSchemaManager is overriding a product schema with a CSM schema.
Remedy : None required.

Exception Log component

The following process control commands are available to the Exception Log component.

Purge Exception_Log records

The Purge Exception_Log Records command purges all records from the Exception Log File (Exception_Log) that have a date prior to a specified date. The Exception_Log contains messages that various Integrated Server (IS) process types could not interpret or process.

Process control syntax

Destination: XMLS or a symbolic process name
Command: Deliver
Component: Exception Log

Command text: PURGERECS *yyyymmdd*

where: *yyyymmdd* is the date prior to which records will be purged.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER ELOG PURGERECS *yyyymmdd*"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

yyyymmdd — The date prior to which records will be purged.

Heap Manager component

The Heap Manager component is bound into all BASE24-eps processes to provide extended memory management services. It allocates and deallocates heap memory—an area of memory reserved for data created at runtime—for each of the process' threads. It also has built-in monitoring functions to help diagnose memory leaks and memory corruption—these built-in monitoring functions work the same across all platforms and are activated using process control commands. Refer to "[Memory management and troubleshooting](#)" for more information about using Heap Manager to diagnose memory leaks and corruption.

Caution: The Heap Manager process control commands are for diagnostic purposes and intended for use in test environments only. Using them in a production system can having severe processing impacts.

Use of Status, Deliver, and Alter commands

Heap Manager process control commands are entered as deliver or alter commands and sometimes as status commands. There are some basic difference in their uses.

- Status commands go to a single process and to the first available thread within that process. Response information is returned to the Process Control window and may generate event messages as well.
- Deliver commands go to a single process and to the first available thread within that process. Response information is generated in the form of event messages only.
- Alter commands go to one or more processes based on the destination you specify, which could be a specific process or multiple processes represented by a service name; the command is received by all threads within those processes receiving the command. Response information is generated in the form of event messages only.

Note: On the IBM and UNIX platforms, this can differ depending on whether the command is sent to a command or message queue. If the command is sent to a destination that uses a command queue (such as PCTL or a symbolic name), the behavior will be as stated; however, if the command is sent to a destination that uses a message queue (such as an Integrated Service process), only one process and one thread will receive the message.

When diagnosing memory problems, it is recommended to focus on one single-threaded process at a time. In this case, either deliver or alter commands can be used because there is only process and one thread.

If you are diagnosing a process running multiple threads, you should use the alter command so that all threads receive the same commands to avoid having different threads in different states.

The following process control commands are available to the Heap Manager component.

Get basic memory status for a process thread

The STATUS command reports basic status information for the Heap Manager associated with a process thread. Basic status information includes:

Total number of bytes of heap memory that are currently allocated for the thread.

Whether or not monitoring is enabled for the Heap Manager (using the MONITOR ON and/or MONITOR STACK ON commands).

Whether or not verification is enabled for the Heap Manager (using the VERIFY ON, VERIFY INTERVAL, and/or VERIFY TXN commands).

Command Syntax

Destination:	Specify the symbolic name of the test process
Command:	Status
Component:	Heap Manager
Command text:	leave blank—no command text

Example Response Text:

HEAP: There are <n> bytes of memory currently allocated.
Heap is not monitored with stack on.
Heap is not verified after <n> allocations/deallocations.
Deletes are not verified.
Memory is not verified upon transaction completion.

where <n> is a numeric value.

Note: The response information is returned to the Response History portion of the Process Control window and generated in the following event message: SSID 1035 (Heap Manager), event 21.

Get detailed memory status for a process thread

The STATUS DETAIL command reports detailed status information for the Heap Manager associated with a process thread. Detailed status information includes:

- Basic status information. The same information reported by the STATUS command.
- The number of memory parcels allocated and available for the thread, by parcel size. The Heap Manager manages and tracks memory parcels (blocks of memory) based on byte size (i.e., 8, 16, 32, 64, 128, 256, 512, 1024, 2048 and 4096).

Command Syntax

Destination:	Specify the symbolic name of the test process
Command:	Status, Deliver, or Alter
Component:	Heap Manager
Command Text:	[DETAIL]

Response Text:

Basic Status Information:

HEAP: There are <n> bytes of memory currently allocated.
 Heap is not monitored with stack on.
 Heap is not verified after <n> allocations/deallocations.
 Deletes are not verified.
 Memory is not verified upon transaction completion.

Detail Status Information for each parcel size:

HEAP: Detail for memory parcels of size = <byte-size>,
 Number of parcels allocated = <n>,
 Number of parcels available = <n>

where <n> is a numeric value.

(Basic and detailed information is repeated for each thread if an Alter command is used ...)

Note: If entered as a status command, the response information is returned to the Response History portion of the Process Control window and generated in the following event messages: SSID 1035 (Heap Manager), event 21; SSID 1036 (Heap Supervisor), event 10. If entered as Deliver or Alter commands, the response information is generated in the event messages only.

Activate heap memory monitoring for a process thread

The MONITOR ON command activates memory monitoring of a thread by the Heap Manager. Refer to “[Memory monitoring](#)” for detailed information about memory monitoring.

Memory monitoring is deactivated using the MONITOR OFF command.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: MONITOR ON

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 80
HEAP: Monitoring heap allocations has been turned on.

Activate stack trace for a thread

The MONITOR STACK ON command activates a stack trace for use with memory monitoring. Refer to [“Memory monitoring”](#) for detailed information about memory monitoring.

Stack traces are deactivated using the MONITOR STACK OFF command.

Note: The MONITOR STACK ON/OFF commands are not required on a HP NonStop platform. When memory monitoring is enabled on a HP NonStop platform, the stack trace is automatically activated as well.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: MONITOR STACK ON

Response Event Messages:

SSID: 1035 (Heap Manager)
Event: 85
HEAP: The monitor stack flag is always ON for HP NonStop

or

SSID: 1035 (Heap Manager)
Event: 85
HEAP: The monitor stack flag is ON

Reset current memory baseline for a process thread

The RESET command sets the current memory parcel count levels tracked by the Heap Manager as a baseline. This baseline is used by the COMPARE command.

To use this command, memory monitoring must be activated (using the MONITOR ON command).

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: RESET

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 130
HEAP: Monitoring heap allocations has been reset.

Compare current memory usage to the process thread baseline

The COMPARE command compares a process thread's current memory usage to the thread's previously saved baseline.

With memory monitoring activated, the Heap Manager tracks the total memory parcels that have been allocated, but not released. Unreleased parcel counts will increase over time if there are memory leaks in a thread. The RESET command sets the current number of memory parcels as an initial baseline for the thread so that it can be subsequently used to detect increases in unreleased parcel counts. The COMPARE command then compares the current memory usage to the baseline and generates an event message to indicate whether the parcel counts have increased and if so, by how much. The COMPARE command also resets the baseline to the current counts to allow for another comparison later.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Status, Deliver, or Alter
Component: Heap Manager
Command Text: COMPARE

Response Event Messages:

HEAP: No memory differences to report.

or

```
Detail comparison for memory parcel size: 8
      Additional parcels used: 2
Detail comparison for memory parcel size: 16
      Additional parcels used: -1
Detail comparison for memory parcel size: 32
      Additional parcels used: 3
Detail comparison for memory parcel size: 64
      Additional parcels used: -1
Detail comparison for various sizes has changed from 1 to 0
```

Note: The “no differences” response is always returned to the Response History portion of the Process Control window only. The latter response messages depend on how the command is entered. If entered as a Status command, the response messages are returned to the Response History portion of the Process Control window and generated in the following event messages: SSID 1036 (Heap Supervisor), event 20. If entered as Deliver or Alter commands, all of the response information is generated in the event messages only.

Report memory allocated but not released for a thread

The MONITOR command reports the memory allocated and not released for the thread since monitoring was enabled or since the last time a new memory baseline was set (by the RESET or COMPARE command).

Note: A heap trace must be started prior to using this command, because the command causes all unreleased memory allocations to be written to the trace output location. Stack information will be written out as well if stack trace has been activated.

Command Syntax

Destination:	Specify the symbolic name of the test process
Command:	Status
Component:	Heap Manager
Command	MONITOR
Text:	

Response Text

Heap monitor information successfully sent to a trace file.

Deactivate stack trace for a thread

The MONITOR STACK OFF command deactivates a stack trace for use with memory monitoring. Refer to [“Memory monitoring”](#) for detailed information about memory monitoring.

Stack traces are activated using the MONITOR STACK ON command.

Note: The MONITOR STACK ON/OFF commands are not required on a HP NonStop platform. When memory monitoring is enabled on a HP NonStop platform, the stack trace is automatically activated.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: MONITOR STACK OFF

Response Event Messages:

SSID: 1035 (Heap Manager)
Event: 85
HEAP: The monitor stack flag is always ON for HP NonStop

or

SSID: 1035 (Heap Manager)
Event: 85
HEAP: The monitor stack flag is OFF

Deactivate heap memory monitoring

The MONITOR OFF command deactivates memory monitoring of a thread by the Heap Manager.

Memory monitoring is activated using the MONITOR ON command.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: MONITOR OFF

Response Event Messages:

SSID: 1035 (Heap Manager)
Event: 90
HEAP: Heap monitoring and verifying has been turned off.

Or, if verify is on

SSID: 1035 (Heap Manager)
Event: 70
HEAP: Monitoring heap allocations has been turned off.

Activate memory verification for a thread

The VERIFY ON command activates memory verification for a thread. Refer to [“Memory verification”](#) for detailed information about memory verification.

This command initiates special memory handling by the Heap Manager, but does not cause an actual memory verification to be performed. In order to initiate memory verification, one of the following commands must be entered to indicate when the verifications are to be performed: VERIFY, VERIFY DELETE, VERIFY INTERVAL, or VERIFY TXN.

Memory verification is deactivated by the VERIFY OFF command.

Note: Memory monitoring need not be activated for verification to function, but its monitoring provides additional information that can be helpful in diagnosing problems. Refer to [“Memory monitoring”](#) for detailed information about memory monitoring.

Command Syntax

Destination:	Specify the symbolic name of the test process
Command:	Deliver or Alter
Component:	Heap Manager
Command Text:	VERIFY ON

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 120
HEAP: Verifying heap allocations has been turned on, interval = 0

Set memory verification to occur for all deleted/released memory parcels

The VERIFY DELETE command activates memory verification for all memory parcels deleted/released by a thread. If memory is corrupted, the heap manager logs a message with as much information as possible and terminates the process. Only those memory parcel released are verified.

Memory verification must be activated (using the VERIFY ON command) before entering this command.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: VERIFY DELETE

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 100
HEAP: Verifying heap when deleting memory has been turned on.

Set memory allocation verification to occur at a specific interval

The VERIFY INTERVAL sets an interval to use for automatically initiating memory verification. The interval represents the number of memory actions (allocations and deallocations) to wait before verifying memory. For example if the interval is set to 100, all memory allocations are verified after every 100 memory actions (new allocations or deleted/released allocations).

An interval of 0 indicates that memory verification is not performed at all. An interval greater than 0 indicates that after the specified number of memory actions, all memory is verified. If a memory corruption is detected, it must have happened since the last time memory was verified, so the smaller the interval, the narrower the scope of code you need to look at for the memory corruption.

Memory verification must be activated (using the VERIFY ON command) before entering this command.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: VERIFY INTERVAL <n>
 <n> = the interval (values can be 0 to 4294967295).

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 150
HEAP: The verify memory allocation interval has been set to <n>

Set memory allocation verification to occur after each transaction

The VERIFY TXN command specifies that all memory allocations should be verified following the completion of a transaction.

If you do not know that you have a memory corruption, you can set verification to occur after each transaction completes so it occurs on a message basis versus number of times memory is allocated/deallocated.

Memory verification must be activated (using the VERIFY ON) command) before entering this command.

Command Syntax

Destination:	Specify the symbolic name of the test process
Command:	Deliver or Alter
Component:	Heap Manager
Command Text:	VERIFY TXN

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 190
HEAP: Verifying heap allocations has been enabled after each transaction completes.

Verify current memory allocations for a thread

The VERIFY command verifies all current memory allocations to ensure they have not been overwritten beyond their expected data lengths. This is a one-time verification and can be used if you want to initiate the verification yourself rather than setting up verification based on an interval (set by the VERIFY INTERVAL command) or for each transaction (set by the VERIFY TXN command).

If memory corruption is detected, the process terminates and memory address information is written to the home terminal for the process. If no memory corruption is detected, a “memory is good” event message is generated.

Memory verification must be activated (using the VERIFY ON command) before entering this command.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: VERIFY

Response Event Message:

SSID: 1035 (Heap Manager)
Event: 180
HEAP: Verify command received, all verified memory is good

Deactivate memory allocation verification

The VERIFY OFF command deactivates memory verification—deactivating the capability of Heap Manager to check memory for corruption.

The VERIFY ON command activates memory verification.

Command Syntax

Destination: Specify the symbolic name of the test process
Command: Deliver or Alter
Component: Heap Manager
Command Text: VERIFY OFF

Response Event Messages:

SSID: 1035 (Heap Manager)
Event: 110
HEAP: Verifying heap allocations has been turned off.

Or if monitor is off

SSID: 1035 (Heap Manager)
Event: 90
HEAP: Heap monitoring and verifying has been turned off.

Key Conversion component

The following process control commands are available to the Key Conversion component.

Convert AKB

The Convert AKB command converts all non-AKB formatted keys in the specified files into AKB format. This command requires the use of an NSP with AKB version software.

You can set optional Environment attributes that specify the AKB header values used when this command is entered. These attributes are described in section 16.

Process control syntax

Destination: XMLS or the symbolic name of the XML Server process
 Command: Deliver
 Component: Key Conversion
 Command text: CONVERT AKB [HSM *security-device-name*] *file-name*, *file-name*,... | *
 where:

Security-device-name is the name of an individual NSP. If you do not specify an individual NSP, the command is executed on all NSPs.

File-name can be one or more of the following file acronyms, with each acronym separated by a comma:

* (converts keys in all TSS files)

CRDVD (converts keys in the Card Verification File)

CSECD (converts keys in the Channel Security File)

DCRDVD (converts keys in the Dynamic Card Verification File)

DNTD (converts keys in the Diebold Number Table File)

DRVKD (converts keys in the DUKPT Derivation Key File)

EMVSD (converts keys in the EMV Security File)

IDESD (converts keys in the IBM DES Verification File)

ISECD (converts keys in the Interface Security File)

VPVVD (converts keys in the Visa PVV Verification File)

If you specify * as one of the entries in the *file-name* list, all other entries are ignored.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER KEYCONV CONVERT AKB [HSM *security-device-name*] *file-name*, *file-name*,... | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

security-device-name, *file-name* — See process control command text above.

Erase LMK or MFK

The Erase LMK or MFK command erases the old LMK key pair in a specified Thales e-Security HSM or all HSMs, or replaces the current MFK with the pending MFK in a specified Atalla NSP or all NSPs. Use this command after changing master file keys using the Master File Key Conversion utility.

Process control syntax

Destination: XMLS or the symbolic name of the XML Server process
Command: Deliver
Component: Key Conversion
Command text: ERASE *security-device-name* | *
where: *security-device-name* is the name of an individual NSP. If you enter an asterisk (*), the command is executed on all NSPs.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER KEYCONV ERASE *security-device-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

security-device-name — See process control command text above.

Validate Check Digits

The Validate Check Digits command validates the check digits stored in the TSS database for keys encrypted by Atalla NSPs (using variant or AKB software). You can use this command to validate check digits in a single file or all TSS files. Valid check digits are required for non-AKB formatted keys before they can be converted by the AKB Conversion utility.

Process Control Syntax

Destination: XMLS or the symbolic name of the XML Server process
Command: Deliver
Component: Key Conversion

Command text: VVK VALIDATE [HSM *security-device-name* | ALL | *] *file-name*, *file-name*,... | *

where:

security-device-name is the name of an individual NSP. If you enter ALL or an asterisk (*), the command is executed on all NSPs.

file-name can be one or more of the following file acronyms, with each acronym separated by a comma:

- * (validates keys in all TSS files)
- CRDVD (validates keys in the Card Verification File)
- CSECD (validates keys in the Channel Security File)
- DCRDVD (validates keys in the Dynamic Card Verification File)
- DNTD (validates keys in the Diebold Number Table File)
- DRVKD (validates keys in the DUKPT Derivation Key File)
- EMVSD (validates keys in the EMV Security File)
- IDESD (validates keys in the IBM DES Verification File)
- ISECD (validates keys in the Interface Security File)
- VPVVD (validates keys in the Visa PVV Verification File)

If you specify * as one of the entries in the *file-name* list, all other entries are ignored.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER KEYCONV VVK VALIDATE [HSM *security-device-name* | ALL | *] *file-name*, *file-name*,... | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

security-device-name — See process control command text above.

file-name — See process control command text above.

Generate check digits

The Generate Check Digits command generates check digits for keys stored in the TSS database that are encrypted by Atalla NSPs (using variant or AKB software). You can use this command to generate check digits in a single file or all TSS files. If check digits already exist for a key in a TSS file, they are overwritten with the check digits generated by this command. Valid check digits are required for non-AKB formatted keys before they can be converted by the AKB Conversion utility.

Process control syntax

Destination:	XMLS or the symbolic name of the XML Server process
Command:	Deliver

Component: Key Conversion

Command text: VVK GENERATE [HSM *security-device-name* | ALL | *] *file-name*,
file-name,... | *

where:

security-device-name is the name of an individual NSP. If you enter ALL or an asterisk (*), the command is executed on all NSPs.

file-name can be one or more of the following file acronyms, with each acronym separated by a comma:

- * (generates check digits for keys in all TSS files)
- CRDVD (generates check digits for keys in the Card Verification File)
- CSECD (generates check digits for keys in the Channel Security File)
- DCRDVD (generates check digits for keys in the Dynamic Card Verification File)
- DNTD (generates check digits for keys in the Diebold Number Table File)
- DRVKD (generates check digits for keys in the DUKPT Derivation Key File)
- EMVSD (generates check digits for keys in the EMV Security File)
- IDESD (generates check digits for keys in the IBM DES Verification File)
- ISECD (generates check digits for keys in the Interface Security File)
- VPVVD (generates check digits for keys in the Visa PVV Verification File)

If you specify * as one of the entries in the *file-name* list, all other entries are ignored.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER KEYCONV VVK GENERATE [HSM *security-device-name* | ALL | *] *file-name*, *file-name*,... | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

security-device-name — See process control command text above.

file-name — See process control command text above.

Timer component

The following process control commands are available to the Timer component.

Status timer statistics

The Status Timer Statistics command returns timer statistics information for named timers.

Process control syntax

Destination:	The assign name of a Transaction Security Services process defined in the MDSDEST
Command:	Status
Component:	Timer

Trace component

The Trace component is the interface to the process tracing capabilities of the system.

The following command can be sent to the Trace component from the Process Control window.

Status information on trace

The following information command returns status information on a trace. When you enter the command without detail, it returns information that the trace has or has not been started. When you enter the command with detail, it returns information about all the trace levels with the corresponding components which have been started.

Command syntax

Destination:	Any selection from the Destination field.
Command:	Status
Component:	Trace
Command text:	[DETAIL]

Transaction Security component

The following process control commands are available to the Transaction Security component.

Disable security device

The Disable Security Device command disables the specified security device or all security devices (denoted by *) by setting the status of the specified devices to inactive, thereby removing the devices from service. The Transaction Security component generates an event message, indicating that the device has been deactivated. This command also cancels the automatic reset timer if the Transaction Security component is currently attempting to enable the device at the intervals specified in the Reset Timer Limit field on the Security Configuration window. This prevents the Transaction Security component from continuing to attempt to re-establish communications with a device when it is to be taken out of service for an extended period of time.

Process control syntax

Destination: IS or a symbolic process name
 Command: Alter
 Component: Transaction Security
 Command text: `DISABLE security-device-name | *`
 where: *security-device-name* is the name of an individual security device. If you enter an asterisk (*), all security devices are disabled.

NCPCOM syntax

DELIVER PROCESS *object-name*, "ALTER TSEC DISABLE *security-device-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

security-device-name — See process control command text above.

Enable security device

The Enable Security Device command causes the Transaction Security component to send a status or echo test message to the specified security device or all security devices (denoted by *). If this message is successfully processed for a device, the component changes the status of the device from inactive to active and sets the consecutive error count to zero, thereby returning the device to service. If this message is not successfully processed for a device, the Transaction Security component generates an event message that indicates the error and specifies whether the status or echo test message will be automatically retried.

The Reset Timer Limit field on the Security Device Configuration window specifies the number of seconds the component waits before retrying the security device call. If the Reset Timer Limit field is set to 0, the component does not automatically attempt to retry the call.

Process control syntax

Destination: IS or a symbolic process name
Command: Alter
Component: Transaction Security
Command text: ENABLE *security-device-name* | *
where: *security-device-name* is the name of an individual security device. If you enter an asterisk (*), all security devices are enabled.

NCPCOM syntax

DELIVER PROCESS *object-name*, "ALTER TSEC ENABLE *security-device-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

security-device-name — See process control command text above.

Load DNT

The Load DNT command loads the Diebold number table stored in the Diebold Number Table File (DIEBOLD_NUM_TBL) into the memory of the specified security device or all security devices (denoted by *).

Process control syntax

Destination: IS or XMLS or a symbolic process name
Command: Deliver
Component: Transaction Security
Command text: DNTLOAD *security-device-name* | *
where: *security-device-name* is the name of an individual security device. If you enter an asterisk (*), the DNT is loaded in all security devices.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER TSEC DNTLOAD *security-device-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

security-device-name — See process control command text above.

Key generate

The Key Generate command generates an RSA key pair for the BASE24 system. This key pair is required for implementing automated key distribution or secure logons and logoffs to and from an interchange using public key cryptography.

Process control syntax

Destination: XMLS or symbolic name of the XML Server process
 Command: Deliver
 Component: Transaction Security
 Command text: See the “[Key Generate \(KEYGEN\) command](#)” topic in section 15.

NCPCOM syntax

DELIVER PROCESS *object-name*, “DELIVER TSEC KEYGEN *host-name*, *dev-typ*, [*common-name*], [*org-name*], [!], [*mod-size*], [*key-opt*] [,*eff-dat*]”

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

host-name, *dev-typ*, *common-name*, *org-name*, *mod-size*, *key-opt*, *eff-dat* — See the “[Key Generate \(KEYGEN\) command](#)” topic in section 15.

Initialize security device

The Initialize Security Device command initializes the specified security device or all security devices (denoted by *). Initializing a security device causes the TSS application to send the text strings configured in the Initialization String and Configuration String fields for the device on the Security Device Configuration window to the device. In addition, if the status of the device is Active, this command causes the Transaction Security component to send a status or echo test message to the security device. If this message is successfully processed, the component sets the consecutive error count to zero. If this message is not successfully processed for a device, the Transaction Security component generates an event message that indicates the error and specifies whether the status or echo test message will be automatically retried. The Reset Timer Limit field on the Security Device Configuration window specifies the number of seconds the component waits before retrying the security device call. If the Reset Timer Limit field is set to 0, the component does not automatically attempt to retry the call.

This command also causes the firmware version and maximum buffer length for the device to be displayed on the Security Device Configuration window. This information is not displayed until you initialize the device.

Process control syntax

Destination: IS or XMLS or symbolic process name
 Command: Deliver
 Component: Transaction Security
 Command text: INIT *security-device-name* | *
 where: *security-device-name* is the name of an individual security device. If you enter an asterisk (*), all security devices are initialized.

NCPCOM syntax

DELIVER PROCESS *object-name*, "DELIVER TSEC INIT *security-device-name* | *"

object-name — The symbolic name assigned to the User Interface (XML Server) process or a Transaction Security Services process in the XPNET node.

security-device-name — See process control command text above.

Refresh individual CSEC records

If you maintain Channel Security File (Channel_Security) records in memory (i.e., the CSEC_MEMORY_USE Environment attribute is set to Y), you can use the CSEC Refresh command to reread an individual channel security record from disk into memory, without deleting any other Channel_Security records from memory.

If you use the Alter Data Source command on the Channel_Security (CHANNEL_SECURITY assign) when Channel_Security records are maintained in memory, all records are deleted from internal memory and are read from disk back into memory as they are needed in processing. Using the CSEC Refresh command, you can avoid flushing all Channel_Security records from memory when only one channel security record has been modified. You do not need to use this command for a new Channel_Security record because it read from disk and added to memory on the first transaction received that requires the record. You can optionally use this command after deleting a Channel_Security record to remove it from memory.

Command syntax

Destination: IS or a symbolic process name
 Command: Alter
 Component: Transaction Security
 Command text: CSEC REFRESH *channel-id*
 where:
channel-id is the channel ID of the Channel_Security record to be warmbooted from disk.

Start statistics

The Start Statistics command resets the transaction counter to zero and starts monitoring the number of transactions processed by the specified Transaction Security Services process. The counter is increased each time the Transaction Security Services process receives a message. You can use this command to monitor transaction load balancing across the Transaction Security Services processes configured in your system.

Process control syntax

Destination:	IS or a symbolic process name
Command:	Alter or Deliver
	If you set this field to Alter and the Destination field is set to IS, the command is broadcast to all processes configured in the XPNET node with a service value of IS.
	If you set this field to Deliver, the command is sent to a single process.
Component:	Transaction Security
Command text:	STATISTICS ON

NCPCOM syntax

DELIVER PROCESS *object-name*, “[ALTER | DELIVER] TSEC STATISTICS ON”

object-name — The symbolic name assigned to a Transaction Security Services process in the XPNET node.

Status security device

The Status Security Device command indicates whether the specified security device or all security devices (denoted by *) are in service and reports the consecutive error count and the maximum error count for the specified device.

Process control syntax

Destination:	The assign name of a Transaction Security Services process defined in the MDSDEST
Command:	Status
Component:	Transaction Security
Command text:	<i>security-device-name</i> *
	where: <i>security-device-name</i> is the name of an individual security device. If you enter an asterisk (*), status information for all security devices is returned.

Status statistics

The Status Statistics command returns the current number of transactions processed by the specified Transaction Security Services process. The transaction counter is increased each time the Transaction Security Services process receives a message. You can use this command to monitor transaction load balancing across the Transaction Security Services processes configured in your system.

Process control syntax

Destination:	The assign name of a Transaction Security Services process defined in the MDSDEST.
Command:	Status
Component:	Transaction Security
Command text:	STATISTICS

Stop statistics

The Stop Statistics command stops monitoring the number of transactions processed by the specified Transaction Security Services process.

Process control syntax

Destination:	IS or a symbolic process name
Command:	Alter or Deliver If you set this field to Alter and the Destination field is set to IS, the command is broadcast to all processes configured in the XPNET node with a service value of IS. If you set this field to Deliver, the command is sent to a single process.
Component:	Transaction Security
Command text:	STATISTICS OFF

NCPCOM syntax

DELIVER PROCESS *object-name*, "[ALTER | DELIVER] TSEC STATISTICS OFF"

object-name — The symbolic name assigned to a Transaction Security Services process in the XPNET node.

User Interface Auditing component

The following process control commands are available to the User Interface Auditing component when the USE_LOCAL_USER_AUDIT Environment attribute is set to a value of N. If this attribute is set to a value of Y, the User Interface Auditing component is not used.

Disable audit store and forwarding

The Disable Audit Store and Forwarding command disables the sending of audit store-and-forward messages to the UAUD application running on the server machine.

Process control syntax

Destination: XMLS or symbolic name of the XML Server process
Command: Deliver
Component: User Interface Auditing
Command text: SAFSTOP

Enable audit store and forwarding

The Enable Audit Store and Forwarding command enables the sending of audit store-and-forward messages to the UAUD application running on the server machine.

Process control syntax

Destination: XMLS or symbolic name of the XML Server process
Command: Deliver
Component: User Interface Auditing
Command text: SAFSEND

Remove audit store and forward record

The Remove Audit Store and Forward Record command removes a specified record from the Audit Store and Forward File (Audit_Store_and_Forward). You can use this command to remove an invalid record.

Process control syntax

Destination: XMLS or symbolic name of the XML Server process
Command: Deliver
Component: User Interface Auditing

Command text: SAFPOP *timestamp session-number*
 where:
 timestamp is the timestamp of the record to be removed.
 session-number is the session number of the record to be removed.

VSP (Version/Service Pack) Manager component

The VSP (Version/Service Pack) Manager component is used to help track what version and service pack(s) a customer has installed (i.e., version level, service pack level for each component, patches for components, and CSMs for components).

A patch is an interim product correction made to address a critical or serious issue. Patches are not provided as standard support for non-critical issues. The product fix corresponding to the patch is also made to the development version.

Version/service pack

The Version/Service Pack command returns the version level and service pack level information, and, if applicable, a list of patches and CSMs, for all components bound into the executable for a specified destination process type.

Command syntax

Destination: Any selection from the Destination field
Command: Info
Component: VSP Manager
Command text: [*]
 Note: The asterisk can be any value. It is not validated.

VSP Command Response. The VSP command response is a list of component IDs and their version and service pack levels for all registered components installed in the executable of the specified destination process. The service pack designation of “.sp*n*” (where *n* is the number of the service pack) is omitted from the response if a service pack is not present for the component in the executable. The component IDs are listed in alphabetical order. After all the components are listed, any patches installed in the executable are listed, followed by any CSMs installed in the executable. Patches and CSMs are only listed if any are installed and are arranged alphabetically after the “PATCHES:” and/or “CSMs:” labels by Patch/CSM ID plus component ID. The version and service pack level is listed for each installed patch or CSM as shown below. If a service pack is not present for the patch or CSM, the “.sp*n*” designation is omitted from the response.

```
PATCHES: <patch-id1> <cmpnt-id>:08.2.sp1;<patch-id2> <cmpnt-id>:08.2.sp1
CSMs: <CSM-id1> <cmpnt-id>:08.2.sp1;<CSM-id2> <cmpnt-id>:08.2.sp1
```

For the System Interface Services (SIS) layer, the date, version, and release of SIS installed in the executable is listed for the “@(#)SIS:” label (e.g., @(#)SIS:22JUN2008_V01R05).

The version and service pack levels for central services (core) components are listed for the “CORE:” and “TDE:” labels. If a service pack is not present for the core component, the “.spn” designation is omitted from the response.

The following is an example VSP command response text for an 08.2 TSS process executable:

```
@(#)SIS:22JUN2008_V01R05  
CORE: 08.2  
TDE: 08.2
```

10: Process tracing

BASE24-eps provides an extensive means of tracing transaction processing for analysis and problem solving. Using the BASE24-eps trace facilities, you can trace messages through your system and provide information to ACI to assist in the remote diagnosis of problems when required.

Through the ACI desktop user interface, you can configure the types of information to be traced, execute traces, and filter the transactions to be included in the trace output. Information recorded during a trace is written to the Trace data source (Trace).

This section addresses process tracing for C++ processes only. Refer to the **ACI Java Infrastructure Java server reference guide** for information about tracing ACI JSF applications.

This section covers the following topics related to tracing:

- Defining traces
- Configuring traces
- Configuring trace filter profiles
- Operating traces
- Controlling traces
- Examining trace output

Note: The Trace facility was designed as a diagnostic tool for development and test environments. ACI strongly recommends that it be used in a controlled and limited manner in a production environment due to the overhead and impacts on performance. In addition, there are potential security risks for fraud when using the Trace facility in a production environment. Using the Trace facility, sensitive customer data (e.g., PANs, clear PIN offsets, encrypted PIN blocks, etc.) can be written to the trace output without any masking of data. You should only use the Trace facility in a production environment to troubleshoot a problem that cannot otherwise be reproduced in a test environment. Security best practices requires that you securely delete trace files immediately after use.

Defining traces

A trace provides various *snapshots* of a transaction as it is processed through different components of the BASE24-eps system. These snapshots can highlight different aspects of a transaction (e.g., TDEs, external messages, decision points, etc.) as configured for the trace.

These snapshots are recorded in a trace data source that can be examined later for analysis and diagnosis of transaction processing problems. You can configure the locations at which the snapshots are taken (i.e., the processes and components to be traced) and the type of information to be included in the snapshots (i.e., the trace levels to be included in the trace). Through the configuration of filters, you can control which transactions are included in the trace output (e.g., you could trace only transactions with a specified PAN, processing code, or institution ID, or other filter criteria).

Process tracing in a production environment

Traces in a production environment should be started with care and monitored closely. If transaction processing is impacted adversely (e.g., you see a rapid increase in transaction queues after starting the trace), you should stop the trace immediately.

The following options can help reduce performance impacts when tracing a production process.

Use a filter

Set up a trace filter to pinpoint the system component of concern. For example, if a particular terminal is causing an issue, you would set up a trace filter profile with the terminal's ID. Only messages from that terminal will be included in the trace output. If a particular endpoint or message source is in question, you could filter on the name of the acquirer endpoint (e.g., interface name) or symbolic message source (e.g., the component ID or symbolic name of the message source). If you configure multiple filters for a trace, a transaction must meet all the filter criteria to be included in the trace.

Partition the Trace data source

By partitioning the Trace data source, you can spread disk I/O operations across all the partitions so that each Integrated Server process and thread of execution (if applicable for your platform) writes to a different disk. In this case, you would partition the Trace data source so that each partition contains the trace data for the symbolic name or ID of a particular Integrated Server process and execution thread.

Trace a single process

When you enter the process control command to alter (warmboot) the Trace Configuration data source (Trace_Config), change the default destination of PCTL to a specific process symbolic name. Do not use one of the default process destination types either (e.g., IS, EOPP, etc.). By using a specific symbolic name, only a single process of the process executable type is traced rather than all processes of the process executable type, thereby reducing the number of I/Os to disk.

Trace only the levels and components needed

Turn on only the trace levels and components needed for the problem or issue in question. Tracing unneeded levels and components can cause extra disk I/Os that may adversely impact the performance of your system.

Do not include trace header data

Leave the Include Header in Trace checkbox unchecked for the trace. Using this option causes header data to be written to the trace, decreasing the performance of the trace.

Masking sensitive cardholder data in trace output



In a production environment, you should always mask sensitive cardholder data (i.e., PANs, account numbers, and PIN blocks) in process traces. Since the primary purpose of tracing is to debug processing issues, you should only allow unmasked sensitive cardholder data in a non-production testing environment.

The following Environment attributes allow you to unmask less than the default number of characters that can be displayed unmasked for primary account numbers (PANs), account numbers, and PIN blocks in generated trace output for all trace levels:

- `PCI_NUM_UNMASKED_LEFT`
- `PCI_NUM_UNMASKED_RIGHT`

For PANs, the default is to display the first six and last four digits unmasked. For account numbers, the default is to display the last four digits unmasked. For PIN blocks, the default is to display no characters unmasked. If a masked PAN, account number, or PIN block appears within Track data in the trace output, other customer data within the Track data, such as the expiration date or CVV, are not considered sensitive and are not masked. This is because the masking of the PAN, account number, or PIN block effectively renders such data to be useless to a fraudster.

The `PCI_NUM_UNMASKED_LEFT` attribute specifies the number of characters, starting from the left, of sensitive cardholder data that are not masked with the pound sign (#) character in any generated trace data. It defaults to 6.

The `PCI_NUM_UNMASKED_RIGHT` attribute specifies the number of characters, starting from the right, of sensitive cardholder data that are not masked with the pound sign (#) character in any generated trace data. It defaults to 4.

If both the `PCI_NUM_UNMASKED_LEFT` and `PCI_NUM_UNMASKED_RIGHT` attributes are set to 0, PANs, account numbers, and PIN blocks are completely unmasked in trace output. Thus, setting both the `PCI_NUM_UNMASKED_LEFT` and `PCI_NUM_UNMASKED_RIGHT` attributes to 0 effectively disables the masking of sensitive cardholder data in trace output.

For example, if you leave these attributes set to their default values, all but the first six characters and last four characters in PANs are masked (e.g., 123456#####3456) while all but the last four characters in account numbers are masked (e.g., #####7890). For PIN blocks, all characters would be masked (e.g., #####).

If you set the PCI_NUM_UNMASKED_LEFT attribute to 4 and the PCI_NUM_UNMASKED_RIGHT attribute to 2, all but the first four characters and last two characters in PANs would be masked (e.g., 1234#####56) while all but the last two characters in account numbers would be masked (e.g., #####90). PIN blocks would remain completely masked.

For detailed information on configuring the above Environment attributes, refer to the ***BASE24-eps environment management user guide***.

Configuring traces

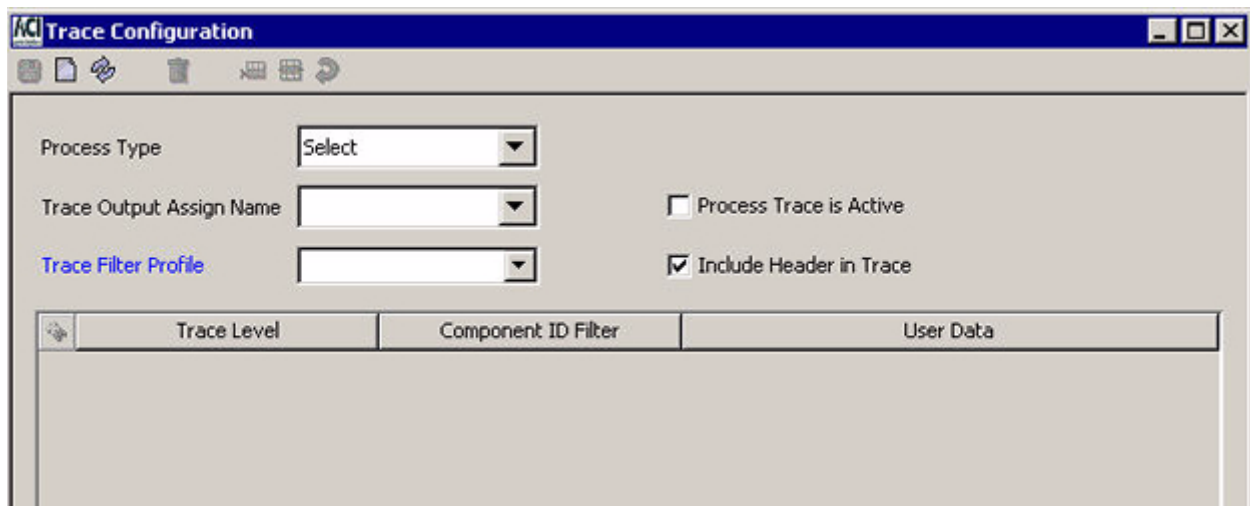
Use the Trace Configuration window on the ACI desktop user interface (accessed from **System Operations > Trace > Trace Configuration**), to configure a trace to be executed. On this window, you can configure the following:

- The type of process on which the trace is to be performed (e.g., IS or XMLS)
- The assign associated with the output data source for the trace
- The trace filter profile to be used for the trace
- Whether header data is included in the trace output
- The levels of tracing to be performed
- The component IDs of components to be traced
- Any optional user data to be used in the trace

You can specify multiple combinations of trace levels, components, and user data to be included in a single trace.

The Trace Configuration window maintains records stored in the Trace Configuration data source (Trace_Config). Because BASE24-eps processes only read the Trace_Config during startup, you must warmboot them to reread (alter) the Trace_Config after making any changes on the window before the changes take effect.

A sample of the Trace Configuration window is shown below, followed by descriptions of its fields.



Process Type

Trace commands should be sent to a single process of a specified process type. You select the process type in the Process Type field. The following are all valid process types for this field. The descriptive name of the destination is shown in parentheses.

- AACR (ACI Collision Resolution process)
- ATBM¹ (ATM Batch Manager process)
- BAUT (Batch Authorization process)
- CMCP (Channel Merchant Cutover process)
- CTXM² (Context Manager process)
- EOPE (End-of-Period process)
- IS (Integrated Server process)
- JQB (Journal Query Batch process)
- RFSH (Partial Refresh process)
- SAFM (SAF Manager process)
- TBP (Transaction Based Pricing process)
- TSS³ (Transaction Security Services process)
- TTLM (Totals Manager process)
- XMLS (XML Server process)

¹The ATM Batch Manager process is only used when C++ ATM Device Handlers are installed.

²The Context Manager process is only used when BASE24-eps is installed on an HP NonStop platform.

³The Transaction Security Services process is only used when ATM (Java) Device Handler or IFX ATM Manager processes are installed.

The process type identifies the type of executable to be traced.

Trace Output Assign

You specify the assign name of the Trace data source (Trace) for the trace in the Trace Output Assign Name field. All assign names currently configured in any Trace Configuration data source (Trace_Config) records are displayed in the selectable list for this field. You can select one of these assign names or enter a new assign name in this field. In either case, the assign name you select or enter must match the assign name configured for the Trace in the Metadata Configuration file (CONFCSV). You can configure multiple Trace assigns in the CONFCSV. The following is an example assign definition for a TRACE assign in the CONFCSV on an HP NonStop platform.

```
"TRACE", "Trace", "1.0", "$vol.subvol.trcd", "Enscribe", "", ""
```

Process Trace is Active

The Process Trace is Active checkbox lets you enable and disable the trace. You click in this field to either enable or disable the trace. A check mark indicates that the trace is currently enabled. You must warmboot the target processes to reread the Trace_Config before a change to this checkbox takes effect. When you enable a trace and warmboot the Trace_Config, the trace is started. When you disable a trace and warmboot the Trace_Config, the trace is stopped.

When you read a trace configuration record on the Trace Configuration window, a trace that is marked as enabled may or may not be running and a trace marked as disabled may or may not be stopped, depending on when the Trace_Config was last warmbooted and a process was started.

Trace Filter Profile

A trace filter profile is an optional feature that enables you to filter the transactions to be included in the trace output. In this field, you can assign a trace filter profile to the trace from the selectable list of all configured profiles. You can also use the hypertext link to navigate directly to the Trace Filter Profile Configuration window if you need to configure a new trace filter profile or you need to view or edit an existing trace filter profile.

Trace filters require matches on all criteria. If a transaction matches all of the criteria configured in the trace filter profile, it is included in the trace output. If a transaction does not match all of the criteria configured in the trace filter profile, it is not included in the trace output. If even one element of the filter criteria is not matched, the transaction is not traced.

Include Header in Trace

The Include Header in Trace checkbox indicates whether headers are added to each trace record written to the Trace. You click in this field to either enable or disable headers in the trace output. A check mark indicates that headers are enabled.

When trace headers are enabled, the following header information is added to each trace record written to the Trace.

```
LEVEL: trace-level COMPONENT: component-ID
```

where:

trace-level is one of the following:

DCSNP	= Decision point
EM	= External message
ENVMT	= Environment
HEAP	= Heap
IO	= Input/Output
JTRC	= Journal trace
OPRND	= Script operand
SCRLN	= Script line number
SCRPT	= Script
TDE	= TDE

component-ID is an identifier of the component that generated the trace data. Following are some examples of component IDs:

ENV	= Environment
HEAP	= Heap Manager
MD	= Message Delivery
TSEC	= Transaction Security

Trace levels

You can configure one or more trace levels in a single trace. Each trace level traces a different aspect of transaction processing as described below.

Decision Point

Decision point tracing traces the logic performed by any component that uses configured data to determine the processing path of a transaction. Information in the trace assists with troubleshooting the cause of failures as shown in the following example.

```
LEVEL: DCSNP
COMPONENT: NCRDH
FUNCTION: tde_rsn_cde_excpt_add
```

```

TDE set: bu_tde_rsn_cde_excpt
Reason code                      = 909
Exception Posted Flag            = 6
Object Id of client which set the ERC = 1543
Record Type                      = 1
LEVEL: DCSNP
COMPONENT: NCRDH
FUNCTION: extrn_auth_rqst
Action code = 909

```

Information in the trace also assists with identifying the flow of a transaction. For example, the following trace level traces the logic performed by the Prefix component for determining the issuer of a transaction and the logic performed by the Router component for determining the authorization destination for a transaction. You can use this trace level to validate, test, and debug the logic for your prefix and routing configurations.

Note: If a wildcard value in the routing configuration is used to determine the authorization destination, the actual value retrieved is displayed in the trace output.

```

LEVEL: DCSNP
COMPONENT: RTE
Route Code determined as: RT_CODE_2
Using Issuer Route Profile: HIS093_ISS_RTE
Acquirer Route Profile: ACQ_RTE_VISA

```



The following is an example of decision point trace output with a PIN block. The PIN block is completely masked.

```

LEVEL: DCSNP
COMPONENT: ATMFRAM
CDH: WDDCDH
data_elmt_get: #####

```

External Message

External message tracing enables you to trace all external request and response messages exchanged between a BASE24-eps process and all external entities or just specified external entities. External entities include security devices, processes, stations, services (HP NonStop platform only), or other BASE24-eps process types.

Each message included in the trace output by the Message Delivery component contains a header (component, process name, timestamp, length (and, when undeliverable, a message failure code), followed by a hexadecimal listing of the message contents. The first nine columns of the trace output is binary data in hexadecimal format. The first column contains a two-byte word counter for the message. Every new message traced starts with the counter set to 000. Counter values are in hexadecimal format (e.g., 010 = 016) and each row in the trace output contains 8-byte words (16 bytes). The last column of the trace output contains the character format of the message. The Message Delivery component always traces in both binary and character format. Trace data for all other components is provided in character format only.

In addition to the above, the trace data for ISO interface components includes the primary and secondary bitmaps, message type indicator, and data element values present for each message parsed or built by the interface.



The following are examples of interface external message trace output for PANs, account numbers, and other sensitive cardholder data.

The first example depicts a 16-digit PAN with all but the first six digits and last four digits masked.

```
LEVEL: EM
COMPONENT: VISAPGM
PAN Field 002 = 16 457545#####1234
```

The next example depicts a 10-digit account number with all but the last four digits masked.

```
LEVEL: EM
COMPONENT: VISAPGM
PCI ACCT Field 102 = 10 #####1234
```

The following example depicts the PAN masked in data element P-35, Track 2 Data. The expiration date and CVV are still unmasked.

```
LEVEL: EM
COMPONENT: VISAPGM
PCI Field 035 = 23 457545#####12341010123
```

Environment

Environment tracing lists the current value and default value for all the environment attributes in the destination process that have been registered. Attributes are written to the Trace in the following format:

```
ATTRIBUTE NAME = name
                VALUE = value
                DEFAULT VALUE = default-value
```

Heap

Heap tracing causes each Heap Supervisor to keep track of memory allocations and to write trace records when you enter a Status Monitor process control command. These trace records report the memory allocated and their contents to assist in analyzing memory management.

Input/Output

Input/Output tracing enables you to trace all database access by a BASE24-eps process, listing the key constraints, values, and field counts. You can trace all input/output tasks that are performed with a data source. Input/Output tracing applies to both physical files and external memory tables (OLTPs).

Journal Trace

Journal tracing enables you to execute a script to trace anything you want after the transaction has completed. The script can retrieve the value of any exported operand from any other data source or class that exports an operand. You can also use control statements to only gather the data desired. Thus, a journal trace can output whatever is appropriate for tracing. For example, if you initialize your authorization data sources with specific balances, you could ensure they were impacted properly by a transaction by executing a script to retrieve the balances. You could also use journal tracing to output specific TDEs to determine what their values are after a transaction is completed. These scripts could be used for regression test bed verification or to gather information if you are attempting to determine what happened in a transaction.

TDE

Transaction Data Element (TDE) tracing writes a record to the Trace with the TDE's contents each time a TDE is added, removed, or modified for a transaction. These records are formatted with one of the following lines preceding the TDE's contents:

```
TDE Added. tde-name
TDE Removed. tde-name
TDE Changed. tde-name
```

Script

Script tracing enables you to trace each authorization script (nested or otherwise) that is executed for a transaction. You can use any one of the following three levels available for script tracing: Script, Script Line Number, and Script Operand. Each of these script tracing levels is described below.

The Script trace level outputs the following information for a script: name of the script executed, execution time of the script in hhmmssmmm format, indication of when the script began and ended, and indication of whether the script was exited successfully, was aborted, or was aborted due to recursive execution.

The Script Line Number trace level outputs each line of the script traced in addition to the information output for the Script trace level described above. Each line is identified by a line number in the output.

The Script Operand trace level outputs the contents of each exported operator used in the script in addition to the information output for both the Script and Script Line Number trace levels described above. Exported operators are identified in the output by the name of the operator and the offset at which they were used.

In the above script tracing level hierarchy, the Script trace level provides the lowest level of tracing and the Script Operand trace level provided the highest level of tracing. If you start tracing a script with more than one of these levels specified, the highest level is used (because the information traced in a lower level is included in all higher levels).

Trace Level, Component ID Filter, and User Data

The Trace Level column of the trace configuration table on the Trace Configuration window defines the type of information to be included in the trace by the component specified in the Component ID Filter column of the same table row. The Component ID Filter column specifies the components that will generate trace data for the trace level specified in the Trace Level column of the same table row. If you specify {none} in the Component ID Filter column, all appropriate components in the transaction path will generate the specified trace data. You can configure up to 100 rows for any allowed combination of trace level, component, and user data to be included in the trace.

You must configure at least one trace level row before enabling a trace.

Only certain combinations of trace level, component ID, and user data are supported as illustrated in the following table.

Trace level	Component ID filter	User data
Decision Point	{ none} APACS40 ATMFRAM (ATM Framework) ATMDS (ATM Data Sources) ATMDBLD (ATM Diebold Device Handler) ATMISO (ATM (Java) Device Handler Interface) ATMNCR (ATM NCR Device Handler) ATMTIDL (ATM Tidel Device Handler) ATMTRIT (ATM Triton Device Handler) ATMWDDC (ATM Wincor DDC Device Handler) ATMWNDC (ATM Wincor NDC Device Handler) B24HISO (BASE24 HISO 8583 Host Interface) Card CNTNGNCY (Contingency Interface) EMVRTAU (EMV Router Auth) FILEUPDT (File Update) FPSM HPDH IFMGR (Interface Manager) IFXHI (IFX Host Interface) IMT (IMT Interface) INTFAEGN (American Express Global Network (AEGN) Interface) INTFAPMS (Application Parameter Management Interface) INTFBAUT (Faster Payments Batch Auth Interface) INTFBNET (BankNet ISO Interface) INTFCSC (CSC (Hogan) Host Interface) INTFEFND (eFunds Interface Base) <i>continued on the following page</i>	Not allowed.

Trace level	Component ID filter	User data
Decision Point <i>continued</i>	INTFEPSN (EPS-Net Interface) INTFHI93 (ISO 93 Host Interface) INTFI93 (ISO 93 Interface Base) INTFIPAY (Interpay BIM Interface) INTFJCB (JCB Interface) INTFLIS5 (LINK ISO Interface) INTFMDS (MDS Interface) INTFNETS (NetWorks (NETS) Interface) INTFNYCE (NYCE Interface) INTFPRST (Presto Interface) INTFPULS (Pulse Interface) INTRFB53 (Fifth Third Interface Base) INTFRTF (Real Time Feed Interface) INTFSHZM (Shazam Interface) INTFSPN2 (SAMA Saudi Payments Network (SPAN2)) INTFSTAR (STAR ISO Interface) INTFTDE (Faster Payments TDE Stream Interface) INTFVDPS (Visa DPS Interface) INTFVISA (VisaNet ISO Interface) ISOBAS (ISO Interface Base) ISOIFI (ISO Interface Issuer Base) ISSBASE (Issuer Base) MCHAN (Merchant Channel Base) Prefix Route SAFEXTR (SAF Extract) SAFMGR (SAF Manager) SPDH UPDTINTF (File Update Interface)	Not allowed.
External Message	{ none } Message Delivery ATMDBLD (ATM Diebold Device Handler) ATMISO (ATM (Java) Device Handler Interface) ATMNCR (ATM NCR Device Handler) ATMTIDL (ATM Tidel Device Handler) ATMTRIT (ATM Triton Device Handler) <i>continued on the following page</i>	Optional for the Message Delivery component. Not allowed for all other components.

Trace level	Component ID filter	User data
External Message <i>continued</i>	ATMWDDC (ATM Wincor DDC Device Handler) ATMWNDC (ATM Wincor NDC Device Handler) B24HISO (BASE24 HISO 8583 Host Interface) CNTNGNCY (Contingency Interface) INTFAEGN (American Express Global Network (AEGN) Interface) IFXHI (IFX Host Interface) INTFAPMS (Application Parameter Management Interface) INTFBAUT (Faster Payments Batch Auth Interface) INTFBNET (BankNet ISO Interface) INTFCSC (CSC (Hogan) Host Interface) INTFEPSN (EPS-Net Interface) INTFHI93 (ISO 93 Host Interface) INTFIPAY (Interpay BIM Interface) INTFJCB (JCB Interface) INTFLIS5 (LINK ISO Interface) INTFMDS (MDS Interface) INTFNETS (NetWorks (NETS) Interface) INTFNYCE (NYCE Interface) INTFPRST (Presto Interface) INTFPULS (Pulse Interface) INTFRTF (Real Time Feed Interface) INTFSHZM (Shazam Interface) INTFSPN2 (SAMA Saudi Payments Network (SPAN2)) INTFSTAR (STAR ISO Interface) INTFTDE (Faster Payments TDE Stream Interface) INTFVDPS (Visa DPS Interface) INTFVISA (VisaNet ISO Interface) TSEC (Transaction Security) ¹	Optional for the Message Delivery component. Not allowed for all other components.
Environment	{ none } Environment	Not allowed.
Heap	{ none } Heap	Not allowed.
I/O	{ none } Data Source	Not allowed.
Journal Trace	{ none } ATM Framework Journal Trace	Required.

Trace level	Component ID filter	User data
TDE	{ none} TDE	Not allowed.
Script	{ none} Script	Not allowed.
Script Line Number	{ none} Script	Not allowed.
Script Operand	{ none} Script	Not allowed.

¹The Transaction Security component only supports filtering for the Thales SRM and Banksys DEP HSMs. The TSEC value is not currently listed in the drop-down values for the Component ID Filter column on the Trace Configuration window.

External message user data

For external message tracing, you can optionally specify entities external to the process type for which external messages are traced. The external entity can be the assign name of a security device, the symbolic name of a process, station, or service (HP NonStop platform only), or another process type.

- If you specify a security device, the assign name must match the assign name configured for a security device in the Message Delivery Service.
- If you specify a process, station, or service, the symbolic name must match the symbolic name configured to the middleware for your system.

If you do not specify an external entity to be traced, external messages for all external entities (e.g., all security devices, processes, and stations) are traced.

Journal trace user data

For journal tracing, you must specify the name of a script to be executed for the journal trace. The script determines what data is generated and output for the trace for completed transactions.

Configuring trace filter profiles

Use the Trace Filter Profile Configuration window on the ACI desktop user interface (accessed from **System Operations > Trace > Trace Filter Profile**), to configure a trace filter profile. The information entered on this window is stored in the Trace Filter Configuration

data source (Trace_Filter). A trace filter profile is an optional feature that enables you to filter the transactions to be included in the trace output based on one or more of the following filter types:

- Action code
- Acquiring endpoint name
- Acquiring institution
- Issuing endpoint name
- Issuing institution
- Primary account number
- Partial primary account number
- Processing code
- Symbolic message destination
- Symbolic message source
- Terminal ID

You can assign one or more values to one or more of these filters in the table displayed on this window. An example of the Trace Filter Profile Configuration window is shown below, followed by a discussion of its fields.

Filter Type	Filter Value

Trace Filter Profile

You enter or select the name of the trace filter profile in the Trace Filter Profile field. You can select the name from the selectable list if the profile is already configured.

Filter Type

The Filter Type column of the table specifies the type of information on which transaction data is to be filtered for the trace. You can select one of the following values from the selectable list for this column in the table:

Acquiring End Point Name	= The name of the acquirer endpoint.
Acquiring Institution	= The ID of the acquirer institution.
Action Code	= A one- to three-digit code indicating the outcome of authorization.
Issuing End Point Name	= The name of the issuer endpoint.
Issuing Institution	= The ID of the issuer institution.
Symbolic Message Destination	= The symbolic name of the destination for the transaction message.
Symbolic Message Source	= The symbolic name of the source for the transaction message.
Primary Account Number	= The primary account number (PAN) used in the transaction.
Partial Primary Account Number	= The first n digits of the PAN used in the transaction, where n is a variable number of configured digits for the filter.
Processing Code	= The processing code for the transaction.
Terminal ID	= The terminal ID associated with the transaction.

If you want to filter on multiple values for one of these filter types, you must enter a separate table row for each value assigned to the filter type. To be included in the trace output, a transaction must match all rows in the filter table.

Filter Value

In the Filter Value column of the table, you enter values associated with the filter type on the same row of the table.

Trace operations

Whenever you make a change to the Trace_Config from the Trace Configuration window or the Trace_Filter from the Trace Filter Profile Configuration window, the change does not take effect in processing until after you have saved the change and either warmbooted (altered) BASE24-eps processes to reread the Trace_Config and Trace_Filter or stopped and restarted the BASE24-eps processes. BASE24-eps processes read the Trace_Config and associated Trace_Filter records during initialization processing.

You alter a process to reread the Trace_Config (and associated Trace_Filter records) by sending an alter data source process control command to the process. The syntax for this command is as follows:

Command Syntax

Destination: PCTL
 Command: Alter
 Component: Data Source
 Command Text: TRACE_CONFIG

For convenience, you can right-click anywhere on the Trace Configuration window or Trace Filter Profile Configuration window and click the “Alter Trace Configuration Data Source” action. When you do so, the Process Control window is displayed with all the fields preset to the values you need to warmboot the appropriate processes to reread Trace_Config (and associated Trace_Filter records). At that point, you need only click Submit () to warmboot the trace configuration.

Controlling traces

Using the ACI desktop, you can start, modify, stop, and status traces.

Starting a trace

Once you have configured a trace, you can execute it by placing a check mark in the Process Trace is Active checkbox on the Trace Configuration window, clicking Save (), and then warmbooting the appropriate processes to reread the Trace Configuration data source (Trace_Config). For convenience, you can right-click anywhere on the Trace Configuration window and click the “Alter Trace Configuration Data Source” action. When you do so, the Process Control window is displayed with all the fields preset to the values you need to warmboot the appropriate processes to reread the Trace_Config. At that point, you need only click Submit () to start the trace or wait for the process that is to be traced to be started.

Modifying a running trace

While a trace is active, you can modify the trace levels, component ID filters, user data for the trace by editing the trace level, component ID filter, and user data table on the Trace Configuration window, or the trace filter profile in the Trace Filter Profile field by editing the trace filter table on the Trace Filter Profile Configuration window. After you have made the desired changes, click Save (), and then warmboot the appropriate processes to reread the Trace Configuration data source (Trace_Config). For convenience, you can right-click anywhere on the Trace Configuration window or the Trace Filter Profile Configuration window

and click the “Alter Trace Configuration Data Source” action. When you do so, the Process Control window is displayed with all the fields preset to the values you need to warmboot the appropriate processes to reread the Trace_Config. At that point, you need only click Submit (▶) for the modified trace levels, component ID filters, user data, and trace filter profile to start being used for the trace or wait for the process that is to be traced to be started.

Stopping a trace

When you are done running the trace, remove the check mark from the Process Trace is Active checkbox on the Trace Configuration window, click Save (💾), and then warmboot the appropriate processes to reread the Trace Configuration data source (Trace_Config). For convenience, you can right-click anywhere on the Trace Configuration window and click the “Alter Trace Configuration Data Source” action. When you do so, the Process Control window is displayed with all the fields preset to the values you need to warmboot the appropriate processes to reread the Trace_Config. At that point, you need only click Submit (▶) to stop the trace or the trace will be disabled the next time the process that is to be traced is started.

Obtaining status information for a trace

You can inquire on the status of a trace by entering a Status command on the Trace component from the Process Control window or from a network control facility. For detailed information on this command, see the [“Status Information on Trace”](#) topic in section 3.

Examining trace output

All trace output is written to the Trace File (Trace). The Trace is an entry-sequenced file with the following primary keys:

- Date and time
- Symbolic process ID/name
- Symbolic thread ID
- Record Number

The date and time of the transaction is provided in YYYYMMDDhhmmsslllccc format.

The symbolic process ID is the symbolic process ID as defined by SIS, which may represent a PPD name, task, transaction type, or whatever uniquely identifies a specific process on a given platform. The symbolic process name is the symbolic name of the process.

The symbolic thread ID identifies the particular thread of execution traced in the process. For the HP NonStop platform, this field is always set to a value of 1.

The record number uniquely identifies multiple rows in the Trace when the trace data generated by the process exceeds the trace buffer limit of 3988 bytes and is written to multiple Trace rows.

You can view trace data written to the Trace using the ACI-provided trace viewer utility.

Note: Consider the amount of time you have when you are using the Trace Viewer utility to read extremely large trace records. These can take considerable time to write to the system.

Instructions for using the Trace Viewer utility as well as a sample of external message trace output are provided below.

Viewing trace data using the trace viewer utility

The ACI-supplied Trace Viewer utility program (TRCVLO executable) displays Trace data in a user-friendly format and is the preferred method for viewing trace data. You can execute this utility using the appropriate run command for your platform.

You can use the RUNTRCV obey file provided at installation or you can create TACL macros (see the example TRCVR and TRCINI TACL macros below) to execute the run command.

Because the Trace Viewer utility uses some of the same configuration design as the DAL Conversational Interface (DALCI), you must include the `-mdbv`, `-ai`, and `-strt` DALCI startup options when running this utility. The utility displays all the data in the data source defined for the TRACE assign in the CONFCSV or in the data source defined for the assign specified in the optional TRCIN runtime option. If the optional TRCIN parameter is not specified at runtime, the utility uses a value of TRACE for the input assign. Therefore, you must enter a value of TRACE or the value of the TRCIN parameter in the Trace Output Assign Name field on the Trace Configuration window if you want to view the trace output with this utility.

The output from the Trace Viewer utility is written to the data source location specified in the MTRCRC assign in the CONFCSV or to the data source defined for the assign specified in the optional TRCOUT runtime option. If the optional TRCOUT parameter is not specified at runtime, the utility uses a value of MTRCRC for the output assign.

Before running the utility, purge any existing data from the existing trace viewer output file specified by the MTRCRC assign or by an assign specified in the TRCOUT runtime parameter. If data already exists in the trace viewer output file when the Trace Viewer utility is run, the data from the new trace is appended to the end of the file. Because no timestamps or dates are associated with external messages written to the trace output file, it may be difficult to determine where one trace stops and another trace starts if the file contains data from more than one trace.

Note: If you specify assigns in the optional TRCIN or TRCOUT runtime parameters, the assigns must be configured in the CONFCSV.

Examples of the run command for the Trace Viewer utility are shown below.

This example uses the default TRACE and MTRCRC assign names.


```
run $vol.subvol.trcvlo/name, &
lib $vol.xnlib.xnlibn/ &
-MDBV=CSV &
-AI=ES &
-STRT=$vol.subvol (location of the CONFCSV file)
```

This example uses the assign names specified in the optional TRCIN and TRCOUT runtime parameters.

```
run $vol.subvol.trcvlo/name, &
lib $vol.xnlib.xnlibn/ &
-MDBV=CSV &
-AI=ES &
-STRT=$vol.subvol (location of the CONFCSV file) &
-TRCIN=TRACE2 &
-TRCOUT=TRC
```

RUNTRCV obey file

ACI provides a default RUNTRCV obey file at installation to run the Trace Viewer utility that you can modify as desired for your site. An example of this obey file is shown below:

```
==
== Obey file to run Trace Viewer.
== Trace output sent to MTRCRC assign.
==
RUN $DATA.TS740BJ.TRCVLO/NAME, LIB $SYSACI.XNLIB.XNLIBN/-MDBV=CSV -AI=ES -
STRT=$CONFIG.TS74CNFG
```

You execute the RUNTRCV obey file by entering the following command at the TACL prompt in the subvolume where the file resides:

```
> OBEY RUNTRCV
```

TRCVR and TRCINI TACL macros

If desired, you can create TRCVR and TRCINI TACL macros to run the Trace Viewer utility. The following is an example TRCVR TACL macro that calls a TRCINI TACL macro.

```
?TACL MACRO

#FRAME

%1%.trcini

==
== exec_it is a macro in the drivini file that will echo out the
== command before executing it.
==
```

```
exec_it run [:exe_loc].trcvlo / lib [:lib] / &
[:startup_options]
```

The following is an example TRCINI TACL macro:

```
?TACL MACRO
```

```
[#DEF exec_it ROUTINE
|BODY|
#OUTPUT [#REST]
[#REST]
] == end def
```

```
#PUSH :exe_loc
#PUSH :lib
#PUSH :startup_options
#PUSH :cmp_loc
#PUSH :data_loc
#PUSH :jrnل_loc
```

```
#SET :exe_loc      $MOAT.FS74exe
#SET :lib          $sysaci.xnlib.xnlibn
#SET :startup_options -mdbv=CSV -ai=ES -strt=$config.r054data
#SET :cmp_loc      $esim.esmacs
#SET :data_loc     $MOAT.FS74data
#SET :jrnل_loc     $MOAT.FS74jrnل
```

If you create these macros and want to use assigns other than the default TRACE and MTRCRC assign names, edit the :startup_options variable in the TRCINI file and set the -TRCIN and -TRCOUT parameters to the desired assign names, as depicted in the next example:

```
?TACL MACRO
```

```
[#DEF exec_it ROUTINE
|BODY|
#OUTPUT [#REST]
[#REST]
] == end def
```

```
#PUSH :exe_loc
#PUSH :lib
#PUSH :startup_options
#PUSH :cmp_loc
#PUSH :data_loc
#PUSH :jrnل_loc
```

```
#SET :exe_loc      $MOAT.FS74exe
#SET :lib          $sysaci.xnlib.xnlibn
#SET :startup_options -mdbv=CSV -ai=ES -strt=$config.cs74data -TRCIN=TRACE2
-TRCOUT=TRC
#SET :cmp_loc      $esim.esmacs
#SET :data_loc     $MOAT.FS74data
#SET :jrnل_loc     $MOAT.FS74jrnل
```

You can execute the TRCVR and TRCINI TACL macros by entering the following command at the TACL prompt in the subvolume where the macros reside:

```
> TRCVR
```

An example of the Trace Viewer utility output for an external message trace is provided below under the [“Example external message trace output”](#) topic. For detailed information on DALCI startup options, refer to the ***DAL Conversational Interface (DALCI) User Guide***.

Example external message trace output

A sample of the trace output for tracing external messages for a specified simulator process and a security device is shown below. For this trace, the trace level is set to EM (external message), the component ID is MD for the Message Delivery component, and the *user-data* specifies two external entities (p3c^sim06 and atalla1). The external messages sent to and received from a security device are highlighted in .

Each message included in the trace output is separated by a blank line and specifies the destination, source, and text of the message. The first nine columns of the trace output is binary data in hexadecimal format. The first column contains a two-byte word counter for the message. Every new message traced starts with the counter set to 000. Counter values are in hexadecimal format (e.g., 010 = 016) and each row in the trace output contains 8-byte words (16 bytes). The last column of the trace output contains the character format of the message.

This sample trace includes seven messages. The first message is a key generate request for a channel device sent from a simulator process to the Transaction Security component in the Transaction Security Services process. The second message is a key generate command for a PIN encryption key sent from the Transaction Security component in the Transaction Security Services process to a security device. The third message is the response returned from the security device to the Transaction Security Services process. The fourth message is change transaction key command sent from the Transaction Security component in the Transaction Security Services process to a security device. This command encrypts the PIN encryption key generated in the previous message for downloading to a channel device. The fifth message is the response returned from the security device to the Transaction Security Services process. The sixth message is the response message sent from the Transaction Security component in the Transaction Security Services process back to the simulator process. The last message is a command sent from a network control facility to the Transaction Security Services process to stop the trace.

```
000:  4465    7374    696E    6174    696F    6E20    3D20    2020  Destination =
008:  5453    4543    2020    2020    2020    2020    2020    2020  TSEC
010:  536F    7572    6365    203D    2020    2020    2020    2020  Source =
018:  5033    435E    5349    4D30    3620    2020    2020    2020  P3C^SIM06
020:  5465    7874    203D    2020    2020    2020    2020    2020  Text =
028:  3034    3031    2043    4841    4E4E    454C    5F52    4143  0401 CHANNEL_RAC
030:  414C    2020    2032    3132    3334    3536    3738    3930  AL  21234567890
038:  3132    3334    3536    3020                                     1234560

000:  4465    7374    696E    6174    696F    6E20    3D20    2020  Destination =
```

008:	4154	414C	4C41	3120	2020	2020	2020	2020	ATALLA1
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	5453	4543	2020	2020	2020	2020	2020	2020	TSEC
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	3C31	3023	3123	3030	3030	3030	3030	3030	<10#1#00000000000
030:	3030	3030	3030	2353	233E				000000#S#>
000:	4465	7374	696E	6174	696F	6E20	3D20	2020	Destination =
008:	5453	4543	2020	2020	2020	2020	2020	2020	TSEC
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	4154	414C	4C41	3120	2020	2020	2020	2020	ATALLA1
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	3C32	3023	3738	4246	3633	4245	3934	3236	<20#78BF63BE9426
030:	4242	3836	2330	4634	4542	4437	3445	3346	BB86#0F4EBD74E3F
038:	3235	4441	3123	4442	4634	233E			25DA1#DBF4#>
000:	4465	7374	696E	6174	696F	6E20	3D20	2020	Destination =
008:	4154	414C	4C41	3120	2020	2020	2020	2020	ATALLA1
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	5453	4543	2020	2020	2020	2020	2020	2020	TSEC
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	3C31	3523	3223	3738	4246	3633	4245	3934	<15#2#78BF63BE94
030:	3236	4242	3836	2335	3342	4531	3342	4341	26BB86#53BE13BCA
038:	3736	3044	3936	3223	3E				760D962#>
000:	4465	7374	696E	6174	696F	6E20	3D20	2020	Destination =
008:	5453	4543	2020	2020	2020	2020	2020	2020	TSEC
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	4154	414C	4C41	3120	2020	2020	2020	2020	ATALLA1
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	3C32	3523	3223	3139	3644	3532	3336	4444	<25#2#196D5236DD
030:	3245	3539	3631	2332	3445	3632	3638	3332	2E5961#24E626832
038:	3636	3341	3537	3723	3E				663A577#>
000:	4465	7374	696E	6174	696F	6E20	3D20	2020	Destination =
008:	5033	435E	5349	4D30	3620	2020	2020	2020	P3C^SIM06
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	5453	4543	2020	2020	2020	2020	2020	2020	TSEC
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	3034	3030	3031	2044	4246	3420	2031	2020	040001 DBF4 1
030:	2020	2020	2020	3139	3644	3532	3336	4444	196D5236DD
038:	3245	3539	3631	2020	2020	2020	2020	2020	2E5961
040:	2020	2020	2020	2020	2020	2020	2020	2020	
048:	2020	2020	2020	2020	2020	2020	2020	2020	
050:	2020	2020	2020	4348	414E	4E45	4C5F	5241	CHANNEL_RA
058:	4341	4C20	2020	3234	4536	3236	3833	3236	CAL 24E6268326
060:	3633	4135	3737						63A577
000:	4465	7374	696E	6174	696F	6E20	3D20	2020	Destination =
008:	5033	435E	4941	5330	3120	2020	2020	2020	P3C^IAS01
010:	536F	7572	6365	203D	2020	2020	2020	2020	Source =
018:	234E	4554	434E	544C	2020	2020	2020	2020	#NETCNTL
020:	5465	7874	203D	2020	2020	2020	2020	2020	Text =
028:	6465	6C69	7665	7220	7472	6320	7374	6F70	deliver trc stop

Security device tracing procedure

To execute a trace of security device interaction, perform the following steps:

1. Before starting a trace, ensure that the trace input file (i.e., the file specified by the TRACE assign in the CONFCSV) does not contain any data. If you do not purge data on the file before starting a trace, then the data for the trace is appended to the end of the file. Thus, you should always issue the following FUP PURGEDATA commands on the TRACE files before starting a trace, taking into account the actual physical TRACE file locations on your system:

```
fup purgedata $vol.tssdata.trcd
fup purgedata $vol.tssdata.trcd0
```

In addition, these files must pre-exist (and be sized) prior to running the trace. Otherwise, the trace will not work. In this case, a message indicating that the trace is not active (e.g., "No trace is in progress") is displayed on the event log.

2. Configure the trace to be performed. On the ACI desktop, access **System Operations > Trace > Trace Configuration**.

Enter values in the following fields as indicated below.

Field	Value
Process Type	TSS
Trace output Assign Name	TRACE
Process Trace is Active	Check the box
Include Header in Trace	Check the box (if the performance costs of doing so are not an issue when tracing in a production system)

Insert a row in the table with the following values:

Trace Level	Component ID filter
External Message	Message Delivery

Click Save () to save the trace configuration.

3. Warmboot (alter) the trace configuration. On the ACI desktop, access **System Operations > Process Control**.

Enter values in the following fields as indicated below.

Field	Value
Destination	IS
Command	Alter

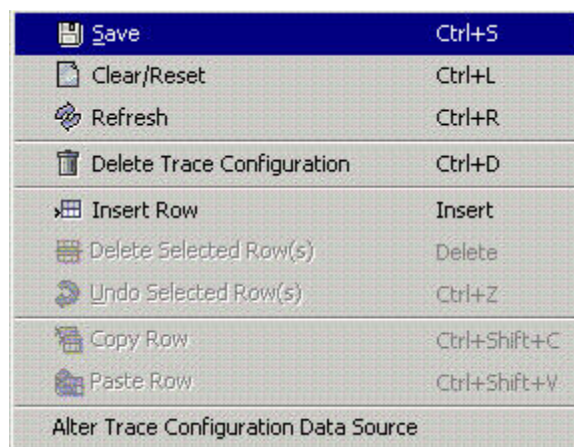
Field	Value
Component	Data Source
Command Text	TRACE_CONFIG

Click Submit (▶). The following should be displayed in EMS:

```
TRC: Tracing enabled with no trace filters configured.
DASRC: Successful warmboot of data source: TRACE_CONFIG
```

At this point, the trace is activated.

(Alternatively, while still displaying the Trace Configuration window, right click on the mouse and the following dialog box is displayed.



Click the Alter Trace Configuration Data Source option. The Process Control window is displayed with all the values populated to warmboot PCTL (which warmboots all processes registered to PCTL: AKDS, IS, and XMLS) to re-read the trace configuration data. Click Submit (▶) and check EMS as described above.)

- Enter the transactions that you want to trace.
- Stop the trace. On the ACI desktop, access **System Operations > Trace > Trace Configuration**. Remove the check mark from the Process Trace is Active field and click Save (💾) to save the trace configuration.
- Warmboot (alter) the trace configuration. On the ACI desktop, access **System Operations > Process Control**.

Enter values in the following fields as indicated below.

Field	Value
Destination	IS
Command	Alter
Component	Data Source
Command Text	TRACE_CONFIG

Click Submit (). The following should be displayed in EMS:

```
TRC: No trace is in progress.
DASRC: Successful warmboot of data source: TRACE_CONFIG
```

At this point, the trace is stopped and the trace output files are closed. You can verify that the trace output files are closed by entering the following TACL FILEINFO command in the data subvolume:

```
>fileinfo tssdata.trc*
```

An "O" is displayed in character position 11 of the command output display if the trace file is currently open. In the following example output display, only the Trace Configuration and Trace Filter files are open. In addition, you can see that the trace output files (TRCD and TRCD0) contain data.

	CODE	EOF	LAST MODIFIED	OWNER	RWEP	PExt	SExt
TRC2D	0	0	29MAR2006 13:09	110,255	NUUU	64	64
TRC2D0	0	0	29MAR2006 13:09	110,255	NUUU	8	8
TRCCFD	0	0A	12288 03APR2006 9:47	110,255	NUUU	64	64
TRCD	0	12288	03APR2006 9:38	110,255	N000	64	64
TRCD0	0	12288	03APR2006 9:38	110,255	NUUU	8	8
TRCFLD	0	0A	0 29MAR2006 13:09	110,255	NUUU	64	64

At this point, you can either leave the trace output data in the trace output file designated by the TRACE assign or you can transfer the data to another file specified by another assign in the CONFCSV to be specified in the TRCIN runtime parameter for the Trace Viewer utility. For example:

```
>fup dup tssdata.trcd,tssdata.trc2d
>fup dup tssdata.trcd0,tssdata.trc2d0
```

7. Run the Trace Viewer utility to format the trace output into a viewable format. Input to the utility is either the file specified by the TRACE assign or by the assign specified in the TRCIN runtime parameter. The viewable output is placed in either the file specified by the MTRCRC assign or in the file specified by the assign in the TRCOUT runtime parameter. See the ["Viewing trace data using the trace viewer utility"](#) topic above for detailed information on running this utility.

```
> trcvr
```

8. Analyze the viewable trace output or attach it to a Clarify case as instructed by a HELP24 analyst.

11: OLTP management

This section describes how to use the OLTP Warmboot Management window to rebuild and warmboot external memory tables (OLTPs). This window provides a quick and more efficient alternative to manually entering Deliver Data Source Load (OLTP Rebuild) and Alter Data Source (OLTP Warmboot) commands from the Process Control window.

Note: Files that do not have any associated OLTPs, but for which the Alter Data Source command may need to be entered (such as the Channel_Security and SEC_DEV_CONFIG), are not managed on the OLTP Warmboot Management window. Continue to enter the Alter Data Source commands for these files from the Process Control window or from a network control facility.

OLTP warmboot files

Whenever a file with an associated OLTP is modified through the ACI desktop user interface, the file server writes a record to the OLTP Warmboot File (OLTP_Warmboot). This new record specifies the Data Access Layer (DAL) assign name of the file, the date and time it was last updated, and the user ID of the person who made the change. Flags indicating whether commands have been submitted to rebuild and warmboot the OLTP are set to null (commands not sent), and the response data for the commands is set to blanks. If a record already exists in the OLTP_Warmboot for the file, the server updates the existing record with the date and time and user ID for the latest activity and resets the warmboot flags to null. Thus, there will never be more than one record in the OLTP_Warmboot for a file, regardless of how many times the file is modified.

Note: The flags indicating whether the OLTP Rebuild and OLTP Warmboot commands have been submitted only reflect commands entered on the OLTP Warmboot Management window. If you enter these commands from the Process Control window or from a network control facility, that activity is not reflected in the OLTP_Warmboot.

Another file called the Data Source OLTP Relationship File (DS_OLTP_Relation), maintains the relationships between file assigns and OLTP assigns as well as the destinations (e.g., PCTL) to which OLTP warmboot commands are sent. The DS_OLTP_Relation is populated using DALCI load scripts at installation and contains one record for each file and OLTP relationship in the BASE24-eps system. Many of these files are not used for BASE24 Transaction Security Services customers. If more than one OLTP is related to a file, a separate record is created for each OLTP related to the file.

OLTP Warmboot Management window

The OLTP Warmboot Management window provides an automated method of tracking whether a file with an associated external memory table (OLTP) has been modified and whether commands to rebuild and warmboot the OLTP have been entered or are still pending. If the OLTP for a modified file has not yet been rebuilt or warmbooted, you can enter either or both commands from this window with the click of a button.

All the records in the OLTP_Warmboot are displayed in a table on the OLTP Warmboot Management window whenever you access the window. The related OLTP assigns are retrieved from the DS_OLTP_Relation and are displayed in the Related OLTP Assign Names column. If multiple OLTP assigns are related to a file, they are separated by commas in this column.

A sample of the OLTP Warmboot Management window when it is accessed is shown on the following page, followed by a discussion of its features, functions, and procedures.

The screenshot shows the 'OLTP Warmboot Management' window. It contains a table with the following columns: Data Source Assign Name, Related OLTP Assign Names, Updated By, Update Time, OLTP Rebuild Submitted, and OLTP Warmboot Submitted. Below the table are two dropdown menus for 'OLTP Rebuild Destination' (set to XMLS) and 'OLTP Warmboot Destinations' (set to PCTL). At the bottom is a 'Response Information' section with a table for Command, Destination, and Response Message.

Data Source Assign Name	Related OLTP Assign Names	Updated By	Update Time	OLTP Rebuild Submitted	OLTP Warmboot Submitted
ACQ_TXN_ALLOWED	ACQUIRER_TXN_ALLOWED_OLTP	SYSADMIN	Sep 14, 2006 10:59:25 PM	No	No
ACT_CDE_EXT_INT	ACTION_CODE_EXTRN_TO_INTRN_OLTP	SYSADMIN	Sep 25, 2006 11:10:16 AM	No	No
ACT_CDE_INT_EXT	ACTION_CODE_INTRN_TO_EXTRN_OLTP	SYSADMIN	Sep 25, 2006 11:10:13 AM	No	No
APACS_CONFIG	APACS_CONFIG_OLTP	SYSADMIN	Sep 25, 2006 11:10:36 AM	No	No
ATM_CONSUMER_MEDIA	ATM_CONSUMER_MEDIA_OLTP	GUESTUSER06	Sep 19, 2006 3:15:00 PM	No	No
ATM_DS1	ATM_DS1_OLTP	GUESTUSER06	Sep 15, 2006 10:14:06 AM	No	No
ATM_FAST_CASH	ATM_FAST_CASH_OLTP	GUESTUSER06	Sep 15, 2006 10:03:50 AM	No	No
ATM_FLOW_CNTL	ATM_FLOW_CNTL_OLTP	GUESTUSER07	Sep 15, 2006 9:45:19 AM	No	No
ATM HOPR	ATM HOPR_OLTP	GUESTUSER06	Sep 15, 2006 10:04:14 AM	No	No

Command Destination

OLTP Rebuild Destination: XMLS OLTP Warmboot Destinations: PCTL

Response Information

Command	Destination	Response Message

2 - Successful view of OLTP Warmboot Management data.

The top portion of the OLTP Warmboot Management window displays a table of all the records in the OLTP_Warmboot. A value of Yes in the OLTP Rebuild Submitted and OLTP Warmboot Submitted columns indicates that a OLTP Rebuild or OLTP Warmboot command, respectively, has been submitted for the OLTP. A value of No in these columns indicates that the respective command still needs to be entered for the associated OLTP(s).

The Command Destination fields in the middle portion of the window specify the process type destinations for rebuilding and warmbooting OLTP process control commands, respectively.

The Response Information table at the bottom of the window displays the command response text returned for rebuild and warmboot OLTP commands entered for one or more selected OLTP_Warmboot rows. If no commands have been entered yet for the selected rows, then no information is displayed. Currently, the only command response text returned is "COMMAND SENT."

Toolbar functions

The Rebuild OLTP button () submits an OLTP Rebuild command for a selected row and the Warmboot OLTP button () submits an OLTP Warmboot command for a selected row. You use the cursor to click in a row to select it and you can select multiple rows at a time. If the OLTP has not yet been rebuilt for the selected row, the Warmboot OLTP button () is disabled.

After the OLTP for a row has been rebuilt and warmbooted, you can delete the row from the OLTP_Warmboot. Use the Delete Selected Row button () to delete one or more selected rows from the table.

You can also select these functions from the right-click menu for the OLTP Warmboot Management window shown below.

	Save	Ctrl+S
	Clear/Reset	Ctrl+L
	Refresh	Ctrl+R
	Delete Selected Row(s)	Delete
	Undo Selected Row(s)	Ctrl+Z
	Copy Row	Ctrl+Shift+C
	Submit OLTP Rebuild for Selected Row(s)	
	Submit OLTP Warmboot for Selected Row(s)	

Sorting OLTP_Warmboot entries

You can display records from the OLTP_Warmboot in the table at the top of the window in ascending or descending order of values for a particular column. When you first access the OLTP Warmboot Management window, records in the OLTP_Warmboot are displayed in ascending order for the OLTP Rebuild Submitted column. Thus, all the rows for which the Rebuild OLTP command has not yet been submitted are displayed at the top of the table.

To change the sorting of OLTP_Warmboot entries, click in the header of the column for which you want rows to be sorted. The triangle () in the column header is highlighted and the sort order is reversed (). The OLTP_Warmboot rows are now listed in descending order for

that column's values. The orientation of the highlighted triangle indicates whether the rows are sorted in ascending (▲) or descending (▼) order. To change the sort order for a column from ascending to descending or vice versa, simply click in the column header again.

In the following example, rows are displayed in descending order by file assign name.

	Data Source Assign Name ▼	Related OLTP Assign Names ▲
	VISA_DPS_INSTITUTION	VISA_DPS_INSTITUTION_OLTP
	TOKEN_CONFIG	TOKEN_CONFIG_OLTP
	SYSTEM_PREFIX	SYSTEM_PREFIX_OLTP
	SURCHARGE	SURCHARGE_OLTP
	STAR_INSTITUTION	STAR_INSTITUTION_OLTP
	PROC_CODE_ISO	PROC_CODE_ISO_OLTP
	PROC_CODE_ATM_TC	PROC_CODE_ATM_TRAN_CODE_OLTP
	MERCH_IM_TYP_RLN	MERCH_INSTR_TYPE_RELATION_OLTP

In the next example, rows are displayed in ascending order by file assign name.

	Data Source Assign Name ▲	Related OLTP Assign Names ▲
	ATM_TC_PROC_CODE	ATM_TRAN_CODE_PROC_CODE_OLTP
	CONTEXT_CONFIG	CONTEXT_CONFIGURATION_OLTP
	EURO_OPTED_CRNCY	EURO_OPTED_CURRENCY_OLTP
	EVENT	EVENT_OLTP
	HISO87_INTERFACE	HISO87_INTERFACE_OLTP
	HISO93_INTERFACE	HISO93_INTERFACE_OLTP
	IM_TYP_ICHG_RLN	INSTRUM_TYP_INTERCHANGE_REL_OLTP
	INTERFACE	INTERFACE_ACQUIRER_OLTP,INTERFACE_ISSUE

You can revert to the initial sort order of ascending for the OLTP Rebuild Submitted column at any time by clicking the Clear/Reset button (🧼).

Rebuilding OLTPs

To rebuild the OLTP(s) for one or more files, perform the following steps:

1. Select the rows for the files for which OLTPs are to be rebuilt. You can click anywhere in a row to select it.
2. Select or enter a value in the OLTP Rebuild Destination field or use the default destination of XMLS. If you enter a value it must be 1–16 characters in length.

Note: The TSS destination is intended only for BASE24-eps customers using a Transaction Security Services process in a BASE24-eps system to differentiate between Integrated Server and Transaction Security Services processes. You should not use this value in a BASE24 system unless you add an XPNET service of “TSS” to your Transaction Security Services processes and corresponding MDSDEST entries (e.g., TSS1, TSS2, TSS3, etc.).

3. Click the Submit OLTP Rebuild for Selected Row(s) button (). The window server builds and submits the respective Rebuild OLTP process control commands. The server submits a command for each OLTP assign listed in the Related OLTP Assign Names column for the selected rows.
4. Review the Response Information portion of the window to ensure the commands were submitted successfully.
5. Check the event log to verify the commands completed successfully.

If you refresh the window at this point, the OLTP Rebuild Submitted column for the file rows will now display a value of Yes.

Warmbooting OLTPs

To warmboot the OLTP(s) for one or more files, perform the following steps:

1. Select the rows for the files for which OLTPs are to be warmbooted. You can click any where in a row to select it.
2. Select or enter a value or values in the OLTP Warmboot Destination field or use the default destination of PCTL if the OLTP needs to be warmbooted. If you need to warmboot the OLTP to multiple destinations, separate each destination with a comma (e.g., IS1,IS2,IS3). Each value you enter must be 1–16 characters in length.

Note: The TSS destination is intended only for BASE24-eps customers using a Transaction Security Services process in a BASE24-eps system to differentiate between Integrated Server and Transaction Security Services processes. You should not use this value in a BASE24 system unless you add an XPNET service of “TSS” to your Transaction Security Services processes and corresponding MDSDEST entries (e.g., TSS1, TSS2, TSS3, etc.).

3. Click the Submit OLTP Warmboot for Selected Row(s) button (). The window server builds and submits the respective Warmboot OLTP process control commands. The server submits a command for each OLTP assign listed in the Related OLTP Assign Names column for the selected rows.
4. Review the Response Information portion of the window to ensure the commands were submitted successfully.
5. Check the event log to verify the commands completed successfully.

If you refresh the window at this point, both the OLTP Rebuild Submitted and OLTP Warmboot Submitted columns for the file row will now display Yes values. You can then delete the OLTP_Warmboot records as described below.

Deleting OLTP_Warmboot records

After all the OLTPs for a file have successfully rebuilt and warmbooted, you will need to delete the file record from the OLTP_Warmboot by performing the following steps:

1. Select the rows to be deleted by clicking anywhere in those rows.
2. Click the Delete Selected Row(s) (DELETE) button () or press the **Delete** key. The selected rows are flagged for deletion and the Save button () is enabled.
3. Click the Save button (). All OLTP_Warmboot entries flagged for deletion are deleted.

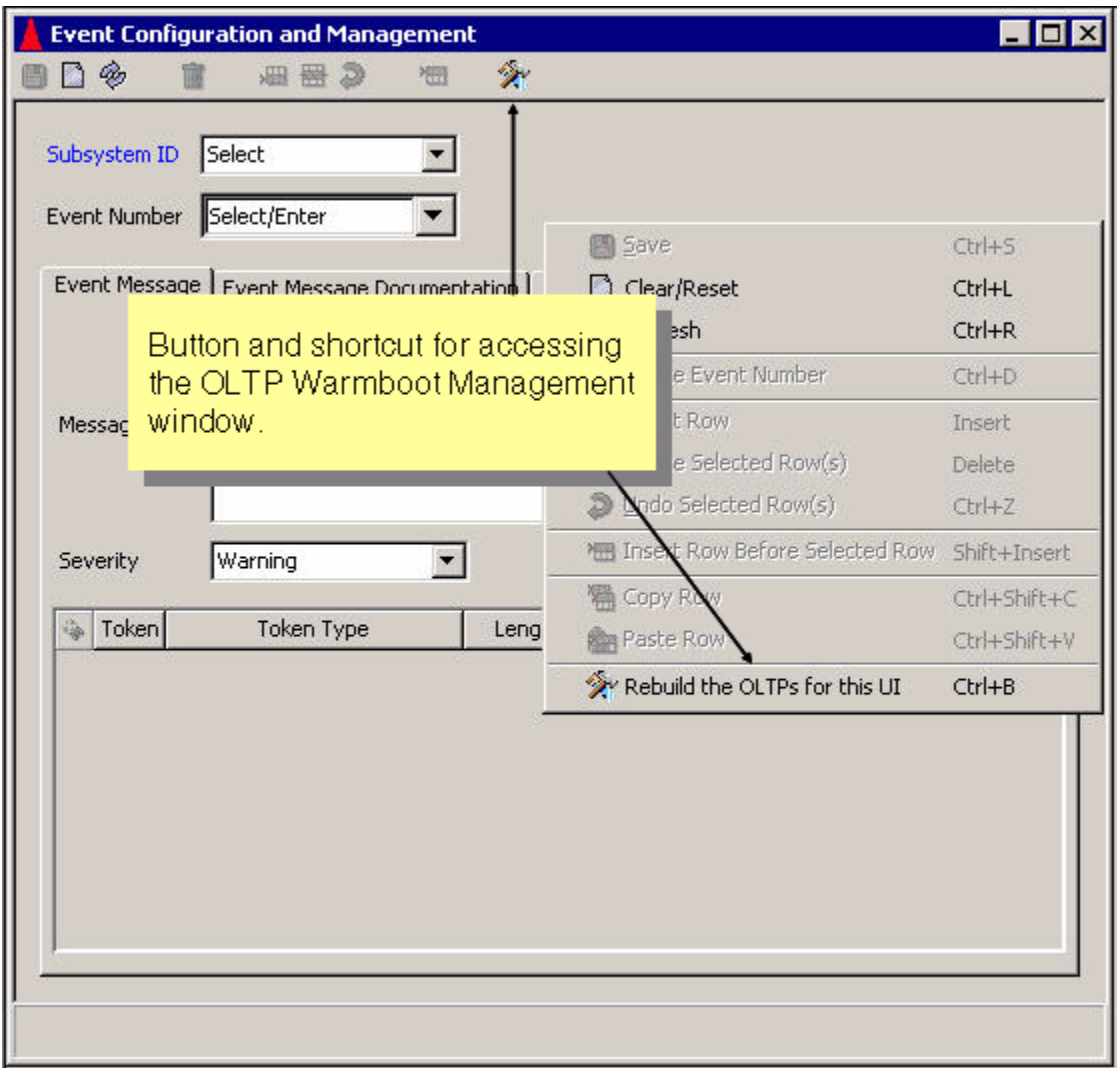
If you refresh the window at this point, the rows for the data sources no longer appear in the table at the top of the window.

Accessing the OLTP Warmboot Management window

For your convenience, a “Rebuild the OLTPs for this UI” button () and a shortcut (Ctrl + B) to access the OLTP Warmboot Management window, have been added to every window on the ACI desktop user interface that maintains a file with one or more associated OLTPs. The button () is displayed on the toolbar for the window and the shortcut syntax is displayed when you right-click anywhere in the window. Thus, you can quickly and easily navigate to the OLTP Warmboot Management window after you make a change to online processing data.

You can also navigate to the OLTP Warmboot Management window by accessing **System Operations > OLTP Warmboot Management**.

The “Rebuild the OLTPs for this UI” button () and the shortcut window for the Event Configuration and Management window is shown below.



12: Memory management and troubleshooting

BASE24-eps provides the capability to troubleshoot process memory issues using its proprietary Heap Manager component, which is bound into all BASE24-eps processes. The ACI Heap Manager component not only provides the routine extended memory management services required by a process, but also supports monitoring functions to identify memory leaks and instances where memory is corrupted by a process. These monitoring functions are accessed and activated using process control commands (refer to [“Heap Manager component”](#)).

- *Memory leaks* occur when a processing thread does not free up memory that it no longer needs, i.e., the thread requests memory from the Heap Manager but does not release it once it is no longer needed.
- Memory corruption is where a processing thread overwrites a memory parcel it has been allocated, e.g., the thread might request 10 bytes of memory, but allow 14 bytes to be written to the address.

Note: Although the ACI Heap Manager component is present in all BASE24-eps processes, its memory troubleshooting capabilities would typically only be of use to customers who have licensed the BASE24-eps Software Development Kit (SDK) and are developing customizations.

About the ACI Heap Manager

The ACI Heap Manager component is bound into all BASE24-eps processes to provide extended memory management services. It allocates and deallocates *heap memory*—an area of memory reserved for data created at runtime—for each of the process' threads. It also has built-in monitoring functions to help diagnose memory leaks and memory corruption—these built-in monitoring functions work the same across all platforms and are accessed and activated using process control commands (refer to [“Heap Manager component”](#)).

Here are several general notes about the ACI Heap Manager:

- The performance characteristics of the ACI Heap Manager component are similar to or better than the platform-supplied heap managers for those platforms on which BASE24-eps runs.

- The ACI Heap Manager component uses the underlying platform-supplied heap manager when performing its own memory allocation and for any memory parcel allocations over 4K.
- When the ACI Heap Manager is used, other memory diagnostic tools that interact with the platform-supplied heap manager are not likely to work correctly, because memory is not returned to the operating system heap.
- Previous versions of ACI Heap Manager required specific compiler settings in order to perform monitoring and diagnosis functions—these functions are now always accessible regardless of how the ACI Heap Manager was compiled.

References to Heap Manager

Unless otherwise differentiated (e.g., the *platform-supplied* heap managers), all references throughout the documentation to the *Heap Manager* refer to the ACI Heap Manager component.

How Heap Manager manages memory

BASE24-eps processes maintain a global pool of heap managers, one for each thread in the process. As threads are started, their heap manager allocates and deallocates the memory parcels used by the thread.

Internally, the Heap Manager component uses heap supervisors to manage memory parcels of different sizes. Each heap supervisor manages a single parcel size, which are as follows: 8-, 16-, 32-, 64-, 128-, 256-, 512-, 1024-, 2048-, and 4096-byte. The individual heap supervisors allocate large chunks of memory from the platform heap manager, splitting the large chunks into uniform parcels that the heap supervisors manage independently.

As memory is required by a thread for processing, the thread requests it using a standard C++ new function call to the heap manager. The requested memory size is rounded up to one of the standard memory sizes and the heap supervisor that manages that parcel size identifies the first available (unallocated) address in its list, marks the address as allocated, and returns the address to the thread.

When memory is no longer needed, the thread returns it using a standard C++ delete function call to the heap manager, and the heap supervisor that manages the parcel size returns it to its list to be reused and marks it as available.

If an address list becomes empty (completely allocated), the heap supervisor allocates another large chunk of memory, splitting it into the correct sizes, and adding all the addresses on the list.

Over time, the allocated memory should reach a peak size and never increase or decrease as memory is obtained and returned to the various lists.

How memory parcels are identified by Heap Manager

Each memory parcel has an eight-byte header that contains an eye catcher, a supervisor index (identifying the parcel size managed by the supervisor) and an optional length value used with memory verification.

Here is the format of the eight-byte header:

aabccdd

aa = An eyecatcher. Eyecatcher values are as follows:

Position		Description
1	2	
>	<i>space</i>	Memory parcel is on the free list and available for allocation.
<	<i>space</i>	Memory parcel was allocated while memory verification was deactivated.
<	<	Memory parcel was allocated while memory verification was activated.
!	<i>space</i>	Memory parcel was allocated from the platform-supplied heap manager while memory verification was deactivated (e.g., for parcels larger than 4K)
!	!	Memory parcel was allocated from the platform-supplied heap manager while memory verification was activated (e.g., for parcels larger than 4K)

b = A one-character, unprintable binary indicator identifying the supervisor that manages the parcel, based on size. Here are the hexadecimal equivalents for the indicator:

Hexadecimal equivalent	Supervisor for memory parcels
00	8 bytes
01	16 bytes
02	32 bytes
03	64 bytes
04	128 bytes
05	256 bytes

Hexadecimal equivalent	Supervisor for memory parcels
06	512 bytes
07	1,024 bytes
08	2,048 bytes
09	4,096 bytes

Note: The Heap Manager interacts directly with the platform-supplied heap manager to allocate/deallocate memory parcels larger than 4,096 bytes.

ccc = **Not used for memory monitoring or verification.**

dd = **A 2-character hex value representing the requested data length. For example, if a 32-byte parcel is allocated for a 20-byte value, *dd* will denote a length of 20-bytes. This value is only added to allocated memory parcels if memory verification is activated.**

Getting thread-level memory status information

The Heap Manager provides access to basic and detailed thread-level memory information using the following commands. Refer to “[Heap Manager component](#)” for information about entering these Process Control commands.

STATUS
STATUS DETAIL

Basic status information includes:

- Total number of bytes of heap memory that are currently allocated for the thread.
- Whether or not memory monitoring and stack tracing is activated for the heap manager.
- Whether or not memory verification is activated for the heap manager and the type of verification to be performed (at intervals, for deletes, or by transaction).

Detailed status information includes the following additional information:

- The number of memory parcels allocated and available for the thread, by parcel size.

Memory monitoring

Memory monitoring by the Heap Manager can be used to identify memory leaks in a process thread.

With memory monitoring activated, the Heap Manager tracks the total memory parcels that have been allocated, but not released. Unreleased parcel counts will increase if there are memory leaks in a thread, and the Heap Manager provides tools to detect increasing unreleased parcel counts.

In addition, because of the way memory monitoring changes the handling of memory parcels, it can also help detect instances where a thread returns a memory parcel to the heap but continues to reference the parcel address contents.

Memory parcel handling with memory monitoring activated

When memory monitoring is activated, all new memory allocations are initialized with ^ values, and memory that is deallocated is reset with * values. Here is an example of a 32-byte memory parcel.

```

1. Free (before):  >.....
2. New Allocation: <.....^.....
3. Returned:      <.....ONLY_USING_THIS_MUCH.....
4. Free (after):  >.....*****

```

Explanation:

1. The memory parcel is on the free list and available for allocation. Monitoring was not activated the last time it was placed in the free list but is now.
2. The memory parcel is allocated to a thread. The parcel is overwritten with ^ values.
3. The thread populates the memory address with "ONLY_USING_THIS_MUCH". The unused portion of the memory parcel remain as ^ values.
4. When the thread returns the memory to the heap, the heap supervisor overwrites the memory address with * values and returns it to the free list.

Note: This approach can also help detect memory that is not being properly released by a thread. If a thread continues to reference a memory address after the memory parcel is returned to the heap, all or portions of the original data may remain intact at the address making it more difficult to detect the error. Overwriting the memory with * values makes it obvious that the original value is no longer available.

Activating/deactivating memory monitoring

Memory monitoring is activated and deactivated using the following Process Control commands:

```
MONITOR ON  
MONITOR OFF
```

On an HP NonStop platform, stack tracing is automatically activated when monitoring is activated. On other platforms, you must activate and deactivate stack tracing using a separate command set:

```
MONITOR STACK ON  
MONITOR STACK OFF
```

Refer to [“Heap Manager component”](#) for information about entering these Process Control commands.

Why use stack tracing with memory monitoring?

When activated, memory monitoring can save procedure stack information with each new memory allocations so that unreleased memory can be tied to the procedure stack that requested (but did not release) it. In order to populate the procedure stack information, stack tracing must be activated. If stack tracing is not activated, the procedure stack information contains “UNKNOWN”.

Starting a Heap Trace

During testing, if a possible memory leak is detected, a Heap Trace must be activated to capture information for analysis. Refer to [“Process tracing”](#) for information about setting up and starting a Heap Trace.

Using memory monitoring to identify memory leaks

Unreleased parcel counts will increase if there are memory leaks in a thread. Once monitoring has been activated, the following Process Control commands can be used to detect increasing numbers of unreleased memory parcels.

```
RESET  
COMPARE
```

The RESET command sets the current number of memory parcels as an initial baseline for the thread and clears the current parcel totals to begin accumulating again. The COMPARE command then compares subsequent memory usage to the baseline and reports increases in unreleased memory parcels. The command also resets the baseline to the current usage and clears the current parcel totals to allow for another comparison later.

Workflow

Here is a basic task workflow for identifying memory leaks:

- Determine whether memory leaks exist - detects possible memory leaks in a process thread and captures the data necessary to find the leaks.
- Find the memory leaks - analyzes the captured data to identify the code causing the memory leaks.

Determine whether memory leaks exist

Used to detect possible memory leaks in a process thread and capture the data necessary to find the leak.

Before you begin

You must have a test environment established that includes a single-threaded version of the process to be tested and a simulator to generate transactions to be tested.

Steps

1. Start the process to be tested.
2. Access the Process Control window (**System Operations > Process Control**).
3. From the Process Control Window, activate memory monitoring. Use the following settings:

Destination:	Specify the symbolic name of the test process
Command:	Deliver
Component:	Heap Manager
Command Text:	MONITOR ON
4. (Optional) From the Process Control Window, activate stack tracing. Use the following settings:

Destination:	Specify the symbolic name of the test process
Command:	Deliver
Component:	Heap Manager
Command Text:	MONITOR STACK ON

Note: This command is not required on an HP NonStop platform, because memory monitoring automatically activates stack tracing on the HP NonStop platform.

Note: Depending on the performance impacts to transactions on the platform, you may need to leave stack tracing off—your test transactions must be able to complete as intended without performance-related errors.

5. From your transactions simulator, send a transaction that exercises the code you want to test for memory leaks. This causes memory to be allocated for the transaction.

Note: The test transaction must complete with the results expected—do not continue to the next step until the test transaction completes as intended.

6. Send the test transaction two more times. This *burns in* the memory allocations for the transaction path you are testing.
7. From the Process Control Window, reset the baseline memory allocation level for the process. Use the following settings:

Destination:	Specify the symbolic name of the test process
Command:	Deliver
Component:	Heap Manager
Command Text:	RESET

8. Send the test transaction three times.

Note: The test transaction must complete consistently with the results expected—without alternative results.

9. From the Process Control Window, enter the COMPARE command to determine whether memory allocations have increased from the reset baseline in step 7. Use the following settings:

Destination:	Specify the symbolic name of the test process
Command:	Status
Component:	Heap Manager
Command Text:	COMPARE

Note: The COMPARE command also sets a new baseline in order to allow for repeat testing.

10. Check the messages returned to the Response History section of the Process Control Window to determine whether memory allocations have increased.

- ☐ If memory allocations have not increased as a result of running the transaction multiple times, there are no memory leaks and you can stop.
- ☐ If memory allocations have increased, continued to the next step.

11. Send the test transaction three more times.

Sometimes a false leak can be reported—running the transaction multiple times can clear it up if it is a false leak. A true leak does not go away.

12. From the Process Control Window, enter the COMPARE command again using the following settings:

Destination:	Specify the symbolic name of the test process
Command:	Status
Component:	Heap Manager
Command Text:	COMPARE

13. Check the messages returned to the Response History section of the Process Control Window to determine whether memory allocations have increased.

- ☐ If memory allocations have not increased again, the first reported increase was not due to a memory leaks and you can stop.

- If memory allocations have increased again, make note of the parcel sizes where the increases have occurred and continue to the next step—the COMPARE command returns the following type of information during testing:

```
Detail comparison for memory parcel size: 8
      Additional parcels used: 2
Detail comparison for memory parcel size: 16
      Additional parcels used: -1
Detail comparison for memory parcel size: 32
      Additional parcels used: 3
Detail comparison for memory parcel size: 64
      Additional parcels used: -1
Detail comparison for various sizes has changed from 1 to 0
```

In this case, unreleased parcel sizes 8 and 32 have increased.

14. Initiate a Heap Trace to capture the data needed to analyze the memory leak.

- a. Access the Trace Configuration window on the ACI desktop user interface (**System Operations > Trace > Trace Configuration**) and set the fields as follows:

Process Type	Specify the type of executable (process) you are testing.
Trace Output Assign Name	Specify the assign name of the Trace data source.
Trace Filter Profile	Do not specify a trace filter profile.
Process Trace is Active	Place a check mark in the box.
Include Header in Trace	Do not place a check mark in the box.

- b. Add a single **Trace Level** entry of HEAP. Let the corresponding **Component ID Filter** and **User Data** fields default.
- c. Click Save. The record is stored in the Trace Configuration data source (Trace_Config).
- d. Right-click anywhere on the Trace Configuration window and select the **Alter Trace Configuration Data Source** action.

The Process Control window is displayed with all the fields preset to the values you need to warmboot the appropriate processes to reread the Trace Configuration data source (Trace_Config).

Destination: PCTL
Command: Alter
Component: Data Source
Command Text: trace_config

- e. Click **Submit**. This causes all of the Heap Trace information to be written out to the specified location where it can be analyzed.

Next

Refer to [“Find the memory leaks”](#)

Find the memory leaks

Before you begin

You need the information you collected in the [“Determine whether memory leaks exist”](#) task.

- A list of the memory parcel sizes in which there were increases in the number of unreleased parcels during testing.
- The trace file created to capture the unreleased memory parcel information from testing.
- You need to be able to open the trace file with the Trace Viewer utility. Refer to [“Examining trace output”](#) for information about starting the trace viewer.

Steps

1. Open the trace file.
2. Review the unreleased parcels for those parcel sizes that have shown increases in unreleased memory during testing—starting with the largest parcels first.
3. Look for repetitive parcel data—unreleased memory data allocated by the same code function to carry the same data. Refer to [“Analyzing Heap Trace output”](#) for an example of the trace output.
4. Identify the code functions causing the memory leaks.
From there, you can review and make changes as needed to the code functions to correct the memory leaks.

Analyzing Heap Trace output

Here are some hints to remember as you reviewing and analyzing the Heap Trace output:

- Correcting larger issues often corrects smaller one. Larger parcels can also be easier to pick out, and there are often fewer of them to look for.
- Just because memory is used does not mean the memory is being leaked. It could simply be allocated while processing the report. Or, by design, it may remain unreleased until the next transaction is processed.
- Additional memory could have been allocated due to an unintentional change in the transaction path during testing from the original transaction burned in.
- Recent changes in a process function or new transaction paths being executed may be areas to focus on.
- For non-obvious leaks, you may need to use a debugging tool. In this case, you can note the addresses within the parcel sizes in question and set breakpoints in the functions identified or in the heap managers OBTAIN function. This would enable you to examine

the code in debug when the noted addresses are returned. Keep in mind that a single address may be used more than once in a single transaction so the first use of an address may not be creating the problem.

Annotated sample

An annotated sample from the Heap Trace file is provided below. Some things to note:

1. For purposes of this example, the following coding errors have been used to create memory leaks in the 8-byte and 32-byte memory parcels.

```
// Memory is leaked because there is no delete.
if ( tkn == "LEAK" )
{
    char * ptr1 = new char[ 6 ];
    strcpy( ptr1, "LEAK1" );
    char * ptr2 = new char[ 30 ];
    strcpy( ptr2, "LEAK 2 for 30 bytes" );
    return;
}
```

2. A test transaction was run three times to create the output. So any leaks should appear three times.
3. The content appears and is annotated in the order it would in a Heap Trace file. However, when working through the output, it is better to start at the end—with largest parcels first—and work backwards.

Sample Heap Trace file output	
Annotation	Content
Identifies the count of unreleased 8-byte parcels. <i>This information is present if any parcels of a particular size are not released.</i> The 8-byte parcels need to be reviewed, because there was a reported increase in unreleased 8-byte parcels.	<pre>***** Memory used for size = 8, count = 11 are: *****</pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcel 1 Identifies the function that requested the unreleased the memory parcel. The function is listed as UNKNOWN if stack tracing is not activated.	Memory obtained in function: __nwa_FUi + 0x290 (UCr).
Unreleased parcel 1 Stack information at the time the COMPARE command was run This information is only present if stack tracing has been activated.	<pre> deliver_cmd__9elog_implFRC13fn_string_var + 0x840 (UCr). deliver_cmd_cbk__9elog_implSFPvPCcUiRQ2_3std10ostream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCcT1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) </pre>
Unreleased parcel 1 Address and address contents of the unrelease memory parcel (in hex and character format).	<pre> Address = 147364624 hex and character contents are: 3C3C000000000064C45414B3100405E '<<.....LEAK1.@^' </pre>
Unreleased parcel 2 Notice that the function, stack and memory contents are identical to the previous parcel. This suggests a possible memory leak.	<pre> Memory obtained in function: __nwa_FUi + 0x290 (UCr). deliver_cmd__9elog_implFRC13fn_string_var + 0x840 (UCr). deliver_cmd_cbk__9elog_implSFPvPCcUiRQ2_3std10ostream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCcT1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147364608 hex and character contents are: 3C3C000000000064C45414B3100405E '<<.....LEAK1.@^' </pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcel 3 <i>Here again, the function, stack, and memory contents are identical to the previous parcels.</i> <i>Since the transaction was run three times, this is a likely situation where memory has not been released.</i> Although you cannot tell with certainty by looking at the stack which function caused the leak, the issue can typically be found in one of the top three functions in the stack. In this example, the leak was caused by the second function (highlighted in red).	<pre>Memory obtained in function: __nwa__FUi + 0x290 (UCr). deliver_cmd_cbk__9elog_implSFPvPCcUiRQ2_3std10ostream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCcT1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147364592 hex and character contents are: 3C3C000000000064C45414B3100405E '<<.....LEAK1.@^'</pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcels 4–7 <i>These are unreleased parcels, but there is no repetition, which you would see with a memory leak because the transaction was run three times.</i> These are the result of sending Process Control commands to get the diagnostics.	<pre> Memory obtained in function: __nw__FUi + 0x290 (UCr). __ct__13fn_md_rcv_msgFPCv + 0x460 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x210 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr). Address = 147366480 hex and character contents are: 3C3C00000000000891A718085D92F0 '<<.....]...' Memory obtained in function: __nwa__FUi + 0x290 (UCr). set__9sis_c_strFPCc + 0x510 (UCr). set__13fn_string_varFPCc + 0xF0 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0x120 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr). Address = 147366656 hex and character contents are: 3C3C0000000000100405E5E5E5E5E5E '<<.....@^^^^^^' Memory obtained in function: __nwa__FUi + 0x290 (UCr). __as__9sis_c_strFRC9sis_c_str + 0x2E0 (UCr). __as__13fn_string_varFRC13fn_string_var + 0xF0 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0x9C0 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr). Address = 147366928 hex and character contents are: 3C3C0000000000075354415455530040 '<<.....STATUS.@" Memory obtained in function: __nwa__FUi + 0x290 (UCr). __as__9sis_c_strFRC9sis_c_str + 0x2E0 (UCr). __as__13fn_string_varFRC13fn_string_var + 0xF0 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0x12E0 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr). Address = 147366944 hex and character contents are: 3C3C0000000000054845415000405E5E '<<.....HEAP.@" </pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcels 8–10 <i>Here again, these are unreleased parcels, but there is no repetition.</i>	<pre> Memory obtained in function: __nw_FUi + 0x290 (UCr). __ct_Q2_3std54basic_streambuf_tm_31_cQ2_3std20char_traits_tm_2 + 0x70. __ct_Q2_3std12strstreambufFi + 0x60 (DLL zcpp3dll). __ct_Q2_3std10ostreamFv + 0x130 (DLL zcpp3dll). msg_pro_9pctl_implFPCCUiR13fn_string_var + 0x1510 (UCr). msg_in_9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk_9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in_13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in_7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg_12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg_7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147366800 hex and character contents are: 3C3C0000000000085E5E5E5E087838A8 '<<.....^x8.'</pre> <pre> Memory obtained in function: __nw_FUi + 0x290 (UCr). stat_cmd_in_cbk_11fn_heap_mgrSFPvPCcUiRQ2_3std10ostream + 0xF0 (UCr). send_cmd_15pctl_coll_entryFPCCt1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd_9pctl_implFPCCPC13fn_string_varT2RQ2_3std10ostream + 0x8B0. msg_pro_9pctl_implFPCCUiR13fn_string_var + 0x1840 (UCr). msg_in_9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk_9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in_13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in_7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg_12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg_7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147366576 hex and character contents are: 3C3C0000000000045E5E5E5E405E5E5E '<<.....^x8.^x8.'</pre> <pre> Memory obtained in function: __nw_FUi + 0x290 (UCr). stat_cmd_in_cbk_11fn_heap_mgrSFPvPCcUiRQ2_3std10ostream + 0x140 (UCr). send_cmd_15pctl_coll_entryFPCCt1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd_9pctl_implFPCCPC13fn_string_varT2RQ2_3std10ostream + 0x8B0. msg_pro_9pctl_implFPCCUiR13fn_string_var + 0x1840 (UCr). msg_in_9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk_9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in_13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in_7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg_12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg_7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147366960 hex and character contents are: 3C3C0000000000045E5E5E5E405E5E5E '<<.....^x8.^x8.'</pre>
Identifies three 16-byte parcels as unreleased.	<pre> **** Memory used for size = 16, count = 3 are: ****</pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcels 1–2 <i>These do not need to be reviewed because there was no reported increase in unreleased 16-byte parcels.</i>	<pre> Memory obtained in function: __nw_FUi + 0x290 (UCr). __ct__8fn_tknzrFRC13fn_string_var + 0x160 (UCr). msg_pro__9pctl_implFPCCUiR13fn_string_var + 0x830 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147243296 hex and character contents are: 3C3C0100000000100000000908C6C0F800000008085D4280 '<<.....]B.'</pre> <pre> Memory obtained in function: __nwa_FUi + 0x290 (UCr). __as__9sis_c_strFRC9sis_c_str + 0x2E0 (UCr). __as__13fn_string_varFRC13fn_string_var + 0xF0 (UCr). __cl__8fn_tknzrFRC13fn_string_varb + 0x540 (UCr). __cl__8fn_tknzrFPCcb + 0x300 (UCr). msg_pro__9pctl_implFPCCUiR13fn_string_var + 0x1440 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147243248 hex and character contents are: 3C3C010000000009204D4F4E49544F5200405E5E5E5E5E '<<..... MONITOR.@^^^^^'</pre>
Unreleased parcel 3 <i>Does not need to be reviewed because there was no reported increase in unreleased 16-byte parcels.</i>	<pre> Memory obtained in function: __nwa_FUi + 0x290 (UCr). __as__9sis_c_strFRC9sis_c_str + 0x2E0 (UCr). __as__13fn_string_varFRC13fn_string_var + 0xF0 (UCr). msg_pro__9pctl_implFPCCUiR13fn_string_var + 0x1490 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147243224 hex and character contents are: 3C3C0100000000094D4F4E49544F520000405E5E5E5E5E '<<.....MONITOR..@^^^^^'</pre>
Identifies five 32-byte parcels as unreleased. The 32-byte parcels need to be reviewed, because there was a reported increase in unreleased 32-byte parcels.	<pre> ***** Memory used for size = 32, count = 5 are: *****</pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcel 1	<pre> Memory obtained in function: __nwa_FUi + 0x290 (UCr). deliver_cmd__9elog_implFRC13fn_string_var + 0x8E0 (UCr). deliver_cmd_cbk__9elog_implSFPvPCCUiRQ2_3stdl0ostrstream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCCt1UiRQ2_3stdl0ostrstream + 0x2E0 (UCr). send_cmd__9pctl_implFPCCPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCCUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147336536 hex and character contents are: 3C3C02000000001E4C45414B203220666F72203330206279 '<<.....LEAK 2 for 30 by' 746573005E5E5E5E5E5E5E5E5E5E405E 'tes.^^^^^^^^^^@^' </pre>
Unreleased parcel 2 <i>Notice that the function, stack and memory contents are identical to the previous parcel. This suggests a possible memory leak.</i>	<pre> Memory obtained in function: __nwa_FUi + 0x290 (UCr). deliver_cmd__9elog_implFRC13fn_string_var + 0x8E0 (UCr). deliver_cmd_cbk__9elog_implSFPvPCCUiRQ2_3stdl0ostrstream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCCt1UiRQ2_3stdl0ostrstream + 0x2E0 (UCr). send_cmd__9pctl_implFPCCPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCCUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147336496 hex and character contents are: 3C3C02000000001E4C45414B203220666F72203330206279 '<<.....LEAK 2 for 30 by' 746573005E5E5E5E5E5E5E5E5E5E405E 'tes.^^^^^^^^^^@^' </pre>

Sample Heap Trace file output	
Annotation	Content
Unreleased parcel 3 <i>Here is a third occurrence in which the function, stack, and memory contents are identical to the previous parcels.</i> <i>Since the transaction was run three times, this is a likely situation where memory has not been released.</i> Although you cannot tell with certainty by looking at the stack which function caused the leak, the issue can typically be found in one of the top three functions in the stack. In this example, the leak was caused by the second function (highlighted in red).	<pre>Memory obtained in function: __nwa__FUi + 0x290 (UCr). deliver_cmd_cbk__9elog_implSFPvPCcUiRQ2_3std10ostream + 0x420 (UCr). send_cmd__15pctl_coll_entryFPCcT1UiRQ2_3std10ostream + 0x2E0 (UCr). send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr). msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr). msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr). msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr). msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr). msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr). wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr). wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr). appl_init + 0xA70 (UCr). main + 0x330 (UCr). _MAIN + 0x90 (UCr) Address = 147336456 hex and character contents are: 3C3C02000000001E4C45414B203220666F72203330206279 '<<.....LEAK 2 for 30 by' 746573005E5E5E5E5E5E5E5E5E5E405E 'tes.^^^^^^^^^^^g^'</pre>

[illegible]

Memory verification

Memory verification can be used to check for corruption caused by processes overwriting their allocated memory. If memory corruption is detected, the Heap Manager immediately writes memory information to the home terminal (from which the process was started) and causes the process to abend.

Enabling / disabling memory verification

Memory verification is activated and deactivated using the following Process Control commands:

VERIFY ON
VERIFY OFF

Refer to “[Heap Manager component](#)” for information about entering these commands.

Setting the memory verification interval

Memory verification can be very CPU-intensive because there may be many thousands of memory allocations to verify at a time. To manage performance impacts, Heap Manager requires that you specify how often the actual verification is to be performed by entering additional commands.

Interval	Command / Description
Default	By default, when memory verification is activated, memory parcel handling is changed to allow for verification, but no memory verification is actually performed.
One-Time	You can initiate a one-time memory verification by entering a VERIFY command. This action verifies all current memory allocations to ensure they all begin with an appropriate eye catcher and have not been overwritten beyond their expected data length. This is a one-time verification and can be used if you want to initiate the verification yourself rather than setting up recurring verification.
After Each Delete	You can have Heap Manager verify memory after each delete by entering a VERIFY DELETE command. In this case, every time memory is deleted/released by the thread, the Heap Manager verifies that parcel has not been corrupted—either by writing over its internal eyecatcher, or by writing into the next contiguous memory address. In this case, only released parcels and their next contiguous memory addresses are verified; the entire list of memory parcels is not verified.

Memory parcel handling with memory verification activated

The primary means by which the Heap Manager identifies overwritten memory is by creating eyecatchers and then verifying that they are not overwritten.

Here is an example of a 32-byte allocated memory parcel with and without memory verification and monitoring activated:

```

1. Allocated Memory: <.....
2. Allocated Memory: <.....^.....
3. Allocated Memory: <<.....dd.....@.....
4. Allocated Memory: <<.....dd.....

```

Explanation:

1. Memory verification is deactivated; memory monitoring is deactivated.

2. The full memory parcel is used; no error is detected in the parcel itself.
3. The next contiguous memory parcel is checked. In this case, it is missing the beginning "<<" eyecatcher, which means the code has written to memory beyond what was allocated and there is a corruption. So the process terminates.

This same approach is used to verify memory parcels allocated prior to activating verification. In this case, some parcels were allocated prior to adding @ eyecatchers. So the only way they can be verified is by verifying the beginning eyecatcher of the next contiguous memory parcel.

In this example, the requested memory allocation was for 20 bytes.

1. Allocated Memory: <.....
2. Returned Memory: <.....OVERWROTE.REQUESTED.MEMORY.AND T
3. Next Parcel: HEN SOME.....

Explanation:

1. Because the memory was allocated prior to activating memory verification or memory monitoring, the starting eyecatcher is set to "< ", and no other eyecatcher is set.
2. The full memory parcel is used even though only 20 bytes were requested; no error is detected in the parcel itself because the Heap Manager is not expecting to find an end eyecatcher because the parcel was allocated before
3. The next contiguous memory parcel is checked. In this case, it is missing the beginning eyecatcher, which means the code has written to memory beyond what was allocated and there is a corruption. So the process terminates.

Detect memory corruption

Use these steps to detect possible memory corruption in a process thread and capture the data necessary to find the corruption.

Before you begin

You must have a test environment established that includes a single-threaded version of the process to be tested and a simulator to generate transactions to be tested.

Steps

1. Start the process to be tested.
2. Access the Process Control window (**System Operations > Process Control**).

3. (Optional) Enter the MONITOR ON command to activate memory monitoring. Use the following settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: MONITOR ON

Note: This step is only required if you can also activate stack tracing (next step). If stack tracing cannot be activated for testing on your platform, skip to step 5.

4. (Optional) Enter the MONITOR STACK ON command to activate stack tracing. Use the following settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: MONITOR STACK ON

Note: This command is not required on an HP NonStop platform, because memory monitoring automatically activates stack tracing on the HP NonStop platform.

Note: Depending on the performance impacts to transactions on the platform, you may need to leave stack tracing off—your test transactions must be able to complete as intended without performance-related errors.

5. Enter the VERIFY ON command to activate memory verification; Use the following settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: VERIFY ON

6. Enter the VERIFY DELETE command to initiate verification after each memory delete. Use the following settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: VERIFY DELETE

7. Enter the VERIFY TXN command to set verification at the transaction level. Use the following command settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: VERIFY TXN

8. Enter the VERIFY INTERVAL command to set an interval for memory verification. Use the following command settings:

Destination: Specify the symbolic name of the test process
Command: Deliver
Component: Heap Manager
Command Text: VERIFY INTERVAL 1

Memory verifications at shorter intervals take longer to run but are easier to analyze. If the transaction takes too long to run, you can set the interval higher or initiate the verifications one at a time by command. The corruption will still be identified, but the window in which the corruption occurred will be larger.

9. (Optional) Enter a STATUS command and check the response to verify Heap Manager is set up properly. Use the following settings:

Destination: Specify the symbolic name of the test process
Command: Status
Component: Heap Manager
Command Text: DETAIL (or leave blank)

10. From your transaction simulator, send a transaction that exercises the code you want to test for memory corruption.

If memory corruption is encountered, the process terminates immediately sending memory information to the home terminal.

11. If the transaction terminates, capture and save the process termination information sent to the home terminal for analysis. Refer to [“Analyzing process termination output”](#).
12. Repeat steps as needed to test the process.

Analyzing process termination output

Of primary importance to analyzing process termination output is that the process termination occurs at the point the memory corruption is detected—not at the point the memory corruption is created.

So, the corruption always occurred *between* the last memory verification that was successful and the current memory verification that fails. If the interval is set to 1, the processing that caused the corruption is located between the previous and current allocation/deallocation actions. The larger the interval, the more memory actions that could have caused the corruption.

Annotated sample 1

Here is an example where memory was allocated in PKLIBS and the problem was found from that point to the next allocation/deallocation performed within the PKLIBS processing. In this case, the code did a STRCPY when it should have been a MEMCPY for proper length, and the STRCPY added a null character beyond the length of the string that was requested.

Sample process termination output	
Annotation	Content
<i>An indication that a parcel has been overwritten.</i>	Own buffer has been overwritten

Note that the ptr3 actions overwrite its requested memory, but the corruption is detected when the ptr 4 memory allocation is requested.

Sample process termination output	
Annotation	Contents
An indication that a parcel has been overwritten.	Own buffer has been overwritten.
Parcel 1 The address that has been overwritten, along with the size of the memory request (6 bytes)	Start of address overwrite = 147367008 size = 6 heap supervisor verify.
Stack information for parcel 1 at the time the process terminated. This information is only present if stack tracing has been activated.	Buffer allocated from __nwa_FUi + 0x290 (UCr)?. deliver_cmd__9elog_implFRC13fn_string_var + 0xB50 (UCr)?. deliver_cmd_cbk__9elog_implSFPvPCcUiRQ2_3std10ostream + 0x420 (UCr)?. send_cmd__15pctl_coll_entryFPCcT1UiRQ2_3std10ostream + 0x2E0 (UCr)?. send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr)?. msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr)?. msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr)?. msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr)?. msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr)?. msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr)?. wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr)?. wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr)?. appl_init + 0xA70 (UCr)?. main + 0x330 (UCr)?. _MAIN + 0x90 (UCr)?.
Address and address contents of the memory parcel (in hex and character format).	hex and character contents are:. 3C3C00000000000064F56455257524954 '<<.....OVERWRT'.

Sample process termination output	
Annotation	Contents
Memory Parcel 2 <i>This is the next adjacent memory parcel in which error was detected. In this case, the initial eyecatcher was overwritten (by the “E” in “OVERWRITE”) from the previous memory parcel.</i> Note: The “CORRUPT” in this memory parcel is from a previous use of the memory parcel—from the “(tkn == "CORRUPT")” in the sample coding error.	<pre> Start of address overwrite = 147367024 size = 16 index into memory = 1057. Buffer allocated from __nwa__FUi + 0x290 (UCr)?.. __as__9sis_c_strFRC9sis_c_str + 0x2E0 (UCr)?.. __as__13fn_string_varFRC13fn_string_var + 0xF0 (UCr)?.. __cl__8fn_tknzrFRC13fn_string_varb + 0x540 (UCr)?.. __cl__8fn_tknzrFPCcb + 0x300 (UCr)?.. deliver_cmd__9elog_implFRC13fn_string_var + 0x740 (UCr)?.. deliver_cmd_cbk__9elog_implSFPvPCCUiRQ2_3std10ostrstream + 0x420 (UCr)?.. send_cmd__15pctl_coll_entryFPCcTlUiRQ2_3std10ostrstream + 0x2E0 (UCr)?.. send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr)?.. msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr)?.. msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr)?.. msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr)?.. msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr)?.. msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr)?.. wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr)?.. wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr)?.. appl_init + 0xA70 (UCr)?.. main + 0x330 (UCr)?.. _MAIN + 0x90 (UCr)?.. Previous buffer allocated from __nwa__FUi + 0x290 (UCr)?.. deliver_cmd__9elog_implFRC13fn_string_var + 0xB50 (UCr)?.. deliver_cmd_cbk__9elog_implSFPvPCCUiRQ2_3std10ostrstream + 0x420 (UCr)?.. send_cmd__15pctl_coll_entryFPCcTlUiRQ2_3std10ostrstream + 0x2E0 (UCr)?.. send_cmd__9pctl_implFPCcPC13fn_string_varT2 + 0x940 (UCr)?.. msg_pro__9pctl_implFPCcUiR13fn_string_var + 0xF80 (UCr)?.. msg_in__9pctl_implFR13fn_md_rcv_msg + 0x6F0 (UCr)?.. msg_in_cbk__9pctl_implSFPvR13fn_md_rcv_msg + 0x410 (UCr)?.. msg_in__13md_coll_entryCFR13fn_md_rcv_msg + 0x200 (UCr)?.. msg_in__7md_implFRC11sis_rcv_msg + 0x1170 (UCr)?.. wait_for_msg__12sis_mds_implFR12sis_msg_rslt + 0x65C0 (UCr)?.. wait_for_msg__7sis_mdsFR12sis_msg_rslt + 0x100 (UCr)?.. appl_init + 0xA70 (UCr)?.. main + 0x330 (UCr)?.. _MAIN + 0x90 (UCr)?.. hex and character contents are:. 45000000000000008434F52525505400 'E.....CORRUPT.'. </pre>

13: Dynamic card verification

This section describes the dynamic card verification support provided by BASE24-pos and the Transaction Security Services process for contactless magnetic stripe cards, or proximity cards. It provides an overview of contactless card technology, standards, and supported contactless card issuer programs. It also describes dynamic card verification processing, the available options, and how to configure the BASE24 and TSS databases to enable dynamic card verification.

Overview

Contactless cards, or proximity cards, use wireless technologies such as Radio Frequency Identifier (RFID) transponders, Bluetooth, Infrared, Wireless Fidelity (WiFi) to exchange data without physical contact. Contactless cards supported by BASE24-pos are based on the ISO/IEC 14443 standard, **Identification cards -- Contactless integrated circuit(s) cards -- Proximity cards**. They contain both a proximity integrated circuit (chip) and an antenna for transmitting data from the chip to a contactless interface when in close proximity.

MasterCard and Visa support contactless card programs called PayPass and payWave, respectively. For both programs, the proximity integrated circuit (chip) can be based on EMV standards or magnetic stripe standards. Visa also has a solution that combines the EMV and magnetic stripe standard for contactless transaction processing. This solution is generally referred to as “MSD with CVN 17”.

The card verification requirements for contactless cards differ, depending on which of these standards they are based and whether a transaction is acquired from a contactless interface or from a contact magnetic stripe or chip reader. Because a contactless interface may not always be available, PayPass and payWave cards also contain a physical magnetic stripe or EMV chip so that transactions can also be initiated from a contact magnetic stripe card reader or EMV card reader.

Note: Dynamic card verification is only performed for contactless magnetic stripe transactions. For contactless EMV transactions, verification of the iCVD for contactless EMV cards is the same as for contact EMV cards, except the Point-of-Service (POS) entry mode value is different. For more information on iCVD verification, refer to the **BASE24 Transaction Security Manual**.

EMV standard

For contactless cards based on the EMV standard, the iCVD or CVD from the transaction is verified using the card verification processing described in the **BASE24 Transaction Security Manual**. For EMV transactions, TLV data is also used for ARQC verification checking to ensure the validity of the authorization request. Refer to the **BASE24-pos EMV Support Manual** for a description of ARQC verification provided for the BASE24-pos product. Verification of the iCVD for contactless cards based on the EMV standard is the same as that performed for contact EMV cards except that the Point-of-Service (POS) entry mode value is different in the transaction request. For contactless EMV transactions, the POS entry mode is set to 07 (Contactless chip transaction) in the transaction request. For contact EMV transactions, the POS entry mode is set to 05 (Integrated circuit card) in the request.

Magnetic stripe standard

For contactless cards based on the magnetic stripe standard, the dynamic card verification digits (DCVD) from the transaction are verified using a different algorithm from the standard algorithm used to verify the CVD. For MasterCard PayPass cards, the DCVD is referred to as the CVC3. For Visa payWave cards, the DCVD is referred to as the dCVV. For American Express Expresspay cards, the DCVD is referred to as the The CVC3 and dCVV are dynamic cryptograms generated by the card.

Note: Static CVC3 cryptograms for PayPass cards are not supported in this release.

The dynamic card verification algorithm differs from the standard card verification algorithm in that it uses additional input data beyond the PAN, card expiration date, and service code to verify the card. For Visa payWave cards, the PAN sequence number and application transaction counter (ATC) are used in addition to the other inputs to compute the dCVV compared to the CVV read from the magnetic stripe on the card. For MasterCard PayPass cards, the PAN sequence number, ATC, static track data, and an unpredictable number are used in addition to the other inputs to compute the CVC3 compared to the CVC read from the magnetic stripe on the card. The unpredictable number is provided by the contactless terminal and is included in the data sent from the terminal to the BASE24 system. All of this additional data input to the dynamic card verification algorithm is carried in the issuer discretionary data (IDD) of the Track 1 or Track 2 data sent in the transaction request.

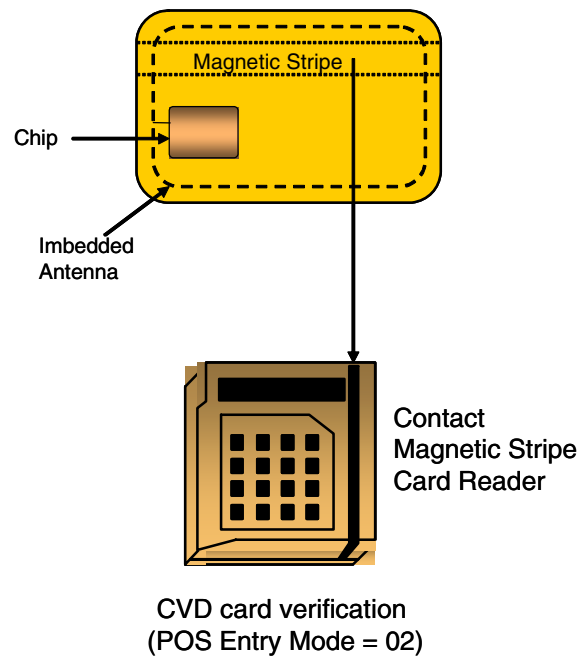
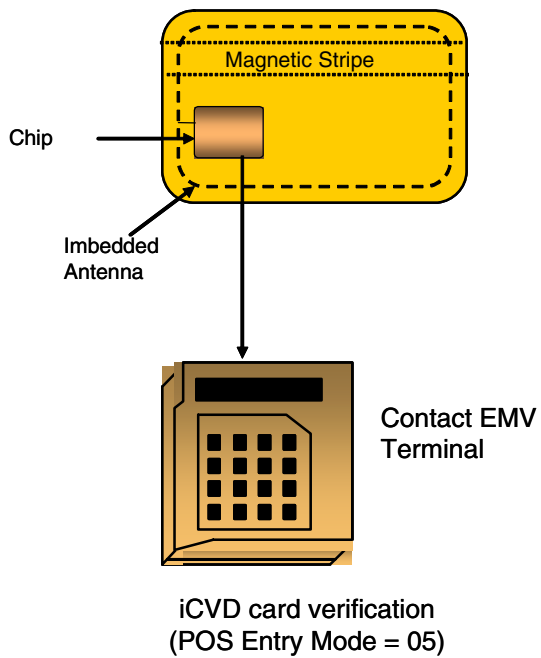
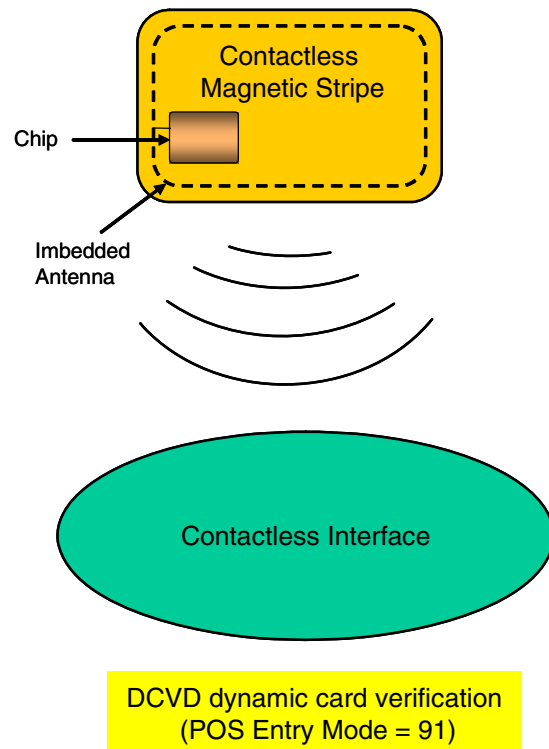
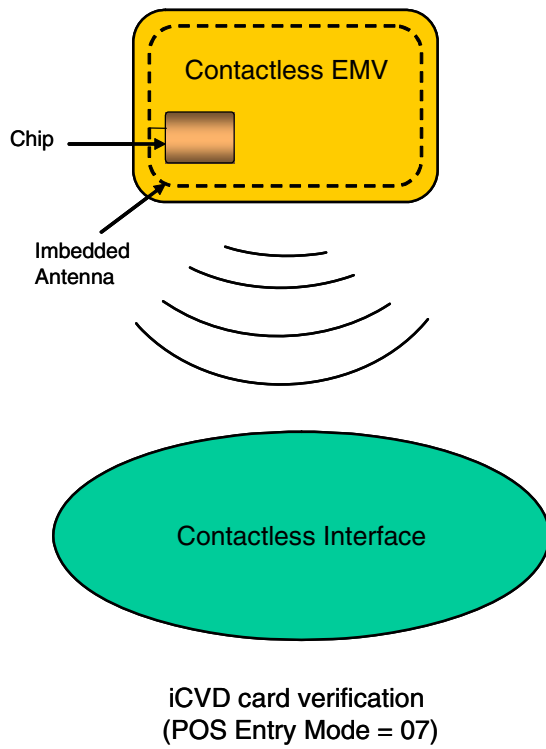
For contactless magnetic stripe transactions, the POS entry mode is set to 91 (Contactless magnetic stripe transaction) in the transaction request. For contact magnetic stripe transactions, the POS entry mode is set to 02 (Magnetic stripe) in the request. For contact magnetic stripe transactions, the actual value of the POS entry mode sent from the terminal may be different (e.g., 80, 90, etc.), but it is translated to 02 in the BASE24 system.

The various ways in which contactless cards can be verified are depicted in the following illustration.

Note: Only dynamic card verification of the DCVD is described in this section. Refer to the “Card Verification” section in the **BASE24 Transaction Security Manual** for detailed descriptions of CVD and iCVD verification processing support.

Visa contactless MSD with CVN 17

Visa also has a solution that combines the EMV and magnetic stripe standards for contactless transaction processing. This solution is generally referred to as “MSD with CVN 17”. As in the magnetic stripe standard, the POS entry mode is set to 91, but the DCVD is not included in the request. Instead, the request includes an ARQC, as in the EMV standard. TLV data is included in the request to allow BASE24-pos to perform ARQC verification.



MasterCard PayPass contactless magnetic stripe requirements

MasterCard's PayPass implementation for contactless magnetic stripe cards has a number of options that can be modified during personalization of the contactless chip. This controls the information sent from the terminal in the issuer discretionary data (IDD) portion of the Track 1 or Track 2 data fields. The following three data elements can be present in the IDD portion of the track data:

Application transaction counter (ATC) — A counter kept on the ICC chip used to generate the RFID signal. This counter is increased by one for each transaction performed using the card. A similar field is maintained by the issuer. The ATC is stored on the card and in the Cardholder Authorization File (CAF) or Dynamic Cardholder Authorization File (CAFD) as 2 binary bytes. However, in contactless magnetic stripe transactions, only the least significant digits of the ATC are sent in the track data. These are used to check whether the ATC value on the card is in synchronization with the ATC value maintained in the database. The entire ATC value is used in calculating the dynamic CVC3 and is sent to the Transaction Security Services process for CVC3 verification. The length and position of this field in the Track 1 or Track 2 data is defined in the ICC personalization data and the BASE24 Card Prefix File (CPF) and can be different for Track 1 and Track 2.

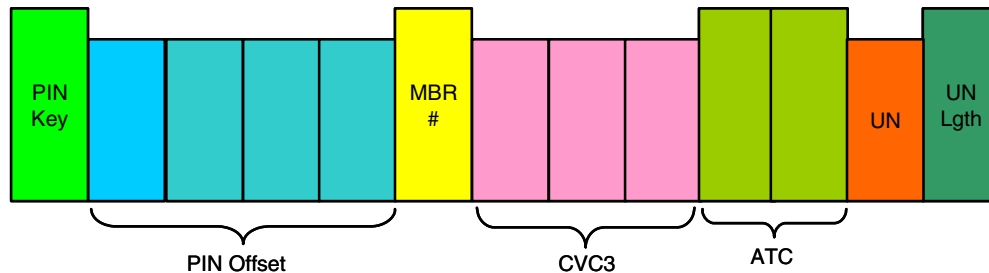
Note: The fewer the number of digits used in ATC checking, the higher the risk exists for the ATC on the card to be considered out of synchronization with the ATC maintained in the CAF or CAFD.

Dynamic CVC3 — A value calculated by the ICC chip on the card using the values in the ATC and the unpredictable number (UN) and other data elements. The least significant 3, 4 or 5 bytes of the CVC3 value are sent as part of the track data. The length and position of this field in the Track 1 or Track 2 data is defined in the ICC personalization data and the CPF. It can be different for Track 1 and Track 2.

Unpredictable number (UN) — A random number generated by the terminal and sent to the card to be used in calculating the dynamic CVC3 value. Zero to eight bytes of data can be included in the track data. The length of the unpredictable number is always provided in the last byte of the IDD. If the length is zero, it means that an unpredictable number was not used in the calculation of the dynamic CVC3. Thus, this value is always zero if the card is using a static CVC3 value. Issuers using a dynamic CVC3, should plan to issue cards that include an unpredictable number of at least one digit. This facilitates the determination of the type of CVC3 calculation (static or dynamic) to perform from the length of the unpredictable number. The position of the unpredictable number in Track1 or Track 2 data is defined in the CPF and can be different for Track 1 and Track 2.

Sample Track 1 and 2 IDD format

The following illustration depicts a sample layout of issuer discretionary data (IDD) for MasterCard PayPass cards. Each of the IDD elements in this sample are described following the illustration.



A total of 13 bytes are available in the issuer discretionary data (IDD). Some IDD elements are optional while some are required for PayPass.

The PAN has a maximum length of 16 bytes.

Optional discretionary data elements include the following, which can be provided in any order as specified by the offsets on CPF screen 3. The order shown here is just an example.

- PIN key indicator in byte 1 (Optional)
- PIN offset in bytes 2–5 (Optional)
- Member code (also known as the card sequence number) in byte 6 (Optional)

PayPass required IDD elements include the following:

- CVC3 in bytes 7–9 (Required)
- ATC in bytes 10–11 (Required)

ACI recommends that you maintain the greatest number of ATC digits possible in the track data. This reduces the risk of having the ATC stored on the track to be considered out of synchronization with the ATC stored in the CAF or CAFD during ATC checking.

- Unpredictable number in byte 12 (Required)

The unpredictable number is provided by the terminal and the least significant digits of the unpredictable number is sent in track IDD.

- Unpredictable number length in byte 13 (Required)

Note: The unpredictable number must always be in the last byte of IDD. There is no offset for this data element in the CPF.

For static CVC3 implementations, the value in this field must be 0 (zero). For dynamic CVC3 implementations, ACI recommends this value be greater than or equal to 1.

Visa payWave contactless magnetic stripe requirements

Visa's payWave contactless magnetic stripe implementation requires the following for issuer discretionary data (IDD) sent to calculate the dCVV:

- The dCVV is always 3 bytes in length.
- The Track 2 ATC is always sent as 4 bytes of numeric (not binary) data. For Track 1 data, the ATC can be split with 2 bytes before and 2 bytes after the 3 bytes that contain the dCVV. Because the largest value that can be sent in the IDD is 9999, the most significant digit for ATC values of 10,000–65,535 is not included in the IDD.

Note: It is assumed that the entire ATC in the card's magnetic stripe emulation data and the entire ATC in the Cardholder Authorization File (CAF) or Dynamic Cardholder Authorization File (CAFD) is used in the dCVV calculation. However, when checking the ATC value from the transaction against the value stored in the CAF or CAFD, if the value in the transaction is over 9999, BASE24-pos removes the most significant digit from the decimal representation of the value stored in the CAF or CAFD (e.g., subtracts 10,000 or 20,000, etc.) before comparing the two values.

Visa's payWave implementation for contactless magnetic stripe cards has a number of options that can be modified during personalization of the contactless chip. This controls the information sent from the terminal in the issuer discretionary data portion of the Track 1 or Track 2 data fields. The following three data elements are present in the issuer discretionary data portion of the track data:

Application transaction counter (ATC) — A counter kept on the ICC chip used to generate the RFID signal. This counter is increased by one for each transaction performed using the card. A similar field is maintained by the issuer. The ATC is stored on the card and in the CAF or CAFD as 2 binary bytes. Because only the four least significant digits of the ATC are sent in Track 1, the first two bytes of the ATC can be separated from the last two bytes of the ATC by the three bytes of the dCVV. In Track 2, all four bytes of the ATC are contiguous. The length and position of this field in the Track 1 or Track 2 data is defined in the ICC personalization data and the CPF and can be different for Track 1 and Track 2.

Note: The ATC value sent in the IDD and the ATC value maintained by the issuer can be different for Visa payWave cards (e.g., if the actual ATC value is between 10,000 and 65,535 inclusive). This is because only four digits of the ATC value can be carried in the IDD. If the ATC value is 10,000 or above, the most significant digit is not sent in the IDD.

Contactless indicator (CI) — A value greater than zero that indicates a contactless transaction. If the CI is not used, it defaults to a value of 1. The length and position of this field in the Track 1 or Track 2 data is defined in the ICC personalization data, but is not defined in the CPF. If present in the track discretionary data sent to the BASE24 system, you must account for its position in the offsets of any data that follows it in the track data. The CI is not currently used in BASE24 processing.

Dynamic CVV (dCVV) — A value calculated by the ICC chip on the card using the value in the ATC (and other data elements). The dCVV value is 3 bytes and must be sent as part of the track data. The length and position of this field in the Track 1 or Track 2 data is defined in the ICC personalization data and the CPF.

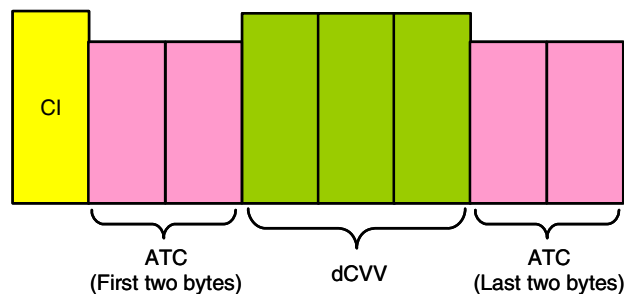
Defaults. If an incorrectly personalized card does not include either dCVV, ATC or Contactless Indicator in Track 2 Equivalent Data, the contactless interface reader builds track data using the following default values: 000 for the dCVV, 0000 for the ATC, and 1 for the CI.

Sample Track 1 and 2 IDD format

The following illustrations depict sample layouts of Track 1 and Track 2 issuer discretionary data (IDD) for Visa payWave cards. Each of the discretionary data elements in these samples are described following the illustrations.

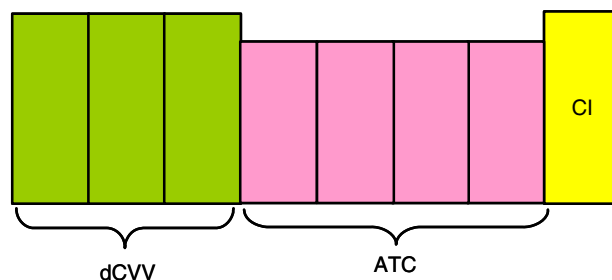
A total of 8 bytes are used in discretionary data for Track 1. The most significant position of the ATC is omitted. All data elements are required for payWave.

Visa Wave Track 1



A variable number of bytes are available in discretionary data for Track 2. The first eight digits are required for payWave and must contain the dCVV, ATC, and Contactless Indicator in that order. If the PVV is included in discretionary data, it must follow the eight bytes of contactless magnetic stripe data.

Visa Wave Track 2



payWave required discretionary data elements include the following:

- Contactless indicator in byte 1 for Track 1 and in byte 8 for Track 2
- ATC in bytes 2–3 and 7–8 for Track 1 and in bytes 4–7 for Track 2
In Track 1, bytes 1 and 2 of the ATC are separated from bytes 3 and 4 by the dCVV. In addition, byte 4 of the ATC is omitted.
- dCVV in bytes 4–6 for Track 1 and in bytes 1–3 for Track 2

Processing inputs

The Transaction Security Services process uses the following inputs for DCVD verification in addition to those used for contact magnetic stripe-based card verification (i.e., PAN, card expiration date, and service code). All of this data is read from the Track 1 or Track 2 IDD sent in the transaction request.

- PAN sequence number (MasterCard and Visa)
- Application transaction counter (ATC) (MasterCard and Visa)
- Unpredictable number (MasterCard only)

The PAN sequence number is a two-digit number used to differentiate cards with the same PAN. If this value is not provided in the transaction, a value of 00 is used.

The ATC is a dynamic value from 0 to 65,535 and is used to generate the DCVD. The microchip on the proximity card maintains the ATC and increments it by one for each transaction performed by the card application. You can elect to check the value of the ATC on the card with an ATC value maintained for the card in the CAF or CAFD. If the value on the card is not greater than the ATC value maintained in the CAF or CAFD, the transaction could be a fraudulent *replayed* transaction (i.e., the same transaction request data sent again). The Router/Authorization module updates the ATC value in the CAF or CAFD to match the ATC value sent from the card if the DCVD is successfully verified.

The unpredictable number is a value generated by the acquiring terminal and is used to provide variability and uniqueness to the generation of a cryptogram.

The service code is always positioned in the first position following the expiration date in Track 1 or 2 of the track data for dynamic card verification. The TRACK1 SRVC CODE OFST and TRACK2 SRVC CODE OFST fields on CPF screen 2 do not apply to the position of the service code in the track data for DCVD verification.

Processing

BASE24-pos dynamic card verification provides two types of verification processing—ATC checking and DCVD checking. If ATC checking is performed, it is always performed prior to checking the DCVD. Dynamic card verification is only performed when the PSTM.PT-SRV-ENTRY-MDE field is set to 91 (Magnetic Stripe Contactless).

Note: When the PSTM.PT-SRV-ENTRY-MDE field is set to 07 (Integrated Chip Card Contactless), card verification of the iCVD and EMV CAM processing is performed instead of dynamic card verification.

ATC checking

The ATC CHECK fields on CPF screens 3 and 11 control whether ATC checking is performed for contactless cards. The value on CPF screen 3 controls ATC checking for contactless magnetic stripe transactions while the value on CPF screen 11 controls ATC checking for contactless EMV transactions.

When application transaction counter (ATC) checking is performed, the Router/Authorization module compares the value of the ATC provided in the transaction to the value of an ATC maintained in the CAF or CAFD. If the CAFD exists, the ATC from the transaction is checked against the ATC in the CAFD. Otherwise, the ATC from the transaction is checked against the ATC in the CAF. The card increases the ATC on the card by 1 before each transaction request is sent for authorization. Therefore, the ATC value in the transaction must be greater than the ATC value maintained in the CAF or CAFD. If not, the action specified in the BAD ATC ACTION field on CPF screen 3 is taken. ATC checking prevents fraudulent *replayed* transactions where the same transaction request data is sent again for authorization. At the end of successful verification of the DCVD for a contactless card transaction, the Router/Authorization module sets the ATC value in the CAF or CAFD to the ATC value sent from the card.

Note: ACI recommends that the ATC be a minimum of 4 digits in length. The fewer the number of digits are used in ATC checking, the higher the risk exists for the ATC on the card to be out of synchronization with the ATC maintained in the CAF or CAFD.

Depending on your card issuer requirements, you can maintain a separate ATC for contactless magnetic stripe and contactless EMV transactions or use a single ATC for both contactless magnetic stripe and contactless EMV transactions. In addition, you can perform ATC checking only for contactless magnetic stripe transactions, only for contactless EMV transactions, or for both. ATCs are maintained in the base or EMV (contact EMV) segment of the CAF or in the CAFD.

For each ATC maintained in the CAF or CAFD, a *second card* ATC is also maintained in the CAF or CAFD. The first ATC value, referred to as the *first card* ATC, is used for checking an active card, while the second card ATC value is used for checking a newly issued card that is not yet active. Because the second card ATC is always initialized to 0000, it allows the new card to become activated with a new ATC without declining the transaction. Once a card is activated, the value of the second card ATC can be moved to the first card ATC and the second card ATC reset to 0000.

To avoid declining contactless magnetic stripe transactions for Visa cards when the ATC in the transaction has least significant digits of 00–05, the Router/Authorization module allows them to be compared to the least significant digits of 95–99 for a first card ATC value in the base segment of the CAF or in the CAFD and still pass the verification check. This allows the counter to roll over from 99 to 100, 999 to 1000 and 9999 to 10000, depending on the number of digits in the transaction. The ATC CHECK field on CPF screen 3 must be set to a value of 2 or 3 for this ATC checking logic to be used.

DCVD Checking

For checking the DCVD in the transaction, different algorithms are used for contactless MasterCard and Visa cards. The value of the DCV CHECK field on CPF screen 3 determines whether DCVD checking is performed and whether the MasterCard or Visa algorithm is used. The following steps describe how BASE24-pos checks the DCVD.

1. Before checking the DCVD, the Router/Authorization module checks the Card Verify Flag field in the BASE24-pos Release 5.0 token (PS50-TKN.CRD-VRFY-FLG) as follows to determine whether the DCVD has already been verified for the transaction:
 - If the Card Verify Flag is set to Y, DCVD checking was completed and dynamic card verification was successful. No further dynamic card verification processing is performed.
 - If the Card Verify Flag is set to C, DCVD checking was completed and the DCVD digits were invalid. In this case, the module takes the action specified in the BAD DCV ACTION field on CPF screen 3.
2. If DCVD checking is to be performed, the module collects the value from the DYN CARD VERIF KEY LOCATOR field on CPF screen 3 and the following information from the PSTM.TRACK2 field:
 - Card expiration date in YYMM format
 - DCVD
 - PAN
 - PAN sequence number if provided in the transaction. If not, 00 is used.
 - ATC
 - Service code
 - Card expiration date in original format provided in the transaction.
3. If the DCV CHECK field on CPF screen 3 is set to a value of 1 (CVC3 ENABLED) or 2 (CVC3/ATC MAY BE 0), the module also collects the following data from the PSTM.TRACK2 field:
 - Unpredictable number
 - Track static data length

Note: If the track static data length is odd and you are using an NSP, the Transaction Security Services process pads the track data with an “F” character and adds 1 to the track length. This ensures that an even number of track characters are sent in the 359 command to an NSP.

 - Track static data

- Track IVCVC3 (Issuer proprietary static data that is only used by Thales e-Security HSMs. If this item is not provided in the transaction data, the Transaction Security Services process calculates it.)
4. If any of the data required to verify the DCVD is not present in the transaction or is in an invalid format, the module takes the action specified in the BAD DCV ACTION field on CPF screen 3. Otherwise, the module formats a message and sends it to the Transaction Security Services process.
 5. The Transaction Security Services process uses the key locator value in the TSS message to retrieve the double-length dynamic card verification key and algorithm (Visa or MasterCard) from the DYN_CARD_VRFCTN. It then formats and sends the appropriate command to the hardware security device.

For Atalla AKB and variant NSPs, one of the following commands is sent:

 - 357 (Visa dCVV Verify)
 - 359 (MasterCard CVC3 Verify)

For Thales e-Security HSMs, the PM (Verify Dynamic CVV) command is sent. This command is used for both Visa dCVV and MasterCard CVC3 verification.
 6. The security device checks the DCVD passed in the message against the DCVD calculated using the Visa or MasterCard algorithm.

For the Visa algorithm, the security device uses the PAN, the ATC, card expiration date, service code, and the two dynamic card verification keys as input to the algorithm. The leftmost four digits of the PAN are replaced with the ATC to create an altered PAN that is input to the algorithm along with the expiration date and service code.

For the MasterCard algorithm, the security device uses the unpredictable number and an initialization vector (IVCVC3) in addition to the same input as defined above for the Visa algorithm. The IVCVC3 is only used with Thales e-Security HSMs.

For both algorithms, the calculated DCVD is compared to the DCVD from the transaction. If the two values match, the card is verified. Refer to the Visa and MasterCard vendor documentation for detailed descriptions of the calculations performed for these algorithms.
 7. If the response from the Transaction Security Services process indicates that dynamic card verification failed, the Router/Authorization module takes the action specified in the BAD DCV ACTION field on CPF screen 3. Otherwise, authorization processing continues.

Configuration

Dynamic card verification controls are configured in the BASE24 and TSS databases as described below.

BASE24 configuration

Configuration settings for dynamic card verification are configured in the Institution Definition File (IDF) and Card Prefix File (CPF).

Institution Definition File (IDF)

IDF Screen 1 indicates the file assigns for the Cardholder Authorization File (CAF) and the optional Dynamic Cardholder Authorization File (CAFD). The CAFD maintains just the ATC values and other dynamic data for a card and is not overwritten (recreated from an input file) during a full-file refresh of the CAF. Thus, by using the CAFD, you can retain values that would normally be reset to default values during a full-file refresh of the CAF. Unlike the CAF, the CAFD is not a segmented file and has relatively small records. Each CAFD record can maintain the first and second card ATC values for contactless magnetic stripe cards. Additionally, each CAFD record has a 20-byte customer user field that you can use to maintain data for ATC management exceptions or for other purposes.

During ATC checking for contactless magnetic stripe cards, the Router/Authorization module uses the ATC from the CAFD if the CAFD exists. Otherwise, it uses the ATC from the CAF.

Card Prefix File (CPF)

The following settings are needed in the Card Prefix File (CPF) for each card prefix using dynamic card verification.

Fields on CPF screen 3 specify dynamic card verification settings that apply only to contactless magnetic stripe cards. One field on CPF screen 11, which specifies card verification settings for EMV cards only, is used to control ATC checking for contactless cards that also contain an EMV application (i.e., the EMV segment is present in the CPF).

Refer to the **BASE24 Base Files Maintenance Manual** for an illustration of CPF screen 3 and the **BASE24-pos EMV Support Manual** for an illustration of CPF screen 11.

CPF screen 3

CPF screen 3 contains dynamic card verification information as well as BASE24 expiration date checking and service code checking controls.

Dynamic Card Verification Information

The following fields define dynamic card verification processing for contactless chip cards with this prefix. These controls only apply when transactions are acquired by a contactless magnetic stripe card reader.

DYN CARD VERIF KEY LOCATOR — The key locator for the Dynamic Card Verification data source (DYN_CARD_VRFCTN) record associated with this card prefix.

DCV CHECK — A code indicating the type of dynamic card verification to be performed for this card prefix. The default value of 0 indicates that dynamic card verification is disabled for the card prefix. Set this field to a value of 1 or 2 for dynamic card verification of the CVC3 value for MasterCard PayPass cards. Set this field to a value of 5 or 6 for dynamic card verification of the dCVV value for Visa payWave cards. A value of 2 indicates that processing

continues even if CVC3 checking fails because the ATC in the CAF base segment is set to 0 (e.g., after a CAF full file refresh) or the ATC in the CAFD is set to 0. A value of 5 indicates that the ATC is located in contiguous positions in the track data. A value of 6 indicates that the first two positions of the ATC are split from the last two positions of the ATC by the three-position DCVD in Track 1 data. If dynamic card verification data is carried in Track 2 for Visa payWave cards, a value of 6 in this field has the same meaning as a value of 5. If the DCVD is checked and is invalid, the action defined in the BAD DCV ACTION field is taken.

TRK1

The following fields define the location and length of dynamic card verification information in Track 1 for this card prefix.

DCVD OFST — A value defining the first position following the name delimiter of the dynamic card verification digits (DCVD) on Track 1 of the card. The name delimiter follows the variable-length name field. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 1 is not used.

DCVD LEN — The length of the DCVD on Track 1 of the card. Valid values are 0 or 3–5, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 1 is not used.

ATC OFST — A value defining the first position following the name delimiter of the application transaction counter (ATC) on Track 1 of the card. The name delimiter follows the variable-length name field. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 1 is not used.

ATC LEN — The length of the ATC on Track 1 of the card. Valid values are 0 through 9, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 1 is not used.

Note: ACI recommends that the ATC be a minimum of 4 digits in length. The fewer the number of digits used in ATC checking, the higher the risk exists for the ATC on the card to be out of synchronization with the ATC maintained in the CAF or CAFD.

UNPRED NUM OFST — A value defining the first position following the name delimiter of the unpredictable number on Track 1 of the card. The name delimiter follows the variable-length name field. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification), 5 (perform dCVV dynamic card verification), 6 (perform dCVV dynamic card verification, split ATC) or Track 1 is not used.

TRK2

The following fields define the location and length of dynamic card verification information in Track 2 for this card prefix.

DCVD OFST — A value defining the first position following the field separator of the dynamic card verification digits (DCVD) on Track 2 of the card. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 2 is not used.

DCVD LEN — The length of the DCVD on Track 2 of the card. Valid values are 0 or 3–5, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 2 is not used.

ATC OFST — A value defining the first position following the field separator of the application transaction counter (ATC) on Track 2 of the card. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 2 is not used.

ATC LEN — The length of the ATC on Track 2 of the card. Valid values are 0 through 9, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification) or Track 2 is not used.

Note: ACI recommends that the ATC be a minimum of 4 digits in length. The fewer the number of digits are used in ATC checking, the higher the risk exists for the ATC on the card to be out of synchronization with the ATC maintained in the CAF or CAFD.

UNPRED NUM OFST — A value defining the first position following the field separator of the unpredictable number on Track 2 of the card. Valid values are 0 through 99, however, 0 is valid only when the value in the DCV CHECK field is 0 (do not perform dynamic card verification), 5 (perform dCVV dynamic card verification), 6 (perform dCVV dynamic card verification, split ATC) or Track 2 is not used.

The following fields on CPF screen 3 are not related to a specific track.

ATC CHECK — A code indicating whether application transaction counter (ATC) checking is performed for this card prefix, and if so, the conditions under which it is performed. To perform ATC checking for contactless magnetic stripe transactions, you must set this field to a value of 2 or 3. Setting this field to a nonzero value causes the ATC fields maintained in the base segment of the CAF or in the CAFD to be updated.

You should set this field in coordination with the ATC CHECK TYPE field on CPF screen 11 and the CAP ATC UPDATE field on CPF screen 13 and take into account whether you want to maintain separate ATCs for contactless magnetic stripe transactions, EMV transactions, and CAP token validation transactions, a single ATC for all three types of transactions, a single ATC for just contactless magnetic stripe transactions, a single ATC for both EMV and CAP token validation transactions, a single ATC for just EMV transactions, a single ATC for just CAP token validation transactions, or no ATCs at all.

Set this field to 0 if you do not want to check the ATC for contactless magnetic stripe transactions.

Set this field to 1 if you do not want to check the ATC for contactless magnetic stripe transactions, but you do want to perform ATC checking for EMV transactions. Use this value only if there is one ATC on the card that is used for both contactless magnetic stripe transactions and EMV transactions.

Set this field to 2 if you want to perform ATC checking for contactless magnetic stripe transactions and maintain an ATC value in the base segment of the CAF or in the CAFD for contactless magnetic stripe transactions only. Use this value if your cards have one ATC for contactless magnetic stripe transactions and a separate ATC for EMV transactions. The ATC CHECK TYPE field on CPF screen 11 must be set to 1 for the EMV ATC to be maintained in the EMV segment of the CAF or in the CAFD.

Set this field to 3 if you want to perform ATC checking for contactless magnetic stripe transactions and maintain a single ATC value in the base segment of the CAF or in the CAFD for both contactless magnetic stripe and EMV transactions. Use this value if your cards have one ATC counter that is used for both contactless magnetic stripe transactions and EMV transactions. The ATC CHECK TYPE field on CPF screen 11 must be set to 8 or 9 for a single ATC value to be maintained in the base segment of the CAF or in the CAFD.

For more information on how the various combinations of values for these fields impact where the ATC is maintained in the BASE24 database, see the [“Application Transaction Counter Checking”](#) topic in section 17 and the **BASE24-pos EMV Support Manual**.

CHECK IF HOST ONLINE

The following two fields indicate whether dynamic card verification (DCV) or application transaction counter (ATC) prescreening checks are performed when transactions are routed to the host for authorization.

DCV — A flag indicating whether the Router/Authorization module prescreens the DCVD before sending a transaction to the host for authorization. Set this field to a value of 1 to prescreen the DCVD. If the DCVD is invalid, the action defined in the BAD DCV ACTION field is taken.

ATC — A flag indicating whether the Router/Authorization module prescreens the ATC before sending a transaction to the host for authorization. Set this field to a value of 1 to prescreen the ATC. If the ATC is invalid, the action defined in the BAD ATC ACTION field is taken.

BAD ATC ACTION — A code indicating the action taken by the Router/Authorization module when an ATC check fails. You can select to note the situation and continue processing, deny and return the card, deny and retain the card, or refer the transaction (BASE24-pos only).

BAD DCV ACTION — A code indicating the action taken by the Router/Authorization module when a DCVD check fails. You can select to note the situation and continue processing, deny and return the card, deny and retain the card, or refer the transaction (BASE24-pos only).

TSS configuration

To support dynamic card verification, you must configure the algorithm (MasterCard or Visa) and dynamic card verification keys on the Contactless tab of the Authentication Profile Configuration window for each card prefix using dynamic card verification. This record must correspond to the key locator value set in the DYN CARD VERIF KEY LOCATOR field on CPF screen 3.

All of this information entered on the Authentication Profile Configuration window is stored in the Dynamic Card Verification data source (DYN_CARD_VRFCTN). A sample of the Contactless tab on the Authentication Profile Configuration window is shown in [section 2](#).

14: Database conversion

To allow existing BASE24 customers to use the Transaction Security Services database files, the existing BASE24 files are converted and the hardware encrypted key data is moved to the new database. The hardware encrypted key fields in existing BASE24 database files are either set to the key value for the record (e.g., the terminal ID for hardware key fields in the BASE24-atm Terminal Data files and BASE24-pos Terminal Data files), an index value created by the conversion program, or are set to binary zeros (e.g., for the Key Authorization File).

TSS database conversion programs also use a runtime parameter called LN-IND to create a key locator value when converting existing BASE24 files. This parameter contains a two-character hexadecimal identifier for a logical network, which allows the Channel_Security and ICHG_SECURITY to be physically partitioned by logical network.

When a database conversion program is run on a BASE24-atm Terminal Data file or BASE24-pos Terminal Data file, the two-character LN-IND value is added to the beginning of the 1–16 character terminal ID. If the terminal ID for the terminal record being converted has more than 14 characters, the last one or two characters are truncated when the key locator value is created.

When converting Key File (KEYF) or Key 6 File (KEY6) records to the TSS database, the two-character LN-IND runtime param value is added to the beginning of a 14-character key locator index value generated by the conversion program.

If you need to partition the TSS database by logical network, the value of the LN-IND runtime parameter must be set to the same value specified in the LN-IND LCONF param. For more information on the LN-IND LCONF param, refer to [section 3](#).

If you do not need to partition the TSS database by logical network, the LN-IND runtime parameter must be set to a value of 00.

When set to the key value for the record or an index value, this value is referred to as the *key locator* value. The key locator is then used as the primary key value in the new database files, which contain the actual hardware key field values. For fields set to binary zeros, the primary key for the existing BASE24 file record is copied to the new database and is used as

the key locator value. The following table shows the relationship of existing BASE24 database files to the new database files. The first column lists the existing BASE24 files and the second column lists the new files accessed by the Transaction Security Services process.

BASE24 file	TSS file
BASE24-atm Terminal Data Dynamic File—general data (ATDD1)	Channel Security File (Channel_Security)
Terminal Data File (TDF)	
BASE24-pos Terminal Data Dynamic File—general data (PTDD1)	
POS Terminal Data File (PTDF)	
Derivation Key File (KEYD)	Derivation Key File (Derivation_Key)
Key Authorization File (KEYA)	Card Verification File (CARD_VRFCTN)
	Diebold Number Table File (DIEBOLD_NUM_TBL)
	IBM DES Verification File (IBM_DES)
	Identikey Verification File (Identikey)
	Visa PVV Verification File (Visa_PVV)
ICC Key File (KEYI)	EMV Authorization File (EMV_Security)
Key File (KEYF)	Interface Security File (ICHG_SECURITY)
Key 6 File (KEY6)	

ACI provides a complete set of utility programs to convert the hardware key fields in existing BASE24 records and add corresponding records to the new database files. Records in the KEYA are converted to one of four new database files, depending on the record type. If the KEY6 is converted, a new record is created in the KEYF for each record converted from the KEY6. After the KEY6 is converted, it is no longer used in processing.

ACI also provides an additional utility program to convert the keys in the new database to the new Atalla key block format, if necessary.

The conversion programs provided by ACI are listed on the following page. For more information on using these conversion programs, refer to the **BASE24 Release 6.0 Implementation Guide**.

Database conversion programs

ACI provides a conversion program for each BASE24 file that contains hardware encrypted key information. These programs are run using obey files. The following table lists the obey files and programs provided by ACI. These files are located on the AT60UC05, BA60UC05, or PS60UC05 subvolumes.

BASE24 file	Obey file	Conversion program
BASE24-atm Terminal Data Dynamic File—general data (ATDD1)	TSSATDR	TSSATD
Terminal Data File (TDF)	TSSTDFR	TSSTDF
BASE24-pos Terminal Data Dynamic File—general data (PTDD1)	TSSPTDR	TSSPTD
Derivation Key File (KEYD)	TSSKEYDR	TSSKEYD
POS Terminal Data File (PTDF)	TSSPTDFR	TSSPTDF
Key Authorization File (KEYA)	TSSKEYAR	TSSKEYA
Key File (KEYF)	TSSKEYFR	TSSKEYF
Key 6 File (KEY6)	TSSKEY6R	TSSKEY6
ICC Key File (KEYI)	TSSKEYIR	TSSKEYI

For detailed procedures on running these conversion programs, refer to the **BASE24 Release 6.0 Implementation Guide** and the **BASE24 Classic 6.0 Installation Guide**.

Note: The TSSATDR obey file creates an entry-sequenced file named TRMF that is simply an output listing of ATMs that are configured for EMV support.

After the existing BASE24 files are converted, all security data maintenance is performed through the ACI desktop user interface, except for certain situations described in the note below. User access to the following BASE24 files maintenance screens must be turned off in the Security File (SEC): KEYA, KEYI, KEYF, and KEY6. Changes to non-hardware encrypted key information in the KEYF are made using the BASE24 KEYF Configuration window accessed from the ACI desktop user interface.

Note: If an interchange encrypts PINs in hardware and the BASE24 system processes PINs internally in the clear, or if an interchange uses clear PINs while the BASE24 system uses PINs encrypted in software, an intermediate key must be configured in the KEYF using the

Pathway-based Key File (KEYF) screens. The ACI desktop user interface does not grant access to intermediate keys. In this case, user access to the KEYF screens must be retained in the SEC.

15: Public key cryptography

This section describes the security-related configuration and implementation requirements for the functions that use public key cryptography:

- BASE24 Automated Key Distribution System (AKDS) add-on product
- Secure logons and logoffs to and from an interchange*

*This function is only currently available in the SPAN2 (SAMA) ISO Interface process.

This section describes the public key cryptography keys and concepts used for the above functions, how these functions are configured in the BASE24 system, the out-of-band and online procedures for implementing these functions, the windows and files used for public key cryptography processing, and the AKDS Capacity Report, which is used with the BASE24 Automated Key Distribution System (AKDS) add-on product only.

Note: For the BASE24 Automated Key Distribution System (AKDS) add-on product, you may have to increase the values of the RECVLEN and XMITLEN XPNET attributes for an ATM device when implementing AKDS to accommodate the larger messages required for AKDS. Refer to the **BASE24 AKDS Implementation Guide** for more information.

Introduction

Public key cryptography, also referred to as asymmetric cryptography or public key infrastructure (PKI), differs from symmetric cryptography, such as DES and triple DES, in that key pairs instead of shared keys are used between the authenticating entities. A unique key pair, consisting of a public key and a private key, is created for each authenticating entity.¹ The two parties then exchange their public keys in a secure manner. The sender encrypts the data using the receiver's public key. The receiver decrypts the data using its private key. The sender and receiver need not share the same key. Thus, public key cryptography is referred to as an *asymmetrical* security scheme as opposed to the *symmetrical* security scheme of DES, where the same keys must be known by both the sender and receiver of a message. PKI provides the following advantages over symmetrical security schemes:

- The receiver of a secured message uses a public key for verifying a message rather than the private (i.e., secret) key that the sender uses to secure the message. This lowers the need for expensive hardware devices at the receiver's site to store shared private keys.

- Each sender requires only one public and private key pair for secure communication with n number of receivers. In symmetric security schemes, the sender would require n number of different keys to secure the communication with n number of receivers. This lowers the complexity (costs) of the security architecture (key management).
- The sender and the receiver do not need to share a private key before actually exchanging secure messages when using public key certificates. This increases the flexibility for unrelated parties to securely exchange information.

¹ For some key loading schemes, such as NCR's enhanced signature remote key loading (RKL) authentication scheme, two unique key pairs are created.

For the BASE24 Automated Key Distribution System add-on product, the authenticating entities are individual ATM terminals and the BASE24 system. For secure logons and logoffs to and from an interchange, the authenticating entities are the interchange and the BASE24 system. The ATM's or interchange's private key, also known as the secret key, is stored within a tamper-resistant module in the ATM terminal or interchange. The BASE24 private key is stored in the TSS database. The public keys are signed by a certificate authority's private key. The entities then authenticate each other and exchange their public keys, signed by the certificate authority's private key.

Signatures provide a method of preventing data from being changed during transmission and of preventing the sender from being impersonated. A secure hashing algorithm is applied to the data and then the result is encrypted. With Rivest, Shamir, and Adleman (RSA) encryption, the signature is generated using the private key. Only the holder of the private key can generate the signature but anybody can verify it using the associated public key.

After both authenticating entities have successfully exchanged public keys, the receiver can use its private key to securely decrypt data received. In this case, a new ATM master key can be generated by the BASE24 system and securely downloaded to an ATM, or an interchange can ensure that communications with a member bank are secure.

Some differences exist in the way Diebold, NCR, Tidel, Triton, and Wincor Nixdorf have implemented public key cryptography. For example, Diebold and Triton use certificates and a third-party certificate authority while NCR and Tidel do not, although Diebold firmware can also emulate NCR firmware using the basic signature scheme and NCR firmware can also emulate Diebold firmware using certificates. For Diebold firmware emulating NCR firmware, the AKDS implementation procedures are exactly the same as for an NCR device using the basic signature scheme. For NCR firmware emulating Diebold firmware, the AKDS implementation procedures are exactly the same as for a Diebold device. NCR supports a basic signature-based AKDS scheme and an enhanced signature-based AKDS scheme. The basic scheme uses a single key pair for the BASE24 system while the enhanced scheme uses two key pairs for the BASE24 system. Signature-based devices (NCR/Wincor) can run on the NCR 5XXX and Diebold 10XX/478X Device Handlers using the shared Wincor Nixdorf signature specification. The NCR enhanced scheme is only supported for NCR devices. Certificate-based (Diebold) devices can also be run from the NCR 5XXX and Diebold 10XX/478X Device Handlers using the shared Wincor Nixdorf certificate specification and use the same certificate-based PKI remote key loading procedures as Diebold devices. For all shared Wincor Nixdorf specifications, you cannot migrate from single-length keys to double-length keys due to limitations in Wincor Nixdorf firmware.

Note: Because Tidel and Triton ATMs are typically accessed using dial-up communications, Tidel and Triton Device Handler processes do not support the shared Wincor Nixdorf signature specification or shared Wincor Nixdorf certificate specification. Although Triton ATMs are typically implemented using dial-up communications, they also support TCP/IP communications over leased lines.

Public key cryptography for interchanges is supported using a signature-based scheme. The specific procedures for validating the RSA key pair generated for the BASE24 system by a certificate authority are beyond the scope of this manual. The means for generating the RSA key pair for the interchange is also beyond the scope of this manual.

NCR and Tidel devices

NCR creates the public and private keys for the encrypting PIN pad (EPP) in each NCR or Tidel ATM during manufacture. NCR also assigns a unique serial number to each EPP and creates a signature for the EPP's public key.

The NCR trusted center serves as a certificate authority that signs the public keys for both the EPP and the BASE24 system. The trusted center sends the root public key of the trusted center (through an offline method) to both ACI and its customers. The NCR trusted center refers to their root public and private keys as the *Global NCR public verification key* and the *Global NCR private signature key*, respectively.

The EPP's public key, private key, public key signature, serial number, serial number signature, and the NCR trusted center's public key are stored within the EPP.

NCR devices use NDC+ message formats while Tidel devices use Tidel's Remote Key Management (RKM) message formats for AKDS processing. For more information on RKM message formats, refer to Tidel's ***RKM Communication Specification***.

NCR supports both a basic and an enhanced signature-based AKDS scheme for remote key loading and authentication. Both of these schemes are described below.

Basic scheme

For NCR's basic signature-based AKDS scheme, a single key pair is generated for the BASE24 system. This key pair is referred to as the *root BASE24 system key pair*. For this scheme, the root BASE24 system public key is exchanged with the NCR device's EPP. This basic scheme is supported for NCR devices, Tidel devices, devices adhering to the shared Wincor Nixdorf signature specification, and interfaces.

Enhanced scheme



To support NCR's enhanced signature-based AKDS scheme, an additional key pair is required for the BASE24 system. This key pair is referred to as the *primary BASE24 system key pair* throughout this section. After the root BASE24 system public key has been signed by the certificate authority, the primary BASE24 system public key

is signed by the root BASE24 system private key. Both the primary and root BASE24 system public keys and their signatures are then exchanged with the NCR device's EPP rather than just the root BASE24 system public key and signature used for the NCR basic scheme.

This enhanced scheme provides for the segmentation of risk to address key compromise. It also allows for the primary BASE24 system key pair to be replaced periodically (e.g., every five years) without having to regenerate the root BASE24 system key pair and having it signed by the certificate authority.

Besides the additional RSA key pair, the enhanced signature scheme provides the ability to delete both the primary and root BASE24 public keys from the EPP using the DELETE PUBLIC KEY device control command (or DELPUBLIC text command). This command sends TSS message 59 (Delete Host Public Key Request) to the TSS process. For more information on these commands, refer to the **BASE24 Device Control Manual** and the **BASE24 Text Command Reference Manual**. Within the enhanced remote key loading scheme, the host keys that have been imported must be deleted before they can be replaced. In addition, to prevent unauthorized deletion of these keys, the public keys are only deleted if the request is accompanied by a signature of the EPP serial number concatenated with the public key to be deleted and the corresponding secret key

If the primary BASE24 system public key is deleted from the EPP, the root BASE24 system public key remains within the EPP and can be used to load another primary BASE24 system public key.

If the root BASE24 system public key is deleted from the EPP, then the primary BASE24 system public key is also automatically deleted from the EPP and the complete set of mutual authentication steps must be performed again.

The NCR enhanced scheme is only supported for NCR devices.

NCR firmware emulating Diebold firmware

When NCR APTRA Advance NDC and APTRA Edge environment NDC Business Services firmware is installed on a foreign vendor terminal (Diebold) with a certificate-based EPP, it must communicate with the BASE24 system using the NCR NDC+ communications protocol. In this case, AKDS is implemented in the TSS environment exactly like a native Diebold device using certificates. You must use a device type of Diebold when generating the RSA key pair for the BASE24 system using the KEYGEN command and on all public key cryptography windows. On these windows, the device type indicates the public key implementation in the EPP rather than the firmware vendor for the device.

For more information on NCR firmware emulating Diebold firmware, refer to NCR's **Support for Initial Key Loading Using Certificates in NDC** document. For convenience, NCR firmware emulating Diebold firmware is referred to as *NCR certificate emulation firmware* throughout this section.

Diebold devices

Diebold EPPs create two RSA key pairs when they are first powered up. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. The public keys for each of these two key pairs are signed by a certificate authority. Diebold has chosen Digital Signature Trust Co. (DST) as their certificate authority. You must use a device type of Diebold for Diebold devices when generating the RSA key pair for the BASE24 system using the KEYGEN command and on all public key cryptography windows.

For each public key created, the certificate authority creates a certificate. These certificates contain the EPP's public keys and signatures. DST creates two certificates for each public key created—a primary certificate and a secondary certificate. Primary and secondary certificates are features of Diebold's disaster recovery procedure, however, BASE24 uses the primary certificate only. The secondary certificate is discarded. The EPP's public key certificates, private signature generation key, private decipherment key, and two certificates (primary and secondary) containing the certificate authority's public signature verification key and signature, are stored in the EPP.

Diebold devices with native firmware use 912 message formats.

Triton devices

Triton EPPs create two RSA key pairs when they are first powered up. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. The public keys for each of these two key pairs are signed by a certificate authority. Triton acts as their own certificate authority using VeriSign as their service provider. You must use a device type of Triton for Triton devices when generating the RSA key pair for the BASE24 system using the KEYGEN command and on all public key cryptography windows.

For each public key created, the certificate authority creates a certificate. These certificates contain the EPP's public keys, signatures, and the EPP's unique identifier. Triton creates one certificate for each public key created. The EPP's public key certificates, private signature generation key, private decipherment key, and root certificate containing the certificate authority's public signature verification key and signature, are stored in the EPP.

Triton devices use Triton's RKT (remote key transport) message formats. Refer to Triton's **Remote Key Transport Host Developers' Guide** for detailed information on RKT messages.

Diebold firmware emulating NCR firmware

When Diebold Agilis 91x XV terminal application firmware is installed on a foreign vendor terminal (NCR) with a signature-based EPP, it must communicate with the BASE24 system using the Diebold 91x communications protocol. In this case, AKDS is implemented in the TSS environment exactly like a native NCR device using the basic signature scheme. You must use a device type of NCR when generating the RSA key pair for the BASE24 system

using the KEYGEN command and on all public key cryptography windows. On these windows, the device type indicates the public key implementation in the EPP rather than the firmware vendor for the device.

For more information on Diebold firmware emulating NCR firmware, refer to Diebold's **91x Message Protocol Extensions for NCR Based RSA Initial Key Loading**. For convenience, Diebold firmware emulating NCR firmware is referred to as *Diebold signature emulation firmware* throughout this section.

Devices supporting Wincor Nixdorf's shared signature specification firmware

Any device that is running firmware compliant with Wincor Nixdorf's shared signature specification, **Wincor Nixdorf NDC/Diebold D91x Message Format Extension for RKL**, and is equipped with a signature-based encrypting PIN pad (EPP), can support the BASE24 Automated Key Distribution System add-on product.

Note: Although Wincor Nixdorf signature firmware can be installed on any signature-based device, the AKDS implementation procedures are different when installed on NCR hardware (EPP). In this case, you must use the NCR out-of-band procedures to generate and sign the root BASE24 system public key (phases 1–3 of the “NCR 5XXX Terminal Implementation Procedures”) rather than the shared Wincor Nixdorf signature device out-of-band procedures (phases 1–3 of the “Shared Wincor Nixdorf Signature Terminal Implementation Procedures”). This is required because an NCR EPP uses only the modulus portion of the public key in transaction data and allows the exponent portion to be defaulted at the device, while the shared Wincor Nixdorf signature scheme uses an ASN.1 formatted public key that includes both the modulus and exponent.

For simplicity, Wincor Nixdorf is assumed to be the device vendor for non-NCR signature devices compliant with the **Wincor Nixdorf NDC/Diebold D91x Message Format Extension for RKL** specification. These devices are referred to as *shared Wincor Nixdorf signature devices* throughout this section while devices compliant with this specification that are installed on NCR hardware are referred to as *shared Wincor Nixdorf/NCR signature devices*.

For shared Wincor Nixdorf signature devices, Wincor Nixdorf creates the public and private keys for the encrypting PIN pad (EPP) in each ATM during manufacture. Wincor Nixdorf also assigns a unique serial number to each EPP and creates a signature for the EPP's public key. The Wincor Nixdorf Customer Key Center – International (CKCI) serves as a certificate authority that signs the public keys for both the EPP and the BASE24 system. CKCI sends a customer-specific Wincor Nixdorf public signature key (through an offline method) to its customers. This key is referred to as the *root Wincor Nixdorf public key* throughout this section. The EPP's public key, private key, public key signature, serial number, serial number signature, and the root Wincor Nixdorf public key are stored within the EPP.

Shared Wincor Nixdorf signature devices running in NCR NDC+ or Diebold 912 emulation mode implement signature-based public key cryptography remote key loading in the same manner as the NCR basic scheme, but with the following differences.

- The root BASE24 system public key is not self-signed when sent to the certificate authority.
- The Wincor Nixdorf Customer Key Center – International (CKCI) serves as the certificate authority that signs the public keys for both the EPP and the BASE24 system.
- CKCI creates customer-specific public and private signature keys.
- A device type of “Wincor Signature” is used with the KEYGEN command, and on the Certificate Management, Host Public Key Perusal, Channel Public Key Perusal, EPP Serial Number Configuration, and Disable KEYGEN windows for shared Wincor Nixdorf signature devices.
- You cannot migrate from single-length keys to double-length keys using the BASE24 Automated Key Distribution System due to limitations in Wincor Nixdorf firmware.

Devices supporting Wincor Nixdorf’s shared certificate specification firmware

A Diebold device that is running firmware compliant with Wincor Nixdorf’s shared certificate specification, **Wincor Nixdorf NDC/Diebold D91x Message Format Extension for RKL**, and is equipped with a certificate-based encrypting PIN pad (EPP), can support the BASE24 Automated Key Distribution System add-on product. Currently, Diebold is the only device vendor that can run the necessary firmware.

AKDS support for shared Wincor Nixdorf certificate devices is exactly the same as that provided for Diebold devices, except you cannot migrate from single-length keys to double-length keys using the BASE24 Automated Key Distribution System due to limitations in Wincor Nixdorf firmware. Use the Diebold implementation procedures provided in this section and the Diebold device type in the KEYGEN command and on ACI desktop windows to implement AKDS for shared Wincor Nixdorf certificate devices.

Wincor Nixdorf creates two RSA key pairs for the encrypting PIN pad (EPP) in each ATM during manufacture. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. Wincor Nixdorf also assigns a unique serial number to each EPP. Digital Signature Trust Co. (DST) serves as the certificate authority and creates certificates for each of these public keys.

For each public key created, the certificate authority creates two certificates—a primary certificate and a secondary certificate. These certificates contain the EPP’s public keys and signatures. Primary and secondary certificates are features of Diebold’s disaster recovery procedure, however, BASE24 uses the primary certificate only. The secondary certificate is discarded. The EPP’s public key certificates, private signature generation key, private decipherment key, and two certificates (primary and secondary) containing the certificate authority’s public signature verification key and signature, are stored in the EPP.

Interfaces

Besides the remote key loading of ATMs, public key cryptography also allows for secure logons and logoffs from a BASE24 Interchange Interface process to an interchange. For example, public key cryptography is supported between the SPAN2 (SAMA) ISO Interface process and the SAMA SPAN2 network. For other interchanges, check with your interchange vendor to determine whether the interchange supports signature-based asymmetric key cryptography and with ACI to determine whether PKI is supported in the corresponding BASE24 Interchange Interface process. The PKI used for interfaces is signature-based.

BASE24 system

The BASE24 system serves as the host in public key exchanges and authentication with ATM terminals or an interchange. Using the KEYGEN command entered from the KEYGEN Command Configuration and Submission window or the Process Control window in the ACI desktop and a hardware security module, one or two unique RSA key pairs is generated for the host. One key pair, referred to as the *root BASE24 system key pair*, is generated for all PKI implementations. A second key pair, referred to as the *primary BASE24 system key pair*, is generated for the NCR enhanced scheme only.

For NCR devices using the NCR basic scheme, Tidel devices, devices with Diebold firmware that emulates NCR firmware using the NCR basic scheme, devices with shared Wincor Nixdorf/NCR signature firmware, or interfaces, the root BASE24 system's public key, private key, and public key signature, as well as the certificate authority's public key are stored in the TSS database.

For NCR devices using the NCR enhanced scheme, the primary BASE24 system's public key and private key are stored in the TSS database in addition to all the keys listed above for the NCR basic scheme.

For Diebold devices, Triton devices, devices with NCR firmware that emulates Diebold firmware, or shared Wincor Nixdorf certificate devices, the root BASE24 system's primary public signature verification key certificate, private signature generation key, the root certificate authority's public verification key, and the primary certificate authority's public keys for signature verification and generation are stored in the TSS database.

For shared Wincor Nixdorf signature devices, the root BASE24 system's public key, private key, and public key signature, as well as the Wincor Nixdorf Customer Key Center – International (CKCI)'s public key are stored in the TSS database.

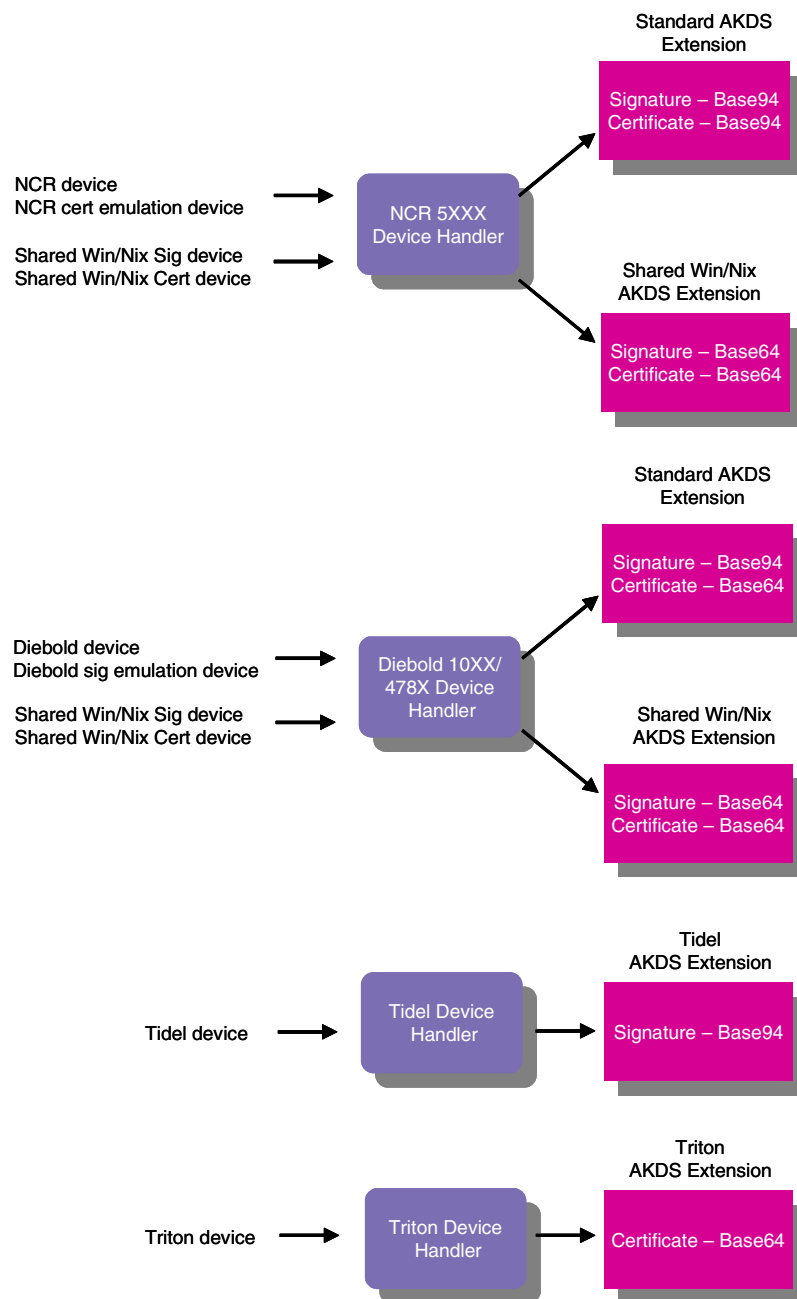
Device definitions

The following table describes the EPP hardware vendor, ATM firmware vendor, message format, Device Handler, and solution used for each of the ATM devices defined in this section.

ATM device	EPP hardware	Firmware	Message format	Device handler	Solution
NCR 5XXX	NCR signature	NCR	NDC+	N50	Native basic NCR Native enhanced NCR
	NCR signature	Diebold	912	D1000	Diebold signature emulation
	NCR signature	Wincor	NDC+	N50	Wincor signature for NCR
			912	D1000	
Diebold 10XX/478X	Diebold certificate	Diebold	912	D1000	Native Diebold
	Diebold certificate	NCR	NDC+	N50	NCR certificate emulation
	Diebold certificate	Wincor	NDC+	N50	Wincor certificate
			912	D1000	
Tidel	NCR signature	NCR	RKM	TIDL	Native basic NCR
Triton	Triton certificate	Triton	RKT	TRIT	Native Triton
Wincor Nixdorf	Wincor signature	Wincor	NDC+	N50	Wincor signature
			912	D1000	

Device handler extensions

AKDS support for the BASE24 Automated Key Distribution System is provided in extensions to the BASE24-atm Diebold 10XX/478X Device Handler, NCR 5XXX Device Handler, Tidel Device Handler, and Triton Device Handler modules. This support is depicted in the following illustration.



Terminology

A number of terms are unique to public key cryptography. These terms are described below.

A-Key — See master key.

Abstract Syntax Notation One (ASN.1) — A formal language used to abstractly describe messages to be exchanged between distributed computer systems. Previously, ASN.1 was used to write application, national and international standards. However more recently with the advent of ASN.1 software tools, ASN.1 has been used to generate programming language code that forms the core of a wide variety of messaging systems applications. ASN.1 encoding rules are sets of rules used to transform data specified in the ASN.1 language into a standard format that can be decoded on any system that has a decoder based on the same set of rules. Diebold, Triton, and shared Wincor Nixdorf certificate firmware use ASN.1. NCR, Tidel, Diebold signature emulation firmware, interfaces, and shared Wincor Nixdorf signature firmware use ASN.1 only when transmitting messages to the certificate authority (e.g., the NCR trusted center, the Wincor Nixdorf Customer Key Center – International, or third-party certificate authority).

Base64 Encoding — A method of encoding used by Diebold, Triton, and shared Wincor Nixdorf signature or certificate firmware to convert a sequence of octets (bytes) into printable ASCII characters. Base64 encoding allows RSA cryptographic fields to be transmitted in messages without exceeding Diebold's 1920 byte limit. Other encoding methods double or triple the size of the original data, while the Base64 encoding method results in data that is only about 33 percent larger than the original un-encoded data.

Native Diebold and shared Wincor Nixdorf signature or certificate firmware use Abstract Syntax Notation One (ASN.1) when transmitting cryptographic fields. This is a standard syntax for defining and encoding data for an open systems environment. The ASN.1 values and types are encoded using the Basic Encoding Rules (BER) and Distinguished Encoding Rules (DER).

ASN.1 produces binary BER and DER data that cannot be directly transmitted over 7-bit ASCII networks. The solution is to convert the binary data into 7-bit ASCII before transmission using Base64 encoding.

Diebold's implementation of the base64 encoding algorithm follows the standard set forth in RFC 1521, section 5.2 - Base64 Content-Transfer-Encoding (N. Borenstein, 1993).

Base94 Encoding — A method of encoding data used by NCR and Tidel firmware to convert binary data into the 94 displayable ASCII characters, ranging from hex 20 (space) to hex 7E (tilde). Using these characters, a base94 algorithm can produce a 5 to 4 ratio of encoded bytes to binary bytes. Given the block size (256 bytes for RSA-encrypted data) and message length constraints, NCR has determined that other encoding methods are unsuitable for transmitting RSA blocks.

Base94 encoding is used for NCR and Tidel firmware and Diebold firmware emulating NCR firmware.

Basic Encoding Rules (BER) — This set of encoding rules was created in the early 1980s and used in a wide range of applications, such as Simple Network Management Protocol (SNMP) for management of the Internet, Message Handling Services (MHS) for exchange of electronic mail and TSAPI for control of telephone/computer interactions. See also Abstract Syntax Notation One.

Certificate — An electronic document binding some pieces of information together, such as an ATM's or host's identity and public key. Diebold and Triton devices use certificates; NCR, Tidel, and Wincor Nixdorf devices and interfaces do not use certificates.

Certificate Authority (CA) — A person or organization that validates and signs public keys. The CA could be an ATM manufacturer, a financial institution, or a third party. Diebold devices use a third party certificate authority—Digital Signature Trust Co. (DST). Triton acts as the certificate authority for Triton devices, using VeriSign as the service provider. The NCR trusted center serves as the certificate authority for NCR and Tidel devices, the Customer Key Center – International serves as the certificate authority for Wincor Nixdorf devices, and the interchange serves as the certificate authority for interfaces.

Certificate Signing Request (CSR) — A request for a certificate to be signed by a certificate authority. This request is always in PKCS #10 Certificate Request Message format.

Cipher — An encryption-decryption algorithm.

Ciphertext — Encrypted data.

Common name — A specific entity to which an X509 certificate applies. This field, in conjunction with the organization name, provides additional security to ensure that a client is authenticating the correct certificate.

Cryptosystem — An encryption-decryption algorithm (cipher), together with all possible plaintexts, ciphertexts and keys.

Distinguished Encoding Rules (DER) — This set of encoding rules is a specialized form of BER that is used in security-conscious applications. These applications, such as electronic commerce, typically involve cryptography, and require that there be one and only one way to encode and decode a message. See *also* Abstract Syntax Notation One.

Encrypting PIN Pad (EPP) — A tamper-proof ATM component comprised of a PIN pad, a security module to perform cryptographic processing, data encryption algorithms (e.g., DES, Triple DES, MAC, RSA), and the cryptographic keys. All cryptographic keys are stored in the EPP. All cryptographic functions are performed by the EPP.

Master Key — A key used to encrypt and decrypt the working keys that are downloaded from the host to the ATM. Also known as the A-Key, Key Exchange Key (KEK) and Terminal Master Key (TMK).

Message Digest — The representation of text in the form of a single string of digits, created using a formula called a one-way hash function. Encrypting a message digest with a private key creates a digital signature, which is an electronic means of authentication. See *also* Secure Hash Algorithm (SHA-1).

Modulus — A large integer that is normally the product of two or more distinct large random prime numbers that is used in the algorithm to generate an RSA key pair. The Transaction Security Services process sends the value configured in the KEYGEN command in RSA key commands to the security device.

Nonce — A randomly generated value used to defeat *playback* attacks in communication protocols. One party randomly generates a nonce and sends it to the other party. The receiver encrypts it using the agreed upon secret key and returns it to the sender. Since the sender randomly generated the nonce, this defeats playback attacks because the replier cannot know in advance the nonce the sender will generate. The receiver denies connections that do not have the correctly encrypted nonce. Diebold and Triton devices use nonces; Wincor Nixdorf and Tidel devices do not. NCR devices using the enhanced scheme use nonces; NCR devices using the basic scheme do not use nonces.

Organization name — The name of the organization or company to which an X509 certificate applies. This field, in conjunction with the common name, provides additional security to ensure that a client is authenticating the correct certificate.

Out-of-Band — A term describing a function that is performed offline, rather than using online transaction processing. For example, manually loading keys at an ATM is a function that is performed out-of-band. Sending a key to a certificate authority to be validated and signed is an example of another out-of-band function.

PKCS #7 Cryptographic Message — A message standard for data that may have cryptography applied to it, such as digital signatures and digital envelopes. The Diebold and Triton certificate authorities return certificates that have been registered with the certificate authority using this format. The Diebold and Triton EPPs transport certificates and other data to and from the BASE24 system using this message. Diebold, Triton, NCR certificate emulation, and shared Wincor Nixdorf certificate devices use the PKCS #7 cryptographic message; NCR, Tidel, Diebold signature emulation, and shared Wincor Nixdorf signature devices and interfaces do not.

Note: Certificates received from the Triton certificate authority must be converted from .cer (SSL certificate) format to .p7b (PKCS #7) format before they can be used in BASE24-eps.

PKCS #10 Certificate Request Message — A message standard for a request for certification of a public key. Also referred to as a certificate signing request (CSR). The root BASE24 system public key must be sent in this format when it is sent to the Diebold or Triton certificate authority. When it is returned from the Diebold certificate authority, it is in PKCS #7 cryptographic message format. When it is returned from the Triton certificate authority, it must be converted from .cer (SSL certificate) format to .p7b (PKCS #7) format before it can be used in BASE24-eps.

Plaintext — The data to be encrypted.

Private Key — One-half of a cryptographic key pair used in a public key algorithm such as Rivest-Shamir-Adleman (RSA). The other half of the cryptographic key pair is the public key. The private key is uniquely associated with an entity and is not available publicly. It is used to decrypt data that was encrypted using its associated public key. It is also used to create

signatures that can only be verified using its associated public key. A mathematical relationship exists between the private key and the public key, but it is virtually impossible to derive one from the other. The private key is also known as the secret key.

Public exponent — A small integer that is often a prime close to a power of 2 (e.g., 3, 5, 7, 17, 257, or 65537) that is used in the algorithm for generating an RSA key pair. The Transaction Security Services process always passes a value of 65537 (hexadecimal value 010001) in RSA key commands to the security device.

Public Key — One-half of a cryptographic key pair used in a public key algorithm such as Rivest-Shamir-Adleman (RSA). The other half of the cryptographic key pair is the private key (also known as the secret key). The public key is uniquely associated with an entity and is available publicly. It is used to encrypt data that can only be decrypted using its associated private key. It is also used to verify signatures that were created with its associated private key. A mathematical relationship exists between the public key and the private key, but it is virtually impossible to derive one from the other.

Public Key Cryptography Standards (PKCS) — A series of cryptographic standards dealing with public key issues, published by RSA Laboratories.

Public Key Infrastructure (PKI) — A security infrastructure whose services are implemented and delivered using public key concepts and techniques.

RSA — A public key algorithm based on the factoring problem. RSA stands for Rivest-Shamir-Adleman, the developers of the RSA public key algorithm.

Secret Key — See private key.

Secure Hash Algorithm (SHA-1) — An algorithm for computing a *condensed representation* of a message or a data file. The *condensed representation* is of fixed length and is known as a *message digest* or *fingerprint*. Because it is conjectured to be computationally infeasible to produce two messages having the same message digest, a SHA-1 hash value can be used to verify the integrity of a file. NCR, Tidel, and Wincor Nixdorf use SHA-1 hash values to ensure the integrity of public key data transported between the certificate authority (i.e., NCR trusted center or Wincor Nixdorf Customer Key Center – International) and an NCR, Tidel, or Wincor Nixdorf customer.

Signature — A method of preventing data from being changed during transmission and of preventing the sender from being impersonated. A secure hashing algorithm is applied to the data and then the result is encrypted. With RSA encryption, the signature is generated using the private key. Only the holder of the private key can generate a valid signature, but anybody can verify it using the associated public key.

For NCR, Tidel, and Wincor Nixdorf devices, signatures are generated using the RSAES-PKCS1-v1_5 scheme. A 256-byte signature block is created. For transmission to the ATM, the block is base94 encoded for devices running NCR and Tidel firmware or Diebold signature emulation firmware and base64 encoded for devices running shared Wincor Nixdorf signature or Triton firmware.

Unique Identifier — The remote key transport (RKT) scheme for Triton devices uses unique identifiers for the certificate authority, the Triton EPP, and the host (BASE24 system). These identifiers are stored in the common name of certificates for each participating party in RKT processing. The unique identifier for the certificate authority is always “Triton Systems of Delaware, Inc.”, while the unique identifiers for Triton EPPs and the host consist of 16 numeric (0-9) digits.

Working Key — A category of keys used to perform specific cryptographic operations. This key is encrypted under the master key prior to being downloaded from the host to the ATM. PIN encryption keys, MAC keys and PIN verification keys are all examples of working keys.

Implementation requirements

The following paragraphs discuss various hardware and software requirements for implementing the BASE24 Automated Key Distribution System add-on product or PKI for secure logons and logoffs between a BASE24 Interchange Interface process and an interchange.

NCR 5XXX devices

The NCR 5XXX (or NCR certificate emulation) device must be running NCR NDC+ 7.00 (or greater) or APTRA Advance NDC 3.00 (or greater) and APTRA Edge environment NDC Business Services software and be equipped with an encrypting PIN pad (EPP) to support the BASE24 Automated Key Distribution System add-on product. NCR supports the BASE24 Automated Key Distribution System add-on product on most 56XX and PersonaS models that meet the above requirements

For native NCR 5XXX devices, the EPP must contain a public and private key pair, a signature of the EPP's public key, the NCR trusted center's public key, and a unique serial number. All of these are created when the EPP is manufactured. See phases 1 and 2 of the implementation procedures for NCR devices.

For NCR certificate emulation firmware on Diebold devices, the EPP must contain two sets of public and secret key pairs. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. Both key pairs are signed by a certificate authority and are stored in the EPP as certificates. For each public key, the certificate authority creates two certificates—a primary certificate and a secondary certificate—for disaster recovery purposes. BASE24 accepts and stores only the primary certificates during registration and certification. Secondary certificates are never used. In addition to the four certificates that contain the EPP's public keys and signatures, two certificates (primary and secondary) containing the certificate authority's public key and signature are stored in the EPP. See phase 1 of the implementation procedures for Diebold devices.

ATM master keys generated for NCR 5XXX devices can be single length (16 hexadecimal characters) or double length (32 hexadecimal characters). The ATM uses a single or double length key depending on the key entry mode selected in supervisor mode or downloaded

using the Load Key Entry Mode command entered from the DCT. Refer to the [“Key entry mode download”](#) topic provided later in this section for more information. NCR recommends that all DES keys be double-length keys. A double-length master key is required to support triple DES encryption between the BASE24 system and an NCR 5XXX device.

Note: NCR's enhanced BAPE software that mimics triple DES encryption does not support the loading of initial keys using the RSA method.

NDC+ 7.00 environment

Previous releases of NDC+ using the BAPE usually preserved the encryption keys stored in the encryptor. No special action was required on the part of NDC+ for this. When NDC+ 7.00 is installed on an ATM with a BAPE encryptor, the initial keys entered using a previous release of NDC+ are preserved. The only exception is release 6.05 where the allocation of key slots differed from other releases (this is corrected in release 6.05.01).

With the EPP, special action is required to preserve the keys in the encryptor when migrating to release 7.00. The EPP is provided in two variants that can emulate either a BAPE (EPPB) or an EKC (EPPE). For NDC+, the EPP must be an EPPB, which is referred to as an EPP throughout this section. The EPP can operate in either its emulation mode or as an EPP. If a previous release of NDC+ has been installed on an ATM with an EPPB, it will operate in BAPE emulation mode. NDC+ release 7.00 will detect this condition, and rather than reprogramming the encryptor as an EPP, it will continue to run in BAPE emulation mode, preserving the encryption keys, until the encryption key entry mode is changed. You can change the key entry mode either locally through supervisor mode or remotely using the Load Key Entry Mode command entered from the DCT. In either case, changing the key entry mode causes all encryption keys to be deleted and the encryptor to be re-programmed as an EPP. You can enter the Load Key Entry Mode command before or after entering the Load Public Key command.

While the EPP is operating in BAPE emulation mode, all the triple DES operations are emulated in software. RSA initial key download is not supported in BAPE emulation mode. No attempt is made to preserve keys loaded onto an EPP; it is always reprogrammed as an EPP.

APTRA environment

APTRA Advance NDC 3.00 (or greater) and APTRA Edge environment NDC Business Services software only supports triple DES and RSA cryptography in hardware and the EPP must operate in EPP mode. In this case, you must enter the Load Public Key command before entering the Load Key Entry Mode command.

Protocol support

The message size required for AKDS messages dictates the protocols supported for NCR 5XXX devices. The following table lists the protocols supported for NCR 5XXX devices as identified by NCR and the corresponding protocol used by the XPNET process.

NCR	XPNET
SNA (LU0)/SDLC ¹	SNA Device Support - Primary LU Type 0, 2
X.25 LAPB/X.21bis	X.25 Network Interface with the HOST or NONE end to ends
SNA/X.25 (QLLC)	X.25 Network Interface with the HOST or NONE end to ends where the HP NonStop system handles the SNA layer.
TCP/IP	TCP/IP
TC500 Asynchronous	6100 Burroughs/NCR Multipoint Supervisor
TC500 Synchronous	

¹ The XPNET process no longer supports straight SDLC.

Diebold 10XX/478X devices

The Diebold 10XX/478X device must be running Terminal Control Software level 1.3 (or greater) and be equipped with the EPP4 encrypting PIN pad. Agilis 91X, TCS Plus, and TCS/OS2 systems that meet the above requirements will support the BASE24 Automated Key Distribution System add-on product. Diebold Agilis 91x XV terminal application firmware is required for Diebold signature emulation firmware.

For native Diebold 10XX/478X devices, the EPP must contain two sets of public and secret key pairs. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. Both key pairs are signed by a certificate authority and are stored in the EPP as certificates. For each public key, the certificate authority creates two certificates—a primary certificate and a secondary certificate—for disaster recovery purposes. BASE24 accepts and stores only the primary certificates during registration and certification. Secondary certificates are never used. In addition to the four certificates that contain the EPP's public keys and signatures, two certificates (primary and secondary) containing the certificate authority's public key and signature are stored in the EPP. See phase 1 of the implementation procedures for Diebold devices.

For Diebold signature emulation firmware on NCR devices, the EPP must contain a public and private key pair, a signature of the EPP's public key, the NCR trusted center's public key, and a unique serial number. All of these are created when the EPP is manufactured. See phases 1 and 2 of the implementation procedures for NCR devices.

ATM master keys generated for Diebold 10XX/478X devices can be single length (16 hexadecimal characters) or double length (32 hexadecimal characters). If the terminal is set to use single-length keys and the new master key downloaded to the terminal is a valid double-length key, the terminal automatically switches to double-length keys. If the terminal is set to use double-length keys and the new master key downloaded to the terminal is a valid single-length key, the terminal automatically switches to single-length keys. Thus, there is no need to configure the key length at the terminal prior to remotely downloading the master key.

Protocol support

The message size required for AKDS messages dictates the protocols supported for Diebold 10XX/478X devices. The following table lists the protocols supported for Diebold 10XX/478X devices as identified by Diebold and the corresponding protocol used by the XPNET process.

Diebold	XPNET
Burroughs Poll Select asynchronous	6100 Burroughs/NCR Multipoint Supervisor
Burroughs Poll Select synchronous	
IBM 3271 Bisync (ASCII)	6100 Bisync Multipoint Supervisor
IBM 3271 Bisync (EBCDIC)	
IBM SNA/SDLC LU types 0 and 2	SNA Device Support - Primary LU Type 0, 2
TCP/IP Stream Sockets (client/server)	TCP/IP
TCP/IP (Diebold TBAPP)	
TCP/IP Dial	
IBM SNA over X.25 (QLLC) LU types 0 and 2 (CCS) ¹	X.25 Network Interface with the HOST or NONE end to ends

- ¹ SNA over X.25(QLLC) is standard X.25 to XPNET (X.25 Network Interface), using either the NONE or HOST end to ends. The SNA part over X.25 is configured and handled by the HP NonStop system. The XPNET process has no knowledge of the SNA layer. The X.25 Network Interface with either the HOST or NONE end to ends are used for standard SVC and PVC circuits.

Shared Wincor Nixdorf signature devices

The shared Wincor Nixdorf signature device must be running firmware compliant with release 1.5 or greater of Wincor Nixdorf's shared signature specification, **Wincor Nixdorf NDC/ Diebold D91x Message Format Extension for RKL**, and be equipped with an encrypting PIN pad (EPP) to support the BASE24 Automated Key Distribution System add-on product.

Note: The device vendor need not be Wincor Nixdorf as long as the device is running firmware compliant with Wincor Nixdorf's shared signature specification and is equipped with a signature-based encrypting PIN pad (EPP) to support the BASE24 Automated Key Distribution System add-on product. Any other device vendor whose ATMs can run the necessary firmware can also be used. For simplicity, Wincor Nixdorf is assumed to be the device vendor for non-NCR devices throughout this section.

For shared Wincor Nixdorf signature devices, the EPP must contain a public and private key pair, a signature of the EPP's public key, the customer-specific Wincor Nixdorf public key, a unique serial number, and a signature of the serial number. The signatures are created by signing the EPP's public key and serial number using the customer-specific Wincor Nixdorf secret key. All of this data is created when the EPP is manufactured. See phases 1 and 2 of the implementation procedures for shared Wincor Nixdorf signature devices.

For NCR devices running shared Wincor Nixdorf signature firmware, the EPP must contain a public and private key pair, a signature of the EPP's public key, the NCR trusted center's public key, and a unique serial number. All of these are created when the EPP is manufactured. See phases 1 and 2 of the implementation procedures for NCR devices.

The message size required for AKDS messages dictates the protocols supported for shared Wincor Nixdorf signature devices running in NDC+ or Diebold 912 emulation mode. Refer to the "Protocol Support" topics for NCR 5XXX and Diebold 10XX/478X devices above for lists of the protocols supported.

Shared Wincor Nixdorf certificate devices

The Diebold device must be running firmware compliant with release 1.5 of Wincor Nixdorf's shared certificate specification, ***Wincor Nixdorf NDC/Diebold D91x Message Format Extension for RKL***, and be equipped with an encrypting PIN pad (EPP) to support the BASE24 Automated Key Distribution System add-on product.

The EPP must contain two sets of public and secret key pairs. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. Both key pairs are signed by a certificate authority and are stored in the EPP as certificates. For each public key, the certificate authority creates two certificates—a primary certificate and a secondary certificate—for disaster recovery purposes. BASE24 accepts and stores primary certificates only during registration and certification. Secondary certificates are never used. In addition to the four certificates that contain the EPP's public keys and signatures, two certificates (primary and secondary) containing the certificate authority's public key and signature are stored in the EPP. See phase 1 of the implementation procedures for Diebold devices.

The message size required for AKDS messages dictates the protocols supported for shared Wincor Nixdorf certificate devices running in NDC+ or Diebold 912 emulation mode. Refer to the "Protocol Support" topics for NCR 5XXX and Diebold 10XX/478X devices above for lists of the protocols supported.

Tidel devices

Tidel ATMs must be running Tidel's 4.22.00 software.

The EPP in a Tidel device must contain a public and private key pair, a signature of the EPP's public key, the NCR trusted center's public key, and a unique serial number. All of these are created when the EPP is manufactured. See phases 1 and 2 of the implementation procedures for NCR 5XXX and Tidel devices.

ATM master keys generated for Tidel devices must be double length (32 hexadecimal characters). Although Tidel devices can support separate master keys for PIN and MAC working keys, only one master key is supported if using the AKDS add-on product.

Tidel ATMs must connect to the BASE24 system using a dialup protocol.

You must use Atalla or Thales security devices to support AKDS for Tidel devices.

Triton devices

Triton ATMs must be running Triton's v2.3 software.

Triton EPPs must contain two sets of public and secret key pairs. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. Both key pairs are signed by a certificate authority and are stored in the EPP as certificates. For each public key, the certificate authority creates a certificate. In addition to the two certificates that contain the EPP's public keys and signatures, one certificate containing the certificate authority's public key and signature are stored in the EPP. See phase 1 of the implementation procedures for Triton devices.

The only EPPs that currently support RKT are Sagem-branded EPPs with firmware versions "414-0391 R1A2".

Triton ATMs can connect to the BASE24 system using a dialup protocol or using TCP/IP over a leased line.

You must use Atalla or Thales security devices to support AKDS for Triton devices.

BASE24 system

The BASE24 Automated Key Distribution System add-on product or secure logons and logoffs to and from an interchange requires a special Transaction Security Services process called the AKDS process, a hardware security module with the appropriate firmware or software, and one of the following versions of BASE24 release 6.0:

- BASE24 release 6.0, version 3 or greater for NCR devices running NCR firmware
- BASE24 release 6.0, version 4 or greater for Diebold devices running Diebold firmware

- BASE24 release 6.0, version 6 or greater for devices running Wincor Nixdorf signature firmware and Diebold devices running NCR certificate emulation firmware
- BASE24 release 6.0, version 7 or greater for Diebold devices running Wincor Nixdorf certificate firmware
- BASE24 release 6.0, version 8 or greater for NCR devices running Diebold signature emulation firmware, NCR devices running Wincor Nixdorf signature firmware, and interchange interfaces
- BASE24 release 6.0, version 9 or greater for Tidel or Triton devices

One or two RSA public/private key pairs must be created for the BASE24 system. Two RSA public/private key pairs are required for the NCR enhanced scheme. One RSA public/private key pair is required for all other implementations. These key pairs are created in a hardware security module and are stored in the Host Public Key Security File (HOST_PUBLIC_KEY). Use the “[Key Generate \(KEYGEN\) command](#)” to generate and store these RSA key pairs. For further information on how this command is used for each of the supported combinations of devices and firmware, see [phase 3](#) of the implementation procedures for NCR devices, Tidel devices, NCR devices running Diebold signature emulation firmware, and NCR devices running Wincor Nixdorf signature firmware, [phase 2](#) of the implementation procedures for Diebold devices, Diebold devices running NCR certificate emulation firmware, and Diebold devices running Wincor Nixdorf certificate firmware, [phase 2](#) of the implementation procedures for Triton devices, or [phase 3](#) of the implementation procedures for devices running Wincor Nixdorf signature firmware. For information on how this command is used for interfaces, see [phase 2](#) of the implementation procedures for interfaces.

Note: The AKDS process is a special Transaction Security Services process that contains all of the components of the Transaction Security Services process plus the components required to support the public key cryptography. The AKDS process uses the AKDSISLO executable object while the regular Transaction Security Services process uses the TSSISLO executable object.

Security device configuration

For each security device to be used for public key cryptography (PKI) functions, you must check the Enable PKI field on the Security Device Configuration window. If this field is not checked for a security device, it is not used for PKI functions. This enables you to consolidate all PKI functions in just one or a few devices.

Atalla public key cryptography commands

An Atalla NSP must be running a minimum of AKB version 2.40 or 1.4 software to support public key cryptography commands using the Rivest-Shamir-Adleman (RSA) scheme. The 2.40 version software is used for A10100, A10150, or A10160 AKB models while the 1.4 version software is used for A10000 AKB models. All of these commands are defaulted to off in the security policy delivered from the factory due to security exposure. BASE24 requires a number of these commands to support public key cryptography. The following commands required for public key cryptography must be enabled in your security policy using the 108 and 109 commands:

- Generate RSA key pair (Command 120)
- Generate AKB for root public key (Command 122)
- Verify public key and generate AKB of public key (Command 123)
- Generate digital signature (Command 124)
- Verify digital signature (Command 125)
- Generate message digest (Command 126)
- Generate ATM master key and encrypt with public key (Command 12F)*

* This command is only used for the BASE24 Automated Key Distribution System add-on product.

The BASE24 Automated Key Distribution System add-on product also uses the Generate Random Number (Command 93) command to generate a random nonce when a new master key is generated for a Diebold or Triton device. Command 93 is enabled in the AKB NSP's default factory security policy.

AKB headers for RSA keys

The following table lists the AKB header assignments to use for RSA public and private keys.

Header assignment	Description
1 K R E E 0 0 0 1 K R E N 0 0 0	The RSA public key used to encrypt key information
1 K R D E 0 0 0 1 K R D N 0 0 0	The RSA private key used to decrypt key information
1 S R G E 0 0 0 1 S R G N 0 0 0	The RSA private key pair used to sign (generate a digital signature)
1 S R V E 0 0 0 1 S R V N 0 0 0	The RSA public key used to verify a digital signature

Banksys DEP HSM public key cryptography commands

Note: You do not need the Banksys DEP RSA KEY GEN & USE or DEP RSA Key Loading programs to generate, authenticate, and store an RSA key pair with the Banksys DEP HSM. Because all RSA keys are stored in the TSS database rather than in the DEP key table, the KEYGEN process control command and Certificate Management window are used instead to perform these functions.

The following commands required for public key cryptography must be enabled in your customer authority level:

- RSA Key Pair Generate (02252900)
- RSA Signature Verify (02250D00 or 02251000)
- RSA Signature Generate (02250C00 or 02250F00)
- RSA Import Public Key (02252A00)
- RSA Generate ATM Master Key (02252B00)*
- RSA Hash Generate (02310800)

* This command is only used for the BASE24 Automated Key Distribution System add-on product.

The Transaction Security Services process performs the same processing for Banksys DEP HSMs that it performs for Thales e-Security HSMs.

Note: Public keys sent to the Banksys DEP HSM must be in the Abstract Syntax Notation One (ASN.1) format using the Distinguished Encoding Rules (DER). The format is as follows:

Sequence identifier	Byte length	Integer identifier	Modulus length	Modulus	Integer identifier	Exponent length	Exponent
---------------------	-------------	--------------------	----------------	---------	--------------------	-----------------	----------

A default value of 65537 is assumed for all public key exponent values.

The AKDS process automatically handles any necessary HSM message format conversions.

The BASE24 Automated Key Distribution System add-on product also uses the Key Generate (022B3000) command to generate a random nonce when a new master key is generated for a Diebold or shared Wincor Nixdorf certificate device.

Thales e-Security public key cryptography commands

Thales e-Security version 6.02 firmware and the optional DSP plug-in (for RSA functions) provides public key cryptography commands using the RSA scheme. BASE24 requires the following commands to support public key cryptography:

- Generate a key (Command A0)
- Generate RSA key set (Command EI)
- Generate a MAC on a public key (Command EO)
- Validate a certificate and generate MAC on its public key (Command ES)
- Generate a signature (Command EW)
- Validate a signature (Command EY)
- Export a DES key (Command GK)
- Hash a Block of Data (GM)

Note: Public keys sent to the Thales HSM must be in the Abstract Syntax Notation One (ASN.1) format using the Distinguished Encoding Rules (DER). The format is as follows:

Sequence identifier	Byte length	Integer identifier	Modulus length	Modulus	Integer identifier	Exponent length	Exponent
---------------------	-------------	--------------------	----------------	---------	--------------------	-----------------	----------

A default value of 65537 is assumed for all public key exponent values.

The AKDS process automatically handles any necessary HSM message format conversions.

LCONF params

The BASE24 Automated Key Distribution System add-on product uses the following LCONF params in addition to the LCONF assigns and params discussed in section 3.

ATM-DH-AKDS-TSS-TIMER param — The number of seconds the NCR 5XXX Device Handler process, Diebold 10XX/478X Device Handler process, Tidel Device Handler process, or Triton Device Handler process waits for a reply from the AKDS process before it considers the Automated Key Distribution System (AKDS) request to have timed out. In this case, the requested load is aborted. Valid values are 10–600 seconds. This param defaults to 600 seconds.

This param must be set in consideration of the Timeout Limit field value on the Security Device Configuration window. The Timeout Limit field value specifies the maximum time, in seconds, to wait for a response from an HSM before the AKDS process considers the request to be timed out. The ATM-DH-AKDS-TSS-TIMER param value must be set to a value large enough to accommodate this length of time plus the time needed to send the request to and receive the response from the AKDS process. Thus, it should always be set to a value greater than that set in the Timeout Limit field on the Security Device Configuration window.

ATM-DH-1000-AKDS-SERIAL-NUM-VRFY — An optional param indicating whether the Diebold 10XX/478X Device Handler process verifies the EPP serial numbers received from the ATM when authenticating and exchanging public keys. If the values match valid EPP serial numbers stored in the EPP Serial Number File (EPP_SERIAL_NUM), it is considered to be valid. Valid values are as follows:

- Y = Yes, validate the EPP serial numbers
- N = No, do not validate the EPP serial numbers

If this param is not configured or contains an invalid value, it defaults to a value of N.

This param applies when using AKDS for any device running off the Diebold 10XX/478X Device Handler process.

ATM-DH-N50-AKDS-SERIAL-NUM-VRFY — An optional param indicating whether the NCR 5XXX Device Handler process verifies the EPP serial number received from the ATM when authenticating and exchanging public keys. If the value matches a valid EPP serial number stored in the EPP Serial Number File (EPP_SERIAL_NUM), it is considered to be valid. Valid values are as follows:

- Y = Yes, validate the EPP serial number
- N = No, do not validate the EPP serial number

If this param is not configured or contains an invalid value, it defaults to a value of N.

This param applies when using AKDS for any device running off the NCR 5XXX Device Handler process.

ATM-DH-TIDL-AKDS-SERIAL-NUM-VRFY — An optional param indicating whether the Tidel Device Handler process verifies the EPP serial number received from the ATM when authenticating and exchanging public keys. If the value matches a valid EPP serial number stored in the EPP Serial Number File (EPP_SERIAL_NUM), it is considered to be valid. Valid values are as follows:

- Y = Yes, validate the EPP serial number
- N = No, do not validate the EPP serial number

If this param is not configured or contains an invalid value, it defaults to a value of N.

This param applies when using AKDS for any device running off the Tidel Device Handler process.

ATM-DH-TIDL-AKDS-HOST-LOAD-TIMER — The number of seconds the ATM waits for a Host RKM Command message to be sent from the Tidel Device Handler process during an AKDS load before aborting the load. Valid values are 10 through 600. This is an optional param. If it is not configured or is invalid, the default value of 60 is used.

The Tidel Device Handler places the value of this parameter in the Group-15 field of a financial or administrative transaction response when a key load is initiated.

ATM-DH-TRIT-AKDS-SERIAL-NUM-VRFY — An optional param indicating whether the Triton Device Handler process verifies the EPP serial number received from the ATM when authenticating and exchanging public keys. If the value matches a valid EPP serial number stored in the EPP Serial Number File (EPP_SERIAL_NUM), it is considered to be valid. Valid values are as follows:

- Y = Yes, validate the EPP serial number
- N = No, do not validate the EPP serial number

If this param is not configured or contains an invalid value, it defaults to a value of N.

This param applies when using AKDS for any device running off the Triton Device Handler process.

NCR 5XXX and Tidel terminal implementation procedures

Implementation of remote initial key processing for NCR 5XXX and Tidel terminals is accomplished in several phases, each of which are described below. The first two phases are performed by NCR alone, ACI and its customers are not involved. These phases are performed out-of-band of the BASE24 system.

Note: Phases 1–3 of the NCR 5XXX terminal implementation procedures also apply to NCR signature devices running Wincor Nixdorf firmware, except that the NCR enhanced signature-based AKDS scheme is not supported.

Phase 1—generate an NCR RSA key pair

The NCR trusted center generates a Rivest-Shamir-Adleman (RSA) key pair composed of a private (secret) key and a public key. These keys are used for signature authentication between an NCR or Tidel ATM and the BASE24 system. The private key is used to generate signatures, while the public key is used to verify signatures. This key pair is referred to as the *root* NCR key pair. The root public key of the NCR trusted center is sent (through an offline method) to you during phase 3.

Phase 2—manufacture encrypting PIN pad (EPP)

When the EPP for a 5XXX or Tidel terminal is manufactured, NCR generates an RSA public and private key pair for each EPP. These keys are used for the encryption and decryption of keys exchanged between the EPP and the BASE24 system. The EPP's public and private keys are stored in the EPP. The public key for the EPP is signed with the root NCR private key. This signature allows the BASE24 system to validate the EPP using the root NCR public key created in phase 1. The signature is stored in the EPP along with the root NCR public key. The root NCR public key is used to validate a signature received from the BASE24 system. Finally, the serial number stored in the EPP is signed by the root NCR private key. This signature is used for additional authentication between the EPP and the BASE24 system. At this point, the following key information is stored in the EPP:

- The EPP's public and private keys
- The EPP's public key signature generated by the root NCR private key
- The root NCR public key
- The EPP's serial number
- The EPP's serial number signature generated by the root NCR private key

Phase 3—generate RSA key pairs for the BASE24 system

To support automated key distribution for ATMs, you must also generate RSA key pairs for the BASE24 system using a hardware security module. One root RSA key pair is required for the basic scheme, while two RSA key pairs are required for the enhanced scheme—the root and the primary. These key pairs are created by entering the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window of the ACI desktop. After the key pair is generated, the NCR trusted center must sign the root public key for the BASE24 system and return the resulting signature to you along with NCR trusted center's root public key. You must use this key to verify the signature returned from NCR. Finally, you must generate a MAC of the self-signed root NCR public key and store all these keys in the TSS database.

Note: The procedures for this phase are essentially the same whether you are generating an RSA key pair for a test or production environment on the BASE24 system or whether you are using the NCR basic or enhanced scheme. For testing, a test EPP provided by NCR must be used instead of a production EPP. When you initially contact NCR to start the process of submitting a public key and getting it signed by NCR, you must specify whether the key to be signed is for testing or production. You should use the same forms and procedures for getting both test and production keys signed, except that for a test key, the key data exchanged with the NCR trusted center need not be delivered on diskette or CD-ROM in tamper-resistant envelopes or using a courier service. Data can be exchanged using an e-mail message instead. The NCR trusted center signs test keys using a root public key that is different from the one it uses to sign production keys.

The above tasks are accomplished in the following steps:

1. To start the process of submitting a public key and getting it signed by NCR, contact your NCR/Reseller Account Manager. The account manager will then establish your name, mailing address, e-mail address, telephone and fax numbers and advise NCR Customer and Sales Support to send you the following forms:
 - Signature Request Process (NCR_001)
 - NCR Approved Contacts (NCR_002)
This form will be completed by NCR before it is sent to you.
 - Signature Request (NCR_003)
2. Generate an RSA public and private key pair or pairs for the BASE24 system. Enter the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window to generate the RSA key pair or pairs for the BASE24 system. Refer to the [“Key Generate \(KEYGEN\) command”](#) topic provided later in this section for detailed information on using this command.

Note: Public and private keys successfully generated by the KEYGEN command are stored in the Host Public Key Security File (HOST_PUBLIC_KEY) immediately.
3. Access the Certificate Management window on the ACI desktop (access **Configure > Transaction Security > Automated Key Distribution > Certificate Management**) to retrieve the root BASE24 system public key and format it in the manner that the NCR trusted center expects it. When the Certificate Management window is displayed, select the host name, device type (i.e., NCR), and effective date used to generate the key. The root BASE24 system public key is retrieved, signed, formatted, and displayed in the Root

Host Public Key Data field. A 40-byte SHA-1 hash value is also generated and displayed in the SHA-1 Hash Value field. The root BASE24 system public key is self-signed and displayed in ASN.1 format. This is the message format required when sending the public key to the NCR trusted center.

4. Copy the root BASE24 system public key cryptogram in the Root Host Public Key Data field, format it as required, and send it on the appropriate media (e.g., a diskette or CD-ROM in a tamper-resistant envelope) to your primary NCR contact specified on form NCR_002. Follow the format and procedures prescribed on form NCR_001.

At the same time, complete form NCR_003 and send it by e-mail message or by facsimile (fax) to all Key Signing Individuals (i.e., the primary and all secondary NCR contacts specified on form NCR_002). Make sure you enter the 40-byte hash value displayed in the SHA-1 Hash Value field on the Root Host Public Key Data tab in the SHA-1 (Fields 1–6) field on form NCR_003.

5. The NCR trusted center validates the root BASE24 system public key signature and SHA-1 hash value. The Key Signing Individual will send an e-mail message to you to acknowledge receipt of the diskette or CD-ROM and to indicate whether the signature and hash value are correct. If either are incorrect, you must resend both form NCR_003 and the root BASE24 system public key. If both are correct, the trusted center will sign the root BASE24 system public key, generate SHA-1 hash values for both the root BASE24 system public key signature and the self-signed root NCR public key, and return them back to you on a diskette or CD-ROM delivered by a courier service. NCR will also complete and issue the following two forms by e-mail message or by facsimile (fax) to the approved customer contact. You must use these forms to verify that the SHA-1 hash values over the self-signed root NCR public key and root BASE24 system public key signature are valid.

- NCR Public Key (NCR_004)
- NCR Signature of Customer Public Key (NCR_005)

The diskette or CD-ROM returned to you will contain several files. Each file contains a key or hash value. You must copy the contents of some of these files and paste them into fields on the Certificate Management window in subsequent steps as shown in the following table.

Value (NCR filename)	Certificate Management field
Self-signed root NCR public key (PK_NCR.txt)	Generate Message Digest > Certificate Authority Public Key (step 6a) and Validate Certificates/Signatures > Certificate Authority Public Key (step 9a)
SHA-1 hash of self-signed NCR public key (PK_NCR_digest.txt)	Generate Message Digest > SHA-1 Hash Value (step 6b)
Root BASE24 system public key signature (PK_(cust)_ncr_sig)	Validate Certificates/Signatures > Host Signature (step 9a)

Value (NCR filename)	Certificate Management field
SHA-1 hash of root BASE24 system public key (PK_(cust)_NCR_sig_digest.txt)	Validate Certificates/Signatures > SHA-1 Hash Value (step 9b)

6. On the Generate Message Digest tab of the Certificate Management window, perform the following:

- a. Copy the self-signed root NCR public key cryptogram returned on diskette or CD-ROM from the NCR trusted center into the Certificate Authority Public Key field. You can click Expand View () to expand this field if needed.

Note: If the cryptogram contains spaces between blocks of hexadecimal characters, the spaces are automatically removed in the message sent to the security device.

- b. Enter the SHA-1 hash value of the self-signed NCR public key into the SHA-1 Hash Value field.
- c. Click Save ()

If you are using an Atalla NSP, the NSP validates the signature of the self-signed root NCR public key, validates the SHA-1 hash value for the key, and generates a message digest of the self-signed root NCR public key, which is displayed in the Message Digest field.

If you are using a Thales e-Security or Banksys DEP HSM, the HSM validates the signature and SHA-1 hash value of the self-signed root NCR public key. Thales e-Security and Banksys DEP HSMs do not create a message digest.

Any validation errors are reported in screen messages on the Certificate Management window. If the key signature or SHA-1 hash value are incorrect, you must request NCR to resend the self-signed root NCR public key, SHA-1 hash value, and form NCR_004. The NSP uses command 126 to create a message digest value that is displayed in the Message Digest field on this tab.

7. If you are using a Thales e-Security or Banksys DEP HSM, you can skip this step and continue with step 9.

For an Atalla NSP only, encrypt the value returned in the Message Digest field from the previous step under variant 30 of the MFK using the SCT or SCA. Variant 30 is automatically assumed by both the SCT and SCA. Both the SCT and SCA require the message digest to be input as two 16-character fields. On the SCT, use the Utilities Menu--> Encrypt Challenge function to encrypt the message digest value. On the SCA, use the Calculate AKB/Cryptogram menu (Challenge/Response) to encrypt the message digest value. You must input the 32-byte message digest as two 16-byte values (Left Challenge and Right Challenge on the SCT or Challenge on the SCA). Because the SCA only allows single-length keys for a challenge/response, you must input the 32-byte message digest value as two separate key components (left 16 bytes and right 16 bytes) and encrypt each separately in the Challenge field. You must then recombine the two encrypted 16 byte values to form the root certificate hash value. Refer to the **Atalla Secure Configuration Assistant User Guide** for detailed SCA procedures.

8. If you are using a Thales e-Security or Banksys DEP HSM, you can skip this step and continue with step 9.

For an Atalla NSP only, copy or enter the 32 byte (16 byte left and 16 byte right) values from the previous step into the Root Certificate Hash Value field on the Validate Certificates/Signatures tab.

9. On the Validate Certificates/Signatures tab of the Certificate Management window, perform the following:
 - a. Copy the self-signed root NCR public key and root BASE24 system public key signature returned from the NCR trusted center into the Certificate Authority Public Key and Host Signature fields, respectively, on the Validate Certificates/Signatures tab of the Certificate Management window. You can click Expand View () to expand either of these fields if needed.
 - b. Enter the SHA-1 hash value for the root BASE24 system public key signature into the SHA-1 Hash Value field.
 - c. Click Save ()

The Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM validates the signature and SHA-1 hash value of the root BASE24 system public key received from NCR. If validated successfully, the root BASE24 system public key signature is stored in the Host Public Key Security File (HOST_PUBLIC_KEY).

For an Atalla NSP only, the self-signed root NCR public key is validated using the hash value input from step 8, imported, and stored in the HOST_PUBLIC_KEY along with the message digest.

Any validation errors are reported in screen messages on the Certificate Management window. If the key signature or SHA-1 hash value are incorrect, you must request NCR to resend the BASE24 system public key signature, SHA-1 hash value, and form NCR_005.

After the above steps are completed, the following key data is now entered into the database for the AKDS process.

- The root BASE24 system public and private keys
- The primary BASE24 system public and private keys (if generated by the KEYGEN command for the NCR enhanced scheme)
- The signature of the root BASE24 system public key signed by the root NCR private key
- The self-signed root NCR public key
- A MAC of the self-signed root NCR public key (for Thales e-Security and Banksys DEP HSMs only)

Phase 4—authenticate and exchange public keys

Before you can automatically distribute (download) an ATM master key to the EPP of an ATM, the EPP and the BASE24 system must authenticate each other and exchange their public keys. This processing is accomplished differently for NCR 5XXX and Tidel terminals.

- For an NCR 5XXX terminal, you enter the Load Public Key command for the ATM.
- For a Tidel terminal, you either enter the Load All Keys command for the ATM or have an ATM technician select the key exchange function from the ATM Supervisor's Host Report menu at the ATM.

NCR 5XXX terminals

For an NCR 5XXX terminal, the Load Public Key command (entered as LOADPUBLIC or LPUBLIC) is sent to the terminal's NCR 5XXX Device Handler or Diebold 10XX/478XX Device Handler process using the Device Control Terminal (DCT) or a network control facility. Refer to the **BASE24-atm NCR 5XXX Device Support Manual** or **BASE24-atm Diebold 10XX/478X Device Support Manual** for detailed Load Public Key command syntax and processing steps performed during this phase.

The Load Public Key command initiates an online exchange of public keys between the BASE24 system and the EPP of a specified terminal.

When you enter the Load Public Key command, the NCR 5XXX Device Handler or Diebold 10XX/478x Device Handler process closes the specified ATM and sends a configuration request message. When the process receives the ATM's configuration response message, it determines whether the ATM terminal has an AKDS-capable EPP installed and is using a protocol that supports the BASE24 Automated Key Distribution System add-on product. If the ATM has an AKDS-capable EPP, the requested load command is allowed to proceed. If the ATM is not capable of processing AKDS commands and is using a protocol that supports AKDS, the Load Public Key command is not performed. In addition, one or more event messages are generated and the process reopens the ATM.

During this phase, the AKDS process interacts with the hardware security module to perform the following:

- Verify the signature of the ATM's EPP serial number using the self-signed root NCR public key retrieved from the Host Public Key Security File (HOST_PUBLIC_KEY).
- Optionally verify the EPP serial number itself against valid EPP serial numbers entered in the EPP Serial Number File (EPP_SERIAL_NUM). The value of the ATM-DH-N50-AKDS-SERIAL-NUM-VRFY, ATM-DH-TIDL-AKDS-SERIAL-NUM-VRFY, ATM-DH-1000-AKDS-SERIAL-NUM-VRFY LCONF, or ATM-DH-TRIT-AKDS-SERIAL-NUM-VRFY param determines whether this additional validation is performed. Valid EPP serial numbers must be manually entered on the EPP Serial Number Configuration window before this validation is performed. Information entered on the EPP Serial Number Configuration window is stored in the EPP_SERIAL_NUM. Refer to the ["Data access"](#) topic provided later in this section for more information on the EPP Serial Number Configuration window and the EPP_SERIAL_NUM.
- Retrieve the root BASE24 system public key and signature from the HOST_PUBLIC_KEY. For the enhanced NCR RKL scheme, the primary BASE24 system public key and signature are also retrieved from the HOST_PUBLIC_KEY. These are sent to the ATM after the EPP serial number signature has been successfully validated.
- Verify the signature of the EPP's public key using the self-signed root NCR public key retrieved from the HOST_PUBLIC_KEY. If the signature is valid, the process generates a MAC of the EPP public key and stores the EPP public key, and the EPP public key MAC in the CHAN_PKEY_SEC.

The NCR 5XXX Device Handler or Diebold 10XX/478XX Device Handler process sends the BASE24 system public key and signature to the ATM's EPP, where they are verified and stored.

At this point, the following key information is stored in the EPP:

- The EPP's public and private keys
- The EPP's public key signature generated by the root NCR private key
- The self-signed root NCR public key
- The EPP's serial number
- The EPP's serial number signed by the root NCR private key
- The root BASE24 system's public key and signature. When using the enhanced NCR RKL scheme, the primary BASE24 system's public key and signature is also stored in the EPP.

At this point, the following key data is entered into the database for the AKDS process.

- The root BASE24 system's public and private keys. When using the enhanced NCR RKL scheme, the primary BASE24 system's public and private keys are also stored.
- The signature of the root BASE24 system's public key signed by the root NCR private key. When using the enhanced NCR RKL scheme, the signature of the primary BASE24 system's public key is also stored.
- The self-signed root NCR public key
- A MAC of the root NCR public key
- The EPP's public key
- The EPP's serial number
- A MAC of the EPP's public key

Tidel terminals

For a Tidel terminal, the Load All Keys command (entered as LOADALLKEYS) is sent to the terminal's Tidel Device Handler process using a network control facility. The Load All Keys command causes the Tidel Device Handler process to set the RKMinInit flag in the next financial or administrative response message. When the Tidel terminal receives the RKMinInit flag (or when an ATM technician selects the key exchange function from the ATM Supervisor's Host Report menu at the ATM), the ATM initiates the following processing:

- Enters an out-of-service state
- Authenticates and exchanges public keys with the BASE24 system by sending an RKM request message to the BASE24 system
- Requests a download of a new ATM master key
- Requests a download of a new PIN encryption (working) key
- Requests a download of a new MAC (working) key if message authentication (MACing) is supported
- Enters an in-service state if the above processing is successful

Authenticate and exchange public keys. During this phase, the AKDS process interacts with the hardware security module to perform the following:

- Verify the signature of the ATM's EPP serial number using the self-signed root NCR public key retrieved from the Host Public Key Security File (HOST_PUBLIC_KEY).
- Optionally verify the EPP serial number itself against valid EPP serial numbers entered in the EPP Serial Number File (EPP_SERIAL_NUM). The value of the ATM-DH-TIDL-AKDS-SERIAL-NUM-VRIFYparam determines whether this additional validation is performed. Valid EPP serial numbers must be manually entered on the EPP Serial Number Configuration window before this validation is performed. Information entered on the EPP Serial Number Configuration window is stored in the EPP_SERIAL_NUM. Refer to the ["Data access"](#) topic provided later in this section for more information on the EPP Serial Number Configuration window and the EPP_SERIAL_NUM.
- Retrieve the BASE24 system public key and signature from the HOST_PUBLIC_KEY. These are sent to the ATM after the EPP serial number signature has been successfully validated.
- Verify the signature of the EPP's public key using the self-signed root NCR public key retrieved from the HOST_PUBLIC_KEY. If the signature is valid, the process generates a MAC of the EPP public key and stores the EPP public key and the EPP public key MAC in the CHAN_PKEY_SEC.

The Tidel Device Handler process sends the BASE24 system public key and signature to the ATM's EPP, where they are verified and stored.

At this point, the following key information is stored in the EPP:

- The EPP's public and private keys
- The EPP's public key signature generated by the root NCR private key
- The self-signed root NCR public key
- The EPP's serial number
- The EPP's serial number signed by the root NCR private key
- The BASE24 system's public key and signature

At this point, the following key data is entered into the database for the AKDS process.

- The BASE24 system's public and private keys
- The signature of the BASE24 system's public key signed by the root NCR private key
- The self-signed root NCR public key
- A MAC of the root NCR public key
- The EPP's public key
- The EPP's serial number
- A MAC of the EPP's public key

Download a new ATM master key. If the public keys are successfully exchanged and authenticated, the AKDS process interacts with the hardware security module to generate a new master key encrypted under the EPP's public key and sign it using the BASE24 system

private key. The new master key and associated check digits are stored in the Channel Security File (Channel_Security). The new master key and signature are also sent to the ATM.

Download a new PIN encryption (working) key. Next, the Tidel Device Handler process requests the AKDS process to generate a new PIN working key. The AKDS process interacts with the hardware security module to generate and download a new PIN working key (inbound key) encrypted under the ATM's master key (exchange key). The new PIN working key and associated check digits are stored in the Channel Security File (Channel_Security). The new PIN working key is also sent to the ATM.

Note: The Tidel Device Handler process always downloads a new PIN working key, regardless of the setting of the PIN ENCRYPT TYPE field on Tidel AnyCard screen 9 of the BASE24-atm Terminal Data files (ATD).

Download a new MAC (working) key. Finally, if the MAC SUPPORT TYPE field on Tidel AnyCard screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM, the Tidel Device Handler process requests the AKDS process to generate a new MAC working key. The AKDS process interacts with the hardware security module to generate and download a new MAC working key (inbound key) encrypted under the ATM's master key (exchange key). The new MAC working key and associated check digits are stored in the Channel Security File (Channel_Security). The new MAC working key is also sent to the ATM.

At this point, both the BASE24 system and the EPP have the terminal's master key, PIN working key, and MAC working key stored in their respective databases.

Key entry mode download

Note: The following procedure is only available for NCR devices running NCR firmware.

After the BASE24 system and the EPP have authenticated each other and exchanged their public keys, the key entry mode can be downloaded to the ATM. This is accomplished by entering the Load Key Entry Mode command (entered as LOADKEYMODE or LKEYMODE) from the Device Control Terminal (DCT) or a network control facility.

Note: This procedure is not necessary if the key entry mode is set through supervisor mode at the terminal itself, prior to authenticating and exchanging public keys. In addition, if the ATM is using NDC+ 7.00 software, you can enter the Load Key Entry Mode command before or after authenticating and exchanging public keys.

Changing the key entry mode of an EPP deletes all the DES keys in the encryptor. To reduce the chances of accidentally changing the mode and deleting the keys, the Load Key Entry Mode command is accepted only when the "Authenticate and Exchange Public Keys" phase has been successfully performed since the last power up. This ensure that the DES keys can be restored after they are deleted by changing the key mode.

Note: NCR allows one exception to the above restriction. To assist in the migration from a previous release of NDC+ to release 7.00, the Load Key Entry Mode command is accepted one time only without having to perform the "Authenticate and Exchange Public Keys" phase if the EPP is operating in Basic Alpha PIN pad and Encryptor (BAPE) emulation mode.

If the EPP is installed with NDC+ 7.00 software and is running in BAPE emulation mode when the key entry mode is changed, either through supervisor mode or by entering the Load Key Entry Mode command, it is also reprogrammed as an EPP and moves out of BAPE emulation mode.

Refer to the **BASE24-atm NCR 5XXX Device Support Manual** for detailed Load Key Entry Mode command syntax and processing steps performed during this phase.

Phase 5—download the master key

Note: This phase does not apply to Tidel devices because master keys are automatically downloaded when you initiate a key exchange by entering the Load All Keys command or when an ATM technician selects the key exchange function from the ATM Supervisor's Host Report menu at the ATM.

After an NCR 5XXX EPP has successfully authenticated and exchanged public keys with the BASE24 system, and the key entry mode has been set, you can securely download the ATM master key to the terminal using the Load Master Key command (entered as LOADMASTER or LMASTER). You can enter this command from the DCT or a network control facility. This command specifies the ID of the terminal to which the ATM master key is to be downloaded. After the master key is successfully loaded in both the EPP and the TSS database, you can securely download working keys to the EPP. Refer to the **BASE24-atm NCR 5XXX Device Support Manual** or **BASE24-atm Diebold 10XX/478X Device Support Manual** for detailed Load Master Key command syntax and processing steps performed during this phase.

When you enter the Load Master Key command, the NCR 5XXX Device Handler or Diebold 10XX/478x Device Handler process closes the specified ATM and sends a configuration request message. When the process receives the ATM's configuration response message, it determines whether the ATM terminal has an AKDS-capable EPP installed and is using a protocol that supports the BASE24 Automated Key Distribution System add-on product. If the ATM has an AKDS-capable EPP, the requested load command is allowed to proceed. If the ATM is not capable of processing AKDS commands and is using a protocol that supports AKDS, the Load Master Key command is not performed. In addition, one or more event messages are generated and the process reopens the ATM.

During this phase, the AKDS process interacts with the hardware security module to generate a new master key encrypted under the EPP's public key and sign it using the BASE24 system private key. The new master key and associated check digits are stored in the Channel Security File (Channel_Security). The new master key and signature are also sent to the ATM.

At this point, both the BASE24 system and the EPP have the terminal's master key stored in their respective databases. Working keys (i.e., the PIN encryption key and MAC key) can now be securely downloaded to the EPP.

Financial Institution Table (FIT) data

The PIN encryption key in the FIT is encrypted under the master key. If a new master key is generated and a PIN encryption key exists in the FIT for local PIN verification, you must manually translate the PIN encryption key from encryption under the old master key to encryption under the new master key, replace the PIN encryption key cryptogram in the FIT, and download the updated FIT data to the ATM.

For the NCR 5XXX Device Handler process, the PIN encryption key is entered in the PEKEY (PIN ENCRYPTION KEY) field on the Financial Institution Table screen that you can access from the NCR 5XXX Configuration File Menu in the Device Control Terminal (DCT). For additional information on this field and downloading FIT data, refer to the ***BASE24-atm NCR 5XXX Device Support Manual***.

For the Diebold 10XX/478X Device Handler process, the PIN encryption key is entered in the PEKEY field on the Financial Institution Table screen that you can access from the Diebold Entry Program Menu. For additional information on this field and downloading FIT data, refer to the ***BASE24-atm Diebold 10XX/478X Device Support Manual***.

Migrating from single-length to double-length keys

Note: The following procedure only applies to NCR devices running NCR firmware. NCR devices running Diebold signature emulation, Tidel devices, or Wincor Nixdorf firmware must already be using double-length keys to support AKDS.

After you have successfully completed all phases of AKDS implementation for an NCR 5XXX terminal, you can use the following procedure to migrate the terminal to double-length keys.

1. Send the Load Key Entry Mode command to the terminal with a value of 3 (Double length with XOR). You can enter this command from the Device Control Terminal (DCT) NCR 5XXX Load Commands and Diebold 10XX Load Commands screen or from a network control facility. Refer to the ***BASE24-atm NCR 5XXX Device Support Manual*** or ***BASE24-atm Diebold 10XX/478X Device Support Manual*** for detailed command syntax.

Changing the key entry mode of an EPP deletes all the DES keys in the encryptor.

An event message is generated when the command is successfully completed.

2. Send the Load Public Key command to the terminal to re-establish the BASE24 system's public key that was erased in step 1 in the terminal's EPP. Refer to the ***BASE24-atm NCR 5XXX Device Support Manual*** or ***BASE24-atm Diebold 10XX/478X Device Support Manual*** for detailed Load Public Key command syntax. Another event message is generated when the processing for this command is successfully completed.
3. On the ACI desktop, read the terminal's current record on the Channel Security Configuration window. Change the Key Length field for the exchange key and inbound key from a value of Single to a value of Double.

If you are using Thales e-Security HSMS, you may also need to change the External Key Block Format field from a value of Variant method to ANSI X9.17 method. If you are using Atalla NSPs, the External Key Block Format field must be set to Standard Atalla. If you are using DEP HSMS, the External Key Block Format field must be set to Standard DEP.

Click Save () to save the changes in the Channel_Security.

4. Warmboot the Transaction Security Services and AKDS processes to reread the Channel Security File (Channel_Security) record into memory. You can warmboot these processes to either reread all Channel_Security records from disk into memory using the [Alter Data Source](#) process control command or just reread the specified Channel_Security terminal record into memory using the [CSEC Refresh](#) process control command.
5. Send the Load Master Key command to the terminal to generate and load a new ATM master key. Refer to the **BASE24-atm NCR 5XXX Device Support Manual** or **BASE24-atm Diebold 10XX/478X Device Support Manual** for detailed Load Master Key command syntax. Another event message is generated when the processing for this command is successfully completed.
6. Generate and download new double-length communications keys (i.e., PIN and/or MAC keys) using the appropriate commands entered from the DCT or a network control facility. As before, check the event message log to make sure the command completed successfully.
7. Bring the ATM back into service. Transactions entered at the ATM should now be processed using the new double-length communications key.

Diebold 10XX/478X terminal implementation procedures

Implementation of remote initial key processing for Diebold 10XX/478X terminals is accomplished in four phases, each of which are described below. This implementation uses digital certificates initially signed by a third-party certificate authority—Digital Signature Trust Co. (DST). The first phase is performed by Diebold and the initial certificate authority, ACI is not involved.

All certificates for the Diebold 10XX/478X implementation follow the International Telecommunications Union (ITU) X.509 standard, **Information Technology - Open Systems Interconnection - The Directory: Public-Key and Attribute Certificate Frameworks**.

Phase 1—initial registration and certification for the EPP

Diebold EPPs create two RSA key pairs when they are first powered up. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. The public keys for each of these two key pairs are signed by a certificate authority. Diebold has chosen Digital Signature Trust Co. (DST) as their certificate authority.

For each public key created, the certificate authority creates two certificates—a primary certificate and a secondary certificate. These certificates contain the EPP's public keys and signatures. Primary and secondary certificates are features of Diebold's disaster recovery procedure, however, BASE24 uses the primary certificate only. The EPP's public key certificates, private signature generation key, private decipherment key, and two additional certificates (primary and secondary) containing the certificate authority's public signature verification key and signature, are stored in the EPP.

Phase 2—initial registration and certification for the BASE24 system

To support automated key distribution for ATMs, you must also generate a root RSA key pair for signature verification and generation for the BASE24 system using a hardware security module. This is accomplished by entering the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window of the ACI desktop. After the key pair is generated, the Diebold certificate authority—Digital Signature Trust Co. (DST)—must verify the BASE24 system's public key and create certificates of the root BASE24 system's public key for a valid date range, signed by the certificate authority's private keys. The BASE24 system's certificate are referred to as the Host Bank server certificates by the certificate authority. The certificate authority also creates certificates of its own public signature verification keys, and root certificate authority public key, signed by the certificate authority's private keys. All of these certificates are returned to you. You must then validate these certificates and store them in the Host Public Key Security File. These tasks are accomplished in the following steps:

1. Generate a root RSA key pair for the BASE24 system for signature verification or generation using the KEYGEN command. Refer to the [“Key Generate \(KEYGEN\) command”](#) topic provided later in this section for detailed information on using this command.
2. Access the Certificate Management window on the ACI desktop (access **Configure > Transaction Security > Automated Key Distribution > Certificate Management**) to retrieve the BASE24 system public key certificate and format it in the manner that the target certificate authority expects it.

When the Certificate Management window is displayed, select the host name, device type (i.e., Diebold), and effective date used to generate the key. The BASE24 system public key certificate is retrieved and displayed in the Root Host Public Key Data field. For Diebold, the key is displayed in PKCS#10 Certificate Request format, Base64 encoded. This is the message format required when sending the public key to Diebold's certificate authority.

3. Copy the BASE24 system public key certificate in the Root Host Public Key Data field and paste it in the appropriate field in the DST-hosted Diebold Host Bank registration pages. Host bank registration involves three phases: authorization, application, and retrieval.

For the authorization phase, you must enter a unique authorization code distributed by Diebold to ensure that you are permitted to receive a Host Bank server certificate. Ask your Diebold System Engineer to contact the Diebold Product Manager for their Remote Key product using an e-mail message. The Diebold Product Manager will send you an e-mail message containing the address of the DST-hosted web site, your authorization code (password), and an HTML demonstration of the registration website. An example of this

e-mail message is shown below. Each time you need new certificates (e.g., for test, production, and certification) you must contact your Diebold System Engineer to request a new authorization code.

This e-mail contains the voucher (Authorization Code) that allows *Financial Institution* to receive host certificates.

The information in this message should be regarded with the appropriate level of security, as this voucher retrieves certificates that allow the holder to be identified as a legitimate ATM host by the EPP.

Website: <https://secure.digsigtrust.com/tsapp/dbold-ssl-start.jsp>

<<https://secure.digsigtrust.com/tsapp/dbold-ssl-start.jsp>>

Authorization Code: 1234567-XXXX-XXXX

Attached is a demonstration of the host registration site. Please use these test html pages to become familiar with the registration site. Unzip the attached file into a directory before beginning at the file named "Start_Here".

For the application phase, you must enter some required and optional contact information (e.g., e-mail address, phone number, and mailing address) for the headquarters of your organization. You must also enter some required and optional contact information about yourself (e.g., your name, job title, and office phone number) and choose a DST passphrase. A DST passphrase is required to renew or revoke your digital certificate, as well as to update your personal information. Finally, you must copy the root BASE24 system public key certificate from the Root Host Public Key Data field on the Certificate Management window and paste it into the CSR field on the DST registration pages.

At this point, the application phase is complete. DST verifies the root BASE24 system public key certificate and creates two certificates of the root BASE24 system public key, signed by the certificate authority's primary and secondary private keys, respectively. The certificate authority also creates two additional certificates. One certificate contains the primary certificate authority's public signature verification key, signed by the certificate authority's primary private key. The second certificate contains the secondary certificate authority's public signature verification key, signed by the certificate authority's secondary private key.

For the retrieval phase, you must copy the following certificates generated or displayed by DST in your browser and paste them in the appropriate fields on the Certificate Management window in the following steps. All certificates are in PKCS #7 Cryptographic Message format. The following table shows you which field on the Certificate Management window to paste each certificate from DST and in which of the following steps you paste the certificate.

DST certificate	Certificate Management field
DST Root CA X3 Certificate	Generate Message Digest > Root Certificate Authority Certificate (step 4) and/or Validate Certificates/Signatures > Root Certificate Authority Certificate (step 7)
DST Primary Sub-CA Certificate	Validate Certificates/Signatures > CA Primary Certificate (step 7)
DST Secondary Sub-CA Certificate	Not applicable. This secondary certificate is never used and can be discarded.
Primary Host Bank Server Certificate	Validate Certificates/Signatures > Host Primary Certificate (step 7)
Secondary Host Bank Server Certificate	Not applicable. This secondary certificate is never used and can be discarded.

4. If you are using a Thales e-Security or Banksys DEP HSM, you can skip to step 7. If you are using an Atalla NSP, create a message digest of the root certificate authority certificate (DST Root CA X3 Certificate). Copy the root certificate authority certificate returned from DST into the Root Certificate Authority Certificate field on the Generate Message Digest tab of the Certificate Management window and click Save (📁). You can click Expand View (🔍) to expand this field if needed. The NSP uses command 126 to create a message digest value that is displayed in the Message Digest field on this tab.
5. Using the SCT, encrypt the value returned in the Message Digest field from the previous step under variant 30 of the MFK. Use the Utilities Menu--> Encrypt Challenge function on the SCT to encrypt the message digest value. You must input the message digest as two 16 byte values (Left Challenge and Right Challenge). The results are two 16 byte values that comprise the root certificate hash value.
6. Copy or enter the 32 byte (16 byte left and 16 byte right) values from the previous step into the Root Certificate Hash Value field on the Validate Certificates/Signatures tab.
7. Copy the root certificate authority certificate, primary certificate authority's public signature verification key certificate, and the primary root BASE24 system public signature verification key certificate returned from DST in step 3 into the appropriate fields on the Validate Certificates/Signatures tab of the Certificate Management window and click Save (📁).

When you click Save (📁), the following processing is performed. This processing is illustrated in the graphic following the steps.

- a. The root certificate authority certificate is validated using the hash value input from step 6 (for Atalla NSPs only), the self-signed signature of the certificate is validated, and the root certificate authority public key is stored in the Host Public Key Security File (HOST_PUBLIC_KEY).

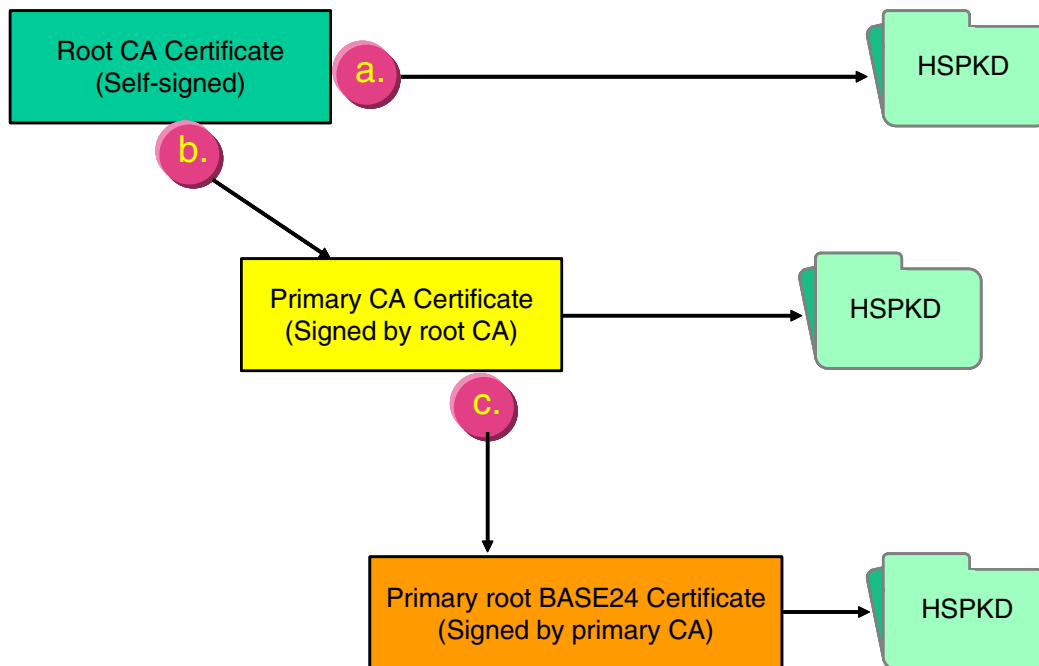
- b. The signature of the primary certificate authority's public signature verification keys is validated using the root certificate authority public key stored in step a. The primary certificate authority public key is stored in the HOST_PUBLIC_KEY.
- c. The signature of the root BASE24 system (host) primary certificate is validated using the primary certificate authority's public signature verification keys stored in step b. The root BASE24 system primary public key signature is stored in the HOST_PUBLIC_KEY.

The validity date range for the certificate is parsed and stored in the HOST_PUBLIC_KEY as the begin and end dates for the certificate. These are the "notBefore" and "notAfter" dates in the PKCS #7 certificate format.

```

30 1E                                validity
17 0D                                notBefore August 13, 2001, 11:50:05am GMT
30 31 30 38 31 33 31 35 35 30 30 35 5A
17 0D                                notAfter August 13, 2011, 11:50:05am GMT
31 31 30 38 31 33 31 35 35 30 30 35 5A

```



Note that the certificates for the certificate authority are parsed and only the public key values are stored in the HOST_PUBLIC_KEY. For the root BASE24 system (host) primary public key, the full certificate signature is stored in the HOST_PUBLIC_KEY.

After the above steps are completed, the following key data is entered into the database for the AKDS process or in the tamper-resistant memory of the hardware security module.

- Root certificate authority public key
- Root BASE24 system's public key signature (signed by the certificate authority's primary private key)

- The BASE24 system's public signature verification key certificate beginning and ending validity dates
- Primary certificate authority's public signature verification key
- The root BASE24 system's private signature generation key

Phase 3—exchange certificates

After the keys and certificates required for automated key distribution have been established at the EPP and the BASE24 system, you can initiate an online exchange of public signature verification key certificates and the EPP public encipherment key certificate. This is accomplished by sending the Load Public Key command (entered as LOADPUBLIC) to the terminal's Diebold 10XX/478XX Device Handler or NCR 5XXX Device Handler process using the Device Control Terminal (DCT) or a network control facility, or by sending an Unsolicited Status message from the terminal with a status indicating that a new network certificate is required. This latter method is only supported on Diebold terminals running Diebold firmware. Refer to the ***BASE24-atm Diebold 10XX/478X Device Support Manual*** or ***BASE24-atm NCR 5XXX Device Support Manual*** for detailed Load Public Key command syntax and processing steps performed during this phase.

The Load Public Key command initiates an online exchange of public signature verification key certificates and the EPP public encipherment key certificate.

When you enter the Load Public Key command, the Diebold 10XX/478X Device Handler or NCR 5XXX Device Handler process closes the specified ATM and sends a configuration request message.

When the Diebold 10XX/478X Device Handler process receives the ATM's configuration response message, it searches the Additional Hardware Configuration Status area for the value "EP0104RK". The presence of this value indicates the ATM has an AKDS-capable EPP installed. If this value is found, the requested load command is allowed to proceed. If this value is not found, the Load Public Key command is not performed. In addition, one or more event messages is generated and the process reopens the ATM.

When the NCR 5XXX Device Handler process receives the ATM's configuration response message, it checks the encryptor type, software ID, and release number in the configuration response message, and the configured terminal line protocol in the ATD, to determine if the ATM is capable of sending the encryptor capabilities and state and whether the device supports certificates. If not, the NCR 5XXX Device Handler process proceeds with the load using the signature-based method (see the "[Phase 4—authenticate and exchange public keys](#)" topic). If so, the NCR 5XXX Device Handler process sends an extended encryption key change (EEKC) message to request the encryptor capabilities and state. The ATM responds with an encryptor initialization data (EID) message that the NCR 5XXX Device Handler checks to determine whether certificate-based downloading can proceed. If not, the Load Public Key command is not performed. In addition, one or more event messages is generated and the process reopens the ATM.

For devices running Wincor Nixdorf firmware, the Device Handler process searches the signature/certificate indicator for the value of 2 (certificate) in the configuration response message. A value of 2 indicates that the ATM supports the Automated Key Distribution

System add-on product. If the ATM is not capable of processing AKDS Distribution commands, the load is discontinued, one or more event messages is generated, and the process reopens the ATM.

During this phase, the AKDS process interacts with the hardware security module to perform the following:

- When the EPP sends its signature to the BASE24 system, optionally verify the EPP serial number itself against valid EPP serial numbers entered in the EPP Serial Number File (EPP_SERIAL_NUM). The value of the ATM-DH-1000-AKDS-SERIAL-NUM-VRFY or ATM-DH-N500-AKDS-SERIAL-NUM-VRFY LCONF param determines whether this additional validation is performed. This optional verification is performed during verification of both the EPP public signature verification key certificate and the EPP public encipherment verification key certificate. The EPP serial numbers in these certificates are appended with the letters V and E, respectively, to indicate which certificate they are used in. Valid EPP serial numbers must be manually entered on the EPP Serial Number Configuration window before this validation is performed. These serial numbers must exactly match the values contained in the certificates. Therefore, you must enter two EPP serial numbers on the entries EPP Serial Number Configuration window for each Diebold 10XX/478X device. For example, if the EPP serial number for a Diebold 10XX/478X device is 05010070, you must enter serial number values of 05010070V and 05010070E, where is a blank space. Information entered on the EPP Serial Number Configuration window is stored in the EPP_SERIAL_NUM. Refer to the [“Data access”](#) topic provided later in this section for more information on the EPP Serial Number Configuration window and the EPP_SERIAL_NUM.
- Retrieve the root BASE24 system's primary public signature verification key from the Host Public Key Security File (HOST_PUBLIC_KEY) and generate a host random nonce. This key and the host random nonce are sent to the ATM along with a request for the terminal's EPP public signature verification key certificate. The generated host random nonce is stored in the CHAN_PKEY_SEC.
- Verify the signature of the EPP's public signature verification key certificate using the primary certificate authority public key retrieved from the HOST_PUBLIC_KEY.
- Validate the EPP's random nonce from the EPP's public signature verification key certificate using the host random nonce from the CHAN_PKEY_SEC.
- Validate that the current system date falls within the beginning and ending validity dates for the EPP's public signature verification key certificate.
- If the signature, EPP random nonce, and validity dates are valid for the EPP's public signature verification key certificate, the process stores them in the Channel Public Key Security File (CHAN_PKEY_SEC).
- Verify the signature of the EPP's public encipherment key certificate using the primary certificate authority public key retrieved from the HOST_PUBLIC_KEY.
- Validate the EPP's random nonce from the EPP's public encipherment key certificate using the host random nonce from the CHAN_PKEY_SEC.
- Validate that the current system date falls within the beginning and ending validity dates for the EPP's public encipherment key certificate.
- If the signature, EPP random nonce, and validity dates are valid for the EPP's public encipherment key certificate, the process stores them in the CHAN_PKEY_SEC.

At this point, the following key information is stored in the EPP:

- The EPP's primary and secondary public signature verification key certificates (signed by the certificate authority's primary and secondary private keys, respectively)
- The EPP's private signature generation key
- The EPP's primary and secondary public encipherment key certificates (signed by the certificate authority's primary and secondary private keys, respectively)
- The EPP's private decipherment key
- The EPP's random nonce
- The certificate authority's primary and secondary public signature verification key certificates (signed by the certificate authority's primary and secondary private keys, respectively)
- The primary root BASE24 system's public signature verification key certificate (signed by the certificate authority's primary private key)
- The primary root BASE24 system's public signature verification key certificate beginning and ending validity dates
- The BASE24 system's (host) random nonce

At this point, the following key data is entered into the database for the AKDS process or in the tamper-resistant memory of the hardware security module.

- The root certificate authority's public key
- The primary root BASE24 system public signature verification key (signed by the certificate authority's primary private key)
- The primary root BASE24 system's public signature verification key certificate beginning and ending validity dates
- The primary certificate authority's public signature verification key
- The BASE24 system's private signature generation key
- The BASE24 system's (host) random nonce
- The EPP's public signature verification key (signed by the certificate authority's primary private key)
- The EPP's public signature verification key certificate beginning and ending validity dates
- The EPP's public encipherment key (signed by the certificate authority's primary private key)
- The EPP's public encipherment key certificate beginning and ending validity dates
- The EPP's random nonce

Phase 4—download the master key

After an EPP has been successfully authenticated and exchanged public key certificates with the BASE24 system, you can securely download the ATM master key to the terminal. You can initiate the download by either entering the Load Master Key command (entered as

LOADMASTER) from the DCT or a network control facility, or by sending an Unsolicited Status message from the ATM terminal to the BASE24 system with a status indicating that remote key transport is required. This latter method is only supported on Diebold devices running Diebold firmware. Refer to the ***BASE24-atm Diebold 10XX/478X Device Support Manual*** or ***BASE24-atm NCR 5XXX Device Support Manual*** for detailed Load Master Key command syntax and processing steps performed during this phase.

When you enter the Load Master Key command, the Diebold 10XX/478X Device Handler or NCR 5XXX Device Handler process closes the specified ATM and sends a configuration request message.

When the Diebold 10XX/478X Device Handler process receives the ATM's configuration response message, it searches the Additional Hardware Configuration Status area for the value "EP0104RK". The presence of this value indicates the ATM has an AKDS-capable EPP installed. If this value is found, the Load Master Key command is allowed to proceed. If this value is not found, the Load Master Key command is not performed. In addition, an event message is generated and the process reopens the ATM.

When the NCR 5XXX Device Handler process receives the ATM's configuration response message, it checks the encryptor type, software ID, and release number in the configuration response message, and the configured terminal line protocol in the ATD, to determine if the ATM is capable of sending the encryptor capabilities and state and whether the device supports certificates. If not, the NCR 5XXX Device Handler process proceeds with the load using the signature-based method (see the ["Phase 5—download the master key"](#) topic). If so, the NCR 5XXX Device Handler process sends an extended encryption key change (EEKC) message to request the encryptor capabilities and state. The ATM responds with an encryptor initialization data (EID) message that the NCR 5XXX Device Handler checks to determine whether certificate-based downloading can proceed. If not, the Load Public Key command is not performed. In addition, one or more event messages is generated and the process reopens the ATM.

For devices running Wincor Nixdorf firmware, the Device Handler process searches the signature/certificate indicator for the value of 2 (certificate) in the configuration response message. A value of 2 indicates that the ATM supports the Automated Key Distribution System add-on product. If the ATM is not capable of processing AKDS Distribution commands, the load is discontinued, one or more event messages is generated, and the process reopens the ATM.

If the ATM is set to use single-length keys and BASE24 downloads a double-length master key, the ATM will automatically switch to double-length keys. If the ATM is set to use double-length keys and BASE24 downloads a single-length master key, the ATM will automatically switch to single-length keys. There is no need to configure the key length at the ATM prior to remotely transporting the master key.

During this phase, the AKDS process interacts with the hardware security module to generate a new master key, encrypt it under the EPP's public encipherment key retrieved from the Channel Public Key Security File (CHAN_PKEY_SEC), and sign it using the BASE24 system's private signature generation key retrieved from the Host Public Key Security File (HOST_PUBLIC_KEY).

At this point, both the BASE24 system and the EPP have the terminal's master key stored in their respective databases. Working keys (i.e., the PIN encryption key and MAC key) can now be securely downloaded to the EPP.

After the AKDS process has generated a new master key and stored it in the TSS database, any errors in subsequent processing (e.g., a failure of the message sent to the ATM to load the new master key or the ATM rejecting the new master key), causes the AKDS process to back out the new master key (i.e., restore it to its previous value). No operator intervention is required to destroy the new master key.

Financial Institution Table (FIT) data

The PIN encryption key in the FIT is encrypted under the master key. If a new master key is generated and a PIN encryption key exists in the FIT for local PIN verification, you must manually translate the PIN encryption key from encryption under the old master key to encryption under the new master key, replace the PIN encryption key cryptogram in the FIT, and download the updated FIT data to the ATM terminal.

For the Diebold 10XX/478X Device Handler process, the PIN encryption key is entered in the PEKEY field on the Financial Institution Table screen that you can access from the Diebold Entry Program Menu. For additional information on this field and downloading FIT data, refer to the ***BASE24-atm Diebold 10XX/478X Device Support Manual***.

For the NCR 5XXX Device Handler process, the PIN encryption key is entered in the PEKEY (PIN ENCRYPTION KEY) field on the Financial Institution Table screen that you can access from the NCR 5XXX Configuration File Menu in the Device Control Terminal (DCT). For additional information on this field and downloading FIT data, refer to the ***BASE24-atm NCR 5XXX Device Support Manual***.

Migrating from single-length to double-length keys

Note: The following procedure only applies to Diebold devices running Diebold firmware. Diebold devices running NCR certificate emulation or Wincor Nixdorf firmware must already be using double-length keys to support AKDS.

After you have successfully completed all phases of AKDS implementation for a Diebold 10XX/478X terminal, you can use the following procedure to migrate the terminal to double-length keys.

1. On the ACI desktop, read the terminal's current record on the Channel Security Configuration window. Change the Key Length field for the exchange key and inbound key from a value of Single to a value of Double.

If you are using Thales e-Security HSMS, you may also need to change the External Key Block Format field from a value of Variant method to ANSI X9.17 method. If you are using Atalla NSPs, the External Key Block Format field must be set to Standard Atalla. If you are using DEP HSMS, the External Key Block Format field must be set to Standard DEP.

Click Save () to save the changes in the Channel_Security.

2. Warmboot the Transaction Security Services and AKDS processes to reread the Channel Security File (Channel_Security) record into memory. You can warmboot these processes to either reread all Channel_Security records from disk into memory using the [Alter Data Source](#) process control command or just reread the specified Channel_Security terminal record into memory using the [CSEC Refresh](#) process control command.
3. Send the Load Master Key command to the terminal to generate and load a new ATM master key. Refer to the **BASE24-atm Diebold 10XX/478X Device Support Manual** or **BASE24-atm NCR 5XXX Device Support Manual** for detailed Load Master Key command syntax. Another event message is generated when the processing for this command is successfully completed.
4. Generate and download new double-length communications keys (i.e., PIN and/or MAC keys) using the appropriate commands entered from the DCT or a network control facility. As before, check the event message log to make sure the command completed successfully.
5. Bring the ATM back into service. Transactions entered at the ATM should now be processed using the new double-length communications key.

Triton terminal implementation procedures

Implementation of remote key transport processing for Triton terminals is accomplished in three phases, each of which are described below. This implementation uses digital certificates initially signed by a third-party certificate authority—Triton, with VeriSign acting as the service provider. The first phase is performed by Triton and the initial certificate authority, ACI is not involved.

All certificates for the Triton implementation follow the International Telecommunications Union (ITU) X.509 standard, **Information Technology - Open Systems Interconnection - The Directory: Public-Key and Attribute Certificate Frameworks**.

Phase 1—initial registration and certification for the EPP

Triton EPPs create two RSA key pairs when they are first powered up. One key pair is used for signature verification and generation. The second key pair is used for encipherment and decipherment. The public keys for each of these two key pairs are signed by a certificate authority.

For each public key created, the certificate authority creates a certificate. These certificates contain the EPP's public keys, signatures, and the EPP's unique identifier.

The EPP's public key certificates, private signature generation key, private decipherment key, and two additional certificates containing the certificate authority's public signature verification key or public encipherment and decipherment key, signatures, and unique identifier for the certificate authority are stored in the EPP.

Phase 2—initial registration and certification for the BASE24 system

To support automated key distribution for ATMs, you must also generate a root RSA key pair for signature verification and generation for the BASE24 system using a hardware security module. This is accomplished by entering the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window of the ACI desktop. After the key pair is generated, the Triton certificate authority—Triton—must verify the BASE24 system's public key and create a certificate of the root BASE24 system's public key for a valid date range, signed by the certificate authority's private key. VeriSign acts as the service provider for this processing. The BASE24 system's certificate is referred to as the bank host certificate by the certificate authority. The certificate authority also creates a self-sign certificate of its own public signature verification key. Both of these certificates are returned to you. You must then validate these certificates and store them in the Host Public Key Security File. These tasks are accomplished in the following steps:

Note: Before you generate a root RSA key pair for the BASE24 system, you must obtain a host (BASE24 system) unique identifier generated by Triton. You must enter this value for the common-name variable in the KEYGEN command. Contact your Triton representative to start the process of getting your host ID issued.

Refer to the ***Applying for a Triton RKT Host Certificate*** document for detailed procedures on submitting your BASE24-eps system's public key and retrieving the appropriate certificates from Triton. To obtain a copy of this document, you must first execute a TDL (Triton Dynamic Language) license agreement with Triton. Contact your Triton representative for more information.

1. Generate a root RSA key pair for the BASE24 system for signature verification or generation using the KEYGEN command. Refer to the [“Key Generate \(KEYGEN\) command”](#) topic provided later in this section for detailed information on using this command.
2. Access the Certificate Management window on the ACI desktop (access **Configure > Transaction Security > Automated Key Distribution > Certificate Management**) to retrieve the BASE24 system public key certificate and format it in the manner that the target certificate authority expects it.

When the Certificate Management window is displayed, select the host name, device type (i.e., Triton), and effective date used to generate the key. The BASE24 system public key certificate is retrieved and displayed in the Root Host Public Key Data field. For Triton, the key is displayed in PKCS#10 Certificate Request format, Base64 encoded. This is the message format required when sending the public key to Triton's certificate authority.

3. Copy the BASE24 system public key certificate in the Root Host Public Key Data field and paste it into a text file with a .csr file extension (e.g., <your-org-name>.csr). Submit your host public key certificate to Triton using Triton's secure web portal with VeriSign as described in Triton's ***Applying for a Triton RKT Host Certificate*** document. Make sure you remove any end-of-line characters as well as the “-----BEGIN NEW CERTIFICATE REQUEST-----” and “-----END NEW CERTIFICATE REQUEST-----” text from the certificate you submit.

When completing the CSR enrollment form, make sure you enter your assigned 16-digit host ID in the Host ID/Name field, your email address in the E-mail Address field, and the URL you used for the org-name variable when generating the host RSA key pair in the Company/Agency/Org field.

Triton will respond with an email that includes your signed host key certificate in text and in an attached cert.p7b file. Your assigned 16-digit host ID appears in the email salutation. If the Triton self-signed certificate is not included, you may need to send a separate request to Triton support to get it.

Note: Be sure to use the BASE64 certificate embedded in the body of the email rather than the attached .p7b file. The format in the attached .p7b file is incorrect.

For the retrieval phase, you must reformat and copy the following certificates generated or returned by Triton and paste them in the appropriate fields on the Certificate Management window in the following steps. All certificates must be in PKCS #7 Cryptographic Message format before you paste them into BASE24. The following table shows you which field on the Certificate Management window to paste each certificate from Triton and in which of the following steps you paste the certificate.

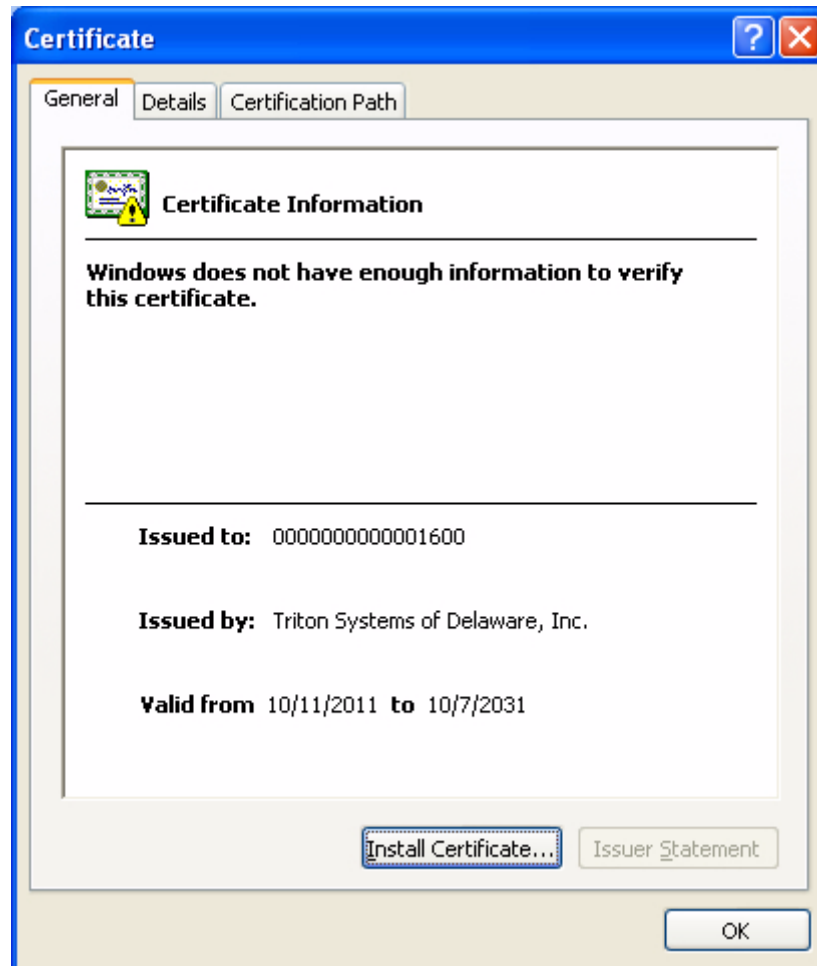
Note: Before you can paste the bank host certificate and Triton CA certificate into BASE24, you must perform the steps listed below the table to reformat the certificate into PKCS #7 Cryptographic Message format.

Triton certificate	Certificate Management field
Triton self-signed CA certificate Note: The self-signed certificate must be in text format, rather than p7b format, and contain signData.	Generate Message Digest > Root Certificate Authority Certificate (step 4) and/or Validate Certificates/Signatures > Root Certificate Authority Certificate (step 7)
Bank host certificate	Validate Certificates/Signatures > Host Primary Certificate (step 7)

a. Reformat the bank host certificate.

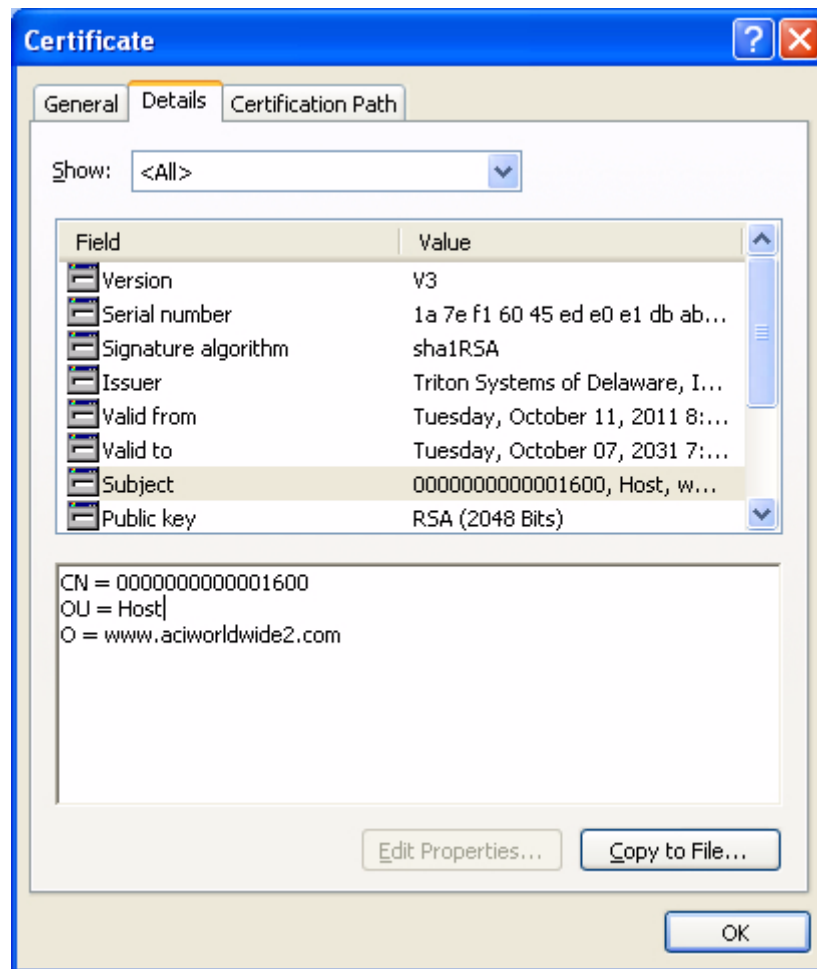
- 1) Save the bank host certificate from the email to a memorable name under Windows, such as "HOST_TRITON_ATM from converted 3-oct.cer".

- 2) Double-click the file to start the Microsoft Management Console - Certificate view/accept utility.¹ Although this utility is normally used to install a certificate to be used for SSL communication, you are using it just to convert the certificate into a format compatible for use with BASE24.

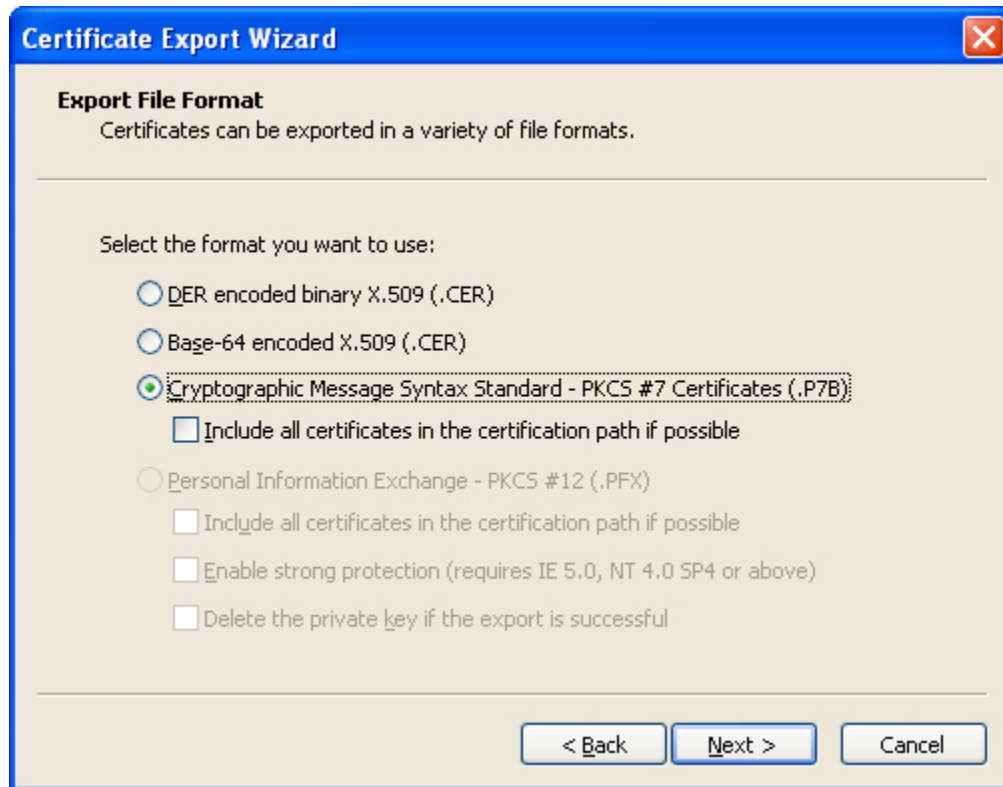


¹ACI Worldwide makes no warranties about this third party utility.

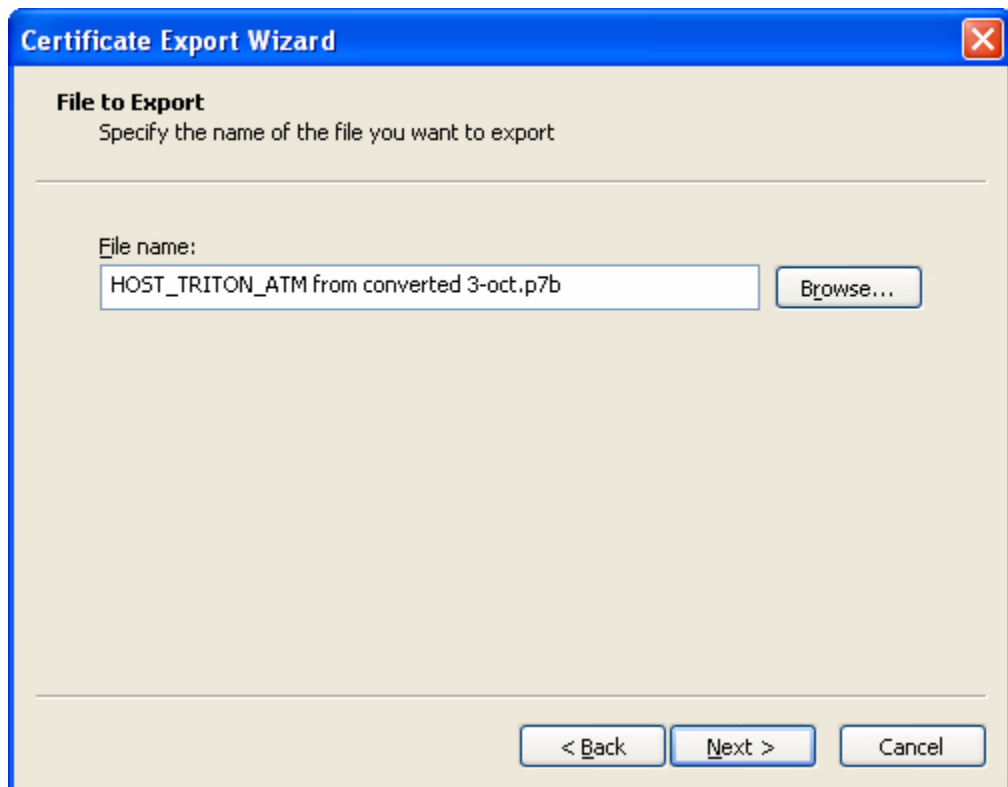
- 3) Select Details and Subject to confirm the certificate is the one you expected to receive from Triton.



- 4) Select Copy to File to activate the export and conversion options. Select the "Cryptographic Message Syntax Standard - PKCS #7 Certificates (.P7B)" option and click Next.



- 5) Specify the name of the file you want to export. Again, give it memorable name under Windows (e.g., HOST_TRITON_ATM from converted 3-oct.p7b) with a .p7b file extension and click Next..

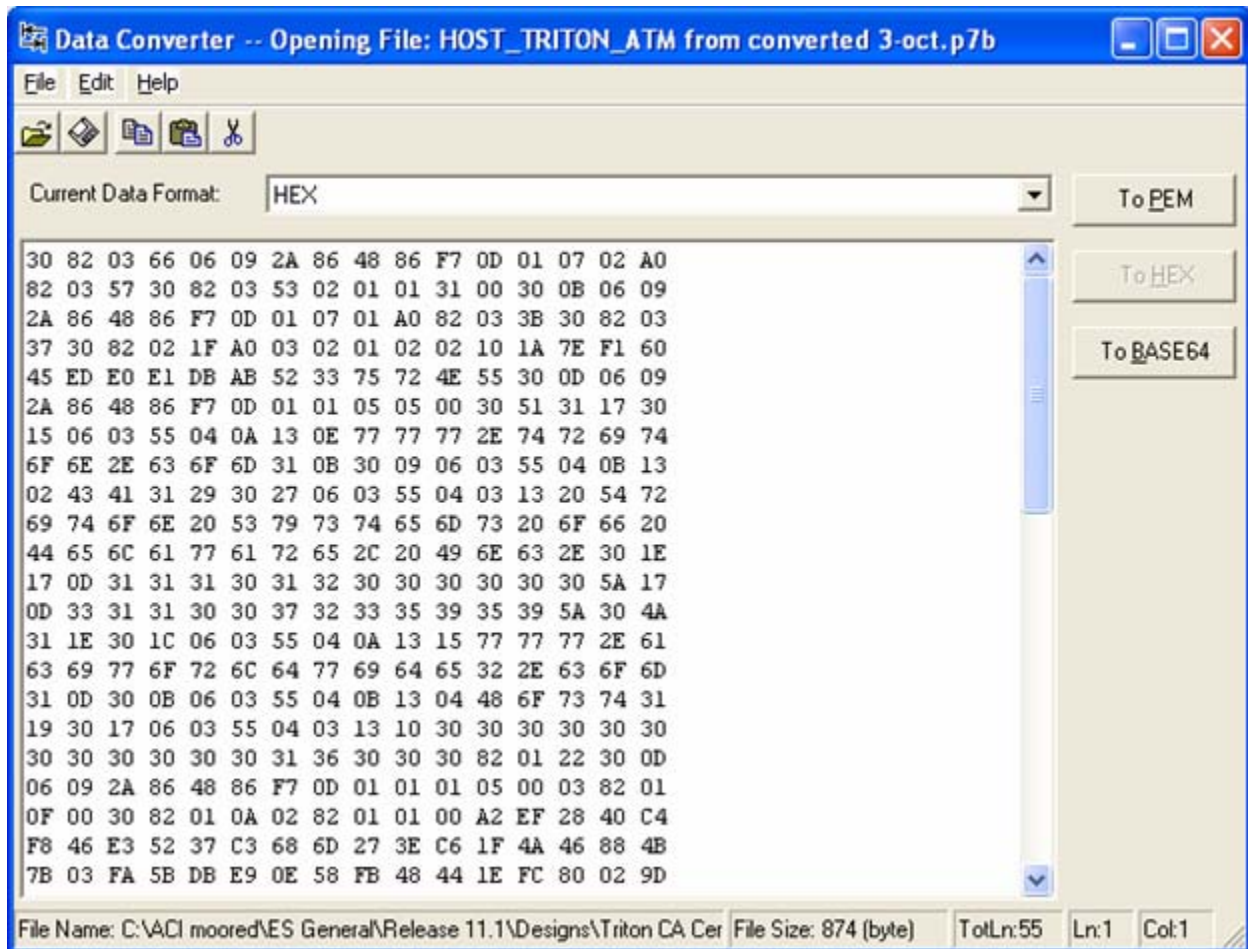


- 6) The resulting file is saved in the P7B format, but is encoded in binary. You can then use a utility such as the ASN.1 Editor² to convert the file from binary to PEM (character encoded) form, which is the format in which BASE24-eps expects to receive it.

The URL for downloading a free copy of ASN.1 is <http://lipingshare.com/Asn1Editor/> for Asn1EditorSetup.msi.

²ACI Worldwide makes no warranties about this third party utility.

- 7) Once installed, you can use the ASN.1 Editor to confirm the certificate is correctly encoded. You can then use the DataConverter utility that comes with the ASN.1 Editor to change the format from binary to PEM format. BASE24 expects the certificate to be in PEM format (.pem).



- 8) Click on the To PEM button and then save the resulting file with a memorable name such as -- HOST_TRITON_ATM from converted 3-oct-PEM.p7b
- 9) This provides a result similar to the following. You must copy and paste the encoded part between the BEGIN and END markers into the Validate Certificates/Signatures > Host Primary Certificate field on the Certificate Management window in step 7.

```
-----BEGIN HOST_TRITON_ATM from converted 3oct.p7b-----
MIIDZgYJKoZIhvcNAQcCoIIDVzCCA1MCAQExADALBgkqhkiG9w0BBwGgggM7MIID
NzCCAh+gAwIBAgIQGo7xYEXt40Hbq1IzdXJOVTANBgkqhkiG9w0BAQUFADBMRcw
FQYDVQKKEw53d3cudHJpdG9uLmNvbTELMakGA1UECxMCQ0ExKTAnBgNVBAMTIFry
aXRvbiBTeXN0ZW1zIG9mIERlbGF3YXJLLCBJbmMuMB4XDTEwMTAxMjAwMDAwMFoX
DTMxMTAwNzIzNTk1OVowSjEeMBwGA1UEChMVd3d3d3LmFjaXZvcmxkd2lkZTIuY29t
MQ0wCwYDVQQLewRlbnN0MRkwFwYDVQQDEwAwMDAwMDAwMDAwMDAxNjAwMIIBIjAN
BgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAou8oQMT4RuNSN8NobSc+Xh9KRohL
ewP6W9vpDlj7SEqe/IACndASdDDKDR0p0eNaPZEzcZ2cEk9XWkI2ZYx8uSLFURnL
gzSw00pxCrYX6ps+fNGHlvq+GsYbnx8bNU97vQXjjEDEMCxhFa3ntkeo/OoZwRZK
```

```
dM6DACCrmAXsK3c9XIOPiWL8as3j4P3gVTTIV7d/mVoeki6HekasosFC1u2KrNYR
r7E39npQV4Vd19cM8af8aDISnlzZ4YINhd8c+CnWF7kP8cw1GRmkieYsQGupvJno
rnBwfnwV2HEoYkNM7bNeF06N+toV9l2yt3FewY4lYz1zWwkQSIPqRTxx7QIDAQAB
oxIwEDA0BgNVHQ8BAf8EBAMgB4AwDQYJKoZIhvcNAQEFBQADggEBAJmdKQC5yCpg
hLxN5P4MKmDH/Dj9Asl7tVqvY0uR4JMtTM8Shanhg7ZgibtWyNDEJJg9cyTbvkp7
NcArHmSKnmX1euQ3WJ5p7G095NDm4tSH85/di0Fh7gF9RnZ2S06Iyc2CTxQZSPlQ
nv4KJgm19C+sHF/cpA7XjtkH1NokWr2+aYj3KSvumsL8kJccUMMkTWStqKF4b2c6
fJZ9d2kjVyiydtaCPgLRRCbf9f8SAKn+7+6vIwrhFSZFhNVTsgCrc1MBrn+Pf34
R4W2/YQ0fc1VAX0J5QIYR50zrIhFxrXTi0YaeQHQocT/cL/vew8HmkgTdorRogMU
gPY8EdF7KaExAA==
```

```
-----END HOST_TRITON_ATM from converted 3oct.p7b-----
```

b. Reformat the Triton CA certificate.

- 1) Make sure you receive the Triton CA certificate in a suitable format, such as P7B.
- 2) Save the certificate to a file with a memorable name such as -- TritonCaCert.p7b
- 3) Use the ASN.1 associated tool DataConverter to change it from binary to PEM format (same as above for the bank host certificate), and save to another memorable file name (e.g., TritonCaCert-PEM.p7b).

This provides a result similar to the following. You must copy and paste the encoded part between the BEGIN and END markers into the Generate Message Digest > Root Certificate Authority Certificate field on the Certificate Management window in step 4 and/or into the Validate Certificates/Signatures > Root Certificate Authority Certificate on the Certificate Management window in step 7.

```
-----BEGIN TritonCaCert.p7b-----
```

```
MIIDzwYJKoZIhvcNAQcCoIIDd62cA7wCAQExADALBgkqhkiG9w0BBwGgggOkMIIID
oDCCAoigAwIBAgIQKlyQxEZNoqDYdMPS9lOQ7zANBgkqhkiG9w0BAQUFADBMRcw
FQYDVQKKEw53d3cudHJpdG9uLmNvbTELMakGA1UECxCQ0ExKTAnBgNVBAMTIFRy
aXRvbiBTeXN0ZW1zIG9mIERlbGF3YXJlLCBjb250b250b250b250b250b250b250
DTMyMTExMTIzNTk1OVowUTEYVBUUA1UEChM0d3d3LnRyaXRvbi5jb20xXzAJBgNV
BAsTAkNBMSkwJWYDVQQDEyBUcm10b24gU3lzdGVtcyBvZiBEZWxhd2FyZSwgSW5j
LjCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAMiJYkYaZDoYRy1qGXyh
19Hq0bu0xeYf1Npb6GdGz0dvZfXHEASX4faZAv2UWxf1MuUI723jm8NkKV2g2W4/
CmLJ0BNkytQrLP7yvsZPRktK+7uMoMzyAXeN16aScEUsbwP/o0HtM9liJZnsCwfb
maLkic9h7utipJUpIc5IPXDK8d7CR0DXL6alaCBqmvz4+RS2Xkg7qq+UhuAdl0i0
Mek6qVw8xP70yYmAP1BfspEja2ubjvLhgj2Jc9UjRgDU8v9Hpr4c lyuH7iHuVUwH
mYX08uDStZZqEA5aqcLlscRHEDB6X/bpJR48rfrvgYJIhLEBPrIG3jBIFtGzbPWF
/eECAwEAAAN0MHIwEgYDVR0TAQH/BAGwBgEB/wIBADA0BgNVHQ8BAf8EBAMCAQYw
LQYDVRR0RBCYwJKQIMCAxHjAcBgNVBAMTFVByaXZhdGVMYWJlbdQTMjA00C01NjAd
BgNVHQ4EFgQUo1T6IKdSPAgCr7GL/ECaWlUHBASwDQYJKoZIhvcNAQEFBQADggEB
ABYYfoxRkdVPrqV+++ffNH+tYsA607nhuA7U50mKgMRNxsu8XmG+ZXHqT3E5hqt1D
h9qd6r/uwoV0Kb3oUabtNbpPBvXXyJSk7XH4rsp5a1KoWqq6yET8v9ZUv4+AqHjq
IQ9iRSpVIdkBQrjTfVlnk02p7zxkqnCsbxXHaeJBpDQF6otg3wK7NvbrgnuBWIZB
1Wnaqf0LF9PR/6oXhquFymxaUwm8qhUN7n8sFSP7eo59TwYQk0gqrHmnYJjaS4Ag
p9Rq6p+A+6Pzb+Y5zN2a6Yh9d27Mu0jky8c0KSyT5uk5HVDJN+yEShcLkGPoKZS0
w46SoN6CJh9uAgCBFL/MxR8xAA==
```

```
-----END TritonCaCert.p7b-----
```

4. If you are using a Thales e-Security HSM, you can skip to step 7. If you are using an Atalla NSP, create a message digest of the root certificate authority certificate (Triton self-signed CA Certificate). Copy the root certificate authority certificate returned from Triton into the Root Certificate Authority Certificate field on the Generate Message Digest tab of

the Certificate Management window and click Save (📁). You can click Expand View (🔍) to expand this field if needed. The NSP uses command 126 to create a message digest value that is displayed in the Message Digest field on this tab.

5. Using the SCT, encrypt the value returned in the Message Digest field from the previous step under variant 30 of the MFK. Use the Utilities Menu--> Encrypt Challenge function on the SCT to encrypt the message digest value. You must input the message digest as two 16 byte values (Left Challenge and Right Challenge). The results are two 16 byte values that comprise the root certificate hash value.
6. Copy or enter the 32 byte (16 byte left and 16 byte right) values from the previous step into the Root Certificate Hash Value field on the Validate Certificates/Signatures tab.
7. Copy the root certificate authority certificate and bank host certificate returned from Triton in step 3 into the appropriate fields on the Validate Certificates/Signatures tab of the Certificate Management window and click Save (📁).

When you click Save (📁), the following processing is performed. This processing is illustrated in the graphic following the steps.

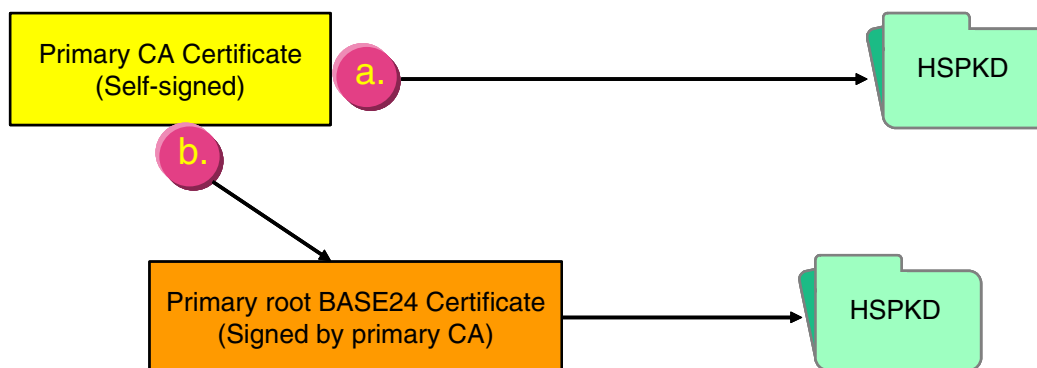
- a. The root certificate authority certificate is validated using the hash value input from step 9 (for Atalla NSPs only), the self-signed signature of the certificate is validated, and the certificate authority public key is stored in the Host Public Key Security File (HOST_PUBLIC_KEY).
- b. The signature of the BASE24 system (host) primary certificate is validated using the primary certificate authority's public signature verification keys stored in step a. The root BASE24 system primary public key signature is stored in the HOST_PUBLIC_KEY.

The validity date range for the certificate is parsed and stored in the HOST_PUBLIC_KEY as the begin and end dates for the certificate. These are the "notBefore" and "notAfter" dates in the PKCS #7 certificate format.

```

30 1E                                validity
17 0D                                notBefore August 13, 2001, 11:50:05am GMT
30 31 30 38 31 33 31 35 35 30 30 35 5A
17 0D                                notAfter August 13, 2011, 11:50:05am GMT
31 31 30 38 31 33 31 35 35 30 30 35 5A

```



Note that the certificate for the certificate authority is parsed and only the public key value is stored in the HOST_PUBLIC_KEY. For the root BASE24 system (host) primary public key, the full certificate signature is stored in the HOST_PUBLIC_KEY.

After the above steps are completed, the following key data is entered into the database for the AKDS process or in the tamper-resistant memory of the hardware security module.

- Certificate authority public key
- Root BASE24 system's public key signature (signed by the certificate authority's private key)
- The BASE24 system's public signature verification key certificate beginning and ending validity dates
- Certificate authority's public signature verification key
- The root BASE24 system's private signature generation key

Phase 3—exchange certificates and download keys

Before you can automatically distribute (download) an ATM master key to the EPP of an ATM, the EPP and the BASE24 system must authenticate each other and exchange their public key certificates. This processing is accomplished by either entering the Load All Keys command for the ATM or having a technician perform a remote key load from the management functions menu at the ATM.

Because the Triton EPP can send its signature verification and encryption/decryption certificates to the BASE24 system in any order, the Transaction Security component uses the values of the EPP_ED_CERT_SUBJ and EPP_SV_CERT_SUBJ Environment attributes to identify the certificates. The component compares the attribute value to the subject distinguished name from the PKCS #7 certificate. Triton sets this text in the certificate's subject's organizational unit name. The value of the EPP_ED_CERT_SUBJ attribute identifies the encryption/decryption certificate. The value of the EPP_SV_CERT_SUBJ attribute identifies the signature verification certificate.

Note: The exchange and authentication of public key certificates is performed only once the first time you initiate a download of all keys to a Triton ATM. Thereafter, only master and working keys are downloaded when entering the Load All Keys command.

The Load All Keys command (entered as LOADALLKEYS) is sent to the terminal's Triton Device Handler process using a network control facility. The Load All Keys command causes the Triton Device Handler process to set the r FID to a value of "a" in the next financial or administrative response message, in the response to a download request, in a host totals response, or in a reversal. This value indicates to the terminal to go temporarily out of service after the call and initiate an RKT sequence. When the Triton terminal receives the "ra" FID (or when an ATM technician performs a remote key load manually at the ATM), the ATM initiates the following processing:

- Enters an out-of-service state
- Authenticates and exchanges public keys with the BASE24 system by sending an RKT request message to the BASE24 system. The BASE24 system sends the root BASE24 system's public key certificate to the ATM. The ATM then sends the EPP's signature verification and encryption certificates to the BASE24 system in any order.

Note: The above processing is only performed once, the first time RKT is initiated. On all subsequent initiations, the authentication and exchange of public keys is bypassed.

- Requests downloads of new ATM master keys
- Requests a download of a new PIN encryption (working) key
- Requests a download of a new MAC (working) key (if message authentication is used)
- Enters an in-service state if the above processing is successful

Authenticate and exchange public keys. During this phase, the AKDS process interacts with the hardware security module to perform the following:

- When the EPP sends its signature to the BASE24 system, optionally verify the EPP serial number itself against valid EPP serial numbers entered in the EPP Serial Number File (EPP_SERIAL_NUM). The value of the ATM-DH-TRIT-AKDS-SERIAL-NUM-VRFY param determines whether this additional validation is performed. This optional verification is performed during verification of both the EPP public signature verification key certificate and the EPP public encipherment verification key certificate. Valid EPP serial numbers must be manually entered on the EPP Serial Number Configuration window before this validation is performed. These serial numbers must exactly match the values contained in the certificates. The same serial number is used to validate both the EPP public signature verification key certificate and the EPP public encipherment verification key certificate from a Triton device. Information entered on the EPP Serial Number Configuration window is stored in the EPP_SERIAL_NUM. Refer to the [“Data access”](#) topic provided later in this section for more information on the EPP Serial Number Configuration window and the EPP_SERIAL_NUM.
- Retrieve the root BASE24 system's primary public signature verification key from the Host Public Key Security File (HOST_PUBLIC_KEY) and generate a host random nonce. This key and the host random nonce are sent to the ATM along with a request for the terminal's EPP public signature verification key certificate. The generated host random nonce is stored in the CHAN_PKEY_SEC.
- Verify the signature of the EPP's public signature verification key certificate using the primary certificate authority public key retrieved from the HOST_PUBLIC_KEY.
- Validate the EPP's random nonce from the EPP's public signature verification key certificate using the host random nonce from the CHAN_PKEY_SEC.
- Validate that the current system date falls within the beginning and ending validity dates for the EPP's public signature verification key certificate.
- If the signature, EPP random nonce, and validity dates are valid for the EPP's public signature verification key certificate, the process stores them in the Channel Public Key Security File (CHAN_PKEY_SEC).
- Verify the signature of the EPP's public encipherment key certificate using the primary certificate authority public key retrieved from the HOST_PUBLIC_KEY.
- Validate the EPP's random nonce from the EPP's public encipherment key certificate using the host random nonce from the CHAN_PKEY_SEC.
- Validate that the current system date falls within the beginning and ending validity dates for the EPP's public encipherment key certificate.
- If the signature, EPP random nonce, and validity dates are valid for the EPP's public encipherment key certificate, the process stores them in the CHAN_PKEY_SEC.

The Triton Device Handler process sends the BASE24 system public key and signature to the ATM's EPP, where they are verified and stored.

The ATM sends its EPP public signature verification and encryption keys to the Triton Device Handler process. The Triton Device Handler process sends them to the AKDS process where they are verified and stored.

At this point, the following key information is stored in the EPP:

- The EPP's public signature verification key certificate (signed by the certificate authority's private key)
- The EPP's private signature generation key
- The EPP's public encipherment key certificate (signed by the certificate authority's private key)
- The EPP's private decipherment key
- The EPP's random nonce
- The certificate authority's public signature verification key certificate (signed by the certificate authority's private key)
- The root BASE24 system's public signature verification key certificate (signed by the certificate authority's private key)
- The root BASE24 system's public signature verification key certificate beginning and ending validity dates
- The BASE24 system's (host) random nonce

At this point, the following key data is entered into the database for the AKDS process.

- The root certificate authority's public key
- The root BASE24 system public signature verification key (signed by the certificate authority's private key)
- The root BASE24 system's public signature verification key certificate beginning and ending validity dates
- The certificate authority's public signature verification key
- The BASE24 system's private signature generation key
- The BASE24 system's (host) random nonce
- The EPP's public signature verification key (signed by the certificate authority's private key)
- The EPP's public signature verification key certificate beginning and ending validity dates
- The EPP's public encipherment key (signed by the certificate authority's private key)
- The EPP's public encipherment key certificate beginning and ending validity dates
- The EPP's random nonce

Download a new ATM PIN master key. If the public keys are successfully exchanged and authenticated during the first initiation of RKT processing and on all subsequent initiations, the AKDS process interacts with the hardware security module to generate a new PIN master

key encrypted under the EPP's public key and signs it using the BASE24 system private key. Triton ATMs use separate master keys for PIN and MAC working keys. The new PIN master key and associated check digits are stored in the Channel Security File (Channel_Security). The new PIN master key and signature are also sent to the ATM.

Download a new PIN encryption (working) key. Next, the Triton Device Handler process requests the AKDS process to generate a new PIN working key. The AKDS process interacts with the hardware security module to generate and download a new PIN working key (inbound key) encrypted under the ATM's PIN master key (exchange key). The new PIN working key and associated check digits are stored in the Channel Security File (Channel_Security). The new PIN working key is also sent to the ATM.

Note: The Triton Device Handler process always downloads a new PIN working key, regardless of the setting of the PIN ENCRYPT TYPE field on TRITON MINIATM SECURITY INFORMATION screen 9 of the BASE24-atm Terminal Data files (ATD).

Download a new ATM MAC master key. If the MAC SUPPORT TYPE field on TRITON MINIATM SECURITY INFORMATION screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM,, the AKDS process interacts with the hardware security module to generate a new MAC master key encrypted under the EPP's public key and signs it using the BASE24 system private key. Triton ATMs use separate master keys for PIN and MAC working keys. The new MAC master key and associated check digits are stored in the Channel Security File (Channel_Security). The new MAC master key and signature are also sent to the ATM.

Download a new MAC (working) key. If the MAC SUPPORT TYPE field on TRITON MINIATM SECURITY INFORMATION screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM, the Triton Device Handler process requests the AKDS process to generate a new MAC working key. The AKDS process interacts with the hardware security module to generate and download a new MAC working key (inbound key) encrypted under the ATM's MAC master key (exchange key). The new MAC working key and associated check digits are stored in the Channel Security File (Channel_Security). The new MAC working key is also sent to the ATM.

At this point, both the BASE24 system and the EPP have the terminal's master keys, PIN working key, and MAC working key stored in their respective databases.

Shared Wincor Nixdorf signature terminal implementation procedures

Implementation of remote initial key processing for Wincor Nixdorf terminals running in NCR NDC+ or Diebold 912 emulation mode is accomplished in five phases, each of which are described below. The first two phases are performed by Wincor Nixdorf alone, ACI and its customers are not involved. These phases are performed out-of-band of the BASE24 system.

Note: The device vendor does not have to be Wincor Nixdorf as long as the device runs firmware that is compliant with Wincor Nixdorf's shared signature specification and is equipped with a signature-based encrypting PIN pad (EPP) to support the BASE24 Automated Key Distribution System add-on product. The following procedures assume Wincor Nixdorf as the device vendor. If you are using another vendor's device, the implementation procedures may differ from those presented here.

For NCR devices running Wincor Nixdorf firmware, use phases 1–3 of the [“NCR 5XXX and Tidel terminal implementation procedures”](#) instead of phases 1–3 below. Phases 4 and 5 below apply equally to both Wincor Nixdorf and NCR devices running Wincor Nixdorf firmware.

Phase 1—Generate a Wincor Nixdorf RSA key pair

The Wincor Nixdorf Customer Key Center – International (CKCI) generates a customer-specific RSA key pair composed of a private (secret) key and a public key. These keys are used for signature authentication between a Wincor Nixdorf ATM and the BASE24 system. The private key is used to generate signatures, while the public key is used to verify signatures. This key pair is referred to as the root Wincor Nixdorf key pair throughout this section. The root Wincor Nixdorf public key is sent (through an offline method) to you during phase 3.

Phase 2—Manufacture encrypting PIN pad (EPP)

When the EPP for a Wincor Nixdorf terminal is manufactured, Wincor Nixdorf generates an RSA public and private key pair for each EPP. These keys are used for the encryption and decryption of keys exchanged between the EPP and the BASE24 system. The EPP's public and private keys are stored in the EPP. The public key for the EPP is signed with the root Wincor Nixdorf private key. This signature allows the BASE24 system to validate the EPP using the root Wincor Nixdorf public key created in phase 1. The signature is stored in the EPP along with the root Wincor Nixdorf public key. The root Wincor Nixdorf public key is used to validate a signature received from the BASE24 system. Finally, the serial number stored in the EPP is signed by the root Wincor Nixdorf private key. This signature is used for additional authentication between the EPP and the BASE24 system. At this point, the following key information is stored in the EPP:

- The EPP's public and private keys
- The EPP's public key signature generated by the root Wincor Nixdorf private key
- The root Wincor Nixdorf public key
- The EPP's serial number
- The EPP's serial number signature generated by the root Wincor Nixdorf private key

Phase 3—Generate an RSA key pair for the BASE24 system

To support automated key distribution for ATMs, you must also generate an RSA key pair for the BASE24 system using a hardware security module. This is accomplished by entering the KEYGEN command. Once the key pair is generated, the Wincor Nixdorf Customer Key Center – International (CKCI) must sign the public key for the BASE24 system and return the resulting signature to you along with Wincor Nixdorf's root public key. You must use this key to verify the signature returned from Wincor Nixdorf. Finally, you must store all these keys in the TSS database.

Note: The procedures are essentially the same whether you are generating an RSA key pair for a test or production environment on the BASE24 system. For testing, a test EPP provided by Wincor Nixdorf must be used instead of a production EPP. When you initially contact Wincor Nixdorf to start the process of submitting a public key and getting it signed by Wincor Nixdorf, you must specify whether the key to be signed is for testing or production. You should use the same forms and procedures for getting both test and production keys. Wincor Nixdorf Customer Key Center – International (CKCI) signs test keys using a root public key that is different from the one it uses to sign production keys.



Cross-industry best practices and card associations require that keys used in production are never used in a test environment or in test devices. Likewise, test keys must never be used in a production environment or production devices. Separate hardware security devices, ATMs, and host systems must be employed for testing purposes.

The above tasks are accomplished in the following steps:

1. To start the process of submitting a public key and getting it signed by Wincor Nixdorf, contact your Wincor Nixdorf representative and request a Key Form.
2. Generate an RSA public and private key pair for the BASE24 system. Enter the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window of the ACI Desktop to generate the RSA key pair for the BASE24 system. Refer to the [“Key Generate \(KEYGEN\) command”](#) topic provided later in this document for detailed information on using this command.
3. Access the Certificate Management window on the ACI Desktop (access **Configure > Transaction Security > Automated Key Distribution > Certificate Management**) to retrieve the BASE24 system public key and format it in the manner expected by the Wincor Nixdorf Customer Key Center – International (CKCI). When the Certificate Management window is displayed, select the host name, device type (i.e., Wincor Signature), and effective date used to generate the key. The BASE24 system public key is retrieved, formatted, and displayed in the Root Host Public Key Data field. A 40-byte SHA-1 hash value is also generated and displayed in the SHA-1 Hash Value field. The BASE24 system public key is displayed in ASN.1 format. This is the message format required when sending the public key to the Wincor Nixdorf Customer Key Center – International (CKCI).
4. Enter the 40-byte hash value displayed in the SHA-1 Hash Value field on the Root Host Public Key Data tab in the Digest (SHA-1) of Customer's Public Key field on the Wincor Nixdorf Key Form. Choose a Key Length of 2048 bits, and an Exchange Format of ASCII Hex. Choosing the ASCII Hex option ensures that the signature of your BASE24 system

public key returned by Wincor Nixdorf will be in the correct format to enter into the ACI Desktop. After completing the remainder of the Key Form, fax it to the Wincor Nixdorf Customer Key Center – International (CKCI).

5. When requested by Wincor Nixdorf, copy the BASE24 system public key cryptogram from the Root Host Public Key Data field on the Certificate Management window into a text file. Send the text file on the appropriate media to Wincor Nixdorf following the format and procedures requested by the Wincor Nixdorf Customer Key Center – International (CKCI).
6. The Wincor Nixdorf Customer Key Center – International (CKCI) validates the SHA-1 hash value for your BASE24 system. Wincor Nixdorf will contact you to indicate whether the hash value is correct. If it is incorrect, you must resend the BASE24 system public key and hash value. If it is correct, the Wincor Nixdorf Customer Key Center – International (CKCI) creates a signature of the BASE24 system public key, generates a SHA-1 hash value for the root Wincor Nixdorf public key, and returns them back to you in the media format you requested. You will receive two files. Each file contains a key or hash value in ASCII Hex format. You must copy the contents of these files and paste them into fields on the Certificate Management window in subsequent steps as shown in the following table.

Value (Wincor Nixdorf filename)	Certificate Management field
Root Wincor Nixdorf public key pkSignWN (pkSignWN<cust>T.mpk if this is a test key, or pkSignWN<cust>P.mpk if this is a production key)	Generate Message Digest > Certificate Authority Public Key (step 7a) and Validate Certificates/Signatures > Certificate Authority Public Key (step 11a)
SHA-1 hash of Wincor Nixdorf public key Digest public key, calculated with SHA1 (pkSignWN<cust>T.mpk if this is for a test key, or pkSignWN<cust>P.mpk if this is for a production key)	Generate Message Digest > SHA-1 Hash Value (step 7b)
BASE24 system public key signature (siHSM<cust>T.sig if this is for a test key, or siHSM<cust>P.sig if this is for a production key)	Validate Certificates/Signatures > Host Signature (step 11b)

7. On the Generate Message Digest tab of the Certificate Management window, perform the following:
 - a. Copy the root Wincor Nixdorf public key cryptogram from the Wincor Nixdorf Customer Key Center – International (CKCI) into the Certificate Authority Public Key field. You can click Expand View to expand this field if needed.
Note: If the cryptogram contains spaces between blocks of hexadecimal characters, the spaces are automatically removed in the message sent to the security device.
 - b. Enter the SHA-1 hash value of the root Wincor Nixdorf public key into the SHA-1 Hash Value field.

- c. Click Save.
8. If you are using an Atalla NSP, the NSP validates the SHA-1 hash value of the root Wincor Nixdorf public key and generates a message digest of the root Wincor Nixdorf public key, which is displayed in the Message Digest field. Any validation errors are reported in screen messages on the Certificate Management window. If the SHA-1 hash value is incorrect, you must request Wincor Nixdorf to resend the root Wincor Nixdorf public key and SHA-1 hash value. The NSP uses command 126 to create a message digest value that is displayed in the Message Digest field on this tab.
9. For an Atalla NSP only, encrypt the value returned in the Message Digest field from the previous step under variant 30 of the MFK using the SCT or SCA. Variant 30 is automatically assumed by both the SCT and SCA. Both the SCT and SCA require the message digest to be input as two 16-character fields. On the SCT, use the Utilities Menu -> Encrypt Challenge function to encrypt the message digest value. On the SCA, use the Calculate AKB/ Cryptogram menu (Challenge/Response) to encrypt the message digest value. You must input the message digest as two 16-byte values (Left Challenge and Right Challenge on the SCT or Component block 1 and Component block 2 on the SCA). The results are two 16-byte values that comprise the root certificate hash value.
10. For an Atalla NSP only, copy or enter the 32-byte (16-byte left and 16-byte right) values from the previous step into the Root Certificate Hash Value field on the Validate Certificates/Signatures tab.
11. On the Validate Certificates/Signatures tab of the Certificate Management window, perform the following:
 - a. Copy the root Wincor Nixdorf public key returned from the Wincor Nixdorf Customer Key Center – International (CKCI) into the Certificate Authority Public Key field on the Validate Certificates/Signatures tab of the Certificate Management window. You can click Expand View to expand this field if needed.
 - b. Copy the BASE24 system public key signature returned from the Wincor Nixdorf Customer Key Center – International (CKCI) into the Host Signature field on the Validate Certificates/Signatures tab of the Certificate Management window. You can click Expand View to expand this field if needed.
- c. Click Save.

The Atalla NSP validates the signature of the BASE24 system public key received from Wincor Nixdorf. If validated successfully, the BASE24 system public key signature is stored in the Host Public Key Security File (HOST_PUBLIC_KEY).

For an Atalla NSP only, the root Wincor Nixdorf public key is validated using the hash value input from step 10, imported, and stored in the HOST_PUBLIC_KEY along with the message digest. Any validation errors are reported in screen messages on the Certificate Management window. If the key signature is incorrect, you must request that Wincor Nixdorf resend the BASE24 system public key signature.

Once the above steps are completed, the following key data is now entered into the database for the AKDS process:

- The BASE24 system's public and private keys
- The signature of the BASE24 system's public key signed by the root Wincor Nixdorf private key

- The root Wincor Nixdorf public key

Phase 4—authenticate and exchange public keys

Before you can automatically distribute (download) an ATM master key to the EPP of an ATM, the EPP and the BASE24 system must authenticate each other and exchange their public keys. You authenticate and exchange public keys with a Wincor Nixdorf terminal (running in NDC+ or Diebold 912 emulation mode) online by sending the Load Public Key command (entered as LOADPUBLIC or LPUBLIC) to the terminal's NCR 5XXX or Diebold 10XX/478X Device Handler process using the Device Control Terminal (DCT) or a network control facility. Refer to the **BASE24-atm NCR 5XXX Device Support Manual** or the **BASE24-atm Diebold 10XX/478X Device Support Manual** for detailed Load Public Key command syntax and processing steps performed during this phase.

The Load Public Key command initiates an online exchange of public keys between the BASE24 system and the EPP of a specified terminal.

When you enter the Load Public Key command, the NCR 5XXX or Diebold 10XX/478X Device Handler process closes the specified ATM and sends a configuration request message. This message requests the ATM to send remote key loading (RKL) information and EPP capabilities to the BASE24 system. If the ATM does not have an AKDS-capable EPP installed, it rejects this request. If the ATM responds, the NCR 5XXX or Diebold 10XX/478X Device Handler process checks the Signature/Certificate status in the response. If the status is not 1 (Signature) or 3 (Signature and certificate), the process aborts the load and re-opens the ATM. Otherwise, it continues the load and requests the EPP serial number and signature from the ATM by sending a Terminal Command message with a command code indicating "Send RKL Information, EPP Serial Number and Signature." The ATM sends the EPP serial number and signature to the BASE24 system. The Device Handler requests the Transaction Security Services process to verify the signature using the root Wincor Nixdorf public key and store the EPP serial number and signature. The Transaction Security Services process returns the BASE24 system public key and signature, which the Device Handler process encodes and downloads to the terminal. The EPP in the terminal verifies the signature of the BASE24 system public key using the root Wincor Nixdorf public key and stores the BASE24 system public key in the EPP. The Device Handler process then requests the terminal to send its EPP public key to the BASE24 system. The EPP sends the EPP public key and signature to the BASE24 system. The Device Handler requests the Transaction Security Services process to verify the signature and store the EPP public key in the TSS database. In addition, one or more event messages are generated and the process reopens the ATM.

During this phase, the AKDS process interacts with the hardware security module to perform the following:

- Verify the signature of the ATM's EPP serial number using the root Wincor Nixdorf public key retrieved from the Host Public Key Security File (HOST_PUBLIC_KEY).
- Optionally, verify the EPP serial number itself against valid EPP serial numbers entered in the EPP Serial Number File (EPP_SERIAL_NUM). The value of the ATM-DH-N50-AKDS-SERIAL-NUM-VRFY or ATM-DH-1000-AKDS-SERIAL-NUM-VRFY LCONF param determines whether this additional validation is performed. Valid EPP serial numbers must be manually entered on the EPP Serial Number Configuration window before this validation is

performed. Information entered on the EPP Serial Number Configuration window is stored in the EPP_SERIAL_NUM. Refer to the [“Data access”](#) topic provided later in this section for more information on the EPP Serial Number Configuration window and the EPP_SERIAL_NUM.

- Retrieve the BASE24 system public key and signature from the HOST_PUBLIC_KEY. These are sent to the ATM after the EPP serial number signature has been successfully validated.
- Verify the signature of the EPP's public key using the root Wincor Nixdorf public key retrieved from the HOST_PUBLIC_KEY. If the signature is valid, the process generates a MAC of the EPP public key and stores the EPP public key, and the EPP public key MAC in the CHAN_PKEY_SEC.

The NCR 5XXX or Diebold 10XX/478X Device Handler process sends the BASE24 system public key and signature to the ATM's EPP, where they are verified and stored.

At this point, the following key information is stored in the EPP:

- The EPP's public and private keys
- The EPP's public key signature generated by the root Wincor Nixdorf private key
- The root Wincor Nixdorf public key
- The EPP's serial number
- The EPP's serial number signed by the root Wincor Nixdorf private key
- The BASE24 system's public key and signature

At this point, the following key data is entered into the database for the AKDS process:

- The BASE24 system's public and private keys
- The signature of the BASE24 system's public key signed by the root Wincor Nixdorf private key
- The root Wincor Nixdorf public key
- A MAC of the root Wincor Nixdorf public key
- The EPP's public key
- The EPP's serial number
- A MAC of the EPP's public key

Phase 5—download the master key

After an EPP has been successfully authenticated and exchanged public keys with the BASE24 system and the key entry mode has been set, you can securely download the ATM master key to the terminal using the Load Master Key command (entered as LOADMASTER or LMASTER). You can enter this command from the DCT or a network control facility. This command specifies the ID of the terminal to which the ATM master key is to be downloaded. After the master key is successfully loaded in both the EPP and the TSS database, you can securely download working keys to the EPP. Refer to the **BASE24-atm NCR 5XXX Device**

Support Manual or **BASE24-atm Diebold 10XX/478X Device Support Manual** for detailed Load Master Key command syntax and processing steps performed during this phase.

When you enter the Load Master Key command, the NCR 5XXX or Diebold 10XX/478X Device Handler process closes the specified ATM and sends a configuration request message. This message requests the ATM to send remote key loading (RKL) information and EPP capabilities to the BASE24 system. If the ATM does not have an AKDS-capable EPP installed, it rejects this request. If the ATM responds, the NCR 5XXX or Diebold 10XX/478X Device Handler process checks the Signature/Certificate status in the response. If the status is not 1 (Signature) or 3 (Signature and certificate), the process aborts the load and re-opens the ATM. Otherwise, it continues the load by requesting the Transaction Security Services process to generate a new master key and signature and downloading the new key and signature to the terminal. The EPP in the terminal verifies the signature using the BASE24 system public key, decrypts the master key using the EPP's private key, and stores the master key in the EPP. The terminal then sends a six-byte key verification value (KVV) for the master key just loaded back to the BASE24 system. Finally, the Device Handler process requests the Transaction Security Services process to validate the KVV.

During this phase, the AKDS process interacts with the hardware security module to generate a new master key encrypted under the EPP's public key and sign it using the BASE24 system private key. The new master key and associated check digits are stored in the Channel Security File (Channel_Security). The new master key and signature are also sent to the ATM.

At this point, both the BASE24 system and the EPP have the terminal's master key stored in their respective databases. Working keys (i.e., the PIN encryption key and MAC key) can now be securely downloaded to the EPP.

Financial Institution Table (FIT) data

The PIN encryption key in the FIT is encrypted under the master key. If a new master key is generated and a PIN encryption key exists in the FIT for local PIN verification, you must manually translate the PIN encryption key from encryption under the old master key to encryption under the new master key, replace the PIN encryption key cryptogram in the FIT, and download the updated FIT data to the ATM.

If your Wincor Nixdorf terminal is running in NDC+ emulation mode, the PIN encryption key is entered in the PEKEY (PIN ENCRYPTION KEY) field on the Financial Institution Table screen that you can access from the NCR 5XXX Configuration File Menu in the Device Control Terminal (DCT). For additional information on this field and downloading FIT data, refer to the **BASE24-atm NCR 5XXX Device Support Manual**.

If your Wincor Nixdorf terminal is running in Diebold 912 emulation mode, the PIN encryption key is entered in the PEKEY field on the Financial Institution Table screen that you can access from the Diebold Entry Program Menu. For additional information on this field and downloading FIT data, refer to the **BASE24-atm Diebold 10XX/478X Device Support Manual**.

Load all keys command

The Load All Keys command performs different processing, depending on the type of device to which it is sent.

- For NCR 5XXX, Diebold 10XX/478X terminals, and Wincor Nixdorf terminals running in NDC+ or Diebold 912 emulation mode, the Load All Keys command performs the same processing as the Load Public Key and Load Master Key commands as well as downloading the PIN and MAC encryption keys to the ATM. The command syntax and processing performed for the Load All Keys command is slightly different between NCR 5XXX, Diebold 10XX/478X terminals, and Wincor Nixdorf terminals running in NDC+ or Diebold 912 emulation mode. Therefore, they are described separately below.
- For Tidel and Triton terminals, the Load All Keys command causes the ATM terminal to authenticate and exchange public keys with the BASE24 system, request new master key(s), request a new PIN encryption (working) key, and request a new MAC (working) key (if applicable). For Tidel terminals, the ATM terminal authenticates and exchanges public keys with the BASE24 system each time the Load All Keys command is entered. For Triton terminals, the ATM terminal authenticates and exchanges public keys with the BASE24 system just once, the first time the Load All Keys command is entered. Also, two new master keys are downloaded for Triton terminals (one for the PIN key and one for the MAC key) while only a single master key is downloaded for Tidel terminals.

Because the Load All Keys command is an integral part of the AKDS implementation for Tidel and Triton terminals, the processing performed for this command is described in the respective implementation procedures rather than here. The Load All Keys command processing performed for Tidel terminals is described during [phase 4](#) of the NCR 5XXX and Tidel terminal implementation procedures, while the Load All Keys command processing performed for Triton terminals is described during [phase 3](#) of the Triton terminal implementation procedures.

NCR 5XXX and Load All Keys Command

When you enter the Load All Keys command (entered as LOADALLKEYS or LALLKEYS), the NCR 5XXX Device Handler process closes the specified ATM and sends a configuration request message. When the process receives the ATM's configuration response message, it determines whether the ATM is capable of processing AKDS commands. If not, the Load All Keys command is not performed. In addition, one or more event messages are generated and the process reopens the ATM.

If the ATM supports AKDS, the NCR 5XXX Device Handler process sequentially performs the following for the Load All Keys command:

1. Performs [phase 4](#)—authenticate and exchange public keys for an NCR device or [phase 3](#)—exchange certificates for a Diebold device.
2. Performs [phase 5](#)—download the master key for an NCR device or [phase 4](#)—download the master key for a Diebold device.

3. Downloads the MAC encryption key if the MAC SUPPORT TYPE field on NCR 5XXX or Diebold 10XX/478X screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM.
4. Downloads the PIN encryption key if the PIN ENCRYPT TYPE field on NCR 5XXX or Diebold 10XX/478X screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 01, 03, 04, or 05 for the ATM.
Note: For Tidel devices, the PIN ENCRYPT TYPE field must be set to a value of 04 or 05.
5. Reopens the ATM.

If the ATM does not support AKDS, the NCR 5XXX Device Handler process sequentially performs only steps 3 and 4 above for the Load All Keys command.

Refer to the **BASE24-atm NCR 5XXX Device Support Manual** for the detailed syntax of the Load All Keys command.

Diebold 10XX/478X load all keys command

When you enter the Load All Keys command (entered as LOADALLKEYS), the Diebold 10XX/478X Device Handler process closes the specified ATM and sends a configuration request message. When the process receives the ATM's configuration response message, it determines whether the ATM is capable of processing AKDS commands. If not, the Load All Keys command only loads the PIN and MAC encryption keys. In addition, one or more event messages is generated and the process reopens the ATM.

If the ATM supports AKDS, the Diebold 10XX/478X Device Handler process sequentially performs the following for the Load All Keys command:

1. Performs [phase 3](#)—exchange certificates for a Diebold device or [phase 4](#)—authenticate and exchange public keys for an NCR device.
2. Performs [phase 4](#)—download the master key for a Diebold device or [phase 5](#)—download the master key for an NCR device.
3. Downloads the MAC encryption key if the MAC SUPPORT TYPE field on Diebold 10XX/478X or NCR 5XXX screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM.
4. Downloads the PIN encryption key if the PIN ENCRYPT TYPE field on Diebold 10XX/478X or NCR 5XXX screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 01, 03, 04, or 05 for the ATM.
5. Reopens the ATM.

If the ATM does not support AKDS, the Diebold 10XX/478X Device Handler process sequentially performs only steps 3 and 4 above for the Load All Keys command.

Refer to the **BASE24-atm Diebold 10XX/478X Device Support Manual** for the detailed syntax of the Load All Keys command.

Wincor Nixdorf load all keys command

When you enter the Load All Keys command (entered as LOADALLKEYS or LALLKEYS), the NCR 5XXX or Diebold 10XX/478X Device Handler process closes the specified ATM and sends a configuration request message. This message requests the ATM to send remote key loading (RKL) information and EPP capabilities to the BASE24 system. If the ATM does not have an AKDS-capable EPP installed, it rejects this request. If the ATM responds, the NCR 5XXX or Diebold 10XX/478X Device Handler process checks the Signature/Certificate status in the response. If the status is not 1 (Signature) or 3 (Signature and certificate), the process aborts the load, generates one or more event messages, and reopens the ATM.

If the ATM supports AKDS, the NCR 5XXX or Diebold 10XX/478X Device Handler process sequentially performs the following for the Load All Keys command:

1. Performs [phase 4](#)—authenticate and exchange public keys.
2. Performs [phase 5](#)—download the master key
3. Downloads the MAC encryption key if the MAC SUPPORT TYPE field on NCR 5XXX screen 9 or on Diebold 10XX/478X screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 3 for the ATM.
4. Downloads the PIN encryption key if the PIN ENCRYPT TYPE field on NCR 5XXX screen 9 or on Diebold 10XX/478X screen 9 of the BASE24-atm Terminal Data files (ATD) is set to a value of 01, 03, 04, or 05 for the ATM.
5. Reopens the ATM.

If the ATM does not support AKDS, the NCR 5XXX or Diebold 10XX/478X Device Handler process sequentially performs only steps 3 and 4 above for the Load All Keys command.

Refer to the **BASE24-atm NCR 5XXX Device Support Manual** or **BASE24-atm Diebold 10XX/478X Device Support Manual** for the detailed syntax of the Load All Keys command.

Interface implementation

The Implementation of public key cryptography processing for secure logons and logoffs for interfaces, such as the SAMA SPAN2 interchange interface, is accomplished in three phases, each of which are described below.

The first phase is performed by the interchange, ACI and its customers are not involved. This phase is performed out-of-band of the BASE24 system and is beyond the scope of this guide.

Phase 1—generate an interchange RSA key pair

The interchange generates an RSA key pair. The means of how this is performed is dictated by the interchange and beyond the scope of this guide.

Phase 2—Generate RSA key pairs for the BASE24 system and exchange public keys

This phase is similar to the phase 3 implementation for NCR devices, except that a third-party certificate authority is not used. The interchange itself is considered to be the certificate authority in relation to the BASE24 system. In addition, the generated root public keys are self-signed. Once the root BASE24 system and interchange RSA key pairs are generated, the public keys can be exchanged (e.g., via email) and entered in the TSS database. Use the following procedures to generate the root BASE24 system key pair, exchange public keys with the interchange, and enter the root interchange public key into the TSS database.

Note: The root interchange public key is entered and stored as the certificate authority public key.

1. Generate a root RSA public and private key pair for the BASE24 system. Refer to the interchange specifications for the size of modulus to use. Enter the KEYGEN command on the KEYGEN Command Configuration and Submission window or Process Control window to generate the RSA key pair or pairs for the BASE24 system. Refer to the [“Key Generate \(KEYGEN\) command”](#) topic provided later in this section for detailed information on using this command.

Note: Public and private keys successfully generated by the KEYGEN command are stored in the Host Public Key Security File (HOST_PUBLIC_KEY) immediately.

2. Access the Certificate Management window on the ACI desktop (access **Configure > Transaction Security > Automated Key Distribution > Certificate Management**) to retrieve the root BASE24 system public key and format it in the manner that the interchange expects it. When the Certificate Management window is displayed, select the host name, device type (i.e., Interface), and effective date used to generate the key. The root BASE24 system public key is retrieved, formatted, and displayed in the Root Host Public Key Data field. The root BASE24 system public key is displayed in ASCII hexadecimal ASN.1 format without a signature. This is the message format required when sending the public key to the interchange.
3. Exchange public keys with the interchange.
 - a. Copy the root BASE24 system public key cryptogram in the Root Host Public Key Data field, format it as required, and send it on the appropriate media (e.g, using email) to the appropriate interchange contact.
 - b. Copy the root interchange public key sent to you from the interchange and enter it into the Certificate Authority Public Key field on the Generate Message Digest tab of the Certificate Management window. You can click Expand View () to expand this field if needed.

Note: If the cryptogram contains spaces between blocks of hexadecimal characters, the spaces are automatically removed in the message sent to the security device.

Click Save ()

If you are using an Atalla NSP, the NSP validates the root interchange public key and generates a message digest of the key, which is displayed in the Message Digest field.

If you are using a Thales e-Security or Banksys DEP HSM, the HSM validates the root interchange public key. Thales e-Security and Banksys DEP HSMs do not create a message digest.

Any validation errors are reported in screen messages on the Certificate Management window. If the key is not valid, you must request the interchange to resend the root interchange public key. The Atalla NSP uses command 126 to create a message digest value that is displayed in the Message Digest field on this tab.

4. If you are using a Thales e-Security or Banksys DEP HSM, you can skip this step and proceed with step 6.

For an Atalla NSP only, encrypt the value returned in the Message Digest field from the previous step under variant 30 of the MFK using the SCT or SCA. Variant 30 is automatically assumed by both the SCT and SCA. Both the SCT and SCA require the message digest to be input as two 16-character fields. On the SCT, use the Utilities Menu--> Encrypt Challenge function to encrypt the message digest value. On the SCA, use the Calculate AKB/Cryptogram menu (Challenge/Response) to encrypt the message digest value. You must input the 32-byte message digest as two 16-byte values (Left Challenge and Right Challenge on the SCT or Challenge on the SCA). Because the SCA only allows single-length keys for a challenge/response, you must input the 32-byte message digest value as two separate key components (left 16 bytes and right 16 bytes) and encrypt each separately in the Challenge field. You must then recombine the two encrypted 16 byte values to form the root certificate hash value. Refer to the **Atalla Secure Configuration Assistant User Guide** for detailed SCA procedures.

5. If you are using a Thales e-Security or Banksys DEP HSM, you can skip this step and proceed with step 6.

For an Atalla NSP only, copy or enter the 32 byte (16 byte left and 16 byte right) values from the previous step into the Root Certificate Hash Value field on the Validate Certificates/Signatures tab.

6. On the Validate Certificates/Signatures tab of the Certificate Management window, perform the following:
 - a. Copy the root interchange public key sent from the interchange into the Certificate Authority Public Key field on the Validate Certificates/Signatures tab of the Certificate Management window. You can click Expand View () to expand this field if needed.
 - b. Click Save ()

The Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM validates the signature of the root BASE24 system public key. If validated successfully, the root BASE24 system public key signature is stored in the Host Public Key Security File (HOST_PUBLIC_KEY).

For an Atalla NSP only, the root interchange public key is validated using the hash value input from step 5, imported, and stored in the HOST_PUBLIC_KEY along with the message digest.

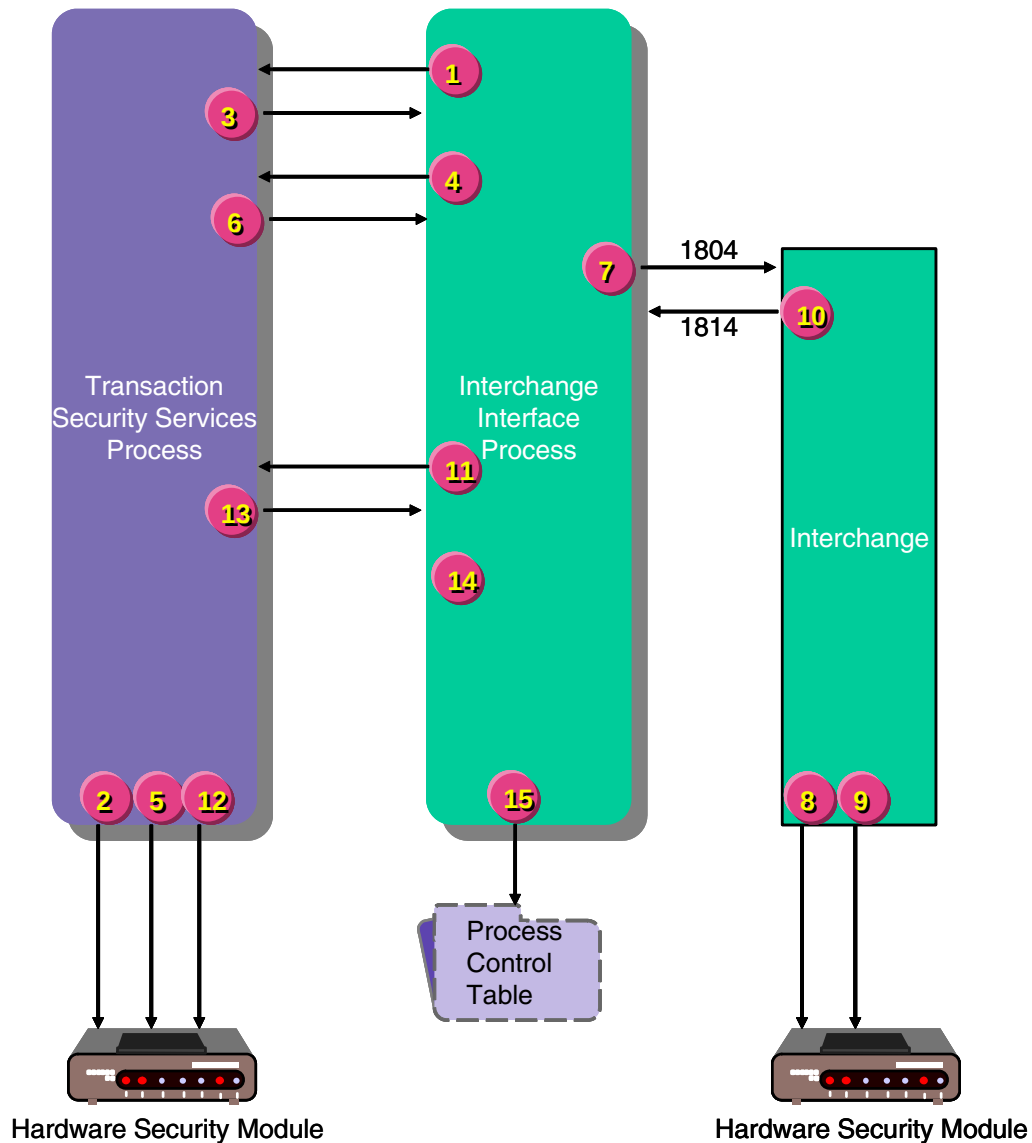
Any validation errors are reported in screen messages on the Certificate Management window.

After the above steps are completed, the following key data is now entered into the database for the AKDS process.

- The root BASE24 system public and private keys
- The root interchange public key
- A MAC of the root interchange public key

Phase 3—Mutual authentication

The diagram below illustrates the flow of processing for mutual authentication between the BASE24 system and an interchange. The steps depicted in the illustration are described following the diagram.



1. When the BASE24 system initiates an 1804 logon or logoff to the interchange, the Interchange Interface process sends a request to the Transaction Security Services process to generate a random nonce.
2. The Transaction Security Services process requests the hardware security device to generate the random nonce.

3. The Transaction Security Services process returns the random nonce to the calling Interchange Interface process.
4. The Interchange Interface process sends a request to the Transaction Security Services process to generate a digital signature using the data elements defined by the interchange to be sent in the 1804 logon/logoff message. For the SAMA SPAN2 interchange, this includes the following:
 - Transmission date and time
 - Transaction originator institution identification code
 - Function/action code
 - Random nonce generated in step 2
5. The Transaction Security Services process requests the hardware security device to generate the digital signature using the root BASE24 system private signature key.
6. The Transaction Security Services process returns the digital signature to the calling Interchange Interface process.
7. The Interchange Interface process sends the 1804 logon or logoff message to the interchange. The data in the message includes the BASE24 system random nonce and the digital signature.
8. The interchange verifies the digital signature in the 1804 logon or logoff request message using the root BASE24 system public signature key. The interchange also ensures that the transmission date and time in the request is greater than the transmission date and time from the previous logon/logoff request received from the BASE24 system to prevent a replay attack (i.e., a third party duplicating a message from a previous logon/logoff message).
9. If the signature is valid, the interchange generates a random nonce and a digital signature of the 1814 logon/logoff response message using the data elements defined by the interchange and its private signature key. For the SAMA SPAN2 interchange, this includes the same data elements identified in step 4.
10. The interchange sends the 1814 response message to the BASE24 system. The data in the message includes the interchange random nonce and the digital signature.
11. The Interchange Interface process sends a request to the Transaction Security Services process to verify the digital signature received in the 1814 response message.
12. The Transaction Security Services process requests the hardware security device to verify the digital signature using the interchange's public signature key.
13. The Transaction Security Services process returns a response indicating whether the digital signature is valid to the calling Interchange Interface process.
14. The Interchange Interface process ensures that the transmission date and time in the response is greater than the transmission date and time from the previous logon/logoff response received from the interchange to prevent a replay attack (i.e., a third party duplicating a message from a previous logon/logoff message).
15. The Interchange Interface process sets the status of the connection according to the outcome of the signature verification and transmission date and time comparison.

Key Generate (KEYGEN) command

The KEYGEN process control command enables you to generate one or two RSA key pairs for the BASE24 system. Typically, this command is only entered one time to generate the root BASE24 system key pair, unless the existing root BASE24 system RSA key pair becomes corrupted in the TSS database or is compromised. You can also use the KEYGEN process control command to periodically generate a primary BASE24 system RSA key pair for the NCR enhanced signature-based AKDS scheme. Some card associations advocate that key distribution hosts (KDHs) replace their keys every five years.

You can use the KEYGEN command to generate multiple keys for the BASE24 system for the following scenarios:

- When host public keys are defined at a regional level
- In a merger or acquisition where two or more institution's host public keys are used in the same BASE24 system
- When multiple certificate authorities are used for a fleet of the same type of ATMs

You must use a different host name for each different RSA key pair or pairs generated for the BASE24 system for a particular device type.

After you have successfully generated a root RSA key pair for the BASE24 system and had the public key signed by a certificate authority, you can disable use of the KEYGEN command from the Disable KEYGEN window for generating the root BASE24 system key pair. You can also re-enable the command from this window if it is needed at a later time. See the [“Disabling the KEYGEN command”](#) topic for detailed information on the Disable KEYGEN window.

Note: Disabling the KEYGEN command only disables the ability to generate a root BASE24 system key pair. It does not disable the ability to generate a primary BASE24 system key pair.

You can enter the KEYGEN command from the KEYGEN Command Configuration and Submission window, from the Process Control window, or from an XPNET network control facility. The KEYGEN Command Configuration and Submission window is provided to simplify the building and submission of this complex command. Procedures for entering this command from the KEYGEN Command Configuration and Submission window and Process Control window are discussed following the command syntax description below. For more information on entering this command from a network control facility, refer to [section 9](#).

The security device used for the KEYGEN command is disabled during key generation, which may take up to 90 seconds. Therefore, you should check to make sure you always have at least one other hardware security module in an enabled state in your production system to authorize transactions before entering this command. Otherwise, a situation could arise where no hardware security modules are available for online transaction processing. This is especially important when testing and production systems share the same hardware security modules, although this is not recommended for security best practices.

For the *host-name*, *common-name*, and *org-name* variables in the KEYGEN command syntax, the following characters are allowed: A–Z, a–z, 0–9, *b*, (,), +, -, ., /, :, ?, and #, where *b* represents a blank space. You cannot use a comma “,” or exclamation point “!” within these variables because they are reserved to denote the end of a variable value and to override the current status in the Host Public Key Security File, respectively. For an ATM device, the *host-name* variable in this command must be set to the value of the KEY SIGNER field on screen 22 of the BASE24-atm Terminal Data files (ATD). If this field is not configured, it defaults to a value of HOST. For an interface, the *host-name* variable in this command must be set to a user-provided value or to the default value of HOST.

Syntax

KEYGEN *host-name*, *dev-typ*, [*common-name*], [*org-name*], [!], [*mod-size*], [*key-opt*] [,*eff-daf*]

host-name — For devices running Wincor Nixdorf firmware, Diebold 10XX/478X devices, NCR 5XXX devices, Tidel devices, and Triton devices, this is the name of the host BASE24 system as configured in the KEY SIGNER field on screen 22 of the BASE24-atm Terminal Data files (ATD). If you need to generate multiple BASE24 system RSA key pairs for a device type, you must use a different host name for each RSA key pair or pairs generated. For all other supported device and firmware combinations or if this field is not configured, it defaults to a value of HOST. For an interface, the *host-name* variable in this command must be set to a user-provided value or to the default value of HOST. The key pair is stored under this name in the Host Public Key Security File. 1–16 alphanumeric characters. Required.

dev-typ — The type of ATM device to which keys are to be distributed. Required. Valid values are:

NCR or 0	= NCR 5XXX or Tidel devices
DIEBOLD, DBLD, or 1	= Diebold 10XX/478X devices
INTERFACE, INTFC or 99	= Interface
TRITON, or 4	= Triton devices
WINCORSIG, WINSIG, or 2	= Wincor Nixdorf devices running in NDC+ or Diebold 912 emulation mode on Wincor Nixdorf firmware.
WINCORNCR, WINNCR or 3	= NCR devices running in NDC+ or Diebold 912 emulation mode on Wincor Nixdorf firmware.

common-name — The specific entity for which the BASE24 system public key certificate is being requested. This value is used in generating the BASE24 system public key certificate for use with Diebold devices running in Diebold 912 emulation or NCR NDC+ emulation modes and Triton devices. For Triton devices, this variable must be the host (BASE24 system) unique identifier returned from Triton.

This variable is not used for NCR devices or Wincor Nixdorf devices running in Diebold 912 emulation or NCR NDC+ emulation modes, Tidel devices, or interfaces, however the intervening comma used to separate this variable is always required. 1–64 alphanumeric characters.

org-name — The name (e.g., ACI Worldwide) or URL (e.g., www.host.com) of the organization or company for which the BASE24 system public key certificate is being requested. This value is used in generating the BASE24 system public key certificate for use with Diebold devices running in Diebold 912 emulation or NCR NDC+ emulation modes and Triton devices. For Triton devices, this variable must contain a URL. This variable is not used for NCR devices or Wincor Nixdorf devices running in Diebold 912 emulation or NCR NDC+ emulation modes, or interfaces, however, the intervening comma used to separate this variable is always required if the command line includes the override flag. 1–64 alphanumeric characters.

! — An optional override flag. This flag forces the KEYGEN command to be performed regardless of the current status in the Host Public Key Security File. Normally, you would only use this flag if the database has been corrupted.

mod-size — The size of the modulus in bits. For an Atalla NSP, this value must be 1024–2048. For a Thales e-Security HSM, this value must be 320–2048. For a Banksys DEP HSM, this value must be 512–2048. For an interface, this value must be 1024–2048. All value ranges are inclusive. If this value is not provided in the command, it defaults to 2048 for all device types, including interfaces.

key-opt — A code indicating the type of RSA key pair(s) to be generated. Valid values are:

- R = Root BASE24 system key pair. Default.
- P = Primary BASE24 system key pair.
- B = Both the root and primary BASE24 system key pairs.

If this value is not provided in the command, it defaults to R. The values P and B are only valid if the device type is NCR.

eff-dat — The date, in YYYYMMDD format, on which the generated RSA key pair(s) are to become effective. If this value is not provided in the command, it defaults to the current system date.

If multiple root or primary BASE24 system key pairs are generated, the key pair with the most current effective date that is less than or equal to the current system date is used.

Note: The “!” optional override flag for the Key Generate command was designed to prevent multiple Key Generate commands from being performed at the same time. This can occur if the command is issued (typically using NCPCOM) to two different processes (either XMLS or IS) or if you issue the command to the XMLS destination using the Process Control window and more than one XMLS process is configured.

If Key Generate commands are issued to two different processes at the same time, the following event message is generated (assuming that you have not included the “!” flag in either command):

```
YY-MM-DD;HH:MM:SS.sss \ARGUS.$W4X12      ACI.1502.1000      70
Mgr PPD: \ARGUS.$W4XN  Generator: P4X^TSS07
TSEC: RSA Key Generation already in progress. Host Name = DUMMY1
. Implementation = 0.
```

You should only use the “!” override flag when the above event message is generated and you know that a key generation is not in progress (this can occur if the process that was executing the Key Generate command ended abnormally before completing the operation).

KEYGEN Command Configuration and Submission window

Because of the complexity of the KEYGEN command, the KEYGEN Command Configuration and Submission window is provided to assist you in building, verifying, and submitting the command. On this window, you enter the required and optional data fields needed for the command in the Command Configuration section of the window. The data fields in which you can enter data, and the default data displayed in active fields, depends on the device type you select in the Device Type field as follows:

- If you select NCR, Interface, or Wincor/NCR Signature, the Common Name and Organization Name fields are disabled. Although the Key Option field is not disabled when you select Interface or Wincor/NCR Signature, the only available value is Root.
- If you select Diebold or Triton, the Key Option field is disabled. The default value of Root is assumed.
- If you select Wincor Signature, the Common Name, Organization Name, and Key Option fields are disabled. The default value of Root is assumed for the key option.
- If you select Interface, the Modulus Size field defaults to 1024. For all other device type selections, the Modulus Size field defaults to 2048.

After you have configured the fields needed for the command, you can click the Build KEYGEN Command button to validate that your entries are correct. If your entries are valid, the window replaces the default text in the Command Text field with the KEYGEN command formatted as it will be sent to the server and enables the Submit KEYGEN Command button. You can then click the Submit KEYGEN Command button to submit the command. The command response is displayed in the Response field.

A sample of the KEYGEN Command Configuration and Submission window when the Diebold device type is selected is shown below.

KEYGEN Command Configuration and Submission

Command Configuration

Host Name: DIEBOLD_1 Device Type: Diebold

Common Name: Organization Name:

Override Current Key Indicator: ☐ Modulus Size: 2048

Key Option: Root Effective Date: August 3, 2007

Functions

Command Text

KEYGEN host-name, dev-ty, [common-name], [org-name], [!], [mod-size], [key-opt] [,eff-dat]

Build KEYGEN Command Submit KEYGEN Command

Response

The next screen image depicts the window after clicking the Build KEYGEN Command button. Notice that the default command text syntax is replaced with the actual values from the data fields and that the Submit KEYGEN Command button is now enabled.

KEYGEN Command Configuration and Submission

Command Configuration

Host Name: DIEBOLD_1 Device Type: Diebold

Common Name: ATM Division Organization Name: Bank A

Override Current Key Indicator: ☐ Modulus Size: 2048

Key Option: Root Effective Date: July 23, 2007

Functions

Command Text

KEYGEN DIEBOLD_1,DIEBOLD,ATM Division,Bank A, ,2048, ,20070723

Build KEYGEN Command Submit KEYGEN Command

Response

To enter the KEYGEN command on the KEYGEN Command Configuration and Submission window, perform the following steps:

1. Log on to the ACI desktop user interface.
2. If necessary, navigate to the Security Device Configuration window (access **Configure > Transaction Security > Security Device Configuration**) and make sure the appropriate hardware security module (i.e., one with PKI-enabled software or firmware) is enabled (i.e., the Enable PKI field is checked).
3. Navigate to the KEYGEN Command Configuration and Submission window (access **Configure > Transaction Security > Automated Key Distribution > KEYGEN Command Configuration and Submission**).

Set the required and optional fields at the top of the window as needed and click the Build KEYGEN Command button. The entered data elements for the command are validated and the command syntax is built and displayed in the Command Text field.

4. If the command is valid and built as desired, click the Submit KEYGEN Command button to send the command to the hardware security device. The command is submitted as a Deliver process control command to the Transaction Security component of the XML Server (XMLS) process.

The device generates the key pair and returns the keys to the Transaction Security component, which stores them in the Host Public Key Security File (HOST_PUBLIC_KEY). Check the command response in the Response field to verify the command was successfully processed.

Process Control window

To enter the KEYGEN command on the Process Control window, perform the following steps:

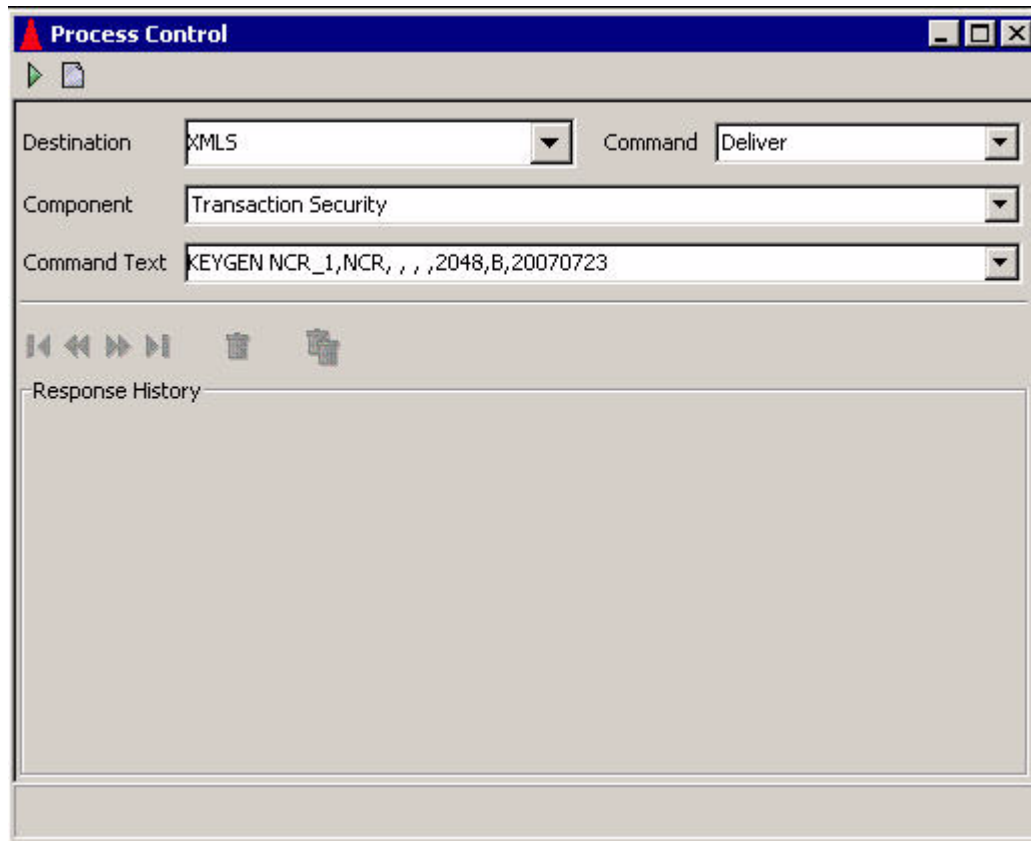
1. Log on to the ACI desktop user interface.
2. If necessary, navigate to the Security Device Configuration window (access **Configure > Transaction Security > Security Device Configuration**) and make sure the appropriate hardware security module (i.e., one with PKI-enabled software or firmware) is enabled (i.e., the Enable PKI field is checked).
3. Navigate to the Process Control window (access **System Operations > Process Control**).

On this window, set the Destination field to XMLS or the symbolic name of the XML Server process, the Command field to Deliver, the Component field to Transaction Security, and the Command Text field according to the command syntax described above. The following are examples of the command text:

```
KEYGEN NCR_1,NCR, , , ,2048,B,20070723
KEYGEN DIEBOLD_1,DIEBOLD,Bank A,ATM Division, ,2048, ,200707
KEYGEN TRITON,TRITON,00000000000000200,www.triton.com, ,2048, ,20081031
KEYGEN HOST,WINCORSIG, , , ,2048, ,20070723
KEYGEN HOST,INTERFACE, , , ,1024,R,20070723
KEYGEN HOST,WINCORNCR, , , ,2048,B,20070723
```

4. Click Submit (▶) to send the command to the hardware security device. The device generates the key pair and returns the keys to the Transaction Security component, which stores them in the Host Public Key Security File (HOST_PUBLIC_KEY). Check the command response to verify the command was successfully processed.

An example of the Process Control window with the KEYGEN command is shown below.

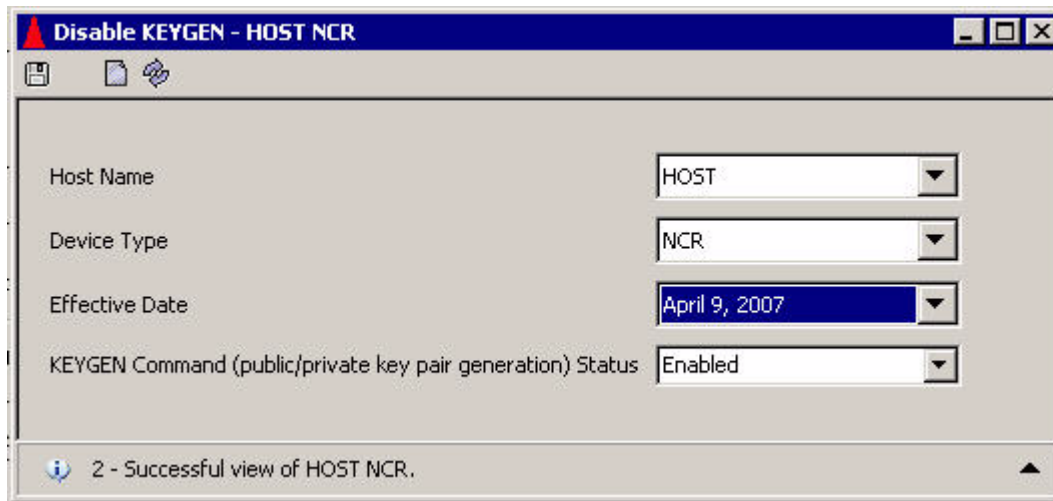


Disabling the KEYGEN command

The Disable KEYGEN window allows you to enable or disable the KEYGEN process control command for generating the root BASE24 system (host) RSA key pair. This window does not disable the KEYGEN command for generating the primary BASE24 system (host) RSA key pair used in NCR's enhanced signature based AKDS scheme. You access the Disable KEYGEN window from **Configure > Transaction Security > Automated Key Distribution > Disable KEYGEN**.

If you enter the KEYGEN command after you have already used it to successfully generate a root RSA key pair for the BASE24 system (host) and had the root public key signed by a certificate authority, the currently signed root keys in the Host Public Key Security File (HOST_PUBLIC_KEY) are deleted and replaced with the new root keys. In this case, the new root keys must be signed by the certificate authority, which can take a number of days to complete. To prevent this from happening inadvertently, you can use the Disable KEYGEN window to disable the KEYGEN process control command for generating the root BASE24 system (host) RSA key pair. You would do this after you have used the command once to successfully generate the root RSA key pair for the BASE24 system, had the key pair signed by a certificate authority, and stored the keys in the HOST_PUBLIC_KEY.

A sample of this window is shown below, followed by descriptions of its fields.



The Host Name field specifies the name of the root BASE24 system (host) public key owner in the HOST_PUBLIC_KEY.

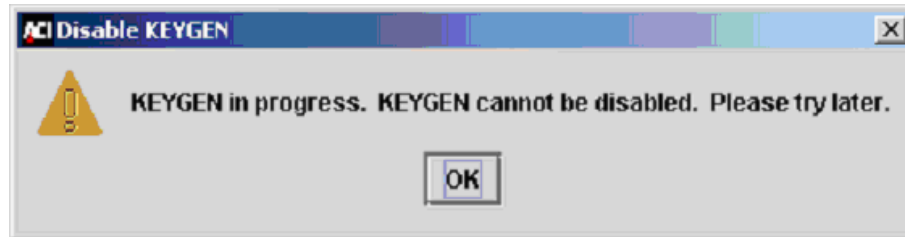
The Device Type field allows you to select the type of device (NCR, Diebold, Interface, Triton, Wincor Signature, or Wincor/NCR Signature) for which the KEYGEN command is to be disabled or enabled.

The Effective Date field lets you select the effective date of the root BASE24 system (host) public key for which the KEYGEN command is to be disabled or enabled. If multiple root or primary BASE24 system key pairs are generated, the key pair with the most current effective date that is less than or equal to the current system date is used in processing.

The Enable/Disable KEYGEN command (public/private key pair generation) field allows you to change the status of the public/private key generation for the root BASE24 system (host) public key pair to a value of Enabled or Disabled. When the KEYGEN command is entered, the Transaction Security Services process checks the status value in the HOST_PUBLIC_KEY to determine whether the command can be performed. The command is executed only if an RSA key generation is not currently in progress and the KEYGEN command is enabled.

The Disabled value effectively disables the use of the KEYGEN command entered from the KEYGEN Command Configuration and Submission window, the Process Control window, or from an XPNET network control facility, while the Enabled value enables the command. To disable the KEYGEN command, select the desired host name in the Host Name field, the desired device type in the Device Type field, and the Disabled value in the Enable/Disable KEYGEN command field, and click Save (). The value is stored in the HOST_PUBLIC_KEY. To re-enable the KEYGEN command, select the desired host name in the Host Name field, the desired device type in the Device Type field, and the Enabled value in the Enable/Disable

KEYGEN command field, and click Save (). The value is stored in the HOST_PUBLIC_KEY. If you attempt to disable the KEYGEN command while an RSA key generation is in progress, the following dialog box is displayed.



Because of the sensitive nature of the KEYGEN command, user access to the Disable KEYGEN window should be highly restricted. The task ID and default permission ID associated with this window are KEYGENLockdown and FullKEYGENLockdown, respectively. In addition, users granted security access to the Disable KEYGEN window must also be granted at least view access to the Host Public Key Perusal window to retrieve the drop-down list for the Host Name field.

Data access

When the BASE24 Automated Key Distribution System add-on product is installed, two additional TSS database files are required to support NCR, Diebold, Tidel, Triton, shared Wincor Nixdorf certificate, or shared Wincor Nixdorf signature EPPs— the Host Public Key Security File (HOST_PUBLIC_KEY) and Channel Public Key Security File (CHAN_PKEY_SEC). Another TSS database file—the EPP Serial Number File (EPP_SERIAL_NUM)—is optional for all types of EPPs. When public key cryptography is implemented for secure logons and logoffs to and from an interchange, only the HOST_PUBLIC_KEY is used.

Data is entered into the HOST_PUBLIC_KEY using the KEYGEN command and the Certificate Management window of the ACI desktop when generating an RSA key pair for the BASE24 system. Data is automatically entered into the CHAN_PKEY_SEC when authenticating and exchanging public keys with an ATM terminal. After data is entered in these files, you can only peruse the data using the Host Public Key Perusal and Channel Public Key Perusal windows on the ACI desktop (access **View > Automated Key Distribution > Host Public Key Perusal** or **Channel Public Key Perusal**).

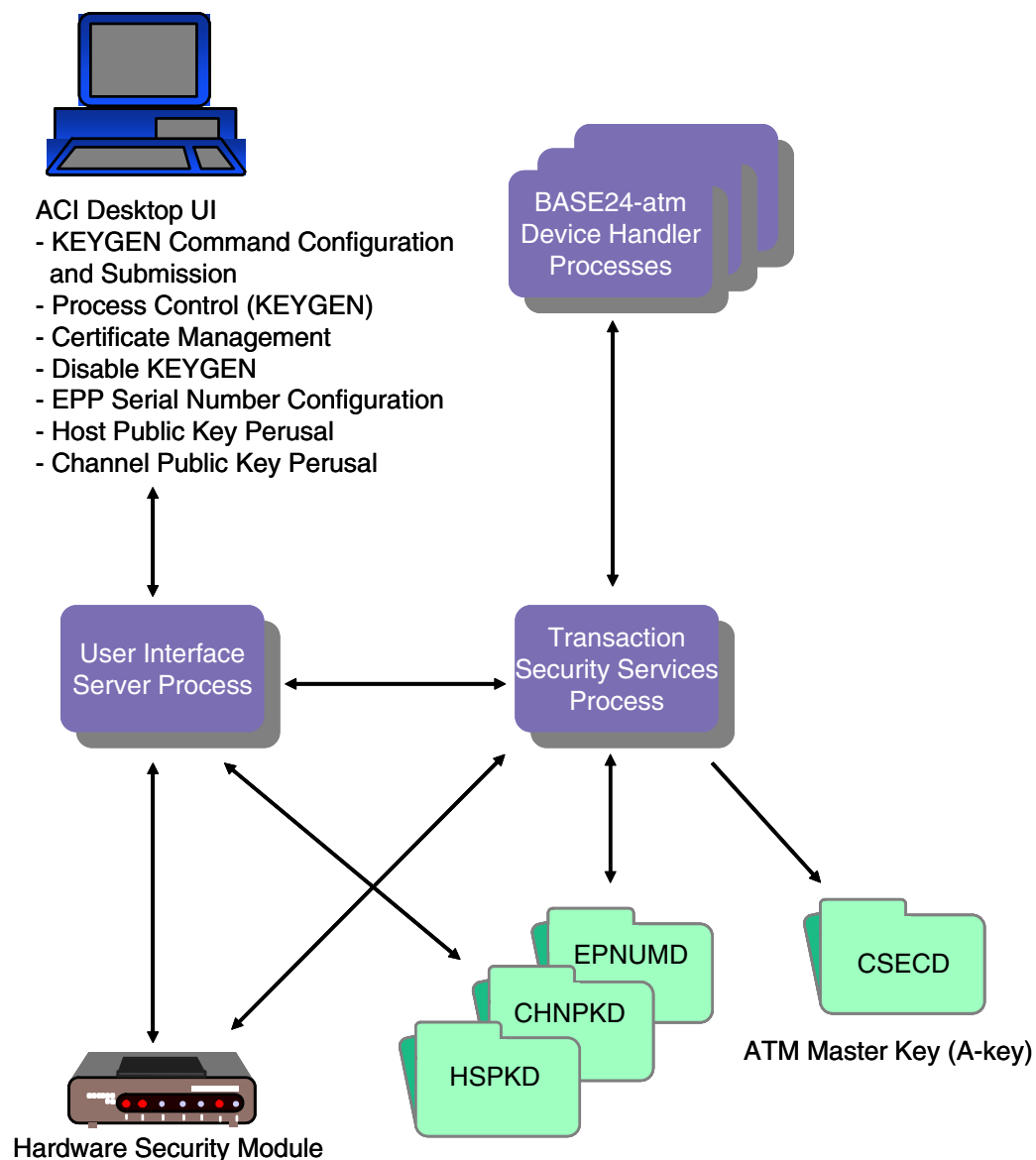
Data is entered into the EPP_SERIAL_NUM using the EPP Serial Number Configuration window on the ACI desktop (access **Configure > Transaction Security > EPP Serial Number**).

Note: The HOST_PUBLIC_KEY, CHAN_PKEY_SEC, and EPP_SERIAL_NUM are not accessed as memory tables.

When the AKDS process generates a new master key, it stores the new key in the Channel Security File (Channel_Security).

The diagram on the following page shows you the relationships between the HOST_PUBLIC_KEY, CHAN_PKEY_SEC, and EPP_SERIAL_NUM and the processing components that access them.

Descriptions of the Certificate Management window, EPP Serial Number Configuration window, Host Public Key Perusal window, and Channel Public Key Perusal window are provided following the diagram.



Certificate Management window

The Certificate Management window enables you to perform several functions when generating an RSA key pair for the BASE24 system. For NCR and Tidel devices, these functions are performed during [phase 3](#). For Diebold devices, these functions are performed during [phase 2](#). For Triton devices, these functions are performed during [phase 2](#). For Wincor Nixdorf devices, these functions are performed during [phase 3](#). For interfaces, these functions are performed during [phase 3](#). The Certificate Management window has three tabs, each of which are described below.

Root host public key data

The Root Host Public Key Data tab on the Certificate Management window enables you to retrieve the root BASE24 system public key after it is generated using the KEYGEN command and format the key in the manner that the target certificate authority expects it.

When the Certificate Management window is displayed, select the host name in the Host Name field, the device type in the Device Type field, and the effective date in the Effective Date field. The root BASE24 system public key that matches this data is retrieved from the HOST_PUBLIC_KEY and displayed in the Root Host Public Key Data field.

For NCR, Tidel, and Wincor Nixdorf devices and for interfaces, the key is displayed in ASN.1 format. For NCR devices, Tidel devices, and interfaces, the BASE24 system public key is self-signed; for Wincor Nixdorf devices, the root BASE24 system public key is not self-signed. In addition, a 40-byte Secure Hash Algorithm 1 (SHA-1) hash value of the root BASE24 system public key is displayed in the SHA-1 Hash Value field for NCR, Tidel, and Wincor Nixdorf devices. Note that the signature of the key is included in the SHA-1 hash calculation for NCR and Tidel devices. Neither a signature or SHA-1 hash is used or displayed for interfaces. For Diebold and Triton devices, the key is displayed in PKCS#10 Certificate Request format, Base64 encoded. This is the message format required when sending the public key to the Diebold and Triton certificate authorities. For Diebold and Triton devices or for interfaces, the SHA-1 Hash Value field is not used.

For NCR and Tidel devices, the certificate authority is the NCR trusted center. To start the process of submitting a root public key and getting it signed by NCR, contact your NCR/Reseller Account Manager. The account manager will then establish your name, mailing address, e-mail address, telephone and fax numbers and advise NCR Customer and Sales Support to send you the proper forms, format, and procedures.

For Wincor Nixdorf devices, the certificate authority is the Wincor Nixdorf Customer Key Center – International (CKCI). To start the process of submitting a root public key and getting it signed by the Wincor Nixdorf Customer Key Center – International (CKCI), contact your Wincor Nixdorf representative and request a Key Form.

For Diebold devices, the certificate authority is the Digital Signature Trust Co. (DST). To begin the process of submitting a root public key certificate and getting it signed by DST, ask your Diebold System Engineer to contact the Diebold Product Manager for their Remote Key product using an e-mail message. The Diebold Product Manager will then send you an e-mail message containing the address of the DST-hosted web site, your authorization code

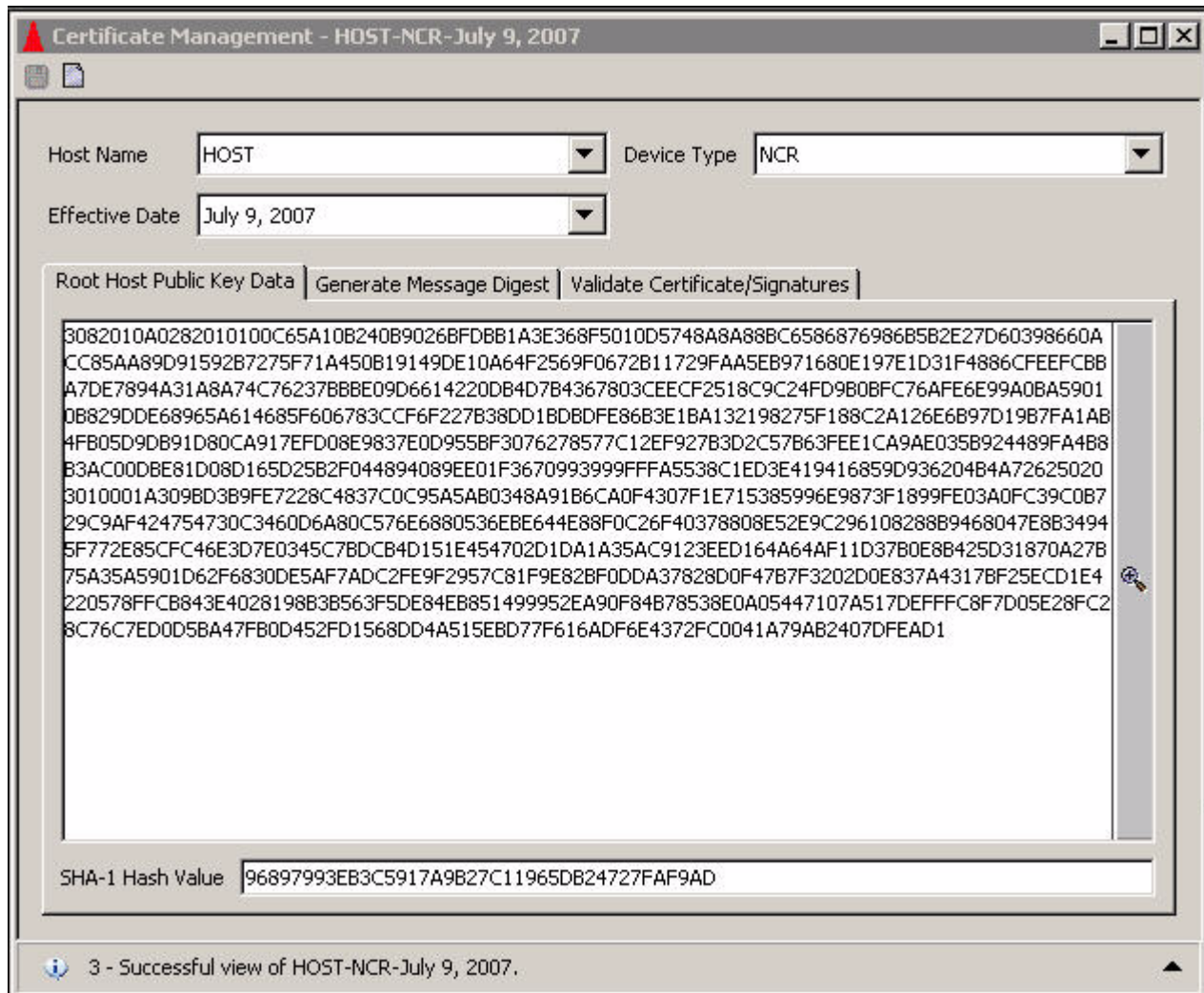
(password), and an HTML demonstration of the registration website. Each time you need new certificates (e.g., for test, production, and certification) you must contact your Diebold System Engineer to request a new authorization code.

For Triton devices, the certificate authority is Triton. To begin the process, contact your Triton representative.

For interfaces, the certificate authority to be used is dictated by the interchange. For example, for the SAMA SPAN2 interchange, a third-party certificate authority is not used.

NCR and Tidel Example. A sample of the Certificate Management window after selecting the host name, NCR device type, and effective date is shown below. In this example, the root BASE24 system public key is a self-signed key displayed in ASN.1 format. You can copy the root BASE24 system public key cryptogram in the Root Host Public Key Data field, format it as required, and send it on the appropriate media (e.g, diskette or CD-ROM) to the NCR trusted center to be validated and signed.

For NCR and Tidel devices, the SHA-1 Hash Value field displays the SHA-1 of the ANS.1 formatted root BASE24 system public key. You must enter the value displayed in this field into the SHA-1 (Fields 1–6) field on form NCR_003 - Signature Request sent to the NCR trusted center. These procedures also apply when shared Wincor Nixdorf signature firmware is used on NCR devices (i.e., the device type is “Wincor/NCR Signature”)



Certificate Management - HOST-NCR-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data | Generate Message Digest | Validate Certificate/Signatures

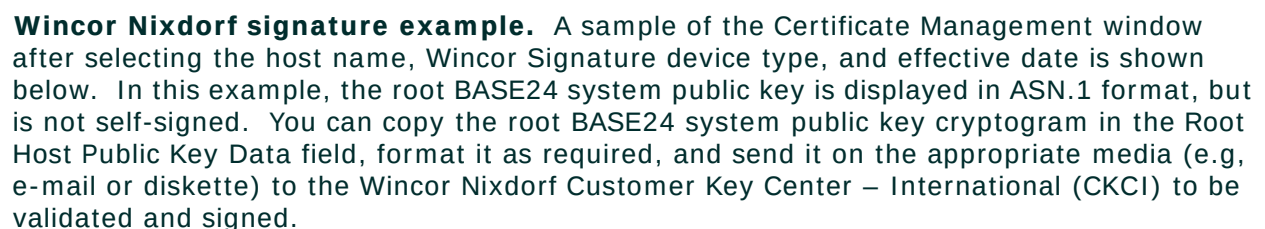
```

3082010A0282010100C65A10B240B9026BFD8B1A3E368F5010D5748A88BC6586876986B5B2E27D60398660A
CC85AA89D91592B7275F71A450B19149DE10A64F2569F0672B11729FAA5EB971680E197E1D31F4886CFEEFCBB
A7DE7894A31A8A74C76237BBBE09D6614220DB4D7B4367803CEECF2518C9C24FD9B0BFC76AFE6E99A0BA5901
0B829DDE68965A614685F606783CCF6F227B38DD1BDBDFE86B3E1BA132198275F188C2A126E6B97D19B7FA1AB
4FB05D9DB91D80CA917EFD08E9837E0D955BF3076278577C12EF927B3D2C57B63FEE1CA9AE035B924489FA4B8
B3AC00DBE81D08D165D25B2F044894089EE01F3670993999FFFA5538C1ED3E419416859D936204B4A72625020
3010001A309BD3B9FE7228C4837C0C95A5AB0348A91B6CA0F4307F1E715385996E9873F1899FE03A0FC39C0B7
29C9AF424754730C3460D6A80C576E6880536EBE644E88F0C26F40378808E52E9C296108288B9468047E8B3494
5F772E85CFC46E3D7E0345C7BDCB4D151E454702D1DA1A35AC9123EED164A64AF11D37B0E8B425D31870A27B
75A35A5901D62F6830DE5AF7ADC2FE9F2957C81F9E82BF0DDA37828D0F47B7F3202D0E837A4317BF25ECD1E4
220578FFCB843E4028198B3B563F5DE84EB851499952EA90F84B78538E0A05447107A517DEFFFC8F7D05E28FC2
8C76C7ED0D5BA47FB0D452FD1568DD4A515EBD77F616ADF6E4372FC0041A79AB2407DFEAD1

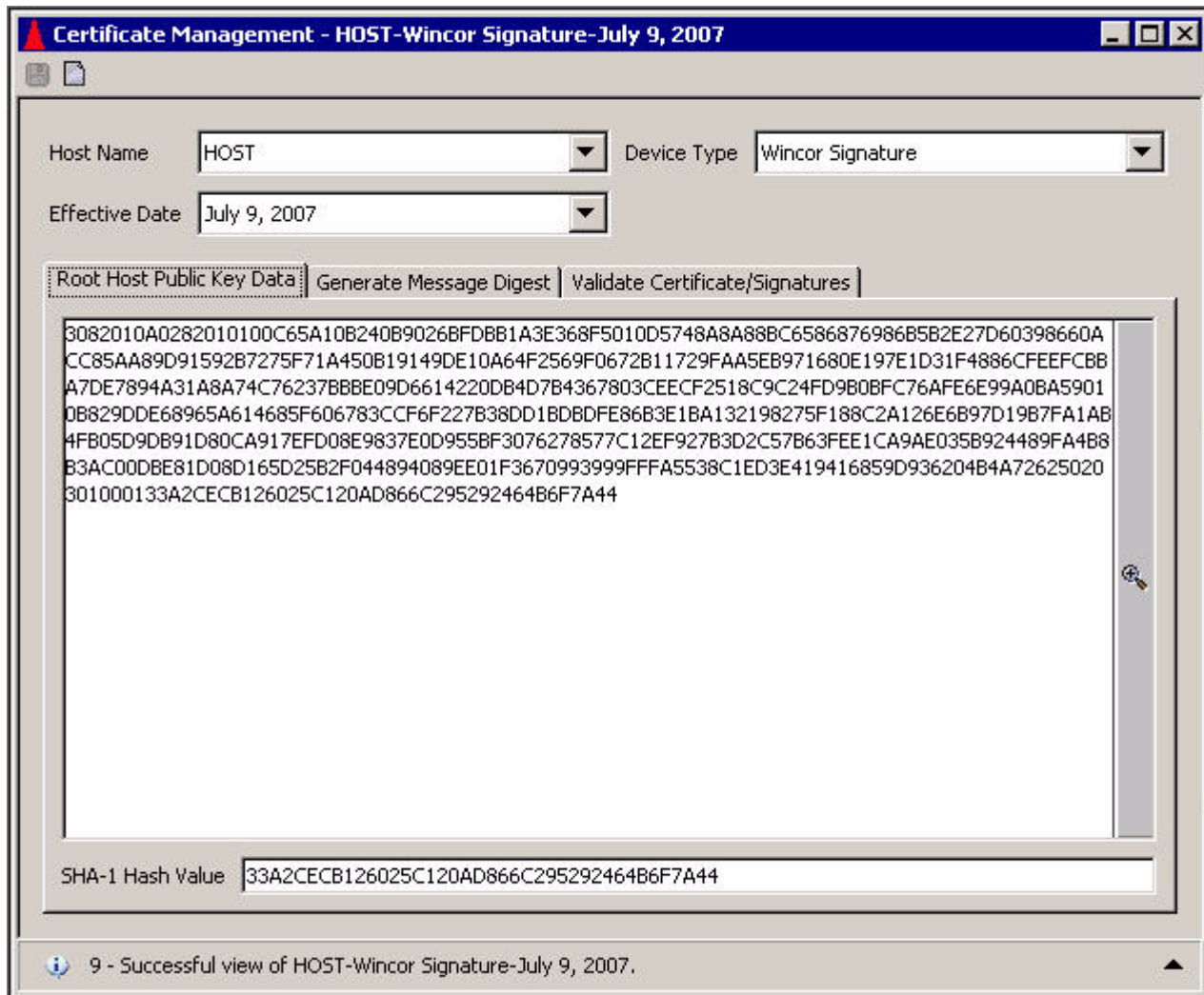
```

SHA-1 Hash Value:

3 - Successful view of HOST-NCR-July 9, 2007.



For Wincor Nixdorf devices, the SHA-1 Hash Value field displays the SHA-1 of the ANS.1 formatted root BASE24 system public key. You must enter the value displayed in this field into the Digest (SHA-1) of Customers Public Key field on the Key Form sent to the Wincor Nixdorf Customer Key Center – International (CKCI).



Certificate Management - HOST-Wincor Signature-July 9, 2007

Host Name: Device Type:

Effective Date:

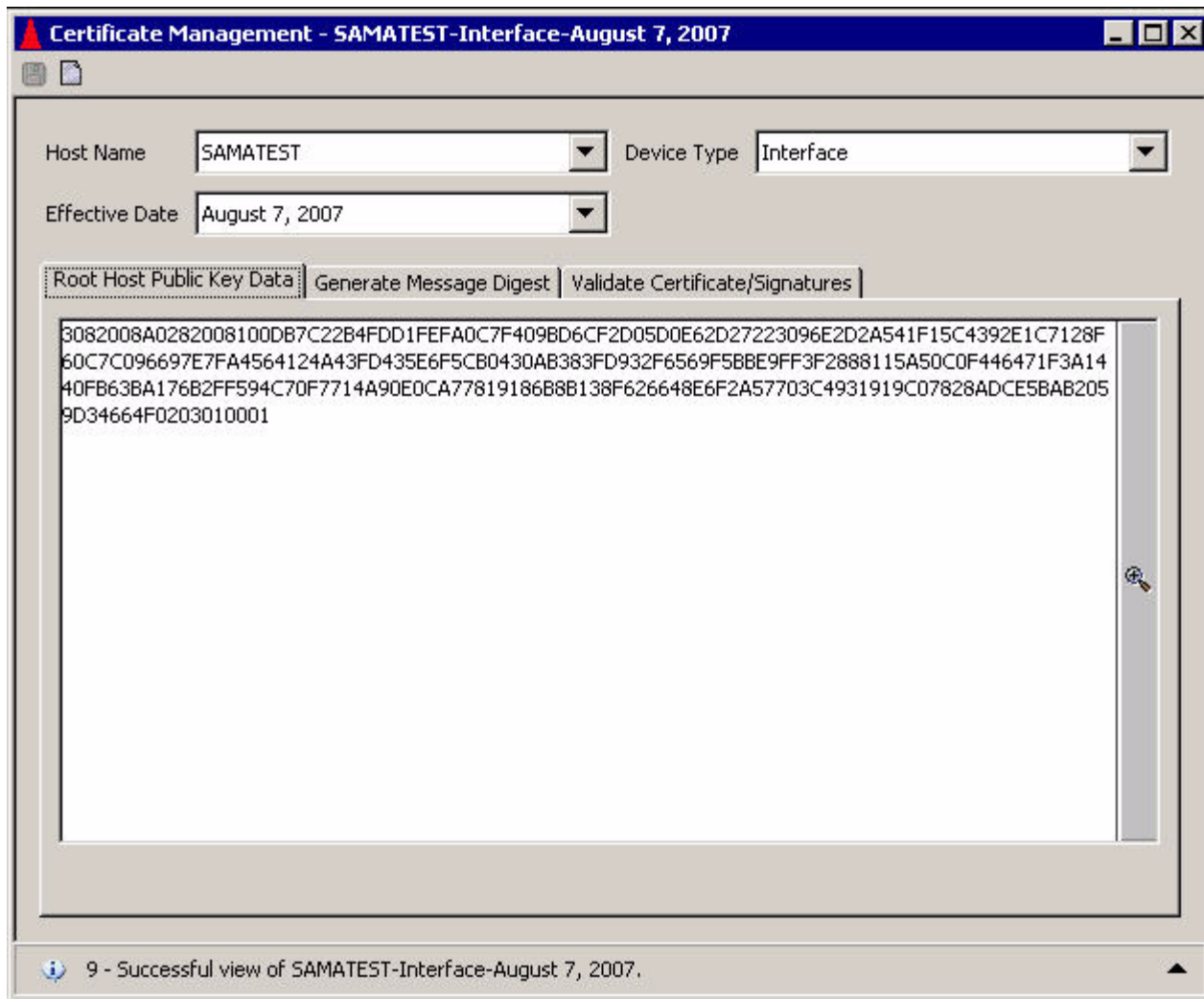
Root Host Public Key Data | Generate Message Digest | Validate Certificate/Signatures

3082010A0282010100C65A10B240B9026BFDDB1A3E368F5010D5748A8A88BC6586876986B5B2E27D60398660A
CC85AA89D91592B7275F71A450B19149DE10A64F2569F0672B11729FAA5EB971680E197E1D31F4886CFEEFCBB
A7DE7894A31A8A74C76237BBE09D6614220DB4D7B4367803CEECF2518C9C24FD9B0BFC76AFE6E99A0BA5901
0B829DDE68965A614685F606783CCF6F227B38DD1BDBDFE86B3E1BA132198275F188C2A126E6B97D19B7FA1AB
4FB05D9DB91D80CA917EFD08E9837E0D955BF3076278577C12EF927B3D2C57B63FEE1CA9AE035B924489FA4B8
B3AC00DBE81D08D165D25B2F044894089EE01F3670993999FFFA5538C1ED3E419416859D936204B4A72625020
301000133A2CECB126025C120AD866C295292464B6F7A44

SHA-1 Hash Value:

9 - Successful view of HOST-Wincor Signature-July 9, 2007.

Interface example. A sample of the Certificate Management window after selecting the host name, Interface device type, and effective date is shown below. In this example, the root BASE24 system public key is displayed in ASN.1 format and is self-signed. You can copy the root BASE24 system public key cryptogram in the Root Host Public Key Data field, format it as required, and send it on the appropriate media (e.g, e-mail) to the interchange.



Generate message digest

The Generate Message Digest tab on the Certificate Management window enables you to create a message digest of the public key returned from the certificate authority. For NCR and Tidel devices, a message digest is created for the self-signed root NCR public key returned from the NCR trusted center. For Diebold and Triton devices, a message digest is created for the root certificate authority certificate returned from DST or the self-signed root certificate authority certificate returned from Triton. For Wincor Nixdorf signature devices, a message digest is created for the root Wincor Nixdorf public key returned from Wincor

Nixdorf Customer Key Center – International (CKCI). For interfaces, a message digest is created for the root certificate authority public key returned from the interchange. A message digest is only required if you are using Atalla NSPs in your BASE24 system.

For NCR, Tidel, and Wincor Nixdorf signature devices, this tab also enables you to validate the SHA-1 hash value of the root certificate authority public key (i.e., the root NCR public key returned to you on form NCR_004 - NCR Public Key or the root Wincor Nixdorf public key returned to you from the Wincor Nixdorf Customer Key Center – International (CKCI). The SHA-1 hash value only needs to be validated for NCR and Wincor Nixdorf signature devices.

If you are using Diebold or Triton devices and Thales e-Security or Banksys DEP HSMs in your BASE24 system, you do not need to use this tab.

On this tab, you can click Expand View () to expand the Certificate Authority Public Key or Root Certificate Authority Certificate field if needed.

NCR, Tidel, Wincor Nixdorf Signature, or interface device types. To use this tab for NCR devices, Tidel devices, Wincor Nixdorf signature devices, or interfaces, perform the following:

1. Copy the root certificate authority public key cryptogram into the Certificate Authority Public Key field. For NCR and Tidel devices, this is the self-signed root NCR public key cryptogram returned from the NCR trusted center. For Wincor Nixdorf devices, this is the root Wincor Nixdorf public key cryptogram returned from the Wincor Nixdorf Customer Key Center – International (CKCI). For interfaces, this is the root certificate authority public key cryptogram returned from the interchange.
2. Enter the SHA-1 hash value of the root certificate authority public key cryptogram into the SHA-1 Hash Value field. For NCR devices, this is provided in the SHA-1 (Fields 1–6) field on form NCR_004 - NCR Public Key. For Wincor Nixdorf devices, this is the “digest public key, calculated with SHA1” returned in the (pkSignWN<cust>T.mpk or pkSignWN<cust>P.mpk file, or the “Digest (SHA-1 Calculation of pkSignWNWINT Field 1-5)” if returned by fax.

Note: This step does not apply for interfaces.

3. Click Save ()

If you are using Atalla NSPs, the NSP creates a value that is displayed in the Message Digest field. You must copy this value and encrypt it using the SCT or SCA as described in [step 7 of phase 3](#) for NCR and Tidel devices, in [step 9 of phase 3](#) for Wincor Nixdorf devices, or in [step 5 of phase 3](#) for interfaces. The Message Digest field is not displayed or used for Thales e-Security or Banksys DEP HSMs.

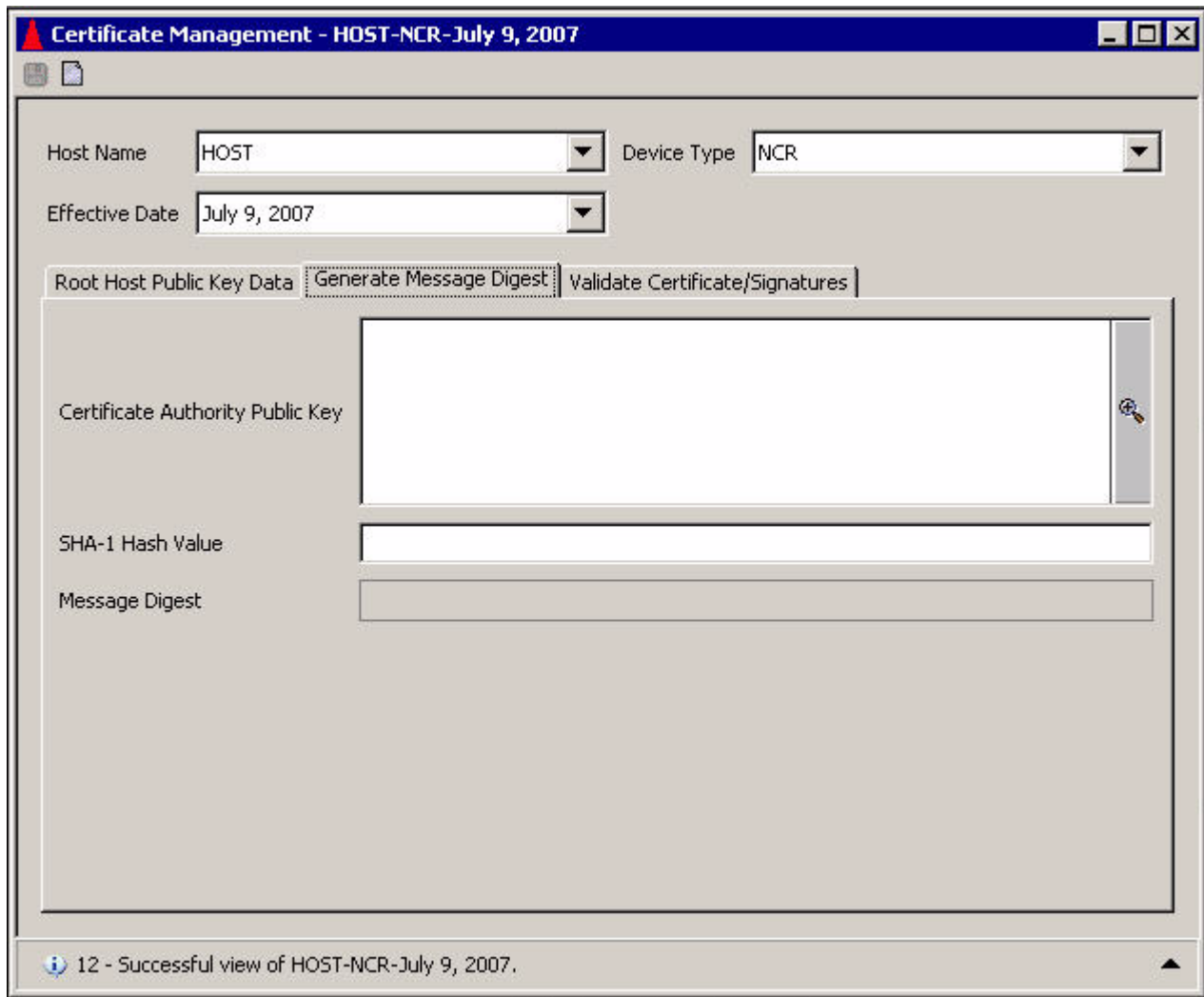
The Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM verifies the signature to ensure that the root public key cryptogram is authentic.

For NCR, Tidel, and Wincor Nixdorf devices, the Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM generates a SHA-1 hash value for the NCR public key cryptogram and compares it to the hash value you entered in the SHA-1 Hash Value field. If the values match, the public key is valid. If the values do not match, either the public key was corrupted or changed during transport or the SHA-1 hash value you entered was incorrectly entered. This SHA-1 hash value comparison is not performed for interfaces.

A message is displayed at the bottom of the window to indicate the success or failure of the signature validation and SHA-1 hash value comparison.

Examples of the Generate Message Digest tab for NCR devices, Tidel devices, Wincor Nixdorf devices, and interfaces with Atalla NSPs and Thales e-Security or Banksys DEP HSMS are shown on the following pages.

The following is a sample of the Generate Message Digest tab displayed for NCR, Tidel, or Wincor/NCR Signature devices and Atalla NSPs.



Certificate Management - HOST-NCR-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data: **Generate Message Digest** | Validate Certificate/Signatures

Certificate Authority Public Key:

SHA-1 Hash Value:

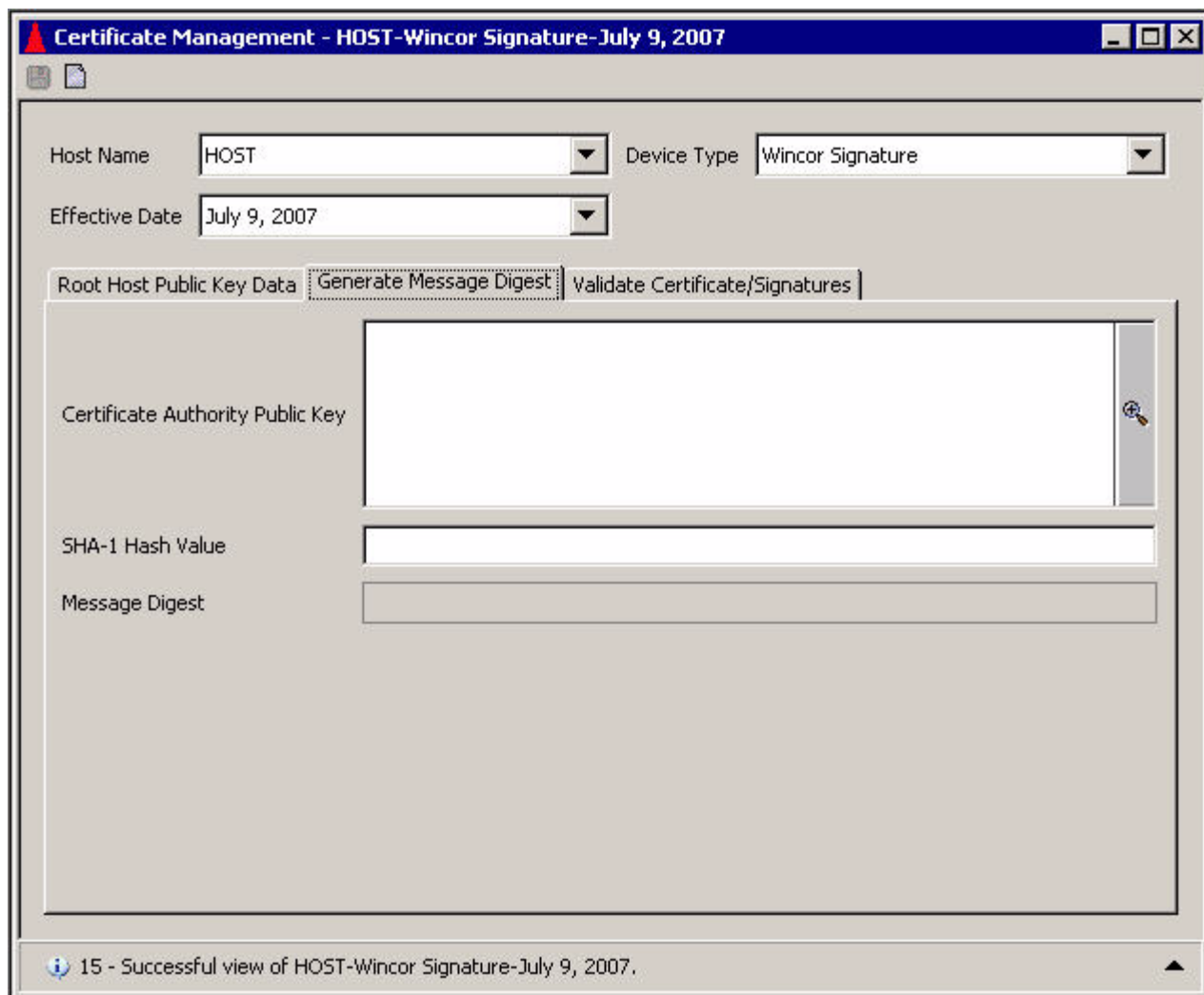
Message Digest:

12 - Successful view of HOST-NCR-July 9, 2007.

The following is a sample of the Generate Message Digest tab displayed for NCR, Tidel, or Wincor/NCR Signature devices and Thales e-Security or Banksys DEP HSMS.

The screenshot shows a software window titled "Certificate Management - HOST-NCR-July 9, 2007". Inside the window, there are three dropdown menus: "Host Name" set to "HOST", "Device Type" set to "NCR", and "Effective Date" set to "July 9, 2007". Below these, there are three tabs: "Root Host Public Key Data", "Generate Message Digest" (which is the active tab), and "Validate Certificate/Signatures". The "Generate Message Digest" tab contains two input fields: "Certificate Authority Public Key" and "SHA-1 Hash Value". The "Certificate Authority Public Key" field is currently empty and has a small key icon on its right side. The "SHA-1 Hash Value" field is also empty. At the bottom of the window, a status bar displays the message "3 - Successful view of HOST-NCR-July 9, 2007.".

The following is a sample of the Generate Message Digest tab displayed for Wincor Nixdorf devices and Atalla NSPs.



Certificate Management - HOST-Wincor Signature-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data: **Generate Message Digest** | Validate Certificate/Signatures

Certificate Authority Public Key:

SHA-1 Hash Value:

Message Digest:

15 - Successful view of HOST-Wincor Signature-July 9, 2007.

The following is a sample of the Generate Message Digest tab displayed for Wincor Nixdorf devices and Thales e-Security or Banksys DEP HSMs.

The following is a sample of the Generate Message Digest tab displayed for interfaces and Atalla NSPs.

The screenshot shows a software window titled "Certificate Management - SAMATEST-Interface-August 7, 2007". Inside the window, there are several input fields and tabs. The "Host Name" field contains "SAMATEST", the "Device Type" dropdown is set to "Interface", and the "Effective Date" dropdown is set to "August 7, 2007". Below these fields are three tabs: "Root Host Public Key Data", "Generate Message Digest" (which is the active tab), and "Validate Certificate/Signatures". The "Generate Message Digest" tab contains a large text area labeled "Certificate Authority Public Key" and a smaller text area labeled "Message Digest". The status bar at the bottom of the window displays the message "9 - Successful view of SAMATEST-Interface-August 7, 2007."

The following is a sample of the Generate Message Digest tab displayed for interfaces and Thales e-Security or Banksys DEP HSMS.

Diebold or Triton devices. To use this tab for Diebold or Triton devices and Atalla NSPs, perform the following:

1. Copy the root certificate authority certificate returned from DST or the self-signed root certificate authority certificate returned from Triton into the Root Certificate Authority Certificate field.
2. Click Save (📁). The NSP creates a value that is displayed in the Message Digest field. You must copy this value and encrypt it using the SCT as described in [step 5 of phase 2](#) for Diebold devices or as described in [step 5 of phase 2](#) for Triton devices.

The following is a sample of the Generate Message Digest tab displayed for Diebold or Triton devices and Atalla NSPs.

The screenshot shows a software window titled "Certificate Management - HOST-Diebold-August 23, 2002". Inside the window, there are several input fields and a tabbed interface. The "Host Name" field contains "HOST", the "Device Type" dropdown is set to "Diebold", and the "Effective Date" dropdown is set to "August 23, 2002". Below these fields is a tabbed interface with three tabs: "Root Host Public Key Data", "Generate Message Digest" (which is the active tab), and "Validate Certificate/Signatures". The "Generate Message Digest" tab contains two large text areas: "Root Certificate Authority Certificate" and "Message Digest". The "Root Certificate Authority Certificate" area is currently empty, while the "Message Digest" area contains a single line of text. At the bottom of the window, a status bar displays the message "18 - Successful view of HOST-Diebold-August 23, 2002.".

Host Name: HOST Device Type: Diebold Effective Date: August 23, 2002

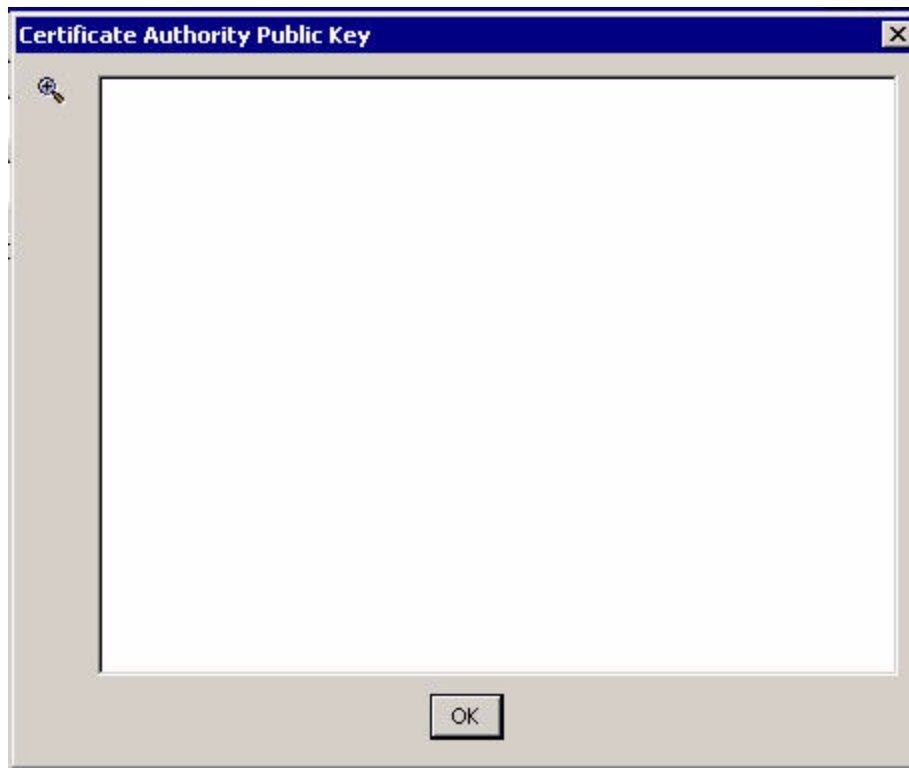
Root Host Public Key Data Generate Message Digest Validate Certificate/Signatures

Root Certificate Authority Certificate

Message Digest

18 - Successful view of HOST-Diebold-August 23, 2002.

Expanded Field. The following image shows you the Certificate Authority Public Key field after it has been expanded.



Validate certificates/signatures

The Validate Certificates/Signatures tab on the Certificate Management window enables you to validate and enter certificate authority and BASE24 system (host) public key information into the TSS database. The fields displayed on this tab differ, depending on the device type selected and type of security devices used.

When you select the NCR or Wincor/NCR Signature device type, the following fields are displayed:

- Root Certificate Hash Value (for Atalla NSPs only)
- Certificate Authority Public Key
- Host Signature
- SHA-1 Hash Value

When you select the Diebold device type, the following fields are displayed:

- Root Certificate Hash Value (for Atalla NSPs only)
- Root Certificate Authority Certificate
- Certificate Authority Primary Certificate
- Host Primary Certificate

When you select the Triton device type, the following fields are displayed:

- Root Certificate Hash Value (for Atalla NSPs only)
- Root Certificate Authority Certificate
- Host Primary Certificate

When you select the Wincor Signature device type, the following fields are displayed:

- Root Certificate Hash Value (for Atalla NSPs only)
- Certificate Authority Public Key
- Host Signature

When you select the Interface device type, the following fields are displayed:

- Root Certificate Hash Value (for Atalla NSPs only)
- Certificate Authority Public Key

You can expand the fields for which Expand View () is displayed.

If you are using Atalla NSPs, you must enter the root certificate hash value encrypted from the SCA or SCT into the Root Certificate Hash Value field as described in [step 8 of phase 3](#) for NCR devices, in [step 6 of phase 2](#) for Diebold devices, in [step 6 of phase 2](#) for Triton devices, in [step 10 of phase 3](#) for Wincor Nixdorf devices, or in [step 6 of phase 3](#) for interfaces.

For NCR, Tidel, or Wincor Nixdorf/NCR signature devices, you must also enter the following:

- Self-signed root NCR public key in the Certificate Authority Public Key.
- Root BASE24 system public key signature returned from the NCR trusted center in the Host Signature field.
- SHA-1 hash value for the root BASE24 system public key returned from the NCR trusted center in the SHA-1 (Field 1) field on form NCR_005 - NCR Signature of Customer Public Key.

For Diebold devices, you must also enter the following:

- The root certificate authority certificate in the Root Certificate Authority Certificate field.
- The primary certificate authority's public signature verification key certificate in the Certificate Authority Primary Certificate field.

- The host (BASE24 system) primary public signature verification key certificate in the Host Primary Certificate field.

For Triton devices, you must also enter the following:

- The primary certificate authority's public signature verification key certificate in the Root Certificate Authority Certificate field.
- The host (BASE24 system) primary public signature verification key certificate in the Host Primary Certificate field.

For Wincor Nixdorf devices, you must also enter the following:

- Root Wincor Nixdorf public key in the Certificate Authority Public Key.
- Root BASE24 system public key signature returned from the Wincor Nixdorf Customer Key Center – International (CKCI) in the Host Signature field.

For interfaces, you must also enter the following:

- The self-signed root certificate authority public key in the Certificate Authority Public Key in ASCII hexadecimal ASN.1 format.

After all the required data is entered, click Save () to validate signatures and hash values and store data in the Host Public Key Security File (HOST_PUBLIC_KEY).

The Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM verifies the signature or certificates returned from the certificate authority to ensure their authenticity.

For NCR and Tidel devices, the Atalla NSP, Thales e-Security HSM, or Banksys DEP HSM generates a SHA-1 hash value for the root BASE24 system public key cryptogram and compares it to the hash value you entered in the SHA-1 Hash Value field. If the values match, the public key is valid. If the values do not match, either the public key was corrupted or changed during transport or the SHA-1 hash value you entered was incorrectly entered.

A message is displayed at the bottom of the window to indicate the success or failure of the signature or certificate validation and SHA-1 hash value comparison.

A sample of the Validate Certificates/Signatures tab is shown below for NCR, Tidel, or Wincor/NCR Signature devices and Atalla NSPs.

The screenshot shows a software window titled "Certificate Management - HOST-NCR-July 9, 2007". The window contains several input fields and tabs. At the top, there are three dropdown menus: "Host Name" with "HOST" selected, "Device Type" with "NCR" selected, and "Effective Date" with "July 9, 2007" selected. Below these are three tabs: "Root Host Public Key Data", "Generate Message Digest", and "Validate Certificate/Signatures", with the third tab being the active one. The main area of the window contains four labels on the left and corresponding input fields on the right: "Root Certificate Hash Value" with a single-line text box, "Certificate Authority Public Key" with a multi-line text box and a key icon, "Host Signature" with a multi-line text box and a key icon, and "SHA-1 Hash Value" with a single-line text box. At the bottom of the window, there is a status bar with an information icon and the text "3 - Successful view of HOST-NCR-July 9, 2007."

Host Name: HOST Device Type: NCR Effective Date: July 9, 2007

Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

Root Certificate Hash Value: [Text Box]

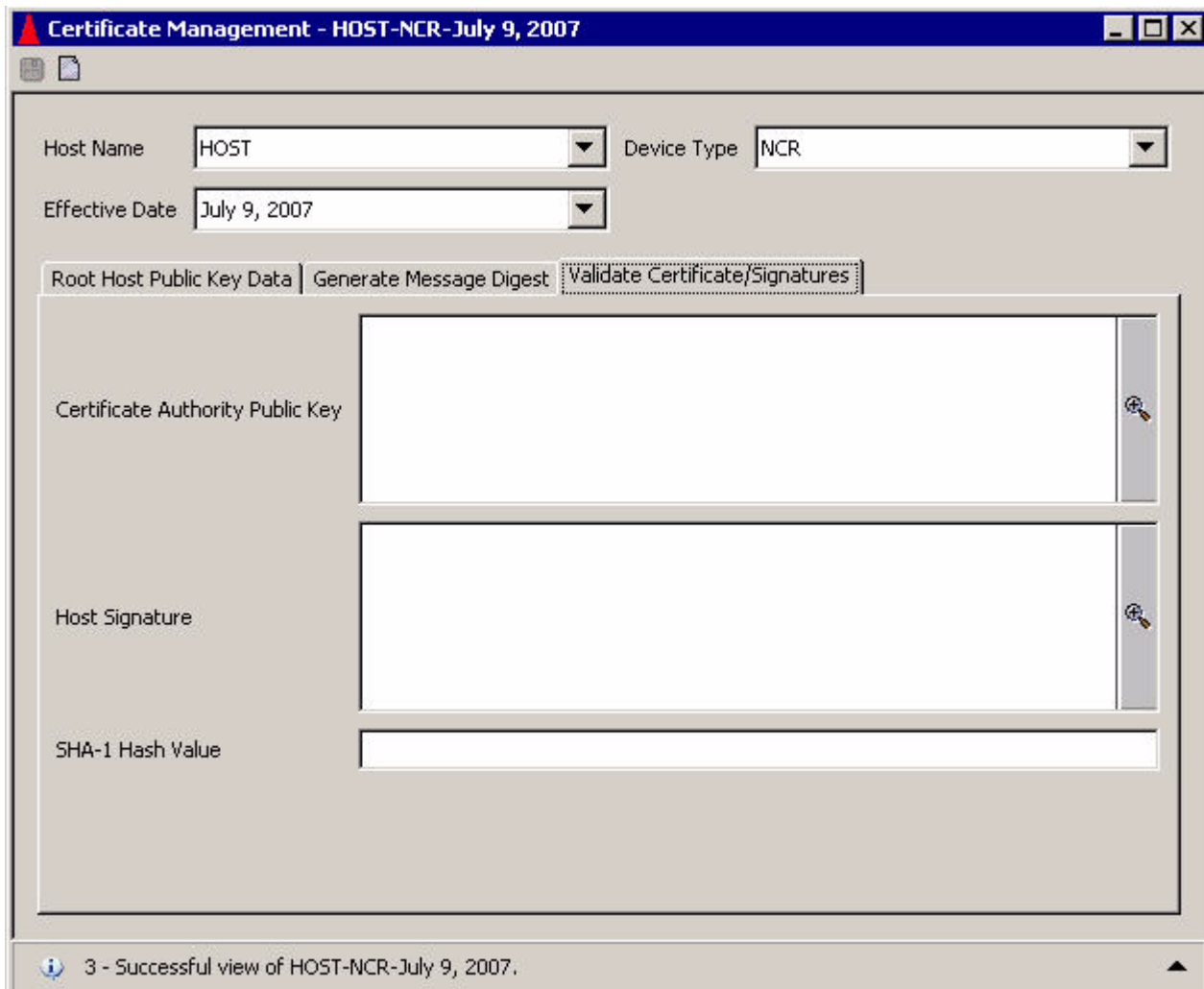
Certificate Authority Public Key: [Text Box] [Key Icon]

Host Signature: [Text Box] [Key Icon]

SHA-1 Hash Value: [Text Box]

3 - Successful view of HOST-NCR-July 9, 2007.

A sample of the Validate Certificates/Signatures tab is shown below for NCR, Tidel, or Wincor/NCR Signature devices and Thales e-Security or Banksys DEP HSMS.



Certificate Management - HOST-NCR-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

Certificate Authority Public Key:

Host Signature:

SHA-1 Hash Value:

3 - Successful view of HOST-NCR-July 9, 2007.

A sample of the Validate Certificates/Signatures tab is shown below for Diebold devices and Atalla NSPs.

The screenshot shows a window titled "Certificate Management - HOST-Diebold-August 23, 2002". The window contains several input fields and tabs. At the top, there are three dropdown menus: "Host Name" with the value "HOST", "Device Type" with the value "Diebold", and "Effective Date" with the value "August 23, 2002". Below these, there are three tabs: "Root Host Public Key Data", "Generate Message Digest", and "Validate Certificate/Signatures". The "Validate Certificate/Signatures" tab is selected. Under this tab, there are four rows of input fields, each with a label on the left and a text box on the right. The labels are "Root Certificate Hash Value", "Root Certificate Authority Certificate", "Certificate Authority Primary Certificate", and "Host Primary Certificate". Each text box is empty. To the right of each text box is a small icon of a magnifying glass over a document. At the bottom of the window, there is a status bar that says "6 - Successful view of HOST-Diebold-August 23, 2002."

Host Name: HOST Device Type: Diebold Effective Date: August 23, 2002

Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

Root Certificate Hash Value

Root Certificate Authority Certificate

Certificate Authority Primary Certificate

Host Primary Certificate

6 - Successful view of HOST-Diebold-August 23, 2002.

A sample of the Validate Certificates/Signatures tab is shown below for Diebold devices and Thales e-Security or Banksys DEP HSMS.

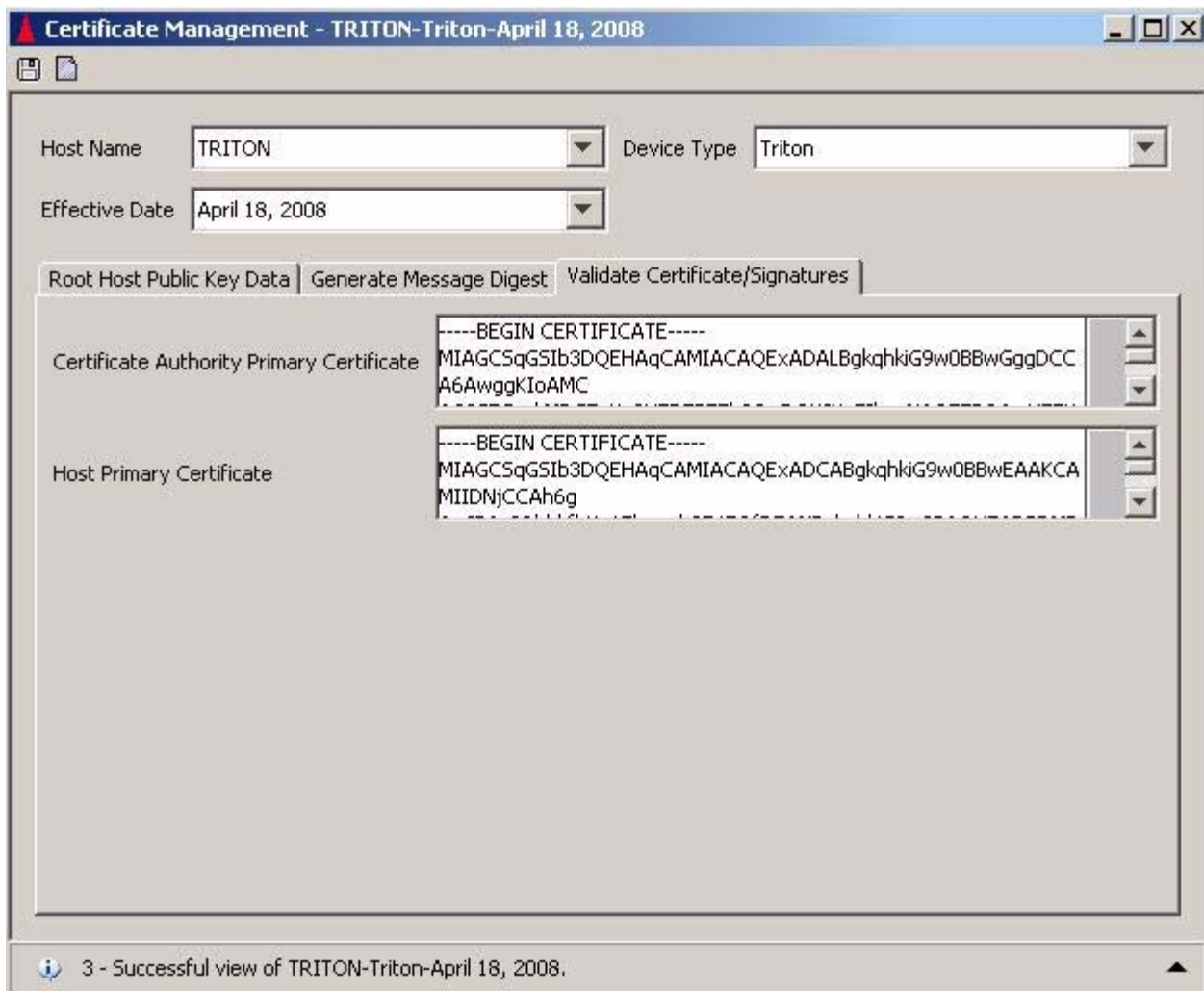
The screenshot shows a software window titled "Certificate Management - HOST-Diebold-February 15, 2005". The window contains several input fields and a tabbed interface. At the top, there are three dropdown menus: "Host Name" set to "HOST", "Device Type" set to "Diebold", and "Effective Date" set to "February 15, 2005". Below these is a tabbed interface with three tabs: "Root Host Public Key Data", "Generate Message Digest", and "Validate Certificate/Signatures". The "Validate Certificate/Signatures" tab is active, showing three rows of input fields for certificates. Each row has a label on the left and a text input field on the right with a key icon button. The labels are "Root Certificate Authority Certificate", "Certificate Authority Primary Certificate", and "Host Primary Certificate". The input fields are empty. At the bottom of the window, a status bar displays the message "15 - Successful view of HOST-Diebold-February 15, 2005."

Host Name	Device Type	Effective Date
HOST	Diebold	February 15, 2005

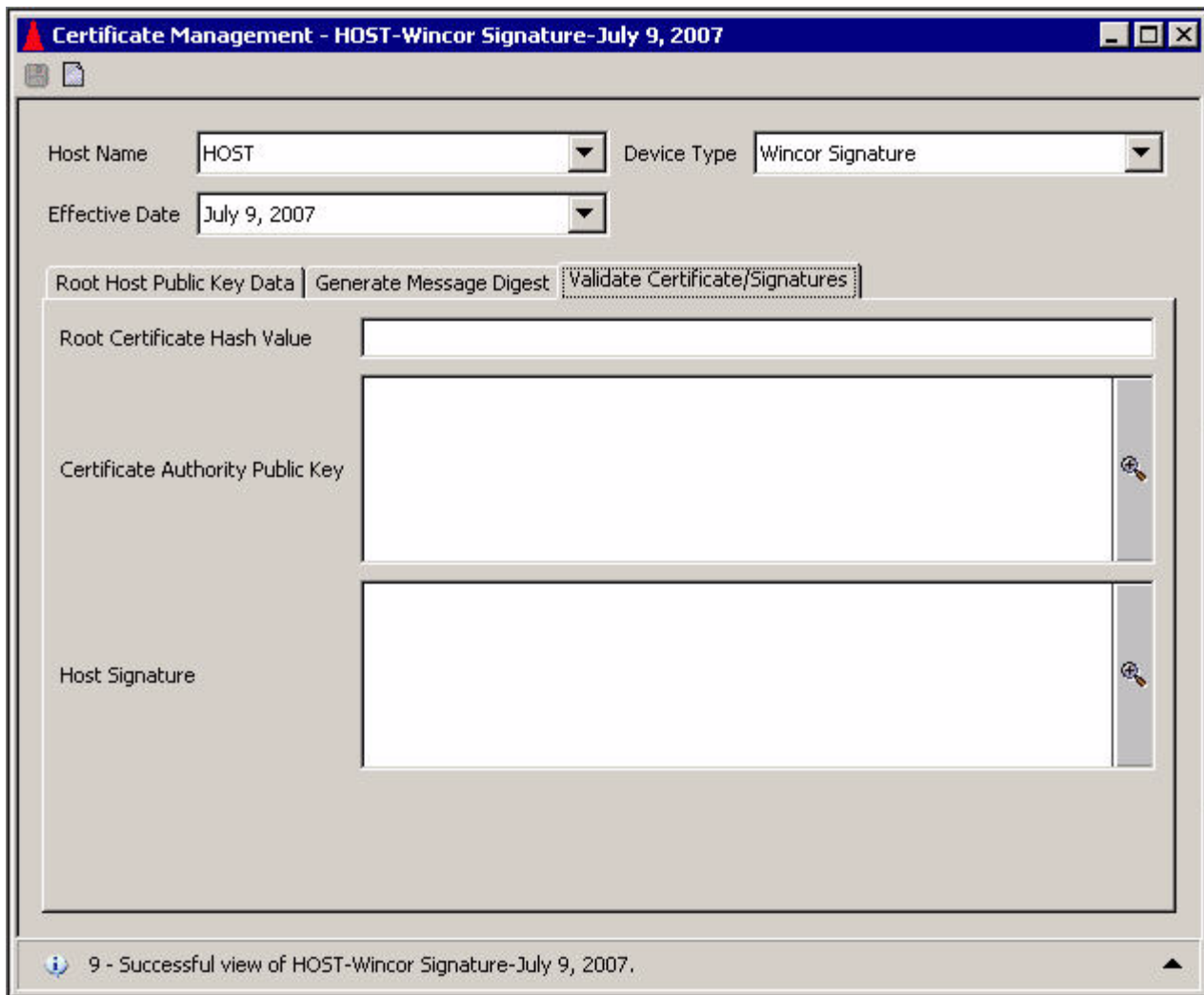
Root Host Public Key Data	Generate Message Digest	Validate Certificate/Signatures
Root Certificate Authority Certificate		
Certificate Authority Primary Certificate		
Host Primary Certificate		

15 - Successful view of HOST-Diebold-February 15, 2005.

A sample of the Validate Certificates/Signatures tab is shown below for Triton devices and Thales e-Security HSMs.



A sample of the Validate Certificates/Signatures tab is shown below for Wincor Nixdorf devices and Atalla NSPs.



Certificate Management - HOST-Wincor Signature-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

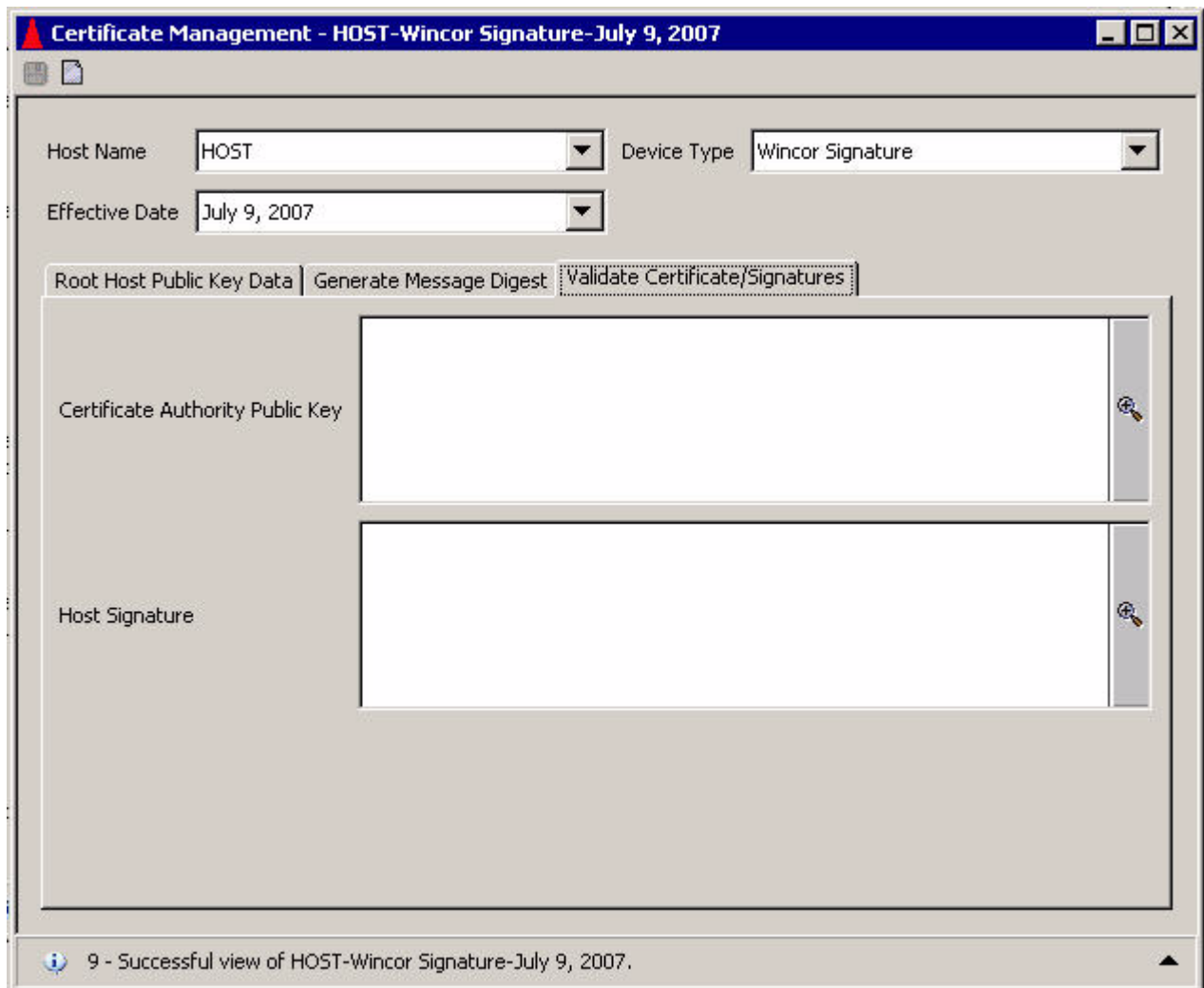
Root Certificate Hash Value:

Certificate Authority Public Key:

Host Signature:

9 - Successful view of HOST-Wincor Signature-July 9, 2007.

A sample of the Validate Certificates/Signatures tab is shown below for Wincor Nixdorf devices and Thales e-Security or Banksys DEP HSMs.



Certificate Management - HOST-Wincor Signature-July 9, 2007

Host Name: Device Type:

Effective Date:

Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

Certificate Authority Public Key

Host Signature

9 - Successful view of HOST-Wincor Signature-July 9, 2007.

A sample of the Validate Certificates/Signatures tab is shown below for interfaces and Atalla NSPs.

Certificate Management - SAMATEST-Interface-August 7, 2007

Host Name: Device Type:

Effective Date:

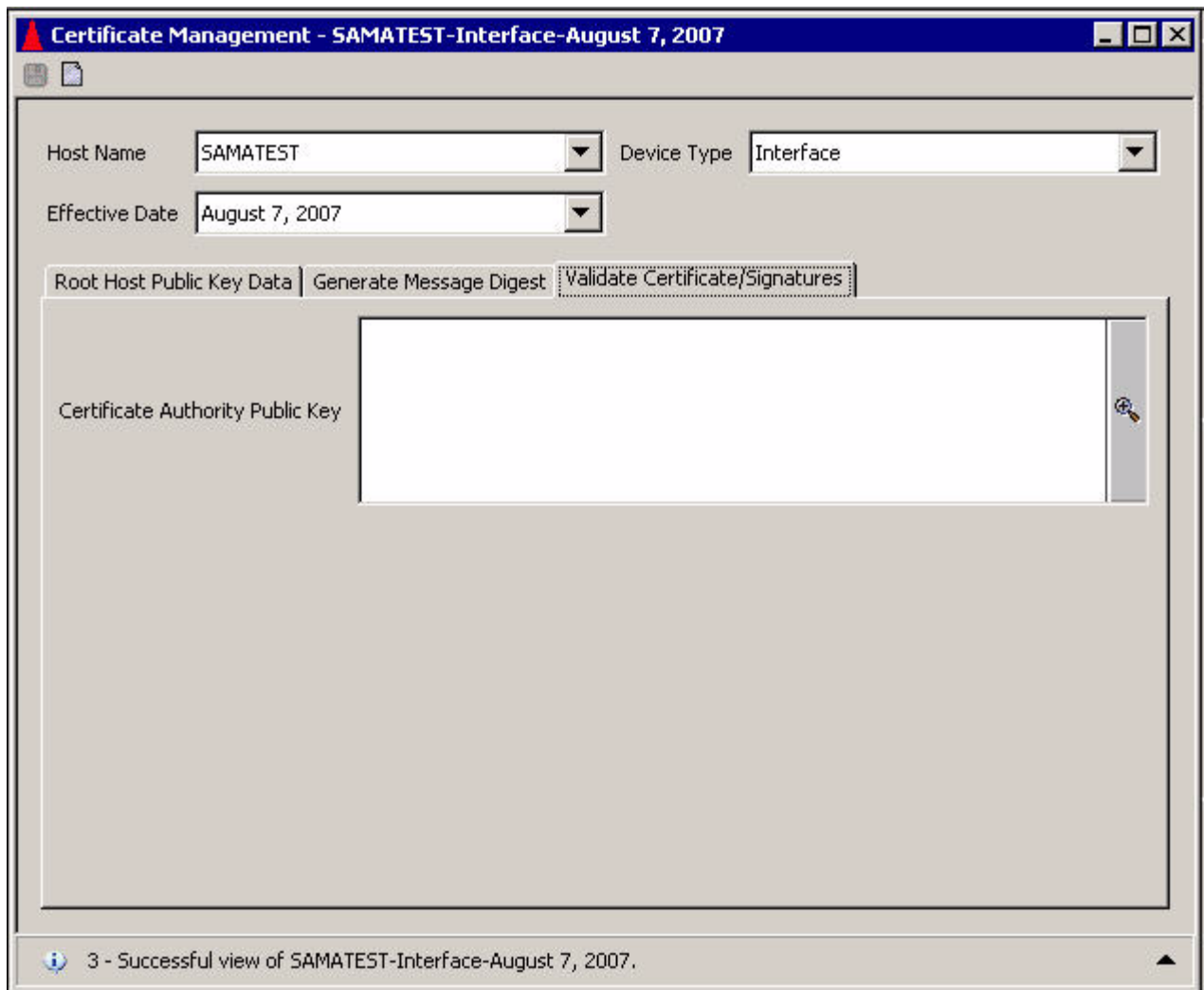
Root Host Public Key Data | Generate Message Digest | **Validate Certificate/Signatures**

Root Certificate Hash Value:

Certificate Authority Public Key:

9 - Successful view of SAMATEST-Interface-August 7, 2007.

A sample of the Validate Certificates/Signatures tab is shown below for interfaces and Thales e-Security or Banksys DEP HSMS.



Host Public Key Perusal window

The Host Public Key Perusal window (access **View > Automated Key Distribution > Host Public Key Perusal**) enables you to view root BASE24 system public keys, primary BASE24 system public keys, and root certificate authority (CA) public keys stored in the Host Public Key File (HOST_PUBLIC_KEY). You can also view the signature (for NCR devices, Tidel devices, Wincor Nixdorf devices, or interfaces) or certificate (for Diebold and Triton devices) associated with the host root (28) public key. For Diebold, Triton, or shared Wincor Nixdorf certificate devices, you can also view primary certificate authority public keys for signature validation and their associated certificates. Each public key is assigned a key type when it is stored in the HOST_PUBLIC_KEY as shown in the following table.

Key type	Description
CA Root (15)	Identifies a root certificate authority public key for all device types.
CA Primary (22)	Identifies a primary certificate authority public key for signature validation for Diebold and Triton devices.
Host Root (28)	Identifies a root BASE24 system public key for all device types.
Host Primary (11)	Identifies a primary BASE24 system public key for the NCR enhanced signature remote key loading authentication scheme.

All of the public keys currently stored in the HOST_PUBLIC_KEY for a host name and device type are displayed in a table on the upper portion of the Host Public Key Perusal window when you access the window and select a host name and device type. Each row in the table identifies the key type and effective date for the public key as well as the public key itself. Additional columns in the table provide the MAC and check digits associated with the public key and any security device specific information such as the AKB header for Atalla AKB NSPs.

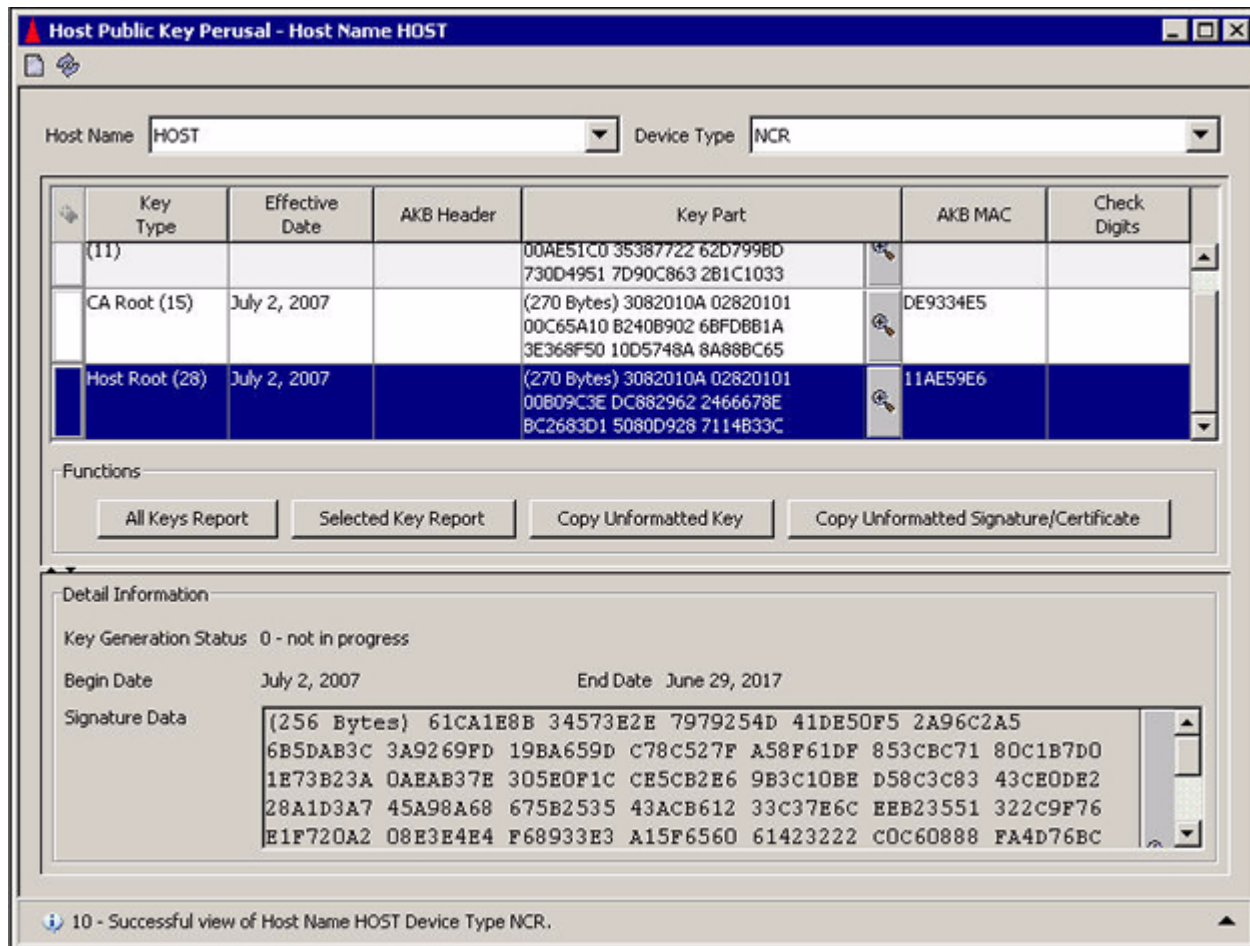
Detail information for the currently selected public key row is displayed on the lower portion of the window. For NCR devices, Tidel devices, Wincor Nixdorf devices, or interfaces, detail information includes the key generation status, begin and end validity dates (if applicable for the key type) and the key signature (if applicable for the key type). For Diebold and Triton devices, detail information includes the key generation status, validity range for the key, and the key certificate. For host root (28) certificates, Diebold device detail information also includes the serial number, common name, and organization name used to create the certificate. For certificate authority root (15) and primary (22) certificates, Diebold device detail information also includes a hash of the distinguished name from the certificate.

The middle portion of the window provides the following four buttons to execute specific functions:

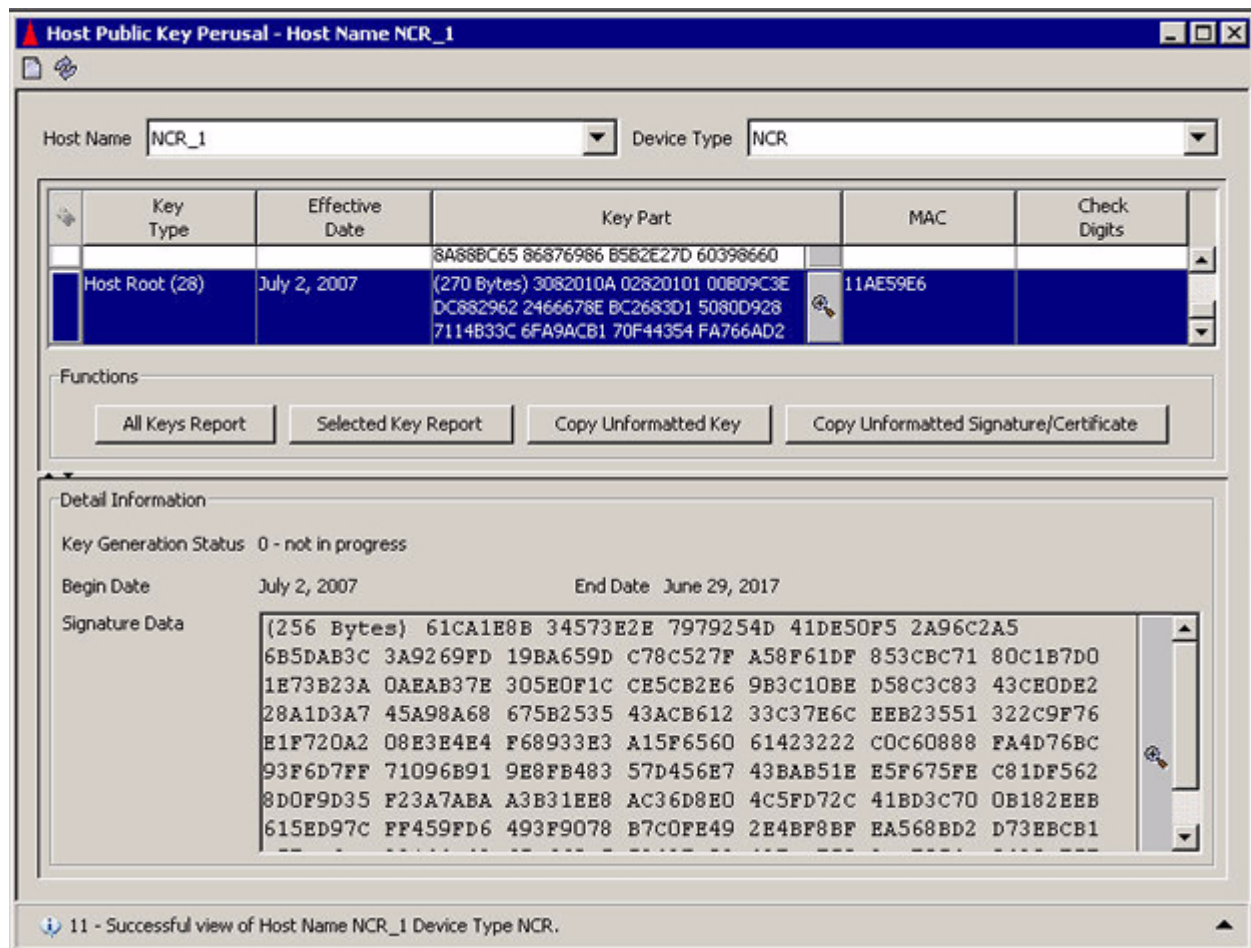
- The All Keys Report button generates a text listing of all stored information for all public keys displayed in the key table.
- The Selected Key Report button generates a text listing of all stored information for the selected public key only.

- The Copy Unformatted Key button copies the selected public key unformatted to the clipboard for pasting into another application.
- The Copy Unformatted Signature/Certificate copies the certificate or signature of the selected public key unformatted to the clipboard for pasting into another application.

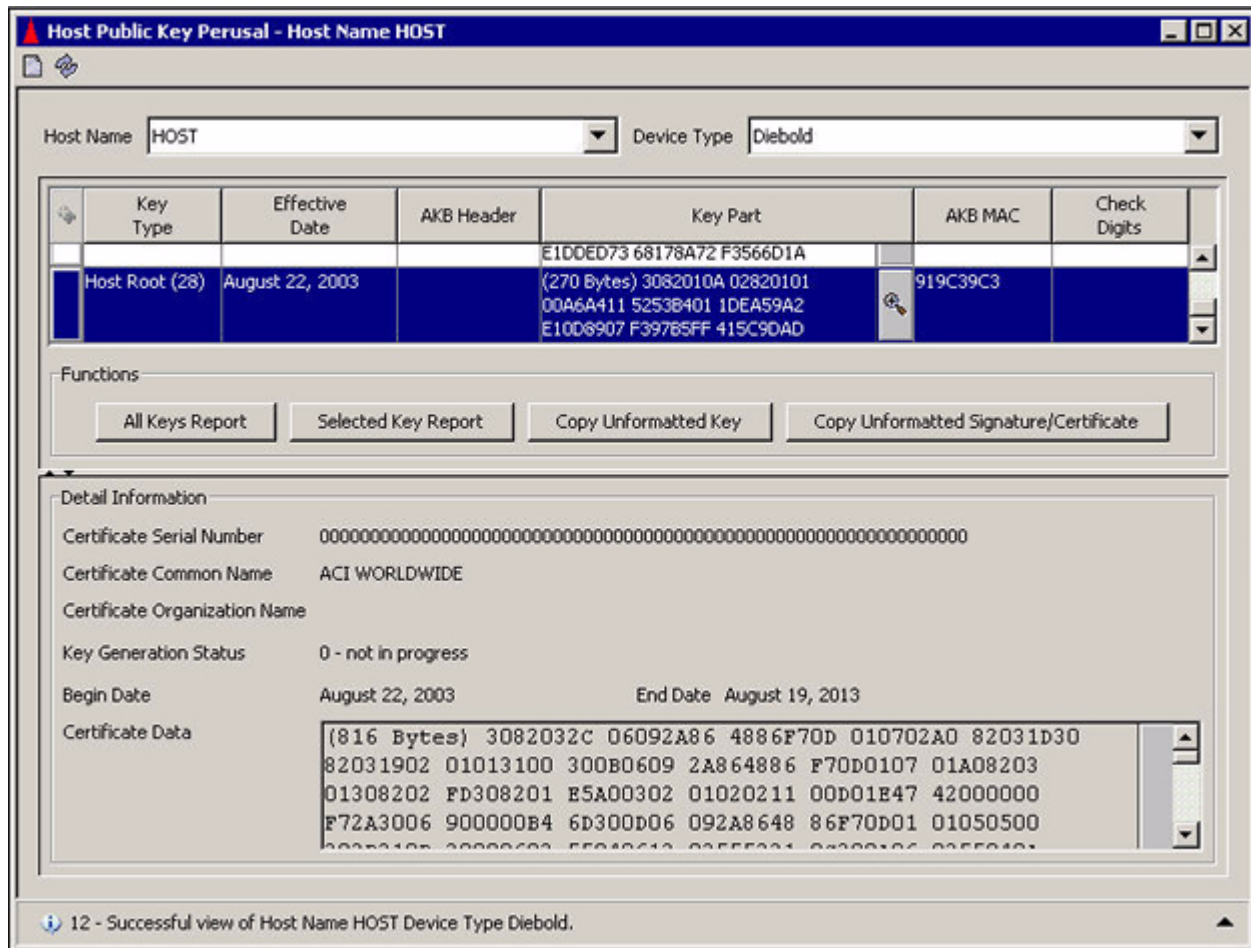
The following illustration shows you a sample of the Host Public Key Perusal window for NCR or Tidel devices and Atalla NSPs. The fields displayed for Wincor Nixdorf devices and interfaces are similar.



The following illustration shows you a sample of the Host Public Key Perusal window for NCR or Tidel devices and Thales e-Security or Banksys DEP HSMs. The fields displayed for Wincor Nixdorf devices and interfaces are similar.

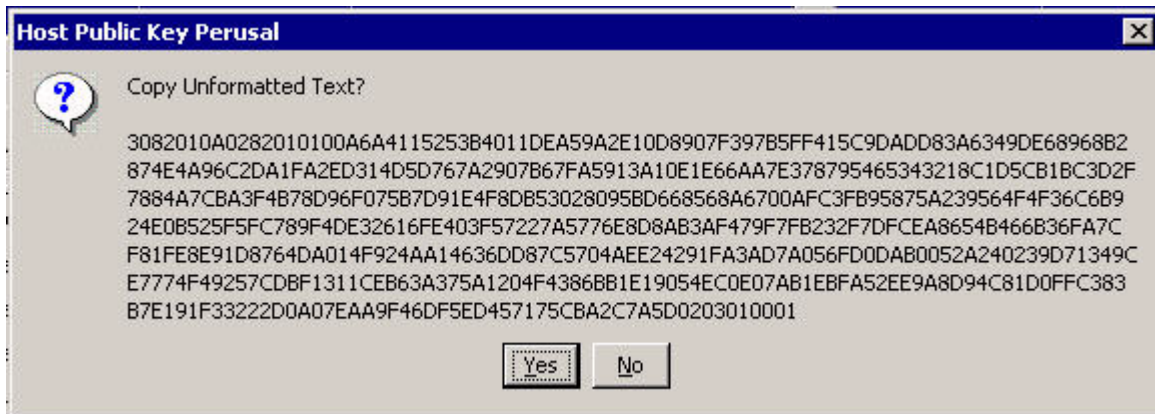


The next illustration shows you a sample of the Host Public Key Perusal window for Diebold or Triton devices and Atalla NSPs.

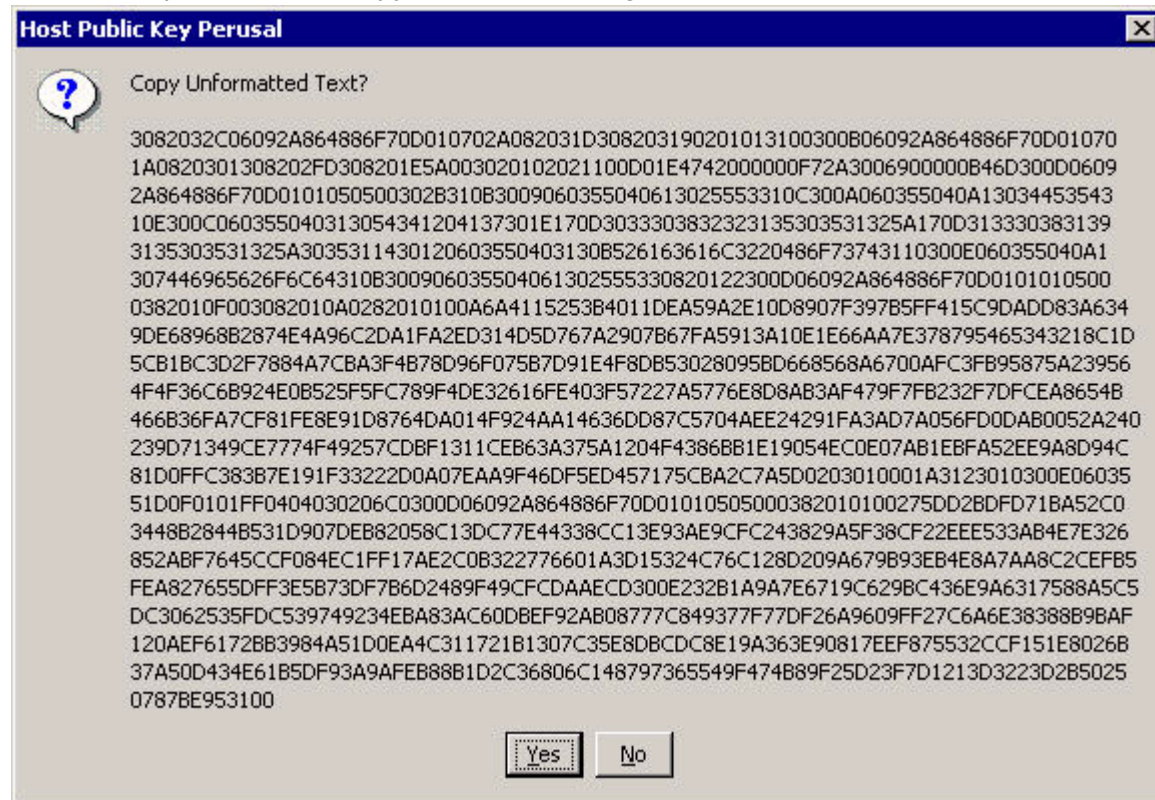


To copy the key text as unformatted text, select a row for the key text you want to copy and click the Copy Unformatted Key or Copy Unformatted Signature/Certificate button. The full unformatted value of the selected key element is displayed in a pop-up window from which you can copy the value to your clipboard by clicking Yes. You can then easily paste the value into another application. A sample of these pop-up windows are shown below.

The first sample is for the Copy Unformatted Key button.



The next sample is for the Copy Unformatted Signature/Certificate button.





The next Key Report window displays all the fields and values for all the keys in the key table. Each key entry is separated by a horizontal dotted line.

Key Report

Host Name	HOST
Device Type	Diebold
Key Type	CA Root (15)
Effective Date	August 22, 2003
Key Part	(270 Bytes) 3082010A 02820101 00DFAFE9 97500883 57B4CC62 65F69082 ECC7D32C 6B30CA5B ECD9C37D C740C118 148BEOE8 3376492A E33F2149 93AC4E0E AF3E48CB 65EEFCD3 210F65D2 2AD9328F 8CE5F777 B0127BB5 95C089A3 A9BAED73 2E7A0C06 3283A27E 8A1430CD 11A0E12A 38B9790A 31FD50BD 8065DFB7 516383C8 E28861EA 4B6181EC 526BB9A2 E24B1A28 9F48A39E OCDA098E 3E172E1E DD20DF5B C62A8AAB 2EBD70AD C50B1A25 907472C5 7B6AAB34 D63089FF E568137B 540BC8D6 AEEC5A9C 921E3D64 B38CC6DF BFC94170 EC1672D5 26EC3855 3943D0FC FD185C40 F197EBD5 9A9B8D1D BADA25B9 C6D8DFC1 15023AAB DA6EF13E 2EF55C08 9C3CD683 69E4109B 192AB629 57E3E53D 9B9FF002 5D020301 0001
MAC	087706D4
Check Digits	
Distinguished Name Hash Value	
Begin Date	
End Date	

Host Name	HOST

Close

Channel Public Key Perusal window

The Channel Public Key Perusal window (access **View > Automated Key Distribution > Channel Public Key Perusal**) enables you to view the EPP encipherment and signing public keys for individual terminal IDs (channel IDs). All of these keys are stored in the Channel Public Key Security data source (CHAN_PKEY_SEC) when public keys are exchanged with the BASE24 system. Both encipherment and signature types of public keys are displayed in a table on the upper portion of the window when you access the window and enter a channel ID. If you do not know the channel ID, you can use the Find function () to select it from a list. Encipherment public keys are used for all device types, while signing public keys are only used for Diebold or Triton devices.

Detail information for the currently selected public key is displayed on the lower portion of the window. For Diebold and Triton devices, this includes the validity date range for the public key, the EPP random nonce, and the host random nonce. For NCR, Tidel, and Wincor Nixdorf devices, this includes the EPP serial number. The begin and end validity dates do not apply to NCR, Tidel, and Wincor Nixdorf devices.

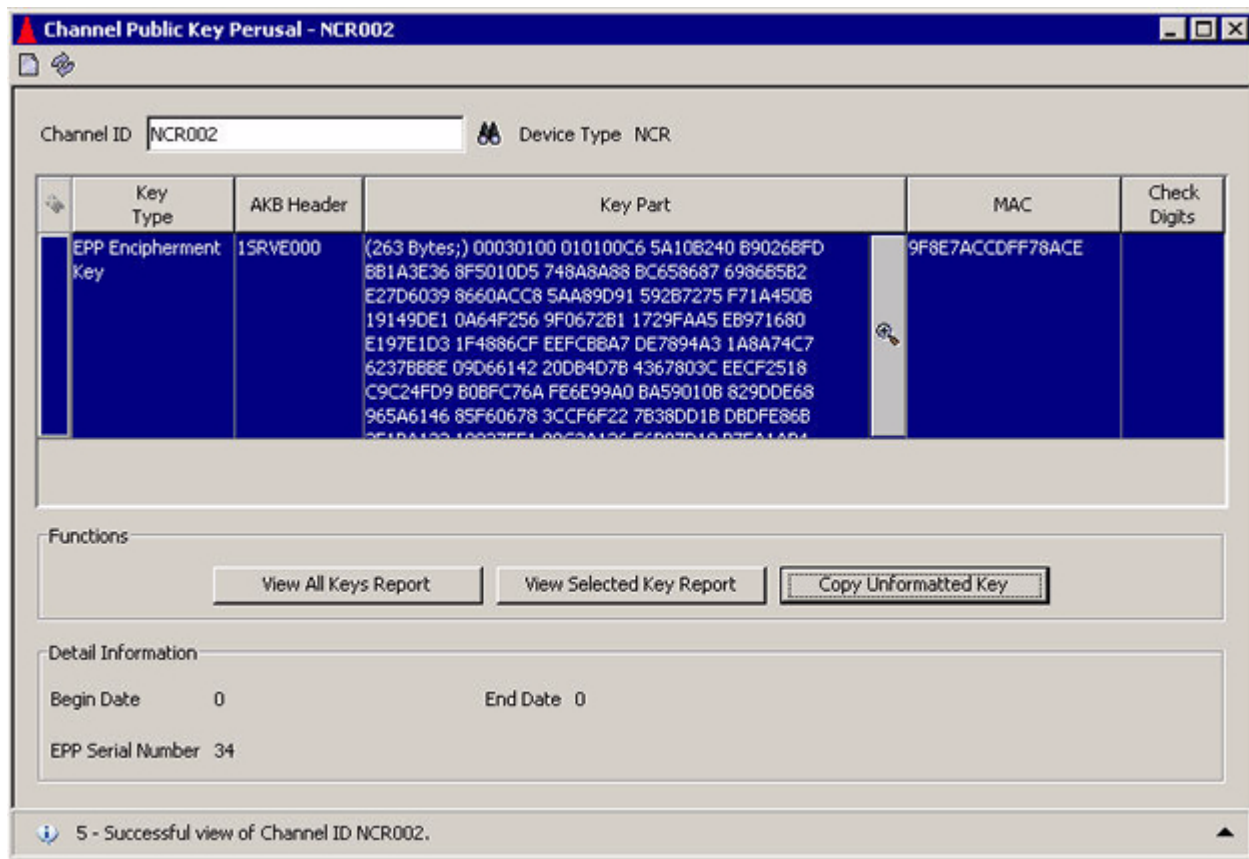
For certificate-based AKDS implementations (i.e., Diebold and Triton devices), the Begin Date and End Date field values are set to the certificate validity dates obtained from the certificate data.

For signature-based AKDS implementations (i.e., NCR, Tidel, and Wincor Nixdorf devices), the Begin Date and End Date field values are set to nulls (0), which translates to the beginning of time (January 1, 1970) on HP NonStop platforms, when the CHAN_PKEY_SEC record is added.

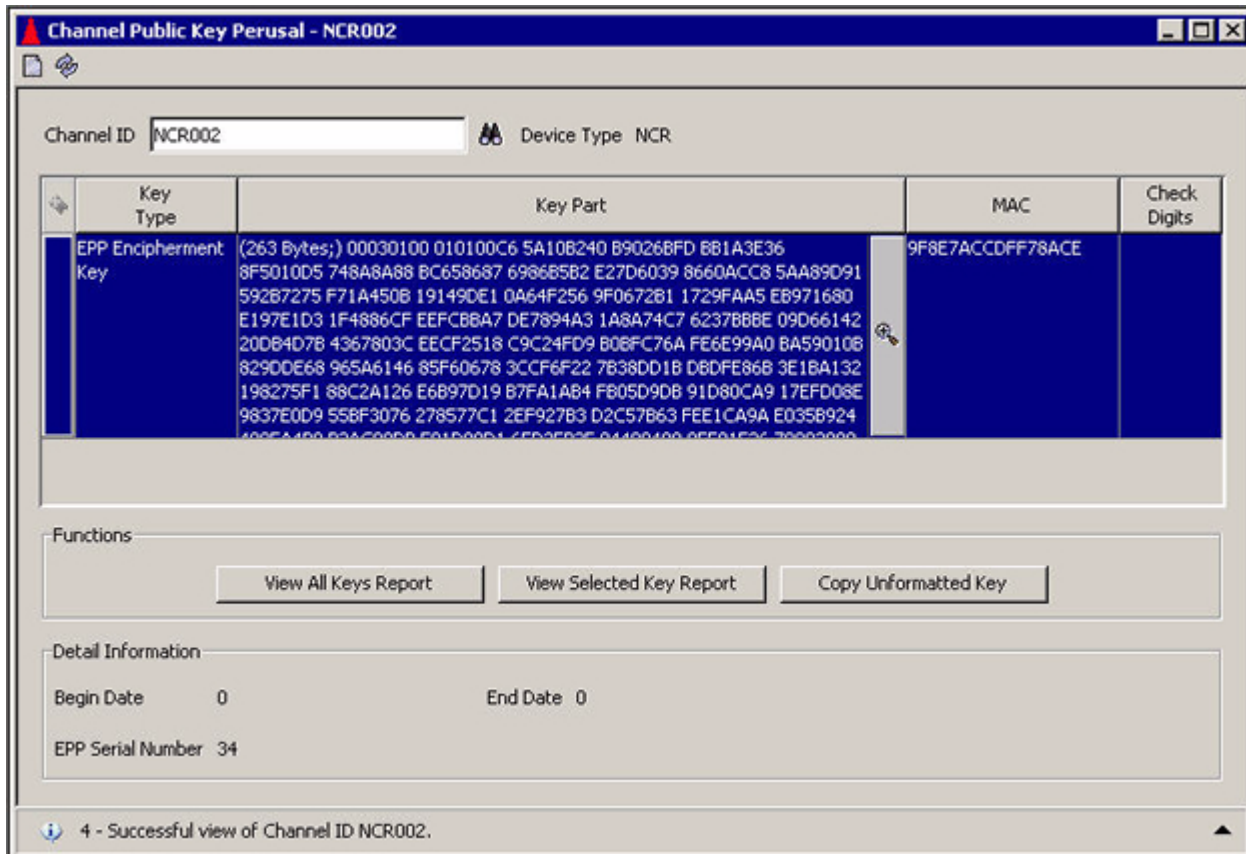
The middle portion of the window provides the following three buttons to execute specific functions:

- The View All Keys Report button generates a text listing of all stored information for all public keys displayed in the key table.
- The View Selected Key Report button generates a text listing of all stored information for the selected public key only.
- The Copy Unformatted Key button copies the selected public key unformatted to the clipboard for pasting into another application.

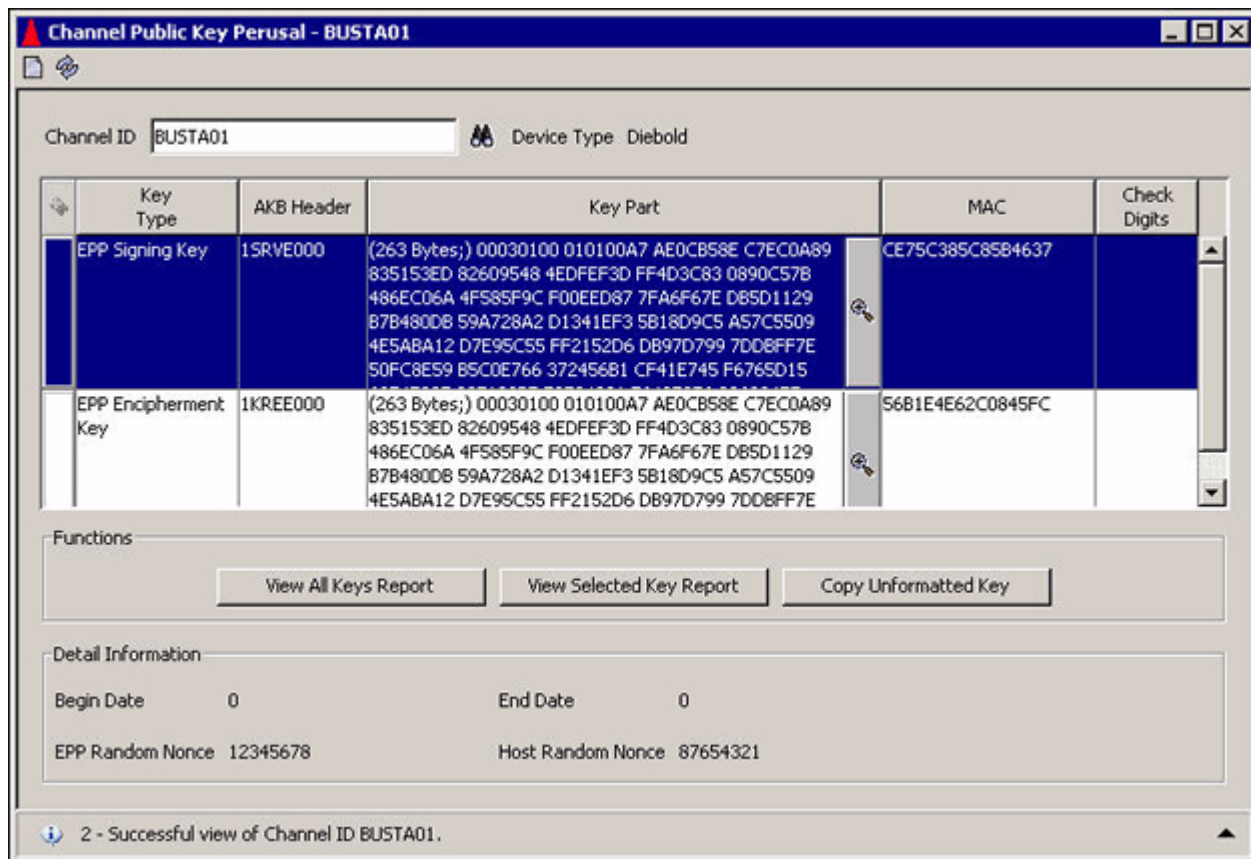
The following illustration shows you a sample of the Channel Public Key Perusal window for an NCR or Tidel device and Atalla NSPs. The display for Wincor Nixdorf devices is similar.



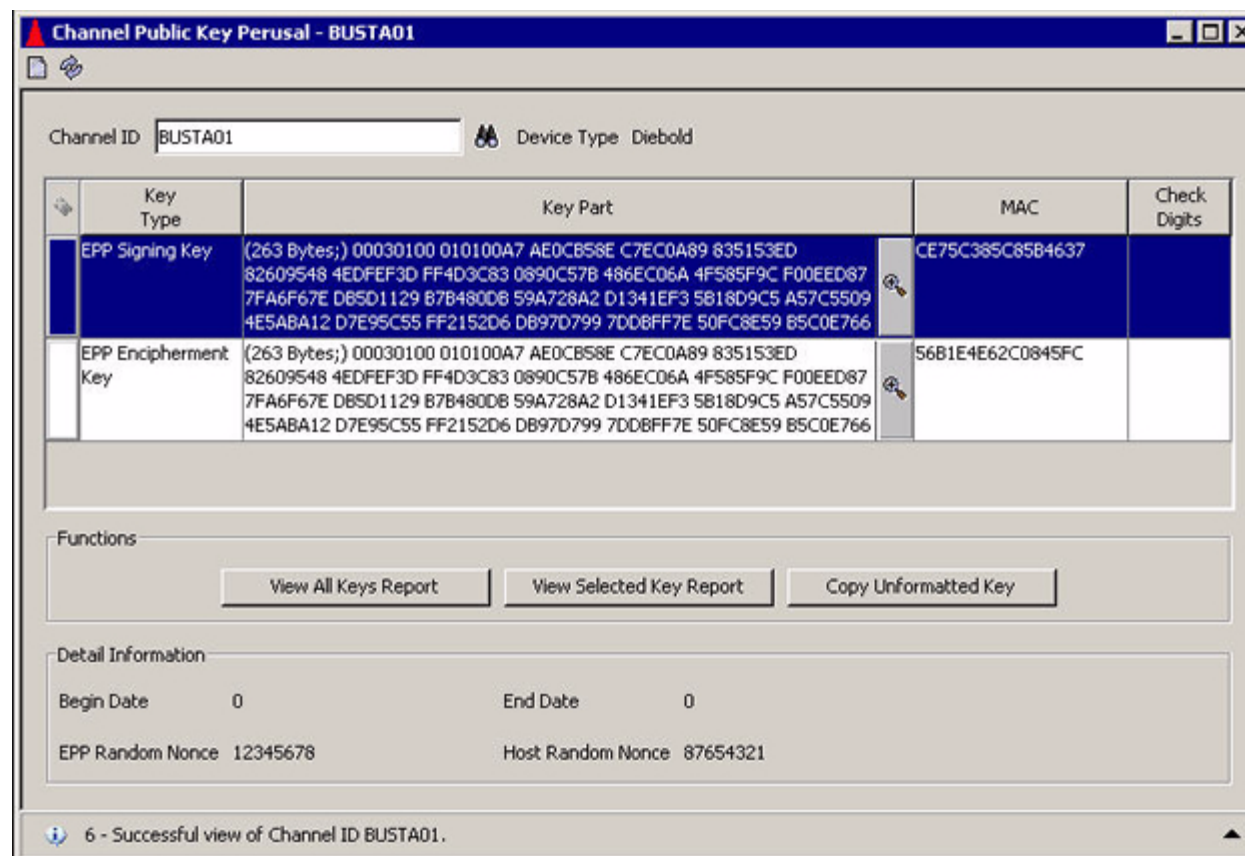
The following illustration shows you a sample of the Channel Public Key Perusal window for an NCR or Tidel device and Thales e-Security or Banksys DEP HSMs. The display for shared Wincor Nixdorf/NCR signature and Wincor Nixdorf devices is similar.



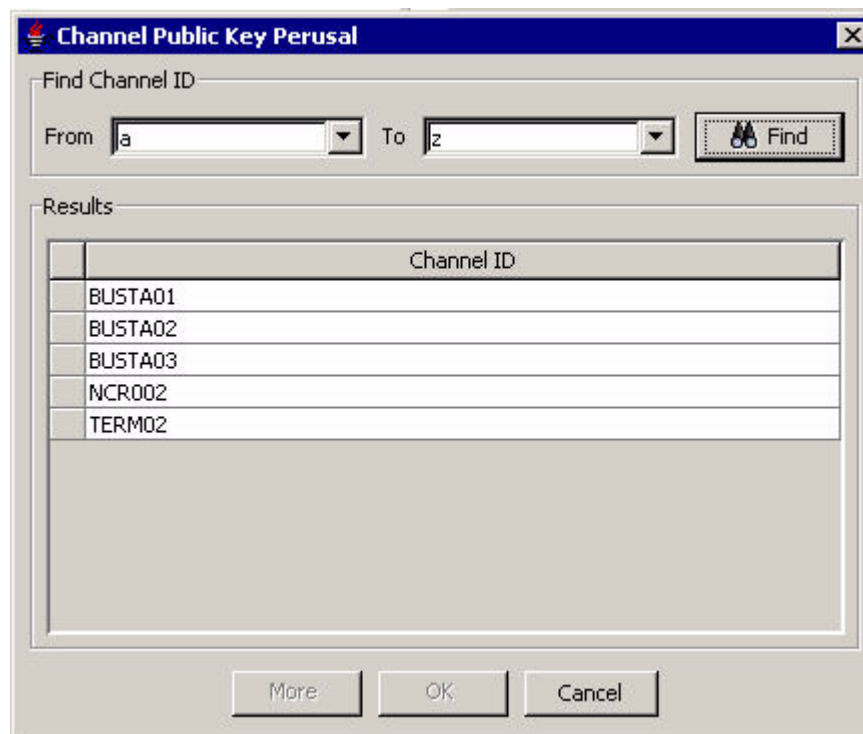
The following illustration shows you a sample of the Channel Public Key Perusal window for a Diebold or Triton device and Atalla NSPs.



The following illustration shows you a sample of the Channel Public Key Perusal window for a Diebold or Triton device and Thales e-Security or Banksys DEP HSMS.



When you click Find () on the Channel Public Key Perusal window, the following search window is displayed.



The image shows a window titled "Channel Public Key Perusal" with a search section and a results table.

Find Channel ID

From To

Results

Channel ID
BUSTA01
BUSTA02
BUSTA03
NCR002
TERM02

More OK Cancel

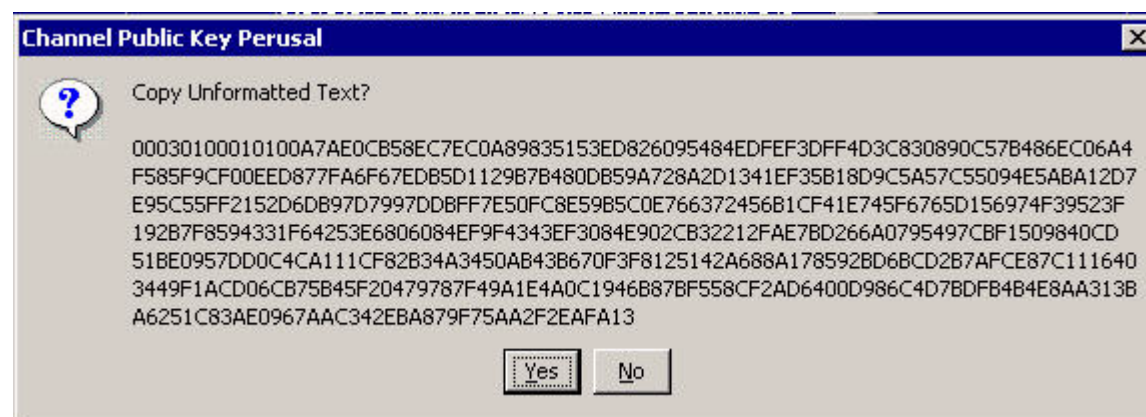
On this window, you can search for CHAN_PKEY_SEC records within a specified range of channel ID values. After you click the Search Now button, the channel IDs for CHAN_PKEY_SEC records within the specified range are displayed in the Results portion of the window. You can then select individual records to be displayed on the Channel Public Key Perusal window by highlighting a row in the Results area and clicking OK.

Copying key text

You can copy the key text as formatted or unformatted text from the Channel Public Key Perusal window. This is a useful feature if you need to paste key data into another application.

To copy the key text as formatted text, first click on the magnifying glass icon next to the Key Part field in the key table. Next, select all the text in the pop-up window and copy it to the clipboard by pressing the **Control-C** keys. You can then paste the text from the clipboard by pressing the **Control-V** keys.

To copy the key text as unformatted text, select a row for the key text you want to copy and click the Copy Unformatted Key button. The full unformatted value of the selected key element is displayed in a pop-up window from which you can copy the value to your clipboard by clicking Yes. You can then easily paste the value into another application. A sample of this pop-up window is shown below.



Key reports

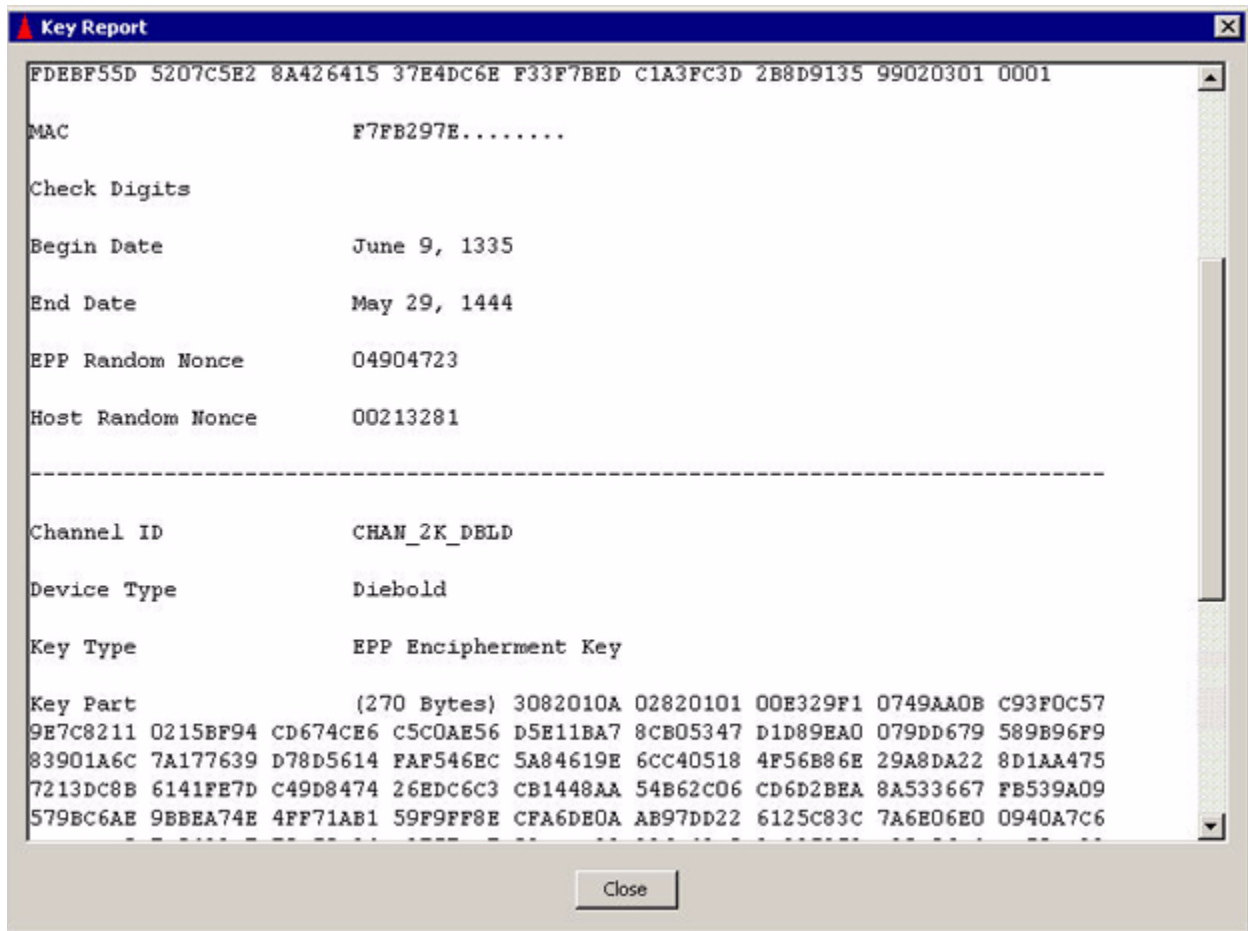
You can generate text reports for a selected key or all keys for a channel ID by clicking the View Selected Key Report or View All Keys Report button, respectively. When you either of these buttons, the key text is displayed in a separate Key Report window as shown below.

The following Key Report window displays all the fields and values for the currently selected key. Tabs are used to separate field headings from their values. You can select, copy, and paste the text into another application if desired for printing or storage.

Channel ID	CHAN_2K_NCR
Device Type	Wincor/NCR Signature
Key Type	EPP Encipherment Key
Key Part	(270 Bytes) 3082010A 02820101 00C65A10 B240B902 6BFDBB1A 3E368F50 10D5748A 8A88BC65 86876986 B5B2E27D 60398660 ACC85AA8 9D91592B 7275F71A 450B1914 9DE10A64 F2569F06 72B11729 FAA5EB97 1680E197 E1D31F48 86CFEEFC BBA7DE78 94A31A8A 74C76237 BBBE09D6 614220DB 4D7B4367 803CEECE 2518C9C2 4FD9B0BF C76AFE6E 99A0BA59 010B829D DE68965A 614685F6 06783CCF 6F227B38 DD1BDBDF E86B3E1B A1321982 75F188C2 A126E6B9 7D19B7FA 1AB4FB05 D9DB91D8 0CA917EF D08E9837 E0D955BF 30762785 77C12EF9 27B3D2C5 7B63FEE1 CA9AE035 B924489F A4B8B3AC 00DBE81D 08D165D2 5B2F0448 94089EE0 1F367099 3999FFFA 5538C1ED 3E419416 859D9362 04B4A726 25020301 0001
MAC	DE9334E5.....
Check Digits	
Begin Date	0
End Date	0
EPP Serial Number	12345678

Close

The next Key Report window displays all the fields and values for all the keys in the key table. Each key entry is separated by a horizontal dotted line.



EPP Serial Number Configuration window

The EPP Serial Number Configuration window (access **Configure > Transaction Security > Automated Key Distribution > EPP Serial Number**) is an optional window that enables you to view, add, and delete valid EPP serial numbers in the EPP_SERIAL_NUM. On this window, you can also search for existing records in the EPP_SERIAL_NUM within a specified serial number range. After a serial number record is added to the EPP_SERIAL_NUM from this window, you can clear out the nonkey field values for the record (i.e., everything but the serial number). New values are assigned to these fields when a Device Handler process sends the EPP serial number to the AKDS process in an online request. This enables you to reset information for an EPP, for example, when it needs to be repaired and temporarily taken out of the system.

The AKDS process optionally validates the EPP serial number returned from an ATM against valid numbers stored in the EPP_SERIAL_NUM when authenticating and exchanging public keys with an ATM. If the EPP serial number sent from the terminal is found in the EPP_SERIAL_NUM, it is considered to be authentic. Otherwise, it is not. The LCONF params ATM-DH-N50-AKDS-SERIAL-NUM-VRFY, ATM-DH-1000-AKDS-SERIAL-NUM-VRFY, ATM-DH-TIDL-AKDS-SERIAL-NUM-VRFY, and ATM-DH-TRIT-AKDS-SERIAL-NUM-VRFY determine whether this additional validation processing is performed for NCR 5XXX, Diebold 10XX/478X, Tidel, and Triton terminals, respectively.

When entering records on the EPP Serial Number Configuration window, the value you enter in the Institution ID field should match the FIID value configured in the BASE24-atm Terminal Data file (ATD) for the terminal.

For Diebold 10XX/478X terminals, you must enter two serial numbers for each channel (terminal) ID. One serial number entry must end with “*b*V” and the other must end with “*b*E”, where *b* is a blank space. The serial number ending with “*b*V” must match the serial number received in the EPP public signature verification key certificate sent from the terminal, while the serial number ending in “*b*E” must match the serial number received in the EPP public encipherment verification key certificate sent from the terminal.

You can enter any of the 95 printable ASCII characters in the Serial Number field, except for the quote (") character.

A sample of the EPP Serial Number Configuration window is shown below.

Serial Number	Channel ID	Validation Date	Validation Time

AKDS Capacity report

When using the BASE24 Automated Key Distribution System (AKDS) add-on product, you must generate the AKDS Capacity Report on a monthly basis and send it to ACI. The report indicates the total number of ATM terminals that have been authenticated and exchanged public keys with the BASE24 system. ACI uses this report to reconcile your AKDS usage (number of ATM terminals) against your AKDS capacity license. ACI recommends that you generate and send this report in conjunction with your monthly BASE24 Transaction Based Pricing reports.

The following paragraphs describe the AKDS Capacity Report, how to configure its output location and the number of lines printed on each page of the report, and how to generate the report using a process control command.

When run, the AKDS Capacity Report reads through the Channel Public Key Security File (CHAN_PKEY_SEC), determines the total number of ATM terminals defined in the CHAN_PKEY_SEC, and writes the total to specified report stream location (e.g., a file). You can optionally specify the report to list the channel ID of each ATM terminal defined in the CHAN_PKEY_SEC.

Refer to the **BASE24-eps Environment Management User Guide** for information on report assign configuration and report formatting.

The following are samples of the AKDS Capacity Report with and without the channel ID listing.

The value of the CUSTOMER_NAME Environment attribute is printed on the first line of the report header for each report page. You must set this attribute to override the default value of "CUSTOMER NAME". The DFLT_DAT_FRMT Environment attribute controls the format of the date printed on the "Date Run:" line of the report header.

The page number is printed at the bottom of each report page. If you specify to include a listing of channel IDs, they are printed in alphabetical order from left to right in four columns across the report. The total number of ATM terminals defined in the CHAN_PKEY_SEC is listed in the "Total Devices:" line at the end of the report.

The first sample, which consists of two report pages, shows you a report where the channel IDs have been listed.

The second sample, which consists of only one report page, shows you the report run against the same CHAN_PKEY_SEC without the optional channel IDs listed.

CUSTOMER NAME - AKDS Capacity Report - Device Detail
Date Run: DD MMM YYYY

Device Ids:

DIEBOLD001	NCR001	TERM01	TERM02
TERM03	TERM04	TERM05	TERM06
TERM07	TERM08	TERM09	TERM10
TERM100	TERM101	TERM102	TERM103
TERM104	TERM105	TERM106	TERM107
TERM108	TERM109	TERM11	TERM110
TERM111	TERM112	TERM113	TERM114
TERM115	TERM116	TERM117	TERM118
TERM119	TERM12	TERM120	TERM121
TERM122	TERM123	TERM124	TERM125
TERM126	TERM127	TERM128	TERM129
TERM13	TERM130	TERM131	TERM132
TERM133	TERM134	TERM135	TERM136
TERM137	TERM138	TERM139	TERM14
TERM140	TERM141	TERM142	TERM143
TERM144	TERM145	TERM146	TERM147
TERM148	TERM149	TERM15	TERM150
TERM151	TERM152	TERM153	TERM154
TERM155	TERM156	TERM157	TERM158
TERM159	TERM16	TERM160	TERM161
TERM162	TERM163	TERM164	TERM165
TERM166	TERM167	TERM168	TERM169
TERM17	TERM170	TERM171	TERM172
TERM173	TERM174	TERM175	TERM176
TERM177	TERM178	TERM179	TERM18
TERM180	TERM181	TERM182	TERM183
TERM184	TERM185	TERM186	TERM187
TERM188	TERM189	TERM19	TERM190
TERM20	TERM21	TERM22	TERM23
TERM24	TERM25	TERM26	TERM27
TERM28	TERM29	TERM30	TERM31
TERM32	TERM33	TERM34	TERM35

Page # 1

CUSTOMER NAME - AKDS Capacity Report - Device Detail
Date Run: DD MMM YYYY

TERM36	TERM37	TERM38	TERM39
TERM40	TERM41	TERM42	TERM43
TERM44	TERM45	TERM46	TERM47
TERM48	TERM49	TERM50	TERM51
TERM52	TERM53	TERM54	TERM55
TERM56	TERM57	TERM58	TERM59
TERM60	TERM61	TERM62	TERM63
TERM64	TERM65	TERM66	TERM67
TERM68	TERM69	TERM70	TERM71
TERM72	TERM73	TERM74	TERM75
TERM76	TERM77	TERM78	TERM79
TERM80	TERM81	TERM82	TERM83
TERM84	TERM85	TERM86	TERM87
TERM88	TERM89	TERM90	TERM91
TERM92	TERM93	TERM94	TERM95
TERM96	TERM97	TERM98	TERM99

Total Devices: 192

CUSTOMER NAME - AKDS Capacity Report - Device Summary
Date Run: DD MMM YYYY

Total Devices: 192

Page # 1

Configuring the report output location

You must configure the output location of the AKDS Capacity Report in the AKDS_CAPACITY_REPORT assign in the CONFCSV. The AKDS Capacity Report must be written to a valid Stream file. Stream files are write-only files to which single lines of variable length string data (up to 4062 bytes) are written as the data is generated. Stream files must be configured in the CONFCSV with the TableName attribute set to "Stream132". Samples of the AKDS_CAPACITY_REPORT assign in the CONFCSV are shown below. The first example directs the output to a spooler location. The second directs the output to a file.

```
"AKDS_CAPACITY_REPORT","Stream132","1.0","$S.#AKDSCAP","WOF","",""  
"AKDS_CAPACITY_REPORT","Stream132","1.0","$VOL.SUBVOL.  
RPT","WOF","",""
```

Configuring the report page length

You can configure the number of report lines printed on each page of the AKDS Capacity Report using the optional AKDS_CAP_RPT_PG_LGTH Environment attribute. If you do not configure this attribute on the Environment Configuration window, it defaults to 60 lines per page. The AKDS Capacity Report component inserts a page break whenever the number of lines specified by this attribute is reached in the report output.

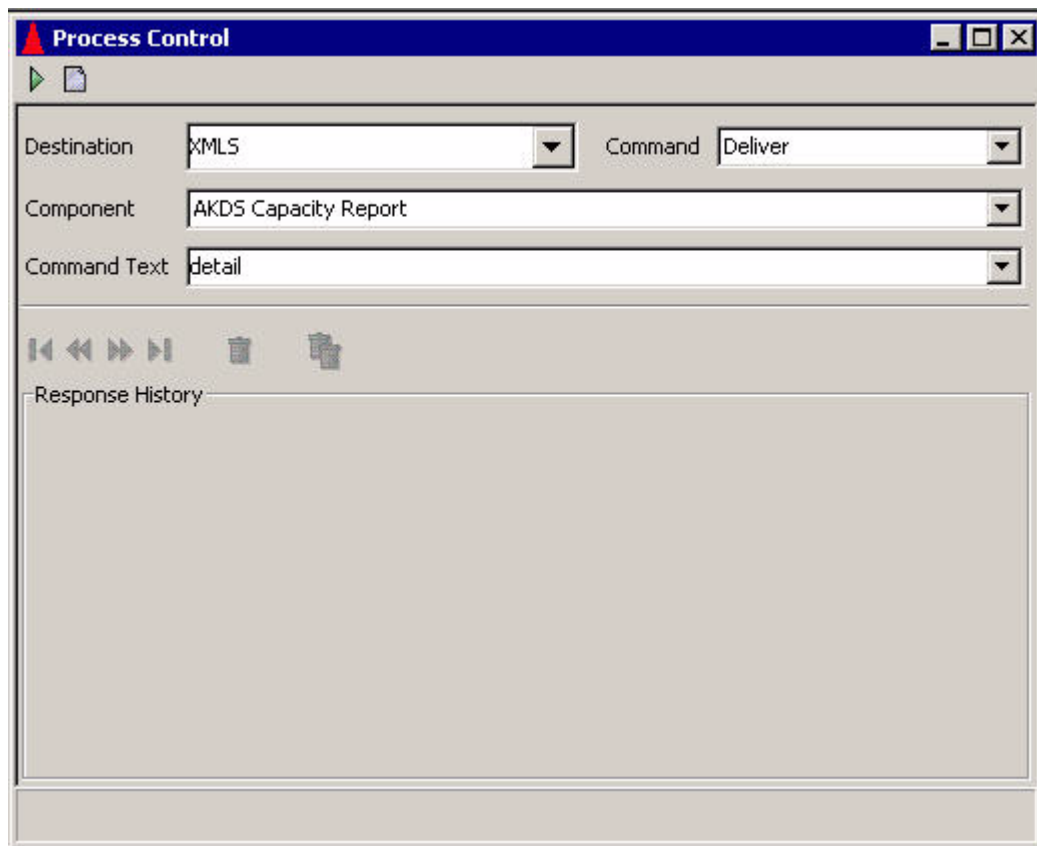
Generating the AKDS Capacity report

You generate the AKDS Capacity Report using a DELIVER process control command entered from the Process Control window on the ACI desktop user interface or from an XPNET network control facility.

Process Control window

When entering this command on the Process Control window, set the Destination field to “XMLS”, the Command field to “Deliver”, and the Component field to “AKDS Capacity Report”. You can optionally enter “detail” or “DETAIL” in the Command Text field to include the channel ID of each ATM terminal in the report output. If you leave the Command Text field

blank, only the total number of ATM terminals configured in the CHAN_PKEY_SEC is displayed on the report. An example of the Process Control window for generating the AKDS Capacity Report with a listing of channel IDs is shown below.



XPNET network control facility

To generate the AKDS Capacity Report from an XPNET network control facility, use the following text command.

Syntax

DELIVER PROCESS *object-name*, "DELIVER AKDSRPT [DETAIL]"

object-name — The symbolic name assigned to the User Interface (XML Server) process in the XPNET node.

Example

The following is an example of a DELIVER PROCESS command sent to the XML Server process to generate the AKDS Capacity Report with a list of the channel IDs for each ATM terminal using AKDS.

```
deliver process p1a^xml,"deliver akdsrpt detail"
```

16: Security device utilities and operations

This section describes the utilities and operations provided in the TSS environment for security device management.

The following utilities are described in this section:

- Check Digit Validation and Generation utility
- AKB Conversion utility
- Master File Key Conversion utility

The following operations procedures performed on hardware security devices are also provided in this section:

- Validating check digits for keys in the TSS database
- Generating check digits for keys in the TSS database
- Converting Atalla keys from non-AKB format to AKB format
- Changing the master file key in a security device
- Disabling a security device
- Enabling a security device
- Initializing a security device
- Obtaining the current status of a security device

Check digit validation and generation utility

ACI provides a check digit validation and generation program to validate and generate check digits for hardware-encrypted keys stored in the TSS database. The program enables you to validate or generate the check digits for keys in one or more files by entering a command on the Process Control window of the ACI desktop.

Note: Currently, this utility only supports check digit validation and generation of keys for Atalla NSPs in either variant or AKB format.

Prerequisites

The Check Digit Validation and Generation utility requires that the Generate Check Digits (7E) command be enabled in the Atalla NSP. This command is enabled in the default factory security policy. The ATALLA_CHK_DGTS_LGTH Environment attribute specifies whether the utility generates and validates four or six check digits. To use the leftmost six digits method in the 7E command, you must also enable option 88 in your Atalla NSP security policy.

For Atalla NSPs running AKB software, the 7E command accepts single-length keys only if the Atalla premium option 6C is enabled in your security policy using commands 108 and 109. This option does not apply and is not needed for Atalla NSPs running variant software.

Procedures

The following paragraphs provide procedures for validating and generating the check digits of keys in one or more files.

Validating check digits

You can validate the check digits of all keys in one or more files using the VVK VALIDATE command entered from the Process Control window (accessed from **System Operations > Process Control**) or from an XPNET network control facility. For each key's check digits validated, the utility requests the Atalla NSP to generate check digits for the key and compares the check digits returned with those stored in the TSS database. If the check digits match, the check digits in the database are valid. Otherwise, they are invalid. You can optionally specify the name of an individual Atalla NSP in the VVK VALIDATE command to perform all check digit validation processing. If you do not specify an individual Atalla NSP in the command, check digit validation requests are distributed in a round-robin manner among all Atalla NSPs in the system.

The syntax for the VVK VALIDATE command is as follows:

```
VVK VALIDATE [HSM security-device-name] filename, filename,...
```

where:

security-device-name is the name of an individual Atalla NSP configured on the Security Device Configuration window, ALL, or *. If you specify ALL or * for the HSM keyword in the command, check digit validation requests are distributed in a round-robin manner among all Atalla NSPs in the system. This has the same effect as excluding the optional HSM keyword syntax altogether from the command.

filename is one or more of the following acronyms for files:

*	= Validates check digits for keys in all files
CRDVD	= Validates check digits for keys in the Card Verification File
CSECD	= Validates check digits for keys in the Channel Security File

DCRDVD	= Validates check digits for keys in the Dynamic Card Verification File
DNTD	= Validates check digits for keys in the Diebold Number Table File
DRVKD	= Validates check digits for keys in the DUKPT Derivation Key File
DVPND ¹	= Validates check digits for keys in the Dual PIN Verification data source (DUAL_PIN_VRFCTN)
EMVSD	= Validates check digits for keys in the EMV Authorization File
IDESD	= Validates check digits for keys in the IBM DES Verification File
ISECD	= Validates check digits for keys in the Interchange Security File
VPVVD	= Validates check digits for keys in the Visa PVV Verification File
ZKA_KGK ¹	= Validates check digits for keys in the ZKA Key Generation Keys data source (ZKA_KGK)

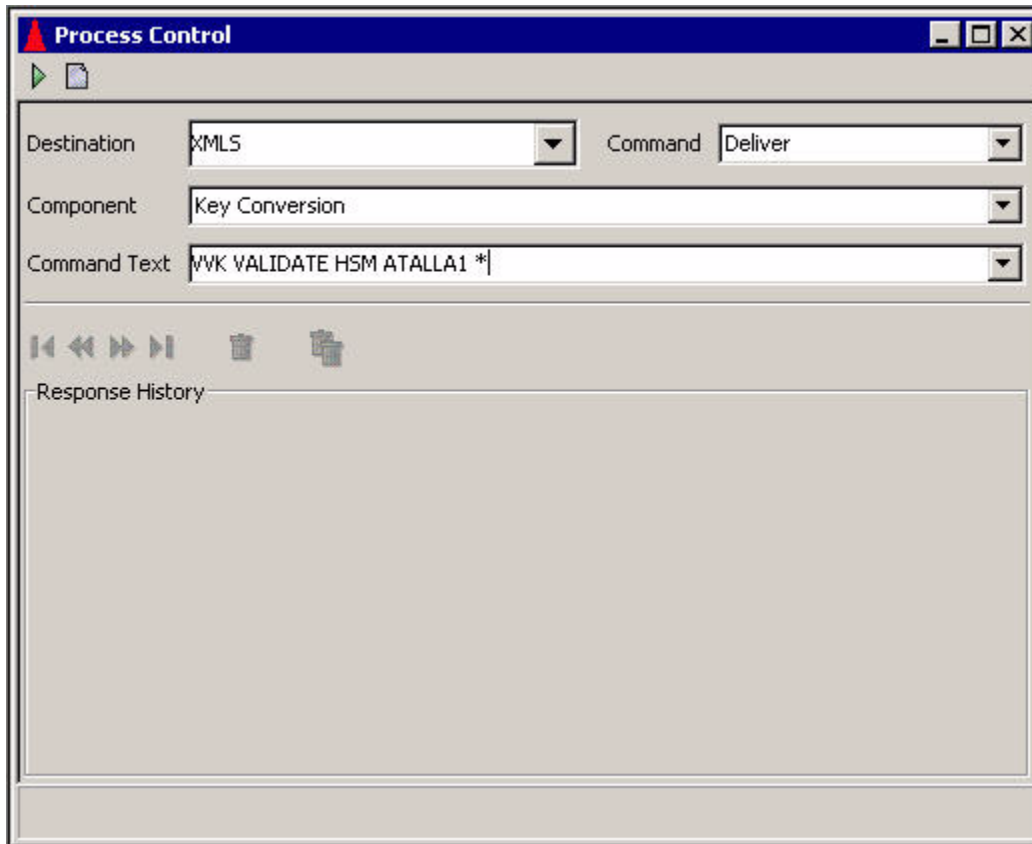
¹ These files are only supported on BASE24-eps.

If you specify more than one file name, each file name must be separated by a comma. If you specify * in the file name list, all other entries in the list are ignored.

When you enter this command from the Process Control window, you must set the Destination, Command, and Component fields as described below. You must enter the VVK VALIDATE command syntax described above in the Command Text field.

Destination:	XMLS or IS
Command:	Deliver
Component:	Key Conversion
Command text:	See VVK VALIDATE syntax above

A sample Process Control window for sending the VVK VALIDATE command is shown below.



A sample NCPCOM text string for sending the VVK VALIDATE command is shown below.

```
DELIVER PROCESS P1A^TSSXML001,"DELIVER KEYCONV VVK VALIDATE HSM
ATALLA2 *"
```

Generating check digits

You can generate the check digits of all keys in one or more files using the VVK GENERATE command entered from the Process Control window (accessed from **System Operations > Process Control**) or from an XPNET network control facility. For each key's check digits generated, the utility requests the Atalla NSP to generate check digits for the key and stores the check digits returned in the TSS database. If a check digit value already exists for a key in a file, it is overwritten by the check digits generated by the NSP. You can specify the name of an individual Atalla NSP in the VVK GENERATE command to perform all check digit generation processing. If you do not specify an individual Atalla NSP in the command, check digit generation requests are distributed in a round-robin manner among all Atalla NSPs in the system. The syntax for the VVK GENERATE command is as follows:

```
VVK GENERATE [HSM security-device-name] filename, filename,...
```


where:

security-device-name is the name of an individual Atalla NSP configured on the Security Device Configuration window, ALL, or *. If you specify ALL or * for the HSM keyword in the command, check digit generation requests are distributed in a round-robin manner among all Atalla NSPs in the system. This has the same effect as excluding the optional HSM keyword syntax altogether from the command.

filename is one or more of the following acronyms for files:

*	= Generates check digits for keys in all files
CRDVD	= Generates check digits for keys in the Card Verification File
CSECD	= Generates check digits for keys in the Channel Security File
DCRDVD	= Generates check digits for keys in the Dynamic Card Verification File
DNTD	= Generates check digits for keys in the Diebold Number Table File
DRVKD	= Generates check digits for keys in the DUKPT Derivation Key File
DVPND ¹	= Validates check digits for keys in the Dual PIN Verification data source (DUAL_PIN_VRFCTN)
EMVSD	= Generates check digits for keys in the EMV Authorization File
IDESD	= Generates check digits for keys in the IBM DES Verification File
ISECD	= Generates check digits for keys in the Interchange Security File
VPVVD	= Generates check digits for keys in the Visa PVV Verification File
ZKA_KGK ¹	= Validates check digits for keys in the ZKA Key Generation Keys data source (ZKA_KGK)

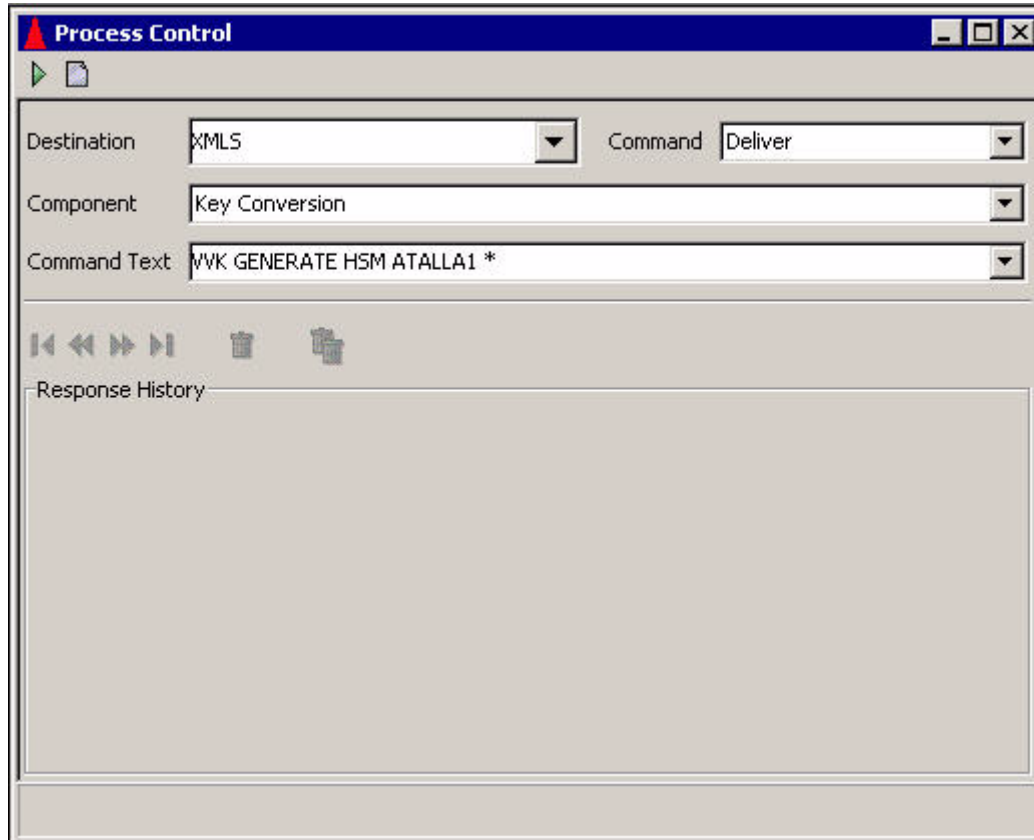
¹ These files are only supported on BASE24-eps.

If you specify more than one file name, each file name must be separated by a comma. If you specify *in the file name list, all other entries in the list are ignored.

When you enter this command from the Process Control window, you must set the Destination, Command, and Component fields as described below. You must enter the VVK VALIDATE command syntax described above in the Command Text field.

Destination:	XMLS or IS
Command:	Deliver
Component:	Key Conversion
Command text:	See VVK GENERATE syntax above

A sample Process Control window for sending the VVK GENERATE command is shown below.



A sample NCPCOM text string for sending the VVK GENERATE command is shown below.

```
DELIVER PROCESS P1A^TSSXML001,"DELIVER KEYCONV VVK GENERATE HSM  
ATALLA2 *"
```

Processing report

A processing report for the Check Digit Validation and Generation utility is printed to the location specified by the KEY_CONVERSION_REPORT assign in the CONFCSV (e.g., \$S.#KEYCONV).

Description

For each file in which check digits were validated or generated, the following is reported:

- The name of the data source (file) being processed, the date and time the utility started processing the file, and text indicating whether check digits are being validated or generated for the file.
- The total number of records read from the file.
- For each type of key stored in the file, the number of keys for which check digits were successfully validated or generated.
- For each type of key stored in the file, the number of keys for which check digit validation or generation failed.
- The total number of keys processed in the records read for which check digits were successfully validated or generated.
- The total number of keys processed in the records read for which the check digits were invalid or processing failed.

If the utility encounters an error while performing check digit validation or generation, an error message is written to the processing report. For each error message, the utility also writes the primary key fields of the record on which the error occurred. The following are examples of error messages that can be reported.

Error opening cursor. Unable to process data source.

<key name> CHECK DIGIT MISMATCH. Generated: <check digit value returned from 7E command> Current: <current check digit value in file>

Error Generating <key name> Check Digits: <error text>

where:

<key name> is the type of key on which the error occurred.

<error text> is one of the following:

No HSM available
Error formatting request to HSM
Unable to send message to HSM
HSM Response: <native HSM response message>

The types of keys that are reported on for each TSS file are shown in the following table.

TSS file	Keys
CARD_VRFCTN	Key Encrypt 1
	Key Encrypt 2

TSS file	Keys
Channel_Security	KKT PIN
	KKT MAC
	Inbound PIN
	Inbound MAC
	KKT DT
	Outbound KDT Lvl1
DIEBOLD_NUM_TBL	DNT Key
Derivation_Key	Derivation Key
DYN_CARD_VRFCTN	Key Encrypt 1
EMV_Security	Secure Message Key
	Secure Message Confidential Key
	Crypto Key
IBM_DES	Key Encrypt

TSS file	Keys
ICHG_SECURITY	KKT PIN MASTER
	KKT MAC MASTER
	KKT IMPORT PIN
	KKT EXPORT PIN
	KKT IMPORT MAC
	KKT EXPORT MAC
	Inbound PIN
	Outbound PIN
	Inbound MAC
	Outbound MAC
	KKT DT IMPORT
	KKT DT EXPORT
	Outbound KDT
	Inbound KDT
Visa_PVV	Key Encrypt

Sample

The following pages show a sample processing report for the Check Digit Validation and Generation utility when validating check digits.

Check Digit Report for Data Source CARD_VRFCTN
 Validating Check Digits
 Oct 09, 2003 08:30:19

Total Records: 3

Key	Success	Fail
-----	-----	-----
Key Encrypt 1	3	0
Key Encrypt 2	3	0
Total Keys Processed	6	0

Check Digit Report for Data Source IBM_DES
Validating Check Digits
Oct 09, 2003 08:30:19

Total Records: 6

Key -----	Success -----	Fail -----
Key Encrypt	6	0
Total Keys Processed	6	0

Check Digit Report for Data Source VISA_PVV
Validating Check Digits
Oct 09, 2003 08:30:19

Total Records: 3

Key -----	Success -----	Fail -----
Key Encrypt	36	0
Total Keys Processed	36	0

Check Digit Report for Data Source CHANNEL_SECURITY
Validating Check Digits
Oct 09, 2003 08:30:19

KKT PIN Key CHECK DIGIT MISMATCH. Generated: 55F7 Current: ' ' '
KKT MAC Key CHECK DIGIT MISMATCH. Generated: 3F8B Current: ' ' '
Inbound PIN Key CHECK DIGIT MISMATCH. Generated: CB02 Current: ' ' '
KKT DT Key CHECK DIGIT MISMATCH. Generated: 12C1 Current: ' ' '
Outbound KDT Lvl1 Data Key CHECK DIGIT MISMATCH. Generated: CA50 Current: ' ' '
Channel ID BEADDECCBEADDECC

Total Records: 8

Key -----	Success -----	Fail -----
KKT PIN	15	1
KKT MAC	15	1
Inbound PIN	15	1
Inbound MAC	16	0
KKT DT	15	1
OUTBOUND KDT	31	1

Total Keys Processed	107	5
----------------------	-----	---

Check Digit Report for Data Source ICHG_SECURITY
Validating Check Digits
Oct 09, 2003 08:30:19

Total Records: 5

Key	Success	Fail
-----	-----	-----
KKT IMPORT PIN	5	0
KKT EXPORT PIN	5	0
KKT IMPORT MAC	5	0
KKT EXPORT MAC	5	0
Inbound PIN	10	0
Outbound PIN	10	0
Inbound MAC	10	0
Outbound MAC	10	0
KKT DT IMPORT	5	0
KKT DT EXPORT	5	0
Outbound KDT	20	0
Inbound KDT	20	0
Total Keys Processed	110	0

Check Digit Report for Data Source EMV_SECURITY
Validating Check Digits
Oct 09, 2003 08:30:19

Total Records: 3

Key	Success	Fail
-----	-----	-----
Secure Message Key	3	0
Secure Message Confidential Key	3	0
Crypto Key	72	0
Total Keys Processed	78	0

AKB conversion utility

ACI provides a conversion program to convert working keys stored in the database in the variant key format (non-AKB format) to the Atalla Key Block (AKB) format. The program enables you to convert the keys in one or more files by entering a command on the Process Control window of the ACI desktop.

Prerequisites

An NSP configured with AKB version 2.30 or later or 1.3 or later software (referred to as AKB NSP throughout this document) is required for the AKB Conversion utility. The version 2.30 or later software is used for A10100, A10150, and A10160 AKB models while the 1.3 version software is used for A10000 AKB models.

11C command

AKB conversion requires that the Atalla DES Key to 3DES AKB Format (11C) command be enabled in the AKB NSP. This command is not enabled in the default factory security policy. Note that with AKB version 2.51 or later software, you must enable the 11C command with a specific count of the number of times the command can be executed. To use this command, you must add it to your security policy using commands 108 and 109.

Command options

The following options must be enabled for the AKB Conversion utility:

- Option 6C must be enabled for the 11C command if any of the keys (KPE, KEK, MAC, etc.) to be converted to AKB format are single length keys or if any keys to be converted are comprised of key pairs (CVV, PVV) and have the same value for key 1 and key 2. This option is not enabled in the default factory security policy. To use this option, you must add it to your security policy using commands 108 and 109.
- Option E3 must be enabled for commands 10 and 1A to support ATM master key headers. This option is not enabled in the default factory security policy. To use this option, you must add it to your security policy using commands 108 and 109.
- Premium option 8C must be enabled for the Verify PIN—Visa PVV Method (32#3) and PIN Change—Visa PVV (37#3) commands to allow single-length Visa PVV keys to be replicated in triple DES operations. Therefore, if you convert single-length Visa PVV keys to AKB format using the AKB Conversion utility, you must enable option 8C to use the 32#3 and 37#3 commands.

11B command

The AKB_KEY_IMPORT_VARIANT_SOURCE environment attribute allows PIN, MAC, and data key exchange keys in the Interface Security File (ICHG_SECURITY) to be converted to AKB format using the key variant or the key type. The value of this attribute indicates whether the Transaction Security Services process passes the key variant or key type in the variant field of the 11B command to the NSP. Valid values for this attribute are as follows:

- 0 = Pass the key type in the variant field of the 11B command. Default value.
- 1 = Pass the key variant in the variant field of the 11B command.

When this attribute is set to a value of 0, one of the following key type values is passed as the key variant in the 11B command:

- 1 = PIN key
- 2 = Data encryption key
- 3 = MAC key

When this attribute is set to a value of 1, the key variant value configured in the Key Variant field on the Interface Security Configuration window for the key is passed in the 11B command.

If your system uses key exchange keys with a key variant of 0, you must set this attribute to a value of 1 to correctly convert key exchange keys. Otherwise, you can set this attribute to 0 or allow it to default to 0 by not configuring it.

Check digits

Note: All keys to be converted in a file must have valid check digits or the conversion will fail. It is possible that the check digits fields in the TSS database may contain blanks or invalid values after BASE24 files are converted because BASE24 does not validate check digits when keys are entered into the BASE24 database. If so, use the [“Check digit validation and generation utility”](#) to generate and store check digits in the TSS database for all keys before running the AKB Conversion utility.

The AKB Conversion utility uses the 7E command to generate check digits using the leftmost four digits or the leftmost six digits method. The ATALLA_CHK_DGTS_LGTH Environment attribute specifies whether the utility generates and validates four or six check digits. To use the leftmost six digits method in the 7E command, you must also enable option 88 in your Atalla NSP security policy.

Quiescent network

The network must be shut down (i.e., transaction processing must be stopped) while the key translation function is performed, although you do not need to stop the Transaction Security Services and AKDS processes. Use of the utility requires careful planning and should only be performed by authorized personnel.

Procedures

The following procedures show you how to use the AKB Conversion utility when both a non-AKB NSP (i.e., an NSP configured with variant version software) and an AKB NSP are available, or when just an AKB NSP is available. The non-AKB NSP is referred to as the Variant NSP throughout this section.

Variant NSP and AKB NSP available

To convert working keys from non-AKB format to AKB format using both a Variant NSP and an AKB NSP, perform the following steps:

1. Back up the current database in case the keys need to be restored to non-AKB format.
2. Generate a triple-length or double-length SuperKey and MFK in AKB format and load them into the AKB NSP using the Secure Configuration Terminal (SCT) or Secure Configuration Assistant (SCA) under appropriate supervision (e.g., split knowledge and dual control). Refer to the appropriate Atalla documentation for detailed information on generating keys and use of the SCT and SCA.

Note: Although the AKB NSP supports both triple- and double-length MFKs, Atalla strongly encourages that you use a triple-length MFK. If you do use a double-length MFK, you cannot use any triple-length keys.

3. Load the first two key parts of the triple-length MFK or both parts of the double-length MFK generated in the above step as the Pending MFK (PMFK) on the Variant NSP using the SCT or SCA.
4. Using the Variant NSP, convert all current keys from encryption under the MFK to encryption under the PMFK using the Master File Key Conversion utility. (Note that this step assumes that the existing TSS database and system are still configured to use the Variant NSP). See the [“Master file key conversion”](#) topic for detailed procedures.
5. Set any optional Environment attributes on the Environment Configuration window for AKB header values that you want to be different from the default value. See the Environment Attributes topic below for a list of these optional Environment attributes and their associated default AKB header values.
6. Modify the Security Device Configuration File to set all Subtype fields to an AKB subtype.
7. Warmboot the Transaction Security Services process to start using the modified Security Device Configuration File (e.g., ALTER DASRC SEC_DEV_CONFIG).
8. Modify all HSM assigns in the MDSDEST file to specify AKB NSPs (or their associated Boxcar processes).

Note: All implementations using the TCP/IP protocol require the Boxcar process.

9. If the SIMDS_DEST_WARMBOOT_DELTA parameter in the CONFCSV is set to a value of 0, then you must stop and restart the User Interface (XML Server) process before the modified MDSDEST file takes effect.
10. Convert all keys from variant format to AKB format using the CONVERT AKB command from the Process Control window or from an XPNET network control facility. You can specify the name of an individual AKB NSP in the CONVERT AKB command to perform all

conversion processing. If you do not specify an individual AKB NSP in the command, conversion function requests are distributed in a round-robin manner among all AKB NSPs in the system. The syntax for the CONVERT AKB command is as follows:

CONVERT AKB [HSM *security-device-name*] *filename, filename,...*

where *security-device-name* is the name of an individual AKB NSP configured on the Security Device Configuration window and *filename* is one or more of the following acronyms for files:

*	= Convert all files
CRDVD	= Card Verification File
CSECD	= Channel Security File
DCRDVD	= Dynamic Card Verification File
DNTD	= Diebold Number Table File
DRVKD	= DUKPT Derivation Key File
ISECD	= Interchange Security File
EMVSD	= EMV Authorization File
IDESD	= IBM DES Verification File
VPVVD	= Visa PVV Verification File

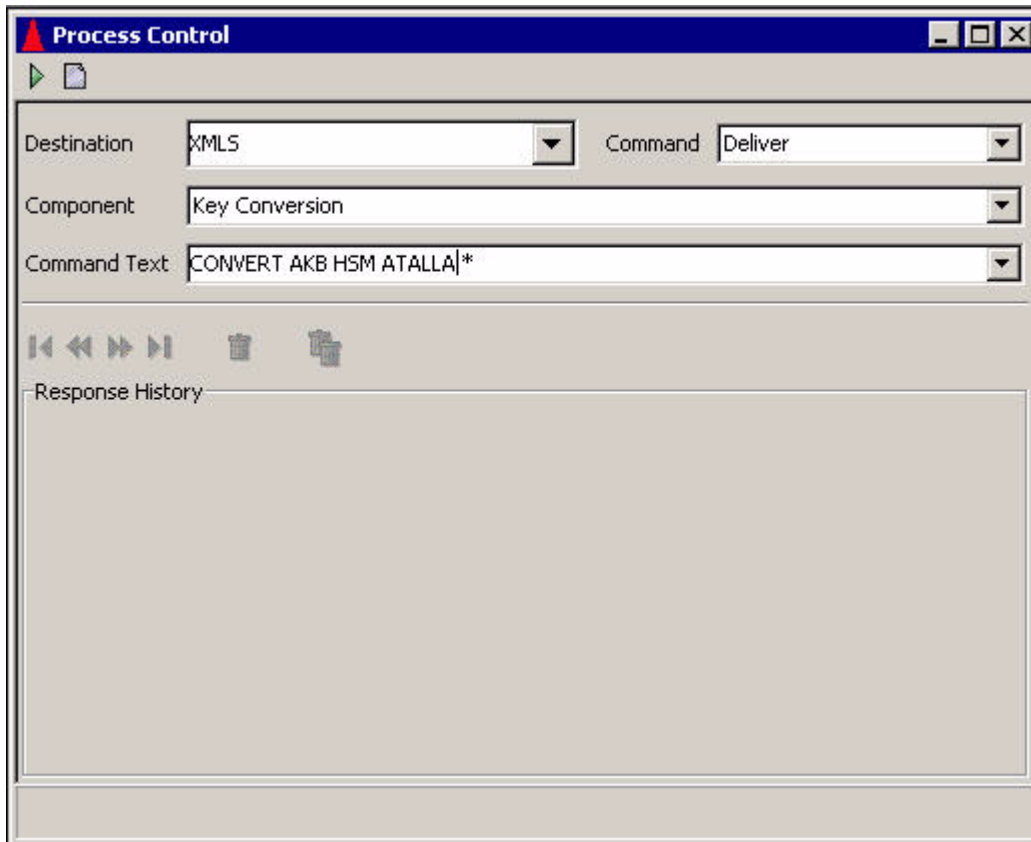
If you specify more than one file name, each file name must be separated by a comma.

If you specify * in the file name list, all other entries in the list are ignored.

Note: Although DNTD is a valid file name in the CONVERT AKB command syntax, the utility does not currently convert keys in the DIEBOLD_NUM_TBL.

Event messages are generated when the key conversion processing is started and completed.

A sample Process Control window for sending the CONVERT AKB command is shown below.



A sample NCPCOM text string for sending the CONVERT AKB command is shown below.

```
DELIVER PROCESS P1A^TSSXML001,"DELIVER KEYCONV CONVERT AKB HSM ATALLA2 *"
```

11. Rebuild and warmboot all external memory tables (i.e., OLTPs).
12. If you did not stop the Transaction Security Services and AKDS processes while the keys were converted, warmboot the Channel Security File (CHANNEL_SECURITY) and Interface Security File (ICHG_SECURITY). If you stopped these processes before the conversion, restart them.
13. Restart transaction processing.

AKB NSP only

To convert working keys from non-AKB format to AKB format using only an AKB NSP, perform the following steps:

1. Back up the current database in case the keys need to be restored to non-AKB format.

2. Generate a triple-length or double-length SuperKey and MFK in AKB format and load them into the AKB NSP using the Secure Configuration Terminal (SCT) or Secure Configuration Assistant (SCA) under appropriate supervision (e.g., split knowledge and dual control). Refer to the appropriate Atalla documentation for detailed information on generating keys and use of the SCT and SCA.

Note: Although the AKB NSP supports both triple- and double-length MFKs, Atalla strongly encourages that you use a triple-length MFK. If you do use a double-length MFK, you cannot use any triple-length keys.

3. Using the SCT or SCA, create the key portion of the variant MFK under which keys in the TSS database are currently encrypted and enter it as an AKB-formatted key using three Environment attributes.

- a. Generate the key portion under appropriate supervision (e.g., split knowledge and dual control). For the SCT, you must use the Keyblock Utility provided by Atalla to generate this key portion. You must modify the Keyblock Utility configuration files in order to use a value of 1KDDN0M0 as the AKB header for this key portion. The following modifications are required:

- 1) Add an entry to the headers.fil file using a text editor such as Notepad. The entry to be added is underlined below.

```
#23456789.123456789.123456789.123456789.123456789.123456789
#Name      Descriptions      Header
MFK        Master File Key    1KDDN0M0
KEK        Key Encryption Key  1KDNE000
```

- 2) Add an entry to the options.fil file using a text editor such as Notepad. The entry to be added is underlined below.

```
** BYTE 6-Special Handling
0*Regular key header
1 Import-KEK header.
N KEK that can import or export other keys.
M MFK
```

- b. Optionally enter the AKB_CNV_DES_3DES_HDR attribute on the Environment Configuration window of the ACI desktop user interface. If you do not enter this attribute, it defaults to a value of 1KDDN0M0.
- c. Enter the resulting 48-byte key block (from step 3a) into the AKB_CNV_DES_3DES_KEY attribute on the Environment Configuration window of the ACI desktop user interface.
- d. Enter the 16-character MAC generated for the key portion (from step 3a) into the AKB_CNV_DES_3DES_MAC attribute on the Environment Configuration window.

If the above Environment attributes are configured when you enter the CONVERT AKB command, the Key Conversion component includes the MFK cryptogram defined by these attributes in the 11C command sent to the AKB NSP, along with the cryptogram of the key to be translated. The AKB NSP then decrypts the key with this MFK before translating the key to AKB format. Otherwise, only the cryptogram of the key to be translated is included in the 11C command.

4. Set any optional Environment attributes on the Environment Configuration window for AKB header values that you want to be different from the default value. See the Environment Attributes topic below for a list of these optional Environment attributes and their associated default AKB header values.

5. Modify the Security Device Configuration File to set all Subtype fields to an AKB subtype.
6. Warmboot the Transaction Security Services process to start using the modified Security Device Configuration File (e.g., ALTER DASRC SEC_DEV_CONFIG).
7. Modify all HSM assigns in the MDSDEST file to specify AKB NSPs (or their associated Boxcar processes).

Note: All implementations using the TCP/IP protocol require the Boxcar process.

8. If the SIMDS_DEST_WARMBOOT_DELTA parameter in the CONFCSV is set to a value of 0, then you must stop and restart the User Interface (XML Server) process before the modified MDSDEST file takes effect.
9. Convert all keys from variant format to AKB format using the CONVERT AKB command from the Process Control window or from an XPNET network control facility. You can specify the name of an individual AKB NSP in the CONVERT AKB command to perform all conversion processing. If you do not specify an individual AKB NSP in the command, conversion function requests are distributed in a round-robin manner among all AKB NSPs in the system. The syntax for the CONVERT AKB command is as follows:

CONVERT AKB [HSM *security-device-name*] *filename, filename,...*

where *security-device-name* is the name of an individual AKB NSP configured on the Security Device Configuration window and *filename* is one or more of the following acronyms for files:

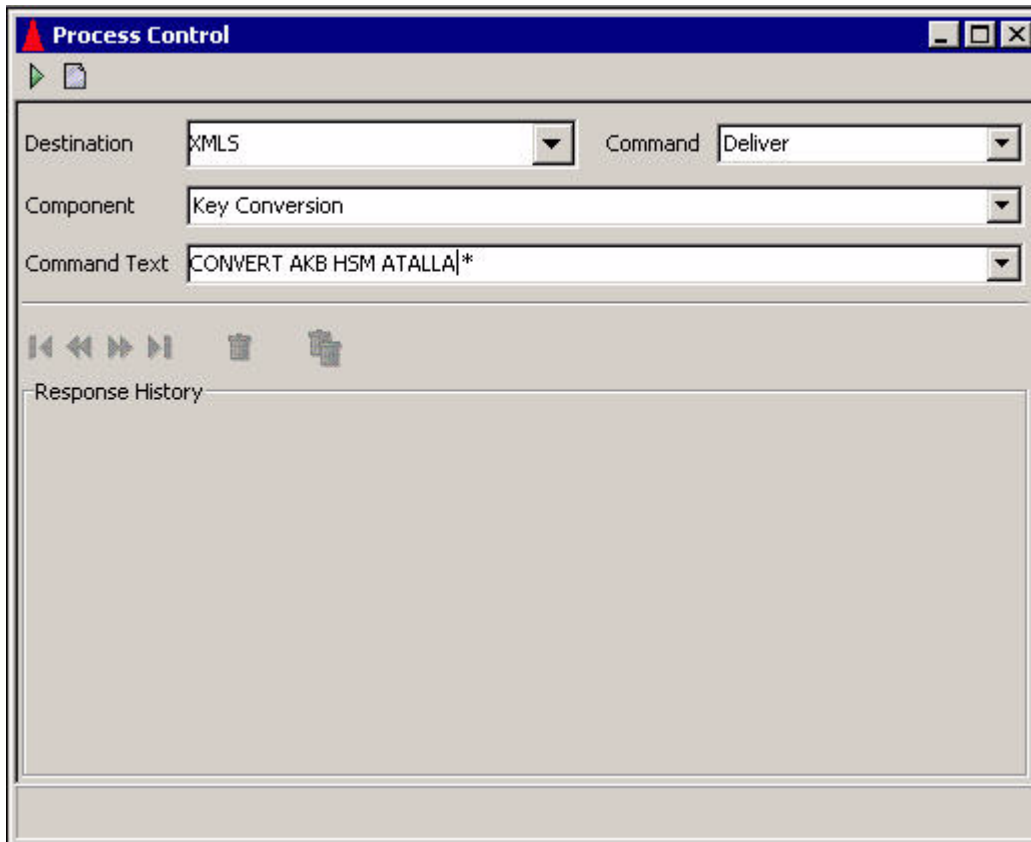
*	= Convert all files
CRDVD	= Card Verification File
CSECD	= Channel Security File
DCRDVD	= Dynamic Card Verification File
DNTD	= Diebold Number Table File
DRVKD	= DUKPT Derivation Key File
ISECD	= Interchange Security File
EMVSD	= EMV Authorization File
IDESD	= IBM DES Verification File
VPVVD	= Visa PVV Verification File

If you specify more than one file name, each file name must be separated by a comma. If you specify * in the file name list, all other entries in the list are ignored.

Note: Although DNTD is a valid file name in the CONVERT AKB command syntax, the utility does not currently convert keys in the DIEBOLD_NUM_TBL.

Event messages are generated when the key conversion processing is started and completed.

A sample Process Control window for sending the CONVERT AKB command is shown below.



A sample NCPCOM text string for sending the CONVERT AKB command is shown below.

```
DELIVER PROCESS P1A^TSSXML001,"DELIVER KEYCONV CONVERT AKB HSM ATALLA2 *"
```

10. Rebuild and warmboot all external memory tables (i.e., OLTPs).

11. If you did not stop the Transaction Security Services and AKDS processes while the keys were converted, warmboot the Channel Security File (CHANNEL_SECURITY) and Interface Security File (ICHG_SECURITY). If you stopped these processes before the conversion, restart them.

12. Restart transaction processing.

Processing report

A processing report for the AKB Conversion utility is printed to the location specified by the KEY_CONVERSION_REPORT assign in the CONFCSV (e.g., \$S.#KEYCONV).

For each data file converted, the following counts are reported after a data file has been completely processed:

- The number of records read
- The number of records successfully converted
- The number of records not converted

For each record not converted, the primary key value for the record is listed.

Note: Although DNTD is a valid file name in the CONVERT AKB command syntax, the utility does not currently convert keys in the DIEBOLD_NUM_TBL. If you include the DNTD file name in the command syntax, the text “Command not supported for DNTD” is written to the processing report.

Optional AKB header environment attributes

The following table lists the optional Environment attributes you can configure if you want to use an AKB header value that differs from the default value when translating keys. The first column of the table lists the name of the Environment attribute. The second column indicates the default AKB header value of the attribute. The third column indicates the file in which the key is stored, and the fourth column indicates the type of key.

Attribute name	Default value	TSS file	Key type
AKB_HDR_ATM_MSTR	1ADNE000	CSECD	PIN and MAC key exchange key when the Key Variant field is set to 5 (ATM Key Variant)
AKB_HDR_CARD_VRFY	1CDNE000	CRDVD	Card verification
AKB_HDR_AMEX_CARD_VRFY	1mXNE000	CRDVD	American Express Card Security Code
AKB_HDR_DNT	1ndNE000	DNTD	Diebold number table
AKB_HDR_DERIVATION	1dDNE000	DRVKD	DUKPT derivation key
AKB_HDR_DYN_CARD_VRFY	1mENN000	DCRDVD	Issuer master key (dynamic card verification)
AKB_HDR_EMV_CRYPT0	1mENE000	EMVSD	EMV ACC master keys (cryptographic keys)
AKB_HDR_EMV_SECURE	1mENE00M	EMVSD	EMV MAC master key (secure message key for integrity)

Attribute name	Default value	TSS file	Key type
AKB_HDR_EMV_SECURE_CONF	1mENE00E	EMVSD	PIN encryption key (secure message key for confidentiality)
AKB_HDR_IBM_DES	1V3NE000	IDESD	IBM DES PIN verification key
AKB_HDR_INBND_MAC	1MDNE000	CSECD ISECD	Inbound MAC encryption key
AKB_HDR_INBND_DT	1DDNE000	ISECD	Inbound data encryption key
AKB_HDR_INBND_IV	1IDNE000	ISECD	Inbound initialization vector
AKB_HDR_INBND_PIN	1PUNE000	CSECD ISECD	Inbound PIN encryption key
AKB_HDR_OUTBND_DT	1DDNE000	CSECD ISECD	Outbound data encryption key
AKB_HDR_OUTBND_IV	1IDNE000	CSECD ISECD	Outbound initialization vector
AKB_HDR_KKT_DT_EXPORT	1KDNE000	ISECD	Export data encryption key exchange key
AKB_HDR_KKT_DT_IMPORT	1DDNN010	CSECD ISECD	Import data encryption key exchange key
AKB_HDR_KKT_PIN	1KDNE000	CSECD	PIN key exchange key when the Key Variant field is set to 0 (Key Exchange Key Variant)
AKB_HDR_KKT_PIN_IMPORT	1PUNN010	ISECD	Import PIN key exchange key
AKB_HDR_KKT_MAC	1KDNE000	CSECD	MAC key exchange key when the Key Variant field is set to 0 (Key Exchange Key Variant)
AKB_HDR_KKT_MAC_IMPORT	1MDNN010	ISECD	Import MAC key exchange key
AKB_HDR_KKT_PIN_EXPORT	1KDNE000	ISECD	Export PIN key exchange key

Attribute name	Default value	TSS file	Key type
AKB_HDR_KKT_MAC_EXPORT	1KDNE000	ISECD	Export MAC key exchange key
AKB_HDR_OUTBND_PIN	1PUNE000	CSECD ISECD	Outbound PIN encryption key
AKB_HDR_OUTBND_MAC	1MDNE000	CSECD ISECD	Outbound MAC encryption key
AKB_HDR_VISA_PVV	1VVNE000	VPVVD	Visa PVV PIN verification key

Master file key conversion

Good security practice as well as several network and card associations require the master file key in a hardware security module to be replaced periodically. Master file key usage varies according to the hardware security module vendor as described below:

- In an Atalla NSP, the master file key is referred to as the Master File Key (MFK). Different types of working keys stored in the database are encrypted and stored under different variants of the MFK when using an NSP.
- In a Thales e-Security HSM, the master file key is referred to as a Local Master Key (LMK), which actually consists of a pair of keys. When using a Thales e-Security HSM, different types of working keys stored in the database are encrypted and stored under different LMK key pairs.

When you change the master file key in a hardware security file, you also have to translate each working key from encryption under the current master file key to encryption under the new master file key. Because of this, manual translation of each working key in the database from one master key to another can be a time-consuming and error-prone process. To alleviate this manual work, ACI provides the Master File Key Conversion utility to automate these conversion tasks. Using the Master File Key Conversion utility, you can translate all the hardware-encrypted keys stored in the database or just the hardware-encrypted keys of a particular type (e.g., PIN verification keys, card verification keys, interface keys, etc.).

Note: When you convert a working key from encryption under the current master file key to encryption under the new master file key, the value of the working key itself is not changed. Therefore, there is no need to download working keys to devices after running this utility on the Channel Security File (Channel_Security).

The material in this document is not meant to be a tutorial on Atalla NSPs or Thales e-Security HSMs; it presumes a familiarity with the device capabilities and nomenclature as discussed in documentation published by Atalla or Thales.

Note: The key translation options of the Master File Key Conversion utility result in the modification of security-related information in the database. The use of this utility by anyone other than qualified, trusted network security staff is discouraged, and should not be attempted without a full understanding of the processing involved.

The keys listed in section 4 of this guide for each of the supported vendors (i.e., [Atalla](#) or [Thales](#)) are converted using this utility.

The public and private keys described in section 15 of this guide can also be converted using this utility when a public key cryptography function is installed. Note that for public keys, the conversion of public keys is actually only a re-calculation of the MAC value.

Prerequisites

Before you can use the Master File Key Conversion utility, the following prerequisites must be met:

- The network must be shut down (i.e., transaction processing must be stopped) while the key translation function is performed, although you do not need to stop the Transaction Security Services and AKDS processes. Use of the utility requires careful planning and should only be performed by authorized personnel.
- Atalla NSPs must contain the old MFK as the current MFK and contain the new MFK as the pending MFK before this utility is used. The utility requires the 9E command (Translate Working Key From Under the Current MFK to the Pending MFK) to translate keys.
- For Atalla NSPs, the Master File Key Conversion utility uses the 7E command to generate check digits using the leftmost four digits or the leftmost six digits method. The ATALLA_CHK_DGTS_LGTH Environment attribute specifies whether the utility generates and validates four or six check digits. To use the leftmost six digits method in the 7E command, you must also enable option 88 in your Atalla NSP security policy.
- For Atalla NSPs, you must enable the 12B (Translate Public or Private Key AKB from MFK to PMFK) premium command in your Atalla NSP security policy to support MFK conversion for keys stored in the Host Public Key Security File (HOST_PUBLIC_KEY) and Channel Public Key Security File (CHAN_PKEY_SEC). The HOST_PUBLIC_KEY and CHAN_PKEY_SEC are used for public key cryptography.
- For Thales e-Security HSMs, the old LMK pair must be moved into key change storage before this utility is used. The utility requires the BW command (Translate Keys From Old LMK to New LMK) to translate keys.
- For Thales e-Security HSMs, you must set the DECIMAL_TBL_ENCRYPT optional environment attribute to a value of Y if you are using encrypted decimalization tables in your system. This attribute allows the Master File Key Conversion utility to properly translate encrypted decimalization tables from the old LMK pair to the new LMK pair. If you are not using encrypted decimalization tables in your system, you do not need to configure this attribute or you can configure it with a value of N.

Accessing the utility

The Master File Key Conversion window on the ACI desktop enables you to run the conversion utility. You can access this window by selecting **Configure > Transaction Security > Master File Key Conversion**. When the window is displayed, enter the name of the security device to be used or ALL from the selectable list in the Security Device Name field. Select ALL if you want to translate keys for all security devices. The security device type is displayed in the Security Device Type field and “to be determined” is displayed in the Report Location field. The report location is controlled by the KEY_CONVERSION_REPORT assign value (e.g., \$S.#KEYCONV) in the Metadata Configuration File (CONFCSV). This is the location where the utility generates reports that list the keys converted in each file. The options displayed in the Select Key Type(s) To Translate area of the window vary according to the type of security device used. Refer to ACI Online Help for detailed descriptions of the Master File Key Conversion window fields and functions.

Event messages are generated when the key conversion processing is started and completed.

Atalla options

For an Atalla NSP, you can translate all keys or any of the following types of keys in the database by selecting to translate all keys or selected keys, clicking on the types of keys you want to translate, and then clicking Submit (▶). The assign names of the files in which each type of key is stored are provided in parentheses:

- Card verification keys (CARD_VRFCTN)
- PIN verification keys (IBM_DES, Visa_PVV)
- EMV keys (EMV_Security)
- Channel keys (Channel_Security)
- Interface keys (ICHG_SECURITY)
- DUKPT derivation keys (Derivation_Key)
- Host (BASE24 system) public and secret keys (HOST_PUBLIC_KEY)
- Channel (ATM terminal) public keys (CHAN_PKEY_SEC)
- Dynamic card verification keys (DYN_CARD_VRFCTN)
- ZKA Key Generation Keys (ZKA_KGK)¹

¹ These keys are only supported on BASE24-eps. They are not available for the Transaction Security Services environment.

A sample of the Master File Key Conversion window for an Atalla NSP is shown below.

Master File Key Conversion - ATALLA

Security Device Name: **ATALLA**

Security Device Type: **Atalla**

Report Location: **to be determined**

Select Translate Type: ☒ All Keys ☐ Selected Keys

Select Key Type(s) To Translate

- ☐ Card Verification Keys
- ☐ PIN Verification Keys
- ☐ EMV Keys
- ☐ Channel Keys
- ☐ Interface Keys
- ☐ DUKPT Derivation Keys
- ☐ Host Public/Secret Keys
- ☐ Channel Public Keys
- ☐ Dynamic Card Verification Keys
- ☐ ZKA Key Generation Keys

2 - MFK UI Security Device Detail - Retrieved

Thales Options

For a Thales e-Security HSM, keys must be translated according to the LMK pair under which they are encrypted. You can translate all keys or select any of the following types of keys in the database to be translated by clicking on each type of key to be translated and clicking Submit (►):

- Zone master keys (LMK pair 04–05)
- Zone working keys (LMK pair 06–07)
- Card verification, PIN verification, terminal master, terminal PIN keys, and transaction key registers¹ (LMK pair 14–15)
- ¹ Transaction key registers are currently only supported for APACS 30/40 devices.
- Terminal MAC keys (LMK pair 16–17)
- IBM DES PIN verification decimalization table (LMK pair 18–19)

- NCR PIN Verification Keys (LMK pair 20–21)
- Zone Authentication Keys (LMK pair 26–27)¹
- EMV, DUKPT derivation, and dynamic card verification keys (LMK pair 28–29)
- Zone Encryption Keys (LMK pair 30–31)¹
- Terminal Encryption Keys (LMK pair 32–33)¹
- Host (BASE24 system) public and secret keys (LMK pairs 34–35 and 36–37)
Private keys are encrypted under LMK pair 34–35 while the MAC values for public keys are encrypted under LMK pair 36–37.
- Channel public keys (LMK pair 36–37)

¹ These keys are only supported on BASE24-eps. They are not available for the Transaction Security Services environment.

The HSM can translate only one LMK pair at a time, regardless of the number of LMK pairs that need to be translated. As a result, it is important to know which files must be processed when a particular LMK pair changes. LMK pairs affect the following files.

LMK pair	CSECD	IDESD	VPVVD	NCRD	CRDVD	EMVSD	ISECD	DRVKD	HSPKD / HSPKSD	CHNPKD	DCRDVD
04-05							✓				
06-07							✓				
14-15	✓	✓	✓		✓						
16-17	✓						✓				
18-19		✓									
20-21				✓							
26-27								✓			
28-29						✓		✓			✓
30-31	✓						✓				
32-33	✓										
34-35									✓		
36-37									✓	✓	

A sample of the Master File Key Conversion window for a Thales HSM is shown below.

Master File Key Conversion - THALES1

Security Device Name: **THALES1**

Security Device Type: **Thales**

Report Location: **to be determined**

Select Translate Type: ☒ All Keys ☐ Selected Keys

Select Key Type(s) To Translate

- ☐ Zone Master Keys (LMK pair 04/05)
- ☐ Zone Working Keys (LMK pair 06/07)
- ☐ Card Verification, PIN Verification, Terminal Master Keys, Terminal PIN Keys and Transaction Key Registers (LMK pair 14/15)
- ☐ Terminal MAC Keys (LMK pair 16/17)
- ☐ NCR PIN Verification Keys (LMK pair 20/21)
- ☐ EMV, DUKPT Derivation, and Dynamic Card Verification Keys (LMK pair 28/29)
- ☐ Decimalization Table (LMK pair 18/19)
- ☐ Host Public/Seret Keys (LMK pairs 34/35 and 36/37)
- ☐ Channel Public Keys
- ☐ Terminal Encryption Keys (LMK pair 32/33)
- ☐ Zone Authentication Keys (LMK pair 26/27)
- ☐ Zone Encryption Keys (LMK pair 30/31)

2 - MFK UI Security Device Detail - Retrieved

Key Translation report

The Master File Key Conversion utility generates a Key Translation Report for each file in which keys were converted. The report lists the total number of keys successfully converted for each type of key in the file, as well as the total number of keys on which errors were encountered. The report displays error messages for any errors encountered during processing. Each error message indicates the type of error that occurred, the response returned from the security device, and the primary key values of the record on which the error occurred.

The format of the Key Translation Report varies slightly between an Atalla device and a Thales e-Security device. Examples of both formats are shown below, followed by descriptions of the fields on the report.

Atalla format

The format of the Key Translation Report for an Atalla NSP is shown below.

```
Key Translation Report for Data Source <data source name>
Translating keys from MFK to Pending MFK
Month DD, YYYY HH:MM:SS
```

(any processing errors appear here)

Total Records:

Key	Success	Fail
-----	-----	-----
<key name 1>		
<key name n>		

Total Keys Processed

Thales e-Security format

The format of the Key Translation Report for a Thales e-Security HSM is shown below.

```
Key Translation Report for Data Source <data source name>
LMK Key Pair <xx/xx>
Translating keys from Old LMK to Current LMK
Month DD, YYYY HH:MM:SS
```

(any processing errors appear here)

Total Records:

Key	Success	Fail
-----	-----	-----
<key name 1>		
<key name n>		

Total Keys Processed

Field Descriptions

<data source name> — The name of the file in which keys were translated.

<xx/xx> — The name of the LMK pair for which keys were translated. This applies to Thales e-Security HSMs only.

Total Records — The total number of records processed for the indicated file and LMK pair.

Key — A descriptive name of the type of key translated for the file.

Success — The total number of keys successfully translated of the indicated key type.

Fail — The total number of keys of the indicated key type for which translation failed.

Total Keys Processed — The total number of key records processed for the file.

Post-Translation procedures

After the selected keys have been successfully translated from encryption under the current master file key to encryption under the pending master file key, perform the following steps:

1. Rebuild and warmboot all external memory tables (i.e., OLTPs).
2. If you did not stop the Transaction Security Services and AKDS processes while the keys were converted, warmboot the Channel Security File (CHANNEL_SECURITY) and Interface Security File (ICHG_SECURITY). If you stopped these processes before the conversion, restart them.
3. Enter the ERASE command on the Process Control window or from an XPNET network control facility to erase the old LMK key pair in a Thales e-Security HSM or make the pending MFK the current MFK in an Atalla NSP. The ERASE command sends the 9A (Network Security Processor Status Key) and 9F (Replace the Current MFK with the Pending MFK) commands to an Atalla NSP and the BS (Erase the Key Change Storage) command to a Thales e-Security HSM.
4. Restart transaction processing.

Process Control window

To enter the ERASE command on the Process Control window, perform the following steps:

1. Log on to the ACI desktop user interface and access System Operations > Process Control. The Process Control window is displayed.
2. In the Destination field, select XMLS from the list of values.
3. In the Command field, select Deliver from the list of values.
4. In the Component field, select Key Conversion from the list of values.
5. In the Command Text field, enter the following command. If you want to execute the ERASE command on a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to execute the ERASE command on all security devices in the BASE24 system, enter an asterisk (*) in the command text.

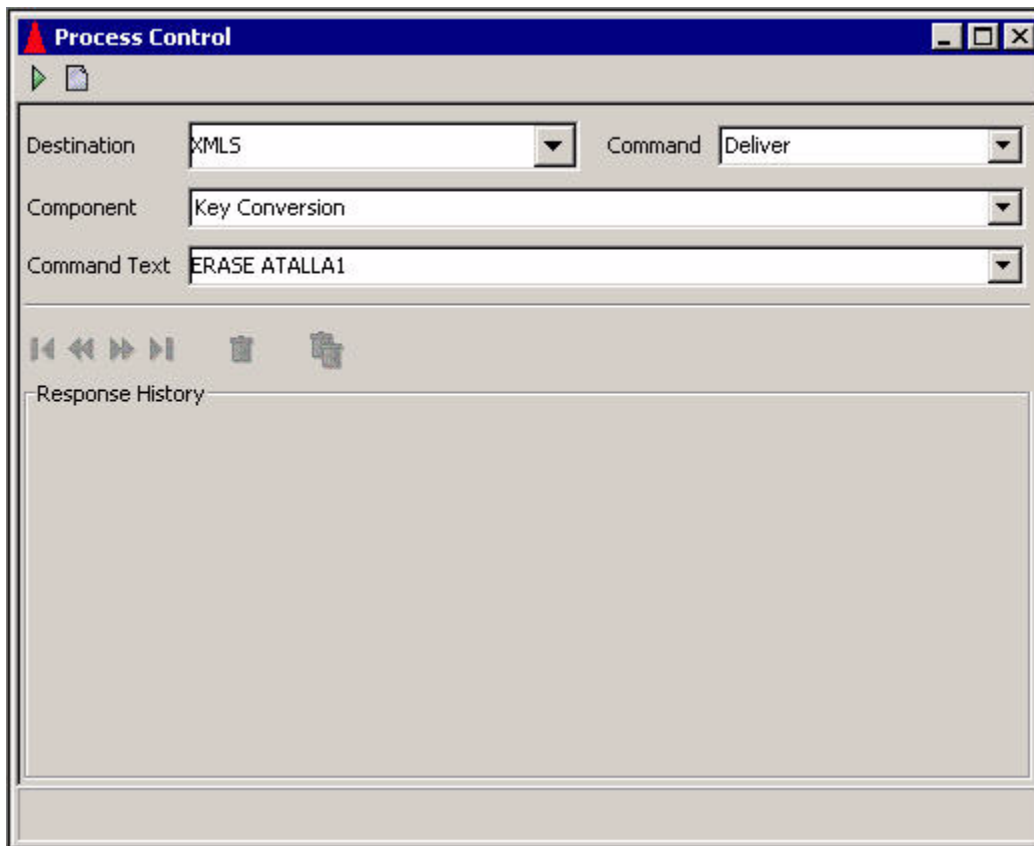
```
ERASE sec-dev-assign-name | *
```

Examples:

```
ERASE ATALLA1  
ERASE *
```

6. Click the Send button at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window.

The following illustration shows the Process Control window values for sending the ERASE command.



Network control facility

An example of the ERASE command entered from an XPNET network control facility (e.g., NCPCOM) is shown below.

```
DELIVER PROCESS P1A^TSSXML001, "DELIVER KEYCONV ERASE THALES1"
```

Disabling a security device

Disabling a security device changes the status of the device from active to inactive, thereby removing the device from service. The Transaction Security component generates an event message, indicating that the device has been deactivated. This command also cancels the automatic reset timer if the Transaction Security component is currently attempting to enable the device at the intervals specified in the Reset Timer Limit field on the Security

Configuration window. This prevents the Transaction Security component from continuing to attempt to re-establish communications with a device when it is to be taken out of service for an extended period of time.

The Transaction Security component does not send requests to inactive devices; it only alternates requests among active devices with a consecutive error count that is less than the maximum.

You can disable a security device by entering a command on the Process Control window or from an XPNET network control facility. Both methods are described below. You do not need to warmboot the Transactions Security Services process for the Security Device Configuration File (SEC_DEV_CONFIG) after disabling the security device.

Note: If you want to permanently disable a security device, set the Status field on the Security Device Configuration window for the device to Inactive and warmboot the SEC_DEV_CONFIG (Destination: PCTL, Command: Alter, Component: Data Source, Command text: SEC_DEV_CONFIG).

Process Control window

To disable one or all security devices from the Process Control window, perform the following:

1. Log on to the ACI desktop user interface and access **System Operations > Process Control**. The Process Control window is displayed.
2. In the Destination field, select IS from the list of values or enter the symbolic name of the XML Server process or a Transaction Security Services process.
3. In the Command field, select Alter from the list of values.
4. In the Component field, select Transaction Security from the list of values.
5. In the Command Text field, enter the following command. If you want to disable a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to disable all security devices in the BASE24 system, enter an asterisk (*) in the command text.

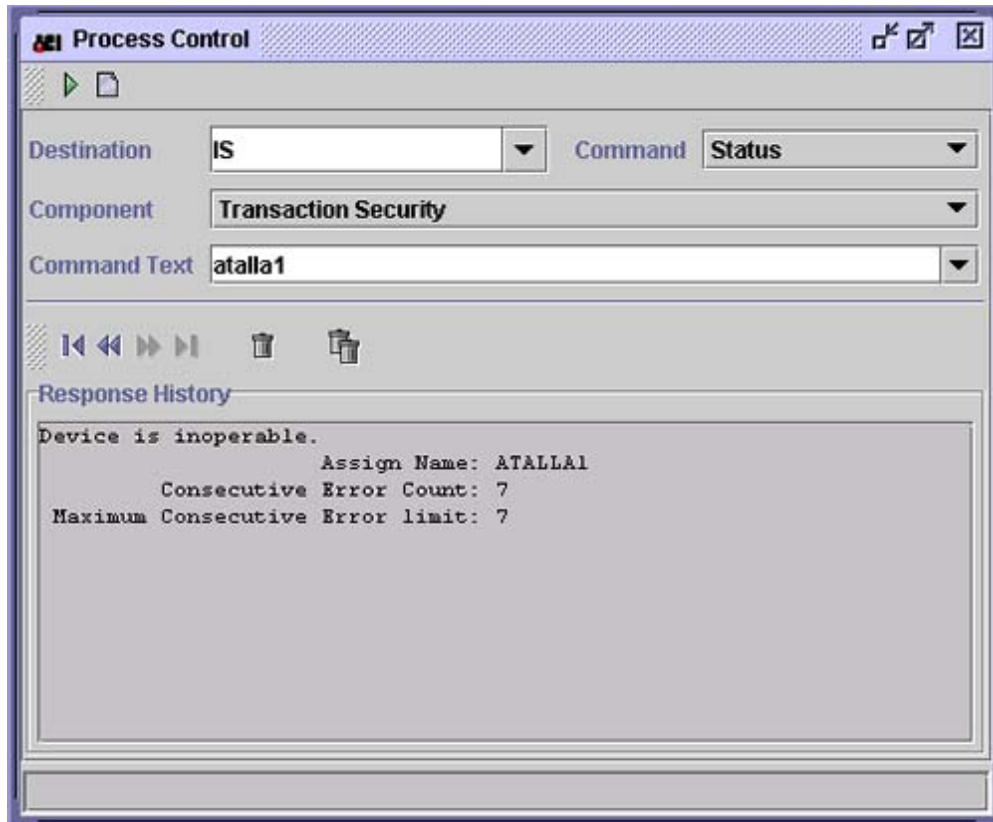
```
DISABLE sec-dev-assign-name | *
```

Examples:

```
DISABLE ATALLA1
DISABLE THALES1
DISABLE ERACOM1
DISABLE BNTNG1
DISABLE DEP1
DISABLE *
```

6. Click the Send button at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window. If the specified security device was successfully disabled, the status of the device on the Security Device Configuration window is changed from “Active” to “Inactive.”

The following illustration shows the response for the Status command after a security device has been disabled. Note that the device is now inoperable and that the consecutive error count has been set to the maximum consecutive error limit value.



Network control facility

An example of the DISABLE command entered from an XPNET network control facility (e.g., NCPCOM) is shown below.

```
DELIVER PROCESS P1A^TSSISL001, "ALTER TSEC DISABLE ATALLA1"
```

Enabling a security device

Enabling a security device causes the Transaction Security component to send a status or echo test message to the security device. If this message is successfully processed, the component changes the status of the device from inactive to active and sets the consecutive error count to zero, thereby returning the device to service. If this message is not successfully processed for a device, the Transaction Security component generates an event

message that indicates the error and specifies whether the status or echo test message will be automatically retried. The Reset Timer Limit field on the Security Device Configuration window specifies the number of seconds the component waits before retrying the security device call. If the Reset Timer Limit field is set to 0, the component does not automatically attempt to retry the call. The Transaction Security component does not send online transaction requests to inactive devices; it only alternates requests among active devices with a consecutive error count that is less than the maximum.

You can enable a security device by entering a command on the Process Control window or from an XPNET network control facility. Both methods are described below. You do not need to warmboot the Transactions Security Services process for the Security Device Configuration File (SEC_DEV_CONFIG) after enabling the security device.

Note: If you want to permanently enable a security device, set the Status field on the Security Device Configuration window for the device to Active and warmboot the SEC_DEV_CONFIG (Destination: PCTL, Command: Alter, Component: Data Source, Command text: SEC_DEV_CONFIG).

Process Control window

To enable one or all security devices from the Process Control window, perform the following:

1. Log on to the ACI desktop user interface and access **System Operations > Process Control**. The Process Control window is displayed.
2. In the Destination field, select IS from the list of values or enter the symbolic name of the XML Server process or a Transaction Security Services process.
3. In the Command field, select Alter from the list of values.
4. In the Component field, select Transaction Security from the list of values.
5. In the Command Text field, enter the following command. If you want to enable a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to enable all security devices in the BASE24 system, enter an asterisk (*) in the command text.

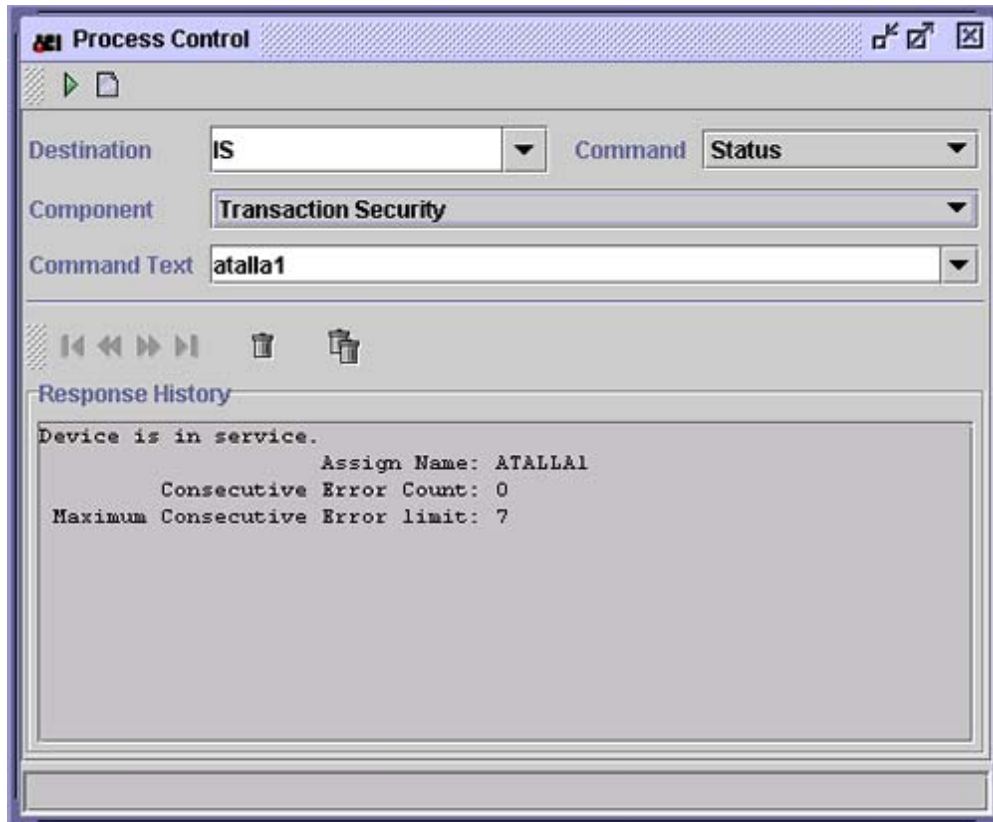
```
ENABLE sec-dev-assign-name | *
```

Examples:

```
ENABLE ATALLA1
ENABLE THALES1
ENABLE ERACOM1
ENABLE BNTNG1
ENABLE DEP1
ENABLE *
```

6. Click the Send button at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window. If the specified security device was successfully enabled, the status of the device on the Security Device Configuration window is changed from “Inactive” to “Active.”

The following illustration shows the response for the Status command after a security device has been enabled. Note that the device is now in service and that the consecutive error count has been reset to zero.



Network control facility

An example of the ENABLE command entered from an XPNET network control facility (e.g., NCPCOM) is shown below.

```
DELIVER PROCESS P1A^TSSISL001, "ALTER TSEC ENABLE  ATALLA1"
```

Initializing a security device

Initializing a security device causes the TSS application to send the text strings configured in the Initialization String and Configuration String fields for the device on the Security Device Configuration window to the device. In addition, if the status of the device is Active, this command causes the Transaction Security component to send a status or echo test message to the security device. If this message is successfully processed, the component sets the

consecutive error count to zero and uses the response to determine the firmware version and maximum buffer length displayed on the Security Device Configuration window for the device. If this message is not successfully processed for a device, the Transaction Security component generates an event message that indicates the error and specifies whether the status or echo test message will be automatically retried. The Reset Timer Limit field on the Security Device Configuration window specifies the number of seconds the component waits before retrying the security device call. If the Reset Timer Limit field is set to 0, the component does not automatically attempt to retry the call.

This command causes the firmware version and maximum buffer length for the device to be displayed on the Security Device Configuration window. This information is not displayed until you initialize the device.

For Bull BNTng HSMS, this command also loads data from the applicable record in the BNTng Initialization File (BNTng_Init_Data) into the HSM.

You can initialize a security device by entering a command on the Process Control window or from an XPNET network control facility. Both methods are described below.

Process Control window

To initialize one or all security devices from the Process Control window, perform the following:

1. Log on to the ACI desktop user interface and access System Operations > Process Control. The Process Control window is displayed.
2. In the Destination field, select IS or XMLS from the list of values or enter the symbolic name of the XML Server process or a Transaction Security Services process.
3. In the Command field, select Deliver from the list of values.
4. In the Component field, select Transaction Security from the list of values.
5. In the Command Text field, enter the following command. If you want to initialize a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to initialize all security devices in the BASE24 system, enter an asterisk (*) in the command text.

```
INIT sec-dev-assign-name | *
```

Examples:

```
INIT ATALLA2
INIT THALES1
INIT ERACOM1
INIT BNTNG1
INIT DEP1
INIT *
```

6. Click the Send button at the top of the Process Control window. A “Command sent” message is displayed in the Response History area of the window.

Network control facility

An example of the INIT command entered from an XPNET network control facility (e.g., NCPCOM) is shown below.

```
DELIVER PROCESS P1A^TSSISL001, "DELIVER TSEC INIT ATALLA1"
```

Obtaining the current status of a security device

The status of a security device indicates whether the device is in service or inoperable, enabled or disabled, and lists the consecutive error count, maximum consecutive error count, firmware version, and maximum buffer length for the device. The Transaction Security component does not send requests to inoperable devices; it only alternates requests among devices in service. To obtain the status of one or all security devices, perform the following:

1. Log on to the ACI desktop user interface and access **System Operations > Process Control**. The Process Control window is displayed.
2. In the Destination field, enter the assign name of a Transaction Security Services process defined in the System Interface Message Delivery Service Destination File (MDSDEST).
3. In the Command field, select Status from the list of values.
4. In the Component field, select Transaction Security from the list of values.
5. In the Command Text field, enter the assign name for the device or an asterisk (*). If you want to obtain the status for a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to obtain the status for all security devices in the BASE24 system, enter an asterisk (*) in the command text.

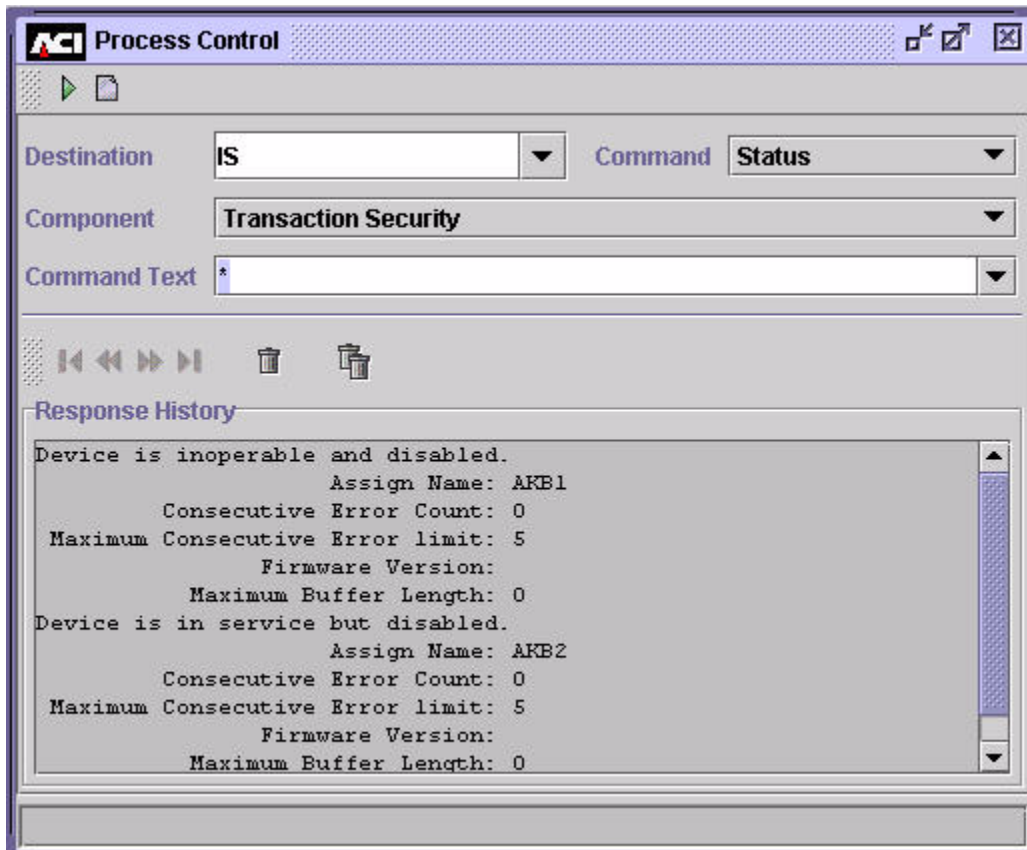
sec-dev-assign-name | *

Examples:

ATALLA1
THALES1
ERACOM1
BNTNG1
DEP1
*

6. Click the Send button at the top of the Process Control window. The status information for the device is displayed in the Response History area of the window. If the specified security device is active, "Device is in service" is displayed. If the specified device is inactive, "Device is inoperable" is displayed. The status response also displays the assign

name for the device, the consecutive error count, the maximum consecutive error limit, firmware version, and maximum buffer length. A sample of the response text for the status of multiple security devices is shown below.



17: Europay, MasterCard, and Visa (EMV)

This section contains miscellaneous topics related to EMV processing when using the Transaction Security Services process.

Cryptographic support

EMV cryptographic support is provided for the following security functions specified by MasterCard, Visa, and Common Core Definitions (CCD):

- For request and response cryptogram processing, Visa uses the derived key stored on the card, whereas MasterCard and CCD generate a session key from the derived key.
- For secure messaging, Visa has a different method of generating the session key from that used by MasterCard and CCD.
- Visa supports secure messaging with both integrity and confidentiality.

CCD cards use the same implementation whether they are issued by MasterCard or Visa.

To support different security implementations, the Transaction Security Services process uses the EMV Scheme field on the EMV tab of the Authentication Profile Configuration window, and the cryptogram version number passed in the EMV request, to determine the key and transaction data fields to use in cryptogram functions. See the “EMV Scheme and Cryptogram Version Number” topic below.

Both Visa and MasterCard permit the use of issuer-proprietary cryptogram processing. This is identified by a specific cryptogram version number. The data fields required to generate the issuer-proprietary cryptogram are set up based on the cryptogram version number in the request.

EMV scheme and cryptogram version number

To support different security implementations for individual payment schemes, the EMV Scheme field on the EMV tab of the Authentication Profile Configuration window identifies the specific EMV implementation associated with an EMV profile (i.e., authentication data). This field, in conjunction with the cryptogram version number passed in requests (or specified in the Cryptographic Version Number field on the EMV tab of the Authentication Profile

Configuration window if the cryptogram version number is not present in the EMV transaction request), is used by the Transaction Security Services process to determine the specific security module calls and security functions to be performed (key derivation method). Each of the valid EMV schemes are explained below.

- Visa—Issuer application data (IAD) format defined by Visa in the **Visa Integrated Circuit Card Specification**. (Note this format is used for UKIS cards).
- MasterCard/Europay—IAD format defined for MasterCard Chip Payment Application (MCPA) version 2.1 (M/Chip 2.1) cards. The MasterCard EPI/MCI derivation method also refers to this scheme.
- M/CHIP 4—IAD format defined for MasterCard Chip Payment Application (MCPA) version 4 (M/Chip 4) cards.
- CCD—IAD format defined for EMV version 4.1 cards. The EMV CSK derivation method also refers to this scheme.
- Interac—IAD format defined for Interac Flash cards. The EMV 2000 derivation method with a few Interac-specific changes is used for this scheme.

Note: On the Authentication Profile Configuration window, only values of Visa, MasterCard/Europay, CCD, and Interac are available in the EMV Scheme field. For MasterCard/Europay, the cryptogram version number is sufficient to differentiate between M/Chip 2.1 and M/Chip 4 cards. For M/Chip 4 cards, the valid cryptogram version number values are %h00, %h11, %h12, %h13, %h14 and %h15. For M/Chip 2.1 cards, the valid cryptogram version number values are %h0x, where x represents the characters A–F.

Cryptogram version numbers in EMV

The Cryptogram Version Number is used (in conjunction with the EMV scheme) to identify the security requirements in EMV online transaction processing. In particular, the Cryptogram Version Number defines the required processing in the following areas:

- Card key derivation method
- Session key derivation method for ARQC calculation
- Input data in ARQC/TC calculation
- Method used to calculate ARQC
- Session key derivation method for ARPC calculation
- Input data and data length in ARPC calculation
- Method used to calculate ARPC
- Session key derivation method for secure messaging
- Data format in secure messaging MAC calculation
- Padding of encrypted data for secure messaging
- Method used to encrypt data in secure messaging

The tables below indicate the security processing options, data formats, and methods used for each supported combination of EMV scheme and cryptogram version number. You can obtain further details on each of these security processing options, data formats, and methods in the EMV and card scheme specifications.

Throughout these tables, *x* represents the characters 0–9 and A–F in cryptogram version number %h0*x*, and *y* represents the characters A–F in cryptogram version number %hy5.

The processing performed within security devices for these functions is depicted in several diagrams at the end of this topic.

Note: For proprietary reasons, the cryptographic methods used for the Interac Flash EMV scheme are not documented.

Card key derivation method

Security processing options	EMV application	
	EMV scheme	CVN
Option A – Format a 16-byte data block by concatenating the PAN and application PAN sequence number (APSN), using padding or truncation if required. The left half of the key is computed by 3DES encrypting the PAN/APSN with the master key. The right half of the key is computed by 3DES encrypting the inverted (bitwise) PAN/APSN with the master key.	VIS	%h0A
	VIS	%h11 ¹
	VIS	%h0E
	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
Option B – Use the SHA-1 hashing algorithm on the PAN and APSN, and then create a 16-byte data block by selecting decimal digits from the output of the hash result. The left half of the key is computed by 3DES encrypting the PAN/APSN with the master key. The right half of the key is computed by 3DES encrypting the inverted (bitwise) PAN/APSN with the master key.	CCD	%hy5

¹Cryptogram version number 17 for Visa cards is used to support quick Visa Smart Debit/Credit card (qVSDC) transactions. These transactions use a minimum of data to ensure quick transactions over a contactless interface, such as one employed for a low value payment toll booth operation.

Session key derivation method for ARQC calculation

Security processing options	EMV application	
	EMV scheme	CVN
None – Use card key.	VIS	%h0A
	VIS	%h11
EMV 2000 – Use a specified function that contains a branch factor, a <i>parent</i> key, and a <i>grandparent</i> key. Use the function to take bits from the ATC to derive a session key that is unique for each value of the ATC.	VIS	%h0E
	M/Chip	%h12
	M/Chip	%h13
MasterCard-proprietary – Format an 8-byte <i>diversification value</i> , which is the 2-byte ATC concatenated with 2 bytes of %h00 and the 4-byte unpredictable number. Use the diversification value to generate the left and right halves of the session key. The left half is computed by replacing the third byte of the data block with %hF0, and 3DES encrypting the result with the card key. The right half is computed by replacing the third byte of the data block with %h0F, and 3DES encrypting the result with the card key.	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
Common – Format an 8-byte <i>diversification value</i> , which is the 2-byte ATC concatenated with 6 bytes of %h00. Use the diversification value to generate the left and right halves of the session key. The left half is computed by replacing the third byte of the data block with %hF0, and 3DES encrypting the result with the card key. The right half is computed by replacing the third byte of the data block with %h0F, and 3DES encrypting the result with the card key.	M/Chip	%h14
	M/Chip	%h15
	CCD	%hy5

Input data in ARQC/TC calculation

Input data format	EMV application	
	EMV scheme	CVN
Amount, Auth Amount, Other Terminal Country Code Terminal Verification Results (TVR) Transaction Currency Code Transaction Date Transaction Type Unpredictable Number Application Interchange Profile (AIP) Application Transaction Counter (ATC) Card Verification Results (4 bytes)	VIS	%h0A
	VIS	%h0E
	M/Chip	%h0x
Amount, Auth Transaction Currency Code Unpredictable Number	VIS	%h11
Amount, Auth Amount, Other Terminal Country Code Terminal Verification Results (TVR) Transaction Currency Code Transaction Date Transaction Type Unpredictable Number Application Interchange Profile (AIP) Application Transaction Counter (ATC) Card Verification Results (6 bytes)	M/Chip	%h10
	M/Chip	%h12
	M/Chip	%h14
Amount, Auth Amount, Other Terminal Country Code Terminal Verification Results (TVR) Transaction Currency Code Transaction Date Transaction Type Unpredictable Number Application Interchange Profile (AIP) Application Transaction Counter (ATC) Card Verification Results (6 bytes) Counters (8 bytes)	M/Chip	%h11
	M/Chip	%h13
	M/Chip	%h15

Input data format	EMV application	
	EMV scheme	CVN
Amount, Auth Amount, Other Terminal Country Code Terminal Verification Results (TVR) Transaction Currency Code Transaction Date Transaction Type Unpredictable Number Application Interchange Profile (AIP) Application Transaction Counter (ATC) Issuer Application Data	CCD	%hy5

Method used to calculate ARQC

Security processing options	EMV application	
	EMV scheme	CVN
Visa-proprietary – Format a data block, comprising the input data padded to the right with bytes of %h00 until the data block is a multiple of 8 bytes in length. Calculate an 8-byte ARQC by using the key to generate a MAC over the data block.	VIS	%h0A
	VIS	%h11
EMV – Format a data block, comprising the ARQC concatenated with the input data and one byte containing %h80. If necessary, append bytes of %h00 until the data block is a multiple of 8 bytes in length. Calculate an 8-byte ARQC by using the key to generate a MAC over the data block.	VIS	%h0E
	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
	CCD	%hy5

Session key derivation method for ARPC calculation

Derivation method	EMV application	
	EMV scheme	CVN
None – Use card key.	VIS	%h0A
	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
EMV 2000 – Use a specified function that contains a branch factor, a <i>parent</i> key, and a <i>grandparent</i> key. Use the function to take bits from the ATC to derive a session key that is unique for each value of the ATC.	VIS	%h0E
	M/Chip	%h12
	M/Chip	%h13
Common – Format an 8-byte <i>diversification value</i> , which is the 2-byte ATC concatenated with 6 bytes of %h00. Use the diversification value to generate the left and right halves of the session key. The left half is computed by replacing the third byte of the data block with %hF0, and 3DES encrypting the result with the card key. The right half is computed by replacing the third byte of the data block with %h0F, and 3DES encrypting the result with the card key.	M/Chip	%h14
	M/Chip	%h15
	CCD	%hy5

Input data and data length in ARPC calculation

Input data and data length	EMV application	
	EMV scheme	CVN
ISO 8583 (1987) Response Code (2 bytes). ARPC is 8 bytes.	VIS	%h0A
	VIS	%h0E
	M/Chip	%h0x

Input data and data length	EMV application	
	EMV scheme	CVN
16-bit ARPC Response Code (2 bytes). ARPC is 8 bytes.	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
Card Status Updates (4 bytes) plus Proprietary Authentication Data (up to 8 bytes). ARPC is 4 bytes (truncated from 8 bytes).	CCD	%hy5

Method used to calculate ARPC

Calculation method	EMV application	
	EMV scheme	CVN
Method 1 – Format an 8-byte data block, comprising the input data padded to the right with bytes of %h00. Calculate an 8-byte ARPC by XORing the data block with the ARQC, and 3DES encrypting the result with the key. Note: this method cannot support more than 8 bytes of input data.	VIS	%h0A
	VIS	%h0E
	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
Method 2 – Format a data block, comprising the ARQC concatenated with the input data and one byte containing %h80. If necessary, append bytes of %h00 until the data block is a multiple of 8 bytes in length. Calculate an 8-byte ARPC by using the key to generate a MAC over the data block. Note: this method is also used for all schemes and CVNs to calculate the MAC required for secure messaging with integrity.	CCD	%hy5

Session key derivation method for secure messaging

Derivation method	EMV application	
	EMV scheme	CVN
Visa-proprietary. Format an 8-byte <i>diversification value</i> , which is the 2-byte ATC right justified and padded to the left with 6 bytes of %h00. Use the diversification value to generate the left and right halves of the session key. The left half is computed by XORing the data block with the left half of the card key. The right half is computed by performing a bitwise inversion and XORing the result with the right half of the card key.	VIS	%h0A
EMV 2000 – Use a specified function that contains a branch factor, a <i>parent</i> key, and a <i>grandparent</i> key. Use the function to take bits from the ATC to derive a session key that is unique for each value of the ATC.	VIS	%h0E
	M/Chip	%h12
	M/Chip	%h13
Common – Format an 8-byte <i>diversification value</i> , which is the ARQC. Use the diversification value to generate the left and right halves of the session key. The left half is computed by replacing the third byte of the data block with %hF0, and 3DES encrypting the result with the card key. The right half is computed by replacing the third byte of the data block with %h0F, and 3DES encrypting the result with the card key.	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h14
	M/Chip	%h15
	CCD	%hy5

Data format in secure messaging MAC calculation

Data format	EMV application	
	EMV scheme	CVN
Format 2 - The data elements (including the MAC) contained in the data field of the secured command are not composed of TLV data objects. Transaction-specific data (e.g., the ATC) is inserted into the data field (immediately before any command data) to ensure that the MAC value generated is unique per transaction.	VIS	%h0A
	VIS	%h0E
	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
Format 1 - The data field of the secured command is composed of TLV data objects. If the command to be secured has command data, then the command data is carried in the first data object, and is itself encapsulated under an EMV tag. The second data object is the MAC, which is encapsulated under tag 8E.	CCD	%hy5

Padding Of encrypted data for secure messaging

Padding method	EMV application	
	EMV scheme	CVN
Visa-proprietary - Prepend enciphered data with one byte containing the length of the enciphered data. Append enciphered data with one byte containing %h80. If necessary, append bytes of %h00 until data block is a multiple of 8 bytes in length.	VIS	%h0A
	VIS	%h0E

Padding method	EMV application	
	EMV scheme	CVN
MasterCard-proprietary - If the enciphered data is not a multiple of 8 bytes in length, then append one byte containing %h80. If necessary, append bytes of %h00 until data block is a multiple of 8 bytes in length.	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
EMV - Append enciphered data with one byte containing %h80. If necessary, append bytes of %h00 until data block is a multiple of 8 bytes in length.	CCD	%hy5

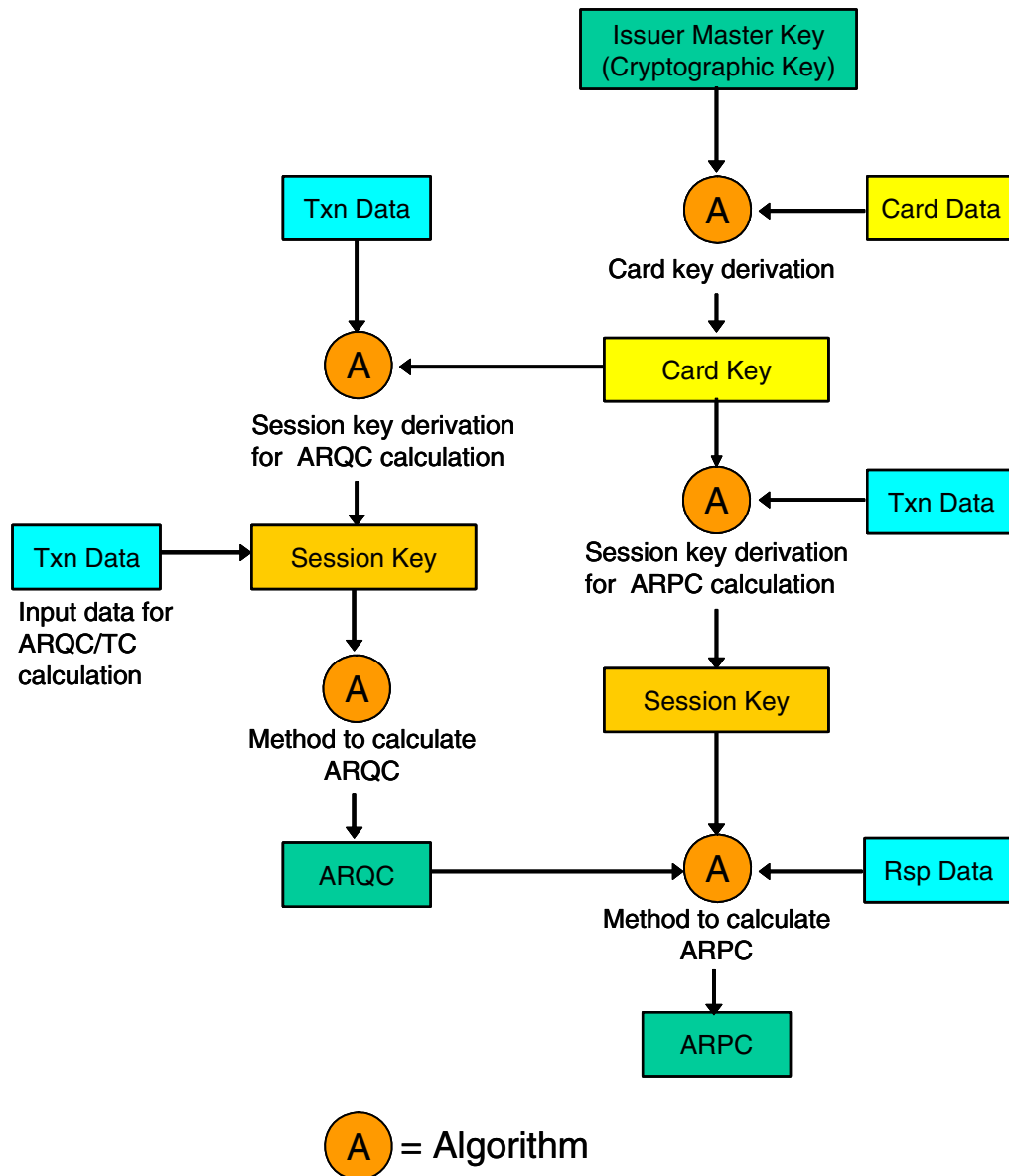
Method used to encrypt data in secure messaging

Encryption method	EMV application	
	EMV scheme	CVN
Electronic Code Book (ECB).	VIS	%h0A
	VIS	%h0E
Cipher Block Chaining (CBC).	M/Chip	%h0x
	M/Chip	%h10
	M/Chip	%h11
	M/Chip	%h12
	M/Chip	%h13
	M/Chip	%h14
	M/Chip	%h15
	CCD	%hy5

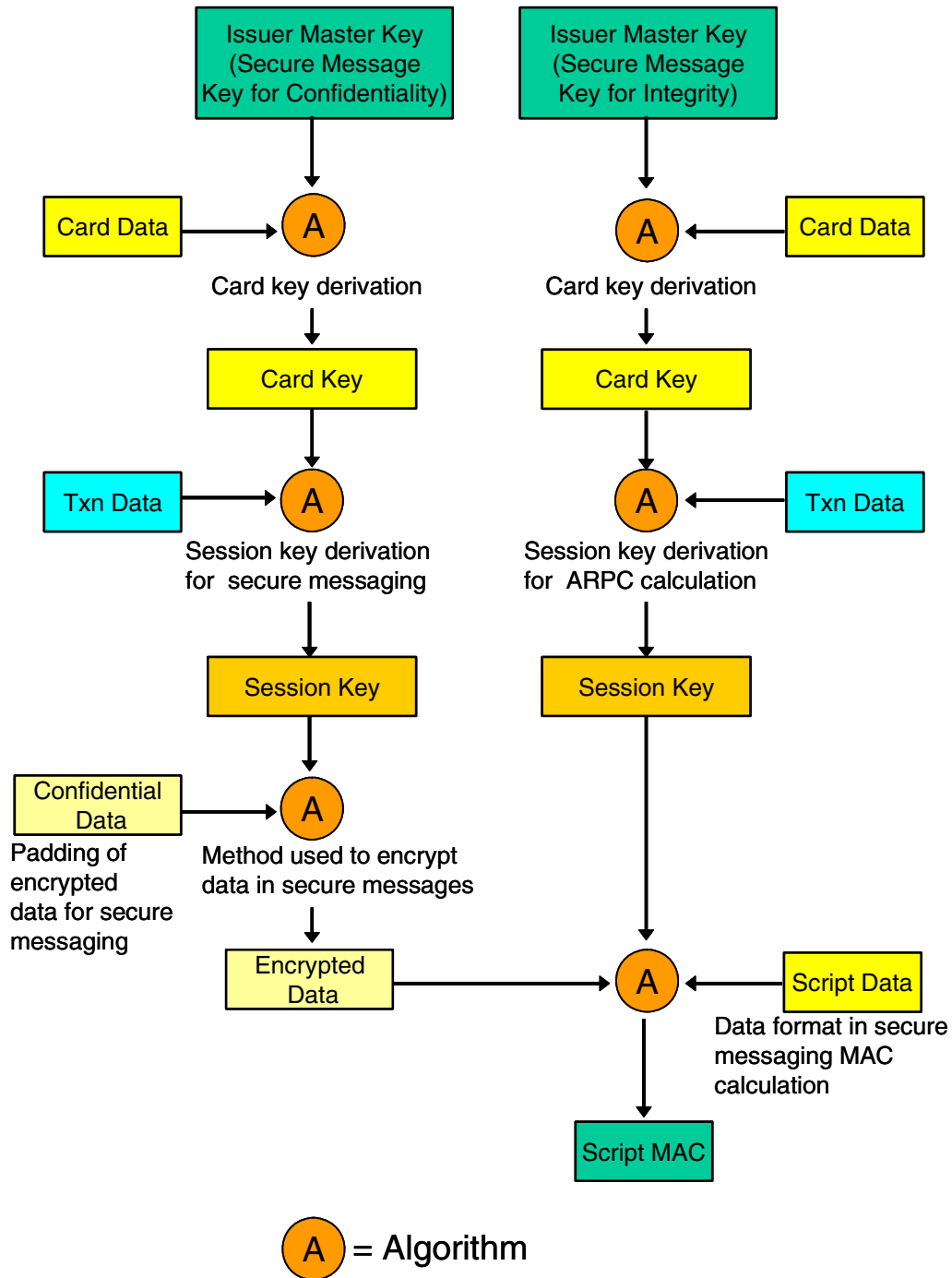
Cryptographic processing flows

The following diagrams depict the flow of cryptographic functions for authorization request and response cryptogram processing and for secure message processing. All of these functions are performed within a hardware security device.

Authorization request and response processing



Secure message processing



Issuer script commands

Issuer script commands allow the EMV issuer to alter the data or functionality of cards already in circulation. A script is a command or a string of commands transmitted by the issuer to the terminal for the purpose of being sent serially to the integrated circuit card (ICC) as commands. The terminal forwards a script command to the card, if provided from the issuer. The card uses the script data to reconfigure itself. For example, issuer script commands can be used to support offline PIN unblocking. PIN unblocking is needed when the PIN try counter on the card has reached zero and needs to be reset to the normal value (i.e., the PIN try limit value).

Script data passed between the EMV issuer and the card is secured with a message authentication code (MAC) generated by a hardware security module. The Secure Message Key for Integrity field on the EMV tab of the Authentication Profile Configuration window contains a key used to derive the key used to generate the MAC added to secure the script command.

EMV issuer script commands are supported in pass-through mode by the BASE24 system.

Offline PIN change

Offline PIN validation allows a cardholder PIN to be verified by the EMV card at an ATM. The management of offline PIN validation requires two administrative functions for card issuers:

- Support for offline PIN changes (i.e., the cardholder changes his or her PIN to a new value at an ATM). The new PIN must be transmitted to the card so that the PIN value on the card can be synchronized with the new PIN offset value in the issuer's database.
- Support for offline PIN unblocking. In this case, the PIN try counter on the EMV card has reached zero and needs to be reset to the normal value (i.e., the maximum PIN try limit).

The Secure Message Key for Confidentiality field on the EMV tab of the Authentication Profile Configuration window contains a key used to derive the key used to encrypt the PIN block for secure transmission to the card. The Destination PIN Block Type field on this tab specifies the PIN block format in which the new PIN is transmitted to the card. Offline PIN unblocking is supported using an issuer script command. A sample of the EMV tab on the Authentication Profile Configuration window is provided in [section 2](#).

Note: An EMV PIN change function is not currently supported by Bull BNTng HSMs.

Notification to host

If you need to notify a card management system of offline PIN changes, set the NEW PIN TRANSPORT KEY LOCATOR field on Card Prefix File (CPF) screen 11 to a PIN block encryption key locator value and the OFFLINE PIN MANAGEMENT ACTION field to a value of 2. The PIN block encryption key locator value must correspond to a primary key value (interface name)

for a record in the ICHG_SECURITY. This record must specify a PIN encryption key in the Outbound Keys field on the PIN information tab of the Interface Security Configuration window. The Transaction Security Services process translates the new PIN block to encryption under this key. The translated PIN block is then written to the PIN Change token in the Transaction Log File (TLF). When the card management system receives the new PIN block (e.g. in a TLF extract), it uses the same key as stored in the ICHG_SECURITY to decrypt the new PIN block.

Destination PIN block

The Transaction Security Services process uses different destination PIN blocks when generating PIN Change issuer script commands, based on the EMV scheme selected for EMV processing in the EMV Scheme field on the EMV tab of the Authentication Profile Configuration window:

- If you select the Visa scheme, the Transaction Security Services process uses the Visa Format Without Using Current PIN for the destination PIN block.
- If you select the MasterCard/Europay or Interac scheme, the Transaction Security Services process uses the Standard EMV PIN Block for the destination PIN block.
- If you select the CCD scheme, the Transaction Security Services process can use either the Standard EMV PIN Block or Visa Format Without Using Current PIN for the destination PIN block, however, CCD specifications mandate use of the Standard EMV PIN Block for the destination PIN block.

The Destination PIN Block Type field on the EMV tab of the Authentication Profile Configuration window is a display-only field when you select Visa or MasterCard/Europay in the EMV Scheme field. It displays the PIN block format value appropriate to the EMV scheme selected.

Refer to the appropriate vendor documentation for detailed descriptions of destination PIN block formats.

EMV 2000

The Transaction Security Services process supports EMV 2000 (key derivation method) processing for Atalla NSPs, Banksys DEP HSMs, Bull BNTng HSMs, SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs), and Thales e-Security hardware security modules (HSMs). The Transaction Security Services process uses the EMV scheme and cryptogram version number to determine the key derivation method used in processing.

Atalla NSPs

EMV 2000 processing is supported on the following Atalla NSPs: Axx100-V or Axx100 (AKB) running version 2.62 or later software or Axx150V/Axx160V or Axx150/Axx160 (AKB) running version 3.0C or later software.

For Atalla NSPs, the Transaction Security Services process sends the branch/height parameter in 350, 351, and 352 commands requests for EMV 2000 processing. In addition, for the 350 and 351 command requests, the Transaction Security Services process passes the ATC instead of a random number for EMV 2000 processing.

Banksys DEP HSMs

For Banksys DEP (HSMs), the Transaction Security Services process supports EMV 2000 processing for firmware with a version of FINTXS 1.0.e or later. For Banksys DEP HSMs, the Transaction Security Services process sends the branch/height parameter in 02260A00, 02262500, 02261B00, and 02260F00 command requests for EMV 2000 processing.

Bull BNTng HSMs

EMV 2000 processing is supported on Bull BNTng HSMs running version 7 or later software. For Bull BNTng HSMs, the Transaction Security component sends the branch/height parameter in E2, E3, and E4 command requests for EMV 2000 processing.

SafeNet (Eracom) HSMs

EMV 2000 processing is supported in SafeNet (Eracom) ProtectHost White Mark II host security modules running software with a minimum release date of 25-Jun- 2002 or later.

The following two functions are used for EMV 2000 processing instead of the EMV CCD Script Crypto (EE2007) and EMV CCD PIN Change/Unblock (EE2016) functions:

- EMV 2000 Script Crypto (EF2013)
- EMV 2000 PIN Change/Unblock (EE2017)

The EF2013 function performs similar processing to the EE2007 function and the EE2017 function performs similar processing to the EE2016 function. The EF2013 and EE2017 functions, however, use the EMV 2000 method (session key derivation function) for generating the session key.

Thales e-Security HSMs

For Thales e-Security hardware security modules (HSMs), the Transaction Security Services process supports EMV 2000 processing for version 2.0 or later software on the HSM 8000, version 3.0 or later for the RG7x10, and version 7.0 or later for the RG7x00.

The following three commands are used for EMV 2000 processing instead of the KQ and KU EMV authorization commands:

- Generate Secure Message with Integrity and optional Confidentiality and PIN Change (KY)
- ARQC (or TC/ACC) Verification or ARPC Generation (KW)
- Verify Encrypted Counters - M/Chip 4 (K0)

The KY command performs a similar function to the KU command and the KW command performs a similar function to the KQ command. The KY and KW commands, however, use the EMV 2000 method (session key derivation function) for generating the session key.

Session key derivation function

This method uses the session key derivation function, which generates a unique session key for each ICC application transaction by generating a *tree* of keys. The ICC master key entered in the Cryptographic Keys field on the EMV tab of the Authentication Profile Configuration window serves as the base of this tree. Several levels of intermediate keys reside above it, with each intermediary key derived from the keys beneath it. At the very top of the tree are the session keys, with one session key per value of the application transaction counter (ATC). For detailed information on session key derivation for EMV 2000, refer to your hardware security module vendor documentation and the appropriate EMV 2000 specifications.

The session key derivation function uses two parameters—the height of the tree (i.e., the number of levels of intermediary keys in the tree excluding the base level) and the *branch* factor (i.e., the number of *child* keys that can be derived from a *parent* key). The parent key must be one level lower in the tree to the child key it derives. You configure the tree height and branch factor in the Branch/Height Parameters field on the EMV tab of the Authentication Profile Configuration window. Possible values for this field are:

- Branch factor 2; Tree Height 16
- Branch factor 4; Tree Height 8

The Transaction Security Services process passes the branch factor and tree height parameters you configure in extended 350, 351, and 352 command requests sent to an Atalla NSP, in E2, E3, and E4 command requests sent to a Bull BNTng HSM, in EF2013, EE2017, and EE2018 function requests sent to a SafeNet (Eracom) ProtectHost White Mark II HSM, and in KY, KW, and K0 command requests to a Thales e-Security HSM.

Atalla NSPs also require a 16-byte initializing value (IV) and index parameters in the extended 350, 351, and 352 commands. The Transaction Security Services process does not send these parameters in command requests. Therefore, the Atalla NSP uses default values of zeros for the IV and 7 for the index.

Data block padding

In the Atalla 350 command, Bull BNTng E2 command, SafeNet (Eracom) EE2018 function, and Thales KW command, the Transaction Security Services process appends the data block with the x80 pad character and additional padding of binary zeros to ensure it is a multiple of 8 bytes.

Common session key derivation

The Transaction Security Services process supports two Common Session Key Derivation methods—MasterCard proprietary and EMV common—for Atalla NSPs, SafeNet (Eracom) ProtectHost White Mark II host security modules (HSMs), and Thales e-Security hardware security modules (HSMs). The Transaction Security Services process uses the EMV scheme and cryptogram version number to determine the key derivation method used in processing.

Atalla NSPs

Common session key derivation processing is supported on the following Atalla NSPs: Axx100-V or Axx100 (AKB) running version 2.82 or later software or Axx150V/Axx160V or Axx150/Axx160 (AKB) running version 3.0C or later software.

For the CCD EMV scheme, the Transaction Security Services process passes the entire PAN in 350, 351, and 352 command requests for common session key derivation processing. For all other EMV schemes, only the first 14 PAN digits are passed in 350, 351, and 352 command requests. Also for the CCD EMV scheme, the CSU is passed in the 351 command request. For all other EMV schemes, the ARC is passed in the 351 command request.

In addition, for the 351 and 352 command requests, the Transaction Security Services process passes the ARQC instead of the ATC for common session key derivation.

SafeNet (Eracom) ProtectHost White Mark II HSMs

Common session key derivation processing is supported on SafeNet (Eracom) ProtectHost White Mark II HSMs running the appropriate release of software.

The Transaction Security Services process uses the EF2013, EE2017, and EE2018 command requests for common session key derivation processing. For all of these requests, the PAN/PAN Sequence Number must be an even number of bytes. If the PAN/PAN Sequence number

is an odd number of bytes, a 0 digit is prepended to the left. Also for the CCD EMV scheme, the CSU is passed in the EE2018 command request. For all other EMV schemes, the ARC is passed in the EE2018 command request.

Thales e-Security HSMs

For Thales e-Security hardware security modules (HSMs), the Transaction Security Services process supports common session key derivation processing for version 2.1 or later software on the HSM 8000, version 3.01 or later for the RG7x10, and version 7.01 or later for the RG7x00.

The Transaction Security Services process uses the KY, KW, and K0 command requests for common session key derivation processing. For all of these requests, the PAN/PAN Sequence Number must be an even number of bytes. If the PAN/PAN Sequence number is an odd number of bytes, a 0 digit is prepended to the left. Also for the CCD EMV scheme, the CSU is passed in the KW command request. For all other EMV schemes, the ARC is passed in the KW command request.

MasterCard proprietary session key derivation

The MasterCard proprietary session key derivation function generates a unique session key for each transaction performed by the application. It does this by enciphering an 8-byte *diversification value* with the 16-byte ICC Master Key (MK) to produce a 16-byte ICC Session Key (SK). The diversification value is used to generate the left and right halves (parts) of the double-length session key. The left half is computed by replacing the third byte of the data block with %hF0, and 3DES encrypting the result with the card key. The right half is computed by replacing the third byte of the data block with %h0F, and 3DES encrypting the result with the card key. The session key is the concatenation of the left and right halves (parts).

The diversification key is used to help prevent replay attacks. The data used for this value should have a high probability of being different for each session key derivation. An important requirement for the diversification function is that the number of possible outputs of the function is sufficiently large and uniformly distributed to prevent an exhaustive key search on the session key.

This method is only used only when deriving the session key for calculating the ARQC.

EMV common session key derivation

The EMV common session key derivation function works just like the proprietary MasterCard method described above, except that it uses a different diversification value.

For ARQC and ARPC calculation, the diversification value is the 2-byte ATC concatenated with 6 bytes of %h00.

For the session keys used for secure messaging, the diversification value is the 8-byte ARQC returned in the first GENERATE AC command. The same session key is used for all script commands in a single transaction.

Data block padding

In the Atalla 350 command, SafeNet (Eracom) ProtectHost White Mark II EE2018 function, and Thales KW command, the Transaction Security Services process appends the data block with the x80 pad character and additional padding of binary zeros to ensure it is a multiple of 8 bytes.

Chip authentication program (CAP) token validation

The Transaction Security Services process supports chip authentication program (CAP) token validation for MasterCard's OneSMART Authentication solution and Visa's Dynamic Passcode Authentication (DPA) solution. CAP token validation provides two factor authentication (2FA) in card-not-present point-of-sale (POS) environments. Because two factor authentication uses two security items—the chip on the card and a PIN—it is more secure than using just chip or PIN verification.

CAP token validation can be used with contact and contactless EMV cards or on chip cards used only for CAP token validation. The Transaction Security Services process supports CAP token validation in Thales e-Security host security modules (HSMs).

Processing overview

When a OneSMART or DPA cardholder performs a transaction, the cardholder inserts his or her chip card in a personal card reader (PCR) and enters a PIN. The PCR generates a unique one-time password (OTP) that the cardholder then conveys to an authentication service (e.g., Internet banking server). Because the OTP is unique for each transaction performed by the cardholder, it is effective in thwarting *phishing* attacks where fraudsters entice users to reveal confidential financial information.

The OTP is a CAP token (i.e., an EMV cryptogram) generated with purely static/default data, except for the ATC on the card. The authentication server can provide a *challenge* to the cardholder in the form of an unpredictable number that is included in the CAP token. The CAP token can also include some variable transaction data (e.g., the transaction amount and currency code) that is also included in the cryptogram generation. If the authentication server does not actually use an unpredictable number in a challenge/response (CR) or the actual transaction amount and currency in generating the CAP token, then it must use default values for this data in generating the CAP token. The PCR must respond to the authentication server challenge with the appropriate response.

For example, the authentication server could use the OTP as the encryption key to transmit the unpredictable number to the PCR. The PCR must return as its response a similarly-encrypted value which is some predetermined function of the originally-offered information, thus proving that it was able to decrypt the challenge.

The authentication service formats a CAP authentication request, including the CAP token, that it sends to the BASE24-pos system. The CAP authentication request is sent as a Card Verification transaction request that includes the CAP token in the Authentication Date (CE) token. The request can be sent through the ISO Host Interface process or any BASE24-pos interface or device handler (e.g., SPDH) that supports the Card Verification transaction and CE token.

Note: Because BASE24-pos also uses card verification transactions for secure internet validation, CAP token validation cannot be implemented in a BASE24 system that already supports secure internet validation, and vice versa.

When the BASE24-pos system receives the Card Verification transaction, the following steps are performed:

1. The acquiring endpoint in the BASE24-pos system sends the Card Verification request to the Router/Authorization module. The Router/Authorization CAP extension module performs limit checking and invokes the CAP Security utilities to perform CAP token validation. The CAP Security utilities perform the following:
 - a. Retrieve the CAP group (key locator value for the Chip Authentication data source (Chip_Authentication)) from the CPF.
 - b. Retrieve the ATC from the last successfully validated CAP token for the card from the CAF or CAFD.
 - c. Format TSS message 60 (CAP Authenticate) and send the message to Transaction Security Services process using a direct write/read interface. This message contains the CAP group, expiration date, derivation key index, PAN, PAN sequence number (PSN), CAP token, ATC, unpredictable number, authorization amount, and transaction currency code.

If the PAN sequence number or derivation key index are not present in the transaction, default values are retrieved from the CAF.
2. The Transaction Security Services process uses the key locator value and expiration date from TSS message 60 to retrieve the following data from the Chip Authentication external memory table (Chip_Authentication_OLTP):
 - Issuer application data
 - Key locator value for the EMV Security external memory table (EMV_Security_OLTP)
 - Application interchange profile
 - Predicted card verification results and initial card verification results
 - A flag indicating how the initial card verification results are to be used
 - Internet authentication flags (IAF)
 - Issuer proprietary bitmap (IPB)
 - Chip authentication application
 - ATC resynchronization flag

- HSM-specific parameters
3. The Transaction Security Services process parses the CAP token to retrieve any of the following data indicated by the IPB and IAF retrieved from the Chip_Authentication_OLTP record:
 - PSN: none, part or all of the PSN value is retrieved from the CAP token and mapped onto the base PSN value from the call
 - CID: bit 8 is retrieved if present and is used to set the appropriate bit in the CVR
 - ATC: all or the lowest significant bits are retrieved
 - AC: any number or combination of bits is supported
 - DKI: none, part or all of the derivation key index (DKI) value is retrieved from the CAP token and mapped onto the base DKI value from the call
 - CVN: none, part or all of the CVN value will be retrieved from the CAP token and mapped onto the base CVN value from the Chip_Authentication_OLTP
 - CVR: none, part or all of the CVR value is retrieved from the CAP token and mapped onto the base CVR value from the Chip_Authentication_OLTP
 4. The Transaction Security Services process calculates the ATC as defined by the MasterCard specification.
 5. The Transaction Security Services process uses the EMV key locator value from the Chip_Authentication_OLTP record and the DKI from the message to retrieve the following data from the EMV Authorization external memory table (EMV_Security_OLTP):
 - EMV scheme/cryptographic version number
 - Issuer master key, using the DKI from the message
 - Branch/height parameters
 6. The Transaction Security Services process alters the default transaction date if the chip authentication application is DPA.
 7. The Transaction Security Services process uses the IAF from the Chip_Authentication_OLTP record to parse the AC part of the IPB into a 16-character string for the call to the HSM.
 8. The Transaction Security Services process determines whether the use_init_crd_vrfy_rslts field (set in Use Initial Card Verification Results field on the Chip Auth tab of the Authentication Profile Configuration window) is set to ATC 0 or ATC 1. If it is, and the ATC in the token is set to the same value, it uses the initial CVR instead of the predicted CVR. See the “[Initial CVR](#)” topic for more information.
 9. The Transaction Security Services process formats a CAP token validation command request using the retrieved data and sends the command to the security device.
 10. The security device validates the CAP token and returns a response indicating whether the token is valid to the Transaction Security Services process.

If the response from the security device indicates a “validation failure” and the Automatic Resynch Attempts field on the Chip Auth tab of the Authentication Profile Configuration window is set to a nonzero value, the process repeats the validation command request the indicated number of attempts. For each attempt, the process increases the value of the ATC by 2^n , where n is the number of ATC bits in the IPB. The process stops once the number of resynch attempts is performed, the CAP token is successfully validated, or the ATC value exceeds 0xFFFF.

11. The Transaction Security Services process formats a response TSS message 60 that indicates the outcome of the CAP validation and also includes the new ATC value if the CAP token was successfully validated. The Transaction Security Services process sends the response back to the CAP security utilities.
12. The Router/Authorization CAP extension module performs the following, based on the response:
 - If the CAP token validation failed, it increments the Bad CAP Token Validation Accumulator for the primary or secondary card in the CAF, declines the transaction by setting an appropriate response code, and performs no further CAP processing.
 - If the CAP token validation was successful, the module updates the CAP ATC in the CAF or CAFD and resets the Bad CAP Token Validation Accumulator for the primary or secondary card in the CAF and returns control to the POS Router/Authorization module, indicating that CAP processing was completed successfully.

Initial CVR

BASE24 TSS chip authentication functionality enables you to specify an initial card verification results value for a chip authentication profile. This is useful when a payments application is used for CAP, or where a card application vendor is unable to make the CVR the same for the initial transaction as for subsequent transactions.

The initial transaction means the first transaction by a user in a PCR. This is not necessarily the transaction at ATC 0001. Testing of the cards by the vendor and insertion of the card in the PCR without generating a token can increase the ATC of the application and the first CAP token. Also, a customer may generate CAP tokens without attempting to validate them on the server, so the first transaction seen by the server is not necessarily the initial transaction with the initial CVR.

This means that for cards where the initial CVR may differ, the CVR used in the first transaction seen by the validation server cannot be determined by looking at the ATC in the transaction. The server must therefore attempt to validate the first transaction on such a card using both the initial CVR and the normal CVR.

If an initial CVR is specified for a given card authentication profile, the BASE24 Transaction Security Services process performs the following extra functionality:

- Any CVR bits in the CAP token are mapped onto both CVRs
- The validation of a CAP token where the last ATC value is 0000 or 0001 (i.e., this is the first observed transaction) results in the Transaction Security Services process first attempting validation using the initial CVR value, and if that attempt fails, attempting validation again using the normal CVR value.
- All subsequent transactions are validated using the normal CVR value

The fact that the normal CVR is tried upon validation failure also means that you can mix cards from different vendors, some of which have a different initial CVR and some of which do not, under the same profile, so long as the normal CVR values are the same.

Implementation

CAP token validation in BASE24-pos can be implemented in any of the following three ways:

- As a stand-alone solution based on CAP-only cards
- As a solution involving the existing EMV card application to be used for CAP token authentication
- As a solution involving multiple application payment/CAP cards for maximum configurability, while still maintaining a single card for the customer.

Configuration

The processing controls for CAP token validation are configured in both the BASE24 and TSS databases. These controls are discussed below.

BASE24 database

Besides invoking the Transaction Security Services process to perform CAP token validation, the Router/Authorization CAP extension module performs limit checking on CAP authentication failures and maintains the application transaction counter (ATC) for CAP authentication transactions. The configuration parameters for these functions are entered on Card Prefix File (CPF) screen 13 and Cardholder Authorization File (CAF) screen 13. The extension module stores the CAP ATC and bad CAP token validation accumulator in the base or EMV segment of the Cardholder Authorization File (CAF) or in the Dynamic Cardholder Authorization File (CAFD) for the primary and secondary card. The ATC counters can be maintained separately or combined with the ATC counters for the EMV application and/or contactless magnetic stripe application. ATC checking controls on CPF screens 3, 11, and 13 control how and where the ATC counters are maintained in the BASE24 database. For more information on how these controls impact how ATC values are maintained in the BASE24 database, see the [“Application transaction counter checking”](#) topic later in this section. Data from the CAF or CAFD is displayed on CAF screen 13.

The CAFD maintains just the ATC values and other dynamic data for a card and is not overwritten (recreated from an input file) during a full-file refresh of the CAF. Thus, by using the CAFD, you can retain values that would normally be reset to default values during a full-file refresh of the CAF. Unlike the CAF, the CAFD is not a segmented file and has relatively small records. Each CAFD record can maintain the first and second card ATC values for EMV cards, the first and second card ATC values for CAP token validation cards, and the first and second card bad CAP token validation accumulators. In addition, each CAFD record has a 20-byte customer user field that you can use to maintain data for ATC management exceptions or for other purposes.

During ATC checking for CAP token validation transactions, the Router/Authorization module uses the ATC from the CAFD if the CAFD exists. Otherwise, it uses the ATC from the CAF.

If you use the CAFD, you must configure the assign for it on IDF screen 1. For more information on this screen, refer to the **BASE24 Base Files Maintenance Manual**.

Refer to the ***BASE24-pos EMV Support Manual*** for detailed information on CPF screens 3, 11, and 13, their fields, limit checking, and ATC maintenance. The key locator value for the Chip Authentication data source (Chip_Authentication) is configured in the CAP GROUP field on CPF screen 13.

TSS database

Processing controls for CAP token validation are entered on the Chip Auth and EMV tabs of the Authentication Profile Configuration window. See [section 2](#) of this manual for a detailed description of the Chip Auth tab and its fields.

Application transaction counter checking

The application transaction counter (ATC) is a value maintained by the microchip and updated for each transaction performed by the card application. A single card can hold multiple ATCs, depending on the number of applications on the card. The ATC is also maintained in the BASE24 Cardholder Authorization File (CAF), as the value may be checked during transaction processing.

To reflect the possibility of multiple ATCs being maintained on the card, there are multiple CAF fields that can be used to hold the ATC. The CAF field used in a particular transaction is determined by the setting of various fields on the CPF:

- ATC CHECK field on CPF screen 3
- ATC CHECK TYPE field on CPF screen 11
- CAP ATC UPDATE field on CPF screen 13

You should set these fields based on which of the following you want to maintain:

- Separate ATCs for contactless magnetic stripe transactions, EMV transactions, and CAP token validation transactions.
- A single ATC for all three types of transactions.
- A single ATC for both contactless magnetic stripe and EMV transactions, and a separate ATC for CAP token validation transactions (or where CAP token validation transactions are not supported).
- A single ATC for both EMV and CAP token validation transactions, and a separate ATC for contactless magnetic stripe transactions (or where contactless magnetic stripe transactions are not supported).
- Separate ATCs for contactless magnetic stripe transactions and EMV transactions (where CAP token validation transactions are not supported).
- Separate ATCs for contactless magnetic stripe transactions and EMV transactions (where contactless magnetic stripe transactions are not supported).
- A single ATC for just contactless magnetic stripe transactions (where EMV transactions and CAP token validation transactions are not supported).

- A single ATC for just EMV transactions (where contactless magnetic stripe transactions and CAP token validation transactions are not supported).

- A single ATC for just CAP token validation transactions (where contactless magnetic stripe transactions and EMV transactions are not supported).
- No ATCs at all.

The relationship between these fields is described in more detail in the ***BASE24-pos EMV Support Manual***.

18: Diebold PIN verification

This section contains miscellaneous topics related to Diebold PIN verification processing when using the Transaction Security Services process.

Diebold number table

A Diebold number table (DNT) is a random set of 256 nonrepeating values used in Diebold PIN verification.

Diebold number table values must be entered in the Diebold Number Table fields under the Diebold tab on the Authentication Profile Configuration window of the ACI desktop user interface. The Diebold number table is downloaded into HSM memory when the Transaction Security component is initialized or when an operator enters the DNTLOAD command from the Process Control window or an XPNET network control facility.

Each value in the Diebold number table consists of a pair of hexadecimal characters (e.g., AA, A1, 9F, etc.). There are 256 possible combinations of hexadecimal pairs, and these combinations can be scrambled to derive a unique set of table entries, subject to the following rules:

- There must be 256 values in the table. Eight values are defined in each row in the table. Thus, 32 rows must be defined in the table.
- No hexadecimal pair can be repeated.
- The first character of the first hexadecimal pair of the table must be a C. Thus, the first hexadecimal pair can be any one of the 16 combinations from C0 to CF

Note: This table functions in much the same way as a DES key; therefore, it must be kept confidential.

Atalla NSPs

For an Atalla NSP, each of the 32 rows in the Diebold number table must be encrypted under the MFK before they are loaded into the NSP's memory. You can enter the Diebold number table values on the Authentication Profile Configuration window either in the clear or in encrypted format. In either case, the values must be entered in AKB format. You cannot enter Diebold number table values in non-AKB (i.e., variant) format.

Encrypted data entry

To enter or update the table in encrypted format, you must first encrypt the Diebold number table values using the Atalla Secure Configuration Terminal (SCT) or Secure Configuration Assistant (SCA) and copy the resulting cryptograms into the Diebold Number Table fields under the Diebold tab on the Authentication Profile Configuration window. Connect the SCT or SCA to the serial port of your PC. Use the Atalla Helper application with the SCT or the Calculate AKB Cryptogram menu on the SCA to set up the AKB header and display the resulting encrypted key. Set the Encrypt Type field on the Authentication Profile Configuration window to a value of Encrypted and copy the encrypted data for each DNT row into the DNT fields. When using the SCA, you can copy the encrypted data from the SCA Remote Management application on your PC. You can copy and paste the entire key into a row of the Diebold number table. The ACI desktop automatically parses the key parts into the correct fields (i.e., Header, Key Part 1, Key Part 2, Key Part 3, MAC, and Check Digits). After you have copied all encrypted data for all rows, click on the Save button to store the table.

Clear data entry

To enter the Diebold number table in the clear, the premium command 75 (Enter Key Part) must be enabled in the security policy of the NSP. If you enter or update the Diebold number table in the Diebold Number Table fields in clear format, set the Encrypt Type field to a value of Clear and click on the Save button after the clear values are entered. The Diebold number table values are automatically encrypted under the MFK using command 75 for storage in the TSS database. After the Diebold number table is entered or updated, you can only view it in encrypted format. When viewing the Diebold number table after it is entered or updated, the Encrypt Type field value indicates whether the table was originally entered in clear or encrypted format.

Warning: The entry of clear Diebold number table values is not secure and is not in compliance with major card association PIN security rules. Entering the Diebold number table in encrypted format is the most secure method for entering the table. In addition, Atalla command 75, which must be enabled to support the entry of clear values, could be used by an adversary to seriously breach the security of an NSP. Therefore, if you do enter Diebold number table values in the clear, command 75 should be immediately disabled after the clear table values are entered and saved.

Thales e-Security HSMs

For a Thales e-Security HSM, the Diebold number table must be entered and updated in encrypted format in the Diebold Number Table fields under the Diebold tab on the Authentication Profile Configuration window. Thus, the Encrypted Type field is not used when entering or updating Diebold number table data. When viewing Diebold number table data, the Encrypted Type field will always display the Encrypted value. To enter the Diebold number table on a Thales e-Security HSM, enter the table data and the location of the data in user storage using the R command on the HSM console. The data is encrypted under the appropriate LMK key pair and the encrypted data is displayed on the screen. Copy the encrypted data from the HSM console and enter it on the Authentication Profile Configuration

window. After all the data is entered, click the Save button to store the encrypted Diebold number table in the TSS database. An Environment attribute, RSM_KEY_STORE_LGTH, specifies the length of the Diebold number table key values stored in HSM memory. The valid values for this attribute are as follows:

- 1 = Single length keys (Default)
- 2 = Double length keys
- 3 = Triple length keys

Relative location

The Relative Location field under the Diebold tab of the Authentication Profile Configuration window specifies the relative location of the Diebold Number Table (DNT) in the security device. When the security device contains one DNT, the value entered in this field is 1. When the security device contains multiple DNTs, the relative location of the DNT used with this authentication profile is entered in this field. The Transaction Security component calculates the actual table index based on the value specified in this field.

Downloading the Diebold number table (DNT)

The DNT entered on the Authentication Profile Configuration window must be downloaded into the memory of your security modules before you can perform PIN verification. This must be done any time you change the DNT values on the Authentication Profile Configuration window. The DNT is not automatically downloaded when you stop and restart a Transaction Security Services process due to the fact that several Transaction Security Services processes can be running in the TSS environment. Therefore, you must manually download the DNT using this procedure.

You can download the DNT by entering a command on the Process Control window or from an XPNET network control facility. Both methods are described below.

Process Control window

To download the DNT into the memory of a hardware security device from the Process Control window, perform the following steps using the ACI desktop:

1. Log on to the ACI desktop user interface and access **System Operations > Process Control**. The Process Control window is displayed.
2. In the Destination field, select IS from the list of values.
3. In the Command field, select Deliver from the list of values.
4. In the Component field, select Transaction Security from the list of values.

5. In the Command Text field, enter the following command. If you want to download the DNT to a single device, enter the name of the device as it is configured on the Security Configuration window in the command text. If you want to download the DNT to all security devices in the BASE24 system, enter an asterisk (*) in the command text.

DNTLOAD *sec-dev-assign-name* | *

Examples:

DNTLOAD ATALLA1

DNTLOAD THALES1

6. Click Submit () at the top of the Process Control window. A “command sent” message is displayed in the Response History area of the window.

Network control facility

An example of the DNTLOAD command entered from an XPNET network control facility (e.g., NCPCOM) is shown below.

```
DELIVER PROCESS P1A^TSSISL001, "DELIVER TSEC DNTLOAD ATALLA1"
```

19: NCR PIN verification support

This section describes BASE24 support of the NCR PIN verification method.

NCR PIN verification can be performed only within Thales e-Security RG7x00 HSMs equipped with version 2.01/6.01 or later firmware.

NCR PIN verification

NCR PIN verification is based on the use of a PIN verification key, a PIN data block, and a PIN offset. NCR PIN verification can be performed using a hardware security module linked to the processor on which the BASE24 system is running.

At card-issuing time, the card issuer generates a unique PIN and PIN offset combination for each card, using validation data taken from the cardholder's account number. If the PIN is issuer-assigned or chosen by the cardholder, the PIN offset is generated to correspond to the PIN; if the PIN offset is a set value, the PIN is generated to correspond to the PIN offset. The PIN offset is then placed in the magnetic stripe data of the card.

The HSM performs the following steps to verify a PIN using the NCR PIN verification method.

1. Decrypts the single- or double-length PIN encryption key from under the appropriate LMK pair as designated by the PIN source flag.
If the key does not have odd parity on each byte, the HSM returns error code 10 (PIN key parity error).
2. Decrypts the *PVK from under LMK pair 20-21(32H) or (1A+32H).
If the key does not have odd parity on each byte, the HSM returns error code 11 (*PVK parity error).
3. Decrypts the PIN block from under the PIN key.
4. Disassembles the PIN block using the format defined by the PIN block format code and, if necessary, the 12 right-most digits of the PAN after the check digit has been removed.
If the PIN block does not contain valid values, the HSM returns error code 20 (PIN block does not contain valid values).
If the PIN length is invalid, the HSM returns error code 24 (PIN < 4 or > 12 digits).
5. Calculates the PIN from the PIN data block and PIN offset according to the NCR PIN verification algorithm.

6. Compares the derived PIN with the customer PIN. If the PIN is valid, the HSM returns error code 00 (No errors). If the PIN is not valid, the HSM returns error code 01 (Verification failure).

Input data

The NCR PIN verification algorithm uses the following input data to verify a PIN:

- PIN encryption key (the key under which the PIN block is encrypted)
- PIN verification key
- PIN source flag
- PIN block
- PIN block format
- PIN data block
- PAN (12 right-most digits, excluding the check digit)
- PIN offset

PIN encryption key and PIN source flag

BASE24 can receive an NCR PIN block from a terminal or from Equens. When the transaction originates from an ATM terminal, the PIN block is encrypted under the terminal PIN key. When the transaction originates from Equens, the PIN block is encrypted under the interface PIN key.

The PIN source flag indicates whether the PIN is encrypted under the Equens PIN encryption key or the terminal PIN encryption key. Valid values are:

- 0 = PIN block from Equens. The PIN key is encrypted under LMK pair 06–07.
- 1 = PIN block from terminal. The PIN key is encrypted under LMK pair 14–15.

Before verifying the PIN, the HSM decrypts the PIN block accordingly.

PIN verification key

NCR PIN verification requires that the validation data be encrypted for processing. The PIN verification key is the key used to encrypt that validation data. In this case, encryption is used to generate a different sequence of digits and to tie that string of digits to a secret value (the key).

For confidentiality, an encrypted key (encrypted by or for the hardware module) is required that can be passed to the HSM for encrypting the validation data. The encrypted PIN verification key must be entered in the Encryption Keys table on the NCR tab of the Authentication Profile Configuration window. The Key Scheme field in the Encryption Keys table specifies whether the Thales e-Security HSM is configured to use the old (ANSI X9.17 method) non-variant (32H) key encryption scheme or the variant (1A+32H) key encryption scheme.

PIN

In NCR PIN verification, the PIN entered by the cardholder is compared to the PIN computed by the hardware security module.

PINs are always verified by a hardware module, and must be passed to the module in encrypted form, along with the key used to encrypt them. The module then decrypts the PIN for comparing to the PIN it computes.

BASE24-atm allows cardholders to select their own PINs. The requirements for this are that PIN offsets be maintained in the CAF and that they initially be blank.

Upon receiving the first transaction from a cardholder, BASE24-atm computes an appropriate PIN offset based on the PIN entered by the cardholder and places that offset in the CAF. From that point on, that PIN entered by the cardholder is the only one allowed.

PINs can be from four to 12 digits in length. However, the length of the PIN offset is the minimum required length of the PIN. For example, if the PIN offset is four digits, the PIN must be at least four digits as well. The PIN can be longer than the PIN offset, however, if the PIN is longer than the PIN offset, only the right-most digits of the PIN are used in the verification.

PIN block and PIN block format

The PIN block is 16 hexadecimal characters that includes the encrypted PIN. NCR PIN verification supports PIN blocks in PIN/PAN (ANSI or ISO Format 0) or PIN PAD format, depending on the transaction source. PINs received from ATMs are in PIN PAD format (Thales PIN block format 03), encrypted under the *TPK. PINs received from Equens are in PIN/PAN (ANSI or ISO Format 0) (Thales PIN block format 01), encrypted under the *KSO. The PIN source flag indicates the source of the transaction.

PIN data block

The PIN data block contains the PIN identifier derived from track 3 of the card. The PIN data block is 16 hexadecimal characters in length. The NCR PIN verification algorithm uses the PIN identifier along with the PIN offset from track 3 to calculate the PIN compared to the PIN entered by the cardholder.

PAN

A primary account number (PAN) is the card number used to identify each card. It must be carried in the magnetic stripe data of the card and is used to disassemble the PIN block if necessary. The NCR PIN verification algorithm uses the 12 right-most digits of the PAN, excluding the check digit.

Index

A

A10100 network security processor, device options and commands, [231](#)

A8100 network security processor, device options and commands, [231](#)

A9100 network security processor, device options and commands, [231](#)

Abstract Syntax Notation One (ASN.1), definition of, [487](#)

ACI desktop user interface, [27](#)

ACI desktop user interface client, [47](#)

ACI Java Server Framework (AJSF), [49](#)

ACI Web Application Services - Java Services servlet container, [22](#)

ACI Web Services HTTP Server
use with ACI desktop user interface, [47](#)

AKB
see Atalla Key Block (AKB) format

AKB Conversion utility
Atalla security devices, [227](#)
Environment attributes, [202](#)
environment attributes, [632](#), [635](#)
introduction, [59](#)
overview, [627](#)
procedures, [629](#)

AKDS Capacity Report
generating, [613](#)
introduction, [608](#)
page length, [613](#)
report output location, [612](#)

AKDS Capacity Report component, [359](#)

AKDS process
configuration of, [28](#)

Alter Attribute command, [361](#)

Alter Data Source command, [359](#)

APPLCNFG
see Application Configuration File (APPLCNFG)

Application Configuration File (APPLCNFG), [49](#)

Application transaction counter, [681](#)

Application transaction counter (ATC)
definition of, MasterCard, [460](#)
definition of, Visa, [462](#)

Application transaction counter (ATC), definition of, [464](#)

AS2805 6.2 and 6.2 PIN block formats, [260](#)

AS2805 standard, [260](#)

Atalla Key Block (AKB) format
conversion utility, [474](#)
description of, [22](#)

Atalla Master File Key (MFK), conversion tool, [223](#)

Atalla NSPs
premium commands, [233](#)
premium options, [232](#)

Atalla security module requirements, [219](#)

ATC
see Application transaction counter (ATC)

ATC, see Application transaction counter

ATD
see BASE24-atm Terminal Data files (ATD)

ATDD1
see BASE24-atm Terminal Data Dynamic File—general data (ATDD1)

Audit Store and Forward Configuration File (Audit_Store_and_Forward_Config), [37](#)

Audit Store and Forward File (Audit_Store_and_Forward)
release 05.4, [37](#)
user auditing, [54](#)

AUDIT_SAF_CONFIG
see Audit Store and Forward Configuration File (AUDIT_SAF_CONFIG)

Audit_Store_and_Forward
see Audit Store and Forward File (Audit_Store_and_Forward)

Authentication Profile Configuration window
Card tab, [140](#)
Contactless tab, [142](#)
DES (IBM 3624) PIN Verification tab, [130](#)
Diebold tab, [132](#)
EMV tab, [135](#)
files maintained by, [27](#)
Identikey tab, [134](#)
maintaining authentication profile security data, [142](#)
NCR tab, [133](#)
overview, [128](#)
Visa PVV Keys tab, [131](#)

B

BASE24 database, Master Key loading from HSM, [273](#)

BASE24 KEYF Configuration window
description of, [96](#)
files maintained by, [27](#)
maintaining KEYF and interface security data, [114](#)

BASE24-atm Terminal Data Dynamic File—general data (ATDD1)
database conversion, [66](#), [474](#)
database conversion program, [475](#)

BASE24-atm Terminal Data files (ATD), database relationships, [25](#)

BASE24-eps architecture, [xvi](#)

- BASE24_KEYF assign, [96](#)
- BASE24-pos Terminal Data Dynamic File—
 - general data (PTDD1)
 - database conversion, [73](#), [474](#)
 - database conversion program, [475](#)
- BASE24-pos Terminal Data files (PTD), database relationships, [25](#)
- Base64 encoding, definition of, [487](#)
- Base94 encoding, definition of, [487](#)
- Basic Encoding Rules (BER), definition of, [488](#)
- BNTng Header File (BNTng_Header)
 - external memory table, [162](#)
- BNTng Initialization File (BNTng_Init_Data), [254](#)
- Bull BNTng hardware security modules (HSMs)
 - configuration requirements, [250](#)
 - connectivity, [241](#), [250](#)
 - supported functions, [251](#)
- C**
- CAP token validation
 - configuration, [680](#)
 - implementation, [680](#)
 - overview, [676](#)
- Card Prefix File (CPF), database relationships, [25](#)
- Card security code (CSC), [140](#)
- Card validation code (CVC), [140](#)
- Card verification code 3 (CSC3), [142](#)
- Card Verification File (Card_Verification)
 - external memory table, [162](#)
- Card Verification File (CARD_VRFCTN)
 - database conversion, [474](#)
 - database relationships, [25](#)
 - security data maintenance, [27](#), [125](#)
- Card verification value (CVV), [140](#)
- CARD_VRFCTN
 - see Card Verification File (CARD_VRFCTN)
- Certificate authority (CA), [488](#)
- Certificate Management window
 - Diebold, [514](#), [524](#)
 - NCR, [503](#), [547](#)
- Certificate signing request (CSR), definition of, [488](#)
- Certificate, definition of, [488](#)
- Changing database access passwords
 - procedure, [331](#)
- Changing IP addresses procedure, [329](#)
- Channel Public Key Security File (CHAN_PKEY_SEC)
 - data access, [561](#)
 - Diebold, [519](#)
 - Triton, [534](#)
- Channel Security Configuration window
 - files maintained by, [27](#)
 - for ATM devices, [69](#)
 - for POS devices, [75](#)
 - maintaining channel security data, [80](#)
- Channel Security File (Channel_Security)
 - database conversion, [474](#)
 - database relationships, [25](#)
 - internal memory table, [178](#)
 - security data maintenance, [27](#), [66](#), [73](#)
- Channel_Security
 - see Channel Security File (Channel_Security)
- Check Digit Validation and Generation utility
 - overview, [616](#)
 - prerequisites, [617](#)
 - procedures, [617](#)
 - processing report, [621](#)
- Check digits
 - ACI desktop user interface, [219](#)
 - BNTng command usage, [254](#)
 - DEP HSM command usage, [246](#)
 - generating, [619](#)
 - validating, [617](#)
- Chip Authentication File (Chip_Authentication)
 - database relationships, [25](#)
 - external memory table, [162](#)
 - security data maintenance, [27](#)
- Chip CVC
 - see Integrated circuit card validation code (Chip CVC)
- Chip_Authentication
 - see Chip Authentication File (Chip_Authentication)
- Cipher block chaining (CBC), [297](#)
- Cipher, definition of, [488](#)
- Ciphertext, definition of, [488](#)
- Comma Separated Values (CSV) tables, [26](#)
- Common name, definition of, [488](#)
- Common procedures
 - changing database access passwords, [331](#)
 - changing IP addresses, [329](#)
- Component, definition of, [21](#)
- Components
 - VSP Manager, [390](#)
- CONFCSV
 - see Metadata Configuration File (CONFCSV)
- CONFEMT, [317](#)
- Configuration files
 - Local.cfg, [47](#)
 - Logical Network Configuration File (LCONF), [184](#), [189](#)
 - Remote.cfg, [47](#)
- Contactless indicator (CI), definition of, [462](#)
- Convert AKB command
 - Atalla security devices, [227](#)
 - process control syntax, [378](#)
- CPF
 - see Card Prefix File (CPF)
- Cryptosystem, definition of, [488](#)
- CSC
 - see Card security code (CSC)
- CSC3
 - see Card verification code 3 (CSC3)
- CVC
 - see Card validation code (CVC)
- CVV
 - see Card verification value (CVV)
- D**
- Data encryption keys
 - configuration of, [308](#)
 - description of, [299](#)
 - overview of, [297](#)

- Data encryption layers, 300
- Data Encryption Peripheral (DEP) HSMs
 - ACI desktop key token entry fields, 242
 - check digits, 246
 - connectivity, 241
 - C-ZAM/DEP, 246
 - external key block format, 241
 - function definitions, 243
 - key storage method, 245
 - key tokens, 242
 - library definitions, 243
 - message formats, 241
 - supported commands, 248
 - supported functions, 241
 - TSS data entry, 246
- Data field encryption
 - cipher block chaining (CBC), 297
 - commands, 309
 - data encryption keys, 299
 - data types, 300
 - dynamic key management, 310
 - electronic code book (ECB), 298
 - functions, 298
 - interface configuration, 304
 - key exchange keys, 308
 - layers, 300
 - MACs, 300
 - overview, 297
 - security module configuration, 308
 - SPDH device configuration, 301
 - tokens, 300
- Data maintenance procedures
 - adding a new authentication profile, 145
 - adding a new authentication profile from a template, 150
 - adding a new channel ID record, 80
 - adding new KEYF and ICHG_SECURITY records, 115
 - creating a new authentication profile from the beginning, 146
 - deleting a channel ID record, 87
 - deleting an authentication profile, 158
 - deleting KEYF and ICHG_SECURITY records, 123
 - displaying an authentication profile, 143
 - modifying an existing authentication profile, 155
 - modifying an existing channel ID record, 83
 - modifying KEYF and ICHG_SECURITY records, 122
- Data Source OLTP Relationship data source (DS_OLTP_Relation), 419
- Database conversion
 - description of, 473
 - file relationships, 24
- Database conversion programs, 475
- Database partitioning, 24
- db.audited.db-service-name, 27
- dCVV
 - see Dynamic card verification value (dCVV)
- Default user IDs, 50
- Deliver Data Source command, 361
- DEP Master Key (DMK), 246
- Derivation Key Configuration window
 - description of, 93
- Derivation Key File (Derivation_Key)
 - database conversion, 474
 - database relationships, 25
 - external memory table, 163
 - security data maintenance, 27, 91
- Derivation Key File (KEYD)
 - database conversion, 91, 474
 - database conversion program, 475
 - database relationships, 25
- Derivation_Key
 - see Derivation Key File (Derivation_Key)
- Destination PIN block, 671
- Destinations
 - process tracing, 396
- Diebold number table (DNT)
 - Atalla NSPs, 684
 - overview, 684
 - relative location, 686
 - Thales e-Security HSMs, 685
- Diebold Number Table File (Diebold_Number_Table)
 - external memory table, 163
- Diebold Number Table File (DIEBOLD_NUM_TBL)
 - database conversion, 474
 - database relationships, 25
 - security data maintenance, 27, 125
- Diebold PIN verification
 - Diebold number table, 684
 - downloading the DNT, 59
 - handling the DNT, Thales, 277
- DIEBOLD_NUM_TBL
 - see Diebold Number Table File (DIEBOLD_NUM_TBL)
- Disable Audit Store and Forwarding command, 389
- Disable Security Device command, 383
- Disabling a security device, 646
- Distinguished Encoding Rules (DER), 488
- DST passphrase, 515
- DUKPT Derivation Key Profile Configuration window
 - files maintained by, 27
- Dutch Key Authorization File (KEYV)
 - database relationships, 25
- Dynamic card verification
 - ATC checking, 465
 - BASE24 configuration, 467
 - DCVD checking, 466
 - overview, 456
 - processing inputs, 464
 - TSS configuration, 472
- Dynamic Card Verification File (DYN_CARD_VRFCTN)
 - database relationships, 25
 - external memory table, 163
 - security data maintenance, 27
- Dynamic card verification value (dCVV), 142
- Dynamic CVC3, definition of, 460
- Dynamic CVV (dCVV), definition of, 463
- DYN_CARD_VRFCTN
 - see Dynamic Card Verification File (DYN_CARD_VRFCTN)

E

- Electronic code book (ECB), 298
- EMV Authorization Key File (EMV_Security)
 - database conversion, 474
 - database relationships, 25
 - external memory table, 163
 - security data maintenance, 27, 125
- EMV_Security
 - see EMV Authorization Key File (EMV_Security)
- Enable Audit Store and Forwarding command, 389
- Enable Security Device command, 383
- Enabling a security device, 648
- Encrypting PIN pad (EPP), definition of, 488
- Environment
 - see Environment File (Environment)
- Environment attributes
 - AKDS_CAP_RPT_PG_LGTH, 202
 - AKEY_SUMM_VIEW_MAX_LINES, 212
 - AMEX_CSC_ALGO, 141, 203
 - ATALLA_CBC_DATA_FRMT, 203
 - ATALLA_CHK_DGTS_LGTH, 204, 223, 617, 628, 638
 - CHAN_DATA_KEY_EXPORT_VARIANT, 205
 - CHNPK_LIST_MAX_LINES, 212
 - CRD_VRFY_LIST_MAX_LINES, 212
 - CSEC_MEMORY_USE, 178, 205
 - CUSTOMER_NAME, 205, 609
 - DECIMAL_TBL_ENCRYPT, 205, 273, 638
 - DFLT_DAT_FRMT, 609
 - EMV_LIST_MAX_LINES, 212
 - ENV_LIST_MAX_LINES, 212
 - EPP_ED_CERT_SUBJ, 533
 - EPP_SV_CERT_SUBJ, 533
 - EVT_DEST, 206
 - HOST_PK_SEC_LIST_MAX_LINES, 212
 - IDKEY_PRFL_LIST_MAX_LINES, 212
 - INTF_SEC_LIST_MAX_LINES, 212
 - KEYF_FIID_LIST_MAX_LINES, 212
 - KEYF_NAM_LIST_MAX_LINES, 212
 - MAX_CSEC_IN_MEM, 178, 206
 - NCR_LIST_MAX_LINES, 212
 - NON_ATALLA_KEY_EXPORT, 206
 - PCI_DATA_SUPPRESS, 394
 - PCI_NUM_UNMASKED_LEFT, 394
 - PCI_NUM_UNMASKED_RIGHT, 394
 - PIN_LGTH_ERR_RETRY_IND, 207
 - PIN_PRFL_LIST_MAX_LINES, 212
 - PIN_VRFY_FAIL_RETRY_IND, 207
 - PIN_XLATE_DISK_READ, 208
 - RAM_MAX_BUF_LGTH, 208
 - RSM_EMV_KEY_SCHEME, 208
 - RSM_KEY_STORE_LGTH, 209
 - RSM_TERM_MASTER_KEY, 189, 209
 - TSEC_STARTUP_MSGS, 210
 - UI_AUDIT_LEVEL, 54, 210
 - UI_AUDIT_OUTPUT_DATA_TYP, 211
 - URAM_MAX_BUF_LGTH, 211
 - USE_LOCAL_USER_AUDIT, 54, 211
- Environment Configuration window, 312
- Environment configuration, overview, 311
- Environment File (Environment), 56
- EPP Serial Number File (EPP_SERIAL_NUM), data access, 561
- Erase LMK or MFK command
 - Atalla security devices, 224
 - process control syntax, 379
 - Thales e-Security devices, 278
- Event
 - see Event File (Event)
- Event configuration
 - cause, effect, and remedy, 337, 340
 - message text and tokens, 339
 - suppressing an event message, 340
- Event Configuration and Management window
 - Event Message tab, 339
 - Event Suppression Information tab, 343
- Event Document File (Event_Document), 56
- Event Documentation File (Event_Document), 49
- Event File (Event), 49, 56
 - external memory table, 163
- Event management
 - cause, effect, and remedy, 337
 - configuring events, 347
 - event response or action, 336
 - generating events, 335
 - Integrated Server Event Configuration and Management window, 339
 - introduction, 62
 - log destination, 336
 - logging events, 335
 - monitoring events, 336
 - overview, 335
 - recognizing events, 336
 - subsystem ID and event number, 337
 - suppressing events, 335
- Event message suppression
 - log state, 344
 - methods, 346
 - overview, 341
 - processing, 342
- Event number, 337
- Event processing
 - event suppression, 341
- Event report generation command, 363
- Event Subsystem ID Configuration window, 352
- Event suppression methods
 - by number of events to skip, 346
 - by timer interval, 346
- Event_Document
 - see Event Document File (Event_Document)
- Exception Log File (Exception_Log)
 - description of, 56
 - purging records from, 366
- Exception_Log
 - see Exception Log File (Exception_Log)
- External Connection File (EXTRCNCT), 49
- External key block format, 219
- External memory table (OLTP) size, 168
- External memory tables (OLTPs)
 - list of, 162
 - rebuilding, 165
 - size of, 168
 - warmbooting, 175
- EXTRCNCT
 - see External Connection File (EXTRCNCT)

F

File formats, [58](#)
 File Partition Catalog File (file_part_catalog), [63](#)
 File Partition Fields File (file_part_fields), [63](#)
 File Partition List File (file_part_list), [63](#)
 File partitioning, [63](#)
 file_part_catalog
 see File Partition Catalog File (file_part_catalog)
 file_part_fields
 see File Partition Fields File (file_part_fields)
 file_part_list
 see File Partition List File (file_part_list)
 Financial Institution Table (FIT)
 Diebold, [522](#)
 NCR, [512](#), [543](#)
 First card ATC, definition of, [465](#)

G

Generate AKDS Capacity Report command, [359](#)
 Generate check digits command
 Atalla, [223](#)
 Bull BNTng, [254](#)
 Check Digit Validation and Generation utility, [619](#)
 DEP HSM, [246](#)
 process control syntax, [380](#)
 Thales e-Security, [277](#)

H

Hash algorithms
 mid-square, [169](#)
 modulo divide, [169](#)
 multiplicative, [169](#)
 hash table
 terminology, [175](#)
 Header locator value, description of, [251](#)
 Heap manager
 about the ACI Heap Manager, [427](#)
 how Heap Manager manages memory, [428](#)
 how memory parcels are identified by Heap Manager, [429](#)
 Host Public Key Security File (HOST_PUBLIC_KEY)
 data access, [561](#)
 Host Public Key Security File (Host_Public_Key)
 NCR, [506](#), [548](#)
 Host Public Key Security Primary File (HOST_PUBLIC_KEY)
 NCR, [516](#), [532](#)
 Hypertext Transfer Protocol (HTTP)
 ACI desktop, [28](#)
 Hypertext Transfer Protocol over SSL (HTTPS)
 ACI desktop, [28](#)

I

IBM DES Verification File (IBM_DES)
 database conversion, [474](#)
 database relationships, [25](#)
 external memory table, [163](#)
 security data maintenance, [27](#), [125](#)

IBM_DES
 see IBM DES Verification File (IBM_DES)
 ICC Key File (KEYI)
 database conversion, [125](#), [474](#)
 database conversion program, [475](#)
 database relationships, [25](#)
 ICHG_SECURITY
 see Interface Security File (ICHG_SECURITY)
 iCSC
 see Integrated circuit card security code (iCSC)
 iCVV
 see Integrated circuit card verification value (CVV)
 Identikey
 see Identikey PIN Verification File (Identikey)
 Identikey PIN Verification File (Identikey)
 database conversion, [474](#)
 database relationships, [25](#)
 external memory table, [163](#)
 security data maintenance, [27](#), [125](#)
 Info Attribute command, [362](#)
 Initialize Security Device command, [385](#)
 Initializing a security device, [650](#)
 Integrated circuit card security code (iCSC), [140](#)
 Integrated circuit card validation code (Chip CVC), [140](#)
 Integrated circuit card verification value (iCVC), [140](#)
 Interac key management support, [101](#)
 Interface Security Configuration window
 Data Encr Exch Info tab, [112](#)
 Data Encr Key Info tab, [113](#)
 Exchange Information tab, [106](#)
 files maintained by, [27](#)
 MAC Information tab, [110](#)
 maintaining KEYF and interface security data, [114](#)
 Master Keys tab, [114](#)
 overview, [103](#)
 PIN Information tab, [108](#)
 Interface Security File (ICHG_SECURITY)
 database conversion, [474](#)
 database relationships, [25](#)
 external memory table, [163](#)
 memory table, [178](#)
 security data maintenance, [27](#), [94](#)
 Intermediate key, [276](#)
 IS process, MDSDEST, [315](#)
 Issuer script commands, [670](#)

K

Key 6 File (KEY6)
 database conversion, [474](#)
 database conversion program, [475](#)
 Key Authorization File (KEYA)
 database conversion, [125](#), [474](#)
 database conversion program, [475](#)
 database relationships, [25](#)
 Key exchange key, [276](#)

Key File (KEYF)
 database conversion, [94](#), [474](#)
 database conversion program, [475](#)
 database relationships, [25](#)
 security data maintenance, [27](#), [94](#)

Key Generate (KEYGEN) command
 description, [552](#)
 process control syntax, [385](#)

Key locator
 database conversion, [473](#)
 definition of, [24](#)

Key token
 Bull BNTng, description of, [251](#)
 DEP HSM, description of, [242](#)

Key Translation report, [643](#)

KEY6
 see Key 6 File (KEY6)

KEYA
 see Key Authorization File (KEYA)

KEYD
 see Derivation Key File (KEYD)

KEYF
 see Key File (KEYF)

KEYGEN command
 Diebold implementation procedures, [514](#)
 NCR implementation procedures, [503](#), [547](#)

KEYI
 see ICC Key File (KEYI)

KEYV
 see Dutch Key Authorization File (KEYV)

L

LISTENER
 see Listener File (LISTENER)

Listener File (LISTENER), [49](#)

Load All Keys command
 Diebold, [545](#)
 NCR, [544](#), [546](#)
 Tidel, [508](#), [533](#)

Load balancing, [213](#)

Load DNT command, [384](#)

Load Key Entry Mode command, NCR, [510](#)

Load Master Key command
 Diebold, [520](#)
 NCR, [511](#), [542](#)

Load Public Key command
 Diebold, [518](#)
 NCR, [507](#), [541](#)

Local Master Key (LMK) key pairs conversion tool, [278](#)

Log state
 always log the event, [344](#)
 log at network threshold, [345](#)
 log at process threshold, [345](#)
 never log the event, [345](#)

Logical Network Configuration File (LCONF)
 assigns
 SECURE-DEV1, [184](#)
 SECURE-DEV2, [184](#)
 SECURE-DEV-xx, [184](#)

Logical Network Configuration File (LCONF)
 params
 ATM-4000-MASTER, [190](#)
 ATM-DH-1000-AKDS-SERIAL-NUM-VRFY, [500](#),
[507](#), [519](#), [541](#)
 ATM-DH-AKDS-TSS-TIMER, [500](#)
 ATM-DH-N50-AKDS-SERIAL-NUM-VRFY, [501](#),
[507](#), [519](#), [541](#)
 ATM-DH-TIDL-AKDS-HOST-LOAD-TIMER, [501](#)
 ATM-DH-TIDL-AKDS-SERIAL-NUM-VRFY, [501](#),
[507](#), [509](#)
 ATM-DH-TRIT-AKDS-SERIAL-NUM-VRFY, [501](#),
[507](#), [534](#)
 DES-TRIPLE-SINGLE, [190](#)
 LN-IND, [24](#), [192](#)
 MULT-LN-PIN-KEY, [192](#)
 POS-4000-MASTER, [190](#)
 RSM-TERM-MASTER-KEY, [189](#)
 SECURE-DEV-ACCESS-TYP, [189](#)
 SECURE-DEV-ACCESS-TYPE, [189](#)
 SECURE-DEV-CONF-CMD100, [189](#)
 SECURE-DEV-CONF-TXT, [189](#)
 SECURE-DEV-FAIL-NOTIFY, [191](#)
 SECURE-DEV-MONITOR-THRESHOLD, [185](#),
[191](#)
 SECURE-DEV-REINSTATE-TIME, [189](#)
 SECURE-DEV-TIM-LMT, [191](#)
 SECURE-DEV-TYP, [184](#), [189](#)
 SOFTWARE-PIN-VERIFICATION-ALWD, [190](#)

Logical network identifier
 BASE24-atm Terminal Data files, [66](#)
 BASE24-pos Terminal Data files, [73](#), [94](#)

M

Master File Key (MFK), [220](#)

Master File Key Conversion utility
 accessing, [639](#)
 Atalla, [223](#)
 Atalla options, [639](#)
 key translation report, [643](#)
 overview, [62](#), [637](#)
 post-translation procedures, [645](#)
 prerequisites, [638](#)
 process control introduction, [60](#)
 Thales e-Security, [278](#)
 Thales options, [640](#)

Master key, definition of, [488](#)

MDBCSV
 see Meta Database Comma Separated Values (MDBCSV) file

MDBEMT, [317](#)

MDSDEST
 see System Interface Message Delivery Service Destination File (MDSDEST)

Memory management and troubleshooting
 activating/deactivating memory
 monitoring, [432](#)
 analyzing heap trace output for memory leaks, [436](#)
 analyzing process termination output for memory verification, [452](#)
 detect memory corruption, [450](#)
 determine whether memory leaks exist, [433](#)
 enabling/disabling memory verification, [446](#)
 find the memory leaks, [436](#)

Memory management and troubleshooting

continued

- getting thread-level memory status information, [430](#)
- how memory is verified, [449](#)
- memory monitoring, [431](#)
- memory parcel handling with memory verification activated, [448](#)
- memory verification, [446](#)
- setting the memory verification interval, [447](#)
- using memory monitoring to identify memory leaks, [432](#)

Message authentication for large messages

- Atalla, [231](#)
- Banksys DEP, [246](#)
- Eracom, [261](#)
- Thales e-Security, [273](#)

Message digest, definition of, [488](#)

Message flows

- card verification, [292](#)
- chip authentication, [293](#)
- interface acquired—on-us card, [289](#)
- terminal acquired—not-on-us card, [285](#)
- terminal acquired—on-us card, [287](#)
- terminal acquired—on-us card—software verification allowed, [294](#)
- terminal acquired—on-us card—software verification not allowed, [295](#)

Meta Database Comma Separated Values (MDBCSV) file, [26](#)

Metadata Configuration File (CONFCSV)

- assign names, [165](#), [166](#), [176](#), [177](#), [181](#), [182](#)
- description of, [26](#)
- external memory table size, [168](#)
- modifying parameters, [326](#)

Metadata EMT Build process, [317](#)Metadata external memory tables, [317](#)Metadata Manager (Metaman) utility, [25](#)Modulus, definition of, [489](#)**N**

NCR PIN Verification File (NCR_PIN_VRFY)

- database relationships, [25](#)
- external memory table, [163](#)
- security data maintenance, [27](#), [125](#)

NCR_PIN_Verification

- see NCR PIN Verification File (NCR_PIN_Verification)

Nonce, definition of, [489](#)NSPDIAG tool, [234](#)**O**Offline PIN change, [670](#)OLTP Warmboot data source (OLTP_Warmboot), [419](#)

OLTP Warmboot Management window

- accessing, [425](#)
- deleting OLTP_Warmboot records, [425](#)
- overview, [420](#)
- rebuilding OLTPs, [423](#)
- warmbooting OLTPs, [424](#)

Organization name, definition of, [489](#)Out-of-band, definition of, [489](#)**P**Pad characters, [42](#)PIN block formats, AS2805 6.2 and 6.4, [260](#)PIN blocks, types supported, [42](#)PIN translation, Atalla NSPs, [228](#)PIN verification methods, [23](#)

PKI

- see Public key infrastructure (PKI)

Plaintext, definition of, [489](#)

POS Terminal Data File (PTDF)

- database conversion, [474](#)
- database conversion program, [475](#)

PPD names, [188](#)Premium commands, Atalla, [232](#)

PROCESS

- see Process File (PROCESS)

Process control

- commands, [358](#)
- overview, [353](#)
- Transaction Security Services environment, [59](#)
- XPNET environment, [59](#)

Process control commands

- activate heap memory monitoring for a process thread, [369](#)
- activate memory allocation for a thread, [374](#)
- activate stack trace for a thread, [370](#)
- alter attribute, [361](#)
- alter CSEC REFRESH *channel-id*, [386](#)
- alter data source, [180](#), [359](#)
- compare current memory usage to the process thread baseline, [371](#)
- convert AKB, [227](#), [378](#)
- CSEC refresh, [180](#)
- deactivate heap memory monitoring, [373](#)
- deactivate memory allocation verification, [377](#)
- deactivate stack trace for a thread, [372](#)
- deliver data source, [361](#)
- deliver DETAIL, [359](#)
- disable audit store and forwarding, [389](#)
- disable security device, [383](#)
- disabling a security device, [646](#)
- enable audit store and forwarding, [389](#)
- enable security device, [383](#)
- enabling a security device, [648](#)
- erase LMK or MFK, [224](#), [278](#), [379](#)
- event report generation, [363](#)
- generate check digits, [380](#)
- get basic memory status for a process thread, [368](#)
- get detailed memory status for a process thread, [369](#)
- info attribute, [362](#)
- info VSP *, [390](#)
- initialize security device, [385](#), [650](#)
- key generate, [385](#)
- load DNT, [384](#)
- obtaining the status of a security device, [652](#)
- purge Exception_Log records, [366](#)
- remove audit store and forwarding record, [389](#)
- report memory allocated but not released for a thread, [372](#)
- reset current memory baseline for a process thread, [371](#)
- reset suppression, [363](#)

- Process control commands *continued*
 - set memory allocation verification to occur after each transaction, 376
 - set memory allocation verification to occur at a specific interval, 375
 - set memory verification to occur for all deleted/released memory parcels, 374
 - start statistics, 387
 - status DETAIL, 382
 - status security device, 387
 - status statistics, 388
 - status timer statistics, 382
 - stop statistics, 388
 - validate check digits, 379
 - verify current memory allocations for a thread, 376
 - Process Control window
 - description of, 354
 - introduction, 60
 - Process File (PROCESS), 49
 - Process tracing
 - active flag, 397
 - component IDs, 402
 - configuring, 395
 - controlling traces, 409
 - example external message trace output, 414
 - external message user data, 406
 - filter profile, 397
 - introduction, 392
 - journal trace user data, 406
 - operations, 408
 - output, 397, 410
 - process types, 396
 - security device tracing procedure, 416
 - trace filter profile configuration, 406
 - trace filter types, 408
 - trace headers, 398
 - trace levels, 398
 - viewing trace data using the Trace Viewer utility, 411
 - Proof-of-endpoint processing, 261
 - PTD
 - see BASE24-pos Terminal Data files (PTD)
 - PTDD1
 - see BASE24-pos Terminal Data Dynamic File—general data (PTDD1)
 - PTDF
 - see POS Terminal Data File (PTDF)
 - Public exponent, definition of, 490
 - Public key cryptography
 - data access, 561
 - implementation procedures, Diebold, 513, 523
 - implementation procedures, NCR, 502
 - implementation requirements, Atalla, 497
 - implementation requirements, Banksys DEP HSM, 498
 - implementation requirements, BASE24, 496
 - implementation requirements, Diebold, 493
 - implementation requirements, NCR, 491
 - implementation requirements, shared Wincor Nixdorf certificate devices, 495
 - implementation requirements, shared Wincor Nixdorf signature devices, 494
 - implementation requirements, Thales e-Security, 499
 - implementation requirements, Tidel devices, 496
 - implementation requirements, Triton devices, 496
 - interface implementation, 546
 - overview, 477
 - terminology, 487
 - Public key cryptography commands
 - Atalla, 497
 - Banksys DEP HSM, 498
 - Thales e-Security, 499
 - Public Key Cryptography Standards (PKCS)
 - #10 certificate request message, 489
 - #7 cryptographic message, 489
 - definition of, 490
 - Public key infrastructure (PKI), 490
 - Public key, definition of, 490
 - Purge Exception_Log Records command, 366
- ## R
- Racal
 - see Thales e-Security
 - Racal Access Module (RAM) process, 278
 - Raw send flag, 314
 - Refresh individual CSEC records command, 386
 - Remove Audit Store and Forward Record command, 389
 - Reset Suppression command, 363
 - Rivest-Shamir-Adleman (RSA) scheme
 - definition of, 490
 - introduction, 43
- ## S
- SafeNet (Eracom) Host Security Modules (HSMs)
 - bad PIN and key synchronization errors, 260
 - configuration requirements, 258
 - connectivity, 258
 - external key block format, 260
 - supported functions, 258
 - supported HSM online functions, 267
 - SEC_DEV_CONFIG
 - see Security Device Configuration File (SEC_DEV_CONFIG)
 - Second card ATC, definition of, 465
 - Secret key, definition of, 490
 - Secure Configuration Assistant (SCA)
 - preparing data for the TSS files, 220
 - Secure Configuration Terminal (SCT)
 - preparing data for TSS files, 220
 - Secure Configuration Terminal (SCT), AKB version software, 223
 - Secure Hash Algorithm (SHA-1), definition of, 490
 - Secure Sockets Layer (SSL) protocol, 55
 - Security best practices symbol, xx
 - Security data maintenance, 27
 - Security device
 - adding a new security device, 327
 - MDSDEST, 314
 - Security Device Configuration File (SEC_DEV_CONFIG)
 - internal memory table, 178
 - security data maintenance, 27

- Security Device Configuration window
 - assigns for, [188](#)
 - description of, [193](#)
 - enabling PKI, [497](#)
 - error notify interval, [199](#)
 - files maintained by, [27](#)
 - maximum consecutive errors, [199](#)
 - timeout limit, [191](#), [500](#)
 - Security module requirements
 - Atalla, [219](#)
 - Thales e-Security, [271](#)
 - Security modules
 - Atalla, preparing data for TSS files, [220](#), [246](#)
 - BNTng, preparing data for TSS files, [255](#)
 - SafeNet (Eracom), preparing data for TSS files, [262](#)
 - Thales e-Security, preparing data for TSS files, [273](#)
 - Shutdown, [59](#)
 - Signature, definition of, [490](#)
 - SIMDS_DEST_FILENAME parameter, [314](#), [326](#)
 - SIMDS_DEST_WARMBOOT_DELTA parameter, [326](#), [633](#)
 - SSL
 - see Secure Sockets Layer (SSL) protocol
 - Start Statistics command, [387](#)
 - Startup, [59](#)
 - Status a security device, [652](#)
 - Status information on trace command, [382](#)
 - Status Security Device command, [387](#)
 - Status Statistics command, [388](#)
 - Status Timer Statistics command, [382](#)
 - Stop Statistics command, [388](#)
 - Subsystem ID, [337](#)
 - Syntax notations, [xx](#)
 - System Interface Message Delivery Service
 - Destination File (MDSDEST)
 - creating, [316](#)
 - description of, [313](#)
 - introduction to, [56](#)
 - PPD names, [188](#), [194](#)
- T**
- TDF
 - see Terminal Data File (TDF)
 - Terminal Data File (TDF)
 - database conversion, [474](#)
 - database conversion program, [475](#)
 - Terminal ID
 - key locator in BASE24-atm Terminal Data files, [66](#)
 - key locator in BASE24-pos Terminal Data files, [73](#)
 - Thales e-Security HSMs
 - changing local master key pairs, [278](#)
 - console, [273](#)
 - console commands, [274](#)
 - double- and triple-length key encryption scheme, [272](#)
 - external key block format, [272](#)
 - generate check digits command, [277](#)
 - key encryption scheme for double- and triple-length keys, [272](#)
 - preparing data for TSS files, [273](#)
 - RAM process, [278](#)
 - supported commands, [279](#)
 - triple DES enabled commands, [278](#)
 - Third-party JARS
 - Java Authentication and Authorization Service (JAAS), [36](#)
 - Timer process
 - adding additional processes, [328](#)
 - function of, [48](#)
 - MDSDEST, [315](#)
 - TMF
 - see Transaction Management Facility (TMF)
 - Trace Configuration window, [395](#)
 - Trace viewer utility, [411](#)
 - Trace, definition of, [392](#)
 - Transaction Management Facility (TMF), [63](#)
 - Transaction Security component, [21](#)
 - Transaction Security Services environment support, [47](#)
 - Transaction Security Services process
 - adding additional processes, [327](#)
 - ATM device support, [40](#)
 - balancing transactions between, [186](#)
 - checking the secondary, [185](#)
 - communications with, [32](#)
 - configuration of, [28](#)
 - examples in transaction processing, [65](#)
 - implementation requirements, [32](#)
 - integration in a BASE24 system, [31](#)
 - interface support, [41](#)
 - operations, [59](#)
 - overview, [22](#)
 - POS device support, [41](#)
 - process control, [59](#)
 - security module support, [38](#)
 - software support, [32](#)
 - warmbooting external memory tables, [175](#)
 - warmbooting internal memory tables, [180](#)
 - Transmission Control Protocol/Internet Protocol (TCP/IP), ACI desktop, [28](#)
 - Triple DES
 - Atalla migration software requirements, [226](#)
 - enabled Atalla commands, [225](#)
 - enabled Thales e-Security commands, [278](#)
 - Two factor authentication (2FA), [676](#)
- U**
- UAUD
 - see User Auditing (UAUD) application
 - UDP/IP protocol
 - see User Datagram Protocol/Internet Protocol (UDP/IP)
 - Unique identifier, definition of, [491](#)
 - Unpredictable number (UN), definition of, [460](#)
 - URAM-MSG-MDFY parameter, [279](#)
 - USEC
 - see User Security (USEC) application
 - User Auditing (UAUD) application, [53](#)
 - User Datagram Protocol/Internet Protocol (UDP/IP), [278](#)

User interface
 security data maintenance, [27](#)
 support, [47](#)
User Interface Server process, functions of, [48](#)
User Security (USEC) application, [50](#)

V

Validate check digits command
 Check Digit Validation and Generation
 utility, [617](#)
 process control syntax, [379](#)
Version Checker server, [47](#)
Version/Service Pack command, [390](#)
Visa PVV Verification File (Visa_PVV)
 database conversion, [474](#)
 database relationships, [25](#)
 external memory table, [163](#)
 security data maintenance, [27](#), [125](#)
Visa_PVV
 see Visa PVV Verification File (Visa_PVV)
VPVVD
 see Visa PVV Verification File (VPVVD)
VSP Manager component, [390](#)

W

Warmbooting
 external memory tables (OLTPs), OLTP
 Warmboot Management window, [424](#)
 external memory tables (OLTPs), process
 control commands, [175](#)
 internal memory tables, [180](#)
WebGate
 see ACI Web Services
Working key, definition of, [491](#)

X

X9.19 message authentication
 channel security, [77](#)
 interface security, [110](#)
XML Server process
 see User Interface process

Z

Zone master key (ZMK), [276](#)